

Code Export

Source Code PDF Report

Generated 2026-05-05 17:27

Root C:\Users\LENOVO\Documents\Java_run\Smart_EmployeeManagementSystem\SmartEmployeeManagementSys
directory

Files 174
found

Total lines 22,161

Table of Contents

1. Parent: README.md (3 lines)
2. Current: .mvn\wrapper\maven-wrapper.properties (4 lines)
3. Current: pom.xml (138 lines)
4. Current: src\main\java\com\example\EmployeeManagementSystem\code\CodeToPdf.java (348 lines)
5. Current: src\main\java\com\example\EmployeeManagementSystem\config\BasicAuthConfig.java (52 lines)
6. Current: src\main\java\com\example\EmployeeManagementSystem\config\JWTauth\JWTAuthConfig.java (197 lines)
7. Current: src\main\java\com\example\EmployeeManagementSystem\config\LeaveTypeDataInitializer.java (44 lines)
8. Current: src\main\java\com\example\EmployeeManagementSystem\config\Seassion\SessionAuthConfig.java (32 lines)
9. Current: src\main\java\com\example\EmployeeManagementSystem\config\swaggerConfig.java (38 lines)
10. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\AdminController.java (182 lines)
11. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\ApiKeyController.java (51 lines)
12. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\AuthController.java (184 lines)
13. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\DeliveryController.java (33 lines)
14. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\DeviceController.java (71 lines)
15. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\EmployeeController.java (59 lines)
16. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\GlobalExceptionHandler.java (141 lines)
17. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\HolidayController.java (40 lines)
18. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\LeaveDashboardController.java (119 lines)
19. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\LeaveRequestController.java (72 lines)
20. Current: src\main\java\com\example\EmployeeManagementSystem\Controller>LoginController.java (17 lines)
21. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\ManagerController.java (72 lines)
22. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\NotificationController.java (60 lines)
23. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\OauthController.java (52 lines)
24. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\RestaurantController.java (124 lines)
25. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\SchedulerController.java (103 lines)

26. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\SchedulerTestController.java (63 lines)

27. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\ServiceRequestController.java (74 lines)

28. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\SessionAuthController.java (96 lines)

29. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\SubscriptionController.java (73 lines)

30. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\TotpController.java (134 lines)

31. Current: src\main\java\com\example\EmployeeManagementSystem\Controller\VendorController.java (59 lines)

32. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AccessTokenRequest.java (18 lines)

33. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\ActionDTO.java (34 lines)

34. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AdminActionDTO.java (34 lines)

35. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AdminEmployeeDTO.java (61 lines)

36. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AssignDeviceRequest.java (14 lines)

37. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AuthRequest.java (28 lines)

38. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\AuthResponse.java (28 lines)

39. Current:
src\main\java\com\example\EmployeeManagementSystem\DTO\DeviceAssignmentResponseDTO.java (91 lines)

40. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\DeviceDTO.java (45 lines)

41. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\DeviceResponseDTO.java (103 lines)

42. Current:
src\main\java\com\example\EmployeeManagementSystem\DTO\EmployeeAttendanceResponseDTO.java (43 lines)

43. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\EmployeeDetails.java (52 lines)

44. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\EmployeeDTO.java (61 lines)

45. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\EmployeeStatusDTO.java (71 lines)

46. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\HolidayDTO.java (50 lines)

47. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\LeaveDashboardResponse.java (122 lines)

48. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\LeaveRequestDTO.java (48 lines)

49. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\LeaveResponseDTO.java (82 lines)

50. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\ManagerDTO.java (43 lines)

51. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\ManagerEmployeeDetailsDTO.java (72 lines)

52. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\ManagerLeaveResponseDTO.java (73 lines)

53. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\MaternityLeaveRequestDTO.java (25 lines)

54. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\PromoteRequest.java (23 lines)

55. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\RepairDTO.java (34 lines)

56. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\RestaurantDTO.java (42 lines)

57. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\RestaurantRequest.java (46 lines)

58. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\ServiceRequestDTO.java (44 lines)

59. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\ServiceRequestResponseDTO.java (116 lines)

60. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\SubscriptionDTO.java (103 lines)

61. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\SubscriptionRequest.java (72 lines)

62. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\VendorDTO.java (26 lines)

63. Current: src\main\java\com\example\EmployeeManagementSystem\DTO\VendorRequest.java (47 lines)

64. Current:
src\main\java\com\example\EmployeeManagementSystem\EmployeeManagementSystemApplication.java (16 lines)

65. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\ApiKey.java (79 lines)

66. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Delivery.java (93 lines)

67. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Device.java (107 lines)

68. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\DeviceAssignment.java (68 lines)

69. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Employee.java (197 lines)

70. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\FixedHoliday.java (61 lines)

71. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\LeaveEntitlement.java (104 lines)

72. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\LeaveRequest.java (105 lines)

73. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\LeaveType.java (49 lines)

74. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\LeaveWarning.java (58 lines)

75. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Notification.java (42 lines)

76. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\RefreshToken.java (62 lines)

77. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Restaurant.java (58 lines)

78. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\ServiceRequest.java (141 lines)

79. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Subscription.java (105 lines)

80. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\Vendor.java (124 lines)

81. Current: src\main\java\com\example\EmployeeManagementSystem\Entity\YearEndCarryForwardLog.java (62 lines)

82. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\AssignmentStatus.java (8 lines)

83. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\DeliveryStatus.java (9 lines)

84. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\DeviceStatus.java (10 lines)

85. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\DeviceType.java (15 lines)

86. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\Gender.java (5 lines)

87. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\LeaveStatus.java (9 lines)

88. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\LeaveType.java (9 lines)

89. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\MealSlot.java (8 lines)

90. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\Permission.java (21 lines)

91. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\Role.java (60 lines)

92. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\ScheduleType.java (7 lines)

93. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\ServiceRequestStatus.java (9 lines)

94. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\ServiceRequestType.java (10 lines)

95. Current: src\main\java\com\example\EmployeeManagementSystem\Enum>Status.java (8 lines)

96. Current: src\main\java\com\example\EmployeeManagementSystem\Enum\SubscriptionStatus.java (8 lines)

97. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\DuplicateRequestException.java (8 lines)

98. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\EmployeeNotFound.java (10 lines)

99. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\InvalidEndDateException.java (8 lines)

100. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\InvalidManagerException.java (8 lines)

101. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\InvalidStartDateException.java (8 lines)

102. Current:
src\main\java\com\example\EmployeeManagementSystem\Exception\LeaveRequestNotFoundException.java (8 lines)

103. Current:
src\main\java\com\example\EmployeeManagementSystem\Exception\OverlappingLeaveException.java (8 lines)

104. Current:
src\main\java\com\example\EmployeeManagementSystem\Exception\RestaurantNotFoundException.java (8 lines)

105. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\SubscriptionAlreadyExists.java (8 lines)

106. Current:
src\main\java\com\example\EmployeeManagementSystem\Exception\UnauthorizedAccessException.java (7 lines)

107. Current: src\main\java\com\example\EmployeeManagementSystem\Exception\VendorNotFoundException.java (7 lines)

108. Current: src\main\java\com\example\EmployeeManagementSystem\Filter\ApiKeyFilter.java (67 lines)

109. Current: src\main\java\com\example\EmployeeManagementSystem\Filter\JwtAuthFilter.java (83 lines)

110. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\ApiKeyRepository.java (34 lines)

111. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\DeliveryRepository.java (23 lines)

112. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\DeviceAssignmentRepo.java (21 lines)

113. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\DeviceRepository.java (13 lines)

114. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\EmployeeRepo.java (26 lines)

115. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\FixedHolidayRepository.java (13 lines)

116. Current:
src\main\java\com\example\EmployeeManagementSystem\Repository\LeaveEntitlementRepository.java (31 lines)

117. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\LeaveRequestRepo.java (134 lines)

118. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\LeaveTypeRepository.java (13 lines)

119. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\LeaveWarningRepository.java (21 lines)

120. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\NotificationRepository.java (12 lines)

121. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\RefreshTokenRepository.java (19 lines)

122. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\RestaurantRepository.java (31 lines)

123. Current:
src\main\java\com\example\EmployeeManagementSystem\Repository\ServiceRequestRepository.java (23 lines)

124. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\SubscriptionRepository.java (32 lines)

125. Current: src\main\java\com\example\EmployeeManagementSystem\Repository\VendorRepo.java (13 lines)

126. Current:
src\main\java\com\example\EmployeeManagementSystem\Repository\YearEndCarryForwardLogRepository.java (18 lines)

127. Current: src\main\java\com\example\EmployeeManagementSystem\Scheduler\DeliveryScheduler.java (30 lines)

128. Current: src\main\java\com\example\EmployeeManagementSystem\Scheduler\LeaveAccrualScheduler.java (71 lines)

129. Current: src\main\java\com\example\EmployeeManagementSystem\Scheduler\LeaveStatusScheduler.java (107 lines)

130. Current:
src\main\java\com\example\EmployeeManagementSystem\security\ApiKeyAuthenticationProvider.java (86 lines)

131. Current: src\main\java\com\example\EmployeeManagementSystem\security\ApiKeyAuthenticationToken.java (84 lines)

132. Current: src\main\java\com\example\EmployeeManagementSystem\Service\ApiKeyService.java (52 lines)

133. Current:
src\main\java\com\example\EmployeeManagementSystem\Service\CarryForwardWarningService.java (126 lines)

134. Current: src\main\java\com\example\EmployeeManagementSystem\Service\CombinedUserDetailService.java (41 lines)

135. Current: src\main\java\com\example\EmployeeManagementSystem\Service\CustomOAuth2UserService.java (87 lines)

136. Current: src\main\java\com\example\EmployeeManagementSystem\Service\DeliveryService.java (178 lines)

137. Current: src\main\java\com\example\EmployeeManagementSystem\Service\DeliveryTimeService.java (74 lines)

138. Current: src\main\java\com\example\EmployeeManagementSystem\Service\DeviceService.java (193 lines)

139. Current: src\main\java\com\example\EmployeeManagementSystem\Service\EmployeeService.java (214 lines)

140. Current: src\main\java\com\example\EmployeeManagementSystem\Service\HolidayService.java (39 lines)

141. Current: src\main\java\com\example\EmployeeManagementSystem\Service\LeaveAccrualService.java (135 lines)

142. Current: src\main\java\com\example\EmployeeManagementSystem\Service\LeaveDashboardService.java (215 lines)

143. Current: src\main\java\com\example\EmployeeManagementSystem\Service\LeaveRequestService.java (589 lines)

144. Current: src\main\java\com\example\EmployeeManagementSystem\Service\ManagerService.java (219 lines)

145. Current: src\main\java\com\example\EmployeeManagementSystem\Service\MaternityLeaveService.java (129 lines)

146. Current: src\main\java\com\example\EmployeeManagementSystem\Service\MealSlotService.java (22 lines)

147. Current: src\main\java\com\example\EmployeeManagementSystem\Service\NotificationService.java (57 lines)

148. Current: src\main\java\com\example\EmployeeManagementSystem\Service\RefreshTokenService.java (41 lines)

149. Current: src\main\java\com\example\EmployeeManagementSystem\Service\RestaurantService.java (162 lines)

150. Current: src\main\java\com\example\EmployeeManagementSystem\Service\ServiceRequestService.java (209 lines)

151. Current: src\main\java\com\example\EmployeeManagementSystem\Service\SickLeaveResetService.java (67 lines)

152. Current: src\main\java\com\example\EmployeeManagementSystem\Service\SubscriptionService.java (228 lines)

153. Current: src\main\java\com\example\EmployeeManagementSystem\Service\TotpService.java (45 lines)

154. Current:
src\main\java\com\example\EmployeeManagementSystem\Service\YearEndCarryForwardService.java (224 lines)

155. Current: src\main\java\com\example\EmployeeManagementSystem\Util\AuthUtil.java (21 lines)

156. Current: src\main\java\com\example\EmployeeManagementSystem\Util\JWTUtil.java (76 lines)

157. Current: src\main\java\com\example\EmployeeManagementSystem\Util\OAuth2SuccessHandler.java (85 lines)

158. Current: src\main\resources\application.yaml (69 lines)

159. Current: src\main\resources\static\dashboard.html (2,217 lines)

160. Current: src\main\resources\static\employee-dashboard.html (1,775 lines)

161. Current: src\main\resources\static\google.html (764 lines)

162. Current: src\main\resources\static\leave_management.html (818 lines)

163. Current: src\main\resources\static\login.html (10 lines)

164. Current: src\main\resources\static\manager-dashboard.html (2,325 lines)

165. Current: src\main\resources\static\pending.html (76 lines)

166. Current: src\main\resources\static\tech-vendor-dashboard.html (1,149 lines)

167. Current: src\main\resources\static\otp-setup.html (442 lines)

168. Current: src\main\resources\static\vendor-dashboard.html (529 lines)

169. Current: src\main\resources\static\vendor-login.html (512 lines)

170. Current:
src\test\java\com\example\EmployeeManagementSystem\EmployeeManagementSystemApplicationTests.java (14 lines)

171. Current: src\test\java\com\example\EmployeeManagementSystem\service\LeaveAccrualServiceTest.java (112 lines)

172. Current: src\test\java\com\example\EmployeeManagementSystem\service\LeaveRequestServiceTest.java (243 lines)

173. Current: src\test\java\com\example\EmployeeManagementSystem\service\SickLeaveResetServiceTest.java (135 lines)

174. Current:
src\test\java\com\example\EmployeeManagementSystem\service\YearEndCarryForwardServiceTest.java (137 lines)

1 Parent: README.md

3 lines

```
1 # SmartEmployeeManagementSystem
2 A secure and scalable Spring Boot backend for employee management, leave tracking, and meal subscription
  services with JWT, OAuth2, API key authentication, and role-based access control.
3
```

2 Current: .mvn\wrapper\maven-wrapper.properties

4 lines

```
1 wrapperVersion=3.3.4
2 distributionType=only-script
3 distributionUrl=https://repo.maven.apache.org/maven2/org/apache/maven/apache-maven/3.9.14/apache-maven-
3.9.14-bin.zip
4
```

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
4      <modelVersion>4.0.0</modelVersion>
5      <parent>
6          <groupId>org.springframework.boot</groupId>
7          <artifactId>spring-boot-starter-parent</artifactId>
8          <version>4.0.5</version>
9          <relativePath/> <!-- lookup parent from repository -->
10     </parent>
11     <groupId>com.example</groupId>
12     <artifactId>EmployeeLeaveManagementSystem</artifactId>
13     <version>0.0.1-SNAPSHOT</version>
14     <name/>
15     <description/>
16     <url/>
17     <licenses>
18         <license/>
19     </licenses>
20     <developers>
21         <developer/>
22     </developers>
23     <scm>
24         <connection/>
25         <developerConnection/>
26         <tag/>
27         <url/>
28     </scm>
29     <properties>
30         <java.version>21</java.version>
31     </properties>
32     <dependencies>
33         <dependency>
34             <groupId>org.springframework.boot</groupId>
35             <artifactId>spring-boot-starter-data-jpa</artifactId>
36         </dependency>
37         <dependency>
38             <groupId>org.springframework.boot</groupId>
39             <artifactId>spring-boot-starter-webmvc</artifactId>
40         </dependency>
41         <dependency>
42             <groupId>org.springdoc</groupId>
43             <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
44             <version>3.0.2</version>
45         </dependency>
46         <dependency>
47             <groupId>dev.samstevens.totp</groupId>
48             <artifactId>totp-spring-boot-starter</artifactId>
49             <version>1.7.1</version>
50         </dependency>
51         <dependency>
52             <groupId>org.springframework.session</groupId>
53             <artifactId>spring-session-core</artifactId>
54         </dependency>
55         <!-- TOTP -->
56         <dependency>
57             <groupId>dev.samstevens.totp</groupId>
58             <artifactId>totp-spring-boot-starter</artifactId>
59             <version>1.7.1</version>
60         </dependency>
61         <!-- -->
62         <dependency>
63             <groupId>org.springframework.boot</groupId>
64             <artifactId>spring-boot-devtools</artifactId>
65             <scope>runtime</scope>
66             <optional>true</optional>
67         </dependency>
68         <dependency>
69             <groupId>com.mysql</groupId>
70             <artifactId>mysql-connector-j</artifactId>
71             <scope>runtime</scope>
72     </dependencies>

```

```

73     <dependency>
74         <groupId>org.springframework.boot</groupId>
75         <artifactId>spring-boot-starter-data-jpa-test</artifactId>
76         <scope>test</scope>
77     </dependency>
78     <dependency>
79         <groupId>com.itextpdf</groupId>
80         <artifactId>layout</artifactId>
81         <version>7.2.5</version>
82     </dependency>
83     <dependency>
84         <groupId>org.springframework.boot</groupId>
85         <artifactId>spring-boot-starter-security</artifactId>
86     </dependency>
87
88     <dependency>
89         <groupId>org.springframework.boot</groupId>
90         <artifactId>spring-boot-starter-security-test</artifactId>
91         <scope>test</scope>
92     </dependency>
93     <dependency>
94         <groupId>org.springframework.boot</groupId>
95         <artifactId>spring-boot-starter-webmvc-test</artifactId>
96         <scope>test</scope>
97     </dependency>
98     <dependency>
99         <groupId>io.jsonwebtoken</groupId>
100        <artifactId>jjwt-api</artifactId>
101        <version>0.13.0</version>
102        <scope>compile</scope>
103    </dependency>
104    <dependency>
105        <groupId>io.jsonwebtoken</groupId>
106        <artifactId>jjwt-impl</artifactId>
107        <version>0.13.0</version>
108        <scope>runtime</scope>
109    </dependency>
110    <dependency>
111        <groupId>io.jsonwebtoken</groupId>
112        <artifactId>jjwt-jackson</artifactId>
113        <version>0.13.0</version>
114        <scope>runtime</scope>
115    </dependency>
116    <dependency>
117        <groupId>org.springframework.boot</groupId>
118        <artifactId>spring-boot-starter-security-oauth2-client</artifactId>
119    </dependency>
120
121    <dependency>
122        <groupId>org.springframework.boot</groupId>
123        <artifactId>spring-boot-starter-security-oauth2-client-test</artifactId>
124        <scope>test</scope>
125    </dependency>
126 </dependencies>
127
128 <build>
129     <plugins>
130         <plugin>
131             <groupId>org.springframework.boot</groupId>
132             <artifactId>spring-boot-maven-plugin</artifactId>
133         </plugin>
134     </plugins>
135 </build>
136
137 </project>
138

```

```

1 package com.example.EmployeeManagementSystem.code;
2
3 import com.itextpdf.kernel.colors.ColorConstants;
4 import com.itextpdf.kernel.colors.DeviceRgb;
5 import com.itextpdf.kernel.font.PdfFont;
6 import com.itextpdf.kernel.font.PdfFontFactory;
7 import com.itextpdf.kernel.geom.PageSize;
8 import com.itextpdf.kernel.pdf.PdfDocument;
9 import com.itextpdf.kernel.pdf.PdfWriter;
10 import com.itextpdf.layout.Document;
11 import com.itextpdf.layout.element.AreaBreak;
12 import com.itextpdf.layout.element.Cell;
13 import com.itextpdf.layout.element.Paragraph;
14 import com.itextpdf.layout.element.Table;
15 import com.itextpdf.layout.element.Text;
16 import com.itextpdf.layout.properties.AreaBreakType;
17 import com.itextpdf.layout.properties.UnitValue;
18 import com.itextpdf.io.font.constants.StandardFonts;
19
20 import java.io.File;
21 import java.io.IOException;
22 import java.nio.file.*;
23 import java.nio.file.attribute.BasicFileAttributes;
24 import java.time.LocalDateTime;
25 import java.time.format.DateTimeFormatter;
26 import java.util.*;
27
28 /**
29  * CodeToPdf - Extracts all source code files from the current, parent,
30  * and immediate child directories into a formatted PDF document.
31  *
32  * Dependencies (add to your pom.xml or build.gradle):
33  *   com.itextpdf:itext7-core:7.2.5 (or itext7 bundle)
34  *
35  * Compile:
36  *   javac -cp itext7-core-7.2.5.jar:. CodeToPdf.java
37  *
38  * Run:
39  *   java -cp itext7-core-7.2.5.jar:. CodeToPdf
40  *
41  * Or with Maven:
42  *   mvn compile exec:java -Dexec.mainClass=CodeToPdf
43  */
44 public class CodeToPdf {
45
46     // Configuration
47
48     /** Output PDF filename */
49     private static final String OUTPUT_FILE = "code_export.pdf";
50
51     /** Maximum file size to include (skip very large generated files) */
52     private static final long MAX_FILE_SIZE_BYTES = 500_000; // 500 KB
53
54     /** Source code file extensions to include */
55     private static final Set<String> CODE_EXTENSIONS = new LinkedHashSet<>(Arrays.asList(
56         // JVM languages
57         ".java", ".kt", ".groovy", ".scala", ".clj",
58         // Web
59         ".js", ".ts", ".jsx", ".tsx", ".html", ".css", ".scss", ".less",
60         // Python / Ruby / PHP
61         ".py", ".rb", ".php",
62         // Systems / low-level
63         ".c", ".cpp", ".cc", ".h", ".hpp", ".rs", ".go", ".swift",
64         // Data / config
65         ".xml", ".json", ".yaml", ".yml", ".toml", ".properties", ".env",
66         // Shell / scripts
67         ".sh", ".bash", ".zsh", ".bat", ".ps1",
68         // Build
69         ".gradle", ".cmake", "Makefile", "Dockerfile",
70         // Other
71         ".sql", ".md", ".txt"

```

```

72     });
73
74     /** Directories to skip entirely */
75     private static final Set<String> SKIP_DIRS = new HashSet<>(Arrays.asList(
76         ".git", ".svn", ".hg", "node_modules", "target", "build",
77         "out", ".idea", ".vscode", "__pycache__", ".gradle", "dist", "vendor"
78     ));
79
80     // Colors
81     private static final DeviceRgb COLOR_HEADER_BG = new DeviceRgb(0x18, 0x5F, 0xA5); // blue
82     private static final DeviceRgb COLOR_HEADER_TEXT = new DeviceRgb(0xFF, 0xFF, 0xFF);
83     private static final DeviceRgb COLOR_FILE_TITLE = new DeviceRgb(0x18, 0x5F, 0xA5);
84     private static final DeviceRgb COLOR_CODE_BG = new DeviceRgb(0xF7, 0xF7, 0xF7);
85     private static final DeviceRgb COLOR_LINE_NUM = new DeviceRgb(0x99, 0x99, 0x99);
86     private static final DeviceRgb COLOR_TOC_HEADER = new DeviceRgb(0x18, 0x5F, 0xA5);
87
88     // Font sizes
89     private static final float FONT_TITLE = 20f;
90     private static final float FONT_SECTION = 13f;
91     private static final float FONT_FILE = 11f;
92     private static final float FONT_CODE = 7.5f;
93     private static final float FONT_TOC = 10f;
94
95     // State
96     private final List<FileEntry> fileEntries = new ArrayList<>();
97     private int totalLines = 0;
98
99     //
100
101     public static void main(String[] args) throws Exception {
102         new CodeToPdf().run();
103     }
104
105     private void run() throws Exception {
106         Path currentDir = Paths.get("").toAbsolutePath();
107         Path parentDir = currentDir.getParent();
108
109         System.out.println("=== Code to PDF Extractor ===");
110         System.out.println("Current directory : " + currentDir);
111         System.out.println("Parent directory : " + (parentDir != null ? parentDir : "(none)"));
112         System.out.println();
113
114         // 1. Collect files
115
116         // Parent directory (top-level files only, not recursive)
117         if (parentDir != null && Files.exists(parentDir)) {
118             collectFiles(parentDir, false, "Parent");
119         }
120
121         // Current directory (recursive – includes all child subdirectories)
122         collectFiles(currentDir, true, "Current");
123
124         System.out.printf("%nFound %d code file(s) with %,d total lines.%n",
125             fileEntries.size(), totalLines);
126
127         if (fileEntries.isEmpty()) {
128             System.out.println("No code files found. Exiting.");
129             return;
130         }
131
132         // 2. Generate PDF
133         generatePdf(currentDir);
134
135         System.out.println("PDF saved to: " + Paths.get(OUTPUT_FILE).toAbsolutePath());
136     }
137
138     // File Collection
139
140     private void collectFiles(Path root, boolean recursive, String scope) throws IOException {
141         if (!Files.exists(root)) return;
142
143         Files.walkFileTree(root, new SimpleFileVisitor<>() {
144             @Override
145             public FileVisitResult preVisitDirectory(Path dir, BasicFileAttributes attrs) {
146                 String name = dir.getFileName() == null ? "" : dir.getFileName().toString();
147                 // Never recurse into skip-dirs; always allow the root itself

```

```

148         if (!dir.equals(root) && SKIP_DIRS.contains(name)) {
149             return FileVisitResult.SKIP_SUBTREE;
150         }
151         // If not recursive, skip any subdirectory of root
152         if (!recursive && !dir.equals(root)) {
153             return FileVisitResult.SKIP_SUBTREE;
154         }
155         return FileVisitResult.CONTINUE;
156     }
157
158     @Override
159     public FileVisitResult visitFile(Path file, BasicFileAttributes attrs) throws IOException {
160         if (isCodeFile(file) && attrs.size() <= MAX_FILE_SIZE_BYTES) {
161             String content = Files.readString(file);
162             int lines = content.split("\n", -1).length;
163             String relativePath = scope + ": " + root.relativize(file);
164             fileEntries.add(new FileEntry(relativePath, file, content, lines));
165             totalLines += lines;
166             System.out.println("  " + relativePath + " (" + lines + " lines)");
167         }
168         return FileVisitResult.CONTINUE;
169     }
170 });
171 }
172
173 private boolean isCodeFile(Path file) {
174     String name = file.getFileName().toString();
175     // Check for exact filename match (e.g. "Makefile", "Dockerfile")
176     if (CODE_EXTENSIONS.contains(name)) return true;
177     // Check extension
178     int dot = name.lastIndexOf('.');
179     if (dot >= 0) {
180         return CODE_EXTENSIONS.contains(name.substring(dot));
181     }
182     return false;
183 }
184
185 // PDF Generation
186
187 private void generatePdf(Path currentDir) throws IOException {
188     PdfWriter writer = new PdfWriter(OUTPUT_FILE);
189     PdfDocument pdfDoc = new PdfDocument(writer);
190     Document doc = new Document(pdfDoc, PageSize.A4);
191     doc.setMargins(40, 40, 40, 40);
192
193     PdfFont sansFont = PdfFontFactory.createFont(StandardFonts.HELVETICA);
194     PdfFont boldFont = PdfFontFactory.createFont(StandardFonts.HELVETICA_BOLD);
195     PdfFont monoFont = PdfFontFactory.createFont(StandardFonts.COURIER);
196
197     // Cover page
198     addCoverPage(doc, sansFont, boldFont, currentDir);
199     doc.add(new AreaBreak(AreaBreakType.NEXT_PAGE));
200
201     // Table of contents
202     addTableOfContents(doc, sansFont, boldFont);
203     doc.add(new AreaBreak(AreaBreakType.NEXT_PAGE));
204
205     // Code sections
206     for (int i = 0; i < fileEntries.size(); i++) {
207         FileEntry entry = fileEntries.get(i);
208         addCodeSection(doc, entry, i + 1, sansFont, boldFont, monoFont);
209         if (i < fileEntries.size() - 1) {
210             doc.add(new AreaBreak(AreaBreakType.NEXT_PAGE));
211         }
212     }
213
214     doc.close();
215 }
216
217 private void addCoverPage(Document doc, PdfFont sansFont, PdfFont boldFont, Path dir)
218     throws IOException {
219
220     doc.add(new Paragraph("\n\n\n"));
221
222     // Big title
223     doc.add(new Paragraph("Code Export"))

```

```

224         .setFont(boldFont).setFontSize(FONT_TITLE + 10)
225         .setFillColor(COLOR_FILE_TITLE)
226         .setMarginBottom(6));
227
228     doc.add(new Paragraph("Source Code PDF Report")
229         .setFont(sansFont).setFontSize(FONT_SECTION)
230         .setFillColor(ColorConstants.DARK_GRAY)
231         .setMarginBottom(24));
232
233     // Stats table
234     Table stats = new Table(UnitValue.createPercentArray(new float[]{40, 60}))
235         .setWidth(UnitValue.createPercentValue(80));
236     addStatRow(stats, "Generated",      LocalDateTime.now()
237         .format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm")), sansFont, boldFont);
238     addStatRow(stats, "Root directory", dir.toString(), sansFont, boldFont);
239     addStatRow(stats, "Files found",   String.valueOf(fileEntries.size()), sansFont, boldFont);
240     addStatRow(stats, "Total lines",   String.format("%,d", totalLines), sansFont, boldFont);
241     doc.add(stats);
242 }
243
244 private void addStatRow(Table t, String label, String value,
245     PdfFont sans, PdfFont bold) {
246     t.addCell(new Cell().add(new Paragraph(label)
247         .setFont(bold).setFontSize(10)
248         .setFillColor(ColorConstants.DARK_GRAY))
249         .setBorder(null).setPaddingBottom(6));
250     t.addCell(new Cell().add(new Paragraph(value)
251         .setFont(sans).setFontSize(10))
252         .setBorder(null).setPaddingBottom(6));
253 }
254
255 private void addTableOfContents(Document doc, PdfFont sansFont, PdfFont boldFont)
256     throws IOException {
257     doc.add(new Paragraph("Table of Contents")
258         .setFont(boldFont).setFontSize(FONT_SECTION + 2)
259         .setFillColor(COLOR_TOC_HEADER)
260         .setMarginBottom(12));
261
262     for (int i = 0; i < fileEntries.size(); i++) {
263         FileEntry entry = fileEntries.get(i);
264         String label = String.format("%d.  %s  (%,d lines)",
265             i + 1, entry.relativePath, entry.lineCount);
266         doc.add(new Paragraph(label)
267             .setFont(sansFont).setFontSize(FONT_TOC)
268             .setMarginBottom(3));
269     }
270 }
271
272 private void addCodeSection(Document doc, FileEntry entry, int index,
273     PdfFont sansFont, PdfFont boldFont, PdfFont monoFont)
274     throws IOException {
275
276     // Section header bar
277     Table header = new Table(UnitValue.createPercentArray(new float[]{5, 75, 20}))
278         .setWidth(UnitValue.createPercentValue(100))
279         .setMarginBottom(8);
280
281     header.addCell(new Cell().add(new Paragraph(String.valueOf(index))
282         .setFont(boldFont).setFontSize(FONT_FILE)
283         .setFillColor(COLOR_HEADER_TEXT))
284         .setBackgroundColor(COLOR_HEADER_BG)
285         .setBorder(null).setPadding(6));
286
287     header.addCell(new Cell().add(new Paragraph(entry.relativePath)
288         .setFont(boldFont).setFontSize(FONT_FILE)
289         .setFillColor(COLOR_HEADER_TEXT))
290         .setBackgroundColor(COLOR_HEADER_BG)
291         .setBorder(null).setPadding(6));
292
293     header.addCell(new Cell().add(new Paragraph(entry.lineCount + " lines")
294         .setFont(sansFont).setFontSize(FONT_FILE - 1)
295         .setFillColor(COLOR_HEADER_TEXT))
296         .setBackgroundColor(COLOR_HEADER_BG)
297         .setBorder(null).setPadding(6));
298
299     doc.add(header);

```



```

300
301 // Code block with line numbers
302 String[] lines = entry.content.split("\n", -1);
303 int padWidth = String.valueOf(lines.length).length();
304
305 for (int ln = 0; ln < lines.length; ln++) {
306     String lineNum = String.format("%" + padWidth + "d", ln + 1);
307     String codeLine = lines[ln]
308         .replace("\t", "    ") // expand tabs
309         .replace("\r", "");     // strip CR
310
311     // Truncate very long lines
312     if (codeLine.length() > 200) {
313         codeLine = codeLine.substring(0, 197) + "...";
314     }
315
316     Text numText = new Text(lineNum + " ").setFont(monoFont)
317         .setFontSize(FONT_CODE).setFontColor(COLOR_LINE_NUM);
318     Text codeText = new Text(codeLine).setFont(monoFont)
319         .setFontSize(FONT_CODE).setFontColor(ColorConstants.BLACK);
320
321     Paragraph p = new Paragraph().add(numText).add(codeText)
322         .setBackgroundColor(ln % 2 == 0 ? COLOR_CODE_BG : ColorConstants.WHITE)
323         .setMargin(0).setPaddingLeft(4).setPaddingRight(4)
324         .setMultipliedLeading(1.3f)
325         .setFixedLeading(FONT_CODE + 2.5f);
326
327     doc.add(p);
328 }
329 }
330
331 // Data class
332
333 private static class FileEntry {
334     final String relativePath;
335     final Path absolutePath;
336     final String content;
337     final int lineCount;
338
339     FileEntry(String relativePath, Path absolutePath, String content, int lineCount) {
340         this.relativePath = relativePath;
341         this.absolutePath = absolutePath;
342         this.content = content;
343         this.lineCount = lineCount;
344     }
345 }
346 }
347
348

```

```

1 //package com.example.EmployeeManagementSystem.config;
2 //
3 //import org.springframework.beans.factory.annotation.Qualifier;
4 //import org.springframework.context.annotation.Bean;
5 //import org.springframework.context.annotation.Configuration;
6 //import org.springframework.core.annotation.Order;
7 //import org.springframework.http.HttpMethod;
8 //import org.springframework.security.authentication.AuthenticationManager;
9 //import org.springframework.security.authentication.ProviderManager;
10 //import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
11 //import org.springframework.security.config.Customizer;
12 //import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
13 //import org.springframework.security.config.annotation.web.builders.HttpSecurity;
14 //import org.springframework.security.core.userdetails.UserDetailsService;
15 //import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
16 //import org.springframework.security.crypto.password.PasswordEncoder;
17 //import org.springframework.security.web.SecurityFilterChain;
18 //
19 //
20 // @Configuration
21 // @EnableMethodSecurity
22 // public class BasicAuthConfig {
23 //     @Bean
24 //     @Order(3)
25 //     public SecurityFilterChain basicAuth(HttpSecurity security){
26 //         security
27 //             .csrf(csrf-> csrf.disable())
28 //             .authorizeHttpRequests(
29 //                 auth->auth
30 //                 .requestMatchers("/employee/register","/employee/register/manager").permitAll()
31 //                 .requestMatchers(HttpMethod.POST,"/vendors").permitAll()
32 //                 .anyRequest().authenticated()
33 //             )
34 //             .httpBasic(Customizer.withDefaults());
35 //         return security.build();
36 //     }
37 //
38 //     @Bean
39 //     public PasswordEncoder passwordEncoder(){
40 //         return new BCryptPasswordEncoder();
41 //     }
42 //
43 //     @Bean
44 //     public AuthenticationManager authenticationManager(@Qualifier("combinedUserDetailService")
45 // UserDetailsService userDetailsService,
46 // PasswordEncoder passwordEncoder){
47 //         DaoAuthenticationProvider employeeAuthenticationProvider=new
48 // DaoAuthenticationProvider(userDetailsService);
49 //         employeeAuthenticationProvider.setPasswordEncoder(passwordEncoder);
50 //         return new ProviderManager(employeeAuthenticationProvider);
51 //     }
52 // }

```

```

1 // src/main/java/com/example/EmployeeManagementSystem/config/JWTAuth/JWTAuthConfig.java
2 package com.example.EmployeeManagementSystem.config.JWTAuth;
3
4 import com.example.EmployeeManagementSystem.Filter.ApiKeyFilter;
5 import com.example.EmployeeManagementSystem.Filter.JwtAuthFilter;
6 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
7 import com.example.EmployeeManagementSystem.Repository.RefreshTokenRepository;
8 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
9 import com.example.EmployeeManagementSystem.Service.CustomOAuth2UserService;
10 import com.example.EmployeeManagementSystem.Service.RefreshTokenService;
11 import com.example.EmployeeManagementSystem.Util.JWTUtil;
12 import com.example.EmployeeManagementSystem.Util.OAuth2SuccessHandler;
13 import com.example.EmployeeManagementSystem.security.ApiKeyAuthenticationProvider;
14 import org.springframework.beans.factory.annotation.Qualifier;
15 import org.springframework.context.annotation.Bean;
16 import org.springframework.context.annotation.Configuration;
17 import org.springframework.context.annotation.Primary;
18 import org.springframework.http.HttpMethod;
19 import org.springframework.security.authentication.AuthenticationManager;
20 import org.springframework.security.authentication.ProviderManager;
21 import org.springframework.security.authentication.dao.DaoAuthenticationProvider;
22 import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
23 import org.springframework.security.config.annotation.web.builders.HttpSecurity;
24 import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
25 import org.springframework.security.config.http.SessionCreationPolicy;
26 import org.springframework.security.core.userdetails.UserDetailsService;
27 import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
28 import org.springframework.security.crypto.password.PasswordEncoder;
29 import org.springframework.security.web.SecurityFilterChain;
30 import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
31 import org.springframework.security.web.context.DelegatingSecurityContextRepository;
32 import org.springframework.security.web.context.HttpSessionSecurityContextRepository;
33 import org.springframework.security.web.context.RequestAttributeSecurityContextRepository;
34 import org.springframework.web.client.RestTemplate;
35 import org.springframework.web.cors.CorsConfiguration;
36 import org.springframework.web.cors.CorsConfigurationSource;
37 import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
38
39 import java.util.List;
40
41 @Configuration
42 @EnableWebSecurity
43 @EnableMethodSecurity
44 public class JWTAuthConfig {
45
46     private final JwtAuthFilter jwtAuthFilter;
47     private final ApiKeyAuthenticationProvider apiKeyAuthProvider;
48     private final EmployeeRepo employeeRepo;
49     private final VendorRepo vendorRepo;
50     private final RefreshTokenService refreshTokenService;
51     private final RefreshTokenRepository refreshTokenRepository;
52     private final JWTUtil jwtUtil;
53
54     public JWTAuthConfig(JwtAuthFilter jwtAuthFilter,
55                         ApiKeyAuthenticationProvider apiKeyAuthProvider,
56                         EmployeeRepo employeeRepo,
57                         VendorRepo vendorRepo,
58                         RefreshTokenService refreshTokenService,
59                         RefreshTokenRepository refreshTokenRepository,
60                         JWTUtil jwtUtil) {
61         this.jwtAuthFilter = jwtAuthFilter;
62         this.apiKeyAuthProvider = apiKeyAuthProvider;
63         this.employeeRepo = employeeRepo;
64         this.vendorRepo = vendorRepo;
65         this.refreshTokenService = refreshTokenService;
66         this.refreshTokenRepository = refreshTokenRepository;
67         this.jwtUtil = jwtUtil;
68     }
69
70     @Bean
71     public RestTemplate getRestTemplate() {

```

```

72     return new RestTemplate();
73 }
74
75 @Bean
76 public ApiKeyFilter apiKeyFilter(
77     @Qualifier("apiKeyAuthenticationManager") AuthenticationManager apiKeyAuthManager) {
78     return new ApiKeyFilter(apiKeyAuthManager);
79 }
80
81 @Bean
82 public PasswordEncoder passwordEncoder() {
83     return new BCryptPasswordEncoder();
84 }
85
86 @Bean
87 public AuthenticationManager authenticationManager(
88     @Qualifier("combinedUserDetailsService") UserDetailsService userDetailsService,
89     PasswordEncoder passwordEncoder) {
90     DaoAuthenticationProvider provider = new DaoAuthenticationProvider(userDetailsService);
91     provider.setPasswordEncoder(passwordEncoder);
92     return new ProviderManager(provider);
93 }
94
95 @Bean
96 public CustomOAuth2UserService customOAuth2UserService() {
97     return new CustomOAuth2UserService(employeeRepo, vendorRepo, passwordEncoder());
98 }
99
100 @Bean
101 public OAuth2SuccessHandler oAuth2SuccessHandler() {
102     return new OAuth2SuccessHandler(jwtUtil, employeeRepo, vendorRepo,
103         refreshTokenService, refreshTokenRepository);
104 }
105
106 @Bean
107 @Qualifier("apiKeyAuthenticationManager")
108 public AuthenticationManager apiKeyAuthenticationManager() {
109     return new ProviderManager(apiKeyAuthProvider);
110 }
111
112 @Bean
113 public SecurityFilterChain securityFilterChain(HttpSecurity http, ApiKeyFilter apiKeyFilter) throws
Exception {
114     http
115         .csrf(csrf -> csrf.disable())
116         .cors(cors -> cors.configurationSource(corsConfigurationSource()))
117         .securityContext(ctx -> ctx
118             .securityContextRepository(new DelegatingSecurityContextRepository(
119                 new HttpSessionSecurityContextRepository(),
120                 new RequestAttributeSecurityContextRepository()
121             ))
122         )
123         .sessionManagement(session -> session
124             .sessionCreationPolicy(SessionCreationPolicy.IF_REQUIRED)
125             .maximumSessions(1)
126             .maxSessionsPreventsLogin(false)
127         )
128         .authorizeHttpRequests(auth -> auth
129             // Auth endpoints
130             .requestMatchers("/Authenticate",
131                 "/Authenticate/totp", "/Authenticate/refresh").permitAll()
132             .requestMatchers("/session/login", "/session/logout").permitAll()
133             // OAuth2 endpoints
134             .requestMatchers("/oauth2/**", "/login/oauth2/**").permitAll()
135             .requestMatchers("/auth/google/init").permitAll()
136             // API docs
137             .requestMatchers("/swagger-ui/**", "/v3/api-docs/**").permitAll()
138             // Shared login page
139             .requestMatchers("/login.html", "/google.html").permitAll()
140             .requestMatchers("/error").permitAll()
141             // Dashboards – served as static HTML, actual data is protected by JWT
142             .requestMatchers("/login.html",

```

```

146         "/google.html",
147         "/vendor-login.html",
148         "/vendor-dashboard.html",
149         "/tech-vendor-dashboard.html",
150         "/employee-dashboard.html",
151         "/manager-dashboard.html",
152         "/dashboard.html",
153         "/pending.html",
154         "/leave_management.html",
155         "/totp-setup.html").permitAll()
156
157         // Vendor pages
158         .requestMatchers("/vendor-login.html").permitAll()
159         .requestMatchers("/vendor-dashboard.html").permitAll()
160
161         // API key generation requires authentication
162         .requestMatchers("/api-keys/**").authenticated()
163         .requestMatchers("/manager/createAdmin").permitAll()
164         // totp
165         .requestMatchers("/totp/**").authenticated()
166         // notification
167         .requestMatchers(HttpMethod.GET, "/notifications").authenticated()
168         .requestMatchers(HttpMethod.GET, "/notifications/unread-count").authenticated()
169         .requestMatchers(HttpMethod.POST, "/notifications/mark-read").authenticated()
170         // Everything else requires a valid token
171         .anyRequest().authenticated()
172     )
173     .oauth2Login(oauth -> oauth
174         .userInfoEndpoint(userInfo -> userInfo
175             .userService(customOAuth2UserService()))
176         )
177     .successHandler(oAuth2SuccessHandler())
178 )
179 .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class)
180 .addFilterBefore(apiKeyFilter, UsernamePasswordAuthenticationFilter.class);
181
182     return http.build();
183 }
184 @Bean
185 public CorsConfigurationSource corsConfigurationSource() {
186     CorsConfiguration config = new CorsConfiguration();
187     config.setAllowedOrigins(List.of("http://192.168.1.3:8080"));
188     config.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE", "OPTIONS"));
189     config.setAllowedHeaders(List.of("*"));
190     config.setAllowCredentials(true);
191
192     UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
193     source.registerCorsConfiguration("/**", config);
194     return source;
195 }
196 }
197

```

```
1 package com.example.EmployeeManagementSystem.config;
2
3 import com.example.EmployeeManagementSystem.Entity.LeaveType;
4 import com.example.EmployeeManagementSystem.Enum.Gender;
5 import com.example.EmployeeManagementSystem.Repository.LeaveTypeRepository;
6 import org.springframework.boot.CommandLineRunner;
7 import org.springframework.stereotype.Component;
8
9 @Component
10 public class LeaveTypeDataInitializer implements CommandLineRunner {
11
12     private final LeaveTypeRepository leaveTypeRepository;
13
14     public LeaveTypeDataInitializer(LeaveTypeRepository leaveTypeRepository) {
15         this.leaveTypeRepository = leaveTypeRepository;
16     }
17
18     @Override
19     public void run(String... args) {
20         ensureLeaveType("SICK", false, true, 0, false, null);
21         ensureLeaveType("CASUAL", true, false, 30, false, null);
22         ensureLeaveType("MATERNITY", false, false, 0, true, Gender.F);
23         ensureLeaveType("UNPAID", false, false, 0, false, null);
24     }
25
26     private void ensureLeaveType(String name,
27                                boolean carriesForward,
28                                boolean monthlyReset,
29                                int maxCarryForward,
30                                boolean genderRestricted,
31                                Gender genderRestriction) {
32         leaveTypeRepository.findByName(name).orElseGet(() -> {
33             LeaveType leaveType = new LeaveType();
34             leaveType.setName(name);
35             leaveType.setCarriesForward(carriesForward);
36             leaveType.setMonthlyReset(monthlyReset);
37             leaveType.setMaxCarryForward(maxCarryForward);
38             leaveType.setGenderRestricted(genderRestricted);
39             leaveType.setGenderRestriction(genderRestriction);
40             return leaveTypeRepository.save(leaveType);
41         });
42     }
43 }
44
```

8 Current:

src\main\java\com\example\EmployeeManagementSystem\config\Session\SessionAuthConfig.java

```
1 //package com.example.EmployeeManagementSystem.config.Session;
2 //
3 //import org.springframework.context.annotation.Bean;
4 //import org.springframework.context.annotation.Configuration;
5 //import org.springframework.core.annotation.Order;
6 //import org.springframework.security.config.annotation.web.builders.HttpSecurity;
7 //import org.springframework.security.web.SecurityFilterChain;
8 //import org.springframework.security.web.context.HttpSessionSecurityContextRepository;
9 //
10 //@Configuration
11 //public class SessionAuthConfig {
12 //
13 //    @Bean
14 //    @Order(1)
15 //    public SecurityFilterChain sessionFilterChain(HttpSecurity http) throws Exception {
16 //        http
17 //            .securityMatcher("/session/**", "/web/**")
18 //            .csrf(csrf -> csrf.disable())
19 //            .authorizeHttpRequests(auth -> auth
20 //                .requestMatchers("/session/login", "/session/logout").permitAll()
21 //                .anyRequest().authenticated()
22 //            )
23 //            .sessionManagement(session -> session
24 //                .maximumSessions(1) // one session per user
25 //                .maxSessionsPreventsLogin(false) // new login kicks old session
26 //            )
27 //            .securityContext(ctx -> ctx
28 //                .securityContextRepository(new HttpSessionSecurityContextRepository())
29 //            );
30 //        return http.build();
31 //    }
32 //}
```

```
1 package com.example.EmployeeManagementSystem.config;
2
3 import io.swagger.v3.oas.models.Components;
4 import io.swagger.v3.oas.models.OpenAPI;
5 import io.swagger.v3.oas.models.security.SecurityRequirement;
6 import io.swagger.v3.oas.models.security.SecurityScheme;
7 import org.springframework.context.annotation.Bean;
8 import org.springframework.context.annotation.Configuration;
9
10 @Configuration
11 public class swaggerConfig {
12     @Bean
13     public OpenAPI openAPI() {
14         return new OpenAPI()
15             .components(new Components()
16                 .addSecuritySchemes("bearerAuth",
17                     new SecurityScheme()
18                         .type(SecurityScheme.Type.HTTP)
19                         .scheme("bearer")
20                         .bearerFormat("JWT")
21                 )
22             )
23             .addSecurityItem(new SecurityRequirement().addList("bearerAuth"));
24     }
25     // @Bean
26     // public OpenAPI openAPI() {
27     //     return new OpenAPI()
28     //         .components(new Components()
29     //             .addSecuritySchemes("basicAuth",
30     //                 new SecurityScheme()
31     //                     .type(SecurityScheme.Type.HTTP)
32     //                     .scheme("basic")
33     //             )
34     //         )
35     //         .addSecurityItem(new SecurityRequirement().addList("basicAuth"));
36     // }
37 }
38
```



```

1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.*;
4 import com.example.EmployeeManagementSystem.Entity.Employee;
5 import com.example.EmployeeManagementSystem.Service.*;
6 import org.springframework.format.annotation.DateTimeFormat;
7 import org.springframework.http.HttpStatus;
8 import org.springframework.http.ResponseEntity;
9 import org.springframework.security.access.prepost.PreAuthorize;
10 import org.springframework.web.bind.annotation.*;
11
12 import java.time.LocalDate;
13 import java.util.List;
14
15 @RestController
16 @RequestMapping("/admin")
17 @PreAuthorize("hasRole('ADMIN')")
18 public class AdminController {
19
20     private final EmployeeService employeeService;
21     private final DeliveryService.VendorService vendorService;
22     private final LeaveRequestService leaveRequestService;
23     private final RestaurantService restaurantService;
24     private final SubscriptionService subscriptionService;
25     private final LeaveAccrualService accrualService;
26     private final SickLeaveResetService resetService;
27
28     public AdminController(
29         EmployeeService employeeService,
30         DeliveryService.VendorService vendorService,
31         LeaveRequestService leaveRequestService,
32         RestaurantService restaurantService,
33         SubscriptionService subscriptionService,
34         LeaveAccrualService accrualService, SickLeaveResetService resetService) {
35         this.employeeService = employeeService;
36         this.vendorService = vendorService;
37         this.leaveRequestService = leaveRequestService;
38         this.restaurantService = restaurantService;
39         this.subscriptionService = subscriptionService;
40         this.accrualService = accrualService;
41         this.resetService = resetService;
42     }
43
44     //
45     // EMPLOYEE BRANCH
46     //
47
48     /** List every employee in the system. */
49     @GetMapping("/employees")
50     public ResponseEntity<?> getAllEmployees() {
51         return ResponseEntity.ok(employeeService.getAllEmployees());
52     }
53
54     /** Create a regular employee. */
55     @PostMapping("/employees")
56     public ResponseEntity<Employee> createEmployee(@RequestBody AdminEmployeeDTO dto) {
57         return ResponseEntity.status(HttpStatus.CREATED)
58             .body(employeeService.createEmployee(dto));
59     }
60
61     @PreAuthorize("hasRole('ADMIN')")
62     @GetMapping("/managers")
63     public ResponseEntity<List<ManagerDTO>> getAllManagers(){
64         return employeeService.getAllManagers();
65     }
66
67
68     /** Promote / create a manager. */
69     @PostMapping("/employees/manager")
70     public ResponseEntity<Employee> createManager(@RequestBody EmployeeDTO dto) {
71         return ResponseEntity.status(HttpStatus.CREATED)

```

```

72         .body(employeeService.createManager(dto));
73     }
74
75     /** Update any employee record. */
76     @PutMapping("/employees/{id}")
77     public ResponseEntity<Employee> updateEmployee(
78         @PathVariable long id,
79         @RequestBody EmployeeDTO dto) {
80         return ResponseEntity.ok(employeeService.updateEmployee(id, dto));
81     }
82
83     /** Hard-delete an employee. */
84     @DeleteMapping("/employees/{id}")
85     public ResponseEntity<String> deleteEmployee(@PathVariable long id) {
86         employeeService.deleteEmployee(id);
87         return ResponseEntity.ok("Employee with id: " + id + " has been deleted.");
88     }
89
90     /** Soft-deactivate an employee (sets status to INACTIVE). */
91     @PutMapping("/employees/{id}/inactive")
92     public ResponseEntity<String> deactivateEmployee(@PathVariable long id) {
93         return employeeService.inactivateUser(id);
94     }
95
96     //
97     // VENDOR BRANCH (only admin can register vendors)
98     //
99
100    /** List every vendor. */
101    @GetMapping("/vendors")
102    public ResponseEntity<List<VendorDTO>> getAllVendors() {
103        return ResponseEntity.ok(vendorService.getAllVendors());
104    }
105
106    @PostMapping("/vendors")
107    public ResponseEntity<VendorDTO> registerVendor(@RequestBody VendorRequest request) {
108        return ResponseEntity.status(HttpStatus.CREATED)
109            .body(vendorService.registerVendor(request));
110    }
111
112    /** Remove a vendor by ID. */
113    @DeleteMapping("/vendors/{vendorId}")
114    public ResponseEntity<String> deleteVendor(@PathVariable Long vendorId) {
115        vendorService.deleteVendorById(vendorId);
116        return ResponseEntity.ok("Vendor with id: " + vendorId + " has been removed.");
117    }
118
119    //
120    // LEAVE MANAGEMENT BRANCH
121    //
122
123    /** Fetch all pending leave requests across the organisation. */
124    @GetMapping("/leaves/pending")
125    public ResponseEntity<?> getPendingLeaves() {
126        return ResponseEntity.ok(leaveRequestService.getPendingLeaves());
127    }
128
129    /** Approve a leave request. */
130    @PutMapping("/leaves/{id}/approve")
131    public ResponseEntity<?> approveLeave(@PathVariable Long id) {
132        return ResponseEntity.ok(leaveRequestService.approveLeave(id));
133    }
134
135    /** Reject a leave request. */
136    @PutMapping("/leaves/{id}/reject")
137    public ResponseEntity<?> rejectLeave(@PathVariable Long id) {
138        return ResponseEntity.ok(leaveRequestService.rejectLeave(id));
139    }
140
141    //
142    // RESTAURANT MANAGEMENT BRANCH
143    //
144
145    /** View all restaurants. */
146    @GetMapping("/restaurants")
147    public ResponseEntity<?> getAllRestaurants() {

```

```
148         return ResponseEntity.ok(restaurantService.getAllActiveRestaurants());
149     }
150
151     //
152     // SUBSCRIPTION MANAGEMENT BRANCH
153     //
154
155     /** View all active subscriptions. */
156     @GetMapping("/subscriptions")
157     public ResponseEntity<?> getAllSubscriptions() {
158         return ResponseEntity.ok(subscriptionService.getAllSubscriptions());
159     }
160
161     @PostMapping("/leave/accrue")
162     @PreAuthorize("hasRole('ADMIN')")
163     public ResponseEntity<String> triggerAccrual(
164         @RequestParam(required = false)
165         @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate date
166     ) {
167         LocalDate accrualDate = (date != null) ? date : LocalDate.now();
168         accrualService.runMonthlyAccrual(accrualDate);
169         return ResponseEntity.ok("Accrual complete for " + accrualDate);
170     }
171
172     @PostMapping("/reset-sick")
173     @PreAuthorize("hasRole('ADMIN')")
174     public ResponseEntity<String> triggerSickReset(
175         @RequestParam(required = false)
176         @DateTimeFormat(iso = DateTimeFormat.ISO.DATE) LocalDate date
177     ) {
178         LocalDate resetDate = (date != null) ? date : LocalDate.now();
179         resetService.runMonthlyReset(resetDate);
180         return ResponseEntity.ok("Sick leave reset complete for " + resetDate);
181     }
182 }
```

```

1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.Entity.ApiKey;
4 import com.example.EmployeeManagementSystem.Repository.ApiKeyRepository;
5 import com.example.EmployeeManagementSystem.Service.ApiKeyService;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.security.access.prepost.PreAuthorize;
8 import org.springframework.web.bind.annotation.*;
9
10 import java.util.List;
11
12 @RestController
13 @RequestMapping("/api-keys")
14 public class ApiKeyController {
15
16     private final ApiKeyService apiKeyService;
17     private final ApiKeyRepository apiKeyRepository; // instance, lowercase
18
19     public ApiKeyController(ApiKeyService apiKeyService,
20                             ApiKeyRepository apiKeyRepository) {
21         this.apiKeyService = apiKeyService;
22         this.apiKeyRepository = apiKeyRepository;
23     }
24
25     @PostMapping("/vendor/{vendorId}/generate")
26     @PreAuthorize("hasRole('ADMIN')")
27     public ResponseEntity<ApiKey> generateVendorKey(
28         @PathVariable Long vendorId,
29         @RequestParam String name,
30         @RequestParam(defaultValue = "VENDOR_WRITE") String permissions,
31         @RequestParam(defaultValue = "365") int validDays) {
32         return ResponseEntity.ok(
33             apiKeyService.generateForVendor(vendorId, name, permissions, validDays));
34     }
35
36     @GetMapping("/vendor/{vendorId}")
37     @PreAuthorize("hasRole('ADMIN')")
38     public ResponseEntity<List<ApiKey>> getVendorKeys(@PathVariable Long vendorId) {
39         return ResponseEntity.ok(apiKeyRepository.findByVendor_IdAndActiveTrue(vendorId));
40     }
41
42     @DeleteMapping("/{keyId}/revoke")
43     @PreAuthorize("hasRole('ADMIN')")
44     public ResponseEntity<String> revokeKey(@PathVariable Long keyId) {
45         apiKeyRepository.findById(keyId).ifPresent(key -> {
46             key.setActive(false);
47             apiKeyRepository.save(key);
48         });
49         return ResponseEntity.ok("Key revoked.");
50     }
51 }

```

```

1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.AccessTokenRequest;
4 import com.example.EmployeeManagementSystem.DTO.AuthRequest;
5 import com.example.EmployeeManagementSystem.DTO.AuthResponse;
6 import com.example.EmployeeManagementSystem.Entity.Employee;
7 import com.example.EmployeeManagementSystem.Entity.RefreshToken;
8 import com.example.EmployeeManagementSystem.Entity.Vendor;
9 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
10 import com.example.EmployeeManagementSystem.Repository.RefreshTokenRepository;
11 import com.example.EmployeeManagementSystem.Service.RefreshTokenService;
12 import com.example.EmployeeManagementSystem.Service.TotpService;
13 import com.example.EmployeeManagementSystem.Util.JWTUtil;
14 import org.springframework.http.ResponseEntity;
15 import org.springframework.security.authentication.AuthenticationManager;
16 import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
17 import org.springframework.security.core.Authentication;
18 import org.springframework.security.core.userdetails.UserDetails;
19 import org.springframework.web.bind.annotation.*;
20
21 import java.time.Instant;
22 import java.util.Map;
23 import java.util.UUID;
24 import java.util.concurrent.ConcurrentHashMap;
25
26 @RestController
27 @RequestMapping("/Authenticate")
28 public class AuthController {
29
30     private final AuthenticationManager authenticationManager;
31     private final JWTUtil jwtUtil;
32     private final RefreshTokenService refreshTokenService;
33     private final RefreshTokenRepository refreshTokenRepository;
34     private final TotpService totpService;
35     private final EmployeeRepo employeeRepo;
36
37     // Temporary store for pre-auth tokens (username -> pre-auth token)
38     // In production, use Redis or DB-backed store with TTL
39     private static final ConcurrentHashMap<String, PreAuthEntry> PRE_AUTH_STORE = new ConcurrentHashMap<>();
40
41     private record PreAuthEntry(String username, String role, long expiresAt) {}
42
43     public AuthController(AuthenticationManager authenticationManager,
44                           JWTUtil jwtUtil,
45                           RefreshTokenService refreshTokenService,
46                           RefreshTokenRepository refreshTokenRepository,
47                           TotpService totpService,
48                           EmployeeRepo employeeRepo) {
49         this.authenticationManager = authenticationManager;
50         this.jwtUtil = jwtUtil;
51         this.refreshTokenService = refreshTokenService;
52         this.refreshTokenRepository = refreshTokenRepository;
53         this.totpService = totpService;
54         this.employeeRepo = employeeRepo;
55     }
56
57     /**
58      * Step 1 of login. If TOTP is enabled, returns a pre-auth token instead of a JWT.
59      * The client must then call /Authenticate/totp with that pre-auth token + 6-digit code.
60      */
61     @PostMapping
62     public ResponseEntity<?> authenticateToken(@RequestBody AuthRequest authRequest) {
63         try {
64             Authentication auth = authenticationManager.authenticate(
65                 new UsernamePasswordAuthenticationToken(
66                     authRequest.getUsername(),
67                     authRequest.getPassword()
68                 )
69             );
70
71             UserDetails user = (UserDetails) auth.getPrincipal();

```

```

72
73     String role = user.getAuthorities().stream()
74         .map(a -> a.getAuthority())
75         .filter(a -> a.startsWith("ROLE_"))
76         .findFirst()
77         .map(a -> a.substring("ROLE_".length()))
78         .orElseThrow(() -> new RuntimeException("Role not found"));
79
80     // Handle Vendor login – no TOTP, issue tokens directly
81     if (user instanceof Vendor vendor) {
82         return ResponseEntity.ok(issueTokens(vendor, role));
83     }
84
85     // Handle Employee login
86     Employee employee = employeeRepo.findByEmail(authRequest.getUsername())
87         .orElseThrow(() -> new RuntimeException("Employee not found"));
88
89     if (employee.isTotpEnabled()) {
90         String preAuthToken = UUID.randomUUID().toString();
91         PRE_AUTH_STORE.put(preAuthToken, new PreAuthEntry(
92             employee.getEmail(),
93             role,
94             System.currentTimeMillis() + 5 * 60 * 1000L
95         ));
96         return ResponseEntity.ok(Map.of(
97             "requiresTotp", true,
98             "preAuthToken", preAuthToken
99         ));
100     }
101
102     return ResponseEntity.ok(issueTokens(user, role));
103
104 } catch (Exception e) {
105     throw new RuntimeException(e);
106 }
107 }
108
109 /**
110  * Step 2 of login when TOTP is enabled.
111  * Submit the pre-auth token + 6-digit TOTP code to get the real JWT.
112  */
113 @PostMapping("/totp")
114 public ResponseEntity<?> completeTotpLogin(@RequestBody Map<String, String> body) {
115     String preAuthToken = body.get("preAuthToken");
116     String code = body.get("code");
117
118     if (preAuthToken == null || code == null) {
119         return ResponseEntity.badRequest().body(Map.of("error", "preAuthToken and code are required"));
120     }
121
122     PreAuthEntry entry = PRE_AUTH_STORE.get(preAuthToken);
123     if (entry == null || entry.expiresAt() < System.currentTimeMillis()) {
124         PRE_AUTH_STORE.remove(preAuthToken);
125         return ResponseEntity.status(401).body(Map.of("error", "Pre-auth token expired or invalid.
Please login again."));
126     }
127
128     Employee employee = employeeRepo.findByEmail(entry.username())
129         .orElseThrow(() -> new RuntimeException("Employee not found"));
130
131     if (!totpService.verifyCode(employee.getTotpSecret(), code)) {
132         return ResponseEntity.status(401).body(Map.of("error", "Invalid TOTP code"));
133     }
134
135     // TOTP verified – consume the pre-auth token and issue JWT
136     PRE_AUTH_STORE.remove(preAuthToken);
137     return ResponseEntity.ok(issueTokens(employee, entry.role()));
138 }
139
140 /**
141  * Refresh token endpoint – unchanged from original.
142  */
143 @PostMapping("/refresh")
144 public AuthResponse refresh(@RequestBody AccessTokenRequest request) {
145     RefreshToken storedToken = refreshTokenRepository.findByToken(request.getRefreshToken())
146         .orElseThrow(() -> new RuntimeException("Invalid refresh token"));

```

```

147
148     if (storedToken.getExpiryTime().isBefore(Instant.now())) {
149         refreshTokenRepository.delete(storedToken);
150         throw new RuntimeException("Refresh token expired");
151     }
152
153     String username;
154     String role;
155     if (storedToken.getEmployee() != null) {
156         username = storedToken.getEmployee().getUsername();
157         role = storedToken.getEmployee().getRole().name();
158     } else if (storedToken.getVendor() != null) {
159         username = storedToken.getVendor().getUsername();
160         role = storedToken.getVendor().getRole().name();
161     } else {
162         throw new RuntimeException("Refresh token has no associated user");
163     }
164
165     String newAccessToken = jwtUtil.generateToken(username, role);
166     return new AuthResponse(newAccessToken, storedToken.getToken());
167 }
168
169 // ---- helper ----
170
171 private AuthResponse issueTokens(UserDetails user, String role) {
172     String accessToken = jwtUtil.generateToken(user.getUsername(), role);
173     RefreshToken refreshToken;
174     if (user instanceof Employee employee) {
175         refreshToken = refreshTokenService.createForEmployee(employee);
176     } else if (user instanceof Vendor vendor) {
177         refreshToken = refreshTokenService.createForVendor(vendor);
178     } else {
179         throw new RuntimeException("Unknown user type");
180     }
181     refreshTokenRepository.save(refreshToken);
182     return new AuthResponse(accessToken, refreshToken.getToken());
183 }
184 }

```

```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.Entity.Delivery;
4 import com.example.EmployeeManagementSystem.Enum.DeliveryStatus;
5 import com.example.EmployeeManagementSystem.Service.DeliveryService;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.web.bind.annotation.*;
8
9 import java.util.List;
10
11 @RestController
12 @RequestMapping("/delivery")
13 public class DeliveryController {
14     private final DeliveryService deliveryService;
15
16     public DeliveryController(DeliveryService deliveryService) {
17         this.deliveryService = deliveryService;
18     }
19
20     @PutMapping("/{id}/status")
21     public ResponseEntity<Delivery> updateDeliveryStatus(
22         @PathVariable Long id,
23         @RequestParam DeliveryStatus status) {
24         return ResponseEntity.ok(deliveryService.updateDeliveryStatus(id, status));
25     }
26
27     @GetMapping("/subscription/{subscriptionId}")
28     public ResponseEntity<List<Delivery>> getDeliveriesBySubscription(
29         @PathVariable Long subscriptionId) {
30         return ResponseEntity.ok(deliveryService.getDeliveriesBySubscription(subscriptionId));
31     }
32 }
33
```



```

1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.AssignDeviceRequest;
4 import com.example.EmployeeManagementSystem.DTO.DeviceAssignmentResponseDTO;
5 import com.example.EmployeeManagementSystem.DTO.DeviceDTO;
6 import com.example.EmployeeManagementSystem.DTO.DeviceResponseDTO;
7 import com.example.EmployeeManagementSystem.Entity.Device;
8 import com.example.EmployeeManagementSystem.Entity.DeviceAssignment;
9 import com.example.EmployeeManagementSystem.Service.DeviceService;
10 import org.springframework.http.HttpStatus;
11 import org.springframework.http.ResponseEntity;
12 import org.springframework.security.access.prepost.PreAuthorize;
13 import org.springframework.security.core.Authentication;
14 import org.springframework.web.bind.annotation.*;
15
16 import java.util.List;
17
18 @RestController
19 @RequestMapping("/devices")
20 public class DeviceController {
21     private final DeviceService deviceService;
22
23     public DeviceController(DeviceService deviceService) {
24         this.deviceService = deviceService;
25     }
26
27     @PostMapping
28     @PreAuthorize("hasRole('TECH_VENDOR')")
29     public ResponseEntity<Device> addDevice(@RequestBody DeviceDTO deviceDTO,
30                                             Authentication authentication) {
31         return ResponseEntity.status(HttpStatus.CREATED)
32             .body(deviceService.addDevice(deviceDTO, authentication));
33     }
34
35     @GetMapping("/vendor/me")
36     @PreAuthorize("hasRole('TECH_VENDOR')")
37     public ResponseEntity<List<DeviceResponseDTO>> getMyVendorDevices(Authentication authentication) {
38         return ResponseEntity.ok(deviceService.getDevicesForLoggedInVendor(authentication));
39     }
40
41     @GetMapping("/my")
42     @PreAuthorize("hasAnyRole('EMPLOYEE', 'MANAGER', 'ADMIN')")
43     public ResponseEntity<List<DeviceResponseDTO>> getMyAssignedDevices(Authentication authentication) {
44         return ResponseEntity.ok(deviceService.getDevicesForLoggedInEmployee(authentication));
45     }
46
47     @GetMapping("/unassigned")
48     @PreAuthorize("hasRole('ADMIN')")
49     public ResponseEntity<List<DeviceResponseDTO>> getUnassignedDevices() {
50         return ResponseEntity.ok(deviceService.getUnassignedDevices());
51     }
52
53     @PostMapping("/{deviceId}/assign")
54     @PreAuthorize("hasRole('ADMIN')")
55     public ResponseEntity<String> assignDevice(@PathVariable Long deviceId,
56                                              @RequestBody AssignDeviceRequest request) {
57         DeviceAssignment assignment = deviceService.assignDevice(deviceId, request);
58         return ResponseEntity.status(HttpStatus.CREATED)
59             .body("Device " + assignment.getDevice().getId()
60                 + " assigned to employee " + assignment.getAssignedTo().getEmployeeId());
61     }
62
63     @GetMapping("/assignments")
64     @PreAuthorize("hasRole('TECH_VENDOR')")
65     public List<DeviceAssignmentResponseDTO> getDeviceAssignmentOfVendor(Authentication authentication){
66         return deviceService.getDeviceAssignmentOfVendor(authentication);
67     }
68 }
69
70 }
71

```

```

1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.EmployeeAttendanceResponseDTO;
4 import com.example.EmployeeManagementSystem.DTO.EmployeeDTO;
5 import com.example.EmployeeManagementSystem.Entity.Employee;
6 import com.example.EmployeeManagementSystem.Service.EmployeeService;
7 import org.springframework.http.HttpStatus;
8 import org.springframework.http.ResponseEntity;
9 import org.springframework.security.access.prepost.PreAuthorize;
10 import org.springframework.security.core.Authentication;
11 import org.springframework.web.bind.annotation.*;
12 @RestController
13 @RequestMapping("/employee")
14 public class EmployeeController {
15     private final EmployeeService employeeService;
16
17     public EmployeeController(EmployeeService employeeService) {
18         this.employeeService = employeeService;
19     }
20
21     @GetMapping
22     @PreAuthorize("hasAnyRole('MANAGER', 'ADMIN', 'EMPLOYEE', 'VENDOR')")
23     public ResponseEntity<?> getAllEmployees(){
24         return ResponseEntity.ok(employeeService.getAllEmployees());
25     }
26
27     @GetMapping("/me")
28     @PreAuthorize("hasAnyRole('ADMIN', 'MANAGER', 'EMPLOYEE')")
29     public ResponseEntity<Employee> myAccount(Authentication authentication){
30         return ResponseEntity.ok(employeeService.getAccount(authentication));
31     }
32
33     @GetMapping("/attendance")
34     @PreAuthorize("hasAnyRole('ADMIN', 'MANAGER', 'EMPLOYEE', 'VENDOR')")
35     public ResponseEntity<EmployeeAttendanceResponseDTO> getAttendanceOverview() {
36         return ResponseEntity.ok(employeeService.getEmployeeAttendanceOverview());
37     }
38
39     @PutMapping("/{id}")
40     @PreAuthorize("hasRole('ADMIN')")
41     public ResponseEntity<Employee> updateEmployee(@PathVariable long id, @RequestBody EmployeeDTO
employeeDTO){
42         return ResponseEntity.ok(employeeService.updateEmployee(id, employeeDTO));
43     }
44
45     @DeleteMapping("/{id}")
46     @PreAuthorize("hasAnyRole('ADMIN', 'MANAGER')")
47     public ResponseEntity<?> deleteEmployee(@PathVariable long id){
48         employeeService.deleteEmployee(id);
49         return ResponseEntity.ok("Employee with id: "+id+" is deleted");
50     }
51
52     //should be done by manager
53     @PutMapping("/inactive")
54     @PreAuthorize("hasAnyRole('MANAGER', 'ADMIN')")
55     public ResponseEntity<String> inactivateEmployee(@RequestParam long employeeId) {
56         return employeeService.inactivateUser(employeeId);
57     }
58 }
59

```

16 Current:

src\main\java\com\example\EmployeeManagementSystem\Controller\GlobalExceptionHandler.java

```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.Exception.*;
4 import org.springframework.dao.DataIntegrityViolationException;
5 import org.springframework.http.HttpStatus;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.web.bind.annotation.ExceptionHandler;
8 import org.springframework.web.bind.annotation.RestControllerAdvice;
9 import org.springframework.web.context.request.WebRequest;
10 import org.springframework.security.access.AccessDeniedException;
11
12 import java.time.Instant;
13 import java.time.LocalDateTime;
14 import java.util.LinkedHashMap;
15 import java.util.Map;
16
17 @RestControllerAdvice
18 public class GlobalExceptionHandler {
19
20     private Map<String, Object> buildError(HttpStatus status, String message, WebRequest request) {
21         Map<String, Object> body = new LinkedHashMap<>();
22         body.put("timestamp", Instant.now());
23         body.put("status", status.value());
24         body.put("error", status.getReasonPhrase());
25         body.put("message", message);
26         body.put("path", request.getDescription(false).replace("uri=", ""));
27         return body;
28     }
29
30     @ExceptionHandler(EmployeeNotFoundException.class)
31     public ResponseEntity<Map<String, Object>> handleEmployeeNotFound(
32         EmployeeNotFoundException ex, WebRequest request) {
33         return new ResponseEntity<>(buildError(HttpStatus.NOT_FOUND, ex.getMessage(), request),
34             HttpStatus.NOT_FOUND);
35     }
36
37     @ExceptionHandler(InvalidStartDateException.class)
38     public ResponseEntity<Map<String, Object>> handleInvalidStartDate(
39         InvalidStartDateException ex, WebRequest request) {
40         return new ResponseEntity<>(buildError(HttpStatus.BAD_REQUEST, ex.getMessage(), request),
41             HttpStatus.BAD_REQUEST);
42     }
43
44     @ExceptionHandler(InvalidEndDateException.class)
45     public ResponseEntity<Map<String, Object>> handleInvalidEndDate(
46         InvalidEndDateException ex, WebRequest request) {
47         return new ResponseEntity<>(buildError(HttpStatus.BAD_REQUEST, ex.getMessage(), request),
48             HttpStatus.BAD_REQUEST);
49     }
50
51     @ExceptionHandler(OverlappingLeaveException.class)
52     public ResponseEntity<Map<String, Object>> handleOverlappingLeave(
53         OverlappingLeaveException ex, WebRequest request) {
54         return new ResponseEntity<>(buildError(HttpStatus.BAD_REQUEST, ex.getMessage(), request),
55             HttpStatus.BAD_REQUEST);
56     }
57
58     @ExceptionHandler(DuplicateRequestException.class)
59     public ResponseEntity<Map<String, Object>> handleDuplicateRequest(
60         DuplicateRequestException ex, WebRequest request) {
61         return new ResponseEntity<>(buildError(HttpStatus.BAD_REQUEST, ex.getMessage(), request),
62             HttpStatus.BAD_REQUEST);
63     }
64
65     @ExceptionHandler(LeaveRequestNotFoundException.class)
66     public ResponseEntity<Map<String, Object>> handleLeaveNotFound(
67         LeaveRequestNotFoundException ex, WebRequest request) {
68         return new ResponseEntity<>(buildError(HttpStatus.NOT_FOUND, ex.getMessage(), request),
69             HttpStatus.NOT_FOUND);
70     }
71 }
```

```

72     @ExceptionHandler(InvalidManagerException.class)
73     public ResponseEntity<Map<String, Object>> handleInvalidManager(
74         InvalidManagerException ex, WebRequest request) {
75         return new ResponseEntity<>(buildError(HttpStatus.FORBIDDEN, ex.getMessage(), request),
76             HttpStatus.FORBIDDEN);
77     }
78
79     @ExceptionHandler(SubscriptionAlreadyExists.class)
80     public ResponseEntity<Map<String, Object>> handleSubscriptionExists(
81         SubscriptionAlreadyExists ex, WebRequest request) {
82         return new ResponseEntity<>(buildError(HttpStatus.CONFLICT, ex.getMessage(), request),
83             HttpStatus.CONFLICT);
84     }
85
86
87     @ExceptionHandler(VendorNotFoundException.class)
88     public ResponseEntity<Map<String, Object>> handleVendorNotFound(
89         VendorNotFoundException ex, WebRequest request) {
90         return new ResponseEntity<>(buildError(HttpStatus.NOT_FOUND, ex.getMessage(), request),
91             HttpStatus.NOT_FOUND);
92     }
93
94     @ExceptionHandler(RestaurantNotFoundException.class)
95     public ResponseEntity<Map<String, Object>> handleRestaurantNotFound(
96         RestaurantNotFoundException ex, WebRequest request) {
97         return new ResponseEntity<>(buildError(HttpStatus.NOT_FOUND, ex.getMessage(), request),
98             HttpStatus.NOT_FOUND);
99     }
100
101     @ExceptionHandler(UnauthorizedAccessException.class)
102     public ResponseEntity<Map<String, Object>> handleUnauthorized(
103         UnauthorizedAccessException ex, WebRequest request) {
104         return new ResponseEntity<>(buildError(HttpStatus.FORBIDDEN, ex.getMessage(), request),
105             HttpStatus.FORBIDDEN);
106     }
107
108     @ExceptionHandler(IllegalArgumentException.class)
109     public ResponseEntity<Map<String, Object>> handleIllegalArgument(
110         IllegalArgumentException ex, WebRequest request) {
111         return new ResponseEntity<>(buildError(HttpStatus.BAD_REQUEST, ex.getMessage(), request),
112             HttpStatus.BAD_REQUEST);
113     }
114
115     @ExceptionHandler(DataIntegrityViolationException.class)
116     public ResponseEntity<Map<String, Object>> handleDataIntegrity(
117         DataIntegrityViolationException ex, WebRequest request) {
118         return new ResponseEntity<>(
119             buildError(HttpStatus.CONFLICT, "Cannot delete: related records exist", request),
120             HttpStatus.CONFLICT);
121     }
122
123     @ExceptionHandler(AccessDeniedException.class)
124     public ResponseEntity<Map<String, Object>> handleAccessDenied(AccessDeniedException ex) {
125         return ResponseEntity.status(HttpStatus.FORBIDDEN).body(Map.of(
126             "error", "Access denied",
127             "message", ex.getMessage(),
128             "timestamp", LocalDateTime.now().toString()
129         ));
130     }
131
132     @ExceptionHandler(IllegalStateException.class)
133     public ResponseEntity<Map<String, Object>> handleIllegalState(IllegalStateException ex) {
134         return ResponseEntity.status(HttpStatus.BAD_REQUEST).body(Map.of(
135             "error", ex.getMessage(),
136             "message", ex.getMessage(),
137             "timestamp", LocalDateTime.now().toString()
138         ));
139     }
140
141 }

```

```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.HolidayDTO;
4 import com.example.EmployeeManagementSystem.Service.HolidayService;
5 import org.springframework.http.ResponseEntity;
6 import org.springframework.security.access.prepost.PreAuthorize;
7 import org.springframework.web.bind.annotation.*;
8
9 import java.time.LocalDate;
10
11 @RestController
12 @RequestMapping("/holidays")
13 public class HolidayController {
14     private final HolidayService holidayService;
15
16     public HolidayController(HolidayService holidayService) {
17         this.holidayService = holidayService;
18     }
19
20     @PostMapping
21     @PreAuthorize("hasRole('ADMIN')")
22     public ResponseEntity<?> addHoliday(@RequestBody HolidayDTO dto) {
23         return ResponseEntity.ok(holidayService.addHoliday(dto));
24     }
25
26     @GetMapping
27     public ResponseEntity<?> getHolidays(
28         @RequestParam(defaultValue = "0") int year
29     ) {
30         int targetYear = (year == 0) ? LocalDate.now().getYear() : year;
31         return ResponseEntity.ok(holidayService.getHolidaysByYear(targetYear));
32     }
33
34     @DeleteMapping("/{id}")
35     @PreAuthorize("hasRole('ADMIN')")
36     public ResponseEntity<?> deleteHoliday(@PathVariable Long id) {
37         holidayService.deleteHoliday(id);
38         return ResponseEntity.ok("Holiday deleted");
39     }
40 }
```

18 Current:

src\main\java\com\example\EmployeeManagementSystem\Controller\LeaveDashboardController.java

```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.LeaveDashboardResponse;
4 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
5 import com.example.EmployeeManagementSystem.Service.LeaveDashboardService;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.security.core.Authentication;
8 import org.springframework.security.core.userdetails.UserDetails;
9 import org.springframework.security.oauth2.core.user.OAuth2User;
10 import org.springframework.web.server.ResponseStatusException;
11 import org.springframework.web.bind.annotation.*;
12
13 import java.util.List;
14
15 import static org.springframework.http.HttpStatus.UNAUTHORIZED;
16
17 @RestController
18 @RequestMapping("/api")
19 public class LeaveDashboardController {
20
21     private final LeaveDashboardService dashboardService;
22     private final EmployeeRepo employeeRepository;
23
24     public LeaveDashboardController(LeaveDashboardService dashboardService,
25                                     EmployeeRepo employeeRepository) {
26         this.dashboardService = dashboardService;
27         this.employeeRepository = employeeRepository;
28     }
29
30     // GET own dashboard (any authenticated employee)
31     // GET /api/employees/me/leave-dashboard
32     @GetMapping("/employees/me/leave-dashboard")
33     public ResponseEntity<LeaveDashboardResponse> getMyDashboard(
34         Authentication authentication) {
35
36         Long callerId = resolveEmployeeId(authentication);
37         return ResponseEntity.ok(
38             dashboardService.getDashboard(callerId, callerId, resolveRole(authentication)));
39     }
40
41     // GET any employee's dashboard (manager/admin only)
42     // GET /api/employees/5/leave-dashboard
43     @GetMapping("/employees/{id}/leave-dashboard")
44     public ResponseEntity<LeaveDashboardResponse> getEmployeeDashboard(
45         @PathVariable Long id,
46         Authentication authentication) {
47
48         Long callerId = resolveEmployeeId(authentication);
49         String callerRole = resolveRole(authentication);
50         return ResponseEntity.ok(
51             dashboardService.getDashboard(id, callerId, callerRole));
52     }
53
54     // GET team dashboard (manager/admin only)
55     // GET /api/employees/me/team-dashboard
56     @GetMapping("/employees/me/team-dashboard")
57     public ResponseEntity<List<LeaveDashboardResponse>> getTeamDashboard(
58         Authentication authentication) {
59
60         Long callerId = resolveEmployeeId(authentication);
61         String callerRole = resolveRole(authentication);
62         return ResponseEntity.ok(
63             dashboardService.getTeamDashboard(callerId, callerRole));
64     }
65
66     // PATCH mark warning read
67     // PATCH /api/warnings/42/read
68     @PatchMapping("/warnings/{warningId}/read")
69     public ResponseEntity<Void> markWarningRead(
70         @PathVariable Long warningId,
71         Authentication authentication) {
```

```

72
73     Long callerId = resolveEmployeeId(authentication);
74     String callerRole = resolveRole(authentication);
75     dashboardService.markWarningRead(warningId, callerId, callerRole);
76     return ResponseEntity.noContent().build();
77 }
78
79 // Supports both JWT/session UserDetails and Google OAuth2 principals.
80 private Long resolveEmployeeId(Authentication authentication) {
81     String email = resolveEmail(authentication);
82     return employeeRepository
83         .findByEmail(email)
84         .orElseThrow(() -> new IllegalStateException("Authenticated user not found"))
85         .getEmployeeId();
86 }
87
88 private String resolveRole(Authentication authentication) {
89     if (authentication == null || !authentication.isAuthenticated()) {
90         throw new ResponseStatusException(UNAUTHORIZED, "Authentication required");
91     }
92     return authentication.getAuthorities().stream()
93         .map(a -> a.getAuthority())
94         .filter(authority -> authority.startsWith("ROLE_"))
95         .findFirst()
96         .map(authority -> authority.replace("ROLE_", ""))
97         .orElse("EMPLOYEE");
98 }
99
100 private String resolveEmail(Authentication authentication) {
101     if (authentication == null || !authentication.isAuthenticated()) {
102         throw new ResponseStatusException(UNAUTHORIZED, "Authentication required");
103     }
104
105     Object principal = authentication.getPrincipal();
106     if (principal instanceof UserDetails userDetails) {
107         return userDetails.getUsername();
108     }
109     if (principal instanceof OAuth2User oAuth2User) {
110         String email = oAuth2User.getAttribute("email");
111         if (email != null && !email.isBlank()) {
112             return email;
113         }
114     }
115
116     throw new ResponseStatusException(UNAUTHORIZED, "Authenticated email not found");
117 }
118 }
119

```

19 Current:

src\main\java\com\example\EmployeeManagementSystem\Controller\LeaveRequestController.java

```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.ActionDTO;
4 import com.example.EmployeeManagementSystem.DTO.LeaveRequestDTO;
5 import com.example.EmployeeManagementSystem.DTO.LeaveResponseDTO;
6 import com.example.EmployeeManagementSystem.DTO.MaternityLeaveRequestDTO;
7 import com.example.EmployeeManagementSystem.Service.LeaveRequestService;
8 import com.example.EmployeeManagementSystem.Service.MaternityLeaveService;
9 import org.springframework.http.ResponseEntity;
10 import org.springframework.security.access.prepost.PreAuthorize;
11 import org.springframework.security.core.Authentication;
12 import org.springframework.web.bind.annotation.*;
13
14 import java.util.List;
15
16 @RestController
17 @RequestMapping("leave_requests")
18 public class LeaveRequestController {
19     private final LeaveRequestService leaveRequestService;
20     private final MaternityLeaveService maternityLeaveService;
21
22     public LeaveRequestController(LeaveRequestService leaveRequestService, MaternityLeaveService
23 maternityLeaveService) {
24         this.leaveRequestService = leaveRequestService;
25         this.maternityLeaveService = maternityLeaveService;
26     }
27
28     @GetMapping
29     @PreAuthorize("hasAnyRole('ADMIN','MANAGER')")
30     public ResponseEntity<List<LeaveResponseDTO>> getAllLeaveRequest() {
31         return ResponseEntity.ok(leaveRequestService.getAllTheLeaveRequest());
32     }
33
34     @PostMapping
35     @PreAuthorize("hasAnyRole('ADMIN','MANAGER','EMPLOYEE')")
36     public ResponseEntity<?> addLeaveRequest(Authentication authentication,@RequestBody LeaveRequestDTO
37 dto){
38         return leaveRequestService.createRequest(authentication,dto);
39     }
40
41     @GetMapping("/pending")
42     @PreAuthorize("hasAnyRole('ADMIN','MANAGER')")
43     public ResponseEntity<List<LeaveResponseDTO>> getAllTheLeaveRequests() {
44         return ResponseEntity.ok(leaveRequestService.getAllThePendingLeaveRequests());
45     }
46
47     @PutMapping("/approval")
48     @PreAuthorize("hasAnyRole('ADMIN','MANAGER')")
49     public ResponseEntity<?> updateLeaveRequestStatus(@RequestBody ActionDTO actionDTO,Authentication
50 authentication){
51         return leaveRequestService.updateLeaveRequestStatus(actionDTO,authentication);
52     }
53
54     @PutMapping("/cancel/{leaveId}")
55     @PreAuthorize("hasAuthority('LEAVE_CANCEL')")
56     public ResponseEntity<?> cancelLeaveRequest(Authentication authentication,@PathVariable long leaveId){
57         return leaveRequestService.cancelLeaveRequest(authentication, leaveId);
58     }
59
60     @GetMapping("/employee")
61     @PreAuthorize("hasAnyRole('ADMIN','MANAGER','EMPLOYEE')")
62     public ResponseEntity<?> getEmployeeLeaves(Authentication authentication) {
63         return leaveRequestService.getLeaveRequestsByEmployee(authentication);
64     }
65
66     @PostMapping("/maternity/apply")
67     @PreAuthorize("hasAnyRole('ADMIN','MANAGER','EMPLOYEE')")
68     public ResponseEntity<?> applyMaternityLeave(
69         Authentication authentication,
70         @RequestBody MaternityLeaveRequestDTO dto) {
71         return maternityLeaveService.applyMaternityLeave(authentication, dto);
72     }
73 }
```



```
69 }
```

```
70
```

```
71 }
```

```
72
```

```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import org.springframework.http.ResponseEntity;
4 import org.springframework.web.bind.annotation.GetMapping;
5 import org.springframework.web.bind.annotation.RestController;
6
7 import java.util.Map;
8
9 @RestController
10 public class LoginController {
11
12     @GetMapping("/login")
13     public ResponseEntity<?> login() {
14         return ResponseEntity.ok().body(Map.of("message", "Please use POST to /api/auth/login with
credentials"));
15     }
16 }
17
```

```

1  package com.example.EmployeeManagementSystem.Controller;
2
3  import com.example.EmployeeManagementSystem.DTO.*;
4  import com.example.EmployeeManagementSystem.Entity.Employee;
5  import com.example.EmployeeManagementSystem.Service.ManagerService;
6  import org.springframework.http.ResponseEntity;
7  import org.springframework.security.access.prepost.PreAuthorize;
8  import org.springframework.security.core.Authentication;
9  import org.springframework.web.bind.annotation.*;
10
11 import java.util.List;
12
13 @RestController
14 @RequestMapping("/manager")
15 public class ManagerController {
16     private final ManagerService manangerService;
17
18     public ManagerController(ManagerService manangerService) {
19         this.manangerService = manangerService;
20     }
21
22     @PostMapping("/createAdmin")
23     public Employee createAdmin(@RequestBody EmployeeDetails employeeDetails){
24         return manangerService.createAdmin(employeeDetails);
25     }
26
27     @PostMapping("/employees")
28     @PreAuthorize("hasAnyRole('MANAGER')")
29     public Employee createEmployee(@RequestBody EmployeeDetails employeeDetails, Authentication
authentication){
30         return manangerService.createEmp(employeeDetails,authentication);
31     }
32
33     @GetMapping("/employees")
34     @PreAuthorize("hasRole('MANAGER')")
35     public List<Employee> getAllEmployeeByManager(Authentication authentication){
36         return manangerService.getAllEmployeeByManager(authentication);
37     }
38
39     @GetMapping("/requests")
40     @PreAuthorize("hasRole('MANAGER')")
41     public List<ManagerLeaveResponseDTO> getAllLeaveRequestByManager(Authentication authentication){
42         return manangerService.getAllLeaveRequestByManager(authentication);
43     }
44
45     @GetMapping("/Users")
46     @PreAuthorize("hasAnyRole('MANAGER','ADMIN')")
47     public List<EmployeeDTO> getUsers(){
48         return manangerService.getUsers();
49     }
50
51     @PutMapping("/promote")
52     @PreAuthorize("hasRole('MANAGER')")
53     public ResponseEntity<?> promoteUser(Authentication authentication, @RequestBody PromoteRequest
promoteRequest){
54         return manangerService.promoteUser(authentication,promoteRequest);
55     }
56
57     @GetMapping("/pending")
58     @PreAuthorize("hasAnyRole('MANAGER')")
59     public ResponseEntity<List<LeaveResponseDTO>> getAllTheLeaveRequests(Authentication authentication) {
60         return ResponseEntity.ok(manangerService.getAllThePendingLeaveRequests(authentication));
61     }
62
63     @GetMapping("/employees/search")
64     @PreAuthorize("hasAnyRole('MANAGER')")
65     public ResponseEntity<ManagerEmployeeDetailsDTO> getEmployeeDetails( @RequestParam String name){
66         return ResponseEntity.ok(manangerService.getEmployeeDetailsByName(name));
67     }
68
69

```

70

71 }

72

```

1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.Notification;
5 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
6 import com.example.EmployeeManagementSystem.Service.NotificationService;
7 import org.springframework.http.ResponseEntity;
8 import org.springframework.security.core.Authentication;
9 import org.springframework.security.oauth2.core.user.OAuth2User;
10 import org.springframework.security.oauth2.client.authentication.OAuth2AuthenticationToken;
11 import org.springframework.web.bind.annotation.*;
12 import java.util.List;
13 import java.util.Map;
14
15 @RestController
16 @RequestMapping("/notifications")
17 public class NotificationController {
18
19     private final NotificationService notificationService;
20     private final EmployeeRepo employeeRepo;
21
22     public NotificationController(NotificationService notificationService,
23                                 EmployeeRepo employeeRepo) {
24         this.notificationService = notificationService;
25         this.employeeRepo = employeeRepo;
26     }
27
28     /** GET /notifications – all notifications for logged-in user */
29     @GetMapping
30     public ResponseEntity<List<Notification>> getAll(Authentication authentication) {
31         Employee employee = getEmployee(authentication);
32         return ResponseEntity.ok(notificationService.getForEmployee(employee));
33     }
34
35     /** GET /notifications/unread-count – for the badge number */
36     @GetMapping("/unread-count")
37     public ResponseEntity<?> unreadCount(Authentication authentication) {
38         Employee employee = getEmployee(authentication);
39         return ResponseEntity.ok(Map.of("count",
40                                         notificationService.getUnreadCount(employee)));
41     }
42
43     /** POST /notifications/mark-read – mark all as read when bell is opened */
44     @PostMapping("/mark-read")
45     public ResponseEntity<?> markRead(Authentication authentication) {
46         Employee employee = getEmployee(authentication);
47         notificationService.markAllRead(employee);
48         return ResponseEntity.ok(Map.of("message", "All notifications marked as read"));
49     }
50
51     private Employee getEmployee(Authentication authentication) {
52         final String email = (authentication instanceof OAuth2AuthenticationToken oauth)
53             ? oauth.getPrincipal().<String>getAttribute("email")
54             : authentication.getName(); // JWT path – getName() returns email
55
56         return employeeRepo.findByEmail(email)
57             .orElseThrow(() -> new RuntimeException("Employee not found: " + email));
58     }
59 }
60

```

```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.AuthResponse;
4 import com.example.EmployeeManagementSystem.Entity.Employee;
5 import com.example.EmployeeManagementSystem.Entity.RefreshToken;
6 import com.example.EmployeeManagementSystem.Entity.Vendor;
7 import com.example.EmployeeManagementSystem.Enum.Role;
8 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
9 import com.example.EmployeeManagementSystem.Repository.RefreshTokenRepository;
10 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
11 import com.example.EmployeeManagementSystem.Service.CombinedUserDetailsService;
12 import com.example.EmployeeManagementSystem.Service.RefreshTokenService;
13 import com.example.EmployeeManagementSystem.Util.JWTUtil;
14 import jakarta.servlet.http.HttpServletRequest;
15 import jakarta.servlet.http.HttpServletResponse;
16 import jakarta.servlet.http.HttpSession;
17 import org.springframework.beans.factory.annotation.Autowired;
18 import org.springframework.beans.factory.annotation.Value;
19 import org.springframework.http.*;
20 import org.springframework.security.core.userdetails.UserDetails;
21 import org.springframework.security.core.userdetails.UsernameNotFoundException;
22 import org.springframework.security.crypto.password.PasswordEncoder;
23 import org.springframework.util.LinkedMultiValueMap;
24 import org.springframework.util.MultiValueMap;
25 import org.springframework.web.bind.annotation.GetMapping;
26 import org.springframework.web.bind.annotation.RequestMapping;
27 import org.springframework.web.bind.annotation.RequestParam;
28 import org.springframework.web.bind.annotation.RestController;
29 import org.springframework.web.client.RestTemplate;
30
31 import java.io.IOException;
32 import java.util.Collections;
33 import java.util.Map;
34 import java.util.Optional;
35 import java.util.UUID;
36
37 @RestController
38 @RequestMapping("/auth/google")
39 public class OAuthController {
40
41     @GetMapping("/init")
42     public void initOAuth(@RequestParam(defaultValue = "EMPLOYEE") String role,
43                          @RequestParam(defaultValue = "Asia/Kolkata") String timezone,
44                          HttpServletRequest request,
45                          HttpServletResponse response) throws IOException {
46
47         // Now hand off to Spring's built-in OAuth2 flow
48         System.out.println("Now hand off to Spring's built-in OAuth2 flow");
49         response.sendRedirect("/oauth2/authorization/google");
50     }
51 }
52
```

```

1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.RestaurantDTO;
4 import com.example.EmployeeManagementSystem.DTO.RestaurantRequest;
5 import com.example.EmployeeManagementSystem.Enum.MealSlot;
6 import com.example.EmployeeManagementSystem.Service.RestaurantService;
7 import org.springframework.http.HttpStatus;
8 import org.springframework.http.ResponseEntity;
9 import org.springframework.security.access.prepost.PreAuthorize;
10 import org.springframework.security.core.Authentication;
11 import org.springframework.web.bind.annotation.*;
12
13 import java.util.List;
14 import java.util.stream.Collectors;
15
16 /**
17  * Restaurant management endpoints.
18  *
19  * VENDOR-ONLY (write operations)
20  * POST /restaurants - Create a restaurant (vendor supplies vendorId in body)
21  * PUT /restaurants/{id} - Update restaurant details (vendor ownership verified)
22  * PUT /restaurants/{id}/deactivate - Soft-delete a restaurant (vendor ownership verified)
23  * GET /restaurants/vendor/{vendorId} - List MY restaurants (vendor views their own)
24  *
25  * EMPLOYEE / MANAGER (read-only browse)
26  * GET /restaurants - Browse all active restaurants
27  * GET /restaurants/{id} - Get a specific restaurant
28  * GET /restaurants/by-slot?slot=LUNCH - Find restaurants that serve a given meal slot
29  *
30  * Employees use the browse endpoints to pick a restaurant when creating a subscription.
31  * Employees and managers cannot create, update, or deactivate restaurants.
32  */
33 @RestController
34 @RequestMapping("/restaurants")
35 public class RestaurantController {
36
37     private final RestaurantService restaurantService;
38
39     public RestaurantController(RestaurantService restaurantService) {
40         this.restaurantService = restaurantService;
41     }
42
43     // VENDOR-ONLY write operations
44
45     /**
46      * Create a new restaurant.
47      * Request body must include vendorId – the service validates the vendor exists.
48      */
49     @PostMapping
50     @PreAuthorize("hasRole('FOOD_VENDOR')")
51     public ResponseEntity<RestaurantDTO> createRestaurant(@RequestBody RestaurantRequest
52 request, Authentication authentication) {
53         return ResponseEntity.status(HttpStatus.CREATED)
54             .body(restaurantService.createRestaurant(request, authentication));
55     }
56
57     /**
58      * Update a restaurant's name, address, description, or supported meal slots.
59      * vendorId in the request body is used to verify ownership before any change is made.
60      */
61     @PutMapping("/{id}")
62     @PreAuthorize("hasRole('FOOD_VENDOR')")
63     public ResponseEntity<RestaurantDTO> updateRestaurant(@PathVariable Long id,
64 @RequestBody RestaurantRequest request,
65 Authentication authentication) {
66         return ResponseEntity.ok(restaurantService.updateRestaurant(id, request, authentication));
67     }
68
69     /**
70      * Soft-deactivate a restaurant. Active subscriptions remain in the DB but
71      * new deliveries will stop since the restaurant is inactive.

```

```

71     * Pass vendorId as a query param so ownership can be verified.
72     */
73     @PutMapping("/{id}/deactivate")
74     @PreAuthorize("hasRole('FOOD_VENDOR')")
75     public ResponseEntity<String> deactivateRestaurant(@PathVariable Long id,
76                                                       Authentication authentication) {
77         restaurantService.deactivateRestaurant(id, authentication);
78         return ResponseEntity.ok("Restaurant " + id + " has been deactivated.");
79     }
80
81     /**
82     * List all active restaurants owned by a specific vendor.
83     * Only the vendor themselves should call this to see their portfolio.
84     */
85     @GetMapping("/vendor")
86     @PreAuthorize("hasRole('FOOD_VENDOR')")
87     public ResponseEntity<List<RestaurantDTO>> getRestaurantsByVendor(Authentication authentication) {
88         return ResponseEntity.ok(restaurantService.getRestaurantsByVendor(authentication));
89     }
90
91     // EMPLOYEE / MANAGER read-only browse
92
93     /**
94     * Browse all active restaurants.
95     * Employees use this to discover available restaurants before subscribing.
96     */
97     @GetMapping
98     @PreAuthorize("hasAnyRole('ADMIN') or hasAuthority('RESTAURANTS_READ')")
99     public ResponseEntity<List<RestaurantDTO>> getAllActiveRestaurants() {
100         return ResponseEntity.ok(restaurantService.getAllActiveRestaurants());
101     }
102
103     /**
104     * Get a specific restaurant by id.
105     */
106     @GetMapping("/{id}")
107     @PreAuthorize("hasAnyRole('ADMIN') or hasAuthority('RESTAURANTS_READ')")
108     public ResponseEntity<RestaurantDTO> getRestaurant(@PathVariable Long id) {
109         return ResponseEntity.ok(restaurantService.getRestaurant(id));
110     }
111
112     /**
113     * Find all active restaurants that serve a specific meal slot.
114     * Example: GET /restaurants/by-slot?slot=LUNCH
115     * Employees call this when building their subscription to see valid options for each slot.
116     */
117     @GetMapping("/by-slot")
118     @PreAuthorize("hasAnyRole('ADMIN', 'VENDOR', 'EMPLOYEE', 'MANAGER')")
119     public ResponseEntity<List<RestaurantDTO>> getRestaurantsByMealSlot(@RequestParam MealSlot slot) {
120         return ResponseEntity.ok(restaurantService.getRestaurantsByMealSlot(slot));
121     }
122 }
123
124

```



```

1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.Scheduler.LeaveStatusScheduler;
4 import com.example.EmployeeManagementSystem.Service.LeaveAccrualService;
5 import com.example.EmployeeManagementSystem.Service.SickLeaveResetService;
6 import org.slf4j.Logger;
7 import org.slf4j.LoggerFactory;
8 import org.springframework.http.ResponseEntity;
9 import org.springframework.security.access.prepost.PreAuthorize;
10 import org.springframework.web.bind.annotation.*;
11
12 import java.time.LocalDate;
13 import java.time.YearMonth;
14 import java.util.Map;
15
16 /**
17  * Controller for Scheduler Operations
18  */
19 @RestController
20 @RequestMapping("/scheduler")
21 public class SchedulerController {
22
23     private static final Logger log = LoggerFactory.getLogger(SchedulerController.class);
24
25     private final LeaveStatusScheduler leaveStatusScheduler;
26     private final SickLeaveResetService sickLeaveResetService; // FIX: was missing
27     private final LeaveAccrualService leaveAccrualService; // FIX: was missing
28
29     public SchedulerController(LeaveStatusScheduler leaveStatusScheduler,
30                               SickLeaveResetService sickLeaveResetService,
31                               LeaveAccrualService leaveAccrualService) {
32         this.leaveStatusScheduler = leaveStatusScheduler;
33         this.sickLeaveResetService = sickLeaveResetService;
34         this.leaveAccrualService = leaveAccrualService;
35     }
36
37     /**
38      * Manual trigger for leave status scheduler
39      */
40     @PostMapping("/run")
41     @PreAuthorize("hasRole('ADMIN') or hasAuthority('SCHEDULER_TRIGGER')")
42     public ResponseEntity<?> runScheduler() {
43         try {
44             leaveStatusScheduler.runManually();
45             return ResponseEntity.ok(Map.of(
46                 "status", "success",
47                 "message", "Leave status scheduler executed successfully"
48             ));
49         } catch (Exception e) {
50             log.error("Leave status scheduler failed", e);
51             return ResponseEntity.internalServerError().body(Map.of(
52                 "status", "error",
53                 "message", e.getMessage()
54             ));
55         }
56     }
57
58     /**
59      * FIX: Manual trigger for sick leave reset (missing from original).
60      * Useful for recovery and testing without waiting for the end-of-month cron.
61      */
62     @PostMapping("/sick-leave-reset")
63     @PreAuthorize("hasRole('ADMIN')")
64     public ResponseEntity<?> runSickLeaveReset() {
65         try {
66             LocalDate lastDayOfMonth = YearMonth.now().atEndOfMonth();
67             sickLeaveResetService.runMonthlyReset(lastDayOfMonth);
68             return ResponseEntity.ok(Map.of(
69                 "status", "success",
70                 "message", "Sick leave reset executed for " + lastDayOfMonth
71             ));

```

```

72         } catch (Exception e) {
73             log.error("Sick leave reset failed", e);
74             return ResponseEntity.internalServerError().body(Map.of(
75                 "status", "error",
76                 "message", e.getMessage()
77             ));
78         }
79     }
80
81     /**
82      * FIX: Manual trigger for monthly leave accrual (missing from original).
83      * Useful for onboarding new employees or recovering from a missed cron run.
84      */
85     @PostMapping("/accrue")
86     @PreAuthorize("hasRole('ADMIN')")
87     public ResponseEntity<?> runMonthlyAccrual() {
88         try {
89             LocalDate today = LocalDate.now();
90             leaveAccrualService.runMonthlyAccrual(today);
91             return ResponseEntity.ok(Map.of(
92                 "status", "success",
93                 "message", "Monthly leave accrual executed for " + today
94             ));
95         } catch (Exception e) {
96             log.error("Monthly accrual failed", e);
97             return ResponseEntity.internalServerError().body(Map.of(
98                 "status", "error",
99                 "message", e.getMessage()
100             ));
101         }
102     }
103 }

```

26 Current:

src\main\java\com\example\EmployeeManagementSystem\Controller\SchedulerTestController.java

```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.Service.CarryForwardWarningService;
4 import com.example.EmployeeManagementSystem.Service.LeaveAccrualService;
5 import com.example.EmployeeManagementSystem.Service.SickLeaveResetService;
6 import com.example.EmployeeManagementSystem.Service.YearEndCarryForwardService;
7 import org.springframework.http.ResponseEntity;
8 import org.springframework.security.access.prepost.PreAuthorize;
9 import org.springframework.web.bind.annotation.PostMapping;
10 import org.springframework.web.bind.annotation.RequestMapping;
11 import org.springframework.web.bind.annotation.RequestParam;
12 import org.springframework.web.bind.annotation.RestController;
13
14 import java.time.LocalDate;
15
16 @RestController
17 @RequestMapping("/admin/scheduler")
18 @PreAuthorize("hasRole('ADMIN')")
19 public class SchedulerTestController {
20
21     private final LeaveAccrualService accrualService;
22     private final SickLeaveResetService resetService;
23     private final CarryForwardWarningService warningService;
24     private final YearEndCarryForwardService yearEndService;
25
26     public SchedulerTestController(LeaveAccrualService accrualService,
27                                   SickLeaveResetService resetService,
28                                   CarryForwardWarningService warningService,
29                                   YearEndCarryForwardService yearEndService) {
30         this.accrualService = accrualService;
31         this.resetService = resetService;
32         this.warningService = warningService;
33         this.yearEndService = yearEndService;
34     }
35
36     // Trigger monthly accrual as if today were the 1st of the month
37     @PostMapping("/run-accrual")
38     public ResponseEntity<String> runAccrual(
39         @RequestParam(required = false) String date) {
40         LocalDate target = (date != null) ? LocalDate.parse(date) : LocalDate.now();
41         accrualService.runMonthlyAccrual(target);
42         warningService.runWarningCheck(target);
43         return ResponseEntity.ok("Accrual ran for date: " + target);
44     }
45
46     // Trigger sick leave reset as if today were end of month
47     @PostMapping("/run-sick-reset")
48     public ResponseEntity<String> runSickReset(
49         @RequestParam(required = false) String date) {
50         LocalDate target = (date != null) ? LocalDate.parse(date) : LocalDate.now();
51         resetService.runMonthlyReset(target);
52         return ResponseEntity.ok("Sick leave reset ran for date: " + target);
53     }
54
55     // Trigger year-end carry forward
56     @PostMapping("/run-year-end")
57     public ResponseEntity<String> runYearEnd(
58         @RequestParam(required = false) Integer year) {
59         int targetYear = (year != null) ? year : LocalDate.now().getYear();
60         yearEndService.runYearEnd(targetYear);
61         return ResponseEntity.ok("Year-end ran for year: " + targetYear);
62     }
63 }
```

27 Current:

src\main\java\com\example\EmployeeManagementSystem\Controller\ServiceRequestController.java

```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.AdminActionDTO;
4 import com.example.EmployeeManagementSystem.DTO.RepairDTO;
5 import com.example.EmployeeManagementSystem.DTO.ServiceRequestDTO;
6 import com.example.EmployeeManagementSystem.DTO.ServiceRequestResponseDTO;
7 import com.example.EmployeeManagementSystem.Entity.ServiceRequest;
8 import com.example.EmployeeManagementSystem.Service.ServiceRequestService;
9 import org.springframework.http.HttpStatus;
10 import org.springframework.http.ResponseEntity;
11 import org.springframework.security.access.prepost.PreAuthorize;
12 import org.springframework.security.core.Authentication;
13 import org.springframework.web.bind.annotation.GetMapping;
14 import org.springframework.web.bind.annotation.PostMapping;
15 import org.springframework.web.bind.annotation.PutMapping;
16 import org.springframework.web.bind.annotation.RequestBody;
17 import org.springframework.web.bind.annotation.RequestMapping;
18 import org.springframework.web.bind.annotation.RestController;
19
20 import java.util.List;
21
22 @RestController
23 @RequestMapping("/service-requests")
24 public class ServiceRequestController {
25     private final ServiceRequestService serviceRequestService;
26
27     public ServiceRequestController(ServiceRequestService serviceRequestService) {
28         this.serviceRequestService = serviceRequestService;
29     }
30
31     @PostMapping
32     @PreAuthorize("hasAnyRole('EMPLOYEE', 'MANAGER', 'ADMIN')")
33     public ResponseEntity<ServiceRequest> createServiceRequest(
34         @RequestBody ServiceRequestDTO serviceRequestDTO,
35         Authentication authentication) {
36         return ResponseEntity.status(HttpStatus.CREATED)
37             .body(serviceRequestService.createServiceRequest(serviceRequestDTO, authentication));
38     }
39
40     @GetMapping("/my")
41     @PreAuthorize("hasAnyRole('EMPLOYEE', 'MANAGER', 'ADMIN')")
42     public ResponseEntity<List<ServiceRequestResponseDTO>> getMyServiceRequests(Authentication
43 authentication) {
44         return ResponseEntity.ok(serviceRequestService.getMyServiceRequests(authentication));
45     }
46
47     @GetMapping("/open")
48     @PreAuthorize("hasRole('ADMIN')")
49     public ResponseEntity<List<ServiceRequestResponseDTO>> getAllOpenServiceRequests() {
50         return ResponseEntity.ok(serviceRequestService.getAllOpenServiceRequests());
51     }
52
53     @PutMapping("/admin")
54     @PreAuthorize("hasRole('ADMIN')")
55     public ResponseEntity<ServiceRequestResponseDTO> updateServiceRequestByAdmin(
56         Authentication authentication,
57         @RequestBody AdminActionDTO actionDTO) {
58         return ResponseEntity.ok(serviceRequestService.updateServiceRequestByAdmin(authentication,
59 actionDTO));
60     }
61
62     @GetMapping("/vendor/me")
63     @PreAuthorize("hasRole('TECH_VENDOR')")
64     public ResponseEntity<List<ServiceRequestResponseDTO>> getAllServiceRequestByVendor(Authentication
65 authentication) {
66         return ResponseEntity.ok(serviceRequestService.getAllServiceRequestByVendor(authentication));
67     }
68
69     @PutMapping("/vendor")
70     @PreAuthorize("hasRole('TECH_VENDOR')")
71     public ResponseEntity<ServiceRequestResponseDTO> updateServiceRequestByVendor(
```

```
69         Authentication authentication,
70         @RequestBody RepairDTO repairDTO) {
71         return ResponseEntity.ok(serviceRequestService.updateServiceRequestByVendor(authentication,
72         repairDTO));
73     }
74 }
```

[illegible]

```
70         // Invalidate Spring Security context
71         SecurityContextHolder.clearContext();
72
73         // Invalidate HTTP session (kills OAuth2 session too)
74         session.invalidate();
75
76         // Clear the session cookie
77         Cookie cookie = new Cookie("JSESSIONID", null);
78         cookie.setMaxAge(0);
79         cookie.setPath("/");
80         response.addCookie(cookie);
81
82         return ResponseEntity.ok("Logged out");
83     }
84
85     @GetMapping("/me")
86     public ResponseEntity<?> me(Authentication authentication) {
87         if (authentication == null || !authentication.isAuthenticated()) {
88             return ResponseEntity.status(401).body(Map.of("error", "Not logged in"));
89         }
90         return ResponseEntity.ok(Map.of(
91             "username", authentication.getName(),
92             "roles", authentication.getAuthorities().toString(),
93             "authenticated", true
94         ));
95     }
96 }
```

```

1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.SubscriptionDTO;
4 import com.example.EmployeeManagementSystem.DTO.SubscriptionRequest;
5 import com.example.EmployeeManagementSystem.Entity.Subscription;
6 import com.example.EmployeeManagementSystem.Service.SubscriptionService;
7 import org.springframework.http.HttpStatus;
8 import org.springframework.http.ResponseEntity;
9 import org.springframework.security.access.prepost.PreAuthorize;
10 import org.springframework.security.core.Authentication;
11 import org.springframework.web.bind.annotation.*;
12
13 import java.util.List;
14
15 @RestController
16 @RequestMapping("/subscription")
17 public class SubscriptionController {
18     private final SubscriptionService subscriptionService;
19
20     public SubscriptionController(SubscriptionService subscriptionService) {
21         this.subscriptionService = subscriptionService;
22     }
23
24     //Authentication makes sure that whoever is logged can add subscription and cannot add other
25     //subscription
26     @PostMapping
27     @PreAuthorize("hasAnyRole('EMPLOYEE','MANAGER','ADMIN','VENDOR')")
28     public ResponseEntity<?> addSubscription(@RequestBody SubscriptionRequest request, Authentication
29     authentication) {
30         subscriptionService.addSubscription(request, authentication);
31         return ResponseEntity.status(HttpStatus.CREATED)
32             .body("Added new subscription for user: " + authentication.getName());
33     }
34
35     @GetMapping
36     @PreAuthorize("hasAuthority('SUBSCRIPTION_READ')")
37     public List<SubscriptionDTO> getAllSubscriptions() {
38         return subscriptionService.getAllSubscriptions();
39     }
40
41     @PutMapping("/{id}")
42     @PreAuthorize("hasAnyRole('EMPLOYEE','MANAGER')")
43     public ResponseEntity<Subscription> updateSubscription(
44         @PathVariable Long id,
45         @RequestBody SubscriptionRequest request) {
46         return ResponseEntity.ok(subscriptionService.updateSubscription(id, request));
47     }
48
49     @PutMapping("/{id}/pause")
50     @PreAuthorize("hasAnyRole('EMPLOYEE','MANAGER')")
51     public ResponseEntity<Subscription> pauseSubscription(@PathVariable Long id) {
52         return ResponseEntity.ok(subscriptionService.pauseSubscription(id));
53     }
54
55     @PutMapping("/{id}/resume")
56     @PreAuthorize("hasAnyRole('EMPLOYEE','MANAGER')")
57     public ResponseEntity<Subscription> resumeSubscription(@PathVariable Long id) {
58         return ResponseEntity.ok(subscriptionService.resumeSubscription(id));
59     }
60
61     @PutMapping("/{id}/expire")
62     @PreAuthorize("hasAnyRole('EMPLOYEE','MANAGER')")
63     public ResponseEntity<Subscription> expireSubscription(@PathVariable Long id) {
64         return ResponseEntity.ok(subscriptionService.expireSubscription(id));
65     }
66
67     @GetMapping("/user")
68     @PreAuthorize("hasAuthority('SUBSCRIPTION_READ')")
69     public ResponseEntity<List<SubscriptionDTO>> getSubscriptionOfUser(Authentication authentication){
70         return ResponseEntity.ok(subscriptionService.getSubscriptionOfUser(authentication));
71     }

```


70 }

71

72 }

73

```

1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
5 import com.example.EmployeeManagementSystem.Service.TotpService;
6 import org.springframework.http.ResponseEntity;
7 import org.springframework.security.core.Authentication;
8 import org.springframework.web.bind.annotation.*;
9
10 import java.util.Map;
11
12 @RestController
13 @RequestMapping("/totp")
14 public class TotpController {
15
16     private final TotpService totpService;
17     private final EmployeeRepo employeeRepo;
18
19     public TotpController(TotpService totpService, EmployeeRepo employeeRepo) {
20         this.totpService = totpService;
21         this.employeeRepo = employeeRepo;
22     }
23
24     /**
25      * GET /totp/status
26      * Returns whether TOTP is currently enabled for the logged-in employee.
27      * Used by the frontend to show the correct UI state on load.
28      */
29     @GetMapping("/status")
30     public ResponseEntity<?> status(Authentication authentication) {
31         Employee employee = getEmployee(authentication);
32         return ResponseEntity.ok(Map.of(
33             "totpEnabled", employee.isTotpEnabled()
34         ));
35     }
36
37     /**
38      * POST /totp/setup
39      * Step 1 – Generate a new TOTP secret and return the QR code URI.
40      * Employee scans the QR with Google Authenticator / Authy.
41      * TOTP is NOT enabled yet until /totp/verify-setup succeeds.
42      */
43     @PostMapping("/setup")
44     public ResponseEntity<?> setup(Authentication authentication) {
45         Employee employee = getEmployee(authentication);
46
47         String secret = totpService.generateSecret();
48         employee.setTotpSecret(secret);
49         employee.setTotpEnabled(false); // not active until confirmed
50         employeeRepo.save(employee);
51
52         String qrUri = totpService.getQrCodeUri(secret, employee.getName());
53
54         return ResponseEntity.ok(Map.of(
55             "secret", secret, // backup code for manual entry
56             "qrCodeUri", qrUri // embed in QR image on frontend
57         ));
58     }
59
60     /**
61      * POST /totp/verify-setup
62      * Step 2 – Employee scans QR and types their first code to confirm setup.
63      * Only after this succeeds is totpEnabled set to true.
64      * Body: { "code": "123456" }
65      */
66     @PostMapping("/verify-setup")
67     public ResponseEntity<?> verifySetup(@RequestBody Map<String, String> body,
68                                         Authentication authentication) {
69         Employee employee = getEmployee(authentication);
70
71         if (employee.getTotpSecret() == null) {

```

```

72         return ResponseEntity.badRequest().body(Map.of(
73             "error", "No TOTP secret found. Call /totp/setup first.");
74     }
75
76     String code = body.get("code");
77     if (code == null || code.isBlank()) {
78         return ResponseEntity.badRequest().body(Map.of("error", "code is required"));
79     }
80
81     if (!totpService.verifyCode(employee.getTotpSecret(), code)) {
82         return ResponseEntity.badRequest().body(Map.of("error", "Invalid code. Try again.));
83     }
84
85     employee.setTotpEnabled(true);
86     employeeRepo.save(employee);
87
88     return ResponseEntity.ok(Map.of("message", "TOTP 2FA enabled successfully"));
89 }
90
91 /**
92  * POST /totp/validate
93  * Step 3 – On every login when totpEnabled=true, call this after password auth.
94  * In the new flow this is handled by /Authenticate/totp using a pre-auth token.
95  * This endpoint is kept for compatibility / manual use.
96  * Body: { "code": "123456" }
97  */
98 @PostMapping("/validate")
99 public ResponseEntity<?> validate(@RequestBody Map<String, String> body,
100     Authentication authentication) {
101     Employee employee = getEmployee(authentication);
102
103     if (!employee.isTotpEnabled()) {
104         return ResponseEntity.ok(Map.of("message", "TOTP not enabled for this user"));
105     }
106
107     String code = body.get("code");
108     if (!totpService.verifyCode(employee.getTotpSecret(), code)) {
109         return ResponseEntity.status(401).body(Map.of("error", "Invalid TOTP code"));
110     }
111
112     return ResponseEntity.ok(Map.of("message", "TOTP verified", "authenticated", true));
113 }
114
115 /**
116  * DELETE /totp/disable
117  * Disable TOTP for the logged-in employee.
118  */
119 @DeleteMapping("/disable")
120 public ResponseEntity<?> disable(Authentication authentication) {
121     Employee employee = getEmployee(authentication);
122     employee.setTotpSecret(null);
123     employee.setTotpEnabled(false);
124     employeeRepo.save(employee);
125     return ResponseEntity.ok(Map.of("message", "TOTP disabled"));
126 }
127
128 // ---- helper ----
129
130 private Employee getEmployee(Authentication authentication) {
131     return employeeRepo.findByEmail(authentication.getName())
132         .orElseThrow(() -> new RuntimeException("Employee not found"));
133 }
134 }

```

```
1 package com.example.EmployeeManagementSystem.Controller;
2
3 import com.example.EmployeeManagementSystem.DTO.VendorDTO;
4 import com.example.EmployeeManagementSystem.DTO.VendorRequest;
5 import com.example.EmployeeManagementSystem.Service.DeliveryService;
6 import org.springframework.http.HttpStatus;
7 import org.springframework.http.ResponseEntity;
8 import org.springframework.security.access.prepost.PreAuthorize;
9 import org.springframework.security.core.Authentication;
10 import org.springframework.web.bind.annotation.*;
11
12 import java.util.List;
13
14 @RestController
15 @RequestMapping("/vendors")
16 public class VendorController {
17
18     private final DeliveryService.VendorService vendorService;
19
20     public VendorController(DeliveryService.VendorService vendorService) {
21         this.vendorService = vendorService;
22     }
23
24     @PostMapping("/register")
25     @PreAuthorize("hasRole('ADMIN')")
26     public ResponseEntity<VendorDTO> registerVendor(@RequestBody VendorRequest request) {
27         return ResponseEntity.status(HttpStatus.CREATED)
28             .body(vendorService.registerVendor(request));
29     }
30
31     @GetMapping
32     @PreAuthorize("hasAnyRole('FOOD_VENDOR', 'ADMIN', 'TECH_VENDOR')")
33     public ResponseEntity<List<VendorDTO>> getAllVendors() {
34         return ResponseEntity.ok(vendorService.getAllVendors());
35     }
36
37     @GetMapping("/me")
38     @PreAuthorize("hasAuthority('VENDOR_WRITE')")
39     public ResponseEntity<VendorDTO> getVendor(Authentication authentication) {
40         return ResponseEntity.ok(vendorService.getVendor(authentication));
41     }
42
43
44     @PutMapping("/update")
45     @PreAuthorize("hasAuthority('VENDOR_WRITE')")
46     public ResponseEntity<VendorDTO> updateVendor(
47         @RequestBody VendorRequest request,
48         Authentication authentication) {
49         return ResponseEntity.ok(vendorService.updateVendor(request, authentication));
50     }
51
52     @DeleteMapping
53     @PreAuthorize("hasRole('ADMIN')")
54     public ResponseEntity<String> deleteVendor(Authentication authentication) {
55         vendorService.deleteVendor(authentication);
56         return ResponseEntity.ok(
57             "Vendor account for " + authentication.getName() + " has been closed.");
58     }
59 }
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 public class AccessTokenRequest {
4     private String refreshToken;
5
6     public AccessTokenRequest(String refreshToken) {
7         this.refreshToken = refreshToken;
8     }
9
10    public void setRefreshToken(String refreshToken) {
11        this.refreshToken = refreshToken;
12    }
13
14    public String getRefreshToken() {
15        return refreshToken;
16    }
17 }
18
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3
4 public class ActionDTO {
5     private long leaveRequestId;
6     private String action;
7     private String remarks;
8
9     public long getLeaveRequestId() {
10         return leaveRequestId;
11     }
12
13     public void setLeaveRequestId(long leaveRequestId) {
14         this.leaveRequestId = leaveRequestId;
15     }
16
17
18     public String getAction() {
19         return action;
20     }
21
22     public void setAction(String action) {
23         this.action = action;
24     }
25
26     public String getRemarks() {
27         return remarks;
28     }
29
30     public void setRemarks(String remarks) {
31         this.remarks = remarks;
32     }
33 }
34
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.ServiceRequestStatus;
4
5 public class AdminActionDTO {
6     private Long serviceRequestId;
7     private ServiceRequestStatus status;
8     private String adminRemarks;
9
10    public Long getServiceRequestId() {
11        return serviceRequestId;
12    }
13
14    public void setServiceRequestId(Long serviceRequestId) {
15        this.serviceRequestId = serviceRequestId;
16    }
17
18    public String getAdminRemarks() {
19        return adminRemarks;
20    }
21
22    public void setAdminRemarks(String adminRemarks) {
23        this.adminRemarks = adminRemarks;
24    }
25
26    public ServiceRequestStatus getStatus() {
27        return status;
28    }
29
30    public void setStatus(ServiceRequestStatus status) {
31        this.status = status;
32    }
33 }
34
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.Gender;
4
5 public class AdminEmployeeDTO {
6     private String name;
7     private String email;
8     private String password;
9     private Gender gender;
10    private String timezone;
11    private long managerId;
12
13    public String getName() {
14        return name;
15    }
16
17    public String getPassword() {
18        return password;
19    }
20
21    public String getEmail() {
22        return email;
23    }
24
25    public String getTimezone() {
26        return timezone;
27    }
28
29    public Gender getGender() {
30        return gender;
31    }
32
33    public long getManagerId() {
34        return managerId;
35    }
36
37    public void setEmail(String email) {
38        this.email = email;
39    }
40
41    public void setPassword(String password) {
42        this.password = password;
43    }
44
45    public void setManagerId(long managerId) {
46        this.managerId = managerId;
47    }
48
49    public void setTimezone(String timezone) {
50        this.timezone = timezone;
51    }
52
53    public void setGender(Gender gender) {
54        this.gender = gender;
55    }
56
57    public void setName(String name) {
58        this.name = name;
59    }
60 }
61
```



```
1 package com.example.EmployeeManagementSystem.DTO;  
2  
3 public class AssignDeviceRequest {  
4     private Long employeeId;  
5  
6     public Long getEmployeeId() {  
7         return employeeId;  
8     }  
9  
10    public void setEmployeeId(Long employeeId) {  
11        this.employeeId = employeeId;  
12    }  
13 }  
14
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3
4 import com.fasterxml.jackson.annotation.JsonProperty;
5
6 public class AuthRequest {
7     @JsonProperty("username")
8     private String Username;
9     @JsonProperty("password")
10    private String Password;
11
12    public String getPassword() {
13        return Password;
14    }
15
16    public void setPassword(String password) {
17        Password = password;
18    }
19
20    public String getUsername() {
21        return Username;
22    }
23
24    public void setUsername(String username) {
25        Username = username;
26    }
27 }
28
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 public class AuthResponse {
4     private String accessToken;
5     private String refreshToken;
6
7     public AuthResponse(String accessToken, String token) {
8         this.accessToken=accessToken;
9         this.refreshToken=token;
10    }
11
12    public String getAccessToken() {
13        return accessToken;
14    }
15
16    public void setAccessToken(String accessToken) {
17        this.accessToken = accessToken;
18    }
19
20    public String getRefreshToken() {
21        return refreshToken;
22    }
23
24    public void setRefreshToken(String refreshToken) {
25        this.refreshToken = refreshToken;
26    }
27 }
28
```

39 Current:

src\main\java\com\example\EmployeeManagementSystem\DTO\DeviceAssignmentResponseDTO.java

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.AssignmentStatus;
4 import com.example.EmployeeManagementSystem.Enum.DeviceType;
5
6 import java.time.LocalDate;
7
8 public class DeviceAssignmentResponseDTO {
9     private Long id;
10    private Long deviceId;
11    private String deviceName;
12    private String deviceBrand;
13    private DeviceType deviceType;
14    private Long employeeId;
15    private String employeeName;
16    private AssignmentStatus status;
17    private LocalDate assignedDate;
18
19    public Long getId() {
20        return id;
21    }
22
23    public void setId(Long id) {
24        this.id = id;
25    }
26
27    public String getDeviceName() {
28        return deviceName;
29    }
30
31    public void setDeviceName(String deviceName) {
32        this.deviceName = deviceName;
33    }
34
35    public Long getDeviceId() {
36        return deviceId;
37    }
38
39    public void setDeviceId(Long deviceId) {
40        this.deviceId = deviceId;
41    }
42
43    public DeviceType getDeviceType() {
44        return deviceType;
45    }
46
47    public void setDeviceType(DeviceType deviceType) {
48        this.deviceType = deviceType;
49    }
50
51    public String getDeviceBrand() {
52        return deviceBrand;
53    }
54
55    public void setDeviceBrand(String deviceBrand) {
56        this.deviceBrand = deviceBrand;
57    }
58
59    public String getEmployeeName() {
60        return employeeName;
61    }
62
63    public void setEmployeeName(String employeeName) {
64        this.employeeName = employeeName;
65    }
66
67    public Long getEmployeeId() {
68        return employeeId;
69    }
70
71    public void setEmployeeId(Long employeeId) {
```

```
72         this.employeeId = employeeId;
73     }
74
75     public AssignmentStatus getStatus() {
76         return status;
77     }
78
79     public void setStatus(AssignmentStatus status) {
80         this.status = status;
81     }
82
83     public LocalDate getAssignedDate() {
84         return assignedDate;
85     }
86
87     public void setAssignedDate(LocalDate assignedDate) {
88         this.assignedDate = assignedDate;
89     }
90 }
91
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.DeviceType;
4
5 import java.time.LocalDate;
6
7 public class DeviceDTO {
8     private String deviceName;        // e.g. "Dell XPS 15"
9     private String brand;
10    private DeviceType deviceType;
11    private LocalDate warrantyExpiryDate;
12
13    public String getDeviceName() {
14        return deviceName;
15    }
16
17    public void setDeviceName(String deviceName) {
18        this.deviceName = deviceName;
19    }
20
21    public String getBrand() {
22        return brand;
23    }
24
25    public void setBrand(String brand) {
26        this.brand = brand;
27    }
28
29    public DeviceType getDeviceType() {
30        return deviceType;
31    }
32
33    public void setDeviceType(DeviceType deviceType) {
34        this.deviceType = deviceType;
35    }
36
37    public LocalDate getWarrantyExpiryDate() {
38        return warrantyExpiryDate;
39    }
40
41    public void setWarrantyExpiryDate(LocalDate warrantyExpiryDate) {
42        this.warrantyExpiryDate = warrantyExpiryDate;
43    }
44 }
45
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.Vendor;
5 import com.example.EmployeeManagementSystem.Enum.DeviceStatus;
6 import com.example.EmployeeManagementSystem.Enum.DeviceType;
7
8 import java.time.LocalDate;
9
10 public class DeviceResponseDTO {
11     private Long id;
12     private String deviceName;
13     private String brand;
14     private DeviceType deviceType;
15     private DeviceStatus deviceStatus;
16     private String vendorName;
17     private long assignedEmployeeId;
18     private String assignedEmployeeName;
19     private LocalDate assignedDate;
20
21     private LocalDate warrantyExpiryDate;
22
23     public Long getId() {
24         return id;
25     }
26
27     public void setId(Long id) {
28         this.id = id;
29     }
30
31     public String getDeviceName() {
32         return deviceName;
33     }
34
35     public void setDeviceName(String deviceName) {
36         this.deviceName = deviceName;
37     }
38
39     public String getBrand() {
40         return brand;
41     }
42
43     public void setBrand(String brand) {
44         this.brand = brand;
45     }
46
47     public DeviceType getDeviceType() {
48         return deviceType;
49     }
50
51     public void setDeviceType(DeviceType deviceType) {
52         this.deviceType = deviceType;
53     }
54
55     public DeviceStatus getDeviceStatus() {
56         return deviceStatus;
57     }
58
59     public void setDeviceStatus(DeviceStatus deviceStatus) {
60         this.deviceStatus = deviceStatus;
61     }
62
63     public String getVendorName() {
64         return vendorName;
65     }
66
67     public void setVendorName(String vendorName) {
68         this.vendorName = vendorName;
69     }
70
71     public long getAssignedEmployeeId() {
```

```
72         return assignedEmployeeId;
73     }
74
75     public void setAssignedEmployeeId(long assignedEmployeeId) {
76         this.assignedEmployeeId = assignedEmployeeId;
77     }
78
79     public String getAssignedEmployeeName() {
80         return assignedEmployeeName;
81     }
82
83     public void setAssignedEmployeeName(String assignedEmployeeName) {
84         this.assignedEmployeeName = assignedEmployeeName;
85     }
86
87     public LocalDate getAssignedDate() {
88         return assignedDate;
89     }
90
91     public void setAssignedDate(LocalDate assignedDate) {
92         this.assignedDate = assignedDate;
93     }
94
95     public LocalDate getWarrantyExpiryDate() {
96         return warrantyExpiryDate;
97     }
98
99     public void setWarrantyExpiryDate(LocalDate warrantyExpiryDate) {
100         this.warrantyExpiryDate = warrantyExpiryDate;
101     }
102 }
103
```


42 Current:

src\main\java\com\example\EmployeeManagementSystem\DTO\EmployeeAttendanceResponseDTO.

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import java.util.List;
4
5 public class EmployeeAttendanceResponseDTO {
6     private long workingCount;
7     private long onLeaveCount;
8     private List<EmployeeStatusDTO> workingEmployees;
9     private List<EmployeeStatusDTO> onLeaveEmployees;
10
11     public long getWorkingCount() {
12         return workingCount;
13     }
14
15     public void setWorkingCount(long workingCount) {
16         this.workingCount = workingCount;
17     }
18
19     public long getOnLeaveCount() {
20         return onLeaveCount;
21     }
22
23     public void setOnLeaveCount(long onLeaveCount) {
24         this.onLeaveCount = onLeaveCount;
25     }
26
27     public List<EmployeeStatusDTO> getWorkingEmployees() {
28         return workingEmployees;
29     }
30
31     public void setWorkingEmployees(List<EmployeeStatusDTO> workingEmployees) {
32         this.workingEmployees = workingEmployees;
33     }
34
35     public List<EmployeeStatusDTO> getOnLeaveEmployees() {
36         return onLeaveEmployees;
37     }
38
39     public void setOnLeaveEmployees(List<EmployeeStatusDTO> onLeaveEmployees) {
40         this.onLeaveEmployees = onLeaveEmployees;
41     }
42 }
43
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.Gender;
4
5 public class EmployeeDetails {
6     private String name;
7     private String email;
8     private String password;
9     private String timezone;
10    private Gender gender;
11
12    public String getName() {
13        return name;
14    }
15
16    public void setName(String name) {
17        this.name = name;
18    }
19
20    public String getPassword() {
21        return password;
22    }
23
24    public void setPassword(String password) {
25        this.password = password;
26    }
27
28    public String getEmail() {
29        return email;
30    }
31
32    public void setEmail(String email) {
33        this.email = email;
34    }
35
36    public String getTimezone() {
37        return timezone;
38    }
39
40    public void setTimezone(String timezone) {
41        this.timezone = timezone;
42    }
43
44    public Gender getGender() {
45        return gender;
46    }
47
48    public void setGender(Gender gender) {
49        this.gender = gender;
50    }
51 }
52
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3
4 import com.example.EmployeeManagementSystem.Enum.Gender;
5
6 public class EmployeeDTO {
7     private String name;
8     private String email;
9     private String dept;
10    private String password;
11    private Gender gender;
12    private String timezone;
13
14    public String getTimezone() {
15        return timezone;
16    }
17
18    public void setTimezone(String timezone) {
19        this.timezone = timezone;
20    }
21
22    public String getName() {
23        return name;
24    }
25
26    public void setName(String name) {
27        this.name = name;
28    }
29
30    public String getEmail() {
31        return email;
32    }
33
34    public void setEmail(String email) {
35        this.email = email;
36    }
37
38    public String getDept() {
39        return dept;
40    }
41
42    public void setDept(String dept) {
43        this.dept = dept;
44    }
45
46    public String getPassword() {
47        return password;
48    }
49
50    public void setPassword(String password) {
51        this.password = password;
52    }
53
54    public Gender getGender() {
55        return gender;
56    }
57
58    public void setGender(Gender gender) {
59        this.gender = gender;
60    }
61 }
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.Role;
4 import com.example.EmployeeManagementSystem.Enum.Status;
5
6 public class EmployeeStatusDTO {
7     private long employeeId;
8     private String name;
9     private String email;
10    private String dept;
11    private Role role;
12    private Status status;
13    private String timezone;
14
15    public long getEmployeeId() {
16        return employeeId;
17    }
18
19    public void setEmployeeId(long employeeId) {
20        this.employeeId = employeeId;
21    }
22
23    public String getName() {
24        return name;
25    }
26
27    public void setName(String name) {
28        this.name = name;
29    }
30
31    public String getEmail() {
32        return email;
33    }
34
35    public void setEmail(String email) {
36        this.email = email;
37    }
38
39    public String getDept() {
40        return dept;
41    }
42
43    public void setDept(String dept) {
44        this.dept = dept;
45    }
46
47    public Role getRole() {
48        return role;
49    }
50
51    public void setRole(Role role) {
52        this.role = role;
53    }
54
55    public Status getStatus() {
56        return status;
57    }
58
59    public void setStatus(Status status) {
60        this.status = status;
61    }
62
63    public String getTimezone() {
64        return timezone;
65    }
66
67    public void setTimezone(String timezone) {
68        this.timezone = timezone;
69    }
70 }
71
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 public class HolidayDTO {
4     private Long id;
5     private String name;
6     private String date;
7     private String description;
8     private int year;
9
10    public Long getId() {
11        return id;
12    }
13
14    public void setId(Long id) {
15        this.id = id;
16    }
17
18    public String getName() {
19        return name;
20    }
21
22    public void setName(String name) {
23        this.name = name;
24    }
25
26    public String getDate() {
27        return date;
28    }
29
30    public void setDate(String date) {
31        this.date = date;
32    }
33
34    public String getDescription() {
35        return description;
36    }
37
38    public void setDescription(String description) {
39        this.description = description;
40    }
41
42    public int getYear() {
43        return year;
44    }
45
46    public void setYear(int year) {
47        this.year = year;
48    }
49 }
50
```

```

1 package com.example.EmployeeManagementSystem.DTO;
2
3 import java.math.BigDecimal;
4 import java.util.List;
5 import java.util.Map;
6
7 public class LeaveDashboardResponse {
8
9     private Long    employeeId;
10    private String  employeeName;
11    private int     year;
12
13    // Individual leave balances
14    private LeaveBalanceDto sick;
15    private LeaveBalanceDto casual;
16    private LeaveBalanceDto maternity; // null for non-female employees
17
18    // Carry-forward meter (casual + maternity combined)
19    private CarryForwardMeterDto carryForwardMeter;
20
21    // Generic balances keyed by leave type name, e.g. SICK, CASUAL, MATERNITY.
22    private Map<String, LeaveBalanceDto> leaveBalances;
23
24    // Active warnings (unread)
25    private List<WarningDto> warnings;
26
27    // nested DTOs
28
29    public static class LeaveBalanceDto {
30        private String    leaveType;
31        private BigDecimal opening;
32        private BigDecimal accrued;
33        private BigDecimal used;
34        private BigDecimal available;
35        private boolean    carriesForward;
36
37        public LeaveBalanceDto(String leaveType, BigDecimal opening,
38                                BigDecimal accrued, BigDecimal used,
39                                BigDecimal available, boolean carriesForward) {
40            this.leaveType    = leaveType;
41            this.opening      = opening;
42            this.accrued      = accrued;
43            this.used         = used;
44            this.available    = available;
45            this.carriesForward = carriesForward;
46        }
47
48        public String    getLeaveType()    { return leaveType; }
49        public BigDecimal getOpening()     { return opening; }
50        public BigDecimal getAccrued()     { return accrued; }
51        public BigDecimal getUsed()        { return used; }
52        public BigDecimal getAvailable()   { return available; }
53        public boolean    isCarriesForward() { return carriesForward; }
54    }
55
56    public static class CarryForwardMeterDto {
57        private BigDecimal current; // e.g. 28.00
58        private int        cap;    // always 30
59        private String     label;   // "28/30"
60        private String     status;  // SAFE | WARNING | CRITICAL | CAPPED
61
62        public CarryForwardMeterDto(BigDecimal current, int cap) {
63            this.current = current;
64            this.cap     = cap;
65            this.label   = current.toPlainString() + "/" + cap;
66            this.status  = resolveStatus(current, cap);
67        }
68
69        private String resolveStatus(BigDecimal current, int cap) {
70            int cmp = current.compareTo(BigDecimal.valueOf(cap));
71            if (cmp >= 0) return "CAPPED";

```

```

72         if (current.compareTo(BigDecimal.valueOf(cap - 1)) >= 0) return "CRITICAL";
73         if (current.compareTo(BigDecimal.valueOf(cap - 2)) >= 0) return "WARNING";
74         return "SAFE";
75     }
76
77     public BigDecimal getCurrent() { return current; }
78     public int         getCap()      { return cap; }
79     public String      getLabel()    { return label; }
80     public String      getStatus()   { return status; }
81 }
82
83 public static class WarningDto {
84     private Long id;
85     private String type;
86     private String message;
87     private String date;
88
89     public WarningDto(Long id, String type, String message, String date) {
90         this.id = id; this.type = type;
91         this.message = message; this.date = date;
92     }
93
94     public Long getId()      { return id; }
95     public String getType()  { return type; }
96     public String getMessage() { return message; }
97     public String getDate()  { return date; }
98 }
99
100 // root getters & setters
101
102 public Long         getEmployeeId()      { return employeeId; }
103 public String       getEmployeeName()    { return employeeName; }
104 public int          getYear()             { return year; }
105 public LeaveBalanceDto getSick()          { return sick; }
106 public LeaveBalanceDto getCasual()        { return casual; }
107 public LeaveBalanceDto getMaternity()     { return maternity; }
108 public CarryForwardMeterDto getCarryForwardMeter() { return carryForwardMeter; }
109 public Map<String, LeaveBalanceDto> getLeaveBalances() { return leaveBalances; }
110 public List<WarningDto> getWarnings()     { return warnings; }
111
112 public void setEmployeeId(Long id)        { this.employeeId = id; }
113 public void setEmployeeName(String n)     { this.employeeName = n; }
114 public void setYear(int y)                { this.year = y; }
115 public void setSick(LeaveBalanceDto b)     { this.sick = b; }
116 public void setCasual(LeaveBalanceDto b)   { this.casual = b; }
117 public void setMaternity(LeaveBalanceDto b) { this.maternity = b; }
118 public void setCarryForwardMeter(CarryForwardMeterDto m) { this.carryForwardMeter = m; }
119 public void setLeaveBalances(Map<String, LeaveBalanceDto> b) { this.leaveBalances = b; }
120 public void setWarnings(List<WarningDto> w) { this.warnings = w; }
121 }
122

```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.LeaveType;
4
5 import java.time.LocalDate;
6
7 // DTO used when employee applies for leave.
8 // Contains only input fields required from employee.
9 public class LeaveRequestDTO {
10     private LeaveType leaveType;
11     private LocalDate startDate;
12     private LocalDate endDate;
13     private String reason;
14
15
16     public LeaveType getLeaveType() {
17         return leaveType;
18     }
19
20     public void setLeaveType(LeaveType leaveType) {
21         this.leaveType = leaveType;
22     }
23
24     public LocalDate getStartDate() {
25         return startDate;
26     }
27
28     public void setStartDate(LocalDate startDate) {
29         this.startDate = startDate;
30     }
31
32     public LocalDate getEndDate() {
33         return endDate;
34     }
35
36     public void setEndDate(LocalDate endDate) {
37         this.endDate = endDate;
38     }
39
40     public String getReason() {
41         return reason;
42     }
43
44     public void setReason(String reason) {
45         this.reason = reason;
46     }
47 }
48
```



```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
4 import com.example.EmployeeManagementSystem.Enum.LeaveType;
5
6 import java.time.LocalDate;
7
8 public class LeaveResponseDTO {
9     private long leaveRequestId;
10    private String employeeName;
11    private LeaveType leaveType;
12    private LocalDate startDate;
13    private LocalDate endDate;
14    private long numberOfDays;
15    private String reason;
16    private LeaveStatus status;
17
18    public long getLeaveRequestId() {
19        return leaveRequestId;
20    }
21
22    public void setLeaveRequestId(long leaveRequestId) {
23        this.leaveRequestId = leaveRequestId;
24    }
25
26    public String getEmployeeName() {
27        return employeeName;
28    }
29
30    public void setEmployeeName(String employeeName) {
31        this.employeeName = employeeName;
32    }
33
34    public LocalDate getStartDate() {
35        return startDate;
36    }
37
38    public void setStartDate(LocalDate startDate) {
39        this.startDate = startDate;
40    }
41
42    public LeaveType getLeaveType() {
43        return leaveType;
44    }
45
46    public void setLeaveType(LeaveType leaveType) {
47        this.leaveType = leaveType;
48    }
49
50    public LocalDate getEndDate() {
51        return endDate;
52    }
53
54    public void setEndDate(LocalDate endDate) {
55        this.endDate = endDate;
56    }
57
58    public long getNumberOfDays() {
59        return numberOfDays;
60    }
61
62    public void setNumberOfDays(long numberOfDays) {
63        this.numberOfDays = numberOfDays;
64    }
65
66    public String getReason() {
67        return reason;
68    }
69
70    public void setReason(String reason) {
71        this.reason = reason;
```

```
72     }
73
74     public LeaveStatus getStatus() {
75         return status;
76     }
77
78     public void setStatus(L leaveStatus status) {
79         this.status = status;
80     }
81 }
82
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import org.bouncycastle.crypto.signers.ISOTrailers;
4
5 public class ManagerDTO {
6     private long managerId;
7     private String name;
8     private String email;
9     private String dept;
10
11     public long getManagerId() {
12         return managerId;
13     }
14
15     public void setManagerId(long managerId) {
16         this.managerId = managerId;
17     }
18
19     public String getName() {
20         return name;
21     }
22
23     public void setName(String name) {
24         this.name = name;
25     }
26
27     public String getEmail() {
28         return email;
29     }
30
31     public void setEmail(String email) {
32         this.email = email;
33     }
34
35     public String getDept() {
36         return dept;
37     }
38
39     public void setDept(String dept) {
40         this.dept = dept;
41     }
42 }
43
```

51 Current:

src\main\java\com\example\EmployeeManagementSystem\DTO\ManagerEmployeeDetailsDTO.java

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.Status;
4
5 import java.util.List;
6
7 public class ManagerEmployeeDetailsDTO {
8     private Long employeeId;
9     private String name;
10    private String email;
11    private String dept;
12    private Status status;
13    private String attendance;
14    private List<DeviceResponseDTO> devices;
15
16    public Long getEmployeeId() {
17        return employeeId;
18    }
19
20    public void setEmployeeId(Long employeeId) {
21        this.employeeId = employeeId;
22    }
23
24    public String getName() {
25        return name;
26    }
27
28    public void setName(String name) {
29        this.name = name;
30    }
31
32    public String getEmail() {
33        return email;
34    }
35
36    public void setEmail(String email) {
37        this.email = email;
38    }
39
40    public String getDept() {
41        return dept;
42    }
43
44    public void setDept(String dept) {
45        this.dept = dept;
46    }
47
48    public Status getStatus() {
49        return status;
50    }
51
52    public void setStatus(Status status) {
53        this.status = status;
54    }
55
56    public String getAttendance() {
57        return attendance;
58    }
59
60    public void setAttendance(String attendance) {
61        this.attendance = attendance;
62    }
63
64    public List<DeviceResponseDTO> getDevices() {
65        return devices;
66    }
67
68    public void setDevices(List<DeviceResponseDTO> devices) {
69        this.devices = devices;
70    }
71 }
```



```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
4 import com.example.EmployeeManagementSystem.Enum.LeaveType;
5
6 import java.time.LocalDate;
7
8 public class ManagerLeaveResponseDTO {
9     private String employeeName;
10    private LeaveType leaveType;
11    private LocalDate startDate;
12    private LocalDate endDate;
13    private long numberOfDays;
14    private String reason;
15    private LeaveStatus status;
16
17    public String getEmployeeName() {
18        return employeeName;
19    }
20
21    public void setEmployeeName(String employeeName) {
22        this.employeeName = employeeName;
23    }
24
25    public LeaveType getLeaveType() {
26        return leaveType;
27    }
28
29    public void setLeaveType(LeaveType leaveType) {
30        this.leaveType = leaveType;
31    }
32
33    public LocalDate getStartDate() {
34        return startDate;
35    }
36
37    public void setStartDate(LocalDate startDate) {
38        this.startDate = startDate;
39    }
40
41    public LocalDate getEndDate() {
42        return endDate;
43    }
44
45    public void setEndDate(LocalDate endDate) {
46        this.endDate = endDate;
47    }
48
49    public long getNumberOfDays() {
50        return numberOfDays;
51    }
52
53    public void setNumberOfDays(long numberOfDays) {
54        this.numberOfDays = numberOfDays;
55    }
56
57    public String getReason() {
58        return reason;
59    }
60
61    public void setReason(String reason) {
62        this.reason = reason;
63    }
64
65    public LeaveStatus getStatus() {
66        return status;
67    }
68
69    public void setStatus(LeaveStatus status) {
70        this.status = status;
71    }
```



```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import java.time.LocalDate;
4
5 public class MaternityLeaveRequestDTO {
6     private LocalDate startDate; // only start date – end is always fixed
7     private String reason;
8
9     public LocalDate getStartDate() {
10         return startDate;
11     }
12
13     public void setStartDate(LocalDate startDate) {
14         this.startDate = startDate;
15     }
16
17     public String getReason() {
18         return reason;
19     }
20
21     public void setReason(String reason) {
22         this.reason = reason;
23     }
24 }
25
```



```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 public class PromoteRequest {
4     private String email;
5     private String password;
6
7     public String getEmail() {
8         return email;
9     }
10
11     public void setEmail(String email) {
12         this.email = email;
13     }
14
15     public String getPassword() {
16         return password;
17     }
18
19     public void setPassword(String password) {
20         this.password = password;
21     }
22 }
23
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.ServiceRequestStatus;
4
5 public class RepairDTO {
6     private Long requestId;
7     private String resolution;
8     private ServiceRequestStatus status;
9
10    public Long getRequestId() {
11        return requestId;
12    }
13
14    public void setRequestId(Long requestId) {
15        this.requestId = requestId;
16    }
17
18    public String getResolution() {
19        return resolution;
20    }
21
22    public void setResolution(String resolution) {
23        this.resolution = resolution;
24    }
25
26    public ServiceRequestStatus getStatus() {
27        return status;
28    }
29
30    public void setStatus(ServiceRequestStatus status) {
31        this.status = status;
32    }
33 }
34
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.MealSlot;
4
5 import java.util.Set;
6
7 public class RestaurantDTO {
8     private Long id;
9     private String name;
10    private String address;
11    private String description;
12    private Set<MealSlot> supportedMealSlots;
13    private boolean active;
14    private Long vendorId;
15    private String vendorName;
16
17    public Long getId() { return id; }
18    public void setId(Long id) { this.id = id; }
19
20    public String getName() { return name; }
21    public void setName(String name) { this.name = name; }
22
23    public String getAddress() { return address; }
24    public void setAddress(String address) { this.address = address; }
25
26    public String getDescription() { return description; }
27    public void setDescription(String description) { this.description = description; }
28
29    public Set<MealSlot> getSupportedMealSlots() { return supportedMealSlots; }
30    public void setSupportedMealSlots(Set<MealSlot> supportedMealSlots) {
31        this.supportedMealSlots = supportedMealSlots;
32    }
33
34    public boolean isActive() { return active; }
35    public void setActive(boolean active) { this.active = active; }
36
37    public Long getVendorId() { return vendorId; }
38    public void setVendorId(Long vendorId) { this.vendorId = vendorId; }
39
40    public String getVendorName() { return vendorName; }
41    public void setVendorName(String vendorName) { this.vendorName = vendorName; }
42 }
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.MealSlot;
4 import jakarta.validation.constraints.NotBlank;
5 import jakarta.validation.constraints.NotEmpty;
6 import jakarta.validation.constraints.NotNull;
7
8 import java.util.Set;
9
10 public class RestaurantRequest {
11
12     @NotBlank
13     private String name;
14
15     @NotBlank
16     private String address;
17
18     private String description;
19
20     /**
21      * Which meal slots this restaurant serves.
22      * Must be at least one of: BREAKFAST, LUNCH, DINNER.
23      */
24     @NotNull
25     @NotEmpty
26     private Set<MealSlot> supportedMealSlots;
27
28     /**
29      * The vendor ID of the owner making this request.
30      * Used to verify ownership on updates/deletes.
31      */
32
33     public String getName() { return name; }
34     public void setName(String name) { this.name = name; }
35
36     public String getAddress() { return address; }
37     public void setAddress(String address) { this.address = address; }
38
39     public String getDescription() { return description; }
40     public void setDescription(String description) { this.description = description; }
41
42     public Set<MealSlot> getSupportedMealSlots() { return supportedMealSlots; }
43     public void setSupportedMealSlots(Set<MealSlot> supportedMealSlots) {
44         this.supportedMealSlots = supportedMealSlots;
45     }
46 }
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.ServiceRequestType;
4
5 public class ServiceRequestDTO {
6
7     private Long deviceId;
8     private ServiceRequestType requestType;
9     private String issueDescription;
10    private boolean urgent;
11
12    public Long getDeviceId() {
13        return deviceId;
14    }
15
16    public void setDeviceId(Long deviceId) {
17        this.deviceId = deviceId;
18    }
19
20    public ServiceRequestType getRequestType() {
21        return requestType;
22    }
23
24    public void setRequestType(ServiceRequestType requestType) {
25        this.requestType = requestType;
26    }
27
28    public String getIssueDescription() {
29        return issueDescription;
30    }
31
32    public void setIssueDescription(String issueDescription) {
33        this.issueDescription = issueDescription;
34    }
35
36    public boolean isUrgent() {
37        return urgent;
38    }
39
40    public void setUrgent(boolean urgent) {
41        this.urgent = urgent;
42    }
43 }
44
```

59 Current:

src\main\java\com\example\EmployeeManagementSystem\DTO\ServiceRequestResponseDTO.java

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import java.time.LocalDateTime;
4
5 public class ServiceRequestResponseDTO
6 {
7     private Long id;
8     private String requestType;
9     private String status;
10    private String issueDescription;
11    private String adminRemarks;
12    private String resolution;
13    private boolean urgent;
14    private LocalDateTime raisedAt;
15    private LocalDateTime resolvedAt;
16    private String deviceName;
17    private String raisedByName;
18    private String reviewedByName;
19
20    public String getDeviceName() {
21        return deviceName;
22    }
23
24    public void setDeviceName(String deviceName) {
25        this.deviceName = deviceName;
26    }
27
28    public String getReviewedByName() {
29        return reviewedByName;
30    }
31
32    public void setReviewedByName(String reviewedByName) {
33        this.reviewedByName = reviewedByName;
34    }
35
36    public LocalDateTime getResolvedAt() {
37        return resolvedAt;
38    }
39
40    public void setResolvedAt(LocalDateTime resolvedAt) {
41        this.resolvedAt = resolvedAt;
42    }
43
44    public LocalDateTime getRaisedAt() {
45        return raisedAt;
46    }
47
48    public void setRaisedAt(LocalDateTime raisedAt) {
49        this.raisedAt = raisedAt;
50    }
51
52    public boolean isUrgent() {
53        return urgent;
54    }
55
56    public void setUrgent(boolean urgent) {
57        this.urgent = urgent;
58    }
59
60    public String getResolution() {
61        return resolution;
62    }
63
64    public void setResolution(String resolution) {
65        this.resolution = resolution;
66    }
67
68    public Long getId() {
69        return id;
70    }
71}
```

```
72     public void setId(Long id) {
73         this.id = id;
74     }
75
76     public String getRequestType() {
77         return requestType;
78     }
79
80     public void setRequestType(String requestType) {
81         this.requestType = requestType;
82     }
83
84     public String getStatus() {
85         return status;
86     }
87
88     public void setStatus(String status) {
89         this.status = status;
90     }
91
92     public String getAdminRemarks() {
93         return adminRemarks;
94     }
95
96     public void setAdminRemarks(String adminRemarks) {
97         this.adminRemarks = adminRemarks;
98     }
99
100    public String getIssueDescription() {
101        return issueDescription;
102    }
103
104    public void setIssueDescription(String issueDescription) {
105        this.issueDescription = issueDescription;
106    }
107
108    public String getRaisedByName() {
109        return raisedByName;
110    }
111
112    public void setRaisedByName(String raisedByName) {
113        this.raisedByName = raisedByName;
114    }
115 }
116
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.MealSlot;
4 import com.example.EmployeeManagementSystem.Enum.ScheduleType;
5 import com.example.EmployeeManagementSystem.Enum.SubscriptionStatus;
6
7 import java.time.DayOfWeek;
8 import java.time.ZonedDateTime;
9
10 public class SubscriptionDTO {
11     private long subscriptionId;
12     private long userId;
13     private MealSlot mealSlot;
14     private ScheduleType scheduleType;
15     private DayOfWeek dayOfWeek;
16     private SubscriptionStatus status;
17     private ZonedDateTime nextDeliveryTime;
18
19     private Long restaurantId;
20     private String restaurantName;
21     private String restaurantAddress;
22
23     public void setRestaurantId(Long restaurantId) {
24         this.restaurantId = restaurantId;
25     }
26
27     public void setRestaurantName(String restaurantName) {
28         this.restaurantName = restaurantName;
29     }
30
31     public void setRestaurantAddress(String restaurantAddress) {
32         this.restaurantAddress = restaurantAddress;
33     }
34
35     public Long getRestaurantId() {
36         return restaurantId;
37     }
38
39     public String getRestaurantName() {
40         return restaurantName;
41     }
42
43     public String getRestaurantAddress() {
44         return restaurantAddress;
45     }
46
47     public long getSubscriptionId() {
48         return subscriptionId;
49     }
50
51     public void setSubscriptionId(long subscriptionId) {
52         this.subscriptionId = subscriptionId;
53     }
54
55     public long getUserId() {
56         return userId;
57     }
58
59     public void setUserId(long userId) {
60         this.userId = userId;
61     }
62
63     public MealSlot getMealSlot() {
64         return mealSlot;
65     }
66
67     public void setMealSlot(MealSlot mealSlot) {
68         this.mealSlot = mealSlot;
69     }
70
71     public ScheduleType getScheduleType() {
```



```
72         return scheduleType;
73     }
74
75     public void setScheduleType(ScheduleType scheduleType) {
76         this.scheduleType = scheduleType;
77     }
78
79     public DayOfWeek getDayOfWeek() {
80         return dayOfWeek;
81     }
82
83     public void setDayOfWeek(DayOfWeek dayOfWeek) {
84         this.dayOfWeek = dayOfWeek;
85     }
86
87     public SubscriptionStatus getStatus() {
88         return status;
89     }
90
91     public void setStatus(SubscriptionStatus status) {
92         this.status = status;
93     }
94
95     public ZonedDateTime getNextDeliveryTime() {
96         return nextDeliveryTime;
97     }
98
99     public void setNextDeliveryTime(ZonedDateTime nextDeliveryTime) {
100         this.nextDeliveryTime = nextDeliveryTime;
101     }
102 }
103
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.MealSlot;
4 import com.example.EmployeeManagementSystem.Enum.ScheduleType;
5 import jakarta.validation.constraints.NotEmpty;
6 import jakarta.validation.constraints.NotNull;
7
8 import java.time.DayOfWeek;
9 import java.util.List;
10
11 public class SubscriptionRequest {
12     private ScheduleType scheduleType;
13     private DayOfWeek dayOfWeek;
14     @NotNull
15     @NotEmpty
16     private List<MealSlot> mealSlots;
17     @NotNull
18     private List<SlotOrder> slotOrders;
19     public static class SlotOrder {
20         @NotNull
21         private MealSlot mealSlot;
22
23         @NotNull
24         private Long restaurantId;
25
26         public MealSlot getMealSlot() {
27             return mealSlot;
28         }
29
30         public void setMealSlot(MealSlot mealSlot) {
31             this.mealSlot = mealSlot;
32         }
33
34         public Long getRestaurantId() {
35             return restaurantId;
36         }
37
38         public void setRestaurantId(Long restaurantId) {
39             this.restaurantId = restaurantId;
40         }
41     }
42
43     public ScheduleType getScheduleType() {
44         return scheduleType;
45     }
46
47     public void setScheduleType(ScheduleType scheduleType) {
48         this.scheduleType = scheduleType;
49     }
50
51     public List<MealSlot> getMealSlots() {
52         return mealSlots;
53     }
54
55     public void setMealSlots(List<MealSlot> mealSlots) {
56         this.mealSlots = mealSlots;
57     }
58
59     public DayOfWeek getDayOfWeek() {
60         return dayOfWeek;
61     }
62
63     public void setDayOfWeek(DayOfWeek dayOfWeek) {
64         this.dayOfWeek = dayOfWeek;
65     }
66
67
68
69     public List<SlotOrder> getSlotOrders() { return slotOrders; }
70     public void setSlotOrders(List<SlotOrder> slotOrders) { this.slotOrders = slotOrders; }
71 }
```



```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import java.time.LocalDate;
4
5 public class VendorDTO {
6     private Long id;
7     private String name;
8     private String email;
9     private String phone;
10    private LocalDate registeredAt;
11
12    public Long getId() { return id; }
13    public void setId(Long id) { this.id = id; }
14
15    public String getName() { return name; }
16    public void setName(String name) { this.name = name; }
17
18    public String getEmail() { return email; }
19    public void setEmail(String email) { this.email = email; }
20
21    public String getPhone() { return phone; }
22    public void setPhone(String phone) { this.phone = phone; }
23
24    public LocalDate getRegisteredAt() { return registeredAt; }
25    public void setRegisteredAt(LocalDate registeredAt) { this.registeredAt = registeredAt; }
26 }
```

```
1 package com.example.EmployeeManagementSystem.DTO;
2
3 import com.example.EmployeeManagementSystem.Enum.Role;
4 import jakarta.validation.constraints.Email;
5 import jakarta.validation.constraints.NotBlank;
6
7 public class VendorRequest {
8
9     @NotBlank
10     private String name;
11
12     @Email
13     @NotBlank
14     private String email;
15
16     @NotBlank
17     private String phone;
18
19     private String password;
20
21     private Role role;
22
23     public String getName() { return name; }
24     public void setName(String name) { this.name = name; }
25
26     public String getEmail() { return email; }
27     public void setEmail(String email) { this.email = email; }
28
29     public String getPhone() { return phone; }
30     public void setPhone(String phone) { this.phone = phone; }
31
32     public String getPassword() {
33         return password;
34     }
35
36     public void setPassword(String password) {
37         this.password = password;
38     }
39
40     public Role getRole() {
41         return role;
42     }
43
44     public void setRole(Role role) {
45         this.role = role;
46     }
47 }
```

64 Current:

src\main\java\com\example\EmployeeManagementSystem\EmployeeManagementSystemApplication

```
1 package com.example.EmployeeManagementSystem;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5 import org.springframework.scheduling.annotation.EnableScheduling;
6
7 @SpringBootApplication
8 @EnableScheduling
9 public class EmployeeManagementSystemApplication {
10
11     public static void main(String[] args) {
12         SpringApplication.run(EmployeeManagementSystemApplication.class, args);
13     }
14
15 }
16
```

```
1 // src/main/java/com/example/EmployeeManagementSystem/Entity/ApiKey.java
2 package com.example.EmployeeManagementSystem.Entity;
3
4 import jakarta.persistence.*;
5 import java.time.Instant;
6 import java.util.UUID;
7
8 @Entity
9 @Table(name = "api_keys")
10 public class ApiKey {
11
12     @Id
13     @GeneratedValue(strategy = GenerationType.IDENTITY)
14     private Long id;
15
16     @Column(nullable = false, unique = true)
17     private String apiKey;
18
19     @Column(nullable = false)
20     private String keyName;
21
22     @ManyToOne
23     @JoinColumn(name = "employee_id")
24     private Employee employee;
25
26     @ManyToOne
27     @JoinColumn(name = "vendor_id")
28     private Vendor vendor;
29
30     @Column(nullable = false)
31     private String permissions; // Comma-separated permissions
32
33     @Column(nullable = false)
34     private Instant createdAt;
35
36     private Instant expiresAt;
37
38     private Instant lastUsedAt;
39
40     @Column(nullable = false)
41     private boolean active = true;
42
43     @PrePersist
44     public void prePersist() {
45         this.apiKey = "ems_" + UUID.randomUUID().toString().replace("-", "");
46         this.createdAt = Instant.now();
47     }
48
49     // Getters and setters
50     public Long getId() { return id; }
51     public void setId(Long id) { this.id = id; }
52
53     public String getApiKey() { return apiKey; }
54     public void setApiKey(String apiKey) { this.apiKey = apiKey; }
55
56     public String getKeyName() { return keyName; }
57     public void setKeyName(String keyName) { this.keyName = keyName; }
58
59     public Employee getEmployee() { return employee; }
60     public void setEmployee(Employee employee) { this.employee = employee; }
61
62     public Vendor getVendor() { return vendor; }
63     public void setVendor(Vendor vendor) { this.vendor = vendor; }
64
65     public String getPermissions() { return permissions; }
66     public void setPermissions(String permissions) { this.permissions = permissions; }
67
68     public Instant getCreatedAt() { return createdAt; }
69     public void setCreatedAt(Instant createdAt) { this.createdAt = createdAt; }
70
71     public Instant getExpiresAt() { return expiresAt; }
```

```
72     public void setExpiresAt(Instant expiresAt) { this.expiresAt = expiresAt; }
73
74     public Instant getLastUsedAt() { return lastUsedAt; }
75     public void setLastUsedAt(Instant lastUsedAt) { this.lastUsedAt = lastUsedAt; }
76
77     public boolean isActive() { return active; }
78     public void setActive(boolean active) { this.active = active; }
79 }
```



```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import com.example.EmployeeManagementSystem.Enum.DeliveryStatus;
4 import com.example.EmployeeManagementSystem.Enum.MealSlot;
5 import jakarta.persistence.*;
6
7 import java.time.Instant;
8
9 @Entity
10 @Table(name = "delivery", uniqueConstraints = { @UniqueConstraint(name = "uq_subscription_scheduled_time",
11     columnNames = {"subscription_id", "scheduledDeliveryTime"})})
12 public class Delivery {
13     @Id
14     @GeneratedValue(strategy = GenerationType.IDENTITY)
15     private Long id;
16
17     @ManyToOne
18     @JoinColumn(name = "subscription_id")
19     private Subscription subscription;
20     private MealSlot mealSlot;
21     private Instant scheduledDeliveryTime;
22     private Instant actualDeliveryTime;
23
24     @Enumerated(EnumType.STRING)
25     private DeliveryStatus status;
26
27     private Instant createdAt;
28
29     @PrePersist
30     public void prePersist() {
31         createdAt = Instant.now();
32         if (status == null) {
33             status = DeliveryStatus.SCHEDULED;
34         }
35     }
36
37     public MealSlot getMealSlot() {
38         return mealSlot;
39     }
40
41     public void setMealSlot(MealSlot mealSlot) {
42         this.mealSlot = mealSlot;
43     }
44
45     public Subscription getSubscription() {
46         return subscription;
47     }
48
49     public void setSubscription(Subscription subscription) {
50         this.subscription = subscription;
51     }
52
53     public Long getId() {
54         return id;
55     }
56
57     public void setId(Long id) {
58         this.id = id;
59     }
60
61     public Instant getScheduledDeliveryTime() {
62         return scheduledDeliveryTime;
63     }
64
65     public void setScheduledDeliveryTime(Instant scheduledDeliveryTime) {
66         this.scheduledDeliveryTime = scheduledDeliveryTime;
67     }
68
69     public Instant getActualDeliveryTime() {
70         return actualDeliveryTime;
71     }
```

```
72
73     public void setActualDeliveryTime(Instant actualDeliveryTime) {
74         this.actualDeliveryTime = actualDeliveryTime;
75     }
76
77     public DeliveryStatus getStatus() {
78         return status;
79     }
80
81     public void setStatus(DeliveryStatus status) {
82         this.status = status;
83     }
84
85     public Instant getCreatedAt() {
86         return createdAt;
87     }
88
89     public void setCreatedAt(Instant createdAt) {
90         this.createdAt = createdAt;
91     }
92 }
93
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3
4 import com.example.EmployeeManagementSystem.Enum.DeviceStatus;
5 import com.example.EmployeeManagementSystem.Enum.DeviceType;
6 import jakarta.persistence.*;
7
8 import java.time.LocalDate;
9 import java.time.LocalDateTime;
10
11 @Entity
12 public class Device {
13     @Id
14     @GeneratedValue(strategy = GenerationType.AUTO)
15     private Long id;
16
17     private String deviceName;        // e.g. "Dell XPS 15"
18     private String brand;
19
20     @ManyToOne
21     @JoinColumn(name = "vendor_id")
22     private Vendor techVendor;
23
24     @OneToOne(mappedBy = "device")
25     private DeviceAssignment currentAssignment;
26
27     private DeviceStatus deviceStatus;
28
29     private DeviceType deviceType;
30
31     private LocalDateTime createdAt;
32
33     private LocalDate warrantyExpiryDate;
34
35     public Long getId() {
36         return id;
37     }
38
39     public void setId(Long id) {
40         this.id = id;
41     }
42
43     public String getDeviceName() {
44         return deviceName;
45     }
46
47     public void setDeviceName(String deviceName) {
48         this.deviceName = deviceName;
49     }
50
51     public String getBrand() {
52         return brand;
53     }
54
55     public void setBrand(String brand) {
56         this.brand = brand;
57     }
58
59     public Vendor getTechVendor() {
60         return techVendor;
61     }
62
63     public void setTechVendor(Vendor techVendor) {
64         this.techVendor = techVendor;
65     }
66
67     public DeviceType getDeviceType() {
68         return deviceType;
69     }
70
71     public void setDeviceType(DeviceType deviceType) {
```

```
72         this.deviceType = deviceType;
73     }
74
75     public LocalDateTime getCreatedAt() {
76         return createdAt;
77     }
78
79     public void setCreatedAt(LocalDateTime createdAt) {
80         this.createdAt = createdAt;
81     }
82
83     public DeviceStatus getDeviceStatus() {
84         return deviceStatus;
85     }
86
87     public void setDeviceStatus(DeviceStatus deviceStatus) {
88         this.deviceStatus = deviceStatus;
89     }
90
91     public DeviceAssignment getCurrentAssignment() {
92         return currentAssignment;
93     }
94
95     public void setCurrentAssignment(DeviceAssignment currentAssignment) {
96         this.currentAssignment = currentAssignment;
97     }
98
99     public LocalDate getWarrantyExpiryDate() {
100         return warrantyExpiryDate;
101     }
102
103     public void setWarrantyExpiryDate(LocalDate warrantyExpiryDate) {
104         this.warrantyExpiryDate = warrantyExpiryDate;
105     }
106 }
107
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import com.example.EmployeeManagementSystem.Enum.AssignmentStatus;
4 import jakarta.persistence.*;
5
6 import java.time.LocalDate;
7
8 @Entity
9 public class DeviceAssignment {
10
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private Long id;
14
15     @OneToOne
16     @JoinColumn(name = "device_id")
17     private Device device;
18
19     @ManyToOne
20     @JoinColumn(name = "employee_id")
21     private Employee assignedTo;
22
23     @Enumerated(EnumType.STRING)
24     private AssignmentStatus status;
25
26     private LocalDate assignedDate;
27
28     public Long getId() {
29         return id;
30     }
31
32     public void setId(Long id) {
33         this.id = id;
34     }
35
36     public Device getDevice() {
37         return device;
38     }
39
40     public void setDevice(Device device) {
41         this.device = device;
42     }
43
44     public Employee getAssignedTo() {
45         return assignedTo;
46     }
47
48     public void setAssignedTo(Employee assignedTo) {
49         this.assignedTo = assignedTo;
50     }
51
52     public LocalDate getAssignedDate() {
53         return assignedDate;
54     }
55
56     public void setAssignedDate(LocalDate assignedDate) {
57         this.assignedDate = assignedDate;
58     }
59
60     public AssignmentStatus getStatus() {
61         return status;
62     }
63
64     public void setStatus(AssignmentStatus status) {
65         this.status = status;
66     }
67 }
68
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import com.example.EmployeeManagementSystem.Enum.Gender;
4 import com.example.EmployeeManagementSystem.Enum.Role;
5 import com.example.EmployeeManagementSystem.Enum.Status;
6 import com.fasterxml.jackson.annotation.JsonIgnore;
7 import jakarta.persistence.*;
8 import org.springframework.security.core.GrantedAuthority;
9 import org.springframework.security.core.authority.SimpleGrantedAuthority;
10 import org.springframework.security.core.userdetails.UserDetails;
11
12 import java.time.LocalDate;
13 import java.util.Collection;
14 import java.util.HashSet;
15 import java.util.List;
16 import java.util.Set;
17 import java.util.stream.Collectors;
18
19 @Entity
20 public class Employee implements UserDetails {
21     @Id
22     @GeneratedValue(strategy = GenerationType.IDENTITY)
23     private long employeeId;
24     @Column(nullable = true, unique = false)
25     private String name;
26     @Column(unique = true, nullable = false)
27     private String email;
28     @Column(nullable = true)
29     private String dept;
30     @Enumerated(EnumType.STRING)
31     private Status status;
32     private String password;
33
34     // Self-referential – replaces `String manager`
35     @ManyToOne(fetch = FetchType.LAZY)
36     @JoinColumn(name = "manager_id", nullable = true)
37     @JsonIgnore // prevent infinite recursion in serialization
38     private Employee manager;
39
40
41     private LocalDate joined_at;
42     @Enumerated(EnumType.STRING)
43     private Role role;
44     @Column(nullable = true)
45     private String totpSecret;
46     private boolean totpEnabled = false;
47     /** IANA timezone id, e.g. "Asia/Kolkata", "America/New_York". Defaults to UTC. */
48     @Column(nullable = false)
49     private String timezone = "UTC";
50     @Enumerated(EnumType.STRING)
51     @Column(nullable = true)
52     private Gender gender;
53
54     public String getTimezone() {
55         return timezone;
56     }
57
58     public void setTimezone(String timezone) {
59         this.timezone = timezone;
60     }
61
62     public Role getRole() {
63         return role;
64     }
65
66     public void setRole(Role role) {
67         this.role = role;
68     }
69
70     public long getEmployeeId() {
71         return employeeId;
```

```

72     }
73
74     public void setEmployeeId(long employeeId) {
75         this.employeeId = employeeId;
76     }
77
78     public String getName() {
79         return name;
80     }
81
82     public void setName(String name) {
83         this.name = name;
84     }
85
86     public String getEmail() {
87         return email;
88     }
89
90     public void setEmail(String email) {
91         this.email = email;
92     }
93
94     public String getDept() {
95         return dept;
96     }
97
98     public void setDept(String dept) {
99         this.dept = dept;
100    }
101
102    public Status getStatus() {
103        return status;
104    }
105
106    public void setStatus(Status status) {
107        this.status = status;
108    }
109
110    public LocalDate getJoined_at() {
111        return joined_at;
112    }
113
114    public void setJoined_at(LocalDate joined_at) {
115        this.joined_at = joined_at;
116    }
117
118    public Gender getGender() { return gender; }
119    public void setGender(Gender gender) { this.gender = gender; }
120
121    @PrePersist
122    public void initialSetup() {
123        this.status = Status.ACTIVE;
124        this.joined_at = LocalDate.now();
125        if (this.timezone == null || this.timezone.isBlank()) {
126            this.timezone = "UTC";
127        }
128    }
129
130    @Override
131    public Collection<? extends GrantedAuthority> getAuthorities() {
132        Set<SimpleGrantedAuthority> authorities=new HashSet<>();
133        authorities.add(new SimpleGrantedAuthority("ROLE_"+role.name()));
134        authorities.addAll(role.getPermissions().stream()
135            .map(permissions -> new SimpleGrantedAuthority(permissions.name()))
136            .collect(Collectors.toSet()));
137        return authorities;
138    }
139
140    public String getPassword() {
141        return password;
142    }
143
144    @Override
145    public String getUsername() {
146        return email;
147    }

```

```
148
149     @Override
150     public boolean isAccountNonExpired() {
151         return UserDetails.super.isAccountNonExpired();
152     }
153
154     @Override
155     public boolean isAccountNonLocked() {
156         return UserDetails.super.isAccountNonLocked();
157     }
158
159     @Override
160     public boolean isCredentialsNonExpired() {
161         return UserDetails.super.isCredentialsNonExpired();
162     }
163
164     @Override
165     public boolean isEnabled() {
166         return UserDetails.super.isEnabled();
167     }
168
169     public void setPassword(String password) {
170         this.password = password;
171     }
172
173     public String getTotpSecret() {
174         return totpSecret;
175     }
176
177     public boolean isTotpEnabled() {
178         return totpEnabled;
179     }
180
181     public void setTotpSecret(String totpSecret) {
182         this.totpSecret = totpSecret;
183     }
184
185     public void setTotpEnabled(boolean totpEnabled) {
186         this.totpEnabled = totpEnabled;
187     }
188
189     public Employee getManager() {
190         return manager;
191     }
192
193     public void setManager(Employee manager) {
194         this.manager = manager;
195     }
196
197 }
```



```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import jakarta.persistence.*;
4
5 import java.time.LocalDate;
6
7 @Entity
8 public class FixedHoliday {
9
10
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private Long id;
14
15     private String name;          // "Diwali", "Christmas"
16     private LocalDate date;       // 2026-10-20
17     private String description;   // optional note
18     private int year;
19
20
21     public Long getId() {
22         return id;
23     }
24
25     public void setId(Long id) {
26         this.id = id;
27     }
28
29     public String getName() {
30         return name;
31     }
32
33     public void setName(String name) {
34         this.name = name;
35     }
36
37     public LocalDate getDate() {
38         return date;
39     }
40
41     public void setDate(LocalDate date) {
42         this.date = date;
43     }
44
45     public int getYear() {
46         return year;
47     }
48
49     public void setYear(int year) {
50         this.year = year;
51     }
52
53     public String getDescription() {
54         return description;
55     }
56
57     public void setDescription(String description) {
58         this.description = description;
59     }
60 }
61
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import jakarta.persistence.*;
4
5 import java.math.BigDecimal;
6
7 @Entity
8 @Table(name = "leave_entitlement", uniqueConstraints = @UniqueConstraint(columnNames = {"employee_id",
9 "leave_type_id", "year"}))
10 public class LeaveEntitlement {
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private Long id;
14
15     @ManyToOne(fetch = FetchType.LAZY, optional = false)
16     @JoinColumn(name = "employee_id", nullable = false)
17     private Employee employee;
18
19     @ManyToOne(fetch = FetchType.LAZY, optional = false)
20     @JoinColumn(name = "leave_type_id", nullable = false)
21     private LeaveType leaveType;
22
23     @Column(nullable = false)
24     private Integer year;
25
26     @Column(nullable = false, precision = 5, scale = 2)
27     private BigDecimal openingBalance = BigDecimal.ZERO;
28
29     @Column(nullable = false, precision = 5, scale = 2)
30     private BigDecimal accruedThisYear = BigDecimal.ZERO;
31
32     @Column(nullable = false, precision = 5, scale = 2)
33     private BigDecimal usedThisYear = BigDecimal.ZERO;
34
35     @Column(nullable = false, precision = 5, scale = 2)
36     private BigDecimal closingBalance = BigDecimal.ZERO;
37
38     // Convenience: current available = opening + accrued - used
39     @Transient
40     public BigDecimal getAvailableBalance() {
41         return openingBalance.add(accruedThisYear).subtract(usedThisYear);
42     }
43
44     // --- getters & setters ---
45     public Long getId() {
46         return id;
47     }
48
49     public Employee getEmployee() {
50         return employee;
51     }
52
53     public LeaveType getLeaveType() {
54         return leaveType;
55     }
56
57     public Integer getYear() {
58         return year;
59     }
60
61     public BigDecimal getOpeningBalance() {
62         return openingBalance;
63     }
64
65     public BigDecimal getAccruedThisYear() {
66         return accruedThisYear;
67     }
68
69     public BigDecimal getUsedThisYear() {
70         return usedThisYear;
```

```
71     }
72
73     public BigDecimal getClosingBalance() {
74         return closingBalance;
75     }
76
77     public void setEmployee(Employee e) {
78         this.employee = e;
79     }
80
81     public void setLeaveType(LeaveType lt) {
82         this.leaveType = lt;
83     }
84
85     public void setYear(Integer y) {
86         this.year = y;
87     }
88
89     public void setOpeningBalance(BigDecimal b) {
90         this.openingBalance = b;
91     }
92
93     public void setAccruedThisYear(BigDecimal a) {
94         this.accruedThisYear = a;
95     }
96
97     public void setUsedThisYear(BigDecimal u) {
98         this.usedThisYear = u;
99     }
100
101     public void setClosingBalance(BigDecimal c) {
102         this.closingBalance = c;
103     }
104 }
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
4 import com.example.EmployeeManagementSystem.Enum.LeaveType;
5 import jakarta.persistence.*;
6
7 import java.time.LocalDate;
8
9 @Entity
10 public class LeaveRequest {
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private long id;
14     @Enumerated(EnumType.STRING)
15     private LeaveType leaveType;
16     private LocalDate startDate;
17     private LocalDate endDate;
18     private String reason;
19     @Enumerated(EnumType.STRING)
20     private LeaveStatus status;
21     private String Manager;
22     private String remarks;
23     @ManyToOne
24     @JoinColumn(name = "employee_id")
25     private Employee employee;
26
27
28     public long getId() {
29         return id;
30     }
31
32     public void setId(long id) {
33         this.id = id;
34     }
35
36     public LeaveType getLeaveType() {
37         return leaveType;
38     }
39
40     public void setLeaveType(LeaveType leaveType) {
41         this.leaveType = leaveType;
42     }
43
44     public LocalDate getStartDate() {
45         return startDate;
46     }
47
48     public void setStartDate(LocalDate startDate) {
49         this.startDate = startDate;
50     }
51
52     public LocalDate getEndDate() {
53         return endDate;
54     }
55
56     public void setEndDate(LocalDate endDate) {
57         this.endDate = endDate;
58     }
59
60     public LeaveStatus getStatus() {
61         return status;
62     }
63
64     public void setStatus(LeaveStatus status) {
65         this.status = status;
66     }
67
68     public String getManager() {
69         return Manager;
70     }
71 }
```

```
72     public void setManager(String manager) {
73         Manager = manager;
74     }
75
76     public String getRemarks() {
77         return remarks;
78     }
79
80     public void setRemarks(String remarks) {
81         this.remarks = remarks;
82     }
83
84     public Employee getEmployee() {
85         return employee;
86     }
87
88     public void setEmployee(Employee employee) {
89         this.employee = employee;
90     }
91
92     public String getReason() {
93         return reason;
94     }
95
96     public void setReason(String reason) {
97         this.reason = reason;
98     }
99
100     @PrePersist
101     public void init(){
102         this.status=LeaveStatus.PENDING;
103     }
104 }
105
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import com.example.EmployeeManagementSystem.Enum.Gender;
4 import jakarta.persistence.*;
5
6 @Entity
7 @Table(name = "leave_type")
8 public class LeaveType {
9
10     @Id
11     @GeneratedValue(strategy = GenerationType.IDENTITY)
12     private Integer id;
13
14     @Column(nullable = false, unique = true, length = 50)
15     private String name;           // SICK, CASUAL, MATERNITY
16
17     @Column(nullable = false)
18     private boolean carriesForward;
19
20     @Column(nullable = false)
21     private boolean monthlyReset;   // true = sick leave wipes at month end
22
23     @Column(nullable = false)
24     private int maxCarryForward;    // 30 for carry-forwardable types
25
26     @Column(nullable = false)
27     private boolean genderRestricted;
28
29     @Enumerated(EnumType.STRING)
30     @Column(nullable = true)
31     private Gender genderRestriction; // null = open to all
32
33     // --- getters & setters ---
34     public Integer getId() { return id; }
35     public String getName() { return name; }
36     public boolean isCarriesForward() { return carriesForward; }
37     public boolean isMonthlyReset() { return monthlyReset; }
38     public int getMaxCarryForward() { return maxCarryForward; }
39     public boolean isGenderRestricted() { return genderRestricted; }
40     public Gender getGenderRestriction() { return genderRestriction; }
41
42     public void setId(Integer id) { this.id = id; }
43     public void setName(String name) { this.name = name; }
44     public void setCarriesForward(boolean cf) { this.carriesForward = cf; }
45     public void setMonthlyReset(boolean mr) { this.monthlyReset = mr; }
46     public void setMaxCarryForward(int max) { this.maxCarryForward = max; }
47     public void setGenderRestricted(boolean gr) { this.genderRestricted = gr; }
48     public void setGenderRestriction(Gender g) { this.genderRestriction = g; }
49 }
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3
4 import jakarta.persistence.*;
5 import java.math.BigDecimal;
6 import java.time.LocalDate;
7
8 @Entity
9 @Table(name = "leave_warning")
10 public class LeaveWarning {
11
12     public enum WarningType {
13         APPROACHING_28, // 2 months before cap
14         APPROACHING_29, // 1 month before cap
15         CAP_REACHED,    // exactly at 30
16         DAY_WASTED      // already at 30, day lost
17     }
18
19     @Id
20     @GeneratedValue(strategy = GenerationType.IDENTITY)
21     private Long id;
22
23     @ManyToOne(fetch = FetchType.LAZY, optional = false)
24     @JoinColumn(name = "employee_id", nullable = false)
25     private Employee employee;
26
27     @Enumerated(EnumType.STRING)
28     @Column(nullable = false, length = 30)
29     private WarningType warningType;
30
31     @Column(nullable = false, columnDefinition = "TEXT")
32     private String message;
33
34     @Column(nullable = false, precision = 5, scale = 2)
35     private BigDecimal carryForwardBalance;
36
37     @Column(nullable = false)
38     private LocalDate warningDate;
39
40     @Column(nullable = false)
41     private boolean isRead = false;
42
43     // --- getters & setters ---
44     public Long getId() { return id; }
45     public Employee getEmployee() { return employee; }
46     public WarningType getWarningType() { return warningType; }
47     public String getMessage() { return message; }
48     public BigDecimal getCarryForwardBalance() { return carryForwardBalance; }
49     public LocalDate getWarningDate() { return warningDate; }
50     public boolean isRead() { return isRead; }
51
52     public void setEmployee(Employee e) { this.employee = e; }
53     public void setWarningType(WarningType wt) { this.warningType = wt; }
54     public void setMessage(String m) { this.message = m; }
55     public void setCarryForwardBalance(BigDecimal b) { this.carryForwardBalance = b; }
56     public void setWarningDate(LocalDate d) { this.warningDate = d; }
57     public void setRead(boolean r) { this.isRead = r; }
58 }
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import jakarta.persistence.*;
4 import java.time.LocalDateTime;
5
6 @Entity
7 @Table(name = "notification")
8 public class Notification {
9
10     @Id
11     @GeneratedValue(strategy = GenerationType.IDENTITY)
12     private Long id;
13
14     @ManyToOne(fetch = FetchType.LAZY)
15     @JoinColumn(name = "employee_id", nullable = false)
16     private Employee recipient;
17
18     @Column(nullable = false)
19     private String message;
20
21     @Column(nullable = false)
22     private String type; // LEAVE_APPROVED, LEAVE_REJECTED, LEAVE_REQUEST, HOLIDAY_ADDED
23
24     @Column(nullable = false)
25     private boolean isRead = false;
26
27     @Column(nullable = false)
28     private LocalDateTime createdAt = LocalDateTime.now();
29
30     // getters and setters
31     public Long getId() { return id; }
32     public Employee getRecipient() { return recipient; }
33     public void setRecipient(Employee recipient) { this.recipient = recipient; }
34     public String getMessage() { return message; }
35     public void setMessage(String message) { this.message = message; }
36     public String getType() { return type; }
37     public void setType(String type) { this.type = type; }
38     public boolean isRead() { return isRead; }
39     public void setRead(boolean read) { isRead = read; }
40     public LocalDateTime getCreatedAt() { return createdAt; }
41 }
42
```



```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import jakarta.persistence.*;
4
5 import java.time.Instant;
6 @Entity
7 public class RefreshToken {
8     @Id
9     @GeneratedValue(strategy = GenerationType.IDENTITY)
10    private Long id;
11
12    @Column(nullable = false, unique = true)
13    private String token;
14
15    @Column(nullable = false)
16    private Instant expiryTime;
17
18    @ManyToOne
19    @JoinColumn(name = "employee_id")
20    private Employee employee;
21
22    @ManyToOne
23    @JoinColumn(name = "vendor_id")
24    private Vendor vendor;
25
26    public Long getId() {
27        return id;
28    }
29
30    public String getToken() {
31        return token;
32    }
33
34    public void setToken(String token) {
35        this.token = token;
36    }
37
38    public Instant getExpiryTime() {
39        return expiryTime;
40    }
41
42    public void setExpiryTime(Instant expiryTime) {
43        this.expiryTime = expiryTime;
44    }
45
46    public Employee getEmployee() {
47        return employee;
48    }
49
50    public void setEmployee(Employee employee) {
51        this.employee = employee;
52    }
53
54    public Vendor getVendor() {
55        return vendor;
56    }
57
58    public void setVendor(Vendor vendor) {
59        this.vendor = vendor;
60    }
61 }
62
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import com.example.EmployeeManagementSystem.Enum.MealSlot;
4 import jakarta.persistence.*;
5 import java.util.Set;
6
7
8 @Entity
9 public class Restaurant {
10
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private Long id;
14
15     @Column(nullable = false)
16     private String name;
17
18     @Column(nullable = false)
19     private String address;
20
21     private String description;
22
23     @ElementCollection(targetClass = MealSlot.class, fetch = FetchType.EAGER)
24     @CollectionTable(name = "restaurant_meal_slots",
25         joinColumns = @JoinColumn(name = "restaurant_id"))
26     @Enumerated(EnumType.STRING)
27     @Column(name = "meal_slot")
28     private Set<MealSlot> supportedMealSlots;
29
30     private boolean active = true;
31
32     @ManyToOne(fetch = FetchType.LAZY)
33     @JoinColumn(name = "vendor_id", nullable = false)
34     private Vendor vendor;
35
36     public Long getId() { return id; }
37     public void setId(Long id) { this.id = id; }
38
39     public String getName() { return name; }
40     public void setName(String name) { this.name = name; }
41
42     public String getAddress() { return address; }
43     public void setAddress(String address) { this.address = address; }
44
45     public String getDescription() { return description; }
46     public void setDescription(String description) { this.description = description; }
47
48     public Set<MealSlot> getSupportedMealSlots() { return supportedMealSlots; }
49     public void setSupportedMealSlots(Set<MealSlot> supportedMealSlots) {
50         this.supportedMealSlots = supportedMealSlots;
51     }
52
53     public boolean isActive() { return active; }
54     public void setActive(boolean active) { this.active = active; }
55
56     public Vendor getVendor() { return vendor; }
57     public void setVendor(Vendor vendor) { this.vendor = vendor; }
58 }
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import com.example.EmployeeManagementSystem.Enum.ServiceRequestStatus;
4 import com.example.EmployeeManagementSystem.Enum.ServiceRequestType;
5 import jakarta.persistence.*;
6
7 import java.time.LocalDateTime;
8
9 @Entity
10 public class ServiceRequest {
11     @Id
12     @GeneratedValue(strategy = GenerationType.IDENTITY)
13     private Long id;
14
15     @ManyToOne
16     @JoinColumn(name = "device_id")
17     private Device device;
18
19     @ManyToOne
20     @JoinColumn(name = "raised_by")
21     private Employee raisedBy; // employee reporting issue
22
23     @ManyToOne
24     @JoinColumn(name = "reviewed_by")
25     private Employee reviewedBy; // admin who reviewed
26
27     @Enumerated(EnumType.STRING)
28     private ServiceRequestType requestType;
29
30     @Enumerated(EnumType.STRING)
31     private ServiceRequestStatus status;
32
33     private String issueDescription; // "screen cracked", "won't boot"
34     private String adminRemarks; // admin notes on decision
35     private String resolution; // what was done finally
36
37     private boolean urgent; // employee can flag as urgent
38
39     private LocalDateTime raisedAt;
40     private LocalDateTime resolvedAt;
41
42     @PrePersist
43     void onCreate() { this.raisedAt = LocalDateTime.now(); }
44
45     public Employee getRaisedBy() {
46         return raisedBy;
47     }
48
49     public void setRaisedBy(Employee raisedBy) {
50         this.raisedBy = raisedBy;
51     }
52
53     public Device getDevice() {
54         return device;
55     }
56
57     public void setDevice(Device device) {
58         this.device = device;
59     }
60
61     public Long getId() {
62         return id;
63     }
64
65     public void setId(Long id) {
66         this.id = id;
67     }
68
69     public Employee getReviewedBy() {
70         return reviewedBy;
71     }
```

```
72
73     public void setReviewedBy(Employee reviewedBy) {
74         this.reviewedBy = reviewedBy;
75     }
76
77     public ServiceRequestStatus getStatus() {
78         return status;
79     }
80
81     public void setStatus(ServiceRequestStatus status) {
82         this.status = status;
83     }
84
85     public ServiceRequestType getRequestType() {
86         return requestType;
87     }
88
89     public void setRequestType(ServiceRequestType requestType) {
90         this.requestType = requestType;
91     }
92
93     public String getIssueDescription() {
94         return issueDescription;
95     }
96
97     public void setIssueDescription(String issueDescription) {
98         this.issueDescription = issueDescription;
99     }
100
101     public String getAdminRemarks() {
102         return adminRemarks;
103     }
104
105     public void setAdminRemarks(String adminRemarks) {
106         this.adminRemarks = adminRemarks;
107     }
108
109     public String getResolution() {
110         return resolution;
111     }
112
113     public void setResolution(String resolution) {
114         this.resolution = resolution;
115     }
116
117     public boolean isUrgent() {
118         return urgent;
119     }
120
121     public void setUrgent(boolean urgent) {
122         this.urgent = urgent;
123     }
124
125     public LocalDateTime getRaisedAt() {
126         return raisedAt;
127     }
128
129     public void setRaisedAt(LocalDateTime raisedAt) {
130         this.raisedAt = raisedAt;
131     }
132
133     public LocalDateTime getResolvedAt() {
134         return resolvedAt;
135     }
136
137     public void setResolvedAt(LocalDateTime resolvedAt) {
138         this.resolvedAt = resolvedAt;
139     }
140 }
141
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import com.example.EmployeeManagementSystem.Enum.MealSlot;
4 import com.example.EmployeeManagementSystem.Enum.ScheduleType;
5 import com.example.EmployeeManagementSystem.Enum.SubscriptionStatus;
6 import jakarta.persistence.*;
7
8 import java.time.DayOfWeek;
9 import java.time.Instant;
10 import java.time.LocalDate;
11 import java.time.LocalTime;
12 @Entity
13 public class Subscription {
14     @Id
15     @GeneratedValue(strategy = GenerationType.IDENTITY)
16     private long id;
17     @Enumerated(EnumType.STRING)
18     private MealSlot slot;
19     private ScheduleType scheduleType;
20     private DayOfWeek dayOfWeek;
21     @Enumerated(EnumType.STRING)
22     private SubscriptionStatus status;
23     private Instant nextDeliveryTime;
24     private LocalTime created_at;
25
26     @ManyToOne
27     @JoinColumn(name="employee_id")
28     private Employee employee;
29
30     @ManyToOne(fetch = FetchType.EAGER)
31     @JoinColumn(name = "restaurant_id", nullable = false)
32     private Restaurant restaurant;
33
34     @PrePersist
35     public void init(){
36         this.created_at=LocalTime.now();
37     }
38
39     public long getId() {
40         return id;
41     }
42
43     public void setId(long id) {
44         this.id = id;
45     }
46
47     public MealSlot getSlot() {
48         return slot;
49     }
50
51     public void setSlot(MealSlot slot) {
52         this.slot = slot;
53     }
54
55     public ScheduleType getScheduleType() {
56         return scheduleType;
57     }
58
59     public void setScheduleType(ScheduleType scheduleType) {
60         this.scheduleType = scheduleType;
61     }
62
63     public DayOfWeek getDayOfWeek() {
64         return dayOfWeek;
65     }
66
67     public void setDayOfWeek(DayOfWeek dayOfWeek) {
68         this.dayOfWeek = dayOfWeek;
69     }
70
71     public SubscriptionStatus getStatus() {
```

```
72         return status;
73     }
74
75     public void setStatus(SubscriptionStatus status) {
76         this.status = status;
77     }
78
79     public LocalTime getCreated_at() {
80         return created_at;
81     }
82
83     public void setCreated_at(LocalTime created_at) {
84         this.created_at = created_at;
85     }
86
87     public Instant getNextDeliveryTime() {
88         return nextDeliveryTime;
89     }
90
91     public void setNextDeliveryTime(Instant nextDeliveryTime) {
92         this.nextDeliveryTime = nextDeliveryTime;
93     }
94
95     public Employee getEmployee() {
96         return employee;
97     }
98
99     public void setEmployee(Employee employee) {
100         this.employee = employee;
101     }
102     public Restaurant getRestaurant() { return restaurant; }
103     public void setRestaurant(Restaurant restaurant) { this.restaurant = restaurant; }
104 }
105
```

```
1 package com.example.EmployeeManagementSystem.Entity;
2
3 import com.example.EmployeeManagementSystem.Enum.Role;
4 import jakarta.persistence.*;
5 import jakarta.xml.bind.annotation.XmlAttribute;
6 import org.jspecify.annotations.Nullable;
7 import org.springframework.security.core.GrantedAuthority;
8 import org.springframework.security.core.authority.SimpleGrantedAuthority;
9 import org.springframework.security.core.userdetails.UserDetails;
10
11 import java.time.LocalDate;
12 import java.util.Collection;
13 import java.util.HashSet;
14 import java.util.List;
15 import java.util.Set;
16 import java.util.stream.Collectors;
17
18 /**
19  * Represents a restaurant vendor/owner.
20  * Vendors can register and manage their restaurants.
21  * They cannot access employee, leave, or subscription APIs.
22  */
23 @Entity
24 @Table(name = "vendor")
25 public class Vendor implements UserDetails {
26
27     @Id
28     @GeneratedValue(strategy = GenerationType.IDENTITY)
29     private Long id;
30
31     @Column(nullable = false)
32     private String name;
33
34     @Column(unique = true, nullable = false)
35     private String email;
36
37     @Column(nullable = false)
38     private String phone;
39
40     @Column(nullable = false)
41     private String password;
42
43     @Enumerated(EnumType.STRING)
44     private Role role;
45
46     private LocalDate registeredAt;
47
48     @PrePersist
49     public void init() {
50         if (this.registeredAt == null) {
51             this.registeredAt = LocalDate.now();
52         }
53     }
54
55     public Long getId() { return id; }
56     public void setId(Long id) { this.id = id; }
57
58     public String getName() { return name; }
59     public void setName(String name) { this.name = name; }
60
61     public String getEmail() { return email; }
62     public void setEmail(String email) { this.email = email; }
63
64     public String getPhone() { return phone; }
65     public void setPhone(String phone) { this.phone = phone; }
66
67     public LocalDate getRegisteredAt() { return registeredAt; }
68     public void setRegisteredAt(LocalDate registeredAt) { this.registeredAt = registeredAt; }
69
70
71     @Override
```

```
72     public Collection<? extends GrantedAuthority> getAuthorities() {
73         Role effectiveRole = role;
74         Set<SimpleGrantedAuthority> authorities=new HashSet<>();
75         authorities.add(new SimpleGrantedAuthority("ROLE_"+effectiveRole.name()));
76         authorities.addAll(effectiveRole.getPermissions().stream()
77             .map(permissions -> new SimpleGrantedAuthority(permissions.name()))
78             .collect(Collectors.toSet()));
79         return authorities;
80     }
81
82     @Override
83     public @Nullable String getPassword() {
84         return password;
85     }
86
87     @Override
88     public String getUsername() {
89         return email;
90     }
91
92     @Override
93     public boolean isAccountNonExpired() {
94         return UserDetails.super.isAccountNonExpired();
95     }
96
97     @Override
98     public boolean isAccountNonLocked() {
99         return UserDetails.super.isAccountNonLocked();
100    }
101
102    @Override
103    public boolean isCredentialsNonExpired() {
104        return UserDetails.super.isCredentialsNonExpired();
105    }
106
107    @Override
108    public boolean isEnabled() {
109        return UserDetails.super.isEnabled();
110    }
111
112    public void setPassword(String password) {
113        this.password = password;
114    }
115
116    public Role getRole() {
117        return role;
118    }
119
120    public void setRole(Role role) {
121        this.role = role;
122    }
123 }
124
```



```

1 package com.example.EmployeeManagementSystem.Entity;
2
3
4 import jakarta.persistence.*;
5 import java.math.BigDecimal;
6 import java.time.LocalDateTime;
7
8 @Entity
9 @Table(name = "year_end_carry_forward_log")
10 public class YearEndCarryForwardLog {
11
12     @Id
13     @GeneratedValue(strategy = GenerationType.IDENTITY)
14     private Long id;
15
16     @ManyToOne(fetch = FetchType.LAZY, optional = false)
17     @JoinColumn(name = "employee_id", nullable = false)
18     private Employee employee;
19
20     @Column(nullable = false)
21     private Integer processedYear;
22
23     @ManyToOne(fetch = FetchType.LAZY, optional = false)
24     @JoinColumn(name = "leave_type_id", nullable = false)
25     private LeaveType leaveType;
26
27     @Column(nullable = false, precision = 5, scale = 2)
28     private BigDecimal closingBalance; // raw closing before cap
29
30     @Column(nullable = false, precision = 5, scale = 2)
31     private BigDecimal cappedBalance; // after global 30-day cap applied
32
33     @Column(nullable = false, precision = 5, scale = 2)
34     private BigDecimal daysWasted = BigDecimal.ZERO;
35
36     @Column(nullable = false, precision = 5, scale = 2)
37     private BigDecimal openingNextYear; // written into next year's entitlement
38
39     @Column(nullable = false)
40     private LocalDateTime processedAt = LocalDateTime.now();
41
42     // --- getters & setters ---
43     public Long getId() { return id; }
44     public Employee getEmployee() { return employee; }
45     public Integer getProcessedYear() { return processedYear; }
46     public LeaveType getLeaveType() { return leaveType; }
47     public BigDecimal getClosingBalance() { return closingBalance; }
48     public BigDecimal getCappedBalance() { return cappedBalance; }
49     public BigDecimal getDaysWasted() { return daysWasted; }
50     public BigDecimal getOpeningNextYear() { return openingNextYear; }
51     public LocalDateTime getProcessedAt() { return processedAt; }
52
53     public void setEmployee(Employee e) { this.employee = e; }
54     public void setProcessedYear(Integer y) { this.processedYear = y; }
55     public void setLeaveType(LeaveType lt) { this.leaveType = lt; }
56     public void setClosingBalance(BigDecimal b) { this.closingBalance = b; }
57     public void setCappedBalance(BigDecimal b) { this.cappedBalance = b; }
58     public void setDaysWasted(BigDecimal d) { this.daysWasted = d; }
59     public void setOpeningNextYear(BigDecimal b) { this.openingNextYear = b; }
60     public void setProcessedAt(LocalDateTime dt) { this.processedAt = dt; }
61 }
62

```

82 Current:

src\main\java\com\example\EmployeeManagementSystem\Enum\AssignmentStatus.java

8
lines

```
1 package com.example.EmployeeManagementSystem.Enum;
```

```
2
```

```
3 public enum AssignmentStatus {
```

```
4     ACTIVE,
```

```
5     RETURNED,
```

```
6     REVOKED
```

```
7 }
```

```
8
```

83

Current:

src\main\java\com\example\EmployeeManagementSystem\Enum\DeliveryStatus.java

9

lines

```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 public enum DeliveryStatus {  
4     SCHEDULED,  
5     IN_PROGRESS,  
6     DELIVERED,  
7     MISSED  
8 }  
9
```

84

Current:

10

src\main\java\com\example\EmployeeManagementSystem\Enum\DeviceStatus.java

lines

```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 public enum DeviceStatus {  
4     AVAILABLE,  
5     ASSIGNED,  
6     UNDER_REPAIR,  
7     CONDEMNED,  
8     RETURNED_TO_VENDOR  
9 }  
10
```

```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 public enum DeviceType {  
4     LAPTOP,  
5     DESKTOP,  
6     PHONE,  
7     TABLET,  
8     MONITOR,  
9     KEYBOARD,  
10    MOUSE,  
11    HEADSET,  
12    OTHER  
13  
14 }  
15
```

86

Current:

5

src\main\java\com\example\EmployeeManagementSystem\Enum\Gender.java

lines

```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 public enum Gender {  
4     M, F, O  
5 }
```

87

Current:

9

src\main\java\com\example\EmployeeManagementSystem\Enum\LeaveStatus.java

lines

```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 public enum LeaveStatus {  
4     PENDING,  
5     APPROVED,  
6     REJECTED,  
7     CANCELLED  
8 }  
9
```

```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 public enum LeaveType {  
4     SICK,  
5     CASUAL,  
6     MATERNITY,  
7     UNPAID  
8 }  
9
```


89

Current:

8

src\main\java\com\example\EmployeeManagementSystem\Enum\MealSlot.java

lines

```
1 package com.example.EmployeeManagementSystem.Enum;
```

```
2
```

```
3 public enum MealSlot {
```

```
4     BREAKFAST,
```

```
5     LUNCH,
```

```
6     DINNER
```

```
7 }
```

```
8
```

```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 import java.util.Set;  
4  
5 public enum Permission {  
6     LEAVE_READ,  
7     LEAVE_WRITE,  
8     LEAVE_UPDATE,  
9     LEAVE_CANCEL,  
10    SUBSCRIPTION_WRITE,  
11    SUBSCRIPTION_READ,  
12    SUBSCRIPTION_UPDATE,  
13    VENDOR_UPDATE,  
14    VENDOR_DELETE,  
15    RESTAURANTS_ADD,  
16    RESTAURANTS_READ,  
17    RESTAURANTS_DELETE,  
18    PROFILE_READ,  
19    VENDOR_WRITE;  
20 }  
21
```

```
1 package com.example.EmployeeManagementSystem.Enum;
2
3 import java.util.Collection;
4 import java.util.Set;
5
6 import static com.example.EmployeeManagementSystem.Enum.Permission.*;
7
8
9 public enum Role {
10     EMPLOYEE(Set.of(LEAVE_WRITE,
11         LEAVE_CANCEL,
12         SUBSCRIPTION_WRITE,
13         SUBSCRIPTION_UPDATE,
14         RESTAURANTS_READ,
15         SUBSCRIPTION_READ,
16         PROFILE_READ)),
17     MANAGER(Set.of(LEAVE_UPDATE,
18         LEAVE_WRITE,
19         SUBSCRIPTION_WRITE,
20         RESTAURANTS_READ,
21         SUBSCRIPTION_UPDATE,
22         LEAVE_READ,
23         SUBSCRIPTION_READ,
24         PROFILE_READ)),
25     FOOD_VENDOR(Set.of(
26         VENDOR_WRITE,
27         VENDOR_DELETE,
28         RESTAURANTS_ADD,
29         RESTAURANTS_DELETE,
30         RESTAURANTS_READ,
31         PROFILE_READ
32     )),
33     ADMIN(Set.of(
34         LEAVE_READ,
35         LEAVE_WRITE,
36         LEAVE_UPDATE,
37         LEAVE_CANCEL,
38         RESTAURANTS_READ,
39         SUBSCRIPTION_READ,
40         SUBSCRIPTION_WRITE,
41         SUBSCRIPTION_UPDATE,
42         VENDOR_WRITE,
43         VENDOR_UPDATE,
44         VENDOR_DELETE,
45         PROFILE_READ
46     )),
47     TECH_VENDOR(Set.of(VENDOR_WRITE)),
48     USER(Set.of(PROFILE_READ));
49     private final Set<Permission> permissions;
50
51     Role(Set<Permission> permissions) {
52         this.permissions = permissions;
53     }
54
55     public Set<Permission> getPermissions() {
56         return permissions;
57     }
58 }
59
60
```

92

Current:

src\main\java\com\example\EmployeeManagementSystem\Enum\ScheduleType.java

7

lines

```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 public enum ScheduleType {  
4     DAILY,  
5     WEEKLY  
6 }  
7
```

```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 public enum ServiceRequestStatus {  
4     OPEN,  
5     SENT_FOR_REPAIR,  
6     REPAIR_DONE,  
7     REJECTED  
8 }  
9
```

```
1 package com.example.EmployeeManagementSystem.Enum;  
2  
3 public enum ServiceRequestType {  
4     REPAIR,  
5     REPLACEMENT,  
6     UPGRADE,  
7     MAINTENANCE,  
8     RETURN  
9 }  
10
```

95

Current:

8

src\main\java\com\example\EmployeeManagementSystem\Enum\Status.java

lines

```
1 package com.example.EmployeeManagementSystem.Enum;
```

```
2
```

```
3 public enum Status {
```

```
4     ACTIVE,
```

```
5     INACTIVE,
```

```
6     ON_LEAVE
```

```
7 }
```

```
8
```

96 Current:

src\main\java\com\example\EmployeeManagementSystem\Enum\SubscriptionStatus.java

8

lines

```
1 package com.example.EmployeeManagementSystem.Enum;
```

```
2
```

```
3 public enum SubscriptionStatus {
```

```
4     ACTIVE,
```

```
5     PAUSED,
```

```
6     EXPIRED
```

```
7 }
```

```
8
```


97 Current:

src\main\java\com\example\EmployeeManagementSystem\Exception\DuplicateRequestException.java

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 public class DuplicateRequestException extends RuntimeException {
4     public DuplicateRequestException(String message) {
5         super(message);
6     }
7 }
8
```

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 import org.springframework.security.core.userdetails.UsernameNotFoundException;
4
5 public class EmployeeNotFound extends UsernameNotFoundException {
6     public EmployeeNotFound(String message) {
7         super(message);
8     }
9 }
10
```

99 Current:

src\main\java\com\example\EmployeeManagementSystem\Exception\InvalidEndDateException.java

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 public class InvalidEndDateException extends RuntimeException{
4     public InvalidEndDateException(String message) {
5         super(message);
6     }
7 }
8
```

100 Current:

src\main\java\com\example\EmployeeManagementSystem\Exception\InvalidManagerException.java

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 public class InvalidManagerException extends RuntimeException{
4     public InvalidManagerException(String message) {
5         super(message);
6     }
7 }
8
```

101 Current:

src\main\java\com\example\EmployeeManagementSystem\Exception\InvalidStartDateException.java

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 public class InvalidStartDateException extends RuntimeException{
4     public InvalidStartDateException(String message) {
5         super(message);
6     }
7 }
8
```

102 Current:

src\main\java\com\example\EmployeeManagementSystem\Exception\LeaveRequestNotFoundExcep

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 public class LeaveRequestNotFoundException extends RuntimeException{
4     public LeaveRequestNotFoundException(String message) {
5         super(message);
6     }
7 }
8
```

103 Current:

src\main\java\com\example\EmployeeManagementSystem\Exception\OverlappingLeaveException.j

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 public class OverlappingLeaveException extends RuntimeException{
4     public OverlappingLeaveException(String message) {
5         super(message);
6     }
7 }
8
```

104 Current:

src\main\java\com\example\EmployeeManagementSystem\Exception\RestaurantNotFoundException

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 public class RestaurantNotFoundException extends RuntimeException {
4     public RestaurantNotFoundException(String message) {
5         super(message);
6     }
7 }
8
```


105 Current:

src\main\java\com\example\EmployeeManagementSystem\Exception\SubscriptionAlreadyExists.java

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 public class SubscriptionAlreadyExists extends RuntimeException{
4     public SubscriptionAlreadyExists(String message) {
5         super(message);
6     }
7 }
8
```

106 Current:

src\main\java\com\example\EmployeeManagementSystem\Exception\UnauthorizedAccessException

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 public class UnauthorizedAccessException extends RuntimeException {
4     public UnauthorizedAccessException(String message) {
5         super(message);
6     }
7 }
```

107 Current:

src\main\java\com\example\EmployeeManagementSystem\Exception\VendorNotFoundException.java

```
1 package com.example.EmployeeManagementSystem.Exception;
2
3 public class VendorNotFoundException extends RuntimeException {
4     public VendorNotFoundException(String message) {
5         super(message);
6     }
7 }
```

```

1 // src/main/java/com/example/EmployeeManagementSystem/Filter/ApiKeyFilter.java
2 package com.example.EmployeeManagementSystem.Filter;
3
4 import com.example.EmployeeManagementSystem.security.ApiKeyAuthenticationToken;
5 import jakarta.servlet.FilterChain;
6 import jakarta.servlet.ServletException;
7 import jakarta.servlet.http.HttpServletRequest;
8 import jakarta.servlet.http.HttpServletResponse;
9 import org.springframework.beans.factory.annotation.Autowired;
10 import org.springframework.beans.factory.annotation.Qualifier;
11 import org.springframework.security.authentication.AuthenticationManager;
12 import org.springframework.security.core.Authentication;
13 import org.springframework.security.core.context.SecurityContextHolder;
14 import org.springframework.stereotype.Component;
15 import org.springframework.web.filter.OncePerRequestFilter;
16
17 import java.io.IOException;
18 public class ApiKeyFilter extends OncePerRequestFilter {
19
20     private static final String API_KEY_HEADER = "X-API-Key";
21     private final AuthenticationManager authenticationManager;
22
23     public ApiKeyFilter(AuthenticationManager authenticationManager) {
24         this.authenticationManager = authenticationManager;
25     }
26
27     @Override
28     protected void doFilterInternal(HttpServletRequest request,
29                                     HttpServletResponse response,
30                                     FilterChain filterChain) throws ServletException, IOException {
31
32         String apiKey = request.getHeader(API_KEY_HEADER);
33
34         if (apiKey != null && !apiKey.isEmpty()) {
35             try {
36                 // Create the unauthenticated token
37                 ApiKeyAuthenticationToken authRequest = new ApiKeyAuthenticationToken(apiKey);
38                 // Run it through the provider – this hits the DB, checks expiry, loads roles
39                 Authentication authenticated = authenticationManager.authenticate(authRequest);
40                 // Only set it if validation passed
41                 SecurityContextHolder.getContext().setAuthentication(authenticated);
42             } catch (RuntimeException e) {
43                 // Invalid/expired key – clear context and return 401
44                 SecurityContextHolder.clearContext();
45                 response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
46                 response.setContentType("application/json");
47                 response.getWriter().write("{\"error\": \"" + e.getMessage() + "\"}");
48                 return;
49             }
50         }
51
52         filterChain.doFilter(request, response);
53     }
54
55     @Override
56     protected boolean shouldNotFilter(HttpServletRequest request) {
57         String path = request.getRequestURI();
58         return path.startsWith("/Authenticate") ||
59             path.startsWith("/session/login") ||
60             path.startsWith("/session/logout") ||
61             path.startsWith("/oauth2") ||
62             path.startsWith("/login/oauth2") ||
63             path.startsWith("/swagger-ui") ||
64             path.startsWith("/v3/api-docs") ||
65             path.startsWith("/api-keys/generate"); // Allow API key generation without API key
66     }
67 }

```

```
1 package com.example.EmployeeManagementSystem.Filter;
2
3 import com.example.EmployeeManagementSystem.Util.JWTUtil;
4 import io.jsonwebtoken.ExpiredJwtException;
5 import io.jsonwebtoken.MalformedJwtException;
6 import io.jsonwebtoken.UnsupportedJwtException;
7 import jakarta.servlet.FilterChain;
8 import jakarta.servlet.ServletException;
9 import jakarta.servlet.http.HttpServletRequest;
10 import jakarta.servlet.http.HttpServletResponse;
11 import org.springframework.beans.factory.annotation.Autowired;
12 import org.springframework.beans.factory.annotation.Qualifier;
13 import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
14 import org.springframework.security.core.context.SecurityContextHolder;
15 import org.springframework.security.core.userdetails.UserDetails;
16 import org.springframework.security.core.userdetails.UserDetailsService;
17 import org.springframework.stereotype.Component;
18 import org.springframework.web.filter.OncePerRequestFilter;
19
20 import java.io.IOException;
21
22 @Component
23 public class JwtAuthFilter extends OncePerRequestFilter {
24
25     @Autowired
26     @Qualifier("combinedUserDetailsService")
27     private UserDetailsService customUserDetailsService;
28
29     @Autowired
30     private JWTUtil jwtUtil;
31
32     @Override
33     protected void doFilterInternal(HttpServletRequest request,
34                                     HttpServletResponse response,
35                                     FilterChain filterChain) throws ServletException, IOException {
36         String token = null;
37         String username = null;
38
39         String authHeader = request.getHeader("Authorization");
40
41         if (authHeader != null && authHeader.startsWith("Bearer ")) {
42             token = authHeader.substring(7).trim();
43
44             // Reject obviously invalid values before hitting JWT
45             if (token.isEmpty() || token.equals("null") || token.equals("undefined")) {
46                 filterChain.doFilter(request, response);
47                 return;
48             }
49
50             try {
51                 username = jwtUtil.extractUsernameFromToken(token);
52             } catch (ExpiredJwtException e) {
53                 response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
54                 response.setContentType("application/json");
55                 response.getWriter().write("{\""
56                     {
57                         "error": "TOKEN_EXPIRED",
58                         "message": "JWT expired. Please login again to get a new token."
59                     }
60                     "\"");
61                 return;
62             } catch (MalformedJwtException | UnsupportedJwtException | IllegalArgumentException e) {
63                 // Token is garbage (e.g. "null" string, garbled value) –
64                 // don't crash. OAuth2 session auth will still work fine.
65                 logger.debug("JwtAuthFilter: skipping malformed token – " + e.getMessage());
66                 filterChain.doFilter(request, response);
67                 return;
68             }
69         }
70
71         if (username != null && SecurityContextHolder.getContext().getAuthentication() == null) {
```

```
72         UserDetails userDetails = customUserDetailsService.loadUserByUsername(username);
73         if (userDetails != null && jwtUtil.validateToken(username, userDetails, token)) {
74             UsernamePasswordAuthenticationToken authenticationToken =
75                 new UsernamePasswordAuthenticationToken(
76                     userDetails, null, userDetails.getAuthorities());
77             SecurityContextHolder.getContext().setAuthentication(authenticationToken);
78         }
79     }
80
81     filterChain.doFilter(request, response);
82 }
83 }
```

```
1 // src/main/java/com/example/EmployeeManagementSystem/Repository/ApiKeyRepository.java
2 package com.example.EmployeeManagementSystem.Repository;
3
4 import com.example.EmployeeManagementSystem.Entity.ApiKey;
5 import org.springframework.data.jpa.repository.JpaRepository;
6 import org.springframework.data.jpa.repository.Modifying;
7 import org.springframework.data.jpa.repository.Query;
8 import org.springframework.data.repository.query.Param;
9 import org.springframework.transaction.annotation.Transactional;
10
11 import java.time.Instant;
12 import java.util.List;
13 import java.util.Optional;
14
15 public interface ApiKeyRepository extends JpaRepository<ApiKey, Long> {
16
17     Optional<ApiKey> findByApiKeyAndActiveTrue(String apiKey);
18
19
20     List<ApiKey> findByEmployee_EmployeeIdAndActiveTrue(Long employeeId);
21
22
23     List<ApiKey> findByVendor_IdAndActiveTrue(Long vendorId);
24
25     @Modifying
26     @Transactional
27     @Query("UPDATE ApiKey a SET a.lastUsedAt = :now WHERE a.apiKey = :apiKey")
28     void updateLastUsed(@Param("apiKey") String apiKey, @Param("now") Instant now);
29
30     @Modifying
31     @Transactional
32     @Query("UPDATE ApiKey a SET a.active = false WHERE a.expiresAt < :now AND a.active = true")
33     int expireOldKeys(@Param("now") Instant now);
34 }
```

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.Delivery;
4 import com.example.EmployeeManagementSystem.Entity.Subscription;
5 import com.example.EmployeeManagementSystem.Enum.DeliveryStatus;
6 import org.springframework.data.jpa.repository.JpaRepository;
7 import org.springframework.data.jpa.repository.Query;
8 import org.springframework.data.repository.query.Param;
9
10 import java.time.Instant;
11 import java.util.List;
12
13 public interface DeliveryRepository extends JpaRepository<Delivery, Long> {
14     List<Delivery> findByStatusAndScheduledDeliveryTimeLessThanEqual(
15         DeliveryStatus status, Instant time);
16
17     @Query("SELECT d FROM Delivery d WHERE d.subscription = :subscription " +
18         "AND d.status != :status ORDER BY d.scheduledDeliveryTime DESC")
19     List<Delivery> findRecentDeliveriesBySubscription(
20         @Param("subscription") Subscription subscription,
21         @Param("status") DeliveryStatus status);
22 }
23
```


112 Current:

src\main\java\com\example\EmployeeManagementSystem\Repository\DeviceAssignmentRepo.java

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.DeviceAssignment;
4 import com.example.EmployeeManagementSystem.Enum.AssignmentStatus;
5 import org.springframework.data.jpa.repository.JpaRepository;
6 import org.springframework.data.jpa.repository.Query;
7 import org.springframework.data.repository.query.Param;
8
9 import java.util.List;
10 import java.util.Optional;
11
12 public interface DeviceAssignmentRepo extends JpaRepository<DeviceAssignment, Long> {
13     Optional<DeviceAssignment> findByDeviceIdAndStatus(Long deviceId, AssignmentStatus status);
14     List<DeviceAssignment> findByAssignedToEmployeeIdAndStatus(Long employeeId, AssignmentStatus status);
15
16     @Query(value = "SELECT da.* FROM device_assignment da JOIN device d ON da.device_id = d.id WHERE d.vendor_id = :vendorId", nativeQuery = true)
17     List<DeviceAssignment> getAllAssignmentsByVendorId(@Param("vendorId") long vendorId);
18
19
20 }
21
```

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.Device;
4 import com.example.EmployeeManagementSystem.Enum.DeviceStatus;
5 import org.springframework.data.jpa.repository.JpaRepository;
6
7 import java.util.List;
8
9 public interface DeviceRepository extends JpaRepository<Device, Long> {
10     List<Device> findByTechVendorId(Long vendorId);
11     List<Device> findByDeviceStatus(DeviceStatus deviceStatus);
12 }
13
```

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Enum.Role;
5 import com.example.EmployeeManagementSystem.Enum.Status;
6 import org.springframework.data.jpa.repository.JpaRepository;
7 import org.springframework.data.jpa.repository.Query;
8 import org.springframework.data.repository.query.Param;
9 import org.springframework.stereotype.Repository;
10
11 import java.util.List;
12 import java.util.Optional;
13
14 @Repository
15 public interface EmployeeRepo extends JpaRepository<Employee, Long> {
16     Optional<Employee> findByEmail(String email);
17     Optional<Employee> findByName(String name);
18     long countByRole(Role role);
19
20     List<Employee> findByManager(Employee manager);
21     List<Employee> findByStatus(Status status);
22     List<Employee> findByRole(Role role);
23     @Query("SELECT e FROM Employee e WHERE e.manager.employeeId = :managerId")
24     List<Employee> findByManagerEmployeeId(@Param("managerId") Long managerId);
25 }
26
```

115 Current:

src\main\java\com\example\EmployeeManagementSystem\Repository\FixedHolidayRepository.java

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.FixedHoliday;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
6 import java.time.LocalDate;
7 import java.util.List;
8
9 public interface FixedHolidayRepository extends JpaRepository<FixedHoliday, Long> {
10     List<FixedHoliday> findByYear(int year);
11     List<FixedHoliday> findByDateBetween(LocalDate start, LocalDate end);
12 }
13
```

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.data.jpa.repository.Query;
6 import org.springframework.data.repository.query.Param;
7
8 import java.math.BigDecimal;
9 import java.util.List;
10 import java.util.Optional;
11
12 public interface LeaveEntitlementRepository extends JpaRepository<LeaveEntitlement, Long> {
13
14     Optional<LeaveEntitlement> findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
15         Long employeeId, Integer leaveTypeId, Integer year);
16
17     List<LeaveEntitlement> findByEmployeeEmployeeIdAndYear(Long employeeId, Integer year);
18
19     @Query("""
20         SELECT COALESCE(SUM(e.closingBalance), 0)
21         FROM LeaveEntitlement e
22         WHERE e.employee.employeeId = :empId
23             AND e.leaveType.carriesForward = true
24             AND e.year IN :years
25         """)
26     BigDecimal sumCarryForwardBalances(
27         @Param("empId") Long employeeId,
28         @Param("years") List<Integer> years);
29
30     List<LeaveEntitlement> findByLeaveTypeIdAndYear(Integer leaveTypeId, Integer year);
31 }
```

```

1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.LeaveRequest;
5 import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
6 import com.example.EmployeeManagementSystem.Enum.LeaveType;
7 import org.springframework.data.jpa.repository.JpaRepository;
8 import org.springframework.data.jpa.repository.Query;
9 import org.springframework.data.repository.query.Param;
10
11 import java.time.LocalDate;
12 import java.util.List;
13
14 public interface LeaveRequestRepo extends JpaRepository<LeaveRequest, Long> {
15
16     List<LeaveRequest> findByStatus(LeaveStatus status);
17
18     /**
19      * Find leave requests by start date and status
20      * Used by scheduler to find leaves starting today
21      */
22     List<LeaveRequest> findByStartDateAndStatus(LocalDate startDate, LeaveStatus status);
23
24     // Fix the checkDuplicate query (fix parameter names)
25     @Query(value = "SELECT COUNT(*) " +
26         "FROM leave_request l " +
27         "WHERE l.employee_id = :employeeId " +
28         "AND l.start_date = :startDate " +
29         "AND l.end_date = :endDate " +
30         "AND l.status=:status",nativeQuery = true)
31     long checkDuplicate(@Param("employeeId") long employeeId,
32         @Param("startDate") LocalDate startDate,
33         @Param("endDate") LocalDate endDate,
34         @Param("status") String status);
35
36     // Fix overlapping leave query (was commented in your code)
37     @Query(value = "select COUNT(*) " +
38         "from leave_request l " +
39         "where l.employee_id=:employeeId " +
40         "AND l.status IN ('APPROVED','PENDING') " +
41         "AND l.start_date <= :endDate " +
42         "AND l.end_date >= :startDate",nativeQuery = true)
43     long countOverlappingLeave(
44         @Param("employeeId") long employeeId,
45         @Param("startDate") LocalDate startDate,
46         @Param("endDate") LocalDate endDate
47     );
48
49     // Add missing methods
50     List<LeaveRequest> findByEmployeeOrderByStartDateDesc(Employee employee);
51
52
53     List<LeaveRequest> findByEndDateAndStatus(LocalDate endDate, LeaveStatus status);
54
55     // Add for leave duration validation
56     @Query("SELECT COALESCE(SUM(DATEDIFF(l.endDate, l.startDate) + 1), 0) " +
57         "FROM LeaveRequest l WHERE l.employee = :employee " +
58         "AND l.leaveType = :leaveType " +
59         "AND YEAR(l.startDate) = :year " +
60         "AND l.status IN ('APPROVED', 'PENDING'))")
61     long countDaysByEmployeeAndLeaveTypeAndYear(@Param("employee") Employee employee,
62         @Param("leaveType") LeaveType leaveType,
63         @Param("year") int year);
64
65     @Query("SELECT lr FROM LeaveRequest lr JOIN FETCH lr.employee WHERE lr.status = :status")
66     List<LeaveRequest> findApprovedLeavesWithEmployee(LeaveStatus status);
67
68
69     List<LeaveRequest> findByEmployee_Manager(Employee manager);
70
71     List<LeaveRequest> findByStatusAndEmployee_Manager(LeaveStatus status, Employee manager);

```

```

72
73     @Query("""
74     SELECT COALESCE(SUM(DATEDIFF(lr.endDate, lr.startDate) + 1), 0)
75     FROM LeaveRequest lr
76     WHERE lr.employee = :employee
77           AND lr.leaveType = com.example.EmployeeManagementSystem.Enum.LeaveType.SICK
78           AND lr.status = com.example.EmployeeManagementSystem.Enum.LeaveStatus.APPROVED
79           AND YEAR(lr.startDate) = :year
80           AND MONTH(lr.startDate) = :month
81     """)
82     long countApprovedSickDaysInMonth(
83         @Param("employee") Employee employee,
84         @Param("year") int year,
85         @Param("month") int month);
86
87     @Query("""
88     SELECT COALESCE(SUM(DATEDIFF(lr.endDate, lr.startDate) + 1), 0)
89     FROM LeaveRequest lr
90     WHERE lr.employee = :employee
91           AND lr.leaveType = com.example.EmployeeManagementSystem.Enum.LeaveType.CASUAL
92           AND lr.status IN (
93               com.example.EmployeeManagementSystem.Enum.LeaveStatus.APPROVED,
94               com.example.EmployeeManagementSystem.Enum.LeaveStatus.PENDING
95           )
96           AND YEAR(lr.startDate) = :year
97           AND MONTH(lr.startDate) = :month
98     """)
99     long countCasualDaysInMonth(
100         @Param("employee") Employee employee,
101         @Param("year") int year,
102         @Param("month") int month
103     );
104
105     @Query("SELECT COUNT(l) > 0 FROM LeaveRequest l " +
106           "WHERE l.employee = :employee " +
107           "AND l.leaveType = :leaveType " +
108           "AND YEAR(l.startDate) = :year " +
109           "AND l.status NOT IN (:statuses)")
110     boolean existsMaternityLeaveForYear(
111         @Param("employee") Employee employee,
112         @Param("leaveType") LeaveType leaveType,
113         @Param("year") int year,
114         @Param("statuses") List<LeaveStatus> statuses
115     );
116     @Query("""
117     SELECT COALESCE(SUM(DATEDIFF(lr.endDate, lr.startDate) + 1), 0)
118     FROM LeaveRequest lr
119     WHERE lr.employee = :employee
120           AND lr.leaveType = com.example.EmployeeManagementSystem.Enum.LeaveType.UNPAID
121           AND lr.status IN (
122               com.example.EmployeeManagementSystem.Enum.LeaveStatus.APPROVED,
123               com.example.EmployeeManagementSystem.Enum.LeaveStatus.PENDING
124           )
125           AND YEAR(lr.startDate) = :year
126           AND MONTH(lr.startDate) = :month
127     """)
128     long countUnpaidDaysInMonth(
129         @Param("employee") Employee employee,
130         @Param("year") int year,
131         @Param("month") int month
132     );
133 }
134

```

118 Current:

src\main\java\com\example\EmployeeManagementSystem\Repository\LeaveTypeRepository.java

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
4 import com.example.EmployeeManagementSystem.Entity.LeaveType;
5 import org.springframework.data.jpa.repository.JpaRepository;
6
7 import java.util.List;
8 import java.util.Optional;
9
10 public interface LeaveTypeRepository extends JpaRepository<LeaveType, Integer> {
11     Optional<LeaveType> findByName(String name);
12     List<LeaveType> findByMonthlyResetTrue();
13 }
```


119 Current:

src\main\java\com\example\EmployeeManagementSystem\Repository\LeaveWarningRepository.java

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.LeaveWarning;
4 import org.springframework.data.jpa.repository.JpaRepository;
5
6 import java.time.LocalDate;
7 import java.util.List;
8
9 public interface LeaveWarningRepository extends JpaRepository<LeaveWarning, Long> {
10
11     // All unread warnings for an employee – shown on dashboard
12     List<LeaveWarning> findByEmployeeEmployeeIdAndIsReadFalseOrderByWarningDateDesc(Long employeeId);
13
14     // All warnings for an employee (read + unread) – for history view
15     List<LeaveWarning> findByEmployeeEmployeeIdOrderByWarningDateDesc(Long employeeId);
16
17     // Prevent duplicate warnings for same type on same date
18     boolean existsByEmployeeEmployeeIdAndWarningTypeAndWarningDate(
19         Long employeeId, LeaveWarning.WarningType warningType, LocalDate date
20     );
21 }
```

120 Current:

src\main\java\com\example\EmployeeManagementSystem\Repository\NotificationRepository.java

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.Notification;
4 import com.example.EmployeeManagementSystem.Entity.Employee;
5 import org.springframework.data.jpa.repository.JpaRepository;
6 import java.util.List;
7
8 public interface NotificationRepository extends JpaRepository<Notification, Long> {
9     List<Notification> findByRecipientOrderByCreatedAtDesc(Employee recipient);
10    long countByRecipientAndIsReadFalse(Employee recipient);
11 }
12
```

121 Current:

src\main\java\com\example\EmployeeManagementSystem\Repository\RefreshTokenRepository.java

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.RefreshToken;
5 import com.example.EmployeeManagementSystem.Entity.Vendor;
6 import org.springframework.data.jpa.repository.JpaRepository;
7
8 import java.util.Optional;
9
10 public interface RefreshTokenRepository extends JpaRepository<RefreshToken, Long> {
11
12     Optional<RefreshToken> findByToken(String token);
13
14     // Needed so EmployeeService can clean up tokens before deleting an employee
15     void deleteByEmployee(Employee employee);
16
17     // Needed so VendorService can clean up tokens before deleting a vendor
18     void deleteByVendor(Vendor vendor);
19 }
```

122 Current:

src\main\java\com\example\EmployeeManagementSystem\Repository\RestaurantRepository.java

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.Restaurant;
4 import com.example.EmployeeManagementSystem.Enum.MealSlot;
5 import org.springframework.data.jpa.repository.JpaRepository;
6 import org.springframework.data.jpa.repository.Query;
7 import org.springframework.data.repository.query.Param;
8 import org.springframework.stereotype.Repository;
9
10 import java.util.List;
11
12 @Repository
13 public interface RestaurantRepository extends JpaRepository<Restaurant, Long> {
14
15     /** All active restaurants owned by a vendor. */
16     List<Restaurant> findByVendor_EmailAndActiveTrue(String email);
17
18     /** All active restaurants (for employees browsing available options). */
19     List<Restaurant> findByActiveTrue();
20
21     /**
22      * Active restaurants that support a given meal slot.
23      * Employees use this to discover which restaurants they can subscribe to for a given slot.
24      */
25     @Query("SELECT r FROM Restaurant r JOIN r.supportedMealSlots s " +
26            "WHERE r.active = true AND s = :mealSlot")
27     List<Restaurant> findActiveByMealSlot(@Param("mealSlot") MealSlot mealSlot);
28
29     /** Check if a restaurant belongs to a vendor (ownership guard). */
30     boolean existsByIdAndVendor_Email(Long restaurantId, String email);
31 }
```

123 Current:

src\main\java\com\example\EmployeeManagementSystem\Repository\ServiceRequestRepository.java

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3
4 import com.example.EmployeeManagementSystem.Entity.ServiceRequest;
5 import com.example.EmployeeManagementSystem.Entity.Employee;
6 import com.example.EmployeeManagementSystem.Entity.Vendor;
7 import com.example.EmployeeManagementSystem.Enum.ServiceRequestStatus;
8 import jdk.dynalink.linker.LinkerServices;
9 import org.springframework.data.jpa.repository.JpaRepository;
10
11 import java.util.List;
12
13 public interface ServiceRequestRepository extends JpaRepository<ServiceRequest, Long> {
14
15     List<ServiceRequest> findByStatus(ServiceRequestStatus status);
16     List<ServiceRequest> findByRaisedByOrderByRaisedAtDesc(Employee employee);
17     List<ServiceRequest> findByDeviceTechVendorAndStatus(
18         Vendor vendor,
19         ServiceRequestStatus status
20     );
21
22 }
23
```

124 Current:

src\main\java\com\example\EmployeeManagementSystem\Repository\SubscriptionRepository.java

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.Subscription;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.data.jpa.repository.Query;
6 import org.springframework.data.repository.query.Param;
7 import org.springframework.stereotype.Repository;
8
9 import java.time.Instant;
10 import java.util.List;
11
12 @Repository
13 public interface SubscriptionRepository extends JpaRepository<Subscription, Long> {
14     @Query(value = "select count(*) from subscription s where s.employee_id=:employeeId and s.slot=:slot and s.status=:status", nativeQuery = true)
15     int checkSubscriptionExists(
16         @Param("employeeId") long employeeId,
17         @Param("slot") String slot,
18         @Param("status") String status
19     );
20
21     List<Subscription> findByEmployee_employeeId(long id);
22
23     @Query(value = "SELECT * FROM subscription s WHERE s.status = :status AND s.next_delivery_time IS NOT NULL AND s.next_delivery_time <= :currentTime", nativeQuery = true)
24     List<Subscription> findDueSubscriptions(
25         @Param("status") String status,
26         @Param("currentTime") Instant currentTime // was LocalDateTime
27     );
28
29     List<Subscription> findByEmployee_name(String name);
30     List<Subscription> findByEmployee_Email(String email);
31 }
32
```

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3 import com.example.EmployeeManagementSystem.Entity.Vendor;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.stereotype.Repository;
6
7 import java.util.Optional;
8
9 @Repository
10 public interface VendorRepo extends JpaRepository<Vendor, Long> {
11     Optional<Vendor> findByEmail(String email);
12     boolean existsByEmail(String email);
13 }
```

```
1 package com.example.EmployeeManagementSystem.Repository;
2
3
4 import com.example.EmployeeManagementSystem.Entity.YearEndCarryForwardLog;
5 import org.springframework.data.jpa.repository.JpaRepository;
6
7 import java.util.List;
8
9 public interface YearEndCarryForwardLogRepository
10     extends JpaRepository<YearEndCarryForwardLog, Long> {
11
12     List<YearEndCarryForwardLog> findByEmployeeEmployeeIdAndProcessedYear(
13         Long employeeId, Integer year
14     );
15
16     // Check if year-end has already been run for this year (idempotency guard)
17     boolean existsByProcessedYear(Integer year);
18 }
```



```
1 package com.example.EmployeeManagementSystem.Scheduler;
2
3 import com.example.EmployeeManagementSystem.Service.DeliveryService;
4 import org.slf4j.Logger;
5 import org.slf4j.LoggerFactory;
6 import org.springframework.scheduling.annotation.Scheduled;
7 import org.springframework.stereotype.Component;
8
9 @Component
10 public class DeliveryScheduler {
11     private static final Logger logger = LoggerFactory.getLogger(DeliveryScheduler.class);
12     private final DeliveryService deliveryService;
13
14     public DeliveryScheduler(DeliveryService deliveryService) {
15         this.deliveryService = deliveryService;
16     }
17
18     // Runs every hour
19     @Scheduled(cron = "0 0 * * * *")
20     public void processDeliveries() {
21         logger.info("Starting delivery processing scheduler...");
22         try {
23             deliveryService.processDueDeliveries();
24             logger.info("Delivery processing completed successfully");
25         } catch (Exception e) {
26             logger.error("Error processing deliveries: ", e);
27         }
28     }
29 }
30
```

```

1 package com.example.EmployeeManagementSystem.Scheduler;
2
3 import com.example.EmployeeManagementSystem.Service.CarryForwardWarningService;
4 import com.example.EmployeeManagementSystem.Service.LeaveAccrualService;
5 import com.example.EmployeeManagementSystem.Repository.ApiKeyRepository;
6 import com.example.EmployeeManagementSystem.Service.SickLeaveResetService;
7 import com.example.EmployeeManagementSystem.Service.YearEndCarryForwardService;
8 import org.slf4j.Logger;
9 import org.slf4j.LoggerFactory;
10 import org.springframework.scheduling.annotation.Scheduled;
11 import org.springframework.stereotype.Component;
12 import org.springframework.transaction.annotation.Transactional;
13
14 import java.time.Instant;
15 import java.time.LocalDate;
16 import java.time.YearMonth;
17
18 @Component
19 public class LeaveAccrualScheduler {
20
21     private static final Logger log = LoggerFactory.getLogger(LeaveAccrualScheduler.class);
22
23     private final LeaveAccrualService accrualService;
24     private final SickLeaveResetService resetService;
25     private final CarryForwardWarningService warningService;
26     private final YearEndCarryForwardService yearEndService;
27     private final ApiKeyRepository apiKeyRepository;
28
29     public LeaveAccrualScheduler(LeaveAccrualService accrualService,
30                                 SickLeaveResetService resetService,
31                                 CarryForwardWarningService warningService,
32                                 YearEndCarryForwardService yearEndService,
33                                 ApiKeyRepository apiKeyRepository) {
34         this.accrualService = accrualService;
35         this.resetService = resetService;
36         this.warningService = warningService;
37         this.yearEndService = yearEndService;
38         this.apiKeyRepository = apiKeyRepository;
39     }
40
41     // 1st of month at 00:05 – accrue 1 day for all eligible employees, then warn
42     @Scheduled(cron = "0 5 0 1 * ?")
43     public void runMonthlyAccrual() {
44         LocalDate today = LocalDate.now();
45         accrualService.runMonthlyAccrual(today);
46         warningService.runWarningCheck(today);
47     }
48
49     // Last day of month at 23:55 – reset unused sick leave
50     // FIX: use YearMonth to compute the actual last day of the current month,
51     //      so if the scheduler fires even a minute late (past midnight) on the
52     //      1st, we still reset the correct month and log the correct date.
53     @Scheduled(cron = "0 55 23 L * ?")
54     public void runSickLeaveReset() {
55         LocalDate lastDayOfMonth = YearMonth.now().atEndOfMonth();
56         log.info("Running sick leave reset for end-of-month: {}", lastDayOfMonth);
57         resetService.runMonthlyReset(lastDayOfMonth);
58     }
59
60     // Dec 31 at 22:00 – year-end carry-forward cap
61     @Scheduled(cron = "0 0 22 31 12 ?")
62     public void runYearEnd() {
63         int closingYear = LocalDate.now().getYear();
64         yearEndService.runYearEnd(closingYear);
65     }
66     @Scheduled(cron = "0 0 0 * * *") // runs every midnight
67     @Transactional
68     public void expireKeys() {
69         apiKeyRepository.expireOldKeys(Instant.now());
70     }
71 }

```

```

1 package com.example.EmployeeManagementSystem.Scheduler;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.LeaveRequest;
5 import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
6 import com.example.EmployeeManagementSystem.Enum.Status;
7 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
8 import com.example.EmployeeManagementSystem.Repository.LeaveRequestRepo;
9 import org.slf4j.Logger;
10 import org.slf4j.LoggerFactory;
11 import org.springframework.scheduling.annotation.Scheduled;
12 import org.springframework.stereotype.Component;
13 import org.springframework.transaction.annotation.Transactional;
14
15 import java.time.LocalDate;
16 import java.time.ZoneId;
17 import java.util.List;
18 import java.util.Map;
19 import java.util.stream.Collectors;
20
21 @Component
22 public class LeaveStatusScheduler {
23
24     private static final Logger log = LoggerFactory.getLogger(LeaveStatusScheduler.class);
25
26     private final LeaveRequestRepo leaveRequestRepo;
27     private final EmployeeRepo employeeRepo;
28
29     public LeaveStatusScheduler(LeaveRequestRepo leaveRequestRepo,
30                               EmployeeRepo employeeRepo) {
31         this.leaveRequestRepo = leaveRequestRepo;
32         this.employeeRepo = employeeRepo;
33     }
34
35     /**
36      * Runs every day at midnight UTC
37      */
38     @Scheduled(cron = "0 0 0 * * *", zone = "UTC")
39     @Transactional
40     public void updateEmployeeLeaveStatus() {
41
42         log.info("=== Starting Timezone-Aware Leave Scheduler ===");
43
44         List<LeaveRequest> leaves =
45             leaveRequestRepo.findApprovedLeavesWithEmployee(LeaveStatus.APPROVED);
46
47         Map<Long, List<LeaveRequest>> employeeLeavesMap = leaves.stream()
48             .collect(Collectors.groupingBy(lr -> lr.getEmployee().getEmployeeId()));
49
50         int onLeaveCount = 0;
51         int activeCount = 0;
52
53         for (Map.Entry<Long, List<LeaveRequest>> entry : employeeLeavesMap.entrySet()) {
54
55             Employee employee = entry.getValue().get(0).getEmployee();
56
57             ZoneId zone = ZoneId.of(
58                 employee.getTimezone() != null ? employee.getTimezone() : "UTC"
59             );
60
61             LocalDate today = LocalDate.now(zone);
62
63             boolean isOnLeaveToday = entry.getValue().stream().anyMatch(leave ->
64                 !today.isBefore(leave.getStartDate()) &&
65                 !today.isAfter(leave.getEndDate())
66             );
67
68             // CASE 1: Should be ON_LEAVE
69             if (isOnLeaveToday && employee.getStatus() != Status.ON_LEAVE) {
70
71                 employee.setStatus(Status.ON_LEAVE);

```

```

72         employeeRepo.save(employee);
73         onLeaveCount++;
74
75         log.info("Employee {} ({} marked ON_LEAVE | TZ: {}",
76                 employee.getEmployeeId(),
77                 employee.getName(),
78                 zone);
79     }
80
81     // CASE 2: Should be ACTIVE
82     if (!isOnLeaveToday && employee.getStatus() == Status.ON_LEAVE) {
83
84         employee.setStatus(Status.ACTIVE);
85         employeeRepo.save(employee);
86         activeCount++;
87
88         log.info("Employee {} ({} marked ACTIVE | TZ: {}",
89                 employee.getEmployeeId(),
90                 employee.getName(),
91                 zone);
92     }
93 }
94
95     log.info("=== Scheduler Completed ===");
96     log.info("ON_LEAVE updated: {}", onLeaveCount);
97     log.info("ACTIVE updated: {}", activeCount);
98 }
99
100 /**
101  * Manual trigger (for testing)
102  */
103 public void runManually() {
104     log.info("Manual scheduler trigger started...");
105     updateEmployeeLeaveStatus();
106 }
107 }

```

```

1 // src/main/java/com/example/EmployeeManagementSystem/security/ApiKeyAuthenticationProvider.java
2 package com.example.EmployeeManagementSystem.security;
3
4 import com.example.EmployeeManagementSystem.Entity.ApiKey;
5 import com.example.EmployeeManagementSystem.Entity.Employee;
6 import com.example.EmployeeManagementSystem.Entity.Vendor;
7 import com.example.EmployeeManagementSystem.Repository.ApiKeyRepository;
8 import org.springframework.security.access.prepost.PreAuthorize;
9 import org.springframework.security.authentication.AuthenticationProvider;
10 import org.springframework.security.core.Authentication;
11 import org.springframework.security.core.AuthenticationException;
12 import org.springframework.security.core.GrantedAuthority;
13 import org.springframework.security.core.authority.SimpleGrantedAuthority;
14 import org.springframework.stereotype.Component;
15 import org.springframework.transaction.annotation.Transactional;
16
17 import java.time.Instant;
18 import java.util.ArrayList;
19 import java.util.Arrays;
20 import java.util.Collection;
21 import java.util.List;
22 import java.util.stream.Collectors;
23
24 @Component
25 public class ApiKeyAuthenticationProvider implements AuthenticationProvider {
26
27     private final ApiKeyRepository apiKeyRepository;
28
29     public ApiKeyAuthenticationProvider(ApiKeyRepository apiKeyRepository) {
30         this.apiKeyRepository = apiKeyRepository;
31     }
32
33     @Override
34     @Transactional
35     public Authentication authenticate(Authentication authentication) throws AuthenticationException {
36         String apiKey = null;
37
38         // Extract API key from the authentication object
39         if (authentication instanceof ApiKeyAuthenticationToken) {
40             apiKey = ((ApiKeyAuthenticationToken) authentication).getApiKey();
41         }
42
43         if (apiKey == null || apiKey.isEmpty()) {
44             throw new RuntimeException("Invalid API key");
45         }
46
47         ApiKey key = apiKeyRepository.findByApiKeyAndActiveTrue(apiKey)
48             .orElseThrow(() -> new RuntimeException("Invalid or inactive API key"));
49
50         // Check expiration
51         if (key.getExpiresAt() != null && key.getExpiresAt().isBefore(Instant.now())) {
52             throw new RuntimeException("API key has expired");
53         }
54
55         // Update last used timestamp
56         apiKeyRepository.updateLastUsed(apiKey, Instant.now());
57
58         // Build authorities from permissions
59         List<GrantedAuthority> authorities = new ArrayList<>();
60
61         if (key.getPermissions() != null && !key.getPermissions().isEmpty()) {
62             authorities.addAll(Arrays.stream(key.getPermissions().split(","))
63                 .map(String::trim)
64                 .map(SimpleGrantedAuthority::new)
65                 .collect(Collectors.toList()));
66         }
67
68         Object principal;
69         if (key.getEmployee() != null) {
70             principal = key.getEmployee();
71             authorities.add(new SimpleGrantedAuthority("ROLE_" + key.getEmployee().getRole().name()));

```

```
72         } else if (key.getVendor() != null) {
73             principal = key.getVendor();
74             authorities.add(new SimpleGrantedAuthority("ROLE_" + key.getVendor().getRole().name()));
75         } else {
76             throw new RuntimeException("No user associated with API key");
77         }
78
79         return new ApiKeyAuthenticationToken(principal, apiKey, authorities);
80     }
81
82     @Override
83     public boolean supports(Class<?> authentication) {
84         return ApiKeyAuthenticationToken.class.isAssignableFrom(authentication);
85     }
86 }
```

131 Current:

src\main\java\com\example\EmployeeManagementSystem\security\ApiKeyAuthenticationToken.java

```
1 // src/main/java/com/example/EmployeeManagementSystem/security/ApiKeyAuthenticationToken.java
2 package com.example.EmployeeManagementSystem.security;
3
4 import org.springframework.security.core.Authentication;
5 import org.springframework.security.core.GrantedAuthority;
6 import java.util.Collection;
7
8 public class ApiKeyAuthenticationToken implements Authentication {
9
10     private static final long serialVersionUID = 1L;
11
12     private final Object principal;
13     private final Object credentials;
14     private final String apiKey;
15     private final Collection<? extends GrantedAuthority> authorities;
16     private boolean authenticated;
17
18     // Constructor for unauthenticated API key (before validation)
19     public ApiKeyAuthenticationToken(String apiKey) {
20         this.apiKey = apiKey;
21         this.principal = null;
22         this.credentials = null;
23         this.authorities = null;
24         this.authenticated = false;
25     }
26
27     // Constructor for authenticated API key (after validation)
28     public ApiKeyAuthenticationToken(Object principal, Object credentials,
29                                     Collection<? extends GrantedAuthority> authorities) {
30         this.apiKey = null;
31         this.principal = principal;
32         this.credentials = credentials;
33         this.authorities = authorities;
34         this.authenticated = true;
35     }
36
37     @Override
38     public Collection<? extends GrantedAuthority> getAuthorities() {
39         return authorities;
40     }
41
42     @Override
43     public Object getCredentials() {
44         return credentials;
45     }
46
47     @Override
48     public Object getDetails() {
49         return null;
50     }
51
52     @Override
53     public Object getPrincipal() {
54         return principal;
55     }
56
57     @Override
58     public boolean isAuthenticated() {
59         return authenticated;
60     }
61
62     @Override
63     public void setAuthenticated(boolean isAuthenticated) throws IllegalArgumentException {
64         this.authenticated = isAuthenticated;
65     }
66
67     @Override
68     public String getName() {
69         if (principal instanceof org.springframework.security.core.userdetails.UserDetails) {
70             return ((org.springframework.security.core.userdetails.UserDetails) principal).getUsername();
71         }
```

```
72         if (principal instanceof com.example.EmployeeManagementSystem.Entity.Employee) {
73             return ((com.example.EmployeeManagementSystem.Entity.Employee) principal).getUsername();
74         }
75         if (principal instanceof com.example.EmployeeManagementSystem.Entity.Vendor) {
76             return ((com.example.EmployeeManagementSystem.Entity.Vendor) principal).getUsername();
77         }
78         return principal != null ? principal.toString() : apiKey;
79     }
80
81     public String getApiKey() {
82         return apiKey;
83     }
84 }
```



```
1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.Entity.ApiKey;
4 import com.example.EmployeeManagementSystem.Entity.Vendor;
5 import com.example.EmployeeManagementSystem.Repository.ApiKeyRepository;
6 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
7 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
8 import org.springframework.beans.factory.annotation.Autowired;
9 import org.springframework.stereotype.Service;
10 import org.springframework.transaction.annotation.Transactional;
11
12 import java.time.Instant;
13 import java.time.temporal.ChronoUnit;
14 import java.util.List;
15
16 @Service
17 public class ApiKeyService {
18
19     @Autowired
20     private ApiKeyRepository apiKeyRepository;
21     @Autowired private EmployeeRepo employeeRepo;
22     @Autowired private VendorRepo vendorRepo;
23
24     @Transactional
25     public ApiKey generateForVendor(Long vendorId, String name,
26                                     String permissions, int validDays) {
27         Vendor vendor = vendorRepo.findById(vendorId)
28             .orElseThrow(() -> new RuntimeException("Vendor not found"));
29
30         ApiKey key = new ApiKey(); // constructor auto-generates "ems_<uuid>"
31         key.setVendor(vendor);
32         key.setKeyName(name);
33         key.setPermissions(permissions); // e.g. "VENDOR_WRITE" or "TECH_VENDOR"
34         key.setExpiresAt(Instant.now().plus(validDays, ChronoUnit.DAYS));
35         key.setActive(true);
36         return apiKeyRepository.save(key);
37     }
38
39     // Same for employee-issued keys if needed
40     public List<ApiKey> getActiveKeysForVendor(Long vendorId) {
41         return apiKeyRepository.findByVendor_IdAndActiveTrue(vendorId);
42     }
43
44     @Transactional
45     public void revokeKey(Long keyId) {
46         ApiKey key = apiKeyRepository.findById(keyId)
47             .orElseThrow(() -> new RuntimeException("Key not found"));
48         key.setActive(false);
49         apiKeyRepository.save(key);
50     }
51 }
52
```

```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
5 import com.example.EmployeeManagementSystem.Entity.LeaveWarning;
6 import com.example.EmployeeManagementSystem.Enum.Status;
7 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
8 import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
9 import com.example.EmployeeManagementSystem.Repository.LeaveWarningRepository;
10 import org.slf4j.Logger;
11 import org.slf4j.LoggerFactory;
12 import org.springframework.stereotype.Service;
13 import org.springframework.transaction.annotation.Transactional;
14
15 import java.math.BigDecimal;
16 import java.time.LocalDate;
17 import java.util.List;
18
19 @Service
20 public class CarryForwardWarningService {
21
22     private static final Logger log = LoggerFactory.getLogger(CarryForwardWarningService.class);
23
24     private static final BigDecimal CAP = BigDecimal.valueOf(30);
25     private static final BigDecimal WARN_AT_29 = BigDecimal.valueOf(29);
26     private static final BigDecimal WARN_AT_28 = BigDecimal.valueOf(28);
27
28     private final EmployeeRepo employeeRepository;
29     private final LeaveEntitlementRepository entitlementRepository;
30     private final LeaveWarningRepository warningRepository;
31
32     public CarryForwardWarningService(EmployeeRepo employeeRepository,
33                                     LeaveEntitlementRepository entitlementRepository,
34                                     LeaveWarningRepository warningRepository) {
35         this.employeeRepository = employeeRepository;
36         this.entitlementRepository = entitlementRepository;
37         this.warningRepository = warningRepository;
38     }
39
40     @Transactional
41     public void runWarningCheck(LocalDate checkDate) {
42         List<Employee> activeEmployees = employeeRepository.findByStatus(Status.ACTIVE);
43
44         for (Employee employee : activeEmployees) {
45             BigDecimal carryForwardTotal = calculateCarryForwardBalance(employee, checkDate);
46             evaluateAndWarn(employee, carryForwardTotal, checkDate);
47         }
48
49         log.info("Warning check complete for {} – {} employees evaluated",
50               checkDate, activeEmployees.size());
51     }
52
53     // -----
54     // Sum available balance across all carry-forwardable types
55     // for the current year (this is what will carry forward if unused)
56     // -----
57     private BigDecimal calculateCarryForwardBalance(Employee employee, LocalDate date) {
58         int year = date.getYear();
59
60         List<LeaveEntitlement> entitlements =
61             entitlementRepository.findByEmployeeEmployeeIdAndYear(employee.getEmployeeId(), year);
62
63         return entitlements.stream()
64             .filter(e -> e.getLeaveType().isCarriesForward())
65             .map(LeaveEntitlement::getAvailableBalance)
66             .reduce(BigDecimal.ZERO, BigDecimal::add);
67     }
68
69     // -----
70     // Decide which warning to issue based on carry-forward total
71     // -----

```

```

72     private void evaluateAndWarn(Employee employee, BigDecimal balance, LocalDate date) {
73         int cmp30 = balance.compareTo(CAP);
74         int cmp29 = balance.compareTo(WARN_AT_29);
75         int cmp28 = balance.compareTo(WARN_AT_28);
76
77         if (cmp30 > 0) {
78             // Balance EXCEEDS cap – a day was wasted (cap enforcement happens in Phase 5,
79             // but we flag it here so the warning appears immediately after accrual)
80             issueWarning(employee, LeaveWarning.WarningType.DAY_WASTED, balance, date,
81                 "A carry-forward day was wasted this month because your balance exceeded the 30-day
82 cap.");
83         } else if (cmp30 == 0) {
84             issueWarning(employee, LeaveWarning.WarningType.CAP_REACHED, balance, date,
85                 "Carry-forward cap reached (30 days). Next month's unused leave will be wasted.");
86         } else if (cmp29 >= 0) {
87             issueWarning(employee, LeaveWarning.WarningType.APPROACHING_29, balance, date,
88                 "You have " + balance.toPlainString() + " carry-forward days. "
89                 + "1 month until cap. Use leave before it stops accumulating.");
90         } else if (cmp28 >= 0) {
91             issueWarning(employee, LeaveWarning.WarningType.APPROACHING_28, balance, date,
92                 "You have " + balance.toPlainString() + " carry-forward days. "
93                 + "2 months until cap (30). Plan to use leave soon.");
94         }
95         // Below 28 – no warning needed
96     }
97 }
98
99 // -----
100 // Persist warning – skip if same type already issued today
101 // -----
102 private void issueWarning(Employee employee, LeaveWarning.WarningType type,
103     BigDecimal balance, LocalDate date, String message) {
104     boolean alreadyIssued = warningRepository
105         .existsByEmployeeEmployeeIdAndWarningTypeAndWarningDate(employee.getEmployeeId(), type,
106 date);
107     if (alreadyIssued) {
108         log.debug("Warning {} already issued for employee {} on {}", type, employee.getEmployeeId(),
109 date);
110         return;
111     }
112
113     LeaveWarning warning = new LeaveWarning();
114     warning.setEmployee(employee);
115     warning.setWarningType(type);
116     warning.setMessage(message);
117     warning.setCarryForwardBalance(balance);
118     warning.setWarningDate(date);
119
120     warningRepository.save(warning);
121
122     log.info("Warning issued – employee={} type={} balance={} date={}",
123         employee.getEmployeeId(), type, balance, date);
124 }
125 }
126 }

```

134 Current:

src\main\java\com\example\EmployeeManagementSystem\Service\CombinedUserDetailService.java

```
1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.Vendor;
5 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
6 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
7 import org.springframework.security.core.userdetails.UserDetails;
8 import org.springframework.security.core.userdetails.UserDetailsService;
9 import org.springframework.security.core.userdetails.UsernameNotFoundException;
10 import org.springframework.stereotype.Service;
11
12 import java.util.Optional;
13
14 @Service("combinedUserDetailsService")
15 public class CombinedUserDetailsService implements UserDetailsService {
16
17     private final VendorRepo vendorRepository;
18     private final EmployeeRepo employeeRepository;
19
20     public CombinedUserDetailsService(VendorRepo vendorRepository, EmployeeRepo employeeRepository) {
21         this.vendorRepository = vendorRepository;
22         this.employeeRepository = employeeRepository;
23     }
24
25     @Override
26     public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
27         Optional<Employee> employee = employeeRepository.findByEmail(username);
28         if (employee.isPresent()) {
29             return employee.get(); // must implement UserDetails
30         }
31
32         // Then try Vendor
33         Optional<Vendor> vendor = vendorRepository.findByEmail(username);
34         if (vendor.isPresent()) {
35             return vendor.get();
36         }
37
38         throw new UsernameNotFoundException("User not found: " + username);
39     }
40 }
41
```

```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Enum.Role;
5 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
6 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
7 import org.springframework.security.core.GrantedAuthority;
8 import org.springframework.security.core.authority.SimpleGrantedAuthority;
9 import org.springframework.security.crypto.password.PasswordEncoder;
10 import org.springframework.security.oauth2.client.userinfo.DefaultOAuth2UserService;
11 import org.springframework.security.oauth2.client.userinfo.OAuth2UserRequest;
12 import org.springframework.security.oauth2.core.user.DefaultOAuth2User;
13 import org.springframework.security.oauth2.core.user.OAuth2User;
14 import org.springframework.stereotype.Service;
15
16 import java.util.ArrayList;
17 import java.util.List;
18 import java.util.UUID;
19
20 @Service
21 public class CustomOAuth2UserService extends DefaultOAuth2UserService {
22
23     private final EmployeeRepo employeeRepo;
24     private final VendorRepo vendorRepo;
25     private final PasswordEncoder passwordEncoder;
26
27     public CustomOAuth2UserService(EmployeeRepo employeeRepo,
28                                   VendorRepo vendorRepo,
29                                   PasswordEncoder passwordEncoder) {
30         this.employeeRepo = employeeRepo;
31         this.vendorRepo = vendorRepo;
32         this.passwordEncoder = passwordEncoder;
33     }
34
35     @Override
36     public OAuth2User loadUser(OAuth2UserRequest userRequest) {
37         System.out.println("=== CustomOAuth2UserService.loadUser() called ===");
38         OAuth2User oAuth2User = super.loadUser(userRequest);
39
40         String email = oAuth2User.getAttribute("email");
41         String name = oAuth2User.getAttribute("name");
42         System.out.println("OAuth email: " + email);
43
44         boolean employeeExists = employeeRepo.findByEmail(email).isPresent();
45         boolean vendorExists = vendorRepo.findByEmail(email).isPresent();
46         System.out.println("employeeExists=" + employeeExists + ", vendorExists=" + vendorExists);
47
48         List<GrantedAuthority> authorities = new ArrayList<>(oAuth2User.getAuthorities());
49
50         if (employeeExists) {
51             Employee employee = employeeRepo.findByEmail(email).get();
52             authorities.add(new SimpleGrantedAuthority("ROLE_" + employee.getRole().name()));
53
54             // ADD THIS – loads LEAVE_CANCEL, SUBSCRIPTION_READ, etc. into the session
55             employee.getRole().getPermissions().forEach(permission ->
56                 authorities.add(new SimpleGrantedAuthority(permission.name()))
57             );
58
59             System.out.println("Returning existing employee with role: " + employee.getRole());
60             return new DefaultOAuth2User(authorities, oAuth2User.getAttributes(), "sub");
61         }
62
63         if (vendorExists) {
64             // add vendor role if needed
65             authorities.add(new SimpleGrantedAuthority("ROLE_VENDOR"));
66             System.out.println("Returning existing vendor");
67             return new DefaultOAuth2User(authorities, oAuth2User.getAttributes(), "sub");
68         }
69
70         // New user
71         registerAsUser(email, name);

```

```
72     authorities.add(new SimpleGrantedAuthority("ROLE_USER"));
73     return new DefaultOAuth2User(authorities, oAuth2User.getAttributes(), "sub");
74 }
75
76 private void registerAsUser(String email, String name) {
77     Employee e = new Employee();
78     e.setEmail(email);
79     e.setName(name);
80     e.setPassword(passwordEncoder.encode(UUID.randomUUID().toString()));
81     e.setRole(Role.USER);
82     e.setTimezone("Asia/Kolkata");
83     e.setDept("UNASSIGNED");
84     employeeRepo.saveAndFlush(e); // flush immediately so OAuth2SuccessHandler sees the record
85     System.out.println("New USER registered and flushed for: " + email);
86 }
87 }
```

```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.DTO.VendorDTO;
4 import com.example.EmployeeManagementSystem.DTO.VendorRequest;
5 import com.example.EmployeeManagementSystem.Entity.Delivery;
6 import com.example.EmployeeManagementSystem.Entity.Subscription;
7 import com.example.EmployeeManagementSystem.Entity.Vendor;
8 import com.example.EmployeeManagementSystem.Enum.DeliveryStatus;
9 import com.example.EmployeeManagementSystem.Enum.Role;
10 import com.example.EmployeeManagementSystem.Enum.SubscriptionStatus;
11 import com.example.EmployeeManagementSystem.Exception.VendorNotFoundException;
12 import com.example.EmployeeManagementSystem.Repository.DeliveryRepository;
13 import com.example.EmployeeManagementSystem.Repository.SubscriptionRepository;
14 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
15 import com.example.EmployeeManagementSystem.Util.AuthUtil;
16 import org.slf4j.Logger;
17 import org.slf4j.LoggerFactory;
18 import org.springframework.dao.DataIntegrityViolationException;
19 import org.springframework.security.core.Authentication;
20 import org.springframework.security.crypto.password.PasswordEncoder;
21 import org.springframework.stereotype.Service;
22 import org.springframework.transaction.annotation.Transactional;
23
24 import java.time.Instant;
25 import java.util.List;
26 import java.util.stream.Collectors;
27
28 @Service
29 public class DeliveryService {
30     private final SubscriptionRepository subscriptionRepository;
31     private final DeliveryRepository deliveryRepository;
32     private final DeliveryTimeService deliveryTimeService;
33     private static final Logger logger = LoggerFactory.getLogger(DeliveryService.class);
34
35     public DeliveryService(SubscriptionRepository subscriptionRepository,
36                           DeliveryRepository deliveryRepository,
37                           DeliveryTimeService deliveryTimeService) {
38         this.subscriptionRepository = subscriptionRepository;
39         this.deliveryRepository = deliveryRepository;
40         this.deliveryTimeService = deliveryTimeService;
41     }
42
43     // AFTER
44     @Transactional
45     public void processDueDeliveries() {
46         Instant now = Instant.now();
47         List<Subscription> dueSubscriptions = subscriptionRepository
48             .findDueSubscriptions(SubscriptionStatus.ACTIVE.name(), now);
49
50         for (Subscription subscription : dueSubscriptions) {
51             try {
52                 Delivery delivery = new Delivery();
53                 delivery.setSubscription(subscription);
54                 delivery.setMealSlot(subscription.getSlot());
55                 delivery.setScheduledDeliveryTime(subscription.getNextDeliveryTime());
56                 delivery.setStatus(DeliveryStatus.IN_PROGRESS);
57                 deliveryRepository.save(delivery);
58
59                 Instant nextDelivery = deliveryTimeService.getNextDeliveryTime(subscription);
60                 subscription.setNextDeliveryTime(nextDelivery);
61                 subscriptionRepository.save(subscription);
62
63             } catch (DataIntegrityViolationException e) {
64                 // Duplicate delivery already exists – just advance the next delivery time
65                 logger.warn("Duplicate delivery skipped for subscription ID: {}", subscription.getId());
66                 Instant nextDelivery = deliveryTimeService.getNextDeliveryTime(subscription);
67                 subscription.setNextDeliveryTime(nextDelivery);
68                 subscriptionRepository.save(subscription);
69             }
70         }
71     }

```

```

72
73     @Transactional
74     public Delivery updateDeliveryStatus(Long deliveryId, DeliveryStatus status) {
75         Delivery delivery = deliveryRepository.findById(deliveryId)
76             .orElseThrow(() -> new RuntimeException("Delivery not found"));
77
78         delivery.setStatus(status);
79         if (status == DeliveryStatus.DELIVERED) {
80             delivery.setActualDeliveryTime(Instant.now());
81         }
82
83         return deliveryRepository.save(delivery);
84     }
85
86     public List<Delivery> getDeliveriesBySubscription(Long subscriptionId) {
87         Subscription subscription = subscriptionRepository.findById(subscriptionId)
88             .orElseThrow(() -> new RuntimeException("Subscription not found"));
89         return deliveryRepository.findRecentDeliveriesBySubscription(subscription,
DeliveryStatus.SCHEDULED);
90     }
91
92     @Service
93     public static class VendorService {
94
95         private static final Logger log = LoggerFactory.getLogger(VendorService.class);
96         private final VendorRepo vendorRepo;
97         private final PasswordEncoder passwordEncoder;
98
99         public VendorService(VendorRepo vendorRepo, PasswordEncoder passwordEncoder) {
100             this.vendorRepo = vendorRepo;
101             this.passwordEncoder = passwordEncoder;
102         }
103
104         public VendorDTO registerVendor(VendorRequest request) {
105             if (vendorRepo.existsByEmail(request.getEmail())) {
106                 throw new IllegalArgumentException(
107                     "A vendor with email " + request.getEmail() + " already exists");
108             }
109             Vendor vendor = new Vendor();
110             vendor.setName(request.getName());
111             vendor.setEmail(request.getEmail());
112             vendor.setPhone(request.getPhone());
113             vendor.setRole(request.getRole());
114             vendor.setPassword(passwordEncoder.encode(request.getPassword()));
115             Vendor saved = vendorRepo.save(vendor);
116             log.info("Vendor registered: {} (id={})", saved.getName(), saved.getId());
117             return toDTO(saved);
118         }
119
120         public VendorDTO getVendor(Authentication authentication) {
121             String email= AuthUtil.extractEmail(authentication);
122             return toDTO(vendorRepo.findById(email).orElseThrow(
123                 ()->new VendorNotFoundException("Cannot get others account")
124             ));
125         }
126
127         public List<VendorDTO> getAllVendors() {
128             return vendorRepo.findAll().stream()
129                 .map(this::toDTO)
130                 .collect(Collectors.toList());
131         }
132
133         public VendorDTO updateVendor(VendorRequest request, Authentication authentication) {
134             String email= AuthUtil.extractEmail(authentication);
135             Vendor vendor = vendorRepo.findById(email).orElseThrow(
136                 ()->new VendorNotFoundException("Cannot update others account")
137             );
138             if (request.getName() != null) vendor.setName(request.getName());
139             if (request.getPhone() != null) vendor.setPhone(request.getPhone());
140             if (request.getPassword() != null)
vendor.setPassword(passwordEncoder.encode(request.getPassword()));
141             // Email update not allowed to avoid breaking restaurant ownership references
142             return toDTO(vendorRepo.save(vendor));
143         }
144
145         public void deleteVendor(Authentication authentication) {

```



```
146         String email= AuthUtil.extractEmail(authentication);
147         Vendor vendor = vendorRepo.findByEmail(email).orElseThrow(
148             ()->new VendorNotFoundException("Cannot delete others account")
149         );
150         vendorRepo.delete(vendor);
151         log.info("Vendor deleted: email={}", email);
152     }
153
154     public void deleteVendorById(Long vendorId) {
155         Vendor vendor = findOrThrow(vendorId);
156         vendorRepo.delete(vendor);
157         log.info("Vendor deleted by admin: id={}, email={}", vendor.getId(), vendor.getEmail());
158     }
159
160     // helpers
161
162     private Vendor findOrThrow(Long id) {
163         return vendorRepo.findById(id)
164             .orElseThrow(() -> new VendorNotFoundException("Vendor with id " + id + " not found"));
165     }
166
167     private VendorDTO toDTO(Vendor vendor) {
168         VendorDTO dto = new VendorDTO();
169         dto.setId(vendor.getId());
170         dto.setName(vendor.getName());
171         dto.setEmail(vendor.getEmail());
172         dto.setPhone(vendor.getPhone());
173         dto.setRegisteredAt(vendor.getRegisteredAt());
174         return dto;
175     }
176 }
177 }
178
```

```
1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.DTO.SubscriptionRequest;
4 import com.example.EmployeeManagementSystem.Entity.Employee;
5 import com.example.EmployeeManagementSystem.Entity.Subscription;
6 import com.example.EmployeeManagementSystem.Enum.MealSlot;
7 import com.example.EmployeeManagementSystem.Enum.ScheduleType;
8 import org.springframework.stereotype.Service;
9
10 import java.time.*;
11
12 @Service
13 public class DeliveryTimeService {
14
15     private final MealSlotService mealSlotService;
16
17     public DeliveryTimeService(MealSlotService mealSlotService) {
18         this.mealSlotService = mealSlotService;
19     }
20     public Instant calculateNextDelivery(SubscriptionsRequest request, MealSlot slot, Employee employee) {
21         ZoneId userZone=ZoneId.of(employee.getTimezone());
22         if (request.getScheduleType() == ScheduleType.DAILY) {
23             return calculateDailyNextDelivery(slot,userZone);
24         } else {
25             return calculateWeeklyNextDelivery(slot, request.getDayOfWeek(),userZone);
26         }
27     }
28
29     private Instant calculateDailyNextDelivery(MealSlot slot, ZoneId userZone) {
30         LocalDateTime slotTime = mealSlotService.getTime(slot);
31         ZonedDateTime now = ZonedDateTime.now(userZone);
32         ZonedDateTime deliveryTime;
33
34         if (now.toLocalTime().isBefore(slotTime)) {
35             deliveryTime=ZonedDateTime.of(now.toLocalDate(),slotTime,userZone);
36         } else {
37             deliveryTime=ZonedDateTime.of(now.toLocalDate().plusDays(1), slotTime,userZone);
38         }
39         return deliveryTime.toInstant();
40     }
41
42     private Instant calculateWeeklyNextDelivery(MealSlot slot, DayOfWeek dayOfWeek,ZoneId userZone) {
43         LocalDateTime slotTime = mealSlotService.getTime(slot);
44         ZonedDateTime now=ZonedDateTime.now(userZone);
45         DayOfWeek today = now.getDayOfWeek();
46
47         int daysToAdd = dayOfWeek.getValue() - today.getValue();
48
49         if (daysToAdd == 0) {
50             if (now.toLocalTime().isBefore(slotTime)) {
51                 return ZonedDateTime.of(now.toLocalDate(), slotTime,userZone).toInstant();
52             } else {
53                 return ZonedDateTime.of(now.toLocalDate().plusDays(7), slotTime,userZone).toInstant();
54             }
55         }
56         if (daysToAdd < 0) {
57             daysToAdd += 7;
58         }
59
60         ZonedDateTime deliveryTime = ZonedDateTime.of(
61             now.toLocalDate().plusDays(daysToAdd),slotTime,userZone);
62         return deliveryTime.toInstant();
63     }
64
65     public Instant getNextDeliveryTime(Subscriptions subscription) {
66         ZoneId userZone=ZoneId.of(subscription.getEmployee().getTimezone());
67         if (subscription.getScheduleType() == ScheduleType.DAILY) {
68             return calculateDailyNextDelivery(subscription.getSlot(),userZone);
69         } else {
70             return calculateWeeklyNextDelivery(subscription.getSlot(),
71 subscription.getDayOfWeek(),userZone);
72         }
73     }
74 }
```

```
71     }
```

```
72 }
```

```
73 }
```

```
74
```

```

1  package com.example.EmployeeManagementSystem.Service;
2
3  import com.example.EmployeeManagementSystem.DTO.AssignDeviceRequest;
4  import com.example.EmployeeManagementSystem.DTO.DeviceAssignmentResponseDTO;
5  import com.example.EmployeeManagementSystem.DTO.DeviceDTO;
6  import com.example.EmployeeManagementSystem.DTO.DeviceResponseDTO;
7  import com.example.EmployeeManagementSystem.Entity.Device;
8  import com.example.EmployeeManagementSystem.Entity.DeviceAssignment;
9  import com.example.EmployeeManagementSystem.Entity.Employee;
10 import com.example.EmployeeManagementSystem.Entity.Vendor;
11 import com.example.EmployeeManagementSystem.Enum.AssignmentStatus;
12 import com.example.EmployeeManagementSystem.Enum.DeviceStatus;
13 import com.example.EmployeeManagementSystem.Enum.Role;
14 import com.example.EmployeeManagementSystem.Enum.Status;
15 import com.example.EmployeeManagementSystem.Exception.VendorNotFoundException;
16 import com.example.EmployeeManagementSystem.Repository.DeviceAssignmentRepo;
17 import com.example.EmployeeManagementSystem.Repository.DeviceRepository;
18 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
19 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
20 import com.example.EmployeeManagementSystem.Util.AuthUtil;
21 import org.springframework.security.core.Authentication;
22 import org.springframework.stereotype.Service;
23 import org.springframework.transaction.annotation.Transactional;
24
25 import java.time.LocalDate;
26 import java.time.LocalDateTime;
27 import java.util.List;
28 import java.util.Set;
29
30 @Service
31 public class DeviceService {
32     private final DeviceRepository deviceRepository;
33     private final VendorRepo vendorRepo;
34     private final DeviceAssignmentRepo deviceAssignmentRepo;
35     private final EmployeeRepo employeeRepo;
36
37     public DeviceService(DeviceRepository deviceRepository,
38                         VendorRepo vendorRepo,
39                         DeviceAssignmentRepo deviceAssignmentRepo,
40                         EmployeeRepo employeeRepo) {
41         this.deviceRepository = deviceRepository;
42         this.vendorRepo = vendorRepo;
43         this.deviceAssignmentRepo = deviceAssignmentRepo;
44         this.employeeRepo = employeeRepo;
45     }
46
47     public Device addDevice(DeviceDTO deviceDTO, Authentication authentication) {
48         String email = AuthUtil.extractEmail(authentication);
49         Vendor vendor = vendorRepo.findByEmail(email)
50             .orElseThrow(() -> new RuntimeException("Vendor not found"));
51
52         Device device = new Device();
53         device.setDeviceName(deviceDTO.getDeviceName());
54         device.setWarrantyExpiryDate(deviceDTO.getWarrantyExpiryDate());
55         device.setBrand(deviceDTO.getBrand());
56         device.setDeviceStatus(DeviceStatus.AVAILABLE);
57         device.setDeviceType(deviceDTO.getDeviceType());
58         device.setTechVendor(vendor);
59         device.setCreatedAt(LocalDateTime.now());
60         return deviceRepository.save(device);
61     }
62
63     public List<DeviceResponseDTO> getDevicesForLoggedInVendor(Authentication authentication) {
64         String email = AuthUtil.extractEmail(authentication);
65         Vendor vendor = vendorRepo.findByEmail(email)
66             .orElseThrow(() -> new RuntimeException("Vendor not found"));
67
68         return deviceRepository.findByTechVendorId(vendor.getId()).stream()
69             .map(this::toDeviceResponse)
70             .toList();
71     }

```

```

72
73     public List<DeviceResponseDTO> getDevicesForLoggedInEmployee(Authentication authentication) {
74         String email = AuthUtil.extractEmail(authentication);
75         Employee employee = employeeRepo.findByEmail(email)
76             .orElseThrow(() -> new RuntimeException("Employee not found"));
77
78         return deviceAssignmentRepo.findByAssignedToEmployeeIdAndStatus(employee.getEmployeeId(),
79 AssignmentStatus.ACTIVE)
80             .stream()
81             .map(DeviceAssignment::getDevice)
82             .map(this::toDeviceResponse)
83             .toList();
84     }
85
86     public List<DeviceResponseDTO> getUnassignedDevices() {
87         return deviceRepository.findByDeviceStatus(DeviceStatus.AVAILABLE).stream()
88             .map(this::toDeviceResponse)
89             .toList();
90     }
91
92     @Transactional
93     public DeviceAssignment assignDevice(Long deviceId, AssignDeviceRequest request) {
94         if (request.getEmployeeId() == null) {
95             throw new IllegalArgumentException("employeeId is required");
96         }
97
98         Device device = deviceRepository.findById(deviceId)
99             .orElseThrow(() -> new RuntimeException("Device not found"));
100
101         Employee employee = employeeRepo.findById(request.getEmployeeId())
102             .orElseThrow(() -> new RuntimeException("Employee not found"));
103
104         if (employee.getStatus() != Status.ACTIVE) {
105             throw new IllegalArgumentException("Device can only be assigned to an active user");
106         }
107
108         if (!Set.of(Role.EMPLOYEE, Role.MANAGER, Role.ADMIN).contains(employee.getRole())) {
109             throw new IllegalArgumentException("Devices can only be assigned to employees, managers, or
110 admins");
111         }
112
113         if (device.getDeviceStatus() != DeviceStatus.AVAILABLE) {
114             throw new IllegalArgumentException("Only available devices can be assigned");
115         }
116
117         deviceAssignmentRepo.findByDeviceIdAndStatus(deviceId, AssignmentStatus.ACTIVE)
118             .ifPresent(existingAssignment -> {
119                 throw new IllegalArgumentException("Device is already assigned");
120             });
121
122         DeviceAssignment assignment = new DeviceAssignment();
123         assignment.setDevice(device);
124         assignment.setAssignedTo(employee);
125         assignment.setStatus(AssignmentStatus.ACTIVE);
126         assignment.setAssignedDate(LocalDate.now());
127
128         DeviceAssignment savedAssignment = deviceAssignmentRepo.save(assignment);
129         device.setCurrentAssignment(savedAssignment);
130         device.setDeviceStatus(DeviceStatus.ASSIGNED);
131         deviceRepository.save(device);
132
133         return savedAssignment;
134     }
135
136     private DeviceResponseDTO toDeviceResponse(Device device) {
137         DeviceResponseDTO response = new DeviceResponseDTO();
138         response.setId(device.getId());
139         response.setDeviceName(device.getDeviceName());
140         response.setBrand(device.getBrand());
141         response.setWarrantyExpiryDate(device.getWarrantyExpiryDate());
142         response.setDeviceType(device.getDeviceType());
143         response.setDeviceStatus(device.getDeviceStatus());
144
145         if (device.getTechVendor() != null) {

```

```

146         response.setVendorName(device.getTechVendor().getName());
147     }
148
149     DeviceAssignment currentAssignment = device.getCurrentAssignment();
150     if (currentAssignment != null) {
151         response.setAssignedDate(currentAssignment.getAssignedDate());
152
153         Employee assignedEmployee = currentAssignment.getAssignedTo();
154         if (assignedEmployee != null) {
155             response.setAssignedEmployeeId(assignedEmployee.getEmployeeId());
156             response.setAssignedEmployeeName(assignedEmployee.getName());
157         }
158     }
159
160     return response;
161 }
162
163 public List<DeviceAssignmentResponseDTO> getDeviceAssignmentOfVendor(Authentication authentication) {
164     String email=AuthUtil.extractEmail(authentication);
165     Vendor vendor=vendorRepo.findByEmail(email).orElseThrow(
166         ()->new VendorNotFoundException("Vendor:"+email+" not found")
167     );
168     List<DeviceAssignment>
deviceAssignments=deviceAssignmentRepo.getAllAssignmentsByVendorId(vendor.getId());
169     return deviceAssignments.stream().map(deviceAssignment -> toAssignmentDTO(deviceAssignment)).toList();
170 }
171
172 private DeviceAssignmentResponseDTO toAssignmentDTO(DeviceAssignment a) {
173     DeviceAssignmentResponseDTO dto = new DeviceAssignmentResponseDTO();
174     dto.setId(a.getId());
175     dto.setStatus(a.getStatus());
176     dto.setAssignedDate(a.getAssignedDate());
177
178     if (a.getDevice() != null) {
179         dto.setDeviceId(a.getDevice().getId());
180         dto.setDeviceName(a.getDevice().getDeviceName());
181         dto.setDeviceBrand(a.getDevice().getBrand());
182         dto.setDeviceType(a.getDevice().getDeviceType());
183     }
184
185     if (a.getAssignedTo() != null) {
186         dto.setEmployeeId(a.getAssignedTo().getEmployeeId());
187         dto.setEmployeeName(a.getAssignedTo().getName()); // adjust to your Employee field
188     }
189
190     return dto;
191 }
192 }
193

```

```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.DTO.*;
4 import com.example.EmployeeManagementSystem.Entity.Employee;
5 import com.example.EmployeeManagementSystem.Entity.LeaveRequest;
6 import com.example.EmployeeManagementSystem.Enum.Role;
7 import com.example.EmployeeManagementSystem.Enum.Status;
8 import com.example.EmployeeManagementSystem.Exception.EmployeeNotFound;
9 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
10 import com.example.EmployeeManagementSystem.Repository.LeaveRequestRepo;
11 import com.example.EmployeeManagementSystem.Repository.RefreshTokenRepository;
12 import com.example.EmployeeManagementSystem.Util.AuthUtil;
13 import org.slf4j.Logger;
14 import org.slf4j.LoggerFactory;
15 import org.springframework.http.ResponseEntity;
16 import org.springframework.security.core.Authentication;
17 import org.springframework.security.crypto.password.PasswordEncoder;
18 import org.springframework.stereotype.Service;
19 import org.springframework.transaction.annotation.Transactional;
20
21 import java.time.ZoneId;
22 import java.util.List;
23 import java.util.stream.Collectors;
24
25 @Service
26 public class EmployeeService {
27
28     private static final Logger log = LoggerFactory.getLogger(EmployeeService.class);
29
30     private final EmployeeRepo employeeRepo;
31     private final LeaveRequestRepo leaveRequestRepo;
32     private final PasswordEncoder passwordEncoder;
33     private final RefreshTokenRepository refreshTokenRepository; // FIX: added
34
35     public EmployeeService(EmployeeRepo employeeRepo,
36                             LeaveRequestRepo leaveRequestRepo,
37                             PasswordEncoder passwordEncoder,
38                             RefreshTokenRepository refreshTokenRepository) {
39         this.employeeRepo = employeeRepo;
40         this.leaveRequestRepo = leaveRequestRepo;
41         this.passwordEncoder = passwordEncoder;
42         this.refreshTokenRepository = refreshTokenRepository; // FIX: injected
43     }
44
45     public List<Employee> getAllEmployees() {
46         return employeeRepo.findAll();
47     }
48
49     public Employee createEmployee(AdminEmployeeDTO employeeDTO) {
50         log.info("Creating new employee with email: {}", employeeDTO.getEmail());
51         var employee = new Employee();
52         if (employeeDTO.getName() != null) employee.setName(employeeDTO.getName());
53         if (employeeDTO.getEmail() != null) employee.setEmail(employeeDTO.getEmail());
54         if (employeeDTO.getTimezone() != null && isValidTimezone(employeeDTO.getTimezone())) {
55             employee.setTimezone(employeeDTO.getTimezone());
56         } else {
57             employee.setTimezone("UTC");
58         }
59         employee.setRole(Role.EMPLOYEE);
60         if (employeeDTO.getPassword() == null) {
61             throw new RuntimeException("Password is required");
62         }
63         employee.setPassword(passwordEncoder.encode(employeeDTO.getPassword()));
64         employee.setGender(employeeDTO.getGender());
65         Employee manager=employeeRepo.findById(employeeDTO.getManagerId()).orElseThrow(
66             ()->new EmployeeNotFound("Employee with not found"+ employeeDTO.getManagerId())
67         );
68         employee.setDept(manager.getDept());
69         employee.setManager(manager);
70         return employeeRepo.save(employee);
71     }

```

```

72
73     public Employee createManager(EmployeeDTO employeeDTO) {
74         log.info("Creating new manager with email: {}", employeeDTO.getEmail());
75         var employee = new Employee();
76         if (employeeDTO.getName() != null) employee.setName(employeeDTO.getName());
77         if (employeeDTO.getEmail() != null) employee.setEmail(employeeDTO.getEmail());
78         if (employeeDTO.getDept() != null) employee.setDept(employeeDTO.getDept());
79         if (employeeDTO.getTimezone() != null && isValidTimezone(employeeDTO.getTimezone())) {
80             employee.setTimezone(employeeDTO.getTimezone());
81         } else {
82             employee.setTimezone("UTC");
83         }
84         employee.setRole(Role.MANAGER);
85         if (employeeDTO.getPassword() == null) {
86             throw new RuntimeException("Password is required");
87         }
88         employee.setPassword(passwordEncoder.encode(employeeDTO.getPassword()));
89         employee.setGender(employeeDTO.getGender());
90         return employeeRepo.save(employee);
91     }
92
93     public Employee updateEmployee(long id, EmployeeDTO employee) {
94         var updateEmployee = employeeRepo.findById(id).orElseThrow(
95             () -> new EmployeeNotFound("Employee with id:" + id + " not found")
96         );
97         if (employee.getName() != null)
98             updateEmployee.setName(employee.getName());
99         if (employee.getEmail() != null)
100             updateEmployee.setEmail(employee.getEmail());
101         if (employee.getDept() != null)
102             updateEmployee.setDept(employee.getDept());
103         if (employee.getTimezone() != null && isValidTimezone(employee.getTimezone())) {
104             updateEmployee.setTimezone(employee.getTimezone());
105         }
106         return employeeRepo.save(updateEmployee);
107     }
108
109     /**
110      * FIX: Delete order must be:
111      * 1. refresh_token (FK employee)
112      * 2. leave_request (FK employee)
113      * 3. employee
114      *
115      * Previously only leave_request was deleted first, causing a
116      * FK constraint failure on refresh_token when deleting the employee.
117      */
118     @Transactional
119     public void deleteEmployee(long id) {
120         var employee = employeeRepo.findById(id).orElseThrow(
121             () -> new EmployeeNotFound("Employee with id:" + id + " not found")
122         );
123
124         // Step 1: delete refresh tokens for this employee
125         refreshTokenRepository.deleteByEmployee(employee);
126
127         // Step 2: delete leave requests for this employee
128         List<LeaveRequest> leaveRequests =
129             leaveRequestRepo.findByEmployeeOrderByStartDateDesc(employee);
130         leaveRequestRepo.deleteAll(leaveRequests);
131
132         // Step 3: delete the employee
133         employeeRepo.delete(employee);
134     }
135
136     public ResponseEntity<String> inactivateUser(long employeeId) {
137         var employee = employeeRepo.findById(employeeId).orElseThrow(
138             () -> new EmployeeNotFound("Employee with id:" + employeeId + " not found")
139         );
140         if (employee.getStatus() == Status.INACTIVE) {
141             return ResponseEntity.badRequest()
142                 .body("Status of the employee with id: " + employeeId + " is already set to inactive");
143         }
144         employee.setStatus(Status.INACTIVE);
145         employeeRepo.save(employee);
146         return ResponseEntity.ok("Status of the employee with id: " + employeeId + " is set to inactive");
147     }

```



```

148
149  /** Validate that the given string is a recognized IANA timezone id. */
150  private boolean isValidTimezone(String tz) {
151      try {
152          ZoneId.of(tz);
153          return true;
154      } catch (Exception e) {
155          return false;
156      }
157  }
158
159  public Employee getAccount(Authentication authentication) {
160      String email = AuthUtil.extractEmail(authentication);
161      Employee employee = employeeRepo.findByEmail(email).orElseThrow(
162          () -> new EmployeeNotFound("Employee :" + email + " NOT FOUND")
163      );
164      return employee;
165  }
166
167  public EmployeeAttendanceResponseDTO getEmployeeAttendanceOverview() {
168      List<Employee> employees = employeeRepo.findAll();
169
170      List<EmployeeStatusDTO> workingEmployees = employees.stream()
171          .filter(e -> e.getStatus() == Status.ACTIVE)
172          .map(this::toStatusDTO)
173          .collect(Collectors.toList());
174
175      List<EmployeeStatusDTO> onLeaveEmployees = employees.stream()
176          .filter(e -> e.getStatus() == Status.ON_LEAVE)
177          .map(this::toStatusDTO)
178          .collect(Collectors.toList());
179
180      EmployeeAttendanceResponseDTO response = new EmployeeAttendanceResponseDTO();
181      response.setWorkingCount(workingEmployees.size());
182      response.setOnLeaveCount(onLeaveEmployees.size());
183      response.setWorkingEmployees(workingEmployees);
184      response.setOnLeaveEmployees(onLeaveEmployees);
185      return response;
186  }
187
188  private EmployeeStatusDTO toStatusDTO(Employee employee) {
189      EmployeeStatusDTO dto = new EmployeeStatusDTO();
190      dto.setEmployeeId(employee.getEmployeeId());
191      dto.setName(employee.getName());
192      dto.setEmail(employee.getEmail());
193      dto.setDept(employee.getDept());
194      dto.setRole(employee.getRole());
195      return dto;
196  }
197
198  public ResponseEntity<List<ManagerDTO>> getAllManagers() {
199      List<ManagerDTO> managers = employeeRepo.findByRole(Role.MANAGER).stream()
200          .map(this::toManagerDTO)
201          .collect(Collectors.toList());
202      return ResponseEntity.ok(managers);
203  }
204
205  private ManagerDTO toManagerDTO(Employee employee) {
206      ManagerDTO dto = new ManagerDTO();
207      dto.setManagerId(employee.getEmployeeId());
208      dto.setName(employee.getName());
209      dto.setEmail(employee.getEmail());
210      dto.setDept(employee.getDept());
211      return dto;
212  }
213 }
214

```

```
1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.DTO.HolidayDTO;
4 import com.example.EmployeeManagementSystem.Entity.FixedHoliday;
5 import com.example.EmployeeManagementSystem.Repository.FixedHolidayRepository;
6 import org.springframework.stereotype.Service;
7
8 import java.time.LocalDate;
9 import java.util.List;
10 @Service
11 public class HolidayService {
12     private final FixedHolidayRepository holidayRepository;
13
14     public HolidayService(FixedHolidayRepository holidayRepository) {
15         this.holidayRepository = holidayRepository;
16     }
17
18     // Admin adds a holiday
19     public FixedHoliday addHoliday(HolidayDTO dto) {
20         FixedHoliday h = new FixedHoliday();
21         h.setName(dto.getName());
22         h.setDate(LocalDate.parse(dto.getDate()));
23         h.setDescription(dto.getDescription());
24         h.setYear(LocalDate.parse(dto.getDate()).getYear());
25         return holidayRepository.save(h);
26     }
27
28     // Everyone can view holidays
29     public List<FixedHoliday> getHolidaysByYear(int year) {
30         return holidayRepository.findByYear(year);
31     }
32
33     // Admin deletes a holiday
34     public void deleteHoliday(Long id) {
35         holidayRepository.deleteById(id);
36     }
37 }
38
39
```

```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
5 import com.example.EmployeeManagementSystem.Entity.LeaveType;
6 import com.example.EmployeeManagementSystem.Enum.Gender;
7 import com.example.EmployeeManagementSystem.Enum.Status;
8 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
9 import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
10 import com.example.EmployeeManagementSystem.Repository.LeaveTypeRepository;
11 import org.slf4j.Logger;
12 import org.slf4j.LoggerFactory;
13 import org.springframework.stereotype.Service;
14 import org.springframework.transaction.annotation.Transactional;
15
16 import java.math.BigDecimal;
17 import java.time.LocalDate;
18 import java.util.List;
19 import java.util.Set;
20
21 @Service
22 public class LeaveAccrualService {
23
24     private static final Logger log = LoggerFactory.getLogger(LeaveAccrualService.class);
25     private static final BigDecimal ONE_DAY = BigDecimal.ONE;
26     private static final Set<String> SUPPORTED_LEAVE_TYPES = Set.of("SICK", "CASUAL");
27
28     private final EmployeeRepo employeeRepository;
29     private final LeaveTypeRepository leaveTypeRepository;
30     private final LeaveEntitlementRepository entitlementRepository;
31
32     public LeaveAccrualService(EmployeeRepo employeeRepository,
33                               LeaveTypeRepository leaveTypeRepository,
34                               LeaveEntitlementRepository entitlementRepository) {
35         this.employeeRepository = employeeRepository;
36         this.leaveTypeRepository = leaveTypeRepository;
37         this.entitlementRepository = entitlementRepository;
38     }
39
40     @Transactional
41     public void runMonthlyAccrual(LocalDate accrualDate) {
42         int year = accrualDate.getYear();
43
44         List<Employee> activeEmployees = employeeRepository.findByStatus(Status.ACTIVE);
45         List<LeaveType> allLeaveTypes = leaveTypeRepository.findAll();
46
47         log.info("Starting monthly accrual for {} - {} active employees",
48                 accrualDate, activeEmployees.size());
49
50         for (Employee employee : activeEmployees) {
51             for (LeaveType leaveType : allLeaveTypes) {
52
53                 if (!SUPPORTED_LEAVE_TYPES.contains(leaveType.getName())) {
54                     continue;
55                 }
56                 if (!isEligible(employee, leaveType, accrualDate)) {
57                     continue;
58                 }
59
60                 // Find or create entitlement row for this employee/type/year
61                 LeaveEntitlement entitlement = entitlementRepository
62                     .findByEmployeeIdAndLeaveTypeIdAndYear(
63                         employee.getEmployeeId(), leaveType.getId(), year)
64                     .orElseGet(() -> createNewEntitlement(employee, leaveType, year));
65
66                 // Add 1 day accrual
67                 entitlement.setAccruedThisYear(
68                     entitlement.getAccruedThisYear().add(ONE_DAY)
69                 );
70
71                 // Keep closingBalance in sync so balance checks are never stale

```

```

72         entitlement.setClosingBalance(
73             entitlement.getOpeningBalance()
74                 .add(entitlement.getAccruedThisYear())
75                 .subtract(entitlement.getUsedThisYear())
76         );
77
78         entitlementRepository.save(entitlement);
79         log.debug("Accrued 1 {} day for employee {} (year {})",
80             leaveType.getName(), employee.getEmployeeId(), year);
81     }
82 }
83
84     log.info("Monthly accrual complete for {}", accrualDate);
85 }
86
87 // -----
88 // Eligibility check – single place for all rules
89 // -----
90 private boolean isEligible(Employee employee, LeaveType leaveType, LocalDate accrualDate) {
91     // 1. Gender restriction (Maternity = female only)
92     if (leaveType.isGenderRestricted()) {
93         Gender restriction = leaveType.getGenderRestriction();
94         if (restriction != null && restriction != employee.getGender()) {
95             return false;
96         }
97     }
98 }
99
100 // 2. Employee must have joined ON OR BEFORE the accrual date
101 //     (no accrual for employees joining after the 1st of this month)
102 if (employee.getJoined_at() != null && employee.getJoined_at().isAfter(accrualDate)) {
103     return false;
104 }
105
106     return true;
107 }
108
109 // -----
110 // Creates a fresh entitlement row, carrying forward previous year's
111 // closing balance if this is the first row of the year
112 // -----
113 private LeaveEntitlement createNewEntitlement(Employee employee,
114     LeaveType leaveType,
115     int year) {
116     LeaveEntitlement entitlement = new LeaveEntitlement();
117     entitlement.setEmployee(employee);
118     entitlement.setLeaveType(leaveType);
119     entitlement.setYear(year);
120
121     // Carry forward last year's closing balance (if the type supports it)
122     if (leaveType.isCarriesForward()) {
123         BigDecimal lastYearClosing = entitlementRepository
124             .findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
125                 employee.getEmployeeId(), leaveType.getId(), year - 1)
126             .map(LeaveEntitlement::getClosingBalance)
127             .orElse(BigDecimal.ZERO);
128
129         entitlement.setOpeningBalance(lastYearClosing);
130     }
131
132     return entitlementRepository.save(entitlement);
133 }
134 }
135

```

```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.DTO.LeaveDashboardResponse;
4 import com.example.EmployeeManagementSystem.Entity.Employee;
5 import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
6 import com.example.EmployeeManagementSystem.Entity.LeaveType;
7 import com.example.EmployeeManagementSystem.Entity.LeaveWarning;
8 import com.example.EmployeeManagementSystem.Enum.Gender;
9 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
10 import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
11 import com.example.EmployeeManagementSystem.Repository.LeaveTypeRepository;
12 import com.example.EmployeeManagementSystem.Repository.LeaveWarningRepository;
13 import org.springframework.security.access.AccessDeniedException;
14 import org.springframework.stereotype.Service;
15 import org.springframework.transaction.annotation.Transactional;
16
17 import java.math.BigDecimal;
18 import java.time.LocalDate;
19 import java.util.Set;
20 import java.util.LinkedHashMap;
21 import java.util.List;
22 import java.util.Map;
23
24 @Service
25 public class LeaveDashboardService {
26
27     private static final Set<String> SUPPORTED_LEAVE_TYPES = Set.of("SICK", "CASUAL", "MATERNITY");
28
29     private final EmployeeRepo employeeRepository;
30     private final LeaveEntitlementRepository leaveEntitlementRepository;
31     private final LeaveTypeRepository leaveTypeRepository;
32     private final LeaveWarningRepository warningRepository;
33
34     public LeaveDashboardService(EmployeeRepo employeeRepository,
35                                 LeaveEntitlementRepository leaveEntitlementRepository,
36                                 LeaveTypeRepository leaveTypeRepository,
37                                 LeaveWarningRepository warningRepository) {
38         this.employeeRepository = employeeRepository;
39         this.leaveEntitlementRepository = leaveEntitlementRepository;
40         this.leaveTypeRepository = leaveTypeRepository;
41         this.warningRepository = warningRepository;
42     }
43
44     @Transactional
45     public LeaveDashboardResponse getDashboard(Long requestedEmployeeId,
46                                              Long callerEmployeeId,
47                                              String callerRole) {
48         assertAccess(requestedEmployeeId, callerEmployeeId, callerRole);
49
50         Employee employee = employeeRepository.findById(requestedEmployeeId)
51             .orElseThrow(() -> new IllegalArgumentException(
52                 "Employee not found: " + requestedEmployeeId));
53
54         int year = LocalDate.now().getYear();
55
56         ensureCurrentYearEntitlements(employee, year);
57         List<LeaveEntitlement> entitlements =
58             leaveEntitlementRepository.findByEmployeeIdAndYear(requestedEmployeeId, year);
59
60         List<LeaveWarning> warnings =
61             warningRepository.findByEmployeeIdAndIsReadFalseOrderByWarningDateDesc(
62                 requestedEmployeeId);
63
64         LeaveDashboardResponse.LeaveBalanceDto sick = buildBalanceDto(entitlements, "SICK");
65         LeaveDashboardResponse.LeaveBalanceDto casual = buildBalanceDto(entitlements, "CASUAL");
66         LeaveDashboardResponse.LeaveBalanceDto maternity = buildBalanceDto(entitlements, "MATERNITY");
67
68         Map<String, LeaveDashboardResponse.LeaveBalanceDto> leaveBalances = buildBalanceMap(entitlements);
69         BigDecimal carryTotal = entitlements.stream()
70             .filter(e -> isSupportedLeaveType(e.getLeaveType()))
71             .filter(e -> e.getLeaveType().isCarriesForward())

```

```

72         .map(LeaveEntitlement::getAvailableBalance)
73         .reduce(BigDecimal.ZERO, BigDecimal::add);
74
75     LeaveDashboardResponse response = new LeaveDashboardResponse();
76     response.setEmployeeId(employee.getEmployeeId());
77     response.setEmployeeName(employee.getName());
78     response.setYear(year);
79     response.setSick(sick);
80     response.setCasual(casual);
81     response.setMaternity(maternity);
82     response.setLeaveBalances(leaveBalances);
83     response.setCarryForwardMeter(new LeaveDashboardResponse.CarryForwardMeterDto(carryTotal, 30));
84     response.setWarnings(warnings.stream()
85         .map(w -> new LeaveDashboardResponse.WarningDto(
86             w.getId(),
87             w.getWarningType().name(),
88             w.getMessage(),
89             w.getWarningDate().toString()))
90         .toList());
91
92     return response;
93 }
94
95 // Mark warning read
96
97 @Transactional
98 public void markWarningRead(Long warningId, Long callerEmployeeId, String callerRole) {
99     LeaveWarning warning = warningRepository.findById(warningId)
100         .orElseThrow(() -> new IllegalArgumentException("Warning not found: " + warningId));
101
102     assertAccess(warning.getEmployee().getEmployeeId(), callerEmployeeId, callerRole);
103
104     warning.setRead(true);
105     warningRepository.save(warning);
106 }
107
108 // Team dashboard (manager/admin)
109
110 @Transactional
111 public List<LeaveDashboardResponse> getTeamDashboard(Long managerId, String callerRole) {
112     if (!"MANAGER".equals(callerRole) && !"ADMIN".equals(callerRole)) {
113         throw new AccessDeniedException("Only managers and admins can view team dashboards");
114     }
115
116     List<Employee> team = employeeRepository.findByManagerEmployeeId(managerId);
117     return team.stream()
118         .map(emp -> getDashboard(emp.getEmployeeId(), managerId, callerRole))
119         .toList();
120 }
121
122 // Helpers
123
124 private void ensureCurrentYearEntitlements(Employee employee, int year) {
125     List<LeaveType> leaveTypes = leaveTypeRepository.findAll();
126
127     for (LeaveType leaveType : leaveTypes) {
128         if (!isSupportedLeaveType(leaveType)) {
129             continue;
130         }
131         if (!isEligible(employee, leaveType)) {
132             continue;
133         }
134
135         leaveEntitlementRepository
136             .findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
137                 employee.getEmployeeId(), leaveType.getId(), year)
138             .orElseGet(() -> createEntitlement(employee, leaveType, year));
139     }
140 }
141
142 private boolean isEligible(Employee employee, LeaveType leaveType) {
143     if (!leaveType.isGenderRestricted()) {
144         return true;
145     }
146
147     Gender restriction = leaveType.getGenderRestriction();

```

```

148         return restriction == null || restriction == employee.getGender();
149     }
150
151     private LeaveEntitlement createEntitlement(Employee employee, LeaveType leaveType, int year) {
152         LeaveEntitlement entitlement = new LeaveEntitlement();
153         entitlement.setEmployee(employee);
154         entitlement.setLeaveType(leaveType);
155         entitlement.setYear(year);
156
157         if (leaveType.isCarriesForward()) {
158             BigDecimal lastYearClosing = leaveEntitlementRepository
159                 .findByEmployeeIdAndLeaveTypeIdAndYear(
160                     employee.getId(), leaveType.getId(), year - 1)
161                 .map(LeaveEntitlement::getClosingBalance)
162                 .orElse(BigDecimal.ZERO);
163             entitlement.setOpeningBalance(lastYearClosing);
164         }
165
166         return leaveEntitlementRepository.save(entitlement);
167     }
168
169     private Map<String, LeaveDashboardResponse.LeaveBalanceDto> buildBalanceMap(
170         List<LeaveEntitlement> entitlements) {
171         Map<String, LeaveDashboardResponse.LeaveBalanceDto> balances = new LinkedHashMap<>();
172         for (LeaveEntitlement entitlement : entitlements) {
173             String typeName = entitlement.getLeaveType().getName();
174             if (!SUPPORTED_LEAVE_TYPES.contains(typeName)) {
175                 continue;
176             }
177             balances.put(typeName, toBalanceDto(entitlement, typeName));
178         }
179         return balances;
180     }
181
182     private LeaveDashboardResponse.LeaveBalanceDto buildBalanceDto(List<LeaveEntitlement> entitlements,
183                                                                     String typeName) {
184         return entitlements.stream()
185             .filter(e -> typeName.equalsIgnoreCase(e.getLeaveType().getName()))
186             .findFirst()
187             .map(e -> toBalanceDto(e, typeName))
188             .orElse(null); // null = leave type not applicable for this employee
189     }
190
191     private LeaveDashboardResponse.LeaveBalanceDto toBalanceDto(LeaveEntitlement entitlement,
192                                                                 String typeName) {
193         return new LeaveDashboardResponse.LeaveBalanceDto(
194             typeName,
195             entitlement.getOpeningBalance(),
196             entitlement.getAccruedThisYear(),
197             entitlement.getUsedThisYear(),
198             entitlement.getAvailableBalance(),
199             entitlement.getLeaveType().isCarriesForward());
200     }
201
202     private boolean isSupportedLeaveType(LeaveType leaveType) {
203         return leaveType != null && SUPPORTED_LEAVE_TYPES.contains(leaveType.getName());
204     }
205
206     private void assertAccess(Long targetId, Long callerId, String role) {
207         boolean isSelf = targetId.equals(callerId);
208         boolean isManager = "MANAGER".equals(role) || "ADMIN".equals(role);
209         if (!isSelf && !isManager) {
210             throw new AccessDeniedException(
211                 "Employee " + callerId + " cannot view dashboard for " + targetId);
212         }
213     }
214 }
215

```

```

1  package com.example.EmployeeManagementSystem.Service;
2
3  import com.example.EmployeeManagementSystem.DTO.*;
4  import com.example.EmployeeManagementSystem.Entity.Employee;
5  import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
6  import com.example.EmployeeManagementSystem.Entity.LeaveRequest;
7  import com.example.EmployeeManagementSystem.Enum.*;
8  import com.example.EmployeeManagementSystem.Exception.*;
9  import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
10 import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
11 import com.example.EmployeeManagementSystem.Repository.LeaveRequestRepo;
12 import com.example.EmployeeManagementSystem.Repository.LeaveTypeRepository;
13 import com.example.EmployeeManagementSystem.Util.AuthUtil;
14 import org.slf4j.Logger;
15 import com.example.EmployeeManagementSystem.Enum.LeaveType;
16 import org.slf4j.LoggerFactory;
17 import org.springframework.http.HttpStatus;
18 import org.springframework.http.ResponseEntity;
19 import org.springframework.security.core.Authentication;
20 import org.springframework.stereotype.Service;
21 import org.springframework.transaction.annotation.Transactional;
22
23 import java.math.BigDecimal;
24 import java.time.LocalDate;
25 import java.time.ZoneId;
26 import java.time.temporal.ChronoUnit;
27 import java.util.ArrayList;
28 import java.util.List;
29 import java.util.Map;
30 import java.util.stream.Collectors;
31
32 @Service
33 public class LeaveRequestService {
34     private final LeaveRequestRepo leaveRequestRepo;
35     private final EmployeeRepo employeeRepo;
36     private final LeaveTypeRepository leaveTypeRepository;
37     private final LeaveEntitlementRepository leaveEntitlementRepository;
38     private final NotificationService notificationService;
39     private static final Logger log = LoggerFactory.getLogger(LeaveRequestService.class);
40
41     public LeaveRequestService(LeaveRequestRepo leaveRequestRepo,
42                               EmployeeRepo employeeRepo,
43                               LeaveTypeRepository leaveTypeRepository,
44                               LeaveEntitlementRepository leaveEntitlementRepository,
45                               NotificationService notificationService) {
46         this.leaveRequestRepo = leaveRequestRepo;
47         this.employeeRepo = employeeRepo;
48         this.leaveTypeRepository = leaveTypeRepository;
49         this.leaveEntitlementRepository = leaveEntitlementRepository;
50         this.notificationService=notificationService;
51     }
52
53     public List<LeaveResponseDTO> getAllTheLeaveRequest(){
54         List<LeaveRequest> requestList=leaveRequestRepo.findAll();
55         List<LeaveResponseDTO> dtos=new ArrayList<>();
56         for(LeaveRequest request:requestList){
57             dtos.add(convertToDTO(request));
58         }
59         return dtos;
60     }
61
62
63     public ResponseEntity<?> createRequest(Authentication authentication, LeaveRequestDTO requestDTO){
64         var leaveRequest=new LeaveRequest();
65         String email= AuthUtil.extractEmail(authentication);
66         //Validate employee exists and is active
67         var employee=employeeRepo.findByEmail(email).orElseThrow(()->
68             new EmployeeNotFound("Employee with name: "+email+" is not found")
69         );
70         if (employee.getStatus() != Status.ACTIVE) {
71             return ResponseEntity.badRequest()

```



```

72         .body(Map.of("error", "Only active employees can apply for leave"));
73     }
74
75     if (requestDTO.getLeaveType() == LeaveType.MATERNITY) {
76         if(employee.getGender()==Gender.M){
77             return ResponseEntity.status(HttpStatus.BAD_REQUEST)
78                 .body("Maternity leave is not applicable");
79         }
80         return ResponseEntity.status(HttpStatus.BAD_REQUEST)
81             .body("Please use the maternity leave application process.");
82     }
83
84     leaveRequest.setEmployee(employee);
85     //Checking if the start date is before today
86     String timezone= employee.getTimezone();
87     ZoneId zoneId=ZoneId.of(timezone != null && !timezone.isBlank()
88         ? timezone
89         : "UTC");
90     LocalDate today = LocalDate.now(zoneId);
91     if (requestDTO.getStartDate().isBefore(today)) {
92         throw new InvalidStartDateException("Start date cannot be before current date");
93     }
94     //Checking if the end date is before start date
95     if (requestDTO.getEndDate().isBefore(requestDTO.getStartDate())) {
96         throw new InvalidEndDateException("End date must be equal to or greater than start date");
97     }
98     //optional
99     long daysRequested = ChronoUnit.DAYS.between(requestDTO.getStartDate(), requestDTO.getEndDate()) +
100     1;
101     if (daysRequested > 30) {
102         return ResponseEntity.badRequest()
103             .body(Map.of("error", "Leave cannot exceed 30 consecutive days"));
104     }
105     // ----- SICK -----
106     if (requestDTO.getLeaveType() == LeaveType.SICK) {
107         if (daysRequested > 1) {
108             return ResponseEntity.badRequest()
109                 .body(Map.of("error", "Sick leave is limited to 1 day per month."));
110         }
111         long sickDaysUsedThisMonth = leaveRequestRepo.countApprovedSickDaysInMonth(
112             employee, today.getYear(), today.getMonthValue());
113         var sickLeaveType = leaveTypeRepository.findByName("SICK")
114             .orElseThrow(() -> new RuntimeException("SICK leave type not configured"));
115         var entitlement = leaveEntitlementRepository
116             .findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
117                 employee.getEmployeeId(),
118                 sickLeaveType.getId(),
119                 today.getYear())
120             .orElseGet(() -> createEntitlement(employee, sickLeaveType, today.getYear()));
121         BigDecimal availableSick = entitlement.getAvailableBalance();
122         if (sickDaysUsedThisMonth >= 1 || availableSick.compareTo(BigDecimal.ONE) < 0) {
123             requestDTO.setLeaveType(LeaveType.UNPAID);
124         }
125     }
126
127 // ----- CASUAL -----
128 if (requestDTO.getLeaveType() == LeaveType.CASUAL) {
129     com.example.EmployeeManagementSystem.Entity.LeaveType
130     leaveType=leaveTypeRepository.findByName("CASUAL").orElseThrow(
131         ()->new RuntimeException("Leave type Not found")
132     );
133     int leaveYear=requestDTO.getStartDate().getYear();
134     LeaveEntitlement
135     leaveEntitlement=leaveEntitlementRepository.findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
136         employee.getEmployeeId(),
137         leaveType.getId(),
138         leaveYear
139     ).orElseGet(()->createEntitlement(employee, leaveType, leaveYear));
140
141
142
143
144

```

```

145         BigDecimal availableCasual = leaveEntitlement.getAvailableBalance(); // opening + accrued -
used
146         BigDecimal requestedCasual = BigDecimal.valueOf(daysRequested);
147         if (availableCasual.compareTo(BigDecimal.ZERO) <= 0) {
148             return ResponseEntity.badRequest()
149                 .body(Map.of("error", "No casual leave balance available."));
150         }
151
152         if (requestedCasual.compareTo(availableCasual) > 0) {
153             return ResponseEntity.badRequest()
154                 .body(Map.of(
155                     "error",
156                     "Requested casual leave exceeds available balance. Available: " +
availableCasual
157                 ));
158         }
159     }
160 }
161
162 // ----- UNPAID cap -----
163 if (requestDTO.getLeaveType() == LeaveType.UNPAID) {
164     // Max unpaid days = number of leave types exhausted this month
165     // (1 for exhausted sick + 1 for exhausted casual = max 2)
166     long sickUsedThisMonth = leaveRequestRepo.countApprovedSickDaysInMonth(
167         employee, today.getYear(), today.getMonthValue());
168     long casualBalance = getCasualBalance(employee, today.getYear());
169
170     int maxUnpaid = 0;
171     if (sickUsedThisMonth >= 1) maxUnpaid++; // sick exhausted
172     if (casualBalance <= 0) maxUnpaid++; // casual exhausted
173
174     long unpaidDaysThisMonth = leaveRequestRepo.countUnpaidDaysInMonth(
175         employee, today.getYear(), today.getMonthValue());
176
177     if (maxUnpaid == 0) {
178         return ResponseEntity.badRequest()
179             .body(Map.of("error",
180                 "You still have paid leave available. Please use sick or casual leave
first."));
181     }
182     if (unpaidDaysThisMonth >= maxUnpaid) {
183         return ResponseEntity.badRequest()
184             .body(Map.of("error",
185                 "Unpaid leave limit (" + maxUnpaid + " day(s) this month) already
reached."));
186     }
187 }
188
189 long duplicateCount= leaveRequestRepo.checkDuplicate(
190     employee.getEmployeeId(),
191     requestDTO.getStartDate(),
192     requestDTO.getEndDate(),
193     LeaveStatus.PENDING.name()
194 );
195 if(duplicateCount>0){
196     throw new DuplicateRequestException("Duplicate leave request");
197 }
198
199 //checking if there is any overlapping leave request of that employee
200 long count= leaveRequestRepo.countOverlappingLeave(
201     employee.getEmployeeId(),
202     requestDTO.getStartDate(),
203     requestDTO.getEndDate()
204 );
205 if(count>0){
206     throw new OverlappingLeaveException("Overlapping leave exists");
207 }
208 leaveRequest.setStartDate(requestDTO.getStartDate());
209 leaveRequest.setEndDate(requestDTO.getEndDate());
210
211 if(requestDTO.getLeaveType()==null){
212     return ResponseEntity.status(HttpStatus.BAD_REQUEST)
213         .body("Leave type is required..");
214 }
215 LeaveType originalLeaveType = requestDTO.getLeaveType();
216 leaveRequest.setLeaveType(requestDTO.getLeaveType());

```

```

217         if(requestDTO.getReason()!=null){
218             leaveRequest.setReason(requestDTO.getReason());
219         }
220         LeaveRequest savedRequest=leaveRequestRepo.save(leaveRequest);
221
222         Employee manager = employee.getManager(); // assuming getManager() exists
223         if (manager != null) {
224             notificationService.notify(
225                 manager,
226                 employee.getName() + " submitted a " + savedRequest.getLeaveType() ,
227                 "LEAVE_REQUEST"
228             );
229         }
230         // After the existing manager notification block
231         employeeRepo.findByRole(Role.ADMIN).forEach(admin ->
232             notificationService.notify(
233                 admin,
234                 employee.getName() + " submitted a " + requestDTO.getLeaveType() + " leave request
235                 (" +
236                     requestDTO.getStartDate() + " " + requestDTO.getEndDate() + ")",
237                     "LEAVE_REQUEST"
238                 )
239             );
240         LeaveResponseDTO responseDTO = convertToDTO(savedRequest);
241         if (savedRequest.getLeaveType() == LeaveType.UNPAID && savedRequest.getLeaveType() !=
242         originalLeaveType) {
243             Map<String, Object> body = new java.util.LinkedHashMap<>();
244             body.put("leave", responseDTO);
245             body.put("warning", "Your " + originalLeaveType.name().toLowerCase() +
246                 " leave quota for this month is exhausted. Request recorded as Unpaid leave.");
247             return ResponseEntity.status(HttpStatus.CREATED).body(body);
248         }
249         return ResponseEntity.status(HttpStatus.CREATED).body(responseDTO);
250     }
251     //Only for manager
252     public List<LeaveResponseDTO> getAllThePendingLeaveRequests(){
253         List<LeaveRequest> requestList=leaveRequestRepo.findByStatus(LeaveStatus.PENDING);
254         List<LeaveResponseDTO> responseDTOS=new ArrayList<>();
255         for(LeaveRequest request:requestList){
256             responseDTOS.add(convertToDTO(request));
257         }
258         return responseDTOS;
259     }
260     @Transactional
261     public ResponseEntity<?> updateLeaveRequestStatus(ActionDTO actionDTO,Authentication authentication){
262         String email= AuthUtil.extractEmail(authentication);
263         Employee manager=employeeRepo.findByEmail(email).orElseThrow(
264             ()->new EmployeeNotFound("Manager with name: "+email+" Not Found")
265         );
266         if (actionDTO.getAction() == null) {
267             return ResponseEntity.badRequest().body("Action is required");
268         }
269         log.info("Updating leave request status for id: {}", actionDTO.getLeaveRequestId());
270
271         var leaveRequest=leaveRequestRepo.findById(actionDTO.getLeaveRequestId()).orElseThrow(
272             ()-> new LeaveRequestNotFoundException("LeaveRequest with
273             id:"+actionDTO.getLeaveRequestId()+" not found")
274         );
275         if (leaveRequest.getStatus() != LeaveStatus.PENDING) {
276             return ResponseEntity.badRequest()
277                 .body("Only PENDING requests can be approved or rejected. Current status: "
278                     + leaveRequest.getStatus());
279         }
280
281         if(actionDTO.getAction().equalsIgnoreCase("approved")){
282             leaveRequest.setStatus(LeaveStatus.APPROVED);
283             applyApprovedLeaveToEntitlement(leaveRequest);
284             notificationService.notify(
285                 leaveRequest.getEmployee(),
286                 "Your leave request from " + leaveRequest.getStartDate() + " to " +
287                 leaveRequest.getEndDate() + " has been approved.",
288                 "LEAVE_APPROVED"
289             );
290         }

```

```

289         employeeRepo.findByRole(Role.ADMIN).forEach(admin ->
290             notificationService.notify(
291                 admin,
292                 leaveRequest.getEmployee().getName() + "'s leave request has been approved by
" + manager.getName(),
293                 "LEAVE_APPROVED"
294             )
295         );
296     }
297     else if(actionDTO.getAction().equalsIgnoreCase("rejected")){
298         leaveRequest.setStatus(LeaveStatus.REJECTED);
299         notificationService.notify(
300             leaveRequest.getEmployee(),
301             "Your leave request from " + leaveRequest.getStartDate() + " to " +
leaveRequest.getEndDate() + " was rejected." +
302             (actionDTO.getRemarks() != null ? " Reason: " + actionDTO.getRemarks() : ""),
303             "LEAVE_REJECTED"
304         );
305         employeeRepo.findByRole(Role.ADMIN).forEach(admin ->
306             notificationService.notify(
307                 admin,
308                 leaveRequest.getEmployee().getName() + "'s leave request has been rejected by
" + manager.getName(),
309                 "LEAVE_REJECTED"
310             )
311         );
312     }
313     else{
314         return ResponseEntity.badRequest().body("Invalid action,Use APPROVED or REJECTED");
315     }
316     leaveRequest.setManager(manager.getEmail());
317     if(actionDTO.getRemarks()!=null){
318         leaveRequest.setRemarks(actionDTO.getRemarks());
319     }
320     return ResponseEntity.ok(leaveRequestRepo.save(leaveRequest));
321 }
322
323 @Transactional
324 public ResponseEntity<?> cancelLeaveRequest(Authentication authentication,long leaveId){
325     String email= AuthUtil.extractEmail(authentication);
326     var employee=employeeRepo.findByEmail(email).orElseThrow(
327         ()->new EmployeeNotFound("Employee with email:"+email+" not found")
328     );
329     var leaveRequest=leaveRequestRepo.findById(leaveId).orElseThrow(
330         ()->new LeaveRequestNotFoundException("Leave request with id:"+leaveId+" is not found")
331     );
332     if(employee.getEmployeeId()!=leaveRequest.getEmployee().getEmployeeId()){
333         return ResponseEntity.badRequest().body("You cannot cancel others Leave request..");
334     }
335     String timezone= employee.getTimezone();
336     ZoneId zoneId=ZoneId.of(timezone != null && !timezone.isBlank()
337         ? timezone
338         : "UTC");
339     LocalDate today = LocalDate.now(zoneId);
340     if(leaveRequest.getStartDate().isEqual(today)||leaveRequest.getStartDate().isBefore(today)){
341         return ResponseEntity.badRequest().body("You cannot cancel leave request after the leave has
started");
342     }
343     LeaveStatus previousStatus = leaveRequest.getStatus();
344     leaveRequest.setStatus(LeaveStatus.CANCELLED);
345     if (previousStatus == LeaveStatus.APPROVED) {
346         reverseApprovedLeaveFromEntitlement(leaveRequest);
347     }
348     leaveRequestRepo.save(leaveRequest);
349     return ResponseEntity.ok("Leave Request with id:"+ leaveId+" Successfully cancelled");
350 }
351
352 private LeaveResponseDTO convertToDTO(LeaveRequest request){
353     var responseDTO=new LeaveResponseDTO();
354     responseDTO.setLeaveRequestId(request.getId());
355     responseDTO.setEmployeeName(request.getEmployee().getName());
356     responseDTO.setLeaveType(request.getLeaveType());
357     responseDTO.setStartDate(request.getStartDate());
358     responseDTO.setEndDate(request.getEndDate());
359     responseDTO.setNumberOfDays(requestedDays(request).longValue());
360     responseDTO.setReason(request.getReason());

```

```

361         responseDTO.setStatus(request.getStatus());
362         return responseDTO;
363     }
364
365     public ResponseEntity<?> getLeaveRequestsByEmployee(Authentication authentication) {
366         String email= AuthUtil.extractEmail(authentication);
367         log.info("Fetching leave requests for employee: {}", email);
368
369         Employee employee = employeeRepo.findByEmail(email)
370             .orElseThrow(() -> new EmployeeNotFound("Employee not found: " +email ));
371
372         List<LeaveRequest> requests = leaveRequestRepo.findByEmployeeOrderByStartDateDesc(employee);
373         List<LeaveResponseDTO> response = requests.stream()
374             .map(this::convertToDTO)
375             .collect(Collectors.toList());
376
377         return ResponseEntity.ok(Map.of(
378             "employeeName", email,
379             "totalRequests", response.size(),
380             "requests", response
381         ));
382     }
383     public List<LeaveResponseDTO> getPendingLeaves() {
384         List<LeaveRequest> pendingRequests = leaveRequestRepo.findByStatus(LeaveStatus.PENDING);
385         return pendingRequests.stream()
386             .map(this::convertToDTO)
387             .collect(Collectors.toList());
388     }
389
390     /**
391      * Approve a leave request by ID (simplified version for AdminController).
392      * Returns LeaveResponseDTO directly.
393      */
394     @Transactional
395     public LeaveResponseDTO approveLeave(Long id) {
396         LeaveRequest leaveRequest = leaveRequestRepo.findById(id)
397             .orElseThrow(() -> new LeaveRequestNotFoundException("Leave request with id: " + id + " not
found"));
398
399         if (leaveRequest.getStatus() != LeaveStatus.PENDING) {
400             throw new IllegalStateException("Only PENDING requests can be approved. Current status: " +
leaveRequest.getStatus());
401         }
402
403         leaveRequest.setStatus(LeaveStatus.APPROVED);
404         leaveRequest.setManager("ADMIN"); // Or get from security context if needed
405         applyApprovedLeaveToEntitlement(leaveRequest);
406         notificationService.notify(
407             leaveRequest.getEmployee(),
408             "Your leave request from " + leaveRequest.getStartDate() + " to " +
leaveRequest.getEndDate() + " has been approved by admin.",
409             "LEAVE_APPROVED"
410         );
411
412         LeaveRequest saved = leaveRequestRepo.save(leaveRequest);
413         return convertToDTO(saved);
414     }
415
416     /**
417      * Reject a leave request by ID (simplified version for AdminController).
418      * Returns LeaveResponseDTO directly.
419      */
420     @Transactional
421     public LeaveResponseDTO rejectLeave(Long id) {
422         LeaveRequest leaveRequest = leaveRequestRepo.findById(id)
423             .orElseThrow(() -> new LeaveRequestNotFoundException("Leave request with id: " + id + " not
found"));
424
425         if (leaveRequest.getStatus() != LeaveStatus.PENDING) {
426             throw new IllegalStateException("Only PENDING requests can be rejected. Current status: " +
leaveRequest.getStatus());
427         }
428
429         leaveRequest.setStatus(LeaveStatus.REJECTED);
430         leaveRequest.setManager("ADMIN");
431         notificationService.notify(

```

```

432         leaveRequest.getEmployee(),
433         "Your leave request from " + leaveRequest.getStartDate() + " to " +
leaveRequest.getEndDate() + " was rejected by admin.",
434         "LEAVE_REJECTED"
435     );
436
437     LeaveRequest saved = leaveRequestRepo.save(leaveRequest);
438     return convertToDTO(saved);
439 }
440
441 private void applyApprovedLeaveToEntitlement(LeaveRequest leaveRequest) {
442     if (leaveRequest.getLeaveType() == LeaveType.MATERNITY) {
443         grantAndDeductMaternity(leaveRequest); // special maternity flow
444     } else {
445         ensureSufficientBalanceForApproval(leaveRequest);
446         updateUsedLeave(leaveRequest, requestedDays(leaveRequest)); // existing flow
447     }
448 }
449
450 private void grantAndDeductMaternity(LeaveRequest leaveRequest) {
451     int year = leaveRequest.getStartDate().getYear();
452     Employee employee = leaveRequest.getEmployee();
453
454     com.example.EmployeeManagementSystem.Entity.LeaveType maternityType =
455         leaveTypeRepository.findByName("MATERNITY")
456             .orElseThrow(() -> new IllegalStateException(
457                 "Maternity leave type not configured"));
458
459     var entitlement = leaveEntitlementRepository
460         .findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
461             employee.getEmployeeId(), maternityType.getId(), year)
462         .orElseGet(() -> createEntitlement(employee, maternityType, year));
463
464     // Grant 180 days one time on approval
465     entitlement.setAccruedThisYear(BigDecimal.valueOf(180));
466
467     // Deduct the days she is taking (always 180)
468     entitlement.setUsedThisYear(requestedDays(leaveRequest));
469
470     leaveEntitlementRepository.save(entitlement);
471 }
472
473 private void reverseApprovedLeaveFromEntitlement(LeaveRequest leaveRequest) {
474     if (leaveRequest.getLeaveType() == LeaveType.MATERNITY) {
475         reverseMaternityEntitlement(leaveRequest); // wipe the grant entirely
476     } else {
477         updateUsedLeave(leaveRequest, requestedDays(leaveRequest).negate());
478     }
479 }
480
481 private void reverseMaternityEntitlement(LeaveRequest leaveRequest) {
482     int year = leaveRequest.getStartDate().getYear();
483     Employee employee = leaveRequest.getEmployee();
484
485     com.example.EmployeeManagementSystem.Entity.LeaveType maternityType =
486         leaveTypeRepository.findByName("MATERNITY")
487             .orElseThrow(() -> new IllegalStateException(
488                 "Maternity leave type not configured"));
489
490     leaveEntitlementRepository
491         .findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
492             employee.getEmployeeId(), maternityType.getId(), year)
493         .ifPresent(entitlement -> {
494             // Wipe both the grant and the usage
495             entitlement.setAccruedThisYear(BigDecimal.ZERO);
496             entitlement.setUsedThisYear(BigDecimal.ZERO);
497             leaveEntitlementRepository.save(entitlement);
498         });
499 }
500
501 private void updateUsedLeave(LeaveRequest leaveRequest, BigDecimal delta) {
502     int year = leaveRequest.getStartDate().getYear();
503     String leaveTypeName = leaveRequest.getLeaveType().name();
504     var leaveType = leaveTypeRepository
505         .findByName(leaveTypeName)
506         .orElseThrow(() -> new IllegalStateException(

```

```

507         "Leave type not configured: " + leaveTypeName));
508
509         var entitlement = leaveEntitlementRepository
510             .findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
511                 leaveRequest.getEmployee().getEmployeeId(), leaveType.getId(), year)
512             .orElseGet(() -> createEntitlement(leaveRequest.getEmployee(), leaveType, year));
513
514         BigDecimal newUsed = entitlement.getUsedThisYear().add(delta);
515         // Prevent negative
516         newUsed = newUsed.max(BigDecimal.ZERO);
517
518         // Cap used days to the actual entitled total for the year
519         BigDecimal totalEntitled = entitlement.getOpeningBalance().add(entitlement.getAccruedThisYear());
520         if (newUsed.compareTo(totalEntitled) > 0) {
521             newUsed = totalEntitled;
522         }
523         entitlement.setUsedThisYear(newUsed);
524         // Keep closingBalance in sync so it never shows stale/negative values
525         entitlement.setClosingBalance(
526             entitlement.getOpeningBalance()
527                 .add(entitlement.getAccruedThisYear())
528                 .subtract(newUsed)
529         );
530     }
531
532     private void ensureSufficientBalanceForApproval(LeaveRequest leaveRequest) {
533         int year = leaveRequest.getStartDate().getYear();
534         String leaveTypeName = leaveRequest.getLeaveType().name();
535         var leaveType = leaveTypeRepository
536             .findByName(leaveTypeName)
537             .orElseThrow(() -> new IllegalStateException(
538                 "Leave type not configured: " + leaveTypeName));
539
540         var entitlement = leaveEntitlementRepository
541             .findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
542                 leaveRequest.getEmployee().getEmployeeId(), leaveType.getId(), year)
543             .orElseGet(() -> createEntitlement(leaveRequest.getEmployee(), leaveType, year));
544
545         BigDecimal availableBalance = entitlement.getAvailableBalance();
546         BigDecimal requestedDays = requestedDays(leaveRequest);
547
548         if (requestedDays.compareTo(availableBalance) > 0) {
549             throw new IllegalStateException(
550                 "Insufficient " + leaveTypeName.toLowerCase() + " leave balance for approval. " +
551                 "Available: " + availableBalance + ", requested: " + requestedDays);
552         }
553     }
554
555     private com.example.EmployeeManagementSystem.Entity.LeaveEntitlement createEntitlement(
556         Employee employee,
557         com.example.EmployeeManagementSystem.Entity.LeaveType leaveType,
558         int year) {
559         var entitlement = new com.example.EmployeeManagementSystem.Entity.LeaveEntitlement();
560         entitlement.setEmployee(employee);
561         entitlement.setLeaveType(leaveType);
562         entitlement.setYear(year);
563
564         if (leaveType.isCarriesForward()) {
565             BigDecimal lastYearClosing = leaveEntitlementRepository
566                 .findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
567                     employee.getEmployeeId(), leaveType.getId(), year - 1)
568                 .map(com.example.EmployeeManagementSystem.Entity.LeaveEntitlement::getClosingBalance)
569                 .orElse(BigDecimal.ZERO);
570             entitlement.setOpeningBalance(lastYearClosing);
571         }
572
573         return leaveEntitlementRepository.save(entitlement);
574     }
575
576     private BigDecimal requestedDays(LeaveRequest leaveRequest) {
577         long days = ChronoUnit.DAYS.between(leaveRequest.getStartDate(), leaveRequest.getEndDate()) + 1;
578         return BigDecimal.valueOf(days);
579     }
580
581     private long getCasualBalance(Employee employee, int year) {
582         return leaveTypeRepository.findByName("CASUAL")
583             .flatMap(lt -> leaveEntitlementRepository

```

```
583         .findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(  
584             employee.getEmployeeId(), lt.getId(), year))  
585         .map(e -> e.getAvailableBalance().longValue())  
586         .orElse(0L);  
587     }  
588 }  
589
```



```

1  package com.example.EmployeeManagementSystem.Service;
2
3  import com.example.EmployeeManagementSystem.DTO.*;
4  import com.example.EmployeeManagementSystem.Entity.Device;
5  import com.example.EmployeeManagementSystem.Entity.DeviceAssignment;
6  import com.example.EmployeeManagementSystem.Entity.Employee;
7  import com.example.EmployeeManagementSystem.Entity.LeaveRequest;
8  import com.example.EmployeeManagementSystem.Enum.AssignmentStatus;
9  import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
10 import com.example.EmployeeManagementSystem.Enum.Role;
11 import com.example.EmployeeManagementSystem.Enum.Status;
12 import com.example.EmployeeManagementSystem.Exception.EmployeeNotFound;
13 import com.example.EmployeeManagementSystem.Repository.DeviceAssignmentRepo;
14 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
15 import com.example.EmployeeManagementSystem.Repository.LeaveRequestRepo;
16 import com.example.EmployeeManagementSystem.Util.AuthUtil;
17 import org.springframework.http.ResponseEntity;
18 import org.springframework.security.core.Authentication;
19 import org.springframework.security.crypto.password.PasswordEncoder;
20 import org.springframework.stereotype.Service;
21
22 import java.time.temporal.ChronoUnit;
23 import java.util.List;
24 import java.util.stream.Collectors;
25
26 @Service
27 public class ManagerService {
28     private final EmployeeRepo employeeRepo;
29     private final PasswordEncoder passwordEncoder;
30     private final LeaveRequestRepo leaveRequestRepo;
31     private final DeviceAssignmentRepo deviceAssignmentRepo;
32
33     public ManagerService(EmployeeRepo employeeRepo,
34                           PasswordEncoder passwordEncoder,
35                           LeaveRequestRepo leaveRequestRepo,
36                           DeviceAssignmentRepo deviceAssignmentRepo) {
37         this.employeeRepo = employeeRepo;
38         this.passwordEncoder = passwordEncoder;
39         this.leaveRequestRepo = leaveRequestRepo;
40         this.deviceAssignmentRepo = deviceAssignmentRepo;
41     }
42
43     public Employee createEmp(EmployeeDetails employeeDetails, Authentication authentication) {
44         String email= AuthUtil.extractEmail(authentication);
45         Employee manager= employeeRepo.findByEmail(email).orElseThrow(
46             ()->new EmployeeNotFound("Manager :"+email+"Not found")
47         );
48         Employee employee=new Employee();
49         employee.setName(employeeDetails.getName());
50         employee.setEmail(employeeDetails.getEmail());
51         employee.setPassword(passwordEncoder.encode(employeeDetails.getPassword()));
52         employee.setGender(employeeDetails.getGender());
53         employee.setDept(manager.getDept());
54         employee.setRole(Role.EMPLOYEE);
55         employee.setManager(manager);
56         employee.setTimezone(employeeDetails.getTimezone());
57         return employeeRepo.save(employee);
58     }
59
60
61     public List<Employee> getAllEmployeeByManager(Authentication authentication) {
62         String email=AuthUtil.extractEmail(authentication);
63         Employee manager=employeeRepo.findByEmail(email).orElseThrow(
64             ()->new EmployeeNotFound("Manger:"+email+" not found")
65         );
66         return employeeRepo.findByManager(manager);
67     }
68
69     public List<ManagerLeaveResponseDTO> getAllLeaveRequestByManager(Authentication authentication) {
70         String email=AuthUtil.extractEmail(authentication);
71         Employee manager=employeeRepo.findByEmail(email).orElseThrow(

```

```

72         ()->new EmployeeNotFound("Manager with "+email+" not found")
73     };
74     List<LeaveRequest> requests=leaveRequestRepo.findByEmployee_Manager(manager);
75     return requests.stream().map(request->toManagerLeaveResponseDTO(request)).toList();
76 }
77
78 public List<EmployeeDTO> getUsers(){
79     List<Employee> employeeList=employeeRepo.findByRole(Role.USER);
80     return employeeList.stream().map(this::toDTO).toList();
81 }
82
83 public ResponseEntity<?> promoteUser(Authentication authentication, PromoteRequest promoteRequest){
84     Employee user=employeeRepo.findByEmail(promoteRequest.getEmail()).orElseThrow(
85         ()->new EmployeeNotFound("Employee:"+promoteRequest.getEmail()+" not found")
86     );
87     if(user.getRole()!=Role.USER){
88         return ResponseEntity.ok("Employee already promoted");
89     }
90     String managerEmail=AuthUtil.extractEmail(authentication);
91     Employee manager=employeeRepo.findByEmail(managerEmail).orElseThrow(
92         ()->new EmployeeNotFound("Manager:"+managerEmail+" not found")
93     );
94
95     user.setRole(Role.EMPLOYEE);
96     user.setManager(manager);
97     user.setDept(manager.getDept());
98     user.setPassword(passwordEncoder.encode(promoteRequest.getPassword()));
99     Employee promotedEmployee=employeeRepo.save(user);
100    return ResponseEntity.ok(toDTO(promotedEmployee));
101 }
102
103 public EmployeeDTO toDTO(Employee employee){
104     EmployeeDTO employeeDTO=new EmployeeDTO();
105     employeeDTO.setName(employee.getName());
106     employeeDTO.setEmail(employee.getEmail());
107     employeeDTO.setDept(employee.getDept());
108     employeeDTO.setPassword(employee.getPassword());
109     employeeDTO.setTimezone(employee.getTimezone());
110     return employeeDTO;
111 }
112
113 public Employee createAdmin(EmployeeDetails employeeDetails) {
114     Employee employee=new Employee();
115     employee.setName(employeeDetails.getName());
116     employee.setEmail(employeeDetails.getEmail());
117     employee.setPassword(passwordEncoder.encode(employeeDetails.getPassword()));
118     employee.setDept("Main");
119     employee.setRole(Role.ADMIN);
120     employee.setTimezone(employeeDetails.getTimezone());
121     return employeeRepo.save(employee);
122 }
123
124 public List<LeaveResponseDTO> getAllThePendingLeaveRequests(Authentication authentication) {
125     String email=AuthUtil.extractEmail(authentication);
126     Employee manager=employeeRepo.findByEmail(email).orElseThrow(
127         ()->new EmployeeNotFound("Manager with "+email+" not found")
128     );
129     List<LeaveRequest>
leaveRequests=leaveRequestRepo.findByStatusAndEmployee_Manager(LeaveStatus.PENDING,manager);
130     return leaveRequests.stream()
131         .map(this::convertToLeaveResponseDTO)
132         .collect(Collectors.toList());
133 }
134
135 public ManagerEmployeeDetailsDTO getEmployeeDetailsByName( String name) {
136
137     Employee employee = employeeRepo.findByName(name).orElseThrow(
138         () -> new EmployeeNotFound("Employee:" + name + " not found")
139     );
140
141
142
143     ManagerEmployeeDetailsDTO dto = new ManagerEmployeeDetailsDTO();
144     dto.setEmployeeId(employee.getEmployeeId());
145     dto.setName(employee.getName());
146     dto.setEmail(employee.getEmail());

```

```

147         dto.setDept(employee.getDept());
148         dto.setStatus(employee.getStatus());
149         dto.setAttendance(toAttendance(employee.getStatus()));
150         dto.setDevices(deviceAssignmentRepo
151             .findByAssignedToEmployeeIdAndStatus(employee.getEmployeeId(), AssignmentStatus.ACTIVE)
152             .stream()
153             .map(DeviceAssignment::getDevice)
154             .map(this::toDeviceResponseDTO)
155             .toList());
156         return dto;
157     }
158
159     private LeaveResponseDTO convertToLeaveResponseDTO(LeaveRequest leaveRequest) {
160         LeaveResponseDTO dto = new LeaveResponseDTO();
161         dto.setLeaveRequestId(leaveRequest.getId());
162         dto.setEmployeeName(leaveRequest.getEmployee().getName());
163         dto.setLeaveType(leaveRequest.getLeaveType());
164         dto.setStartDate(leaveRequest.getStartDate());
165         dto.setEndDate(leaveRequest.getEndDate());
166         dto.setReason(leaveRequest.getReason());
167         dto.setStatus(leaveRequest.getStatus());
168         return dto;
169     }
170
171     private ManagerLeaveResponseDTO toManagerLeaveResponseDTO(LeaveRequest leaveRequest){
172         ManagerLeaveResponseDTO managerLeaveResponseDTO=new ManagerLeaveResponseDTO();
173         managerLeaveResponseDTO.setEmployeeName(leaveRequest.getEmployee().getName());
174         managerLeaveResponseDTO.setLeaveType(leaveRequest.getLeaveType());
175         managerLeaveResponseDTO.setStartDate(leaveRequest.getStartDate());
176         managerLeaveResponseDTO.setEndDate(leaveRequest.getEndDate());
177         managerLeaveResponseDTO.setNumberOfDays(
178             ChronoUnit.DAYS.between(leaveRequest.getStartDate(), leaveRequest.getEndDate()) + 1);
179         managerLeaveResponseDTO.setStatus(leaveRequest.getStatus());
180         managerLeaveResponseDTO.setReason(leaveRequest.getReason());
181         return managerLeaveResponseDTO;
182     }
183
184     private String toAttendance(Status status) {
185         if (status == Status.ON_LEAVE) {
186             return "ON_LEAVE";
187         }
188         if (status == Status.ACTIVE) {
189             return "PRESENT";
190         }
191         return status.name();
192     }
193
194     private DeviceResponseDTO toDeviceResponseDTO(Device device) {
195         DeviceResponseDTO dto = new DeviceResponseDTO();
196         dto.setId(device.getId());
197         dto.setDeviceName(device.getDeviceName());
198         dto.setBrand(device.getBrand());
199         dto.setDeviceType(device.getDeviceType());
200         dto.setDeviceStatus(device.getDeviceStatus());
201         dto.setWarrantyExpiryDate(device.getWarrantyExpiryDate());
202
203         if (device.getTechVendor() != null) {
204             dto.setVendorName(device.getTechVendor().getName());
205         }
206
207         DeviceAssignment assignment = device.getCurrentAssignment();
208         if (assignment != null && assignment.getStatus() == AssignmentStatus.ACTIVE) {
209             dto.setAssignedDate(assignment.getAssignedDate());
210             if (assignment.getAssignedTo() != null) {
211                 dto.setAssignedEmployeeId(assignment.getAssignedTo().getEmployeeId());
212                 dto.setAssignedEmployeeName(assignment.getAssignedTo().getName());
213             }
214         }
215
216         return dto;
217     }
218 }
219

```

```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.DTO.LeaveResponseDTO;
4 import com.example.EmployeeManagementSystem.DTO.MaternityLeaveRequestDTO;
5 import com.example.EmployeeManagementSystem.Entity.Employee;
6 import com.example.EmployeeManagementSystem.Entity.LeaveRequest;
7 import com.example.EmployeeManagementSystem.Enum.*;
8 import com.example.EmployeeManagementSystem.Exception.EmployeeNotFound;
9 import com.example.EmployeeManagementSystem.Exception.InvalidStartDateException;
10 import com.example.EmployeeManagementSystem.Exception.OverlappingLeaveException;
11 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
12 import com.example.EmployeeManagementSystem.Repository.LeaveRequestRepo;
13 import com.example.EmployeeManagementSystem.Util.AuthUtil;
14 import org.springframework.http.HttpStatus;
15 import org.springframework.http.ResponseEntity;
16 import org.springframework.security.core.Authentication;
17 import org.springframework.stereotype.Service;
18
19 import java.math.BigDecimal;
20 import java.time.LocalDate;
21 import java.time.temporal.ChronoUnit;
22 import java.util.List;
23 import java.util.Map;
24
25 @Service
26 public class MaternityLeaveService {
27
28     private final EmployeeRepo employeeRepo;
29     private final LeaveRequestRepo leaveRequestRepo;
30     private final NotificationService notificationService;
31
32     public MaternityLeaveService(EmployeeRepo employeeRepo, LeaveRequestRepo leaveRequestRepo,
33 NotificationService notificationService) {
34         this.employeeRepo = employeeRepo;
35         this.leaveRequestRepo = leaveRequestRepo;
36         this.notificationService = notificationService;
37     }
38
39     public ResponseEntity<?> applyMaternityLeave(Authentication authentication,
40 MaternityLeaveRequestDTO maternityLeaveRequestDTO){
41         String email = AuthUtil.extractEmail(authentication);
42         Employee employee = employeeRepo.findByEmail(email)
43             .orElseThrow(() -> new EmployeeNotFound("Employee not found: " + email));
44
45         if (employee.getStatus() != Status.ACTIVE) {
46             return ResponseEntity.badRequest()
47                 .body(Map.of("error", "Only active employees can apply"));
48         }
49
50         if (employee.getGender() != Gender.F) {
51             return ResponseEntity.badRequest()
52                 .body(Map.of("error", "Only female employees can apply for maternity leave"));
53         }
54
55         boolean alreadyApplied = leaveRequestRepo
56             .existsMaternityLeaveForYear(
57                 employee,
58                 LeaveType.MATERNITY,
59                 maternityLeaveRequestDTO.getStartDate().getYear(),
60                 List.of(LeaveStatus.CANCELLED, LeaveStatus.REJECTED));
61         if (alreadyApplied) {
62             return ResponseEntity.badRequest()
63                 .body(Map.of("error", "Maternity leave already applied for this year"));
64         }
65
66         LocalDate today = LocalDate.now();
67         if (maternityLeaveRequestDTO.getStartDate().isBefore(today)) {
68             throw new InvalidStartDateException("Start date cannot be before today");
69         }
70     }

```

```

71
72     // 5. Overlapping check
73     LocalDate endDate = maternityLeaveRequestDTO.getStartDate().plusDays(179); // always 180 days
74     long overlapping = leaveRequestRepo.countOverlappingLeave(
75         employee.getEmployeeId(),
76         maternityLeaveRequestDTO.getStartDate(),
77         endDate);
78     if (overlapping > 0) {
79         throw new OverlappingLeaveException("Overlapping leave exists");
80     }
81
82     LeaveRequest request = new LeaveRequest();
83     request.setEmployee(employee);
84     request.setLeaveType(LeaveType.MATERNITY);
85     request.setStartDate(maternityLeaveRequestDTO.getStartDate());
86     request.setEndDate(endDate); // forced 180 days
87     request.setStatus(LeaveStatus.PENDING);
88     request.setReason(maternityLeaveRequestDTO.getReason());
89
90     LeaveRequest saved = leaveRequestRepo.save(request);
91     Employee manager = employee.getManager();
92     if (manager != null) {
93         notificationService.notify(
94             manager,
95             employee.getName() + " has applied for maternity leave.",
96             "MATERNITY_LEAVE_REQUEST"
97         );
98     }
99     employeeRepo.findByRole(Role.ADMIN).forEach(admin ->
100         notificationService.notify(
101             admin,
102             employee.getName() + " applied for maternity leave (" +
103                 maternityLeaveRequestDTO.getStartDate() + " " + endDate + ")",
104             "MATERNITY_LEAVE_REQUEST"
105         )
106     );
107
108     return ResponseEntity.status(HttpStatus.CREATED).body(convertToDTO(saved));
109 }
110
111 private LeaveResponseDTO convertToDTO(LeaveRequest request){
112     var responseDTO=new LeaveResponseDTO();
113     responseDTO.setLeaveRequestId(request.getId());
114     responseDTO.setEmployeeName(request.getEmployee().getName());
115     responseDTO.setLeaveType(request.getLeaveType());
116     responseDTO.setStartDate(request.getStartDate());
117     responseDTO.setEndDate(request.getEndDate());
118     responseDTO.setNumberOfDays(requestedDays(request).longValue());
119     responseDTO.setReason(request.getReason());
120     responseDTO.setStatus(request.getStatus());
121     return responseDTO;
122 }
123
124 private BigDecimal requestedDays(LeaveRequest leaveRequest) {
125     long days = ChronoUnit.DAYS.between(leaveRequest.getStartDate(), leaveRequest.getEndDate()) + 1;
126     return BigDecimal.valueOf(days);
127 }
128 }
129

```

```
1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.Enum.MealSlot;
4 import org.springframework.stereotype.Service;
5
6 import java.time.LocalTime;
7 import java.util.Map;
8
9 @Service
10 public class MealSlotService {
11
12     private final Map<MealSlot, LocalTime> slotLocalTimeMap = Map.of(
13         MealSlot.BREAKFAST, LocalTime.of(8, 0),
14         MealSlot.LUNCH, LocalTime.of(13, 0),
15         MealSlot.DINNER, LocalTime.of(21, 0)
16     );
17
18     public LocalTime getTime(MealSlot slot) {
19         return slotLocalTimeMap.get(slot);
20     }
21 }
22
```

```
1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.Notification;
5 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
6 import com.example.EmployeeManagementSystem.Repository.NotificationRepository;
7 import org.springframework.stereotype.Service;
8 import java.util.List;
9
10 @Service
11 public class NotificationService {
12
13     private final NotificationRepository notificationRepo;
14     private final EmployeeRepo employeeRepo;
15
16     public NotificationService(NotificationRepository notificationRepo,
17                               EmployeeRepo employeeRepo) {
18         this.notificationRepo = notificationRepo;
19         this.employeeRepo = employeeRepo;
20     }
21
22     /** Create a notification for a specific employee */
23     public void notify(Employee recipient, String message, String type) {
24         Notification n = new Notification();
25         n.setRecipient(recipient);
26         n.setMessage(message);
27         n.setType(type);
28         notificationRepo.save(n);
29     }
30
31     /** Notify every employee in the system (e.g. new public holiday) */
32     public void notifyAll(String message, String type) {
33         List<Employee> all = employeeRepo.findAll();
34         for (Employee e : all) {
35             notify(e, message, type);
36         }
37     }
38
39     public List<Notification> getForEmployee(Employee employee) {
40         return notificationRepo.findByRecipientOrderByCreatedAtDesc(employee);
41     }
42
43     public long getUnreadCount(Employee employee) {
44         return notificationRepo.countByRecipientAndIsReadFalse(employee);
45     }
46
47     public void markAllRead(Employee employee) {
48         List<Notification> unread = notificationRepo
49             .findByRecipientOrderByCreatedAtDesc(employee)
50             .stream()
51             .filter(n -> !n.isRead())
52             .toList();
53         unread.forEach(n -> n.setRead(true));
54         notificationRepo.saveAll(unread);
55     }
56 }
57
```

```
1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.RefreshToken;
5 import com.example.EmployeeManagementSystem.Entity.Vendor;
6 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
7 import com.example.EmployeeManagementSystem.Repository.RefreshTokenRepository;
8 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
9 import org.springframework.stereotype.Service;
10
11 import java.time.Instant;
12 import java.time.temporal.ChronoUnit;
13 import java.util.UUID;
14
15 @Service
16 public class RefreshTokenService {
17     private final EmployeeRepo employeeRepo;
18     private final VendorRepo vendorRepo;
19
20     public RefreshTokenService(RefreshTokenRepository refreshTokenRepository, EmployeeRepo employeeRepo,
21     VendorRepo vendorRepo) {
22         this.employeeRepo = employeeRepo;
23         this.vendorRepo = vendorRepo;
24     }
25
26     public RefreshToken createForEmployee(Employee employee) {
27         RefreshToken refreshToken = new RefreshToken();
28         refreshToken.setToken(UUID.randomUUID().toString());
29         refreshToken.setExpiryTime(Instant.now().plus(7, ChronoUnit.DAYS));
30         refreshToken.setEmployee(employee);
31         return refreshToken;
32     }
33
34     public RefreshToken createForVendor(Vendor vendor) {
35         RefreshToken refreshToken = new RefreshToken();
36         refreshToken.setToken(UUID.randomUUID().toString());
37         refreshToken.setExpiryTime(Instant.now().plus(7, ChronoUnit.DAYS));
38         refreshToken.setVendor(vendor);
39         return refreshToken;
40     }
41 }
```



```

1  package com.example.EmployeeManagementSystem.Service;
2
3  import com.example.EmployeeManagementSystem.DTO.RestaurantDTO;
4  import com.example.EmployeeManagementSystem.DTO.RestaurantRequest;
5  import com.example.EmployeeManagementSystem.Entity.Restaurant;
6  import com.example.EmployeeManagementSystem.Entity.Vendor;
7  import com.example.EmployeeManagementSystem.Enum.MealsSlot;
8  import com.example.EmployeeManagementSystem.Exception.RestaurantNotFoundException;
9  import com.example.EmployeeManagementSystem.Exception.UnauthorizedAccessException;
10 import com.example.EmployeeManagementSystem.Exception.VendorNotFoundException;
11 import com.example.EmployeeManagementSystem.Repository.RestaurantRepository;
12 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
13 import com.example.EmployeeManagementSystem.Util.AuthUtil;
14 import org.slf4j.Logger;
15 import org.slf4j.LoggerFactory;
16 import org.springframework.security.core.Authentication;
17 import org.springframework.stereotype.Service;
18
19 import java.util.List;
20 import java.util.stream.Collectors;
21
22 @Service
23 public class RestaurantService {
24
25     private static final Logger log = LoggerFactory.getLogger(RestaurantService.class);
26
27     private final RestaurantRepository restaurantRepository;
28     private final VendorRepo vendorRepo;
29
30     public RestaurantService(RestaurantRepository restaurantRepository, VendorRepo vendorRepo) {
31         this.restaurantRepository = restaurantRepository;
32         this.vendorRepo = vendorRepo;
33     }
34
35     /**
36      * Create a new restaurant.
37      * Only the owning vendor (matched by vendorId in the request) may do this.
38      */
39     public RestaurantDTO createRestaurant(RestaurantRequest request, Authentication authentication) {
40         String email = AuthUtil.extractEmail(authentication);
41         Vendor vendor = vendorRepo.findByEmail(email)
42             .orElseThrow(() -> new VendorNotFoundException(
43                 "Vendor with email " + email + " not found"));
44
45         Restaurant restaurant = new Restaurant();
46         restaurant.setName(request.getName());
47         restaurant.setAddress(request.getAddress());
48         restaurant.setDescription(request.getDescription());
49         restaurant.setSupportedMealsSlots(request.getSupportedMealsSlots());
50         restaurant.setVendor(vendor);
51         restaurant.setActive(true);
52
53         Restaurant saved = restaurantRepository.save(restaurant);
54         log.info("Restaurant created: {} by vendor {}", saved.getName(), vendor.getId());
55         return toDTO(saved);
56     }
57
58     /**
59      * Update restaurant details.
60      * Vendor must own this restaurant – otherwise 403.
61      */
62     public RestaurantDTO updateRestaurant(Long restaurantId, RestaurantRequest request, Authentication authentication) {
63         Restaurant restaurant = findOrThrow(restaurantId);
64         String email = AuthUtil.extractEmail(authentication);
65         Vendor vendor = vendorRepo.findByEmail(email).orElseThrow(
66             () -> new VendorNotFoundException("Vendor Not found")
67         );
68         assertOwnership(restaurantId, vendor.getEmail());
69
70         if (request.getName() != null) restaurant.setName(request.getName());

```

```

71         if (request.getAddress() != null) restaurant.setAddress(request.getAddress());
72         if (request.getDescription() != null) restaurant.setDescription(request.getDescription());
73         if (request.getSupportedMealSlots() != null && !request.getSupportedMealSlots().isEmpty()) {
74             restaurant.setSupportedMealSlots(request.getSupportedMealSlots());
75         }
76         return toDTO(restaurantRepository.save(restaurant));
77     }
78
79     /**
80      * Deactivate a restaurant (soft delete).
81      * Only the owning vendor may do this.
82      * Active subscriptions to this restaurant will no longer receive new deliveries
83      * – the Subscription status should be handled separately.
84      */
85     public void deactivateRestaurant(Long restaurantId, Authentication authentication) {
86         Restaurant restaurant = findOrThrow(restaurantId);
87         String email= AuthUtil.extractEmail(authentication);
88         Vendor vendor=vendorRepo.findByEmail(email).orElseThrow(
89             ()->new VendorNotFoundException("Vendor Not found")
90         );
91         assertOwnership(restaurantId, vendor.getEmail());
92         restaurant.setActive(false);
93         restaurantRepository.save(restaurant);
94         log.info("Restaurant deactivated: id={}", restaurantId);
95     }
96
97     /**
98      * Get all restaurants owned by a vendor.
99      * VENDOR-only endpoint.
100     */
101     public List<RestaurantDTO> getRestaurantsByVendor(Authentication authentication) {
102         String email= AuthUtil.extractEmail(authentication);
103         vendorRepo.findByEmail(email)
104             .orElseThrow(() -> new VendorNotFoundException("Vendor " + email + " not found"));
105         return restaurantRepository.findByVendor_EmailAndActiveTrue(email)
106             .stream().map(this::toDTO).collect(Collectors.toList());
107     }
108
109     /**
110      * Get all active restaurants.
111      * Available to EMPLOYEE so they can browse and choose.
112      */
113     public List<RestaurantDTO> getAllActiveRestaurants() {
114         return restaurantRepository.findByActiveTrue()
115             .stream().map(this::toDTO).collect(Collectors.toList());
116     }
117
118     /**
119      * Get active restaurants that serve a specific meal slot.
120      * Employees call this when subscribing, to see valid restaurant options for a slot.
121      */
122     public List<RestaurantDTO> getRestaurantsByMealSlot(MealSlot mealSlot) {
123         return restaurantRepository.findActiveByMealSlot(mealSlot)
124             .stream().map(this::toDTO).collect(Collectors.toList());
125     }
126
127     public RestaurantDTO getRestaurant(Long id) {
128         return toDTO(findOrThrow(id));
129     }
130
131     // helpers
132
133     private Restaurant findOrThrow(Long id) {
134         return restaurantRepository.findById(id)
135             .orElseThrow(() -> new RestaurantNotFoundException(
136                 "Restaurant with id " + id + " not found"));
137     }
138
139     /**
140      * Throws UnauthorizedAccessException if the restaurant does not belong to the vendor.
141      * This is the core access-control guard for all mutating vendor operations.
142      */
143     private void assertOwnership(Long restaurantId, String email) {
144         if (!restaurantRepository.existsByIdAndVendor_Email(restaurantId, email)) {
145             throw new UnauthorizedAccessException(
146                 "Vendor " + email + " does not own restaurant " + restaurantId);

```

```
147     }
148 }
149
150 public RestaurantDTO toDTO(Restaurant r) {
151     RestaurantDTO dto = new RestaurantDTO();
152     dto.setId(r.getId());
153     dto.setName(r.getName());
154     dto.setAddress(r.getAddress());
155     dto.setDescription(r.getDescription());
156     dto.setSupportedMealSlots(r.getSupportedMealSlots());
157     dto.setActive(r.isActive());
158     dto.setVendorId(r.getVendor().getId());
159     dto.setVendorName(r.getVendor().getName());
160     return dto;
161 }
162 }
```

```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.DTO.AdminActionDTO;
4 import com.example.EmployeeManagementSystem.DTO.RepairDTO;
5 import com.example.EmployeeManagementSystem.DTO.ServiceRequestDTO;
6 import com.example.EmployeeManagementSystem.DTO.ServiceRequestResponseDTO;
7 import com.example.EmployeeManagementSystem.Entity.Device;
8 import com.example.EmployeeManagementSystem.Entity.Employee;
9 import com.example.EmployeeManagementSystem.Entity.ServiceRequest;
10 import com.example.EmployeeManagementSystem.Entity.Vendor;
11 import com.example.EmployeeManagementSystem.Enum.DeviceStatus;
12 import com.example.EmployeeManagementSystem.Enum.ServiceRequestStatus;
13 import com.example.EmployeeManagementSystem.Exception.EmployeeNotFound;
14 import com.example.EmployeeManagementSystem.Exception.VendorNotFoundException;
15 import com.example.EmployeeManagementSystem.Repository.DeviceRepository;
16 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
17 import com.example.EmployeeManagementSystem.Repository.ServiceRequestRepository;
18 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
19 import com.example.EmployeeManagementSystem.Util.AuthUtil;
20 import org.springframework.security.core.Authentication;
21 import org.springframework.stereotype.Service;
22 import org.springframework.transaction.annotation.Transactional;
23 import org.w3c.dom.stylesheets.LinkStyle;
24
25 import java.time.LocalDateTime;
26 import java.util.List;
27 import java.util.Vector;
28
29 @Service
30 public class ServiceRequestService {
31     private final ServiceRequestRepository serviceRequestRepository;
32     private final EmployeeRepo employeeRepo;
33     private final DeviceRepository deviceRepository;
34     private final VendorRepo vendorRepo;
35
36     public ServiceRequestService(ServiceRequestRepository serviceRequestRepository, EmployeeRepo employeeRepo, DeviceRepository deviceRepository, VendorRepo vendorRepo) {
37         this.serviceRequestRepository = serviceRequestRepository;
38         this.employeeRepo = employeeRepo;
39         this.deviceRepository = deviceRepository;
40         this.vendorRepo = vendorRepo;
41     }
42
43     public ServiceRequest createServiceRequest(ServiceRequestDTO dto, Authentication authentication) {
44         String email= AuthUtil.extractEmail(authentication);
45         Employee employee = employeeRepo.findByEmail(email)
46             .orElseThrow(() -> new RuntimeException("Employee not found"));
47
48         // 2. Fetch device
49         Device device = deviceRepository.findById(dto.getDeviceId())
50             .orElseThrow(() -> new RuntimeException("Device not found"));
51
52         // 3. Validate device is assigned to employee
53         if (device.getCurrentAssignment() == null ||
54             !device.getCurrentAssignment().getAssignedTo().equals(employee)) {
55             throw new RuntimeException("Device not assigned to this employee");
56         }
57
58         // 4. Validate device status
59         if (device.getDeviceStatus() != DeviceStatus.ASSIGNED) {
60             throw new RuntimeException("Device is not in usable state");
61         }
62
63         // 5. Create Service Request
64         ServiceRequest request = new ServiceRequest();
65         request.setDevice(device);
66         request.setRaisedBy(employee);
67         request.setRequestType(dto.getRequestType());
68         request.setIssueDescription(dto.getIssueDescription());
69         request.setUrgent(dto.isUrgent());
70         request.setStatus(ServiceRequestStatus.OPEN);

```

```

71         return serviceRequestRepository.save(request);
72     }
73
74
75     public List<ServiceRequestResponseDTO> getAllOpenServiceRequests(){
76         List<ServiceRequest>
serviceRequestList=serviceRequestRepository.findByStatus(ServiceRequestStatus.OPEN);
77         return serviceRequestList.stream().map(serviceRequest ->
toServiceRequestResponseDTO(serviceRequest)).toList();
78
79     }
80
81     public List<ServiceRequestResponseDTO> getMyServiceRequests(Authentication authentication) {
82         String email = AuthUtil.extractEmail(authentication);
83         Employee employee = employeeRepo.findByEmail(email)
84             .orElseThrow(() -> new EmployeeNotFound("Employee not found: " + email));
85
86         List<ServiceRequest> serviceRequestList =
serviceRequestRepository.findByRaisedByOrderByRaisedAtDesc(employee);
87         return serviceRequestList.stream()
88             .map(this::toServiceRequestResponseDTO)
89             .toList();
90     }
91
92     private ServiceRequestDTO toServiceRequestDTO(ServiceRequest serviceRequest){
93         ServiceRequestDTO requestDTO=new ServiceRequestDTO();
94         requestDTO.setDeviceId(serviceRequest.getDevice().getId());
95         requestDTO.setRequestType(serviceRequest.getRequestType());
96         requestDTO.setIssueDescription(serviceRequest.getIssueDescription());
97         requestDTO.setUrgent(serviceRequest.isUrgent());
98         return requestDTO;
99     }
100
101     private ServiceRequestResponseDTO toServiceRequestResponseDTO(ServiceRequest serviceRequest) {
102         ServiceRequestResponseDTO responseDTO = new ServiceRequestResponseDTO();
103         responseDTO.setId(serviceRequest.getId());
104         responseDTO.setRequestType(serviceRequest.getRequestType() != null
105             ? serviceRequest.getRequestType().name()
106             : null);
107         responseDTO.setStatus(serviceRequest.getStatus() != null
108             ? serviceRequest.getStatus().name()
109             : null);
110         responseDTO.setIssueDescription(serviceRequest.getIssueDescription());
111         responseDTO.setAdminRemarks(serviceRequest.getAdminRemarks());
112         responseDTO.setResolution(serviceRequest.getResolution());
113         responseDTO.setUrgent(serviceRequest.isUrgent());
114         responseDTO.setRaisedAt(serviceRequest.getRaisedAt());
115         responseDTO.setResolvedAt(serviceRequest.getResolvedAt());
116         responseDTO.setDeviceName(serviceRequest.getDevice() != null
117             ? serviceRequest.getDevice().getDeviceName()
118             : null);
119         responseDTO.setReviewedByName(serviceRequest.getReviewedBy() != null
120             ? serviceRequest.getReviewedBy().getName()
121             : null);
122         responseDTO.setRaisedByName(serviceRequest.getRaisedBy().getName());
123         return responseDTO;
124     }
125
126
127     @Transactional
128     public ServiceRequestResponseDTO updateServiceRequestByAdmin(Authentication authentication,
AdminActionDTO actionDTO){
129         ServiceRequest
serviceRequest=serviceRequestRepository.findById(actionDTO.getServiceRequestId()).orElseThrow(
130             ()->new RuntimeException("Service Request Not Found")
131         );
132         String email=AuthUtil.extractEmail(authentication);
133         Employee admin=employeeRepo.findByEmail(email).orElseThrow(
134             ()->new EmployeeNotFound("Admin not found:"+email)
135         );
136         Device device=serviceRequest.getDevice();
137         if (actionDTO.getStatus() == ServiceRequestStatus.SENT_FOR_REPAIR) {
138             device.setDeviceStatus(DeviceStatus.UNDER_REPAIR);
139             deviceRepository.save(device);
140         } else if (actionDTO.getStatus() == ServiceRequestStatus.REJECTED) {
141             device.setDeviceStatus(DeviceStatus.ASSIGNED);

```

```

142     }
143     serviceRequest.setReviewedBy(admin);
144     serviceRequest.setAdminRemarks(actionDTO.getAdminRemarks());
145     serviceRequest.setStatus(actionDTO.getStatus());
146     ServiceRequest saved=serviceRequestRepository.save(serviceRequest);
147     return toServiceRequestResponseDTO(saved);
148 }
149
150 public List<ServiceRequestResponseDTO> getAllServiceRequestByVendor(Authentication authentication){
151     String email=AuthUtil.extractEmail(authentication);
152     Vendor vendor=vendorRepo.findByEmail(email).orElseThrow(
153         ()->new VendorNotFoundException("Vendor not found: "+email)
154     );
155
156     List<ServiceRequest>
serviceRequestList=serviceRequestRepository.findByDeviceTechVendorAndStatus(vendor, ServiceRequestStatus.SENT_FOR
_REPAIR);
157     return serviceRequestList.stream()
158         .map(serviceRequest -> toServiceRequestResponseDTO(serviceRequest))
159         .toList();
160 }
161
162 public ServiceRequestResponseDTO updateServiceRequestByVendor(
163     Authentication authentication, RepairDTO repairDTO
164 ) {
165
166     // 1. Get vendor
167     String email = AuthUtil.extractEmail(authentication);
168     Vendor vendor = vendorRepo.findByEmail(email)
169         .orElseThrow(() -> new RuntimeException("Vendor not found"));
170
171     // 2. Get request
172     ServiceRequest request = serviceRequestRepository.findById(repairDTO.getRequestId())
173         .orElseThrow(() -> new RuntimeException("Request not found"));
174
175     // 3. Validate request belongs to vendor
176     if (!request.getDevice().getTechVendor().getId().equals(vendor.getId())) {
177         throw new RuntimeException("Unauthorized access");
178     }
179
180     // 4. Validate status
181     if (request.getStatus() != ServiceRequestStatus.SENT_FOR_REPAIR) {
182         throw new RuntimeException("Request is not in repair stage");
183     }
184
185     Device device = request.getDevice();
186
187     // 5. Update based on repair result
188     if (repairDTO.getStatus()==ServiceRequestStatus.REPAIR_DONE) {
189         request.setResolution(repairDTO.getResolution());
190         request.setStatus(repairDTO.getStatus());
191
192         device.setDeviceStatus(DeviceStatus.ASSIGNED); // back to employee
193     } else {
194         request.setResolution(repairDTO.getResolution());
195         request.setStatus(ServiceRequestStatus.REPAIR_DONE);
196
197         device.setDeviceStatus(DeviceStatus.CONDEMNED); // not usable
198     }
199
200     request.setResolvedAt(LocalDateTime.now());
201
202     deviceRepository.save(device);
203     ServiceRequest serviceRequest=serviceRequestRepository.save(request);
204     return toServiceRequestResponseDTO(serviceRequest);
205 }
206 }
207
208
209

```

```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
4 import com.example.EmployeeManagementSystem.Entity.LeaveType;
5 import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
6 import com.example.EmployeeManagementSystem.Repository.LeaveTypeRepository;
7 import org.slf4j.Logger;
8 import org.slf4j.LoggerFactory;
9 import org.springframework.stereotype.Service;
10 import org.springframework.transaction.annotation.Transactional;
11
12 import java.math.BigDecimal;
13 import java.time.LocalDate;
14 import java.util.List;
15
16 @Service
17 public class SickLeaveResetService {
18
19     private static final Logger log = LoggerFactory.getLogger(SickLeaveResetService.class);
20
21     private final LeaveTypeRepository leaveTypeRepository;
22     private final LeaveEntitlementRepository entitlementRepository;
23
24     public SickLeaveResetService(LeaveTypeRepository leaveTypeRepository,
25                               LeaveEntitlementRepository entitlementRepository) {
26         this.leaveTypeRepository = leaveTypeRepository;
27         this.entitlementRepository = entitlementRepository;
28     }
29
30     @Transactional
31     public void runMonthlyReset(LocalDate resetDate) {
32         // resetDate = last day of the month being reset (e.g. 2025-06-30)
33         int year = resetDate.getYear();
34         int month = resetDate.getMonthValue(); // FIX: scope reset to this month only
35
36         // Find all leave types that reset monthly (only SICK in your design)
37         List<LeaveType> resetTypes = leaveTypeRepository.findByMonthlyResetTrue();
38
39         for (LeaveType leaveType : resetTypes) {
40             // FIX: was findByLeaveTypeIdAndYear – fetched ALL entitlements for the year
41             // and re-reset them every month. Now scoped to current month's accruals.
42             List<LeaveEntitlement> entitlements =
43                 entitlementRepository.findByLeaveTypeIdAndYear(leaveType.getId(), year);
44
45             for (LeaveEntitlement entitlement : entitlements) {
46
47                 BigDecimal available = entitlement.getAvailableBalance();
48
49                 if (available.compareTo(BigDecimal.ZERO) > 0) {
50                     // Wipe unused days – set accrued = used so available = 0
51                     // Preserves the audit trail of what was used
52                     entitlement.setAccruedThisYear(entitlement.getUsedThisYear());
53
54                     log.info("Sick leave reset for employee {} – {} unused day(s) expired at end of {}-{}",
55                             entitlement.getEmployee().getEmployeeId(), available, year, month);
56                 } else {
57                     log.debug("Sick leave reset for employee {} – nothing to expire (month {}-{})",
58                             entitlement.getEmployee().getEmployeeId(), year, month);
59                 }
60
61                 entitlementRepository.save(entitlement);
62             }
63         }
64
65         log.info("Sick leave monthly reset complete for {}", resetDate);
66     }
67 }

```

```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.DTO.SubscriptionDTO;
4 import com.example.EmployeeManagementSystem.DTO.SubscriptionRequest;
5 import com.example.EmployeeManagementSystem.Entity.Employee;
6 import com.example.EmployeeManagementSystem.Entity.Restaurant;
7 import com.example.EmployeeManagementSystem.Entity.Subscription;
8 import com.example.EmployeeManagementSystem.Enum.MealSlot;
9 import com.example.EmployeeManagementSystem.Enum.ScheduleType;
10 import com.example.EmployeeManagementSystem.Enum.SubscriptionStatus;
11 import com.example.EmployeeManagementSystem.Exception.EmployeeNotFound;
12 import com.example.EmployeeManagementSystem.Exception.RestaurantNotFoundException;
13 import com.example.EmployeeManagementSystem.Exception.SubscriptionAlreadyExists;
14 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
15 import com.example.EmployeeManagementSystem.Repository.RestaurantRepository;
16 import com.example.EmployeeManagementSystem.Repository.SubscriptionRepository;
17 import com.example.EmployeeManagementSystem.Util.AuthUtil;
18 import org.springframework.security.core.Authentication;
19 import org.springframework.stereotype.Service;
20
21 import java.time.ZoneId;
22 import java.time.ZonedDateTime;
23 import java.util.ArrayList;
24 import java.util.List;
25
26 @Service
27 public class SubscriptionService {
28
29     private final SubscriptionRepository subscriptionRepository;
30     private final EmployeeRepo employeeRepo;
31     private final DeliveryTimeService deliveryTimeService;
32     private final RestaurantRepository restaurantRepository;
33
34     public SubscriptionService(SubscriptionRepository subscriptionRepository,
35                               EmployeeRepo employeeRepo,
36                               DeliveryTimeService deliveryTimeService,
37                               RestaurantRepository restaurantRepository) {
38         this.subscriptionRepository = subscriptionRepository;
39         this.employeeRepo = employeeRepo;
40         this.deliveryTimeService = deliveryTimeService;
41         this.restaurantRepository = restaurantRepository;
42     }
43
44     /**
45      * Add subscriptions for an employee.
46      *
47      * Each SlotOrder specifies a meal slot AND the restaurant for that slot,
48      * so an employee can have breakfast from Restaurant A and lunch from Restaurant B.
49      *
50      * Validations:
51      * - Employee must exist and be ACTIVE
52      * - Restaurant must exist and be active
53      * - Restaurant must support the requested meal slot
54      * - No duplicate active subscription for the same employee + slot combination
55      */
56     public void addSubscription(SubscriptionsRequest request, Authentication authentication) {
57         String email = AuthUtil.extractEmail(authentication);
58         Employee employee = employeeRepo.findByEmail(email)
59             .orElseThrow(() -> new EmployeeNotFound(
60                 "Employee with id " + email + " not found"));
61
62         if (request.getSlotOrders() == null || request.getSlotOrders().isEmpty()) {
63             throw new IllegalArgumentException("slotOrders cannot be null or empty");
64         }
65
66         if (request.getScheduleType() == ScheduleType.DAILY) {
67             request.setDayOfWeek(null);
68         } else if (request.getScheduleType() == ScheduleType.WEEKLY) {
69             if (request.getDayOfWeek() == null) {
70                 throw new IllegalArgumentException("dayOfWeek is required for WEEKLY schedule");
71             }

```



```

72     }
73
74     for (SubscriptionRequest.SlotOrder slotOrder : request.getSlotOrders()) {
75         MealSlot slot = slotOrder.getMealSlot();
76         Long restaurantId = slotOrder.getRestaurantId();
77
78         // 1. Validate restaurant exists and is active
79         Restaurant restaurant = restaurantRepository.findById(restaurantId)
80             .orElseThrow(() -> new RestaurantNotFoundException(
81                 "Restaurant with id " + restaurantId + " not found"));
82         if (!restaurant.isActive()) {
83             throw new IllegalArgumentException(
84                 "Restaurant '" + restaurant.getName() + "' is not currently active");
85         }
86
87         // 2. Validate the restaurant actually serves this meal slot
88         if (!restaurant.getSupportedMealSlots().contains(slot)) {
89             throw new IllegalArgumentException(
90                 "Restaurant '" + restaurant.getName() + "' does not serve " + slot +
91                 ". Supported slots: " + restaurant.getSupportedMealSlots());
92         }
93
94         // 3. Check for duplicate active subscription (same employee + slot)
95         if (subscriptionRepository.checkSubscriptionExists(
96             employee.getEmployeeId(), slot.name(), SubscriptionStatus.ACTIVE.name()) > 0) {
97             throw new SubscriptionAlreadyExists(
98                 "Employee " + employee.getEmployeeId() +
99                 " already has an active " + slot + " subscription. " +
100                 "Cancel the existing one before subscribing to a new restaurant.");
101         }
102
103         // 4. Create subscription
104         Subscription subscription = new Subscription();
105         subscription.setEmployee(employee);
106         subscription.setSlot(slot);
107         subscription.setRestaurant(restaurant);
108         subscription.setScheduleType(request.getScheduleType());
109         subscription.setDayOfWeek(request.getDayOfWeek());
110         subscription.setStatus(SubscriptionStatus.ACTIVE);
111         subscription.setNextDeliveryTime(
112             deliveryTimeService.calculateNextDelivery(request, slot, employee));
113
114         subscriptionRepository.save(subscription);
115     }
116 }
117
118 public List<SubscriptionDTO> getAllSubscriptions() {
119     List<Subscription> subscriptions = subscriptionRepository.findAll();
120     List<SubscriptionDTO> dtoList = new ArrayList<>();
121     for (Subscription s : subscriptions) {
122         dtoList.add(convertToDTO(s));
123     }
124     return dtoList;
125 }
126
127 public Subscription updateSubscription(Long id, SubscriptionRequest request) {
128     Subscription subscription = subscriptionRepository.findById(id)
129         .orElseThrow(() -> new RuntimeException("Subscription not found"));
130     Employee employee = subscription.getEmployee();
131
132     if (request.getScheduleType() == ScheduleType.DAILY) {
133         subscription.setDayOfWeek(null);
134     } else if (request.getScheduleType() == ScheduleType.WEEKLY) {
135         if (request.getDayOfWeek() == null) {
136             throw new IllegalArgumentException("dayOfWeek is required for WEEKLY schedule");
137         }
138         subscription.setDayOfWeek(request.getDayOfWeek());
139     }
140     subscription.setScheduleType(request.getScheduleType());
141
142     // Update slot & restaurant if a single slotOrder is provided
143     if (request.getSlotOrders() != null && !request.getSlotOrders().isEmpty()) {
144         if (request.getSlotOrders().size() > 1) {
145             throw new IllegalArgumentException(
146                 "Update supports one slot at a time. Use separate subscriptions for multiple
slots.");

```

```

147     }
148     SubscriptionRequest.SlotOrder slotOrder = request.getSlotOrders().get(0);
149     MealSlot newSlot = slotOrder.getMealSlot();
150     Restaurant restaurant = restaurantRepository.findById(slotOrder.getRestaurantId())
151         .orElseThrow(() -> new RestaurantNotFoundException(
152             "Restaurant " + slotOrder.getRestaurantId() + " not found"));
153     if (!restaurant.isActive()) {
154         throw new IllegalArgumentException("Restaurant '" + restaurant.getName() + "' is not
active");
155     }
156     if (!restaurant.getSupportedMealSlots().contains(newSlot)) {
157         throw new IllegalArgumentException(
158             "Restaurant '" + restaurant.getName() + "' does not serve " + newSlot);
159     }
160     subscription.setSlot(newSlot);
161     subscription.setRestaurant(restaurant);
162 }
163
164     subscription.setNextDeliveryTime(
165         deliveryTimeService.calculateNextDelivery(request, subscription.getSlot(), employee));
166
167     return subscriptionRepository.save(subscription);
168 }
169
170 public Subscription pauseSubscription(Long id) {
171     Subscription subscription = subscriptionRepository.findById(id)
172         .orElseThrow(() -> new RuntimeException("Subscription not found"));
173     subscription.setStatus(SubscriptionStatus.PAUSED);
174     subscription.setNextDeliveryTime(null);
175     return subscriptionRepository.save(subscription);
176 }
177
178 public Subscription resumeSubscription(Long id) {
179     Subscription subscription = subscriptionRepository.findById(id)
180         .orElseThrow(() -> new RuntimeException("Subscription not found"));
181     subscription.setStatus(SubscriptionStatus.ACTIVE);
182     subscription.setNextDeliveryTime(deliveryTimeService.getNextDeliveryTime(subscription));
183     return subscriptionRepository.save(subscription);
184 }
185
186 public Subscription expireSubscription(Long id) {
187     Subscription subscription = subscriptionRepository.findById(id)
188         .orElseThrow(() -> new RuntimeException("Subscription not found"));
189     subscription.setStatus(SubscriptionStatus.EXPIRED);
190     subscription.setNextDeliveryTime(null);
191     return subscriptionRepository.save(subscription);
192 }
193
194 public List<SubscriptionDTO> getSubscriptionOfUser(Authentication authentication) {
195     String email= AuthUtil.extractEmail(authentication);
196     List<Subscription> subscriptions = subscriptionRepository.findByEmployee_Email(email);
197     List<SubscriptionDTO> dtoList = new ArrayList<>();
198     for (Subscription s : subscriptions) {
199         dtoList.add(convertToDTO(s));
200     }
201     return dtoList;
202 }
203
204 // helpers
205
206 private SubscriptionDTO convertToDTO(Subscription s) {
207     SubscriptionDTO dto = new SubscriptionDTO();
208     dto.setSubscriptionId(s.getId());
209     dto.setUserId(s.getEmployee().getEmployeeId());
210     dto.setMealSlot(s.getSlot());
211     dto.setScheduleType(s.getScheduleType());
212     if (s.getScheduleType() == ScheduleType.WEEKLY) {
213         dto.setDayOfWeek(s.getDayOfWeek());
214     }
215     dto.setStatus(s.getStatus());
216     if (s.getNextDeliveryTime() != null) {
217         ZoneId userZone = ZoneId.of(s.getEmployee().getTimezone());
218         dto.setNextDeliveryTime(s.getNextDeliveryTime().atZone(userZone));
219     }
220     // Restaurant info
221     if (s.getRestaurant() != null) {

```

```
222         dto.setRestaurantId(s.getRestaurant().getId());
223         dto.setRestaurantName(s.getRestaurant().getName());
224         dto.setRestaurantAddress(s.getRestaurant().getAddress());
225     }
226     return dto;
227 }
228 }
```

```
1 package com.example.EmployeeManagementSystem.Service;
2
3 import dev.samstevens.totp.code.*;
4 import dev.samstevens.totp.secret.DefaultSecretGenerator;
5 import dev.samstevens.totp.secret.SecretGenerator;
6 import dev.samstevens.totp.time.SystemTimeProvider;
7 import dev.samstevens.totp.time.TimeProvider;
8 import org.springframework.stereotype.Service;
9
10 @Service
11 public class TotpService {
12
13     private final SecretGenerator secretGenerator = new DefaultSecretGenerator();
14     private final TimeProvider timeProvider = new SystemTimeProvider();
15     private final CodeGenerator codeGenerator = new DefaultCodeGenerator();
16     private final CodeVerifier codeVerifier = new DefaultCodeVerifier(codeGenerator, timeProvider);
17
18     /** Generate a new Base32 secret for a user */
19     public String generateSecret() {
20         return secretGenerator.generate();
21     }
22
23     /**
24      * Generate a QR code URI – user scans this with Google Authenticator / Authy.
25      * Format: otpauth://totp/{issuer}:{username}?secret={secret}&issuer={issuer}
26      */
27     public String getQrCodeUri(String secret, String username) {
28         return String.format(
29             "otpauth://totp/EmployeeMS:%s?secret=%s&issuer=EmployeeManagementSystem",
30             username, secret
31         );
32     }
33
34     /**
35      * Verify the 6-digit code the user typed.
36      * DefaultCodeVerifier already allows ±1 time step (±30 seconds) for clock skew.
37      */
38     public boolean verifyCode(String secret, String code) {
39         try {
40             return codeVerifier.isValidCode(secret, code);
41         } catch (Exception e) {
42             return false;
43         }
44     }
45 }
```

```

1 package com.example.EmployeeManagementSystem.Service;
2
3 import com.example.EmployeeManagementSystem.Entity.*;
4 import com.example.EmployeeManagementSystem.Enum.Status;
5 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
6 import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
7 import com.example.EmployeeManagementSystem.Repository.LeaveWarningRepository;
8 import com.example.EmployeeManagementSystem.Repository.YearEndCarryForwardLogRepository;
9 import org.slf4j.Logger;
10 import org.slf4j.LoggerFactory;
11 import org.springframework.stereotype.Service;
12 import org.springframework.transaction.annotation.Transactional;
13
14 import java.math.BigDecimal;
15 import java.math.RoundingMode;
16 import java.time.LocalDate;
17 import java.util.List;
18 import java.util.Map;
19 import java.util.Set;
20 import java.util.stream.Collectors;
21
22 @Service
23 public class YearEndCarryForwardService {
24
25     private static final Logger log = LoggerFactory.getLogger(YearEndCarryForwardService.class);
26     private static final BigDecimal CAP = BigDecimal.valueOf(30);
27     private static final Set<String> SUPPORTED_LEAVE_TYPES = Set.of("SICK", "CASUAL", "MATERNITY");
28
29     private final EmployeeRepo employeeRepository;
30     private final LeaveEntitlementRepository entitlementRepository;
31     private final YearEndCarryForwardLogRepository logRepository;
32     private final LeaveWarningRepository warningRepository;
33
34     public YearEndCarryForwardService(
35         EmployeeRepo employeeRepository,
36         LeaveEntitlementRepository entitlementRepository,
37         YearEndCarryForwardLogRepository logRepository,
38         LeaveWarningRepository warningRepository) {
39         this.employeeRepository = employeeRepository;
40         this.entitlementRepository = entitlementRepository;
41         this.logRepository = logRepository;
42         this.warningRepository = warningRepository;
43     }
44
45     // -----
46     // Main entry point – called by scheduler on Dec 31 at 22:00
47     // -----
48     @Transactional
49     public void runYearEnd(int closingYear) {
50
51         // Idempotency guard – never run twice for same year
52         if (logRepository.existsByProcessedYear(closingYear)) {
53             log.warn("Year-end carry-forward already processed for {}. Skipping.", closingYear);
54             return;
55         }
56
57         List<Employee> activeEmployees = employeeRepository.findByStatus(Status.ACTIVE);
58         log.info("Starting year-end carry-forward for {} – {} employees", closingYear,
59             activeEmployees.size());
60         for (Employee employee : activeEmployees) {
61             processEmployeeYearEnd(employee, closingYear);
62         }
63
64         log.info("Year-end carry-forward complete for {}", closingYear);
65     }
66
67     // -----
68     // Per-employee processing
69     // -----
70     private void processEmployeeYearEnd(Employee employee, int closingYear) {

```

```

71     int nextYear = closingYear + 1;
72
73     // Fetch all entitlements for the closing year
74     List<LeaveEntitlement> entitlements =
75         entitlementRepository.findByEmployeeEmployeeIdAndYear(employee.getEmployeeId(),
closingYear);
76
77     // Separate carry-forwardable from non-carry-forwardable
78     List<LeaveEntitlement> carryForwardable = entitlements.stream()
79         .filter(e -> SUPPORTED_LEAVE_TYPES.contains(e.getLeaveType().getName()))
80         .filter(e -> e.getLeaveType().isCarriesForward())
81         .collect(Collectors.toList());
82
83     // Step 1 – compute raw closing balance for each type
84     //         closing = opening + accrued - used
85     Map<LeaveEntitlement, BigDecimal> rawClosing = carryForwardable.stream()
86         .collect(Collectors.toMap(
87             e -> e,
88             e -> e.getOpeningBalance()
89                 .add(e.getAccruedThisYear())
90                 .subtract(e.getUsedThisYear())
91                 .max(BigDecimal.ZERO) // never negative
92         ));
93
94     // Step 2 – sum all carry-forwardable closing balances
95     BigDecimal totalRaw = rawClosing.values().stream()
96         .reduce(BigDecimal.ZERO, BigDecimal::add);
97
98     // Step 3 – apply cap proportionally if total exceeds 30
99     Map<LeaveEntitlement, BigDecimal> cappedClosing = applyCap(rawClosing, totalRaw);
100
101     // Step 4 – persist: update closing balance on current year row,
102     //         create/update opening balance on next year row, write log
103     for (LeaveEntitlement entitlement : carryForwardable) {
104         BigDecimal raw = rawClosing.get(entitlement);
105         BigDecimal capped = cappedClosing.get(entitlement);
106         BigDecimal wasted = raw.subtract(capped);
107
108         // Update closing balance on the current year's row
109         entitlement.setClosingBalance(capped);
110         entitlementRepository.save(entitlement);
111
112         // Write or update next year's opening balance
113         LeaveEntitlement nextYearRow = entitlementRepository
114             .findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
115                 employee.getEmployeeId(), entitlement.getLeaveType().getId(), nextYear)
116             .orElseGet(() -> buildNextYearRow(employee, entitlement.getLeaveType(), nextYear));
117
118         nextYearRow.setOpeningBalance(capped);
119         entitlementRepository.save(nextYearRow);
120
121         // Write audit log
122         writeLog(employee, closingYear, entitlement.getLeaveType(), raw, capped, wasted);
123
124         // Issue DAY_WASTED warning if any days were lost
125         if (wasted.compareTo(BigDecimal.ZERO) > 0) {
126             issueWastedWarning(employee, wasted, capped);
127             log.info("Employee {} lost {} day(s) of {} to cap at year end",
128                 employee.getEmployeeId(), wasted, entitlement.getLeaveType().getName());
129         }
130     }
131
132     // Step 5 – zero out sick leave closing balance (it never carries)
133     entitlements.stream()
134         .filter(e -> SUPPORTED_LEAVE_TYPES.contains(e.getLeaveType().getName()))
135         .filter(e -> !e.getLeaveType().isCarriesForward())
136         .forEach(e -> {
137             e.setClosingBalance(BigDecimal.ZERO);
138             entitlementRepository.save(e);
139         });
140
141     log.info("Year-end processed for employee {} – total raw={{} capped={}",
142         employee.getEmployeeId(), totalRaw, CAP.min(totalRaw));
143 }
144
145 // -----

```

```

146 // Cap logic – distribute cap proportionally across leave types
147 //
148 // Example: casual=20, maternity=16 total=36, over cap by 6
149 // casual gets: 20/36 * 30 = 16.67
150 // maternity gets: 16/36 * 30 = 13.33
151 // total = 30 exactly
152 // -----
153 private Map<LeaveEntitlement, BigDecimal> applyCap(
154     Map<LeaveEntitlement, BigDecimal> rawClosing,
155     BigDecimal totalRaw) {
156
157     if (totalRaw.compareTo(CAP) <= 0) {
158         // Under cap – no adjustment needed
159         return rawClosing;
160     }
161
162     return rawClosing.entrySet().stream()
163         .collect(Collectors.toMap(
164             Map.Entry::getKey,
165             entry -> {
166                 BigDecimal raw = entry.getValue();
167                 // proportion = raw / totalRaw * CAP
168                 return raw
169                     .divide(totalRaw, 10, RoundingMode.HALF_UP)
170                     .multiply(CAP)
171                     .setScale(2, RoundingMode.HALF_UP);
172             }
173         ));
174 }
175
176 // -----
177 // Build a blank next-year entitlement row
178 // -----
179 private LeaveEntitlement buildNextYearRow(Employee employee, LeaveType leaveType, int year) {
180     LeaveEntitlement row = new LeaveEntitlement();
181     row.setEmployee(employee);
182     row.setLeaveType(leaveType);
183     row.setYear(year);
184     row.setOpeningBalance(BigDecimal.ZERO);
185     row.setAccruedThisYear(BigDecimal.ZERO);
186     row.setUsedThisYear(BigDecimal.ZERO);
187     row.setClosingBalance(BigDecimal.ZERO);
188     return entitlementRepository.save(row);
189 }
190
191 // -----
192 // Write audit log entry
193 // -----
194 private void writeLog(Employee employee, int year, LeaveType leaveType,
195     BigDecimal raw, BigDecimal capped, BigDecimal wasted) {
196     YearEndCarryForwardLog entry = new YearEndCarryForwardLog();
197     entry.setEmployee(employee);
198     entry.setProcessedYear(year);
199     entry.setLeaveType(leaveType);
200     entry.setClosingBalance(raw);
201     entry.setCappedBalance(capped);
202     entry.setDaysWasted(wasted);
203     entry.setOpeningNextYear(capped);
204     logRepository.save(entry);
205 }
206
207 // -----
208 // Issue a DAY_WASTED warning on the dashboard
209 // -----
210 private void issueWastedWarning(Employee employee, BigDecimal wasted, BigDecimal newBalance) {
211     LeaveWarning warning = new LeaveWarning();
212     warning.setEmployee(employee);
213     warning.setWarningType(LeaveWarning.WarningType.DAY_WASTED);
214     warning.setCarryForwardBalance(newBalance);
215     warning.setWarningDate(LocalDate.now());
216     warning.setMessage(
217         wasted.toPlainString() + " carry-forward day(s) were lost at year-end " +
218         "because your balance exceeded the 30-day cap. " +
219         "Your opening balance for next year is " + newBalance.toPlainString() + " days."
220     );
221     warningRepository.save(warning);

```

```
222     }
223 }
224
```



```
1 package com.example.EmployeeManagementSystem.Util;
2
3 import org.springframework.security.core.Authentication;
4 import org.springframework.security.oauth2.core.user.OAuth2User;
5 import org.springframework.stereotype.Component;
6
7 @Component
8 public class AuthUtil {
9     public static String extractEmail(Authentication authentication) {
10         if (authentication == null) return null;
11
12         // OAuth2 login (session-based) – principal is OAuth2User
13         if (authentication.getPrincipal() instanceof OAuth2User oAuth2User) {
14             return oAuth2User.getAttribute("email"); // always the actual email
15         }
16
17         // JWT or session username/password login – getName() is already the email
18         return authentication.getName();
19     }
20 }
21
```

```
1 package com.example.EmployeeManagementSystem.Util;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.Vendor;
5 import com.example.EmployeeManagementSystem.Enum.Role;
6 import com.example.EmployeeManagementSystem.Exception.EmployeeNotFound;
7 import com.example.EmployeeManagementSystem.Exception.VendorNotFoundException;
8 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
9 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
10 import io.jsonwebtoken.Claims;
11 import io.jsonwebtoken.Jwts;
12 import io.jsonwebtoken.security.Keys;
13 import jakarta.annotation.PostConstruct;
14 import org.springframework.beans.factory.annotation.Value;
15 import org.springframework.security.core.userdetails.UserDetails;
16 import org.springframework.stereotype.Component;
17
18 import javax.crypto.SecretKey;
19 import java.util.Date;
20 import java.util.Optional;
21
22 @Component
23 public class JWTUtil {
24
25     private final EmployeeRepo employeeRepo;
26     private final VendorRepo vendorRepo;
27
28     // FIX 1: Was 1000*60*60 (1 hour) – caused token expiry mid-session.
29     // Extended to 24 hours to match the session timeout in application.yaml.
30     private final long Expiration_time = 1000L * 60 * 60 * 24;
31
32     @Value("${jwt.secret}")
33     private String secret;
34     private SecretKey key;
35
36     public JWTUtil(EmployeeRepo employeeRepo, VendorRepo vendorRepo) {
37         this.employeeRepo = employeeRepo;
38         this.vendorRepo = vendorRepo;
39     }
40
41     @PostConstruct
42     public void init() {
43         key = Keys.hmacShaKeyFor(secret.getBytes());
44     }
45
46     public String generateToken(String username, String role) {
47         return Jwts.builder()
48             .subject(username)
49             .claim("role", role)
50             .issuedAt(new Date())
51             .expiration(new Date(System.currentTimeMillis() + Expiration_time))
52             .signWith(key)
53             .compact();
54     }
55
56     public String extractUsernameFromToken(String token) {
57         Claims body = extractClaims(token);
58         return body.getSubject();
59     }
60
61     private Claims extractClaims(String token) {
62         return Jwts.parser()
63             .verifyWith(key)
64             .build()
65             .parseSignedClaims(token)
66             .getPayload();
67     }
68
69     public boolean validateToken(String username, UserDetails userDetails, String token) {
70         return username.equals(userDetails.getUsername()) && !isTokenExpired(token);
71     }
```

```
72
73     private boolean isTokenExpired(String token) {
74         return extractClaims(token).getExpiration().before(new Date());
75     }
76 }
```

```

1 package com.example.EmployeeManagementSystem.Util;
2
3 import com.example.EmployeeManagementSystem.Entity.RefreshToken;
4 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
5 import com.example.EmployeeManagementSystem.Repository.RefreshTokenRepository;
6 import com.example.EmployeeManagementSystem.Repository.VendorRepo;
7 import com.example.EmployeeManagementSystem.Service.RefreshTokenService;
8 import jakarta.servlet.http.HttpServletRequest;
9 import jakarta.servlet.http.HttpServletResponse;
10 import org.springframework.security.core.Authentication;
11 import org.springframework.security.oauth2.core.user.OAuth2User;
12 import org.springframework.security.web.authentication.SimpleUrlAuthenticationSuccessHandler;
13 import org.springframework.stereotype.Service;
14
15 import java.io.IOException;
16 import java.net.URLEncoder;
17 import java.nio.charset.StandardCharsets;
18
19 @Service
20 public class OAuth2SuccessHandler extends SimpleUrlAuthenticationSuccessHandler{
21     private final JWTUtil jwtUtil;
22     private final EmployeeRepo employeeRepo;
23     private final VendorRepo vendorRepo;
24     private final RefreshTokenService refreshTokenService;
25     private final RefreshTokenRepository refreshTokenRepository;
26
27     public OAuth2SuccessHandler(JWTUtil jwtUtil,
28                               EmployeeRepo employeeRepo,
29                               VendorRepo vendorRepo,
30                               RefreshTokenService refreshTokenService,
31                               RefreshTokenRepository refreshTokenRepository) {
32         this.jwtUtil = jwtUtil;
33         this.employeeRepo = employeeRepo;
34         this.vendorRepo = vendorRepo;
35         this.refreshTokenService = refreshTokenService;
36         this.refreshTokenRepository = refreshTokenRepository;
37     }
38
39     @Override
40     public void onAuthenticationSuccess(HttpServletRequest request,
41                                       HttpServletResponse response,
42                                       Authentication authentication) throws IOException {
43
44         try{
45             OAuth2User oAuth2User = (OAuth2User) authentication.getPrincipal();
46             String email = oAuth2User.getAttribute("email");
47
48             // Generate refresh token – check which entity type owns this email
49             RefreshToken refreshToken;
50             String role;
51             var employeeOpt = employeeRepo.findByEmail(email);
52             var vendorOpt = vendorRepo.findByEmail(email);
53
54             if (employeeOpt.isPresent()) {
55                 role = employeeOpt.get().getRole().name();
56                 refreshToken = refreshTokenService.createForEmployee(employeeOpt.get());
57             } else if (vendorOpt.isPresent()) {
58                 role = vendorOpt.get().getRole().name();
59                 refreshToken = refreshTokenService.createForVendor(vendorOpt.get());
60             } else {
61                 response.sendError(HttpServletResponse.SC_INTERNAL_SERVER_ERROR, "User not found post-
62 OAuth");
63                 return;
64             }
65             String accessToken = jwtUtil.generateToken(email, role);
66             refreshTokenRepository.save(refreshToken);
67
68             // Redirect frontend to a callback page with tokens as query params
69             // (or use a fragment # so tokens don't hit server logs)
70             String redirectUrl = "http://localhost:8080/google.html" // your frontend page

```

```
71         + "?accessToken=" + URLEncoder.encode(accessToken, StandardCharsets.UTF_8)
72         + "&refreshToken=" + URLEncoder.encode(refreshToken.getToken(),
StandardCharsets.UTF_8);
73
74         getRedirectStrategy().sendRedirect(request, response, redirectUrl);
75     }
76     catch (Exception e) {
77         // This will now print the REAL error in your console
78         System.err.println("=== OAuth2SuccessHandler FAILED ===");
79         e.printStackTrace();
80         response.sendError(HttpServletResponse.SC_INTERNAL_SERVER_ERROR,
81             "OAuth2 login failed: " + e.getMessage());
82     }
83 }
84 }
85
```

```
1  spring:
2    application:
3      name: EmployeeManagementSystem
4
5    datasource :
6      url : jdbc:mysql://localhost:3306/employee_management_db
7      username : ${USER_NAME}
8      password : ${PASSWORD}
9
10   jpa:
11     hibernate :
12       ddl-auto: update
13     show-sql: true
14
15   springdoc:
16     api-docs:
17       path: /v3/api-docs
18       enabled: true
19     swagger-ui:
20       path: /swagger-ui.html
21       enabled: true
22       operations-sorter: method
23       tags-sorter: alpha
24       try-it-out-enabled: true
25       filter: true
26     show-actuator: true
27   mvc:
28     static-path-pattern: /**
29   session:
30     tracking-modes: COOKIE
31   web:
32     resources:
33       static-locations: classpath:/static/
34       add-mappings: true
35   session:
36     timeout: 86400s # 24 hours in seconds (was 1440m which is incorrect format)
37     store-type: none # Use default cookie-based session
38
39   servlet:
40     session:
41       cookie:
42         http-only: true
43         same-site: strict
44         secure: false # set true in production with HTTPS
45
46   security:
47     debug: true
48     oauth2:
49       client:
50         registration:
51           google:
52             client-id: ${client_id}
53             client-secret: ${client_secret}
54             scope:
55               - email
56               - profile
57
58   logging:
59     level:
60       org.springframework.security: DEBUG
61       com.example.EmployeeManagementSystem: DEBUG
62   jwt:
63     secret: ${JWT_SECRET}
64     # Add this to your application.yaml
65   api:
66     key:
67       enabled: true
68       header-name: X-API-Key
69
```

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>EMS - Admin Console</title>
7      <link
href="https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600;700&family=DM+Mono:wght@400;500&display=
swap" rel="stylesheet">
8      <link href="https://cdn.jsdelivr.net/npm/fullcalendar@6.1.8/index.global.min.css" rel="stylesheet">
9      <script src="https://cdn.jsdelivr.net/npm/fullcalendar@6.1.8/index.global.min.js"></script>
10     <style>
11         * { margin: 0; padding: 0; box-sizing: border-box; }
12         :root {
13             --bg: #0d0f14; --bg2: #131720; --bg3: #1a2030;
14             --panel: #1e2436; --panel2: #252c40;
15             --border: rgba(255,255,255,0.07); --border2: rgba(255,255,255,0.12);
16             --text: #e0e0e0; --text2: #8b92a8; --text3: #5557fa;
17             --accent: #4f8cff; --accent-dim: rgba(79,140,255,0.12);
18             --green: #36d399; --green-dim: rgba(54,211,153,0.12);
19             --amber: #f9c74f; --amber-dim: rgba(249,199,79,0.12);
20             --red: #f87171; --red-dim: rgba(248,113,113,0.12);
21             --violet: #a78bfa; --violet-dim: rgba(167,139,250,0.12);
22             --font: 'DM Sans', sans-serif; --mono: 'DM Mono', monospace;
23         }
24         body { font-family: var(--font); background: var(--bg); color: var(--text); font-size: 14px; line-
height: 1.5; overflow: hidden; }
25         .app { display: flex; height: 100vh; }
26         .sidebar { width: 260px; background: var(--bg2); border-right: 1px solid var(--border); display:
flex; flex-direction: column; overflow-y: auto; }
27         .logo { padding: 20px; border-bottom: 1px solid var(--border); }
28         .logo h2 { font-size: 20px; color: var(--text); }
29         .logo span { color: var(--accent); }
30         .logo p { font-size: 11px; color: var(--text3); margin-top: 4px; }
31         .nav-item { padding: 12px 20px; margin: 4px 12px; border-radius: 8px; cursor: pointer; transition:
all 0.2s; color: var(--text2); display: flex; align-items: center; gap: 10px; }
32         .nav-item:hover, .nav-item.active { background: var(--accent-dim); color: var(--accent); }
33         .main-content { flex: 1; display: flex; flex-direction: column; overflow: hidden; }
34         .topbar { background: var(--bg2); padding: 16px 24px; display: flex; justify-content: space-between;
align-items: center; border-bottom: 1px solid var(--border); }
35         .page-title { font-size: 20px; font-weight: 600; }
36         .user-info { display: flex; align-items: center; gap: 16px; }
37         .logout-btn { padding: 8px 16px; background: var(--red-dim); border: 1px solid
rgba(248,113,113,0.3); border-radius: 6px; color: var(--red); cursor: pointer; }
38         .content { flex: 1; overflow-y: auto; padding: 24px; }
39         .view { display: none; }
40         .view.active { display: block; }
41         .stats-grid { display: grid; grid-template-columns: repeat(4, 1fr); gap: 16px; margin-bottom: 24px;
}
42         .stat-card { background: var(--panel); padding: 20px; border-radius: 12px; border: 1px solid var(--
border); }
43         .stat-label { font-size: 11px; color: var(--text3); text-transform: uppercase; margin-bottom: 8px; }
44         .stat-value { font-size: 28px; font-weight: 600; }
45         .form-card { background: var(--panel); border-radius: 12px; border: 1px solid var(--border);
padding: 24px; max-width: 600px; }
46         .form-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; }
47         .form-group { display: flex; flex-direction: column; gap: 6px; }
48         .form-group.full { grid-column: 1 / -1; }
49         label { font-size: 11px; font-weight: 600; color: var(--text3); text-transform: uppercase; }
50         input, select, textarea { padding: 10px 12px; background: var(--bg3); border: 1px solid var(--
border2); border-radius: 8px; color: var(--text); font-family: var(--font); font-size: 13px; }
51         input:focus, select:focus, textarea:focus { outline: none; border-color: var(--accent); }
52         .table-wrap { background: var(--panel); border-radius: 12px; border: 1px solid var(--border);
overflow-x: auto; }
53         table { width: 100%; border-collapse: collapse; }
54         th { text-align: left; padding: 12px 16px; background: var(--bg3); color: var(--text3); font-size:
11px; font-weight: 600; text-transform: uppercase; }
55         td { padding: 12px 16px; border-top: 1px solid var(--border); }
56         .chip { padding: 4px 10px; border-radius: 20px; font-size: 11px; font-weight: 600; display: inline-b
lock; }
57         .chip.active, .chip.approved, .chip.delivered, .chip.repair_done { background: var(--green-dim);
color: var(--green); }

```

```

58     .chip.pending, .chip.scheduled, .chip.sent_for_repair { background: var(--amber-dim); color: var(--
amber); }
59     .chip.open { background: var(--accent-dim); color: var(--accent); }
60     .chip.inactive, .chip.rejected, .chip.cancelled, .chip.expired { background: var(--red-dim); color:
var(--red); }
61     .chip.on_leave { background: var(--violet-dim); color: var(--violet); }
62     .chip.employee { background: var(--accent-dim); color: var(--accent); }
63     .chip.manager { background: var(--violet-dim); color: var(--violet); }
64     .chip.vendor { background: var(--red-dim); color: var(--red); }
65     .chip.under_repair { background: var(--amber-dim); color: var(--amber); }
66     .chip.unpaid, .chip.UNPAID { background: var(--violet-dim); color: var(--violet); }
67     .chip.assigned { background: var(--green-dim); color: var(--green); }
68     .chip.condemned { background: var(--red-dim); color: var(--red); }
69     .chip.admin { background: var(--amber-dim); color: var(--amber); }
70     .btn { padding: 8px 16px; border-radius: 6px; border: none; font-family: var(--font); font-size:
13px; font-weight: 500; cursor: pointer; transition: all 0.2s; }
71     .btn-primary { background: var(--accent); color: white; }
72     .btn-primary:hover { background: #6ba0ff; }
73     .btn-secondary { background: var(--bg3); color: var(--text2); border: 1px solid var(--border2); }
74     .btn-danger { background: var(--red-dim); color: var(--red); border: 1px solid
rgba(248,113,113,0.3); }
75     .btn-sm { padding: 4px 10px; font-size: 11px; margin: 2px; }
76     .section-head { display: flex; justify-content: space-between; align-items: center; margin-bottom:
16px; }
77     .section-title { font-size: 16px; font-weight: 600; }
78     .empty-state { text-align: center; padding: 40px; color: var(--text3); }
79     .modal-overlay { display: none; position: fixed; top: 0; left: 0; right: 0; bottom: 0; background:
rgba(0,0,0,0.7); z-index: 1000; align-items: center; justify-content: center; }
80     .modal-overlay.open { display: flex; }
81     .modal { background: var(--panel); border-radius: 12px; padding: 24px; width: 450px; max-width: 90%;
}
82     .modal-title { font-size: 18px; font-weight: 600; margin-bottom: 16px; }
83     .modal-actions { display: flex; gap: 12px; margin-top: 20px; justify-content: flex-end; }
84     .alert { position: fixed; top: 20px; right: 20px; padding: 12px 20px; border-radius: 8px; z-index:
1100; animation: slideIn 0.3s ease; }
85     .alert-success { background: var(--green-dim); color: var(--green); border: 1px solid var(--green);
}
86     .alert-error { background: var(--red-dim); color: var(--red); border: 1px solid var(--red); }
87     @keyframes slideIn { from { transform: translateX(100%); opacity: 0; } to { transform:
translateX(0); opacity: 1; } }
88     .action-row { display: flex; gap: 6px; flex-wrap: wrap; }
89     .notif-wrapper { position: relative; cursor: pointer; }
90
91     .notif-badge {
92         position: absolute; top: -5px; right: -6px;
93         background: #e53e3e; color: white;
94         font-size: 10px; font-weight: 600;
95         min-width: 16px; height: 16px;
96         border-radius: 50%; display: flex;
97         align-items: center; justify-content: center;
98         padding: 0 3px;
99     }
100    .notif-panel {
101        position: absolute; top: 48px; right: 0;
102        width: 320px; max-height: 400px; overflow-y: auto;
103        background: var(--panel);
104        border: 1px solid var(--border2);
105        border-radius: 10px;
106        box-shadow: 0 4px 24px rgba(0,0,0,0.4);
107        z-index: 999;
108    }
109    .notif-header {
110        padding: 12px 16px; font-weight: 600;
111        color: var(--text);
112        border-bottom: 1px solid var(--border);
113    }
114    .notif-item {
115        padding: 12px 16px; font-size: 13px;
116        color: var(--text2);
117        border-bottom: 1px solid var(--border);
118    }
119    .notif-item.unread { background: var(--accent-dim); color: var(--text); }
120    .notif-time { font-size: 11px; color: var(--text3); margin-top: 4px; }
121
122    /* ===== CALENDAR STYLES ===== */
123    .leave-balance-grid {

```



```

124     display: grid;
125     grid-template-columns: repeat(auto-fit, minmax(160px, 1fr));
126     gap: 14px; padding: 20px;
127     border-bottom: 1px solid var(--border);
128 }
129 .leave-bal-card {
130     background: var(--bg3); border: 1px solid var(--border);
131     border-radius: 10px; padding: 16px 18px;
132     display: flex; flex-direction: column; gap: 6px;
133     position: relative; overflow: hidden; transition: border-color 0.2s;
134 }
135 .leave-bal-card:hover { border-color: var(--border2); }
136 .leave-bal-card::before {
137     content: ''; position: absolute;
138     left: 0; top: 0; bottom: 0; width: 3px;
139     border-radius: 3px 0 0 3px;
140 }
141 .leave-bal-card.sick::before { background: var(--red); }
142 .leave-bal-card.casual::before { background: var(--amber); }
143 .leave-bal-card.mat::before { background: var(--accent); }
144 .leave-bal-card.total::before { background: var(--green); }
145 .lbc-icon { font-size: 18px; }
146 .lbc-label { font-size: 10px; color: var(--text3); text-transform: uppercase; letter-spacing: 1px; font-weight: 600; }
147 .lbc-count { font-size: 24px; font-weight: 700; color: var(--text); font-family: var(--mono); line-height: 1; }
148 .lbc-sub { font-size: 11px; color: var(--text3); }
149 .cal-legend {
150     display: flex; align-items: center; gap: 20px;
151     padding: 12px 20px; border-bottom: 1px solid var(--border); flex-wrap: wrap;
152 }
153 .legend-label { font-size: 11px; color: var(--text3); font-weight: 600; text-transform: uppercase; letter-spacing: 0.8px; margin-right: 4px; }
154 .legend-item { display: flex; align-items: center; gap: 6px; font-size: 12px; color: var(--text2); }
155 .legend-dot { width: 10px; height: 10px; border-radius: 50%; flex-shrink: 0; }
156 .cal-section { background: var(--panel); border: 1px solid var(--border); border-radius: 12px; margin-bottom: 20px; overflow: hidden; }
157 .cal-section-head { padding: 16px 20px; border-bottom: 1px solid var(--border); display: flex; justify-content: space-between; align-items: center; }
158 .cal-section-title { font-size: 16px; font-weight: 600; color: var(--text); }
159 .cal-section-sub { font-size: 12px; color: var(--text2); margin-top: 2px; }
160 #employee-calendar { padding: 16px 20px 20px; }
161 #employee-calendar .fc .fc-toolbar-title { font-family: var(--font); font-size: 15px; font-weight: 600; color: var(--text); }
162 #employee-calendar .fc .fc-button {
163     background: var(--panel2); border: 1px solid var(--border2);
164     color: var(--text2); font-family: var(--font); font-size: 12px;
165     font-weight: 600; border-radius: 6px; padding: 5px 12px;
166     transition: all 0.2s; box-shadow: none; text-transform: capitalize;
167 }
168 #employee-calendar .fc .fc-button:hover,
169 #employee-calendar .fc .fc-button-active,
170 #employee-calendar .fc .fc-button:not(:disabled):active {
171     background: var(--accent-dim); border-color: var(--accent); color: var(--accent); box-shadow: none;
172 }
173 #employee-calendar .fc .fc-button:focus { box-shadow: none; outline: none; }
174 #employee-calendar .fc-theme-standard td,
175 #employee-calendar .fc-theme-standard th,
176 #employee-calendar .fc-theme-standard .fc-scrollgrid { border-color: var(--border); }
177 #employee-calendar .fc .fc-col-header-cell-cushion {
178     font-family: var(--font); font-size: 11px; font-weight: 600;
179     color: var(--text3); text-transform: uppercase; letter-spacing: 0.8px; padding: 8px 4px;
180 }
181 #employee-calendar .fc .fc-daygrid-day-number { font-family: var(--mono); font-size: 12px; color: var(--text3); padding: 6px 8px; }
182 #employee-calendar .fc .fc-daygrid-day.fc-day-today { background: var(--accent-dim) !important; }
183 #employee-calendar .fc .fc-daygrid-day.fc-day-today .fc-daygrid-day-number { color: var(--accent); font-weight: 700; }
184 #employee-calendar .fc-daygrid-day-frame { background: var(--bg3); }
185 #employee-calendar .fc .fc-day-other .fc-daygrid-day-frame { background: var(--bg2); }
186 #employee-calendar .fc-event {
187     border: none !important; border-radius: 4px;
188     font-family: var(--font); font-size: 11px; font-weight: 600; padding: 2px 6px; cursor: pointer;
189 }
190 .cal-tooltip {
191     position: fixed; background: var(--panel2); border: 1px solid var(--border2);

```

```

192     border-radius: 8px; padding: 10px 14px; font-size: 12px; color: var(--text);
193     z-index: 9999; pointer-events: none; max-width: 200px;
194     box-shadow: 0 8px 24px rgba(0,0,0,0.4);
195 }
196 .cal-tooltip .tt-title { font-weight: 600; margin-bottom: 4px; font-size: 13px; }
197 .cal-tooltip .tt-row { color: var(--text2); margin-top: 2px; }
198     </style>
199 </head>
200 <body>
201 <div class="app">
202     <!-- Sidebar -->
203     <div class="sidebar">
204         <div class="logo">
205             <h2>E<span>M</span>S</h2>
206             <p>Admin Control Center</p>
207         </div>
208         <div style="flex: 1;">
209             <div class="nav-item active" onclick="showView('dashboard', event)">&#9632; Dashboard</div>
210             <div class="nav-item" onclick="showView('employees', event)">&#9632; Employees</div>
211             <div class="nav-item" onclick="showView('add-employee', event)">&#9632; Add Employee</div>
212             <div class="nav-item" onclick="showView('add-manager', event)">&#9632; Add Manager</div>
213             <div class="nav-item" onclick="showView('add-vendor', event)">&#9632; Add Vendor</div>
214             <div class="nav-item" onclick="window.location.href='/leave_management.html'">&#9632; Leave
Management</div>
215             <div class="nav-item" onclick="showView('holidays', event)">&#9632; Holidays</div>
216             <div class="nav-item" onclick="showView('restaurants', event)">&#9632; Restaurants</div>
217             <div class="nav-item" onclick="showView('subscriptions', event)">&#9632; Subscriptions</div>
218             <div class="nav-item" onclick="showView('add-subscription', event)">&#9632; Add
Subscription</div>
219             <div class="nav-item" onclick="showView('vendors', event)">&#9632; Vendors</div>
220             <div class="nav-item" onclick="showView('devices', event)">&#9632; Devices</div>
221             <div class="nav-item" onclick="showView('my-devices', event)">&#9632; My Devices</div>
222             <div class="nav-item" onclick="showView('service-requests', event)">&#9632; Service
Requests</div>
223             <div class="nav-item" onclick="showView('admin-service-requests', event)">&#9632; All Service
Requests</div>
224             <div class="nav-item" onclick="showView('my-calendar', event)">&#9632; My Calendar</div>
225             <div class="nav-item" onclick="showView('my-profile', event)">&#9632; My Profile</div>
226             <div class="nav-item" onclick="showView('two-factor', event)">&#9632; Two-Factor Auth</div>
227         </div>
228     </div>
229
230     <!-- Main Content -->
231     <div class="main-content">
232         <div class="topbar">
233             <div class="page-title" id="pageTitle">Dashboard</div>
234             <!-- Bell icon with badge -->
235             <div class="notif-wrapper" onclick="toggleNotifPanel()">
236                 <svg width="22" height="22" viewBox="0 0 24 24" fill="none"
237                     stroke="currentColor" stroke-width="2" stroke-linecap="round">
238                     <path d="M18 8A6 6 0 0 0 6 8c0 7-3 9-3 9h18s-3-2-3-9"/>
239                     <path d="M13.73 21a2 2 0 0 1-3.46 0"/>
240                 </svg>
241                 <span id="notif-badge" class="notif-badge" style="display:none"></span>
242             </div>
243
244             <!-- Dropdown panel -->
245             <div id="notif-panel" class="notif-panel" style="display:none">
246                 <div class="notif-header">Notifications</div>
247                 <div id="notif-list"></div>
248             </div>
249             <div class="user-info">
250                 <span id="username-display"></span>
251                 <button class="logout-btn" onclick="logout()">Logout</button>
252             </div>
253         </div>
254         <div class="content" id="content">
255
256             <!-- DASHBOARD VIEW -->
257             <div id="dashboard-view" class="view active">
258                 <div class="stats-grid">
259                     <div class="stat-card"><div class="stat-label">Total Employees</div><div class="stat-
value" id="total-employees">--</div></div>
260                     <div class="stat-card"><div class="stat-label">Active</div><div class="stat-value"
id="active-employees">--</div></div>
261                     <div class="stat-card"><div class="stat-label">On Leave</div><div class="stat-value"

```

```

id="onleave-employees">--</div></div>
262         <div class="stat-card"><div class="stat-label">Pending Leaves</div><div class="stat-
value" id="pending-leaves">--</div></div>
263         </div>
264         <div class="section-head"><div class="section-title">Recent Leave Requests</div><button
class="btn btn-secondary btn-sm" onclick="loadDashboard()">Refresh</button></div>
265         <div class="table-
wrap"><table><thead><tr><th>Employee</th><th>Type</th><th>Start</th><th>End</th><th>Status</th></tr></thead><tbo
dy id="recent-leaves"></tbody></table></div>
266         </div>
267
268         <!-- EMPLOYEES VIEW -->
269         <div id="employees-view" class="view">
270             <div class="section-head"><div class="section-title">All Employees</div><button class="btn
btn-primary btn-sm" onclick="loadEmployees()">Refresh</button></div>
271             <div class="table-
wrap"><table><thead><tr><th>ID</th><th>Name</th><th>Email</th><th>Department</th><th>Role</th><th>Status</th><th>
>Timezone</th><th>Actions</th></tr></thead><tbody i...
272             </div>
273
274             <div id="add-employee-view" class="view">
275                 <div class="section-title" style="margin-bottom: 20px;">Add Employee</div>
276                 <div class="form-card">
277                     <div class="form-grid">
278                         <div class="form-group"><label>Name</label><input type="text" id="employee-name"
placeholder="Employee name"></div>
279                         <div class="form-group"><label>Email</label><input type="email" id="employee-email"
placeholder="employee@company.com"></div>
280                         <div class="form-group"><label>Password</label><input type="password" id="employee-
password" placeholder="Temporary password"></div>
281                         <div class="form-group">
282                             <label>Gender</label>
283                             <select id="employee-gender">
284                                 <option value="M">Male</option>
285                                 <option value="F">Female</option>
286                                 <option value="O">Other</option>
287                             </select>
288                         </div>
289                         <div class="form-group full">
290                             <label>Manager</label>
291                             <select id="employee-manager-select">
292                                 <option value="">-- Select Manager --</option>
293                             </select>
294                         </div>
295                         <div class="form-group full"><label>Timezone</label>
296                             <select id="employee-timezone">
297                                 <option value="Asia/Kolkata">India – IST UTC+5:30</option>
298                                 <option value="UTC">UTC</option>
299                                 <option value="America/New_York">New York – EST UTC-5</option>
300                                 <option value="America/Los_Angeles">Los Angeles – PST UTC-8</option>
301                                 <option value="Europe/London">London – GMT UTC+0</option>
302                                 <option value="Europe/Paris">Paris – CET UTC+1</option>
303                                 <option value="Asia/Tokyo">Tokyo – JST UTC+9</option>
304                                 <option value="Asia/Dubai">Dubai – GST UTC+4</option>
305                                 <option value="Australia/Sydney">Sydney – AEDT UTC+11</option>
306                                 <option value="Asia/Singapore">Singapore – SGT UTC+8</option>
307                             </select>
308                         </div>
309                     </div>
310                     <div class="modal-actions" style="margin-top: 20px;">
311                         <button class="btn btn-primary" onclick="createEmployee()">Create Employee</button>
312                         <button class="btn btn-secondary"
onclick="clearEmployeeForm('employee')">Clear</button>
313                     </div>
314                 </div>
315             </div>
316
317             <div id="add-manager-view" class="view">
318                 <div class="section-title" style="margin-bottom: 20px;">Add Manager</div>
319                 <div class="form-card">
320                     <div class="form-grid">
321                         <div class="form-group"><label>Name</label><input type="text" id="manager-name-
create" placeholder="Manager name"></div>
322                         <div class="form-group"><label>Email</label><input type="email" id="manager-email-
create" placeholder="manager@company.com"></div>
323                         <div class="form-group"><label>Password</label><input type="password" id="manager-

```

```

password-create" placeholder="Temporary password"></div>
324         <div class="form-group"><label>Department</label><input type="text" id="manager-
dept-create" placeholder="Department"></div>
325         <div class="form-group full"><label>Timezone</label>
326             <select id="manager-timezone-create">
327                 <option value="Asia/Kolkata">India – IST UTC+5:30</option>
328                 <option value="UTC">UTC</option>
329                 <option value="America/New_York">New York – EST UTC-5</option>
330                 <option value="America/Los_Angeles">Los Angeles – PST UTC-8</option>
331                 <option value="Europe/London">London – GMT UTC+0</option>
332                 <option value="Europe/Paris">Paris – CET UTC+1</option>
333                 <option value="Asia/Tokyo">Tokyo – JST UTC+9</option>
334                 <option value="Asia/Dubai">Dubai – GST UTC+4</option>
335                 <option value="Australia/Sydney">Sydney – AEDT UTC+11</option>
336                 <option value="Asia/Singapore">Singapore – SGT UTC+8</option>
337             </select>
338         </div>
339         <div class="form-group"><label>Gender</label>
340             <select id="manager-gender">
341                 <option value="M">Male</option>
342                 <option value="F">Female</option>
343                 <option value="O">Other</option>
344             </select>
345         </div>
346     </div>
347     <div class="modal-actions" style="margin-top: 20px;"><button class="btn btn-primary"
onclick="createManager()">Create Manager</button><button class="btn btn-secondary" onclick="...
348     </div>
349 </div>
350
351     <div id="add-vendor-view" class="view">
352         <div class="section-title" style="margin-bottom: 20px;">Add Vendor</div>
353         <div class="form-card">
354             <div class="form-grid">
355                 <div class="form-group"><label>Name</label><input type="text" id="vendor-name-
create" placeholder="Vendor name"></div>
356                 <div class="form-group"><label>Email</label><input type="email" id="vendor-email-
create" placeholder="vendor@company.com"></div>
357                 <div class="form-group"><label>Password</label><input type="password" id="vendor-
password-create" placeholder="Temporary password"></div>
358                 <div class="form-group"><label>Phone</label><input type="text" id="vendor-phone-
create" placeholder="+91 90000 00000"></div>
359                 <div class="form-group">
360                     <label>Role</label>
361                     <select id="vendor-role">
362                         <option value="TECH_VENDOR">tech vendor</option>
363                         <option value="FOOD_VENDOR">Food Vendor</option>
364                     </select>
365                 </div>
366             </div>
367             <div class="modal-actions" style="margin-top: 20px;"><button class="btn btn-primary"
onclick="createVendor()">Create Vendor</button><button class="btn btn-secondary" onclick="cl...
368             </div>
369         </div>
370
371     <!-- APPLY LEAVE VIEW -->
372     <div id="apply-leave-view" class="view">
373         <div class="section-title" style="margin-bottom: 20px;">Apply for Leave</div>
374         <div class="form-card">
375             <div class="form-grid">
376                 <div class="form-group full"><label>Logged In User</label><input type="email"
id="leave-email" placeholder="your.email@company.com" readonly></div>
377                 <div class="form-group"><label>Leave Type</label><select id="leave-type"><option
value="SICK">Sick Leave</option><option value="CASUAL">Casual Leave</option><option value="M...
378                     Sick leave is limited to <strong>1 day per month</strong>. If already used
this month, please select Unpaid Leave.
379                 </div>
380                 <div id="casual-leave-hint" style="display:none; margin-top:6px; font-
size:11px; color:var(--amber); background:var(--amber-dim); border:1px solid rgba(251,191,36,0.2); ...
381                     Casual leave is limited to <strong>1 day per month</strong>. If already
used this month, it will be automatically recorded as <strong>Unpaid Leave</strong>.
382                 </div>
383             </div>
384             <div class="form-group"><label>Start Date</label><input type="date" id="leave-
start"></div>
385             <div class="form-group"><label>End Date</label><input type="date" id="leave-

```

```

end"></div>
386             <div class="form-group full"><label>Reason</label><textarea id="leave-reason"
rows="3" placeholder="Please provide reason for leave..."></textarea></div>
387         </div>
388         <div class="modal-actions" style="margin-top: 20px;"><button class="btn btn-primary"
onclick="applyLeave()">Submit Leave Request</button><button class="btn btn-secondary" onclie...
389     </div>
390 </div>
391
392 <!-- HOLIDAYS VIEW -->
393 <div id="holidays-view" class="view">
394     <div class="section-head">
395         <div class="section-title">Fixed Holidays</div>
396         <button class="btn btn-primary btn-sm" onclick="loadHolidays()">Refresh</button>
397     </div>
398
399 <!-- Add Holiday Form -->
400 <div class="form-card" style="margin-bottom: 24px;">
401     <div class="form-grid">
402         <div class="form-group">
403             <label>Holiday Name</label>
404             <input type="text" id="h-name" placeholder="e.g. Diwali, Christmas">
405         </div>
406         <div class="form-group">
407             <label>Date</label>
408             <input type="date" id="h-date">
409         </div>
410         <div class="form-group full">
411             <label>Description (optional)</label>
412             <input type="text" id="h-desc" placeholder="Brief description">
413         </div>
414     </div>
415     <div class="modal-actions" style="margin-top: 20px;">
416         <button class="btn btn-primary" onclick="addHoliday()">Add Holiday</button>
417         <button class="btn btn-secondary" onclick="clearHolidayForm()">Clear</button>
418     </div>
419 </div>
420
421 <!-- Holidays Table -->
422 <div class="table-wrap">
423     <table>
424         <thead>
425             <tr>
426                 <th>ID</th>
427                 <th>Name</th>
428                 <th>Date</th>
429                 <th>Description</th>
430                 <th>Action</th>
431             </tr>
432         </thead>
433         <tbody id="holidays-list">
434             <tr><td colspan="5" class="empty-state">Loading...</td></tr>
435         </tbody>
436     </table>
437 </div>
438 </div>
439
440 <!-- MY LEAVES VIEW -->
441 <div id="my-leaves-view" class="view">
442     <div class="section-head"><div class="section-title">My Leave Requests</div><button
class="btn btn-primary btn-sm" onclick="loadMyLeaves()">Refresh</button></div>
443     <div class="table-wrap"><table><thead><tr><th>ID</th><th>Type</th><th>Start
Date</th><th>End Date</th><th>Status</th><th>Reason</th><th>Action</th></tr></thead><tbody id="my-leaves-...
444 </div>
445
446 <!-- PENDING LEAVES VIEW -->
447 <div id="pending-leaves-view" class="view">
448     <div class="section-head"><div class="section-title">Pending Leave Approvals</div><button
class="btn btn-primary btn-sm" onclick="loadPendingLeaves()">Refresh</button></div>
449     <div class="table-
wrap"><table><thead><tr><th>ID</th><th>Employee</th><th>Type</th><th>Start</th><th>End</th><th>Reason</th><th>Ac
tions</th></tr></thead><tbody id="pending-leaves-li...
450 </div>
451
452 <!-- RESTAURANTS VIEW -->
453 <div id="restaurants-view" class="view">

```

```

454         <div class="section-head"><div class="section-title">Restaurants</div><div><select
id="slot-filter" onchange="loadRestaurants()"><option value="">All Slots</option><option value="BR...
455         <div class="table-wrap"><table><thead><tr><th>Name</th><th>Address</th><th>Meal
Slots</th><th>Status</th><th>Vendor</th></tr></thead><tbody id="restaurants-list"></tbody></table></div>
456     </div>
457
458     <!-- MY SUBSCRIPTIONS VIEW -->
459     <div id="subscriptions-view" class="view">
460         <div class="section-head"><div class="section-title">My Meal Subscriptions</div><button
class="btn btn-primary btn-sm" onclick="loadSubscriptions()">Refresh</button></div>
461         <div class="table-wrap"><table><thead><tr><th>ID</th><th>Restaurant</th><th>Meal
Slot</th><th>Schedule</th><th>Status</th><th>Next Delivery</th><th>Actions</th></tr></thead><tbody i...
462     </div>
463
464     <!-- ADD SUBSCRIPTION VIEW -->
465     <div id="add-subscription-view" class="view">
466         <div class="section-title" style="margin-bottom: 20px;">Create Meal Subscription</div>
467         <div class="form-card">
468             <div class="form-grid">
469                 <div class="form-group full">
470                     <label>Employee Email</label>
471                     <input type="email" id="sub-email" placeholder="employee@company.com">
472                 </div>
473                 <div class="form-group full">
474                     <label>Restaurant</label>
475                     <select id="sub-restaurant-select">
476                         <option value="">-- Select Restaurant --</option>
477                     </select>
478                 </div>
479                 <div class="form-group full">
480                     <label>Meal Slots</label>
481                     <div id="meal-slot-checkboxes" style="display:flex; gap:16px; flex-wrap:wrap;
margin-top:6px;">
482                         <label style="display:flex;align-items:center;gap:6px;text-
transform:none;font-size:13px;font-weight:400;color:var(--text);">
483                             <input type="checkbox" value="BREAKFAST"
style="width:15px;height:15px;padding:0;background:var(--bg3);"> Breakfast
484                         </label>
485                         <label style="display:flex;align-items:center;gap:6px;text-
transform:none;font-size:13px;font-weight:400;color:var(--text);">
486                             <input type="checkbox" value="LUNCH"
style="width:15px;height:15px;padding:0;background:var(--bg3);"> Lunch
487                         </label>
488                         <label style="display:flex;align-items:center;gap:6px;text-
transform:none;font-size:13px;font-weight:400;color:var(--text);">
489                             <input type="checkbox" value="DINNER"
style="width:15px;height:15px;padding:0;background:var(--bg3);"> Dinner
490                         </label>
491                     </div>
492                 </div>
493                 <div class="form-group">
494                     <label>Schedule Type</label>
495                     <select id="sub-schedule">
496                         <option value="DAILY">Daily</option>
497                         <option value="WEEKLY">Weekly</option>
498                     </select>
499                 </div>
500                 <div class="form-group" id="day-of-week-group" style="display:none;">
501                     <label>Day of Week</label>
502                     <select id="sub-day">
503                         <option value="MONDAY">Monday</option>
504                         <option value="TUESDAY">Tuesday</option>
505                         <option value="WEDNESDAY">Wednesday</option>
506                         <option value="THURSDAY">Thursday</option>
507                         <option value="FRIDAY">Friday</option>
508                         <option value="SATURDAY">Saturday</option>
509                         <option value="SUNDAY">Sunday</option>
510                     </select>
511                 </div>
512             </div>
513             <div class="modal-actions" style="margin-top: 20px;">
514                 <button class="btn btn-primary" onclick="addSubscription()">Create
Subscription</button>
515                 <button class="btn btn-secondary" onclick="clearSubscriptionForm()">Clear</button>
516             </div>
517         </div>

```

```

518         </div>
519
520         <!-- DELIVERIES VIEW -->
521         <div id="deliveries-view" class="view">
522             <div class="section-head"><div class="section-title">My Deliveries</div><button class="btn
523 btn-primary btn-sm" onclick="loadDeliveries()">Refresh</button></div>
524             <div class="table-wrap"><table><thead><tr><th>ID</th><th>Subscription ID</th><th>Meal
525 Slot</th><th>Scheduled Time</th><th>Status</th></tr></thead><tbody id="deliveries-list"></tbody>...
526         </div>
527
528         <!-- VENDORS VIEW -->
529         <div id="vendors-view" class="view">
530             <div class="section-head"><div class="section-title">Vendors</div><button class="btn btn-
531 primary btn-sm" onclick="loadVendors()">Refresh</button></div>
532             <div class="table-
533 wrap"><table><thead><tr><th>ID</th><th>Name</th><th>Email</th><th>Phone</th><th>Registered</th><th>Actions</th><
534 /tr></thead><tbody id="vendors-list"></tbody></table>...
535         </div>
536
537         <!-- DEVICES VIEW -->
538         <div id="devices-view" class="view">
539             <div class="section-head">
540                 <div class="section-title">Unassigned Devices</div>
541                 <button class="btn btn-primary btn-sm"
542 onclick="loadUnassignedDevices()">Refresh</button>
543             </div>
544             <div class="table-wrap">
545                 <table>
546                     <thead>
547                         <tr>
548                             <th>ID</th>
549                             <th>Device Name</th>
550                             <th>Brand</th>
551                             <th>Type</th>
552                             <th>Warranty</th>
553                             <th>Vendor</th>
554                             <th>Actions</th>
555                         </tr>
556                     </thead>
557                     <tbody id="devices-list"></tbody>
558                 </table>
559             </div>
560         </div>
561
562         <!-- ASSIGN DEVICE MODAL -->
563         <div id="assign-modal" class="modal-overlay">
564             <div class="modal">
565                 <div class="modal-title">Assign Device to Employee</div>
566                 <input type="hidden" id="assign-device-id">
567                 <div class="form-group" style="margin-bottom:12px">
568                     <label>Device</label>
569                     <input type="text" id="assign-device-name" readonly style="color:var(--text2)">
570                 </div>
571                 <div class="form-group" style="margin-bottom:12px">
572                     <label>Employee</label>
573                     <select id="assign-employee-select">
574                         <option value="">-- Select Employee --</option>
575                     </select>
576                 </div>
577                 <div class="modal-actions">
578                     <button class="btn btn-primary" onclick="confirmAssign()">Assign</button>
579                     <button class="btn btn-secondary" onclick="closeAssignModal()">Cancel</button>
580                 </div>
581             </div>
582         </div>
583
584         <!-- MY DEVICES VIEW -->
585         <div id="my-devices-view" class="view">
586             <div class="section-head">
587                 <div class="section-title">My Assigned Devices</div>
588                 <button class="btn btn-primary btn-sm"
589 onclick="loadMyAssignedDevices()">Refresh</button>
590             </div>
591             <div class="table-wrap">
592                 <table>
593                     <thead>

```

```

587         <tr>
588             <th>ID</th>
589             <th>Device Name</th>
590             <th>Brand</th>
591             <th>Type</th>
592             <th>Warranty</th>
593             <th>Assigned On</th>
594             <th>Status</th>
595         </tr>
596     </thead>
597     <tbody id="my-devices-list"></tbody>
598 </table>
599 </div>
600 </div>
601
602 <!-- MY SERVICE REQUESTS VIEW -->
603 <div id="service-requests-view" class="view">
604     <div class="section-head">
605         <div class="section-title">My Service Requests</div>
606         <button class="btn btn-primary btn-sm" onclick="loadServiceRequests()">Refresh</button>
607     </div>
608     <div class="table-wrap" style="margin-bottom:24px">
609         <table>
610             <thead><tr>
611                 <th>ID</th><th>Device</th><th>Type</th>
612                 <th>Description</th><th>Urgent</th><th>Status</th><th>Raised At</th>
613             </tr></thead>
614             <tbody id="service-requests-list"></tbody>
615         </table>
616     </div>
617
618     <div class="section-title" style="margin-bottom:16px">Raise a New Request</div>
619     <div class="form-card">
620         <div class="form-grid">
621             <div class="form-group full">
622                 <label>Device</label>
623                 <select id="sr-device-select">
624                     <option value="">-- Select your device --</option>
625                 </select>
626             </div>
627             <div class="form-group">
628                 <label>Request Type</label>
629                 <select id="sr-type">
630                     <option value="REPAIR">Repair</option>
631                     <option value="REPLACEMENT">Replacement</option>
632                     <option value="UPGRADE">Upgrade</option>
633                     <option value="MAINTENANCE">Maintenance</option>
634                     <option value="RETURN">Return</option>
635                 </select>
636             </div>
637             <div class="form-group" style="display:flex;align-items:center;gap:10px;padding-top:20px">
638                 <input type="checkbox" id="sr-urgent" style="width:16px;height:16px;padding:0">
639                 <label style="text-transform:none;font-size:13px;font-weight:400;color:var(--text);cursor:pointer" for="sr-urgent">Mark as Urgent</label>
640             </div>
641             <div class="form-group full">
642                 <label>Issue Description</label>
643                 <textarea id="sr-description" rows="3" placeholder="Describe the issue in detail..."></textarea>
644             </div>
645         </div>
646         <div class="modal-actions" style="margin-top:20px">
647             <button class="btn btn-primary" onclick="submitServiceRequest()">Submit
Request</button>
648             <button class="btn btn-secondary"
onclick="clearServiceRequestForm()">Clear</button>
649         </div>
650     </div>
651 </div>
652
653 <!-- MY CALENDAR VIEW -->
654 <div id="my-calendar-view" class="view">
655     <div class="cal-section">
656         <div class="cal-section-head">
657             <div>

```



```

658             <div class="cal-section-title">My Leave Calendar</div>
659             <div class="cal-section-sub">All approved leaves and public holidays</div>
660         </div>
661         <button class="btn btn-secondary btn-sm" onclick="refreshCalendar()">
Refresh</button>
662     </div>
663
664     <!-- Balance Cards -->
665     <div class="leave-balance-grid">
666         <div class="leave-bal-card sick">
667             <span class="lbc-icon"></span>
668             <span class="lbc-label">Sick Leave</span>
669             <span class="lbc-count" id="cal-sick-remaining">--</span>
670             <span class="lbc-sub" id="cal-sick-sub">loading...</span>
671         </div>
672         <div class="leave-bal-card casual">
673             <span class="lbc-icon"></span>
674             <span class="lbc-label">Casual Leave</span>
675             <span class="lbc-count" id="cal-casual-remaining">--</span>
676             <span class="lbc-sub" id="cal-casual-sub">loading...</span>
677         </div>
678         <div class="leave-bal-card mat">
679             <span class="lbc-icon"></span>
680             <span class="lbc-label">Maternity Leave</span>
681             <span class="lbc-count" id="cal-mat-remaining">--</span>
682             <span class="lbc-sub" id="cal-mat-sub">loading...</span>
683         </div>
684         <div class="leave-bal-card total">
685             <span class="lbc-icon"></span>
686             <span class="lbc-label">Total Used</span>
687             <span class="lbc-count" id="cal-total-used">--</span>
688             <span class="lbc-sub" id="cal-total-sub">loading...</span>
689         </div>
690     </div>
691
692     <!-- Legend -->
693     <div class="cal-legend">
694         <span class="legend-label">Legend:</span>
695         <div class="legend-item"><div class="legend-dot"
style="background:#36d399"></div>Approved</div>
696         <div class="legend-item"><div class="legend-dot"
style="background:#f9c74f"></div>Pending</div>
697         <div class="legend-item"><div class="legend-dot"
style="background:#f87171"></div>Rejected</div>
698         <div class="legend-item"><div class="legend-dot"
style="background:#a78bfa"></div>Holiday</div>
699     </div>
700
701     <!-- Calendar -->
702     <div id="employee-calendar"></div>
703 </div>
704 </div>
705
706 <!-- MY PROFILE VIEW -->
707 <div id="my-profile-view" class="view">
708     <div class="section-title" style="margin-bottom:20px">My Profile</div>
709
710     <!-- Profile Info -->
711     <div class="form-card" style="max-width:100%;margin-bottom:24px">
712         <div class="form-grid">
713             <div class="form-group">
714                 <label>Name</label>
715                 <input type="text" id="p-name" readonly style="color:var(--text2)">
716             </div>
717             <div class="form-group">
718                 <label>Email</label>
719                 <input type="text" id="p-email" readonly style="color:var(--text2)">
720             </div>
721             <div class="form-group">
722                 <label>Department</label>
723                 <input type="text" id="p-dept" readonly style="color:var(--text2)">
724             </div>
725             <div class="form-group">
726                 <label>Role</label>
727                 <input type="text" id="p-role" readonly style="color:var(--text2)">
728             </div>

```

```

729         <div class="form-group">
730             <label>Status</label>
731             <input type="text" id="p-status" readonly style="color:var(--text2)">
732         </div>
733         <div class="form-group">
734             <label>Timezone</label>
735             <input type="text" id="p-tz" readonly style="color:var(--text2)">
736         </div>
737     </div>
738 </div>
739
740 <!-- Maternity Section – only visible for female admins -->
741 <div id="maternity-apply-btn" style="display:none">
742     <div class="form-card" style="max-width:100%">
743         <div class="section-title" style="margin-bottom:20px"> Maternity Leave</div>
744         <div class="form-grid">
745             <div class="form-group">
746                 <label>Start Date</label>
747                 <input type="date" id="mat-start-date">
748             </div>
749             <div class="form-group">
750                 <label>End Date (Auto)</label>
751                 <input type="text" id="mat-end-date"
752                     placeholder="Auto: start + 180 days"
753                     disabled style="color:var(--text2)">
754             </div>
755             <div class="form-group full">
756                 <label>Reason / Notes</label>
757                 <textarea id="mat-reason" rows="3"
758                     placeholder="Optional notes..."></textarea>
759             </div>
760             <div class="form-group full">
761                 <div style="background:var(--accent-dim);
762                     border:1px solid rgba(79,140,255,0.2);
763                     border-radius:8px;padding:12px 16px;
764                     font-size:12px;color:var(--text2)">
765                     Maternity leave is fixed at
766                     <strong style="color:var(--accent)">180 days</strong>.
767                     End date is auto-calculated. You can apply
768                     <strong style="color:var(--accent)">once per year</strong>.
769                 </div>
770             </div>
771         </div>
772         <div class="modal-actions" style="margin-top:20px">
773             <button class="btn btn-primary" onclick="applyMaternityLeave()">
774                 Submit Maternity Leave
775             </button>
776         </div>
777     </div>
778 </div>
779 </div>
780
781 <!-- 2FA VIEW -->
782 <div id="two-factor-view" class="view">
783     <div class="section-title" style="margin-bottom:20px"> Two-Factor Authentication</div>
784
785     <div class="form-card" style="max-width:500px;margin-bottom:24px">
786         <!-- Status -->
787         <div style="display:flex;align-items:center;justify-content:space-between;margin-
bottom:20px">
788             <div>
789                 <div style="font-size:14px;font-weight:600;color:var(--text)">Authenticator App
(TOTP)</div>
790                 <div style="font-size:12px;color:var(--text2);margin-top:4px">
791                     Google Authenticator, Authy, or any TOTP app
792                 </div>
793             </div>
794             <span id="totp-status-chip" class="chip">Checking...</span>
795         </div>
796
797         <!-- When TOTP is DISABLED – show setup -->
798         <div id="totp-setup-section">
799             <p style="font-size:13px;color:var(--text2);margin-bottom:16px">
800                 Scan the QR code with your authenticator app, then enter the 6-digit code to
activate.
801             </p>

```

```

802         <button class="btn btn-primary" onclick="generateTotpQr()" id="generate-qr-btn">
803             Generate QR Code
804         </button>
805
806         <!-- QR code area – hidden until generated -->
807         <div id="qr-area" style="display:none;margin-top:20px;text-align:center">
808             <img id="qr-img" src="" alt="QR Code"
809                 style="width:180px;height:180px;border-radius:10px;border:1px solid var(--
border2)">
810             <div style="margin-top:12px;font-size:11px;color:var(--text2)">
811                 Can't scan? Enter this key manually:
812             </div>
813             <div id="totp-secret-display"
814                 style="font-family:var(--mono);font-size:13px;color:var(--accent);
815                 background:var(--bg3);border:1px solid var(--border2);
816                 border-radius:6px;padding:8px 12px;margin-top:6px;
817                 word-break:break-all;text-align:center"></div>
818
819             <!-- Verify step -->
820             <div style="margin-top:20px">
821                 <label style="font-size:11px;color:var(--text3);font-weight:600;
822                     text-transform:uppercase;display:block;margin-bottom:8px">
823                     Enter 6-digit code to activate
824                 </label>
825                 <input id="totp-verify-input" type="text" inputmode="numeric"
826                     maxlength="6" placeholder="000000"
827                     style="text-align:center;font-size:20px;font-family:var(--mono);
828                     letter-spacing:8px;padding:12px;width:100%"
829                     oninput="this.value=this.value.replace(/\D/g, '')">
830                 <button class="btn btn-primary" onclick="verifyAndActivateTotp()"
831                     style="width:100%;margin-top:12px">
832                     Activate 2FA
833                 </button>
834                 <div id="totp-verify-error"
835                     style="display:none;margin-top:8px;font-size:12px;color:var(--
red)"></div>
836             </div>
837         </div>
838     </div>
839
840     <!-- When TOTP is ENABLED – show disable -->
841     <div id="totp-disable-section" style="display:none">
842         <div style="background:var(--green-dim);border:1px solid rgba(54,211,153,0.2);
843             border-radius:8px;padding:12px 16px;font-size:13px;
844             color:var(--green);margin-bottom:16px">
845             Two-factor authentication is active on your account.
846         </div>
847         <button class="btn btn-danger" onclick="disableTotp()">
848             Disable 2FA
849         </button>
850     </div>
851 </div>
852 </div>
853
854 <!-- ===== ADMIN SERVICE REQUESTS VIEW ===== -->
855 <div id="admin-service-requests-view" class="view">
856     <div class="section-head">
857         <div class="section-title">All Open Service Requests</div>
858         <button class="btn btn-primary btn-sm"
onclick="loadAdminServiceRequests()">Refresh</button>
859     </div>
860     <div class="table-wrap">
861         <table>
862             <thead>
863                 <tr>
864                     <th>ID</th>
865                     <th>Employee</th>
866                     <th>Device</th>
867                     <th>Type</th>
868                     <th>Description</th>
869                     <th>Urgent</th>
870                     <th>Status</th>
871                     <th>Raised At</th>
872                     <th>Actions</th>
873                 </tr>
874             </thead>

```

```

875         <tbody id="admin-service-requests-list"></tbody>
876     </table>
877 </div>
878 </div>
879
880 </div>
881 </div>
882 </div>
883
884 <!-- Approve/Reject Modal -->
885 <div id="action-modal" class="modal-overlay">
886     <div class="modal">
887         <div class="modal-title" id="modal-title">Approve Leave Request</div>
888         <input type="hidden" id="action-leave-id">
889         <input type="hidden" id="action-type">
890         <div class="form-group"><label>Reviewed By</label><input type="email" id="manager-email"
placeholder="admin@company.com" readonly></div>
891         <div class="form-group"><label>Remarks</label><textarea id="action-remarks" rows="3"
placeholder="Add remarks..."></div>
892         <div class="modal-actions"><button class="btn btn-primary" id="action-btn"
onclick="submitApproval()">Approve</button><button class="btn btn-secondary"
onclick="closeModal()">Cancel</button>...
893     </div>
894 </div>
895
896 <!-- Cancel Leave Modal -->
897 <div id="cancel-modal" class="modal-overlay">
898     <div class="modal">
899         <div class="modal-title">Cancel Leave Request</div>
900         <input type="hidden" id="cancel-leave-id">
901         <div class="form-group"><label>Cancelled By</label><input type="email" id="cancel-email"
placeholder="admin@company.com" readonly></div>
902         <div class="modal-actions"><button class="btn btn-danger" onclick="confirmCancelLeave()">Cancel
Leave</button><button class="btn btn-secondary" onclick="closeCancelModal()">Close</button></div>
903     </div>
904 </div>
905
906 <!-- Admin Service Request Action Modal -->
907 <div id="admin-sr-modal" class="modal-overlay">
908     <div class="modal">
909         <div class="modal-title">Update Service Request <span id="admin-sr-modal-id" style="color:var(--
accent)"></span></div>
910         <input type="hidden" id="admin-sr-id">
911
912         <div class="form-group" style="margin-bottom:14px">
913             <label>Device</label>
914             <input type="text" id="admin-sr-device-display" readonly style="color:var(--text2)">
915         </div>
916         <div class="form-group" style="margin-bottom:14px">
917             <label>Employee</label>
918             <input type="text" id="admin-sr-employee-display" readonly style="color:var(--text2)">
919         </div>
920         <div class="form-group" style="margin-bottom:14px">
921             <label>Update Status</label>
922             <select id="admin-sr-status">
923                 <option value="SENT_FOR_REPAIR">Sent for Repair – device will be marked Under
Repair</option>
924                 <option value="REPAIR_DONE">Repair Done</option>
925                 <option value="REJECTED">Rejected</option>
926             </select>
927         </div>
928         <div class="form-group" style="margin-bottom:4px">
929             <label>Admin Remarks</label>
930             <textarea id="admin-sr-remarks" rows="3" placeholder="Add remarks or instructions for the
vendor..."></div>
931         </div>
932         <div id="admin-sr-repair-hint" style="font-size:11px;color:var(--amber);margin-
top:6px;display:none;">
933             Selecting "Sent for Repair" will automatically mark the device as <strong>Under
Repair</strong>.
934         </div>
935
936         <div class="modal-actions">
937             <button class="btn btn-primary" onclick="submitAdminSRUpdate()">Update Request</button>
938             <button class="btn btn-secondary" onclick="closeAdminSRModal()">Cancel</button>
939         </div>

```

```

940     </div>
941 </div>
942
943 <div class="cal-tooltip" id="cal-tooltip" style="display:none"></div>
944
945 <script>
946     const API = 'http://localhost:8080';
947     let currentUser = null;
948     const ROLE_DASHBOARDS = {
949         EMPLOYEE: '/employee-dashboard.html',
950         MANAGER: '/manager-dashboard.html',
951         VENDOR: '/vendor-dashboard.html',
952         ADMIN: '/dashboard.html'
953     };
954
955     (async function initAuth() {
956         const token = localStorage.getItem('jwt_token');
957         const authType = localStorage.getItem('auth_type');
958         if (!token && authType !== 'session') {
959             window.location.href = '/login.html';
960             return;
961         }
962
963         try {
964             const response = await fetch(`${API}/employee/me`, {
965                 headers: getAuthHeader(),
966                 credentials: 'include'
967             });
968
969             if (response.status === 401) { window.location.href = '/login.html'; return; }
970             if (!response.ok) { window.location.href = '/login.html'; return; }
971
972             const employee = await response.json();
973             const role = employee.role || 'EMPLOYEE';
974             if (role !== 'ADMIN') {
975                 window.location.href = ROLE_DASHBOARDS[role] || '/employee-dashboard.html';
976                 return;
977             }
978
979             currentUser = employee;
980             localStorage.setItem('last_role', role);
981             document.getElementById('username-display').textContent = employee.email || employee.name ||
982 'admin';
983             document.getElementById('leave-email').value = employee.email || '';
984             document.getElementById('manager-email').value = employee.email || '';
985             document.getElementById('cancel-email').value = employee.email || '';
986             loadDashboard();
987         } catch (e) {
988             window.location.href = '/login.html';
989         }
990     })();
991
992     function getAuthHeader() {
993         const token = localStorage.getItem('jwt_token');
994         return token ? { 'Authorization': `Bearer ${token}` } : {};
995     }
996
997     async function apiFetch(path, opts = {}) {
998         try {
999             const response = await fetch(`${API}${path}`, {
1000                 ...opts,
1001                 headers: {
1002                     'Content-Type': 'application/json',
1003                     ...getAuthHeader(),
1004                     ...opts.headers
1005                 },
1006                 credentials: 'include'
1007             });
1008
1009             if (response.status === 401) {
1010                 localStorage.removeItem('jwt_token');
1011                 localStorage.removeItem('auth_type');
1012                 showAlert('Session expired. Please login again.', 'error');
1013                 setTimeout(() => window.location.href = '/login.html', 1500);
1014                 throw new Error('Unauthorized');
1015             }
1016         }

```

```

1015
1016         if (!response.ok) {
1017             const error = await response.text();
1018             throw new Error(error || 'Request failed');
1019         }
1020
1021         const text = await response.text();
1022         if (!text) return null;
1023         try { return JSON.parse(text); } catch { return text; }
1024     } catch (e) {
1025         if (!e.message.includes('Unauthorized')) showAlert(e.message, 'error');
1026         throw e;
1027     }
1028 }
1029
1030 function showAlert(message, type) {
1031     const alert = document.createElement('div');
1032     alert.className = `alert alert-${type}`;
1033     alert.textContent = message;
1034     document.body.appendChild(alert);
1035     setTimeout(() => alert.remove(), 3000);
1036 }
1037
1038 function showView(viewName, event) {
1039     document.querySelectorAll('.view').forEach(v => v.classList.remove('active'));
1040     document.getElementById(`${viewName}-view`).classList.add('active');
1041     document.querySelectorAll('.nav-item').forEach(n => n.classList.remove('active'));
1042     if (event && event.target) event.target.classList.add('active');
1043     document.getElementById('pageTitle').textContent = viewName.split('-').map(w =>
1044         w.charAt(0).toUpperCase() + w.slice(1)).join(' ');
1045     const loaders = {
1046         'dashboard': loadDashboard,
1047         'employees': loadEmployees,
1048         'add-employee': loadManagersDropdown,
1049         'holidays': loadHolidays,
1050         'add-manager': () => {},
1051         'add-vendor': () => {},
1052         'my-leaves': loadMyLeaves,
1053         'pending-leaves': loadPendingLeaves,
1054         'restaurants': loadRestaurants,
1055         'subscriptions': loadSubscriptions,
1056         'add-subscription': loadRestaurantDropdown,
1057         'deliveries': loadDeliveries,
1058         'vendors': loadVendors,
1059         'devices': loadUnassignedDevices,
1060         'my-devices': loadMyAssignedDevices,
1061         'service-requests': loadServiceRequests,
1062         'admin-service-requests': loadAdminServiceRequests,
1063         'my-profile': loadAdminProfile,
1064         'two-factor': loadTotpStatus,
1065         'my-calendar': loadMyCalendar// ADD THIS
1066     };
1067     if (loaders[viewName]) loaders[viewName]();
1068 }
1069
1070 // ===== DASHBOARD =====
1071 async function loadDashboard() {
1072     try {
1073         const [employees, leaves] = await Promise.all([
1074             apiFetch('/employee'),
1075             apiFetch('/leave_requests')
1076         ]);
1077         document.getElementById('total-employees').textContent = employees.length;
1078         document.getElementById('active-employees').textContent = employees.filter(e => e.status ===
1079 'ACTIVE').length;
1080         document.getElementById('onleave-employees').textContent = employees.filter(e => e.status ===
1081 'ON_LEAVE').length;
1082         document.getElementById('pending-leaves').textContent = leaves.filter(l => l.status ===
1083 'PENDING').length;
1084         const recent = leaves.slice(-5).reverse();
1085         const tbody = document.getElementById('recent-leaves');
1086         if (!recent.length) {
1087             tbody.innerHTML = '<tr><td colspan="5" class="empty-state">No leave requests</td></tr>';
1088         } else {
1089             tbody.innerHTML = recent.map(l => `<tr>
1090                 <td>${l.employeeName || 'N/A'}</td>

```

```

1088         <td><span class="chip">${l.leaveType || 'N/A'}</span></td>
1089         <td>${l.startDate || 'N/A'}</td>
1090         <td>${l.endDate || 'N/A'}</td>
1091         <td><span class="chip ${l.status?.toLowerCase()}>${l.status || 'N/A'}</span></td>
1092     </tr>`).join('');
1093     }
1094     } catch (e) { console.error(e); }
1095 }
1096
1097 // ===== EMPLOYEES =====
1098 async function loadEmployees() {
1099     try {
1100         const employees = await apiFetch('/employee');
1101         const tbody = document.getElementById('employees-list');
1102         if (!employees.length) {
1103             tbody.innerHTML = '<tr><td colspan="8" class="empty-state">No employees found</td></tr>';
1104         } else {
1105             tbody.innerHTML = employees.map(e => `<tr>
1106                 <td>${e.employeeId}</td>
1107                 <td>${e.name || 'N/A'}</td>
1108                 <td>${e.email || 'N/A'}</td>
1109                 <td>${e.dept || 'N/A'}</td>
1110                 <td><span class="chip ${e.role?.toLowerCase()}>${e.role || 'N/A'}</span></td>
1111                 <td><span class="chip ${e.status?.toLowerCase()}>${e.status || 'N/A'}</span></td>
1112                 <td>${e.timezone || 'N/A'}</td>
1113                 <td><button class="btn btn-danger btn-sm"
onClick="deleteEmployee(${e.employeeId})">Delete</button></td>
1114             </tr>`).join('');
1115         }
1116     } catch (e) { console.error(e); }
1117 }
1118
1119 async function loadManagersDropdown() {
1120     try {
1121         const managers = await apiFetch('/admin/managers');
1122         const select = document.getElementById('employee-manager-select');
1123         select.innerHTML = '<option value="">-- Select Manager --</option>';
1124         (managers || []).forEach(m => {
1125             const opt = document.createElement('option');
1126             opt.value = m.managerId; // matches ManagerDTO field
1127             opt.textContent = `${m.name} - ${m.dept || 'N/A'} (${m.email})`;
1128             select.appendChild(opt);
1129         });
1130     } catch (e) {
1131         showAlert('Failed to load managers', 'error');
1132     }
1133 }
1134
1135
1136
1137 async function createEmployee() {
1138     const managerId = document.getElementById('employee-manager-select').value;
1139     const body = {
1140         name: document.getElementById('employee-name').value.trim(),
1141         email: document.getElementById('employee-email').value.trim(),
1142         password: document.getElementById('employee-password').value,
1143         gender: document.getElementById('employee-gender').value,
1144         timezone: document.getElementById('employee-timezone').value || 'UTC',
1145         managerId: parseInt(managerId)
1146     };
1147     if (!body.name || !body.email || !body.password) {
1148         showAlert('Please fill all employee fields', 'error');
1149         return;
1150     }
1151     if (!managerId) {
1152         showAlert('Please select a manager', 'error');
1153         return;
1154     }
1155     try {
1156         await apiFetch('/admin/employees', { method: 'POST', body: JSON.stringify(body) });
1157         showAlert('Employee created successfully!', 'success');
1158         clearEmployeeForm('employee');
1159         loadEmployees();
1160     } catch (e) {
1161         showAlert('Failed to create employee: ' + e.message, 'error');
1162     }

```

```

1163 }
1164
1165 async function createManager() {
1166     const body = {
1167         name: document.getElementById('manager-name-create').value.trim(),
1168         email: document.getElementById('manager-email-create').value.trim(),
1169         password: document.getElementById('manager-password-create').value,
1170         dept: document.getElementById('manager-dept-create').value.trim(),
1171         gender: document.getElementById('manager-gender').value,
1172         timezone: document.getElementById('manager-timezone-create').value || 'UTC'
1173     };
1174     if (!body.name || !body.email || !body.password || !body.dept || !body.gender) {
1175         showAlert('Please fill all manager fields', 'error');
1176         return;
1177     }
1178     try {
1179         await apiFetch('/admin/employees/manager', { method: 'POST', body: JSON.stringify(body) });
1180         showAlert('Manager created successfully!', 'success');
1181         clearEmployeeForm('manager');
1182         loadEmployees();
1183     } catch (e) { showAlert('Failed to create manager: ' + e.message, 'error'); }
1184 }
1185
1186 function clearEmployeeForm(type) {
1187     const fields = type === 'manager'
1188         ? ['manager-name-create', 'manager-email-create', 'manager-password-create', 'manager-dept-
create', 'manager-timezone-create']
1189         : ['employee-name', 'employee-email', 'employee-password', 'employee-timezone'];
1190     fields.forEach(id => document.getElementById(id).value = '');
1191     if (type === 'employee') {
1192         document.getElementById('employee-manager-select').value = '';
1193     }
1194 }
1195
1196 async function deleteEmployee(employeeId) {
1197     if (!confirm('Delete employee #{employeeId}?')) return;
1198     try {
1199         await apiFetch(`/admin/employee/${employeeId}`, { method: 'DELETE' });
1200         showAlert('Employee deleted successfully!', 'success');
1201         loadEmployees();
1202         loadDashboard();
1203     } catch (e) { showAlert('Failed to delete employee: ' + e.message, 'error'); }
1204 }
1205
1206 // ===== LEAVE REQUESTS =====
1207 async function applyLeave() {
1208     const leaveType = document.getElementById('leave-type').value;
1209     const startDate = document.getElementById('leave-start').value;
1210     const endDate = document.getElementById('leave-end').value;
1211     const reason = document.getElementById('leave-reason').value.trim();
1212     if (!startDate || !endDate) { showAlert('Please fill all required fields', 'error'); return; }
1213     try {
1214         const data = await apiFetch('/leave_requests', {
1215             method: 'POST',
1216             body: JSON.stringify({ leaveType, startDate, endDate, reason })
1217         });
1218         if (data && data.warning) {
1219             showAlert(data.warning, 'warning'); // amber toast
1220         } else {
1221             showAlert('Leave request submitted successfully!', 'success');
1222         }
1223         clearLeaveForm();
1224         loadMyLeaves();
1225     } catch (e) { showAlert('Failed to submit: ' + e.message, 'error'); }
1226 }
1227 document.getElementById('leave-type').addEventListener('change', function() {
1228     document.getElementById('sick-leave-hint').style.display =
1229         this.value === 'SICK' ? 'block' : 'none';
1230     document.getElementById('casual-leave-hint').style.display =
1231         this.value === 'CASUAL' ? 'block' : 'none';
1232 });
1233
1234 async function loadMyLeaves() {
1235     try {
1236         const data = await apiFetch('/leave_requests/employee');
1237         const leaves = data.requests || [];

```



```

1238         const tbody = document.getElementById('my-leaves-list');
1239         if (!leaves.length) {
1240             tbody.innerHTML = '<tr><td colspan="7" class="empty-state">No leave requests
found</td></tr>';
1241         } else {
1242             tbody.innerHTML = leaves.map(l => `<tr>
1243                 <td>${l.leaveRequestId}</td>
1244                 <td><span class="chip">${l.leaveType}</span></td>
1245                 <td>${l.startDate}</td>
1246                 <td>${l.endDate}</td>
1247                 <td><span class="chip ${l.status?.toLowerCase()}">${l.status}</span></td>
1248                 <td style="max-width:200px;overflow:hidden;text-overflow:ellipsis;">${l.reason || '-
'}</td>
1249                 <td>${l.status === 'PENDING' ? `<button class="btn btn-danger btn-sm"
onClick="openCancelModal(${l.leaveRequestId})">Cancel</button>` : '-'}</td>
1250             </tr>`).join('');
1251         }
1252     } catch (e) { console.error(e); }
1253 }
1254
1255 async function loadPendingLeaves() {
1256     try {
1257         const leaves = await apiFetch('/leave_requests/pending');
1258         const tbody = document.getElementById('pending-leaves-list');
1259         if (!leaves.length) {
1260             tbody.innerHTML = '<tr><td colspan="7" class="empty-state">No pending leave
requests</td></tr>';
1261         } else {
1262             tbody.innerHTML = leaves.map(l => `<tr>
1263                 <td>${l.leaveRequestId}</td>
1264                 <td>${l.employeeName}</td>
1265                 <td><span class="chip">${l.leaveType}</span></td>
1266                 <td>${l.startDate}</td>
1267                 <td>${l.endDate}</td>
1268                 <td style="max-width:150px;overflow:hidden;text-overflow:ellipsis;">${l.reason || '-
'}</td>
1269                 <td class="action-row">
1270                     <button class="btn btn-primary btn-sm"
onClick="openActionModal(${l.leaveRequestId}, 'APPROVED')">Approve</button>
1271                     <button class="btn btn-danger btn-sm"
onClick="openActionModal(${l.leaveRequestId}, 'REJECTED')">Reject</button>
1272                 </td>
1273             </tr>`).join('');
1274         }
1275     } catch (e) { console.error(e); }
1276 }
1277
1278 function openActionModal(leaveId, action) {
1279     document.getElementById('action-leave-id').value = leaveId;
1280     document.getElementById('action-type').value = action;
1281     document.getElementById('modal-title').textContent = `${action} Leave Request #${leaveId}`;
1282     document.getElementById('action-btn').textContent = action;
1283     document.getElementById('action-btn').className = action === 'APPROVED' ? 'btn btn-primary' : 'btn
btn-danger';
1284     document.getElementById('action-modal').classList.add('open');
1285 }
1286
1287 async function submitApproval() {
1288     const leaveRequestId = parseInt(document.getElementById('action-leave-id').value);
1289     const action = document.getElementById('action-type').value;
1290     const remarks = document.getElementById('action-remarks').value;
1291     try {
1292         await apiFetch('/leave_requests/approval', {
1293             method: 'PUT',
1294             body: JSON.stringify({ leaveRequestId, action, remarks })
1295         });
1296         showAlert(`Leave request ${action.toLowerCase()} successfully!`, 'success');
1297         closeModal();
1298         loadPendingLeaves();
1299     } catch (e) { showAlert('Failed: ' + e.message, 'error'); }
1300 }
1301
1302 function openCancelModal(leaveId) {
1303     document.getElementById('cancel-leave-id').value = leaveId;
1304     document.getElementById('cancel-modal').classList.add('open');
1305 }

```

```

1306
1307     async function confirmCancelLeave() {
1308         const leaveId = document.getElementById('cancel-leave-id').value;
1309         try {
1310             await apiFetch(`/leave_requests/cancel/${leaveId}`, { method: 'PUT' });
1311             showAlert('Leave cancelled successfully!', 'success');
1312             closeCancelModal();
1313             loadMyLeaves();
1314         } catch (e) { showAlert('Failed: ' + e.message, 'error'); }
1315     }
1316
1317     function clearLeaveForm() {
1318         document.getElementById('leave-start').value = '';
1319         document.getElementById('leave-end').value = '';
1320         document.getElementById('leave-reason').value = '';
1321     }
1322
1323     // ===== RESTAURANTS =====
1324     async function loadRestaurants() {
1325         try {
1326             const slot = document.getElementById('slot-filter').value;
1327             const path = slot ? `/restaurants/by-slot?slot=${slot}` : '/restaurants';
1328             const restaurants = await apiFetch(path);
1329             const tbody = document.getElementById('restaurants-list');
1330             if (!restaurants.length) {
1331                 tbody.innerHTML = '<tr><td colspan="5" class="empty-state">No restaurants
found</td></tr>';
1332             } else {
1333                 tbody.innerHTML = restaurants.map(r => `<tr>
1334                     <td><strong>${r.name}</strong></td>
1335                     <td>${r.address || 'N/A'}</td>
1336                     <td>${r.supportedMealSlots ? r.supportedMealSlots.map(s => `<span
class="chip">${s}</span>`).join(' ') : 'N/A'}</td>
1337                     <td><span class="chip ${r.active ? 'active' : 'inactive'}">${r.active ? 'ACTIVE' :
'INACTIVE'}</span></td>
1338                     <td>${r.vendorName || r.vendorId || 'N/A'}</td>
1339                 </tr>`).join('');
1340             }
1341         } catch (e) { console.error(e); }
1342     }
1343
1344     // ===== SUBSCRIPTIONS =====
1345     async function loadRestaurantDropdown() {
1346         try {
1347             const restaurants = await apiFetch('/restaurants');
1348             const arr = Array.isArray(restaurants) ? restaurants : [];
1349             const select = document.getElementById('sub-restaurant-select');
1350             select.innerHTML = '<option value="">-- Select Restaurant --</option>';
1351             arr.filter(r => r.active).forEach(r => {
1352                 const opt = document.createElement('option');
1353                 opt.value = r.id || r.restaurantId;
1354                 opt.textContent = r.name;
1355                 opt.dataset.slots = JSON.stringify(r.supportedMealSlots || []);
1356                 select.appendChild(opt);
1357             });
1358         } catch (e) { console.error('Failed to load restaurants:', e); }
1359     }
1360
1361     document.getElementById('sub-restaurant-select')?.addEventListener('change', function () {
1362         const selected = this.options[this.selectedIndex];
1363         const supported = selected?.dataset?.slots ? JSON.parse(selected.dataset.slots) : [];
1364         const boxes = document.querySelectorAll('#meal-slot-checkboxes input[type=checkbox]');
1365         boxes.forEach(cb => {
1366             const wrapper = cb.parentElement;
1367             if (!supported.length || supported.includes(cb.value)) {
1368                 wrapper.style.display = 'flex'; wrapper.style.opacity = '1'; cb.disabled = false;
1369             } else {
1370                 cb.checked = false; cb.disabled = true; wrapper.style.opacity = '0.3';
1371             }
1372         });
1373     });
1374
1375     async function loadSubscriptions() {
1376         try {
1377             const subscriptions = await apiFetch('/subscription/user');
1378             const arr = Array.isArray(subscriptions) ? subscriptions : [];

```

```

1379     const tbody = document.getElementById('subscriptions-list');
1380     tbody.innerHTML = arr.length
1381       ? arr.map(s => `<tr>
1382         <td>${s.subscriptionId}</td>
1383         <td>${s.restaurantName || 'N/A'}</td>
1384         <td><span class="chip">${s.mealSlot}</span></td>
1385         <td>${s.scheduleType}</td>
1386         <td><span class="chip ${s.status?.toLowerCase()}>${s.status}</span></td>
1387         <td>${s.nextDeliveryTime ? new Date(s.nextDeliveryTime).toLocaleDateString() : 'Not
scheduled'}</td>
1388         <td class="action-row">
1389           ${s.status === 'ACTIVE' ? `<button class="btn btn-secondary btn-sm"
onclick="pauseSubscription(${s.subscriptionId})">Pause</button>` : ''}
1390           ${s.status === 'PAUSED' ? `<button class="btn btn-primary btn-sm"
onclick="resumeSubscription(${s.subscriptionId})">Resume</button>` : ''}
1391           <button class="btn btn-danger btn-sm"
onclick="expireSubscription(${s.subscriptionId})">Expire</button>
1392         </td>
1393       </tr>`).join('')
1394       : `<tr><td colspan="7" class="empty-state">No subscriptions found</td></tr>`;
1395   } catch (e) { console.error(e); }
1396 }
1397
1398   async function addSubscription() {
1399     const email = document.getElementById('sub-email').value.trim();
1400     const restaurantId = parseInt(document.getElementById('sub-restaurant-select').value);
1401     const scheduleType = document.getElementById('sub-schedule').value;
1402     const dayOfWeek = document.getElementById('sub-day').value;
1403     const selectedSlots = [...document.querySelectorAll('#meal-slot-checkboxes
input[type=checkbox]:checked')].map(cb => cb.value);
1404     if (!email) { showAlert('Please enter employee email', 'error'); return; }
1405     if (!restaurantId) { showAlert('Please select a restaurant', 'error'); return; }
1406     if (!selectedSlots.length) { showAlert('Please select at least one meal slot', 'error'); return; }
1407     const body = {
1408       scheduleType,
1409       mealsSlots: selectedSlots,
1410       slotOrders: selectedSlots.map(slot => ({ mealSlot: slot, restaurantId }))
1411     };
1412     if (scheduleType === 'WEEKLY') body.dayOfWeek = dayOfWeek;
1413     try {
1414       await apiFetch('/subscription', { method: 'POST', body: JSON.stringify(body) });
1415       showAlert('Subscription created successfully!', 'success');
1416       clearSubscriptionForm();
1417       loadSubscriptions();
1418     } catch (e) { showAlert('Failed: ' + e.message, 'error'); }
1419   }
1420
1421   async function pauseSubscription(id) {
1422     try { await apiFetch(`/subscription/${id}/pause`, { method: 'PUT' }); showAlert('Subscription
paused', 'success'); loadSubscriptions(); }
1423     catch (e) { showAlert('Failed: ' + e.message, 'error'); }
1424   }
1425
1426   async function resumeSubscription(id) {
1427     try { await apiFetch(`/subscription/${id}/resume`, { method: 'PUT' }); showAlert('Subscription
resumed', 'success'); loadSubscriptions(); }
1428     catch (e) { showAlert('Failed: ' + e.message, 'error'); }
1429   }
1430
1431   async function expireSubscription(id) {
1432     try { await apiFetch(`/subscription/${id}/expire`, { method: 'PUT' }); showAlert('Subscription
expired', 'success'); loadSubscriptions(); }
1433     catch (e) { showAlert('Failed: ' + e.message, 'error'); }
1434   }
1435
1436   function clearSubscriptionForm() {
1437     document.getElementById('sub-email').value = '';
1438     document.getElementById('sub-restaurant-select').value = '';
1439     document.getElementById('sub-schedule').value = 'DAILY';
1440     document.getElementById('day-of-week-group').style.display = 'none';
1441     document.querySelectorAll('#meal-slot-checkboxes input[type=checkbox]').forEach(cb => {
1442       cb.checked = false; cb.disabled = false;
1443       cb.parentElement.style.opacity = '1'; cb.parentElement.style.display = 'flex';
1444     });
1445   }
1446

```

```

1447     document.getElementById('sub-schedule')?.addEventListener('change', function () {
1448         document.getElementById('day-of-week-group').style.display = this.value === 'WEEKLY' ? 'block' :
1449         'none';
1450     });
1451     // ===== DELIVERIES =====
1452     async function loadDeliveries() {
1453         try {
1454             const subscriptions = await apiFetch('/subscription/user');
1455             let allDeliveries = [];
1456             for (const sub of subscriptions) {
1457                 try {
1458                     const deliveries = await apiFetch(`/delivery/subscription/${sub.subscriptionId}`);
1459                     allDeliveries.push(...deliveries);
1460                 } catch (e) {}
1461             }
1462             const tbody = document.getElementById('deliveries-list');
1463             if (!allDeliveries.length) {
1464                 tbody.innerHTML = '<tr><td colspan="5" class="empty-state">No deliveries found</td></tr>';
1465             } else {
1466                 tbody.innerHTML = allDeliveries.map(d => `<tr>
1467                     <td>${d.id}</td>
1468                     <td>${d.subscription?.id || 'N/A'}</td>
1469                     <td><span class="chip">${d.mealSlot || 'N/A'}</span></td>
1470                     <td>${d.scheduledDeliveryTime ? new Date(d.scheduledDeliveryTime).toLocaleString() :
1471                     'N/A'}</td>
1472                     <td><span class="chip ${d.status?.toLowerCase()}>${d.status ||
1473                     'SCHEDULED'}</span></td>
1474                 </tr>`).join('');
1475             } catch (e) { console.error(e); }
1476         }
1477     // ===== VENDORS =====
1478     async function loadVendors() {
1479         try {
1480             const vendors = await apiFetch('/vendors');
1481             const tbody = document.getElementById('vendors-list');
1482             if (!vendors.length) {
1483                 tbody.innerHTML = '<tr><td colspan="6" class="empty-state">No vendors found</td></tr>';
1484             } else {
1485                 tbody.innerHTML = vendors.map(v => `<tr>
1486                     <td>${v.id}</td>
1487                     <td><strong>${v.name}</strong></td>
1488                     <td>${v.email}</td>
1489                     <td>${v.phone || 'N/A'}</td>
1490                     <td>${v.registeredAt || 'N/A'}</td>
1491                     <td><button class="btn btn-danger btn-sm"
1492                     onclick="deleteVendor(${v.id})">Delete</button></td>
1493                 </tr>`).join('');
1494             } catch (e) { console.error(e); }
1495         }
1496     }
1497     async function createVendor() {
1498         const body = {
1499             name: document.getElementById('vendor-name-create').value.trim(),
1500             email: document.getElementById('vendor-email-create').value.trim(),
1501             password: document.getElementById('vendor-password-create').value,
1502             phone: document.getElementById('vendor-phone-create').value.trim(),
1503             role: document.getElementById('vendor-role').value
1504         };
1505         if (!body.name || !body.email || !body.password || !body.phone) {
1506             showAlert('Please fill all vendor fields', 'error'); return;
1507         }
1508         try {
1509             await apiFetch('/vendors/register', { method: 'POST', body: JSON.stringify(body) });
1510             showAlert('Vendor created successfully!', 'success');
1511             clearVendorForm();
1512             loadVendors();
1513         } catch (e) { showAlert('Failed to create vendor: ' + e.message, 'error'); }
1514     }
1515
1516     function clearVendorForm() {
1517         ['vendor-name-create', 'vendor-email-create', 'vendor-password-create', 'vendor-phone-create']
1518         .forEach(id => document.getElementById(id).value = '');

```

```

1519     }
1520
1521     async function deleteVendor(vendorId) {
1522         if (!confirm('Delete vendor #{vendorId}?')) return;
1523         try {
1524             await apiFetch(`/vendors/${vendorId}`, { method: 'DELETE' });
1525             showAlert('Vendor deleted successfully!', 'success');
1526             loadVendors();
1527         } catch (e) { showAlert('Failed to delete vendor: ' + e.message, 'error'); }
1528     }
1529
1530     // ===== MODAL HELPERS =====
1531     function closeModal() {
1532         document.getElementById('action-modal').classList.remove('open');
1533         document.getElementById('action-remarks').value = '';
1534         document.getElementById('manager-email').value = currentUser?.email || '';
1535     }
1536
1537     function closeCancelModal() {
1538         document.getElementById('cancel-modal').classList.remove('open');
1539         document.getElementById('cancel-email').value = currentUser?.email || '';
1540     }
1541
1542     // ===== DEVICES =====
1543     async function loadUnassignedDevices() {
1544         try {
1545             const devices = await apiFetch('/devices/unassigned');
1546             const tbody = document.getElementById('devices-list');
1547             if (!devices || !devices.length) {
1548                 tbody.innerHTML = '<tr><td colspan="7" class="empty-state">No unassigned
devices</td></tr>';
1549                 return;
1550             }
1551             const today = new Date();
1552             tbody.innerHTML = devices.map(d => {
1553                 const warrantyExpiry = d.warrantyExpiryDate ? new Date(d.warrantyExpiryDate) : null;
1554                 const warrantyDisplay = warrantyExpiry
1555                     ? (warrantyExpiry > today
1556                         ? `<span style="color:var(--green);font-size:11px">
${d.warrantyExpiryDate}</span>`
1557                         : `<span style="color:var(--red);font-size:11px"> ${d.warrantyExpiryDate}</span>`)
1558                     : `<span style="color:var(--text3);font-size:11px">--</span>`;
1559                 return `<tr>
1560                     <td>#{d.id}</td>
1561                     <td><strong>${d.deviceName || '-'}</strong></td>
1562                     <td>${d.brand || '-'}</td>
1563                     <td><span class="chip">${d.deviceType || '-'}</span></td>
1564                     <td>${warrantyDisplay}</td>
1565                     <td>${d.vendorName || '-'}</td>
1566                     <td><button class="btn btn-primary btn-sm" onclick="openAssignModal(${d.id},
'${d.deviceName}')">Assign</button></td>
1567                 </tr>`;
1568             }).join('');
1569         } catch (e) { console.error(e); }
1570     }
1571
1572     async function openAssignModal(deviceId, deviceName) {
1573         document.getElementById('assign-device-id').value = deviceId;
1574         document.getElementById('assign-device-name').value = deviceName;
1575         try {
1576             const employees = await apiFetch('/employee');
1577             const select = document.getElementById('assign-employee-select');
1578             select.innerHTML = '<option value="">-- Select Employee --</option>';
1579             employees.filter(e => e.status === 'ACTIVE').forEach(e => {
1580                 const opt = document.createElement('option');
1581                 opt.value = e.employeeId;
1582                 opt.textContent = `${e.name} (${e.email}) - ${e.dept || 'N/A'}`;
1583                 select.appendChild(opt);
1584             });
1585         } catch (e) { showAlert('Failed to load employees', 'error'); return; }
1586         document.getElementById('assign-modal').classList.add('open');
1587     }
1588
1589     async function confirmAssign() {
1590         const deviceId = document.getElementById('assign-device-id').value;
1591         const employeeId = document.getElementById('assign-employee-select').value;

```

```

1592         if (!employeeId) { showAlert('Please select an employee', 'error'); return; }
1593         try {
1594             const message = await apiFetch(`/devices/${deviceId}/assign`, {
1595                 method: 'POST',
1596                 body: JSON.stringify({ employeeId: parseInt(employeeId) })
1597             });
1598             showAlert(message || 'Device assigned successfully!', 'success');
1599             closeAssignModal();
1600             loadUnassignedDevices();
1601         } catch (e) { showAlert('Failed: ' + e.message, 'error'); }
1602     }
1603
1604     function closeAssignModal() {
1605         document.getElementById('assign-modal').classList.remove('open');
1606         document.getElementById('assign-employee-select').value = '';
1607         document.getElementById('assign-device-id').value = '';
1608         document.getElementById('assign-device-name').value = '';
1609     }
1610
1611     // ===== MY DEVICES =====
1612     async function loadMyAssignedDevices() {
1613         try {
1614             const devices = await apiFetch('/devices/my');
1615             const tbody = document.getElementById('my-devices-list');
1616             if (!devices || !devices.length) {
1617                 tbody.innerHTML = '<tr><td colspan="7" class="empty-state">No devices assigned to
you</td></tr>';
1618                 return;
1619             }
1620             const today = new Date();
1621             tbody.innerHTML = devices.map(d => {
1622                 const warrantyExpiry = d.warrantyExpiryDate ? new Date(d.warrantyExpiryDate) : null;
1623                 const warrantyDisplay = warrantyExpiry
1624                     ? (warrantyExpiry > today
1625                         ? `<span style="color:var(--green);font-size:11px">
${d.warrantyExpiryDate}</span>`
1626                         : `<span style="color:var(--red);font-size:11px"> Expired
${d.warrantyExpiryDate}</span>`)
1627                     : `<span style="color:var(--text3);font-size:11px">--</span>`;
1628                 return `<tr>
1629                     <td>#${d.id}</td>
1630                     <td><strong>${d.deviceName || '-'}</strong></td>
1631                     <td>${d.brand || '-'}</td>
1632                     <td><span class="chip">${(d.deviceType || '').toLowerCase()}</span></td>
1633                     <td>${warrantyDisplay}</td>
1634                     <td style="font-size:12px;color:var(--text2)">${d.assignedDate || '-'}</td>
1635                     <td><span class="chip">${(d.deviceStatus || '').toLowerCase()}</span>${d.deviceStatus || '-'}</td>
1636                 </tr>`;
1637             }).join('');
1638         } catch (e) { console.error(e); }
1639     }
1640
1641     // ===== MY SERVICE REQUESTS =====
1642     async function loadServiceRequests() {
1643         await Promise.all([loadServiceRequestsList(), loadMyDevicesForSR()]);
1644     }
1645
1646     async function loadServiceRequestsList() {
1647         try {
1648             const requests = await apiFetch('/service-requests/my');
1649             const tbody = document.getElementById('service-requests-list');
1650             if (!requests || !requests.length) {
1651                 tbody.innerHTML = '<tr><td colspan="7" class="empty-state">No service requests
yet</td></tr>';
1652                 return;
1653             }
1654             tbody.innerHTML = requests.map(r => `<tr>
1655                 <td>#${r.id}</td>
1656                 <td>${r.deviceName || '-'}</td>
1657                 <td><span class="chip">${r.requestType}</span></td>
1658                 <td style="max-width:200px;overflow:hidden;text-overflow:ellipsis;white-
space:nowrap">${r.issueDescription || '-'}</td>
1659                 <td>${r.urgent ? `<span class="chip pending">YES</span>` : `<span style="color:var(--
text2)">No</span>`}</td>
1660                 <td><span class="chip">${r.status?.toLowerCase()}</span>${r.status}</td>
1661                 <td style="font-size:12px;color:var(--text2)">${r.raisedAt ? new

```

```

Date(r.raisedAt).toLocaleDateString() : '-'}</td>
1662         </tr>`).join('');
1663     } catch (e) { console.error(e); }
1664 }
1665
1666     async function loadMyDevicesForSR() {
1667         try {
1668             const devices = await apiFetch('/devices/my');
1669             const select = document.getElementById('sr-device-select');
1670             select.innerHTML = '<option value="">-- Select your device --</option>';
1671             (devices || []).forEach(d => {
1672                 const opt = document.createElement('option');
1673                 opt.value = d.id;
1674                 opt.textContent = `${d.deviceName} (${d.brand || d.deviceType || 'Device'})`;
1675                 select.appendChild(opt);
1676             });
1677         } catch (e) { console.error('Failed to load devices for SR', e); }
1678     }
1679
1680     async function submitServiceRequest() {
1681         const deviceId = parseInt(document.getElementById('sr-device-select').value);
1682         const requestType = document.getElementById('sr-type').value;
1683         const issueDescription = document.getElementById('sr-description').value.trim();
1684         const urgent = document.getElementById('sr-urgent').checked;
1685         if (!deviceId) { showAlert('Please select a device', 'error'); return; }
1686         if (!issueDescription) { showAlert('Please describe the issue', 'error'); return; }
1687         try {
1688             await apiFetch('/service-requests', {
1689                 method: 'POST',
1690                 body: JSON.stringify({ deviceId, requestType, issueDescription, urgent })
1691             });
1692             showAlert('Service request submitted!', 'success');
1693             clearServiceRequestForm();
1694             loadServiceRequestsList();
1695         } catch (e) { showAlert('Failed: ' + e.message, 'error'); }
1696     }
1697
1698     function clearServiceRequestForm() {
1699         document.getElementById('sr-device-select').value = '';
1700         document.getElementById('sr-type').value = 'REPAIR';
1701         document.getElementById('sr-description').value = '';
1702         document.getElementById('sr-urgent').checked = false;
1703     }
1704
1705     // ===== ADMIN SERVICE REQUESTS =====
1706     async function loadAdminServiceRequests() {
1707         try {
1708             const requests = await apiFetch('/service-requests/open');
1709             const tbody = document.getElementById('admin-service-requests-list');
1710             if (!requests || !requests.length) {
1711                 tbody.innerHTML = '<tr><td colspan="9" class="empty-state">No open service
requests</td></tr>';
1712                 return;
1713             }
1714             tbody.innerHTML = requests.map(r => `<tr>
1715                 <td>#${r.id}</td>
1716                 <td style="font-size:12px;color:var(--text2)">${r.raisedByName || r.employeeId ||
1717                 '-'}</td>
1718                 <td><strong>${r.deviceName || '-'}</strong></td>
1719                 <td><span class="chip">${r.requestType}</span></td>
1720                 <td style="max-width:180px;overflow:hidden;text-overflow:ellipsis;white-space:nowrap;font-
size:12px;color:var(--text2)" title="${r.issueDescription || ''}">${r.issueDescription || '...
1721                 <td>${r.urgent ? '<span class="chip pending">YES</span>' : '<span style="color:var(--
text2);font-size:12px">No</span>'}</td>
1722                 <td><span class="chip ${r.status?.toLowerCase()}>${r.status}</span></td>
1723                 <td style="font-size:12px;color:var(--text2)">${r.raisedAt ? new
Date(r.raisedAt).toLocaleDateString() : '-'}</td>
1724                 <td>
1725                     <button class="btn btn-primary btn-sm" onclick="openAdminSRModal(${r.id},
'${(r.deviceName || '').replace(/'/g, '\\\'')}', '${(r.raisedByName || r.employeeId || '').replace(/'/g,...
1726                     update
1727                 </button>
1728                 </td>
1729                 </tr>`).join('');
1730         } catch (e) { console.error(e); }
1731     }

```

```

1731
1732     function openAdminSRModal(requestId, deviceName, employeeName) {
1733         document.getElementById('admin-sr-id').value = requestId;
1734         document.getElementById('admin-sr-modal-id').textContent = `#${requestId}`;
1735         document.getElementById('admin-sr-device-display').value = deviceName;
1736         document.getElementById('admin-sr-employee-display').value = employeeName;
1737         document.getElementById('admin-sr-status').value = 'SENT_FOR_REPAIR';
1738         document.getElementById('admin-sr-remarks').value = '';
1739         // Show hint for default selection
1740         document.getElementById('admin-sr-repair-hint').style.display = 'block';
1741         document.getElementById('admin-sr-modal').classList.add('open');
1742     }
1743
1744     // Show/hide the repair hint based on status selection
1745     document.getElementById('admin-sr-status')?.addEventListener('change', function () {
1746         document.getElementById('admin-sr-repair-hint').style.display =
1747             this.value === 'SENT_FOR_REPAIR' ? 'block' : 'none';
1748     });
1749
1750     async function submitAdminSRUpdate() {
1751         const serviceRequestId = parseInt(document.getElementById('admin-sr-id').value);
1752         const status = document.getElementById('admin-sr-status').value;
1753         const adminRemarks = document.getElementById('admin-sr-remarks').value.trim();
1754
1755         if (!adminRemarks) {
1756             showAlert('Please add remarks before updating', 'error');
1757             return;
1758         }
1759
1760         try {
1761             await apiFetch('/service-requests/admin', {
1762                 method: 'PUT',
1763                 body: JSON.stringify({ serviceRequestId, status, adminRemarks })
1764             });
1765             const statusLabel = status.replace(/_/g, ' ').toLowerCase();
1766             showAlert('Service request marked as "${statusLabel}" successfully!', 'success');
1767             closeAdminSRModal();
1768             loadAdminServiceRequests();
1769         } catch (e) { showAlert('Failed: ' + e.message, 'error'); }
1770     }
1771
1772     function closeAdminSRModal() {
1773         document.getElementById('admin-sr-modal').classList.remove('open');
1774         document.getElementById('admin-sr-id').value = '';
1775         document.getElementById('admin-sr-remarks').value = '';
1776         document.getElementById('admin-sr-repair-hint').style.display = 'none';
1777     }
1778
1779     // ===== LOGOUT =====
1780     function logout() {
1781         localStorage.removeItem('jwt_token');
1782         localStorage.removeItem('refresh_token');
1783         localStorage.removeItem('auth_type');
1784         localStorage.removeItem('last_role');
1785         fetch(`${API}/session/logout`, { method: 'POST', credentials: 'include' })
1786             .finally(() => {
1787                 document.cookie = 'JSESSIONID=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/;';
1788                 window.location.href = '/google.html';
1789             });
1790     }
1791
1792     async function loadNotifCount() {
1793         try {
1794             const data = await apiFetch('/notifications/unread-count');
1795             const badge = document.getElementById('notif-badge');
1796
1797             // Handles: plain number, { count: N }, or { unreadCount: N }
1798             const count = typeof data === 'number'
1799                 ? data
1800                 : (data?.count ?? data?.unreadCount ?? 0);
1801
1802             if (count > 0) {
1803                 badge.textContent = count;
1804                 badge.style.display = 'flex';
1805             } else {
1806                 badge.style.display = 'none';
1807             }
1808         }
1809     }

```



```

1807     } catch(e) { console.warn('Notif count error', e); }
1808 }
1809
1810 async function toggleNotifPanel() {
1811     const panel = document.getElementById('notif-panel');
1812     if (panel.style.display === 'none') {
1813         panel.style.display = 'block';
1814         await loadNotifications();
1815         try { await apiFetch('/notifications/mark-read', { method: 'POST' }); } catch(e) {}
1816         document.getElementById('notif-badge').style.display = 'none';
1817     } else {
1818         panel.style.display = 'none';
1819     }
1820 }
1821
1822 async function loadNotifications() {
1823     try {
1824         const data = await apiFetch('/notifications');
1825         const list = document.getElementById('notif-list');
1826         list.innerHTML = (data && data.length)
1827             ? data.map(n => `
1828                 <div class="notif-item ${n.read ? '' : 'unread'}">
1829                     <div>${n.message}</div>
1830                     <div class="notif-time">${new Date(n.createdAt).toLocaleString()}</div>
1831                 </div>`).join('')
1832             : '<div class="notif-item">No notifications yet.</div>';
1833     } catch(e) { console.warn('Notif load error', e); }
1834 }
1835
1836 loadNotifCount();
1837 setInterval(loadNotifCount, 30000);
1838
1839 document.addEventListener('click', e => {
1840     const panel = document.getElementById('notif-panel');
1841     const wrapper = document.querySelector('.notif-wrapper');
1842     if (panel && wrapper && !panel.contains(e.target) && !wrapper.contains(e.target)) {
1843         panel.style.display = 'none';
1844     }
1845 });
1846
1847 // Set today's date as default for leave form
1848 document.getElementById('leave-start').valueAsDate = new Date();
1849 document.getElementById('leave-end').valueAsDate = new Date();
1850
1851 // ===== HOLIDAYS =====
1852 async function loadHolidays() {
1853     try {
1854         const year = new Date().getFullYear();
1855         const holidays = await apiFetch(`/holidays?year=${year}`);
1856         const tbody = document.getElementById('holidays-list');
1857         if (!holidays || !holidays.length) {
1858             tbody.innerHTML = '<tr><td colspan="5" class="empty-state">No holidays added yet</td></tr>';
1859             return;
1860         }
1861         tbody.innerHTML = holidays.map(h => `<tr>
1862             <td>#${h.id}</td>
1863             <td><strong>${h.name}</strong></td>
1864             <td>${h.date}</td>
1865             <td>${h.description || '-'}</td>
1866             <td>
1867                 <button class="btn btn-danger btn-sm" onclick="deleteHoliday(${h.id})">Delete</button>
1868             </td>
1869         </tr>`).join('');
1870     } catch(e) { console.error(e); }
1871 }
1872
1873 async function addHoliday() {
1874     const name = document.getElementById('h-name').value.trim();
1875     const date = document.getElementById('h-date').value;
1876     const description = document.getElementById('h-desc').value.trim();
1877     if (!name || !date) { showAlert('Please enter name and date', 'error'); return; }
1878     try {
1879         await apiFetch('/holidays', {
1880             method: 'POST',
1881             body: JSON.stringify({ name, date, description })
1882         });

```

```

1883     showAlert('Holiday added!', 'success');
1884     clearHolidayForm();
1885     loadHolidays();
1886   } catch(e) { showAlert('Failed: ' + e.message, 'error'); }
1887 }
1888
1889 async function deleteHoliday(id) {
1890   if (!confirm('Delete this holiday?')) return;
1891   try {
1892     await apiFetch(`/holidays/${id}`, { method: 'DELETE' });
1893     showAlert('Holiday deleted', 'success');
1894     loadHolidays();
1895   } catch(e) { showAlert('Failed: ' + e.message, 'error'); }
1896 }
1897
1898 function clearHolidayForm() {
1899   document.getElementById('h-name').value = '';
1900   document.getElementById('h-date').value = '';
1901   document.getElementById('h-desc').value = '';
1902 }
1903
1904 // ===== MY PROFILE =====
1905 async function loadAdminProfile() {
1906   try {
1907     const me = await apiFetch('/employee/me');
1908     if(me) {
1909       document.getElementById('p-name').value = me.name || '-';
1910       document.getElementById('p-email').value = me.email || '-';
1911       document.getElementById('p-dept').value = me.dept || '-';
1912       document.getElementById('p-role').value = me.role || 'ADMIN';
1913       document.getElementById('p-status').value = me.status || '-';
1914       document.getElementById('p-tz').value = me.timezone || '-';
1915
1916       // Show maternity section only for female admins
1917       document.getElementById('maternity-apply-btn').style.display =
1918         me.gender === 'F' ? 'block' : 'none';
1919     }
1920   } catch(e) {
1921     showAlert('Could not load profile', 'error');
1922   }
1923 }
1924
1925 // Auto calculate maternity end date
1926 document.getElementById('mat-start-date').addEventListener('change', function(){
1927   if(this.value) {
1928     const end = new Date(this.value);
1929     end.setDate(end.getDate() + 179);
1930     document.getElementById('mat-end-date').value =
1931       end.toISOString().split('T')[0];
1932   } else {
1933     document.getElementById('mat-end-date').value = '';
1934   }
1935 });
1936
1937 // Submit maternity leave
1938 async function applyMaternityLeave() {
1939   const startDate = document.getElementById('mat-start-date').value;
1940   const reason = document.getElementById('mat-reason').value.trim();
1941
1942   if(!startDate) {
1943     showAlert('Please select a start date', 'error');
1944     return;
1945   }
1946
1947   try {
1948     await apiFetch('/leave_requests/maternity/apply', {
1949       method: 'POST',
1950       body: JSON.stringify({ startDate, reason })
1951     });
1952     showAlert('Maternity leave submitted successfully!', 'success');
1953     document.getElementById('mat-start-date').value = '';
1954     document.getElementById('mat-end-date').value = '';
1955     document.getElementById('mat-reason').value = '';
1956     loadMyLeaves();
1957   } catch(e) {
1958     showAlert('Failed: ' + e.message, 'error');

```

```

1959     }
1960 }
1961
1962     // ===== CALENDAR =====
1963     let employeeCalendar = null;
1964     let calendarInitialized = false;
1965
1966     const STATUS_COLORS = {
1967         APPROVED: { bg:'rgba(54,211,153,0.25)', border:'#36d399', text:'#36d399' },
1968         PENDING: { bg:'rgba(249,199,79,0.25)', border:'#f9c74f', text:'#f9c74f' },
1969         REJECTED: { bg:'rgba(248,113,113,0.25)', border:'#f87171', text:'#f87171' },
1970         HOLIDAY: { bg:'rgba(167,139,250,0.25)', border:'#a78bfa', text:'#a78bfa' },
1971     };
1972     const TYPE_LABEL = {
1973         SICK:' Sick', CASUAL:' Casual',
1974         MATERNITY:' Maternity', UNPAID:' Unpaid'
1975     };
1976
1977     async function loadMyCalendar() {
1978         await loadCalendarBalanceCards();
1979         if(!calendarInitialized) {
1980             initFullCalendar();
1981         } else {
1982             await refreshCalendarEvents();
1983         }
1984     }
1985
1986     async function loadCalendarBalanceCards() {
1987         ['cal-sick-remaining','cal-casual-remaining','cal-mat-remaining','cal-total-used']
1988             .forEach(id => document.getElementById(id).textContent = '--');
1989         try {
1990             const dashboard = await apiFetch('/api/employees/me/leave-dashboard');
1991             const balances = dashboard?.leaveBalances || {};
1992             let totalUsed = 0, totalAllowed = 0;
1993
1994             const sick = balances['SICK'] || {};
1995             const sickRem = Number(sick.available || 0);
1996             const sickUsed = Number(sick.total || 0) - sickRem;
1997             document.getElementById('cal-sick-remaining').textContent = sickRem;
1998             document.getElementById('cal-sick-sub').textContent = `${sickUsed} used of
1999             ${Number(sick.total||0)} days`;
2000
2001             const cas = balances['CASUAL'] || {};
2002             const casRem = Number(cas.available || 0);
2003             const casUsed = Number(cas.total || 0) - casRem;
2004             document.getElementById('cal-casual-remaining').textContent = casRem;
2005             document.getElementById('cal-casual-sub').textContent = `${casUsed} used of
2006             ${Number(cas.total||0)} days`;
2007
2008             const mat = balances['MATERNITY'] || {};
2009             const matRem = Number(mat.available || 0);
2010             const matUsed = Number(mat.total || 0) - matRem;
2011             document.getElementById('cal-mat-remaining').textContent = matRem;
2012             document.getElementById('cal-mat-sub').textContent = `${matUsed} used of ${Number(mat.total||0)}
2013             days`;
2014
2015             Object.values(balances).forEach(b => {
2016                 totalUsed += (Number(b.total||0) - Number(b.available||0));
2017                 totalAllowed += Number(b.total||0);
2018             });
2019             document.getElementById('cal-total-used').textContent = totalUsed;
2020             document.getElementById('cal-total-sub').textContent = `out of ${totalAllowed} days total`;
2021         } catch(e) {
2022             ['cal-sick-sub','cal-casual-sub','cal-mat-sub','cal-total-sub']
2023                 .forEach(id => document.getElementById(id).textContent = 'unavailable');
2024         }
2025     }
2026
2027     async function buildCalendarEvents() {
2028         const events = [];
2029         try {
2030             const data = await apiFetch('/leave_requests/employee');
2031             const leaves = data?.requests || data || [];
2032             leaves.forEach(l => {
2033                 const s = (l.status || 'PENDING').toUpperCase();

```

```

2032         const c = STATUS_COLORS[s] || STATUS_COLORS.PENDING;
2033         let endDate = l.endDate;
2034         if(endDate) {
2035             const d = new Date(endDate);
2036             d.setDate(d.getDate() + 1);
2037             endDate = d.toISOString().split('T')[0];
2038         }
2039         events.push({
2040             id: `leave-${l.leaveRequestId}`,
2041             title: `${TYPE_LABEL[l.leaveType] || l.leaveType}
2042             (${s.charAt(0)+s.slice(1).toLowerCase()})`,
2043             start: l.startDate, end: endDate, allDay: true,
2044             backgroundColor: c.bg, borderColor: c.border, textColor: c.text,
2045             extendedProps: { type:'leave', status:s, reason:l.reason||'-', leaveType:l.leaveType }
2046         });
2047     } catch(e) { console.warn('Could not load leaves for calendar'); }
2048
2049     try {
2050         const holidays = await apiFetch('/holidays');
2051         (holidays || []).forEach(h => {
2052             const c = STATUS_COLORS.HOLIDAY;
2053             events.push({
2054                 id: `holiday-${h.id||Math.random()}`,
2055                 title: ` ${h.name || 'Holiday'}`,
2056                 start: h.date || h.holidayDate, allDay: true,
2057                 backgroundColor: c.bg, borderColor: c.border, textColor: c.text,
2058                 extendedProps: { type:'holiday', description:h.description||'' }
2059             });
2060         });
2061     } catch(e) { console.warn('Could not load holidays for calendar'); }
2062
2063     return events;
2064 }
2065
2066 function initFullCalendar() {
2067     employeeCalendar = new FullCalendar.Calendar(
2068         document.getElementById('employee-calendar'), {
2069             initialView: 'dayGridMonth',
2070             height: 'auto',
2071             headerToolbar: {
2072                 left:'prev,next today', center:'title', right:'dayGridMonth,dayGridWeek,listMonth'
2073             },
2074             buttonText: { today:'Today', month:'Month', week:'Week', listMonth:'List' },
2075             dayMaxEvents: 3,
2076             navLinks: true,
2077             events: [],
2078             eventMouseEnter: function(info) {
2079                 const ep = info.event.extendedProps;
2080                 const tt = document.getElementById('cal-tooltip');
2081                 let html = `<div class="tt-title">${info.event.title}</div>`;
2082                 if(ep.type === 'leave') {
2083                     html += `<div class="tt-row"> ${info.event.startStr}</div>`;
2084                     if(ep.reason && ep.reason !== '-')
2085                         html += `<div class="tt-row"> ${ep.reason.substring(0,60)}</div>`;
2086                     html += `<div class="tt-row">Status: <strong
2087 style="color:${STATUS_COLORS[ep.status]?.text||'#fff'}">${ep.status}</strong></div>`;
2088                 } else if(ep.description) {
2089                     html += `<div class="tt-row">${ep.description}</div>`;
2090                 }
2091                 tt.innerHTML = html;
2092                 tt.style.display = 'block';
2093             },
2094             eventMouseLeave: function() {
2095                 document.getElementById('cal-tooltip').style.display = 'none';
2096             },
2097             eventClick: function(info) {
2098                 if(info.event.extendedProps.type === 'leave') {
2099                     showView('my-leaves', null);
2100                 }
2101             }
2102         });
2103     employeeCalendar.render();
2104     calendarInitialized = true;
2105     refreshCalendarEvents();

```

```

2106
2107 async function refreshCalendarEvents() {
2108     if(!employeeCalendar) return;
2109     const events = await buildCalendarEvents();
2110     employeeCalendar.removeAllEvents();
2111     events.forEach(e => employeeCalendar.addEvent(e));
2112 }
2113
2114 async function refreshCalendar() {
2115     await loadCalendarBalanceCards();
2116     await refreshCalendarEvents();
2117     showAlert('Calendar refreshed!', 'success');
2118 }
2119
2120 document.addEventListener('mousemove', function(e) {
2121     const tt = document.getElementById('cal-tooltip');
2122     if(tt && tt.style.display === 'block') {
2123         tt.style.left = (e.clientX + 14) + 'px';
2124         tt.style.top = (e.clientY + 14) + 'px';
2125         const rect = tt.getBoundingClientRect();
2126         if(rect.right > window.innerWidth)
2127             tt.style.left = (e.clientX - rect.width - 10) + 'px';
2128         if(rect.bottom > window.innerHeight)
2129             tt.style.top = (e.clientY - rect.height - 10) + 'px';
2130     }
2131 });
2132 // ===== TWO-FACTOR AUTH =====
2133 async function loadTotpStatus() {
2134     const chip = document.getElementById('totp-status-chip');
2135     chip.textContent = 'Checking...';
2136     chip.className = 'chip';
2137     // Reset UI
2138     document.getElementById('qr-area').style.display = 'none';
2139     document.getElementById('totp-verify-input').value = '';
2140     document.getElementById('totp-verify-error').style.display = 'none';
2141
2142     try {
2143         const data = await apiFetch('/totp/status');
2144         if (data.totpEnabled) {
2145             chip.textContent = 'ENABLED';
2146             chip.className = 'chip active';
2147             document.getElementById('totp-setup-section').style.display = 'none';
2148             document.getElementById('totp-disable-section').style.display = 'block';
2149         } else {
2150             chip.textContent = 'DISABLED';
2151             chip.className = 'chip inactive';
2152             document.getElementById('totp-setup-section').style.display = 'block';
2153             document.getElementById('totp-disable-section').style.display = 'none';
2154         }
2155     } catch(e) {
2156         chip.textContent = 'Error';
2157         chip.className = 'chip pending';
2158     }
2159 }
2160
2161 async function generateTotpQr() {
2162     const btn = document.getElementById('generate-qr-btn');
2163     btn.textContent = 'Generating...';
2164     btn.disabled = true;
2165     try {
2166         const data = await apiFetch('/totp/setup', { method: 'POST' });
2167         // Show secret as manual entry fallback
2168         document.getElementById('totp-secret-display').textContent = data.secret || '';
2169         // Build QR image from the otpauth URI using a free QR API
2170         const qrUri = encodeURIComponent(data.qrCodeUri || '');
2171         document.getElementById('qr-img').src =
2172             `https://api.qrserver.com/v1/create-qr-code/?size=180x180&data=${qrUri}`;
2173         document.getElementById('qr-area').style.display = 'block';
2174     } catch(e) {
2175         showAlert('Failed to generate QR: ' + e.message, 'error');
2176     } finally {
2177         btn.textContent = 'Regenerate QR Code';
2178         btn.disabled = false;
2179     }
2180 }
2181

```

```
2182     async function verifyAndActivateTotp() {
2183         const code = document.getElementById('totp-verify-input').value.trim();
2184         const errEl = document.getElementById('totp-verify-error');
2185         errEl.style.display = 'none';
2186         if (code.length !== 6) {
2187             errEl.textContent = 'Please enter a 6-digit code.';
2188             errEl.style.display = 'block';
2189             return;
2190         }
2191         try {
2192             await apiFetch('/totp/verify-setup', {
2193                 method: 'POST',
2194                 body: JSON.stringify({ code })
2195             });
2196             showAlert(' 2FA activated! You will need your authenticator on next login.', 'success');
2197             loadTotpStatus(); // refresh UI
2198         } catch(e) {
2199             errEl.textContent = 'Invalid code. Try again.';
2200             errEl.style.display = 'block';
2201         }
2202     }
2203
2204     async function disableTotp() {
2205         if (!confirm('Are you sure you want to disable two-factor authentication?')) return;
2206         try {
2207             await apiFetch('/totp/disable', { method: 'DELETE' });
2208             showAlert('2FA has been disabled.', 'success');
2209             loadTotpStatus();
2210         } catch(e) {
2211             showAlert('Failed to disable 2FA: ' + e.message, 'error');
2212         }
2213     }
2214 </script>
2215 </body>
2216 </html>
2217
```

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>EMS – Employee Portal</title>
7      <link
href="https://fonts.googleapis.com/css2?family=Sora:wght@300;400;500;600;700&family=JetBrains+Mono:wght@400;500&
display=swap" rel="stylesheet">
8      <!-- FullCalendar CDN -->
9      <link href="https://cdn.jsdelivr.net/npm/fullcalendar@6.1.8/index.global.min.css" rel="stylesheet">
10     <script src="https://cdn.jsdelivr.net/npm/fullcalendar@6.1.8/index.global.min.js"></script>
11     <style>
12         *{margin:0;padding:0;box-sizing:border-box}
13         :root{
14             --bg:#080c10;--bg2:#0d1117;--bg3:#111820;
15             --panel:#141c26;--panel2:#1a2332;
16             --border:rgba(99,179,237,0.08);--border2:rgba(99,179,237,0.15);
17             --text:#d4e6f1;--text2:#7a9bb5;--text3:#4a6d87;
18             --accent:#38bdf8;--accent-dim:rgba(56,189,248,0.1);
19             --green:#34d399;--green-dim:rgba(52,211,153,0.1);
20             --amber:#fbbf24;--amber-dim:rgba(251,191,36,0.1);
21             --red:#f87171;--red-dim:rgba(248,113,113,0.1);
22             --font:'Sora', sans-serif;--mono:'JetBrains Mono', monospace;
23             --sidebar:240px;
24         }
25         body{font-family:var(--font);background:var(--bg);color:var(--text);font-
size:14px;overflow:hidden;height:100vh}
26         .app{display:flex;height:100vh}
27
28         /* SIDEBAR */
29         .sidebar{width:var(--sidebar);background:var(--bg2);border-right:1px solid var(--
border);display:flex;flex-direction:column;position:relative;flex-shrink:0}
30         .sidebar::before{content:'';position:absolute;top:0;left:0;right:0;height:1px;background:linear-
gradient(90deg,transparent, var(--accent), transparent)}
31         .logo{padding:24px 20px;border-bottom:1px solid var(--border)}
32         .logo-mark{display:flex;align-items:center;gap:10px;margin-bottom:6px}
33         .logo-icon{width:32px;height:32px;background:linear-gradient(135deg, var(--accent), #0ea5e9);border-
radius:8px;display:flex;align-items:center;justify-content:center;font-weight:700;font-size...
34         .logo h2{font-size:16px;font-weight:600;color:var(--text);letter-spacing:0.5px}
35         .logo p{font-size:11px;color:var(--text3);font-family:var(--mono);margin-top:2px}
36         .role-badge{display:inline-flex;align-items:center;gap:5px;padding:4px 10px;background:var(--accent-dim);border:1px solid rgba(56,189,248,0.25);border-radius:20px;font-size:10px;font-weight...
37         .role-badge::before{content:'';width:6px;height:6px;background:var(--accent);border-
radius:50%;animation:pulse 2s infinite}
38         @keyframes pulse{0%,100%{opacity:1}50%{opacity:0.4}}
39         .nav{flex:1;padding:12px 0;overflow-y:auto}
40         .nav-section{padding:4px 20px 8px;font-size:10px;font-weight:600;color:var(--text3);letter-
spacing:1.5px;text-transform:uppercase;margin-top:8px}
41         .nav-item{display:flex;align-items:center;gap:10px;padding:10px 20px;margin:2px 8px;border-
radius:8px;cursor:pointer;transition:all 0.2s;color:var(--text2);font-size:13px;font-weight:500}
42         .nav-item:hover{background:var(--accent-dim);color:var(--accent)}
43         .nav-item.active{background:var(--accent-dim);color:var(--accent);border:1px solid
rgba(56,189,248,0.2)}
44         .nav-item .icon{width:20px;text-align:center;font-size:14px;flex-shrink:0}
45         .sidebar-footer{padding:16px 20px;border-top:1px solid var(--border)}
46         .user-card{display:flex;align-items:center;gap:10px}
47         .avatar{width:32px;height:32px;background:linear-gradient(135deg, #0ea5e9, var(--accent));border-
radius:50%;display:flex;align-items:center;justify-content:center;font-size:13px;font-weight:6...
48         .user-info-side .name{font-size:12px;font-weight:600;color:var(--text)}
49         .user-info-side .email{font-size:10px;color:var(--text3);font-family:var(--mono);white-
space:nowrap;overflow:hidden;text-overflow:ellipsis;max-width:140px}
50
51         /* MAIN */
52         .main{flex:1;display:flex;flex-direction:column;overflow:hidden}
53         .topbar{background:var(--bg2);padding:14px 24px;display:flex;justify-content:space-between;align-
items:center;border-bottom:1px solid var(--border);flex-shrink:0}
54         .page-title{font-size:16px;font-weight:600;color:var(--text)}
55         .topbar-right{display:flex;align-items:center;gap:12px}
56         .logout-btn{padding:7px 16px;background:transparent;border:1px solid var(--border2);border-
radius:6px;color:var(--text2);font-family:var(--font);font-size:12px;cursor:pointer;transition:all...
57         .logout-btn:hover{background:var(--red-dim);border-color:var(--red);color:var(--red)}

```

```

58     .notif-wrapper { position: relative; cursor: pointer; }
59
60     .notif-badge {
61         position: absolute; top: -5px; right: -6px;
62         background: #e53e3e; color: white;
63         font-size: 10px; font-weight: 600;
64         min-width: 16px; height: 16px;
65         border-radius: 50%; display: flex;
66         align-items: center; justify-content: center;
67         padding: 0 3px;
68     }
69
70     .notif-panel {
71         position: absolute; top: 48px; right: 16px;
72         width: 320px; max-height: 400px; overflow-y: auto;
73         background: var(--panel); border: 1px solid #e2e8f0;
74         border-radius: 10px; box-shadow: 0 4px 16px rgba(0,0,0,0.1);
75         z-index: 999;
76     }
77
78     .notif-header {
79         padding: 12px 16px; font-weight: 600;
80         border-bottom: 1px solid #e2e8f0;
81     }
82
83     .notif-item {
84         padding: 12px 16px; font-size: 14px;
85         border-bottom: 1px solid #f1f5f9;
86     }
87
88     .notif-item.unread { background: #eff6ff; }
89     .notif-time { font-size: 11px; color: #94a3b8; margin-top: 4px; }
90
91     /* CONTENT */
92     .content{flex:1;overflow-y:auto;padding:24px}
93     .view{display:none}
94     .view.active{display:block;animation:fadeIn 0.3s ease}
95     @keyframes fadeIn{from{opacity:0;transform:translateY(8px)}to{opacity:1;transform:translateY(0)}}
96
97     /* CARDS */
98     .stats-grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(160px,1fr));gap:16px;margin-
bottom:24px}
99     .stat-card{background:var(--panel);border:1px solid var(--border);border-
radius:12px;padding:20px;position:relative;overflow:hidden;transition:border-color 0.2s}
100     .stat-card:hover{border-color:var(--border2)}
101     .stat-card::after{content:'';position:absolute;top:0;left:0;right:0;height:2px}
102     .stat-card.blue::after{background:var(--accent)}
103     .stat-card.green::after{background:var(--green)}
104     .stat-card.amber::after{background:var(--amber)}
105     .stat-card.red::after{background:var(--red)}
106     .stat-label{font-size:11px;color:var(--text3);text-transform:uppercase;letter-spacing:1px;margin-
bottom:10px;font-weight:500}
107     .stat-value{font-size:28px;font-weight:700;color:var(--text);font-family:var(--mono)}
108     .stat-sub{font-size:11px;color:var(--text3);margin-top:4px}
109
110     /* SECTION */
111     .section{background:var(--panel);border:1px solid var(--border);border-radius:12px;margin-
bottom:20px;overflow:hidden}
112     .section-head{padding:16px 20px;border-bottom:1px solid var(--border);display:flex;justify-
content:space-between;align-items:center}
113     .section-title{font-size:14px;font-weight:600;color:var(--text)}
114     .section-sub{font-size:12px;color:var(--text3)}
115
116     /* TABLE */
117     .tablewrap{overflow-x:auto}
118     table{width:100%;border-collapse:collapse}
119     th{padding:10px 16px;text-align:left;font-size:11px;color:var(--text3);font-weight:600;letter-
spacing:0.8px;text-transform:uppercase;background:var(--bg3);border-bottom:1px solid var(--bord...
120     td{padding:12px 16px;border-bottom:1px solid var(--border);font-size:13px;color:var(--text2)}
121     tr:last-child td{border-bottom:none}
122     tr:hover td{background:rgba(56,189,248,0.03);color:var(--text)}
123
124     /* CHIPS */
125     .chip{display:inline-flex;align-items:center;padding:3px 10px;border-radius:20px;font-
size:11px;font-weight:600;letter-spacing:0.5px}
126     .chip.pending{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(251,191,36,0.2)}

```



```

127     .chip.open{background:var(--accent-dim);color:var(--accent)}
128     .chip.approved{background:var(--green-dim);color:var(--green);border:1px solid
rgba(52,211,153,0.2)}
129     .chip.rejected{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
130     .chip.active{background:var(--green-dim);color:var(--green);border:1px solid rgba(52,211,153,0.2)}
131     .chip.sick{background:var(--red-dim);color:var(--red)}
132     .chip.casual{background:var(--amber-dim);color:var(--amber)}
133     .chip.maternity{background:var(--accent-dim);color:var(--accent)}
134     .chip.under_repair { background: var(--amber-dim); color: var(--amber); }
135     .chip.unpaid { background: rgba(139,92,246,0.15); color: #a78bfa; border: 1px solid
rgba(139,92,246,0.2); }
136     .chip.assigned{background:var(--green-dim);color:var(--green)}
137     .chip.condemned{background:var(--red-dim);color:var(--red)}
138
139     /* FORM */
140     .form-card{padding:24px}
141     .form-grid{display:grid;grid-template-columns:1fr 1fr;gap:16px}
142     .form-group{display:flex;flex-direction:column;gap:6px}
143     .form-group.full{grid-column:1/-1}
144     label{font-size:11px;font-weight:600;color:var(--text3);text-transform:uppercase;letter-
spacing:0.8px}
145     input,select,textarea{background:var(--bg3);border:1px solid var(--border);border-
radius:8px;padding:10px 14px;color:var(--text);font-family:var(--font);font-size:13px;transition:border-col...
146     input:focus,select:focus,textarea:focus{outline:none;border-color:var(--accent);background:var(--
bg2)}
147     input::placeholder,textarea::placeholder{color:var(--text3)}
148     select option{background:var(--bg2)}
149
150     /* BUTTONS */
151     .btn{padding:9px 18px;border-radius:8px;border:none;font-family:var(--font);font-size:13px;font-
weight:600;cursor:pointer;transition:all 0.2s}
152     .btn-primary{background:var(--accent);color:#fff}
153     .btn-primary:hover{background:#0ea5e9;transform:translateY(-1px);box-shadow:0 4px 12px
rgba(56,189,248,0.3)}
154     .btn-secondary{background:var(--panel2);color:var(--text2);border:1px solid var(--border)}
155     .btn-secondary:hover{border-color:var(--border2);color:var(--text)}
156     .btn-sm{padding:6px 12px;font-size:12px}
157     .btn-danger{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
158     .btn-danger:hover{background:var(--red);color:#fff}
159     .form-actions{display:flex;gap:10px;margin-top:20px}
160
161     /* ALERT */
162     .alert{position:fixed;top:20px;right:20px;padding:12px 20px;border-radius:10px;z-
index:1100;animation:slideIn 0.3s ease;font-size:13px;font-weight:500}
163     .alert-success{background:var(--green-dim);color:var(--green);border:1px solid
rgba(52,211,153,0.3)}
164     .alert-error{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.3)}
165     @keyframes slideIn{from{transform:translateX(120%);opacity:0}to{transform:translateX(0);opacity:1}}
166
167     .empty-state{text-align:center;color:var(--text3);font-style:italic;padding:30px}
168     .profile-grid{display:grid;grid-template-columns:1fr 1fr;gap:16px;padding:24px}
169     .profile-field{display:flex;flex-direction:column;gap:4px}
170     .profile-field .field-label{font-size:10px;color:var(--text3);text-transform:uppercase;letter-
spacing:0.8px;font-weight:600}
171     .profile-field .field-value{font-size:14px;color:var(--text);padding:10px 14px;background:var(--
bg3);border-radius:8px;border:1px solid var(--border);font-family:var(--mono)}
172
173     .subscription-card{display:grid;grid-template-columns:repeat(auto-
fill,minmax(200px,1fr));gap:12px;padding:20px}
174     .sub-item{background:var(--bg3);border:1px solid var(--border);border-
radius:10px;padding:16px;transition:border-color 0.2s}
175     .sub-item:hover{border-color:var(--border2)}
176     .sub-item .meal-name{font-weight:600;color:var(--text);margin-bottom:6px}
177     .sub-item .meal-meta{font-size:11px;color:var(--text3)}
178
179     .chip.paused{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(251,191,36,0.2)}
180     .chip.expired{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
181
182     /* ===== */
183     /* CALENDAR VIEW STYLES */
184     /* ===== */
185
186     /* Leave balance summary cards above the calendar */
187     .leave-balance-grid {
188         display: grid;
189         grid-template-columns: repeat(auto-fit, minmax(160px, 1fr));

```

```

190         gap: 14px;
191         padding: 20px;
192         border-bottom: 1px solid var(--border);
193     }
194     .leave-bal-card {
195         background: var(--bg3);
196         border: 1px solid var(--border);
197         border-radius: 10px;
198         padding: 16px 18px;
199         display: flex;
200         flex-direction: column;
201         gap: 6px;
202         position: relative;
203         overflow: hidden;
204         transition: border-color 0.2s;
205     }
206     .leave-bal-card:hover { border-color: var(--border2); }
207     .leave-bal-card::before {
208         content: '';
209         position: absolute;
210         left: 0; top: 0; bottom: 0;
211         width: 3px;
212         border-radius: 3px 0 0 3px;
213     }
214     .leave-bal-card.sick::before { background: var(--red); }
215     .leave-bal-card.casual::before { background: var(--amber); }
216     .leave-bal-card.mat::before { background: var(--accent); }
217     .leave-bal-card.total::before { background: var(--green); }
218     .lbc-icon { font-size: 18px; }
219     .lbc-label { font-size: 10px; color: var(--text3); text-transform: uppercase; letter-spacing: 1px;
font-weight: 600; }
220     .lbc-count { font-size: 24px; font-weight: 700; color: var(--text); font-family: var(--mono); line-
height: 1; }
221     .lbc-sub { font-size: 11px; color: var(--text3); }
222
223     /* Legend row */
224     .cal-legend {
225         display: flex;
226         align-items: center;
227         gap: 20px;
228         padding: 12px 20px;
229         border-bottom: 1px solid var(--border);
230         flex-wrap: wrap;
231     }
232     .legend-label { font-size: 11px; color: var(--text3); font-weight: 600; text-transform: uppercase;
letter-spacing: 0.8px; margin-right: 4px; }
233     .legend-item { display: flex; align-items: center; gap: 6px; font-size: 12px; color: var(--text2);
}
234     .legend-dot { width: 10px; height: 10px; border-radius: 50%; flex-shrink: 0; }
235
236     /* FullCalendar dark-theme overrides */
237     #employee-calendar {
238         padding: 16px 20px 20px;
239     }
240     /* Toolbar */
241     #employee-calendar .fc .fc-toolbar-title {
242         font-family: var(--font);
243         font-size: 15px;
244         font-weight: 600;
245         color: var(--text);
246     }
247     #employee-calendar .fc .fc-button {
248         background: var(--panel2);
249         border: 1px solid var(--border2);
250         color: var(--text2);
251         font-family: var(--font);
252         font-size: 12px;
253         font-weight: 600;
254         border-radius: 6px;
255         padding: 5px 12px;
256         transition: all 0.2s;
257         box-shadow: none;
258         text-transform: capitalize;
259     }
260     #employee-calendar .fc .fc-button:hover,
261     #employee-calendar .fc .fc-button-active,

```

```

262     #employee-calendar .fc .fc-button:not(:disabled):active {
263         background: var(--accent-dim);
264         border-color: var(--accent);
265         color: var(--accent);
266         box-shadow: none;
267     }
268     #employee-calendar .fc .fc-button:focus { box-shadow: none; outline: none; }
269
270     /* Grid */
271     #employee-calendar .fc-theme-standard td,
272     #employee-calendar .fc-theme-standard th,
273     #employee-calendar .fc-theme-standard .fc-scrollgrid {
274         border-color: var(--border);
275     }
276     #employee-calendar .fc .fc-col-header-cell-cushion {
277         font-family: var(--font);
278         font-size: 11px;
279         font-weight: 600;
280         color: var(--text3);
281         text-transform: uppercase;
282         letter-spacing: 0.8px;
283         padding: 8px 4px;
284     }
285     #employee-calendar .fc .fc-daygrid-day-number {
286         font-family: var(--mono);
287         font-size: 12px;
288         color: var(--text3);
289         padding: 6px 8px;
290     }
291     #employee-calendar .fc .fc-daygrid-day.fc-day-today {
292         background: var(--accent-dim) !important;
293     }
294     #employee-calendar .fc .fc-daygrid-day.fc-day-today .fc-daygrid-day-number {
295         color: var(--accent);
296         font-weight: 700;
297     }
298     #employee-calendar .fc .fc-daygrid-day:hover {
299         background: rgba(56,189,248,0.03);
300     }
301     #employee-calendar .fc-daygrid-day-frame {
302         background: var(--bg3);
303     }
304     #employee-calendar .fc .fc-day-other .fc-daygrid-day-frame {
305         background: var(--bg2);
306     }
307
308     /* Events */
309     #employee-calendar .fc-event {
310         border: none !important;
311         border-radius: 4px;
312         font-family: var(--font);
313         font-size: 11px;
314         font-weight: 600;
315         padding: 2px 6px;
316         cursor: pointer;
317     }
318     #employee-calendar .fc-event-title { font-weight: 600; }
319
320     /* Event tooltip */
321     .cal-tooltip {
322         position: fixed;
323         background: var(--panel2);
324         border: 1px solid var(--border2);
325         border-radius: 8px;
326         padding: 10px 14px;
327         font-size: 12px;
328         color: var(--text);
329         z-index: 9999;
330         pointer-events: none;
331         max-width: 200px;
332         box-shadow: 0 8px 24px rgba(0,0,0,0.4);
333     }
334     .cal-tooltip .tt-title { font-weight: 600; margin-bottom: 4px; font-size: 13px; }
335     .cal-tooltip .tt-row { color: var(--text2); margin-top: 2px; }
336
337     /* balance loading state */

```

```
338         .lbc-count.loading { color: var(--text3); font-size: 16px; }
339     </style>
340 </head>
341 <body>
342     <div class="app">
343         <!-- SIDEBAR -->
344         <div class="sidebar">
345             <div class="logo">
346                 <div class="logo-mark">
347                     <div class="logo-icon">E</div>
348                     <h2>EMS Portal</h2>
349                 </div>
350                 <p>Employee Management</p>
351                 <div class="role-badge">Employee</div>
352             </div>
353             <nav class="nav">
354                 <div class="nav-section">Overview</div>
355                 <div class="nav-item active" onclick="showView('dashboard',this)"><span class="icon"></span>
Dashboard</div>
356                 <div class="nav-section">Leave</div>
357                 <div class="nav-item" onclick="showView('apply-leave',this)"><span class="icon"></span> Apply
Leave</div>
358                 <div class="nav-item" onclick="showView('my-leaves',this)"><span class="icon"></span> My Leave
History</div>
359                 <div class="nav-item" onclick="showView('my-calendar',this)"><span class="icon"></span> My
Calendar</div>
360                 <div class="nav-section">Meals</div>
361                 <div class="nav-item" onclick="showView('restaurants',this)"><span class="icon"></span>
Restaurants</div>
362                 <div class="nav-item" onclick="showView('subscriptions',this)"><span class="icon"></span> My
Subscriptions</div>
363                 <div class="nav-item" onclick="showView('add-subscription',this)"><span class="icon"></span>
Add Subscription</div>
364                 <div class="nav-section">Account</div>
365                 <div class="nav-item" onclick="showView('profile',this)"><span class="icon"></span> My
Profile</div>
366                 <div class="nav-item" onclick="showView('my-devices',this)"><span class="icon"></span> My
Devices</div>
367                 <div class="nav-item" onclick="showView('service-requests',this)"><span class="icon"></span>
Service Requests</div>
368                 <div class="nav-item" onclick="showView('two-factor',this)"><span class="icon"></span> Two-
Factor Auth</div>
369             </nav>
370             <div class="sidebar-footer">
371                 <div class="user-card">
372                     <div class="avatar" id="avatar-initials">?</div>
373                     <div class="user-info-side">
374                         <div class="name" id="sidebar-name">Loading...</div>
375                         <div class="email" id="sidebar-email">--</div>
376                     </div>
377                 </div>
378             </div>
379         </div>
380
381         <!-- MAIN -->
382         <div class="main">
383             <div class="topbar">
384                 <div class="page-title" id="pageTitle">Dashboard</div>
385                 <!-- Bell icon with badge -->
386                 <div class="notif-wrapper" onclick="toggleNotifPanel()">
387                     <svg width="22" height="22" viewBox="0 0 24 24" fill="none"
388                         stroke="currentColor" stroke-width="2" stroke-linecap="round">
389                         <path d="M18 8A6 6 0 0 0 6 8c0 7-3 9-3 9h18s-3-2-3-9"/>
390                         <path d="M13.73 21a2 2 0 0 1-3.46 0"/>
391                     </svg>
392                     <span id="notif-badge" class="notif-badge" style="display:none"></span>
393                 </div>
394
395                 <!-- Dropdown panel -->
396                 <div id="notif-panel" class="notif-panel" style="display:none">
397                     <div class="notif-header">Notifications</div>
398                     <div id="notif-list"></div>
399                 </div>
400                 <div class="topbar-right">
401                     <span style="font-size:12px;color:var(--text3);font-family:var(--mono)" id="topbar-
email"></span>
```

```
402         <button class="logout-btn" onclick="logout()">Sign Out</button>
403     </div>
404 </div>
405 <div class="content">
406
407     <!-- DASHBOARD -->
408     <div id="dashboard-view" class="view active">
409         <div class="stats-grid">
410             <div class="stat-card blue">
411                 <div class="stat-label">Leave Balance</div>
412                 <div class="stat-value" id="stat-balance">--</div>
413                 <div class="stat-sub">days remaining</div>
414             </div>
415             <div class="stat-card amber">
416                 <div class="stat-label">Pending Leaves</div>
417                 <div class="stat-value" id="stat-pending">--</div>
418                 <div class="stat-sub">awaiting approval</div>
419             </div>
420             <div class="stat-card green">
421                 <div class="stat-label">Approved Leaves</div>
422                 <div class="stat-value" id="stat-approved">--</div>
423                 <div class="stat-sub">this year</div>
424             </div>
425             <div class="stat-card red">
426                 <div class="stat-label">Rejected Leaves</div>
427                 <div class="stat-value" id="stat-rejected">--</div>
428                 <div class="stat-sub">this year</div>
429             </div>
430         </div>
431         <div class="section">
432             <div class="section-head">
433                 <div>
434                     <div class="section-title">Recent Leave Requests</div>
435                     <div class="section-sub">Your last 5 requests</div>
436                 </div>
437                 <button class="btn btn-secondary btn-sm" onclick="loadDashboard()">
Refresh</button>
438             </div>
439             <div class="tablewrap">
440                 <table>
441
442 <thead><tr><th>Type</th><th>From</th><th>To</th><th>Reason</th><th>Status</th></tr></thead>
442                 <tbody id="recent-leaves"><tr><td colspan="5" class="empty-
state">Loading...</td></tr></tbody>
443                 </table>
444             </div>
445         </div>
446         <div class="section">
447             <div class="section-head">
448                 <div class="section-title">Active Subscriptions</div>
449             </div>
450             <div id="dashboard-subs" class="subscription-card"></div>
451         </div>
452     </div>
453
454     <!-- APPLY LEAVE -->
455     <div id="apply-leave-view" class="view">
456         <div class="section">
457             <div class="section-head"><div class="section-title">Apply for Leave</div></div>
458             <div class="form-card">
459                 <div class="form-grid">
460                     <div class="form-group">
461                         <label>Leave Type</label>
462                         <select id="leave-type">
463                             <option value="SICK">Sick Leave</option>
464                             <option value="CASUAL">Casual Leave</option>
465                             <option value="UNPAID">Unpaid Leave</option>
466                         </select>
467                         <div id="sick-leave-hint" style="display:none;margin-top:6px;font-
size:11px;color:var(--amber);background:var(--amber-dim);border:1px solid rgba(251,191,36,0.2);bord...
468                             Sick leave is limited to <strong>1 day per month</strong>. If already
used this month, please select Unpaid Leave.
469                         </div>
470                         <div id="casual-leave-hint" style="display:none; margin-top:6px; font-
size:11px; color:var(--accent); background:var(--accent-dim); border:1px solid rgba(56,189,248,...
471                             You have <strong id="casual-balance-text">...</strong> casual leave
```

```

days available.
472                 </div>
473             </div>
474             <div class="form-group">
475                 <label>Start Date</label>
476                 <input type="date" id="leave-start">
477             </div>
478             <div class="form-group">
479                 <label>End Date</label>
480                 <input type="date" id="leave-end">
481             </div>
482             <div class="form-group full">
483                 <label>Reason</label>
484                 <textarea id="leave-reason" rows="3" placeholder="Please describe the
reason for your leave..."></textarea>
485             </div>
486         </div>
487         <div class="form-actions">
488             <button class="btn btn-primary" onclick="applyLeave()">Submit Request</button>
489             <button class="btn btn-secondary" onclick="clearLeaveForm()">Clear</button>
490         </div>
491     </div>
492 </div>
493 </div>
494
495 <!-- MY LEAVES -->
496 <div id="my-leaves-view" class="view">
497     <div class="section">
498         <div class="section-head">
499             <div><div class="section-title">My Leave History</div><div class="section-sub">All
your leave requests</div></div>
500             <button class="btn btn-secondary btn-sm" onclick="loadMyLeaves()"> Refresh</button>
501         </div>
502         <div class="tablewrap">
503             <table>
504
505 <thead><tr><th>Type</th><th>From</th><th>To</th><th>Days</th><th>Reason</th><th>Status</th><th>Action</th></tr><
/thead>
506             <tbody id="my-leaves-list"><tr><td colspan="7" class="empty-
state">Loading...</td></tr></tbody>
507         </div>
508     </div>
509 </div>
510
511 <!-- ===== -->
512 <!-- MY CALENDAR VIEW (NEW) -->
513 <!-- ===== -->
514 <div id="my-calendar-view" class="view">
515     <div class="section">
516         <div class="section-head">
517             <div>
518                 <div class="section-title">My Leave Calendar</div>
519                 <div class="section-sub">All approved leaves and public holidays</div>
520             </div>
521             <button class="btn btn-secondary btn-sm" onclick="refreshCalendar()">
Refresh</button>
522         </div>
523
524     <!-- Leave Balance Summary Cards -->
525     <div class="leave-balance-grid">
526         <!-- Sick Leave Card -->
527         <div class="leave-bal-card sick">
528             <span class="lbc-icon"></span>
529             <span class="lbc-label">Sick Leave</span>
530             <span class="lbc-count" id="cal-sick-remaining">--</span>
531             <span class="lbc-sub" id="cal-sick-sub">loading...</span>
532         </div>
533         <!-- Casual Leave Card -->
534         <div class="leave-bal-card casual">
535             <span class="lbc-icon"></span>
536             <span class="lbc-label">Casual Leave</span>
537             <span class="lbc-count" id="cal-casual-remaining">--</span>
538             <span class="lbc-sub" id="cal-casual-sub">loading...</span>
539         </div>
540         <!-- Maternity Leave Card -->

```

```

541         <div class="leave-bal-card mat">
542             <span class="lbc-icon"></span>
543             <span class="lbc-label">Maternity Leave</span>
544             <span class="lbc-count" id="cal-mat-remaining">--</span>
545             <span class="lbc-sub" id="cal-mat-sub">loading...</span>
546         </div>
547         <!-- Total Used Card -->
548         <div class="leave-bal-card total">
549             <span class="lbc-icon"></span>
550             <span class="lbc-label">Total Used</span>
551             <span class="lbc-count" id="cal-total-used">--</span>
552             <span class="lbc-sub" id="cal-total-sub">loading...</span>
553         </div>
554     </div>
555
556     <!-- Legend -->
557     <div class="cal-legend">
558         <span class="legend-label">Legend:</span>
559         <div class="legend-item">
560             <div class="legend-dot" style="background:#34d399"></div>
561             Approved Leave
562         </div>
563         <div class="legend-item">
564             <div class="legend-dot" style="background:#fbbf24"></div>
565             Pending Leave
566         </div>
567         <div class="legend-item">
568             <div class="legend-dot" style="background:#f87171"></div>
569             Rejected Leave
570         </div>
571         <div class="legend-item">
572             <div class="legend-dot" style="background:#38bdf8"></div>
573             Public Holiday
574         </div>
575     </div>
576
577     <!-- FullCalendar Mount Point -->
578     <div id="employee-calendar"></div>
579 </div>
580 </div>
581 <!-- END CALENDAR VIEW -->
582
583 <!-- RESTAURANTS -->
584 <div id="restaurants-view" class="view">
585     <div class="section">
586         <div class="section-head">
587             <div class="section-title">Available Restaurants</div>
588             <button class="btn btn-secondary btn-sm" onclick="loadRestaurants()">
Refresh</button>
589         </div>
590         <div class="tablewrap">
591             <table>
592                 <thead><tr><th>Name</th><th>Meal
Slot</th><th>Status</th><th>Action</th></tr></thead>
593                 <tbody id="restaurants-list"><tr><td colspan="4" class="empty-
state">Loading...</td></tr></tbody>
594             </table>
595         </div>
596     </div>
597 </div>
598
599 <!-- SUBSCRIPTIONS -->
600 <div id="subscriptions-view" class="view">
601     <div class="section">
602         <div class="section-head">
603             <div class="section-title">My Meal Subscriptions</div>
604             <button class="btn btn-secondary btn-sm" onclick="loadSubscriptions()">
Refresh</button>
605         </div>
606         <div class="tablewrap">
607             <table>
608                 <thead>
609                     <tr>
610                         <th>Restaurant</th><th>Meal Slot</th><th>Schedule</th>
611                         <th>Next Delivery</th><th>Status</th><th>Action</th>
612                     </tr>

```

```
613         </thead>
614         <tbody id="subscriptions-list"><tr><td colspan="6" class="empty-
state">Loading...</td></tr></tbody>
615     </table>
616 </div>
617 </div>
618 </div>
619
620 <!-- ADD SUBSCRIPTION -->
621 <div id="add-subscription-view" class="view">
622     <div class="section">
623         <div class="section-head"><div class="section-title">Subscribe to Meal Plan</div></div>
624         <div class="form-card">
625             <div class="form-grid">
626                 <div class="form-group">
627                     <label>Meal Slot</label>
628                     <select id="sub-meal-slot">
629                         <option value="BREAKFAST">Breakfast</option>
630                         <option value="LUNCH">Lunch</option>
631                         <option value="DINNER">Dinner</option>
632                     </select>
633                 </div>
634                 <div class="form-group">
635                     <label>Restaurant</label>
636                     <select id="sub-restaurant-id">
637                         <option value="">Select a restaurant</option>
638                     </select>
639                 </div>
640                 <div class="form-group">
641                     <label>Schedule Type</label>
642                     <select id="sub-schedule-type">
643                         <option value="DAILY">Daily</option>
644                         <option value="WEEKLY">Weekly</option>
645                     </select>
646                 </div>
647                 <div class="form-group" id="day-of-week-group" style="display:none">
648                     <label>Day of Week</label>
649                     <select id="sub-day-of-week">
650                         <option value="MONDAY">Monday</option>
651                         <option value="TUESDAY">Tuesday</option>
652                         <option value="WEDNESDAY">Wednesday</option>
653                         <option value="THURSDAY">Thursday</option>
654                         <option value="FRIDAY">Friday</option>
655                         <option value="SATURDAY">Saturday</option>
656                         <option value="SUNDAY">Sunday</option>
657                     </select>
658                 </div>
659             </div>
660             <div class="form-actions">
661                 <button class="btn btn-primary" onclick="addSubscription()">Subscribe</button>
662             </div>
663         </div>
664     </div>
665 </div>
666
667 <!-- MY DEVICES -->
668 <div id="my-devices-view" class="view">
669     <div class="section">
670         <div class="section-head">
671             <div>
672                 <div class="section-title">My Assigned Devices</div>
673                 <div class="section-sub">Devices currently assigned to you</div>
674             </div>
675             <button class="btn btn-secondary btn-sm" onclick="loadMyAssignedDevices()">
Refresh</button>
676         </div>
677         <div class="tablewrap">
678             <table>
679                 <thead>
680                     <tr>
681                         <th>ID</th><th>Device Name</th><th>Brand</th>
682                         <th>Type</th><th>Warranty</th><th>Assigned On</th><th>Status</th>
683                     </tr>
684                 </thead>
685                 <tbody id="my-devices-list"></tbody>
686             </table>
```



```

687         </div>
688     </div>
689 </div>
690
691 <!-- SERVICE REQUESTS -->
692 <div id="service-requests-view" class="view">
693     <div class="section" style="margin-bottom:20px">
694         <div class="section-head">
695             <div>
696                 <div class="section-title">My Service Requests</div>
697                 <div class="section-sub">All requests you have raised</div>
698             </div>
699             <button class="btn btn-secondary btn-sm" onclick="loadServiceRequestsList()">
Refresh</button>
700         </div>
701         <div class="tablewrap">
702             <table>
703                 <thead>
704                     <tr>
705                         <th>ID</th><th>Device</th><th>Type</th>
706                         <th>Description</th><th>Urgent</th><th>Status</th><th>Raised At</th>
707                     </tr>
708                 </thead>
709                 <tbody id="service-requests-list">
710                     <tr><td colspan="7" class="empty-state">Loading...</td></tr>
711                 </tbody>
712             </table>
713         </div>
714     </div>
715     <div class="section">
716         <div class="section-head"><div class="section-title">Raise a New Request</div></div>
717         <div class="form-card">
718             <div class="form-grid">
719                 <div class="form-group full">
720                     <label>Device</label>
721                     <select id="sr-device-select">
722                         <option value="">-- Select your device --</option>
723                     </select>
724                 </div>
725                 <div class="form-group">
726                     <label>Request Type</label>
727                     <select id="sr-type">
728                         <option value="REPAIR">Repair</option>
729                         <option value="REPLACEMENT">Replacement</option>
730                         <option value="UPGRADE">Upgrade</option>
731                         <option value="MAINTENANCE">Maintenance</option>
732                         <option value="RETURN">Return</option>
733                     </select>
734                 </div>
735                 <div class="form-group" style="display:flex;align-
items:center;gap:10px;padding-top:22px">
736                     <input type="checkbox" id="sr-urgent"
style="width:16px;height:16px;padding:0;flex-shrink:0">
737                     <label for="sr-urgent" style="text-transform:none;font-size:13px;font-
weight:500;color:var(--text);cursor:pointer;letter-spacing:0">Mark as Urgent</label>
738                 </div>
739                 <div class="form-group full">
740                     <label>Issue Description</label>
741                     <textarea id="sr-description" rows="3" placeholder="Describe the issue in
detail..."></textarea>
742                 </div>
743             </div>
744             <div class="form-actions">
745                 <button class="btn btn-primary" onclick="submitServiceRequest()">Submit
Request</button>
746                 <button class="btn btn-secondary"
onclick="clearServiceRequestForm()">Clear</button>
747             </div>
748         </div>
749     </div>
750 </div>
751 <!-- TWO-FACTOR AUTH -->
752 <div id="two-factor-view" class="view">
753     <div class="section">
754         <div class="section-head">
755             <div>

```

```

756             <div class="section-title"> Two-Factor Authentication</div>
757             <div class="section-sub">Protect your account with an authenticator app</div>
758         </div>
759         <span id="totp-status-chip" class="chip">Checking...</span>
760     </div>
761
762     <div style="padding:24px;max-width:480px">
763
764         <!-- DISABLED state – setup flow -->
765         <div id="totp-setup-section">
766             <p style="font-size:13px;color:var(--text2);margin-bottom:16px;line-
height:1.6">
767                 Use Google Authenticator, Authy, or any TOTP app.
768                 Scan the QR code below, then enter the 6-digit code to activate.
769             </p>
770             <button class="btn btn-primary" onclick="generateTotpQr()" id="generate-qr-
btn">
771                 Generate QR Code
772             </button>
773
774             <div id="qr-area" style="display:none;margin-top:24px;text-align:center">
775                 <img id="qr-img" src="" alt="QR Code"
776                     style="width:180px;height:180px;border-radius:10px;
777                     border:1px solid var(--border2);display:block;margin:0 auto">
778                 <div style="margin-top:12px;font-size:11px;color:var(--text3)">
779                     Can't scan? Enter this key manually:
780                 </div>
781                 <div id="totp-secret-display"
782                     style="font-family:var(--mono);font-size:13px;color:var(--accent);
783                     background:var(--bg3);border:1px solid var(--border2);
784                     border-radius:6px;padding:8px 12px;margin-top:6px;
785                     word-break:break-all"></div>
786
787                 <div style="margin-top:20px;text-align:left">
788                     <label style="font-size:11px;color:var(--text3);font-weight:600;
789                     text-transform:uppercase;letter-spacing:0.8px;
790                     display:block;margin-bottom:8px">
791                         Enter 6-digit code to activate
792                     </label>
793                     <input id="totp-verify-input" type="text" inputmode="numeric"
794                         maxlength="6" placeholder="000000"
795                         style="text-align:center;font-size:20px;
796                         font-family:var(--mono);letter-spacing:8px;padding:12px"
797                         oninput="this.value=this.value.replace(/\D/g, '')">
798                     <button class="btn btn-primary"
799                         onclick="verifyAndActivateTotp()"
800                         style="width:100%;margin-top:10px">
801                         Activate 2FA
802                     </button>
803                     <div id="totp-verify-error"
804                         style="display:none;margin-top:8px;font-size:12px;
805                         color:var(--red)"></div>
806                 </div>
807             </div>
808         </div>
809
810         <!-- ENABLED state -->
811         <div id="totp-disable-section" style="display:none">
812             <div style="background:var(--green-dim);border:1px solid rgba(52,211,153,0.2);
813             border-radius:8px;padding:12px 16px;font-size:13px;
814             color:var(--green);margin-bottom:16px">
815                 Two-factor authentication is active on your account.
816             </div>
817             <button class="btn btn-danger" onclick="disableTotp()">
818                 Disable 2FA
819             </button>
820         </div>
821     </div>
822 </div>
823 </div>
824 <!-- PROFILE -->
825 <div id="profile-view" class="view">
826     <div class="section">
827         <div class="section-head"><div class="section-title">My Profile</div></div>
828         <div class="profile-grid" id="profile-data">
829             <div class="profile-field"><div class="field-label">Name</div><div class="field-

```

```

value" id="p-name"></div></div>
830         <div class="profile-field"><div class="field-label">Email</div><div class="field-
value" id="p-email"></div></div>
831         <div class="profile-field"><div class="field-label">Department</div><div
class="field-value" id="p-dept"></div></div>
832         <div class="profile-field"><div class="field-label">Role</div><div class="field-
value" id="p-role"></div></div>
833         <div class="profile-field"><div class="field-label">Status</div><div class="field-
value" id="p-status"></div></div>
834         <div class="profile-field"><div class="field-label">Timezone</div><div
class="field-value" id="p-tz"></div></div>
835     </div>
836 </div>
837
838     <div class="section" id="maternity-apply-btn" style="display:none">
839         <div class="section-head">
840             <div>
841                 <div class="section-title"> Maternity Leave</div>
842                 <div class="section-sub">Apply for 180 days maternity leave</div>
843             </div>
844         </div>
845         <div class="form-card">
846             <div class="form-grid">
847                 <div class="form-group">
848                     <label>Start Date</label>
849                     <input type="date" id="mat-start-date">
850                 </div>
851                 <div class="form-group">
852                     <label>End Date (Auto)</label>
853                     <input type="text" id="mat-end-date"
854                         placeholder="Auto: start + 180 days"
855                         disabled
856                         style="color:var(--text3);cursor:not-allowed">
857                 </div>
858                 <div class="form-group full">
859                     <label>Reason / Notes</label>
860                     <textarea id="mat-reason" rows="3"
861                         placeholder="Optional notes..."></textarea>
862                 </div>
863                 <!-- Info box -->
864                 <div class="form-group full">
865                     <div style="background:var(--accent-dim);border:1px solid
rgba(56,189,248,0.2);
866                         border-radius:8px;padding:12px 16px;font-size:12px;color:var(--text2)">
867                         Maternity leave is fixed at <strong style="color:var(--accent)">180
days</strong>.
868                         The end date will be automatically calculated.
869                         You can apply <strong style="color:var(--accent)">once per
year</strong>.
870                     </div>
871                 </div>
872             </div>
873             <div class="form-actions">
874                 <button class="btn btn-primary" onclick="applyMaternityLeave()">
875                     Submit Maternity Leave
876                 </button>
877             </div>
878         </div>
879     </div>
880
881 </div>
882
883 </div><!-- /content -->
884 </div><!-- /main -->
885 </div><!-- /app -->
886
887 <!-- Tooltip for calendar events -->
888 <div class="cal-tooltip" id="cal-tooltip" style="display:none"></div>
889
890 <script>
891     const API = 'http://localhost:8080';
892     let availableRestaurants = [];
893
894     /* =====
895     CALENDAR GLOBALS
896     ===== */

```

```

897     let employeeCalendar = null;    // FullCalendar instance
898     let calendarInitialized = false;
899     let currentEmployeeGender = null;
900     let currentEmployeeId = null;
901
902     // Event color map
903     const STATUS_COLORS = {
904         APPROVED : { bg:'rgba(52,211,153,0.25)', border:'#34d399', text:'#34d399' },
905         PENDING  : { bg:'rgba(251,191,36,0.25)',  border:'#fbbf24', text:'#fbbf24' },
906         REJECTED : { bg:'rgba(248,113,113,0.25)', border:'#f87171', text:'#f87171' },
907         HOLIDAY  : { bg:'rgba(56,189,248,0.25)', border:'#38bdf8', text:'#38bdf8' },
908     };
909
910     const TYPE_LABEL = { SICK:' Sick', CASUAL:' Casual', MATERNITY:' Maternity', UNPAID:' Unpaid'
911     };
912
913     /* =====
914     AUTH / INIT
915     ===== */
916     function extractRoleFromToken(token) {
917         try {
918             const payload = JSON.parse(atob(token.split('.')[1]));
919             return payload.role || payload.roles || payload.authorities || '';
920         } catch(e) { return ''; }
921     }
922
923     (function(){
924         const token = localStorage.getItem('jwt_token');
925         const authType = localStorage.getItem('auth_type');
926         if(!token && authType !== 'session') { window.location.href = '/google.html'; return; }
927         if(token) {
928             try {
929                 const roles = String(extractRoleFromToken(token));
930                 if(roles.includes('ROLE_MANAGER')) { window.location.href='/manager-dashboard.html';
return; }
931                 if(roles.includes('ROLE_VENDOR')) { window.location.href='/vendor-dashboard.html';
return; }
932                 if(roles.includes('ROLE_ADMIN') || roles.includes('ADMIN')) {
window.location.href='/dashboard.html'; return; }
933             } catch(e) { console.warn('Could not decode JWT for role check'); }
934         }
935         initUser();
936     })();
937
938     function getAuthHeader(){
939         const token = localStorage.getItem('jwt_token');
940         return token ? {'Authorization':`Bearer ${token}`} : {};
941     }
942
943     async function apiFetch(path, opts={}){
944         const token = localStorage.getItem('jwt_token');
945         const authType = localStorage.getItem('auth_type');
946         const res = await fetch(`${API}${path}`, {
947             ...opts,
948             headers: {'Content-Type': 'application/json', ...getAuthHeader(), ...(opts.headers||{})},
949             credentials: (!token && authType==='session') ? 'include' : 'omit'
950         });
951         if(res.status===401){
952             localStorage.removeItem('jwt_token'); localStorage.removeItem('auth_type');
953             showAlert('Session expired. Please login again.','error');
954             setTimeout(()=>window.location.href='/google.html',1500);
955             throw new Error('Unauthorized');
956         }
957         if(res.status===403) throw new Error('ACCESS_DENIED');
958         if(!res.ok){
959             const errText = await res.text();
960             throw new Error(errText || 'Request failed');
961         }
962         const text = await res.text();
963         try { return text ? JSON.parse(text) : null; }
964         catch { return text; }
965     }
966
967     function asArray(value){ return Array.isArray(value) ? value : []; }
968
969     function daysBetween(start,end){

```

```

970         if(!start||!end) return '-';
971         const d=(new Date(end)-new Date(start))/86400000+1;
972         return d>0?d:'-';
973     }
974
975     async function initUser(){
976         try {
977             let email='', name='';
978             const token = localStorage.getItem('jwt_token');
979             if(token){
980                 try{ const p=JSON.parse(atob(token.split('.')[1])); email=p.sub||p.email||''; }catch(e){}
981             }
982             try{
983                 const me = await apiFetch('/employee/me');
984                 if(me){
985                     name=me.name||email; email=me.email||email;
986                     currentEmployeeGender = me.gender; // store gender
987                     currentEmployeeId = me.employeeId;
988                     document.getElementById('p-name').textContent=me.name||'-';
989                     document.getElementById('p-email').textContent=me.email||'-';
990                     document.getElementById('p-dept').textContent=me.dept||'-';
991                     document.getElementById('p-role').textContent=me.role||'EMPLOYEE';
992                     document.getElementById('p-status').textContent=me.status||'-';
993                     document.getElementById('p-tz').textContent=me.timezone||'-';
994
995                     const maternityBtn = document.getElementById('maternity-apply-btn');
996                     if(maternityBtn) {
997                         maternityBtn.style.display = me.gender === 'F' ? 'block' : 'none';
998                     }
999                 }
1000             }catch(e){ console.warn('Could not load profile'); }
1001             const initials=(name||email||'?').charAt(0).toUpperCase();
1002             document.getElementById('avatar-initials').textContent=initials;
1003             document.getElementById('sidebar-name').textContent=name||email||'Employee';
1004             document.getElementById('sidebar-email').textContent=email;
1005             document.getElementById('topbar-email').textContent=email;
1006         }catch(e){ console.error(e); }
1007         loadDashboard();
1008     }
1009
1010     /* =====
1011     VIEW SWITCHING
1012     ===== */
1013     function showView(name, el){
1014         document.querySelectorAll('.view').forEach(v=>v.classList.remove('active'));
1015         document.getElementById(`${name}-view`).classList.add('active');
1016         document.querySelectorAll('.nav-item').forEach(n=>n.classList.remove('active'));
1017         if(el) el.classList.add('active');
1018         document.getElementById('pageTitle').textContent=
1019             name.split('-').map(w=>w.charAt(0).toUpperCase()+w.slice(1)).join(' ');
1020
1021         const loaders={
1022             dashboard: loadDashboard,
1023             'my-leaves': loadMyLeaves,
1024             'apply-leave': ()=>{},
1025             'my-calendar': loadMyCalendar,
1026             restaurants: loadRestaurants,
1027             subscriptions: loadSubscriptions,
1028             'add-subscription': ()=>{},
1029             profile: ()=>{},
1030             'my-devices': loadMyAssignedDevices,
1031             'service-requests': loadServiceRequests,
1032             'two-factor': loadTotpStatus
1033         };
1034         if(loaders[name]) loaders[name]();
1035         if(name==='add-subscription') loadRestaurantChoices();
1036     }
1037
1038     /* =====
1039     DASHBOARD
1040     ===== */
1041     async function loadDashboard(){
1042         const [leavesResult, dashboardResult] = await Promise.allSettled([
1043             apiFetch('/leave_requests/employee'),
1044             apiFetch('/api/employees/me/leave-dashboard')
1045         ]);

```

```

1046     const leavesPayload = leavesResult.status==='fulfilled' ? leavesResult.value : null;
1047     const dashboard      = dashboardResult.status==='fulfilled' ? dashboardResult.value : null;
1048     const arr = asArray(leavesPayload?.requests || leavesPayload);
1049     const pending  = arr.filter(l=>l.status==='PENDING').length;
1050     const approved = arr.filter(l=>l.status==='APPROVED').length;
1051     const rejected = arr.filter(l=>l.status==='REJECTED').length;
1052     document.getElementById('stat-pending').textContent=pending;
1053     document.getElementById('stat-approved').textContent=approved;
1054     document.getElementById('stat-rejected').textContent=rejected;
1055     const balances  = dashboard?.leaveBalances ? Object.values(dashboard.leaveBalances) : [];
1056     const available = balances.reduce((t,b)=>t+Number(b.available||0),0);
1057     document.getElementById('stat-balance').textContent=available;
1058     const recent=arr.slice(-5).reverse();
1059     document.getElementById('recent-leaves').innerHTML=recent.length
1060       ? recent.map(l=><tr>
1061         <td><span class="chip
1062   ${l.leaveType||''}.toLowerCase()">${l.leaveType||'N/A'}</span></td>
1063   <td>${l.startDate||'-'}</td><td>${l.endDate||'-'}</td>
1064   <td>${(l.reason||'-').substring(0,30)}</td>
1065   <td><span class="chip ${l.status||''}.toLowerCase()">${l.status||'-'}</span></td>
1066   </tr>`).join('')
1067   : leavesResult.status==='rejected'
1068   ? '<tr><td colspan="5" class="empty-state">Could not load leave data</td></tr>'
1069   : '<tr><td colspan="5" class="empty-state">No leave requests yet</td></tr>';
1070
1071   try{
1072     const subs = await apiFetch('/subscription/user');
1073     const arr2 = subs || [];
1074     document.getElementById('dashboard-subs').innerHTML = arr2.length
1075       ? arr2.filter(s=>s.status==='ACTIVE').slice(0,4).map(s=>`
1076         <div class="sub-item">
1077           <div class="meal-name">${s.restaurantName||'Restaurant'}</div>
1078           <div class="meal-meta">${s.mealSlot||''} · ${s.scheduleType||''}${s.dayOfWeek?' '
1079             '+s.dayOfWeek:''}</div>
1080           <div class="meal-meta" style="margin-top:4px;color:var(--accent)">Next:
1081             ${s.nextDeliveryTime?new Date(s.nextDeliveryTime).toLocaleString():'N/A'}</div>
1082         </div>`).join('')
1083       : '<div style="padding:16px;color:var(--text3);font-style:italic">No active
1084         subscriptions</div>';
1085     }catch(e){
1086       document.getElementById('dashboard-subs').innerHTML='<div style="padding:16px;color:var(--
1087         text3)">Could not load subscriptions</div>';
1088     }
1089   }
1090
1091   /* =====
1092   MY LEAVES
1093   ===== */
1094   async function loadMyLeaves(){
1095     try{
1096       const data=await apiFetch('/leave_requests/employee');
1097       const leaves=data?.requests||data||[];
1098       document.getElementById('my-leaves-list').innerHTML=leaves.length
1099       ? leaves.map(l=><tr>
1100         <td><span class="chip
1101   ${l.leaveType||''}.toLowerCase()">${l.leaveType||'N/A'}</span></td>
1102   <td>${l.startDate||'-'}</td><td>${l.endDate||'-'}</td>
1103   <td>${l.numberOfDays||daysBetween(l.startDate,l.endDate)}</td>
1104   <td>${(l.reason||'-').substring(0,30)}</td>
1105   <td><span class="chip ${l.status||''}.toLowerCase()">${l.status}</span></td>
1106   <td>${l.status==='PENDING'?`<button class="btn btn-danger btn-sm"
1107     onclick="cancelLeave(${l.leaveRequestId})">Cancel</button>`:'-'}</td>
1108   </tr>`).join('')
1109       : '<tr><td colspan="7" class="empty-state">No leave requests found</td></tr>';
1110     }catch(e){
1111       document.getElementById('my-leaves-list').innerHTML='<tr><td colspan="7" class="empty-
1112         state">Could not load leave requests</td></tr>';
1113     }
1114   }
1115
1116   async function applyLeave(){
1117     const leaveType = document.getElementById('leave-type').value;
1118     const startDate = document.getElementById('leave-start').value;
1119     const endDate   = document.getElementById('leave-end').value;
1120     const reason     = document.getElementById('leave-reason').value.trim();
1121
1122     if(!startDate || !endDate){ showAlert('Please fill all required fields','error'); return; }

```

```

1114
1115     try{
1116         const data = await apiFetch('/leave_requests', {
1117             method: 'POST',
1118             body: JSON.stringify({ leaveType, startDate, endDate, reason })
1119         });
1120         if(data && data.warning){
1121             showAlert(data.warning, 'warning');
1122         } else {
1123             showAlert('Leave request submitted successfully!', 'success');
1124         }
1125         clearLeaveForm();
1126         loadDashboard();
1127         loadMyLeaves();
1128     } catch(e) {
1129         // Try to parse the error message as JSON to extract the "error" field
1130         let msg = e.message;
1131         try {
1132             const parsed = JSON.parse(msg);
1133             if(parsed.error) msg = parsed.error;
1134         } catch(_) { /* not JSON, use raw message */ }
1135         showAlert(msg, 'error');
1136     }
1137 }
1138
1139 document.getElementById('leave-type').addEventListener('change', async function() {
1140     document.getElementById('sick-leave-hint').style.display =
1141         this.value === 'SICK' ? 'block' : 'none';
1142
1143     const casualHint = document.getElementById('casual-leave-hint');
1144     if(this.value === 'CASUAL') {
1145         casualHint.style.display = 'block';
1146         document.getElementById('casual-balance-text').textContent = 'loading...';
1147         try {
1148             const dashboard = await apiFetch('/api/employees/me/leave-dashboard');
1149             const balances = dashboard?.leaveBalances || {};
1150             const cas = balances['CASUAL'] || balances['casual'] || {};
1151             const available = Number(cas.available ?? cas.availableBalance ?? 0);
1152             document.getElementById('casual-balance-text').textContent =
1153                 available > 0 ? `${available}` : '0 (no balance remaining)';
1154         } catch(e) {
1155             document.getElementById('casual-balance-text').textContent = 'unavailable';
1156         }
1157     } else {
1158         casualHint.style.display = 'none';
1159     }
1160 });
1161
1162 async function cancelLeave(id){
1163     if(!confirm('Cancel this leave request?')) return;
1164     try{
1165         await apiFetch(`/leave_requests/cancel/${id}`, {method: 'PUT'});
1166         showAlert('Request cancelled successfully', 'success');
1167         loadMyLeaves();
1168     }catch(e){
1169         if(e.message==='ACCESS_DENIED') showAlert('You do not have permission to cancel this
leave', 'error');
1170         else showAlert('Failed to cancel: ' + e.message, 'error');
1171     }
1172 }
1173
1174 function clearLeaveForm(){
1175     ['leave-start', 'leave-end', 'leave-reason'].forEach(id=>document.getElementById(id).value='');
1176 }
1177
1178 /* =====
1179 MY CALENDAR (NEW)
1180 ===== */
1181 async function loadMyCalendar(){
1182     // 1. Load balance cards
1183     await loadCalendarBalanceCards();
1184     // 2. Build or refresh FullCalendar
1185     if(!calendarInitialized){
1186         initFullCalendar();
1187     } else {
1188         await refreshCalendarEvents();

```

```

1189     }
1190 }
1191
1192 async function loadCalendarBalanceCards(){
1193     // Default state
1194     ['cal-sick-remaining', 'cal-casual-remaining', 'cal-mat-remaining', 'cal-total-used']
1195     .forEach(id=>{ document.getElementById(id).textContent='--'; });
1196
1197     try{
1198         const dashboard = await apiFetch('/api/employees/me/leave-dashboard');
1199         const balances = dashboard?.leaveBalances || {};
1200
1201         let totalUsed=0, totalAllowed=0;
1202
1203         // SICK
1204         const sick = balances['SICK'] || balances['sick'] || {};
1205         const sickRem = Number(sick.available || 0);
1206         const sickTotal = Number(sick.total || 0);
1207         const sickUsed = sickTotal - sickRem;
1208         document.getElementById('cal-sick-remaining').textContent = sickRem;
1209         document.getElementById('cal-sick-sub').textContent = `${sickUsed} used of ${sickTotal}
days`;
1210
1211         // CASUAL
1212         const cas = balances['CASUAL'] || balances['casual'] || {};
1213         const casRem = Number(cas.available || 0);
1214         const casTotal = Number(cas.total || 0);
1215         const casUsed = casTotal - casRem;
1216         document.getElementById('cal-casual-remaining').textContent = casRem;
1217         document.getElementById('cal-casual-sub').textContent = `${casUsed} used of ${casTotal}
days`;
1218
1219         // MATERNITY
1220         const mat = balances['MATERNITY'] || balances['maternity'] || {};
1221         const matRem = Number(mat.available || 0);
1222         const matTotal = Number(mat.total || 0);
1223         const matUsed = matTotal - matRem;
1224         document.getElementById('cal-mat-remaining').textContent = matRem;
1225         document.getElementById('cal-mat-sub').textContent = `${matUsed} used of ${matTotal}
days`;
1226
1227         // TOTAL
1228         Object.values(balances).forEach(b=>{
1229             totalUsed += (Number(b.total||0) - Number(b.available||0));
1230             totalAllowed += Number(b.total||0);
1231         });
1232         document.getElementById('cal-total-used').textContent = totalUsed;
1233         document.getElementById('cal-total-sub').textContent = `out of ${totalAllowed} days total`;
1234
1235     }catch(e){
1236         console.error('Could not load leave balance for calendar:', e);
1237         ['cal-sick-sub', 'cal-casual-sub', 'cal-mat-sub', 'cal-total-sub']
1238         .forEach(id=>{ document.getElementById(id).textContent='unavailable'; });
1239     }
1240 }
1241
1242 async function buildCalendarEvents(){
1243     const events = [];
1244     try{
1245         // - Employee leaves -
1246         const data = await apiFetch('/leave_requests/employee');
1247         const leaves = data?.requests || data || [];
1248         leaves.forEach(l=>{
1249             const s = (l.status||'PENDING').toUpperCase();
1250             const c = STATUS_COLORS[s] || STATUS_COLORS.PENDING;
1251             const typeLabel = TYPE_LABEL[l.leaveType] || l.leaveType || 'Leave';
1252
1253             // FullCalendar end date is exclusive, so add 1 day for multi-day events
1254             let endDate = l.endDate;
1255             if(endDate){
1256                 const d = new Date(endDate);
1257                 d.setDate(d.getDate()+1);
1258                 endDate = d.toISOString().split('T')[0];
1259             }
1260
1261             events.push({

```



```

1262         id      : `leave-${l.leaveRequestId}`,
1263         title   : `${typeLabel} (${s.charAt(0)+s.slice(1).toLowerCase()}`,
1264         start   : l.startDate,
1265         end     : endDate,
1266         allDay  : true,
1267         backgroundColor : c.bg,
1268         borderColor : c.border,
1269         textColor : c.text,
1270         extendedProps : {
1271             type : 'leave',
1272             status : s,
1273             reason : l.reason || '-',
1274             leaveType: l.leaveType
1275         }
1276     });
1277 });
1278 }catch(e){ console.warn('Could not load leaves for calendar:', e); }
1279
1280     try{
1281         // - Public holidays (admin-added) -
1282         const holidays = await apiFetch('/holidays');
1283         (holidays||[]).forEach(h=>{
1284             const c = STATUS_COLORS.HOLIDAY;
1285             events.push({
1286                 id      : `holiday-${h.id||h.holidayId||Math.random()}`,
1287                 title   : `${h.name||h.holidayName||'Holiday'}`,
1288                 start   : h.date||h.holidayDate,
1289                 allDay  : true,
1290                 backgroundColor : c.bg,
1291                 borderColor : c.border,
1292                 textColor : c.text,
1293                 extendedProps : { type:'holiday', description: h.description||'' }
1294             });
1295         });
1296     }catch(e){ console.warn('Could not load holidays for calendar:', e); }
1297
1298     return events;
1299 }
1300
1301 function initFullCalendar(){
1302     const calendarEl = document.getElementById('employee-calendar');
1303
1304     employeeCalendar = new FullCalendar.Calendar(calendarEl, {
1305         initialView: 'dayGridMonth',
1306         height : 'auto',
1307         headerToolbar:{
1308             left : 'prev,next today',
1309             center: 'title',
1310             right : 'dayGridMonth,dayGridWeek,listMonth'
1311         },
1312         buttonText:{
1313             today : 'Today',
1314             month : 'Month',
1315             week : 'Week',
1316             listMonth: 'List'
1317         },
1318         dayMaxEvents: 3, // "+N more" link if overflowing
1319         navLinks : true, // click day number day view
1320         events : [], // start empty, populate below
1321
1322         // Event hover tooltip
1323         eventMouseEnter: function(info){
1324             const ep = info.event.extendedProps;
1325             const tooltip = document.getElementById('cal-tooltip');
1326             let html = `<div class="tt-title">${info.event.title}</div>`;
1327             if(ep.type==='leave'){
1328                 html += `<div class="tt-row"> ${info.event.startStr}
1329 ${info.event.endStr||info.event.startStr}</div>`;
1330                 if(ep.reason && ep.reason!=='-'){
1331                     html += `<div class="tt-row"> ${ep.reason.substring(0,60)}</div>`;
1332                     html += `<div class="tt-row">Status: <strong
1333 style="color:${STATUS_COLORS[ep.status]?.text||'#fff'}">${ep.status}</strong></div>`;
1334                 } else {
1335                     if(ep.description) html += `<div class="tt-row">${ep.description}</div>`;
1336                 }
1337             }
1338             tooltip.innerHTML = html;

```

```

1336         tooltip.style.display = 'block';
1337     },
1338     eventMouseLeave: function(){
1339         document.getElementById('cal-tooltip').style.display='none';
1340     },
1341     eventClick: function(info){
1342         // Click navigates to leave history
1343         const ep = info.event.extendedProps;
1344         if(ep.type==='leave'){
1345             showView('my-leaves', document.querySelector('.nav-item:nth-child(4)'));
1346         }
1347     },
1348 });
1349
1350 employeeCalendar.render();
1351 calendarInitialized = true;
1352
1353 // Load events asynchronously after render
1354 refreshCalendarEvents();
1355 }
1356
1357 async function refreshCalendarEvents(){
1358     if(!employeeCalendar) return;
1359     const events = await buildCalendarEvents();
1360     // Remove all existing events and re-add
1361     employeeCalendar.removeAllEvents();
1362     events.forEach(e=>employeeCalendar.addEvent(e));
1363 }
1364
1365 async function refreshCalendar(){
1366     await loadCalendarBalanceCards();
1367     await refreshCalendarEvents();
1368     showAlert('Calendar refreshed!', 'success');
1369 }
1370
1371 // Move tooltip with mouse
1372 document.addEventListener('mousemove', function(e){
1373     const tt = document.getElementById('cal-tooltip');
1374     if(tt.style.display==='block'){
1375         tt.style.left = (e.clientX+14)+'px';
1376         tt.style.top = (e.clientY+14)+'px';
1377         // Keep tooltip on screen
1378         const rect = tt.getBoundingClientRect();
1379         if(rect.right > window.innerWidth)
1380             tt.style.left = (e.clientX - rect.width - 10)+'px';
1381         if(rect.bottom > window.innerHeight)
1382             tt.style.top = (e.clientY - rect.height - 10)+'px';
1383     }
1384 });
1385
1386 /* =====
1387 RESTAURANTS
1388 ===== */
1389 async function loadRestaurants(){
1390     try{
1391         const data=await apiFetch('/restaurants');
1392         availableRestaurants=data||[];
1393         document.getElementById('restaurants-list').innerHTML=data?.length
1394             ? data.map(r=>`<tr>
1395                 <td>${r.name||'-'}</td>
1396                 <td>${r.supportedMealsSlots?.join(', ')}||'-'}</td>
1397                 <td><span class="chip
1398 ${r.active?'active':'rejected'}>${r.active?'Active':'Inactive'}</span></td>
1399                 <td><button class="btn btn-secondary btn-sm"
1400 onClick='openSubscriptionForRestaurant(${r.restaurantId||r.id},${JSON.stringify(r.supportedMealsSlots||[])}>
1401                 Subscribe</button></td>
1402                 </tr>`).join('')
1403             : '<tr><td colspan="4" class="empty-state">No restaurants available</td></tr>';
1404     }catch(e){
1405         document.getElementById('restaurants-list').innerHTML='<tr><td colspan="4" class="empty-
1406 state">Could not load restaurants</td></tr>';
1407     }
1408 }
1409
1410 async function loadRestaurantChoices(){

```

```

1409     const select=document.getElementById('sub-restaurant-id');
1410     try{
1411         const data=await apiFetch('/restaurants');
1412         availableRestaurants=data||[];
1413         select.innerHTML=availableRestaurants.length
1414             ? '<option value="">Select a restaurant</option>'+
1415             availableRestaurants.map(r=>`<option
value="${r.restaurantId||r.id}">${r.name}</option>`).join('')
1416             : '<option value="">No restaurants available</option>';
1417     }catch(e){ select.innerHTML='<option value="">Could not load restaurants</option>'; }
1418     }
1419
1420     function openSubscriptionForRestaurant(restaurantId, supportedMealSlots){
1421         showView('add-subscription',null);
1422         if(supportedMealSlots?.length) document.getElementById('sub-meal-
slot').value=supportedMealSlots[0];
1423         loadRestaurantChoices().then(()=>{ document.getElementById('sub-restaurant-
id').value=String(restaurantId); });
1424     }
1425
1426     /* =====
1427     SUBSCRIPTIONS
1428     ===== */
1429     async function loadSubscriptions(){
1430         try{
1431             const data=await apiFetch('/subscription/user');
1432             const arr=data||[];
1433             document.getElementById('subscriptions-list').innerHTML=arr.length
1434                 ? arr.map(s=>`<tr>
1435                 <td>${s.restaurantName||s.restaurantId||'-'}</td>
1436                 <td>${s.mealSlot||'-'}</td>
1437                 <td>${s.scheduleType||'-'}${s.dayOfWeek?' . '+s.dayOfWeek:'}</td>
1438                 <td>${s.nextDeliveryTime?new Date(s.nextDeliveryTime).toLocaleString():'-'}</td>
1439                 <td><span class="chip
${(s.status||'').toLowerCase()}>${s.status||'ACTIVE'}</span></td>
1440                 <td><button class="btn btn-danger btn-sm"
onclick="cancelSub(${s.subscriptionId})">Cancel</button></td>
1441                 </tr>`).join('')
1442                 : '<tr><td colspan="6" class="empty-state">No subscriptions found</td></tr>';
1443             }catch(e){
1444                 document.getElementById('subscriptions-list').innerHTML='<tr><td colspan="6" class="empty-
state">Could not load subscriptions</td></tr>';
1445             }
1446         }
1447
1448         document.getElementById('sub-schedule-type').addEventListener('change',function(){
1449             document.getElementById('day-of-week-group').style.display=this.value==='WEEKLY'? 'block': 'none';
1450         });
1451         document.getElementById('sub-meal-slot').addEventListener('change', loadRestaurantChoices);
1452
1453         async function addSubscription(){
1454             const restaurantId=parseInt(document.getElementById('sub-restaurant-id').value);
1455             const mealSlot =document.getElementById('sub-meal-slot').value;
1456             const scheduleType=document.getElementById('sub-schedule-type').value;
1457             const dayOfWeek =document.getElementById('sub-day-of-week')?.value;
1458             if(!restaurantId||!mealSlot){showAlert('Please fill all fields','error');return;}
1459             const body={scheduleType,mealSlots:[mealSlot],slotOrders:[{mealSlot,restaurantId}]};
1460             if(scheduleType==='WEEKLY'&&dayOfWeek) body.dayOfWeek=dayOfWeek;
1461             try{
1462                 await apiFetch('/subscription',{method:'POST',body:JSON.stringify(body)});
1463                 showAlert('Subscribed successfully!','success');
1464                 loadSubscriptions(); loadDashboard();
1465             }catch(e){ showAlert('Subscription failed: '+e.message,'error'); }
1466         }
1467
1468         async function cancelSub(id){
1469             if(!confirm('Cancel this subscription?')) return;
1470             try{
1471                 await apiFetch(`/subscription/${id}/expire`,{method:'PUT'});
1472                 showAlert('Subscription cancelled','success');
1473                 loadSubscriptions();
1474             }catch(e){ showAlert('Failed to cancel: '+e.message,'error'); }
1475         }
1476
1477         /* =====
1478         MY DEVICES

```

```

1479  ===== */
1480  async function loadMyAssignedDevices(){
1481      try{
1482          const devices=await apiFetch('/devices/my');
1483          const tbody=document.getElementById('my-devices-list');
1484          if(!devices||!devices.length){
1485              tbody.innerHTML=<tr><td colspan="7" class="empty-state">No devices assigned to
you</td></tr>';
1486              return;
1487          }
1488          const today=new Date();
1489          tbody.innerHTML=devices.map(d=>{
1490              const we=d.warrantyExpiryDate?new Date(d.warrantyExpiryDate):null;
1491              const wd=we?(we>today
1492                  ?`<span style="color:var(--green);font-size:11px"> ${d.warrantyExpiryDate}</span>`
1493                  :`<span style="color:var(--red);font-size:11px"> Expired
${d.warrantyExpiryDate}</span>`)
1494                  :`<span style="color:var(--text3);font-size:11px">--</span>`;
1495              return `<tr>
1496                  <td>#${d.id}</td>
1497                  <td><strong>${d.deviceName||'-'}</strong></td>
1498                  <td>${d.brand||'-'}</td>
1499                  <td><span class="chip">${(d.deviceType||'').toLowerCase()}</span></td>
1500                  <td>${wd}</td>
1501                  <td style="font-size:12px;color:var(--text2)">${d.assignedDate||'-'}</td>
1502                  <td><span class="chip
${(d.deviceStatus||'').toLowerCase()}>${d.deviceStatus||'-'}</span></td>
1503                  </tr>`;
1504              }).join('');
1505          }catch(e){ console.error(e); }
1506      }
1507
1508  /* =====
1509  SERVICE REQUESTS
1510  ===== */
1511  async function loadServiceRequests(){
1512      await Promise.all([loadServiceRequestsList(),loadMyDevicesForSR()]);
1513  }
1514
1515  async function loadServiceRequestsList(){
1516      try{
1517          const requests=await apiFetch('/service-requests/my');
1518          const tbody=document.getElementById('service-requests-list');
1519          if(!requests||!requests.length){
1520              tbody.innerHTML=<tr><td colspan="7" class="empty-state">No service requests
yet</td></tr>';
1521              return;
1522          }
1523          tbody.innerHTML=requests.map(r=>`<tr>
1524              <td style="color:var(--text3);font-family:var(--mono)">#${r.id}</td>
1525              <td>${r.deviceName||'-'}</td>
1526              <td><span class="chip pending">${r.requestType}</span></td>
1527              <td style="max-width:200px;overflow:hidden;text-overflow:ellipsis;white-
space:nowrap">${r.issueDescription||'-'}</td>
1528              <td>${r.urgent?'<span class="chip rejected">YES</span>':`<span style="color:var(--
text3)">No</span>`}</td>
1529              <td><span class="chip ${((r.status||'').toLowerCase())}">${r.status||'-'}</span></td>
1530              <td style="font-size:12px;color:var(--text3)">${r.raisedAt?new
Date(r.raisedAt).toLocaleDateString():'-'}</td>
1531              </tr>`).join('');
1532          }catch(e){ console.error(e); }
1533      }
1534
1535  async function loadMyDevicesForSR(){
1536      try{
1537          const devices=await apiFetch('/devices/my');
1538          const select=document.getElementById('sr-device-select');
1539          select.innerHTML=<option value="">-- Select your device --</option>';
1540          (devices||[]).forEach(d=>{
1541              const opt=document.createElement('option');
1542              opt.value=d.id;
1543              opt.textContent=`${d.deviceName} (${d.brand||d.deviceType||'Device'})`;
1544              select.appendChild(opt);
1545          });
1546          }catch(e){ console.error('Failed to load devices for SR',e); }
1547      }

```

```

1548
1549     async function submitServiceRequest(){
1550         const deviceId      =parseInt(document.getElementById('sr-device-select').value);
1551         const requestType   =document.getElementById('sr-type').value;
1552         const issueDescription=document.getElementById('sr-description').value.trim();
1553         const urgent        =document.getElementById('sr-urgent').checked;
1554         if(!deviceId){showAlert('Please select a device','error');return;}
1555         if(!issueDescription){showAlert('Please describe the issue','error');return;}
1556         try{
1557             await apiFetch('/service-
requests',{method:'POST',body:JSON.stringify({deviceId,requestType,issueDescription,urgent})});
1558             showAlert('Service request submitted!','success');
1559             clearServiceRequestForm();
1560             loadServiceRequestsList();
1561         }catch(e){ showAlert('Failed: '+e.message,'error'); }
1562     }
1563
1564     function clearServiceRequestForm(){
1565         document.getElementById('sr-device-select').value='';
1566         document.getElementById('sr-type').value='REPAIR';
1567         document.getElementById('sr-description').value='';
1568         document.getElementById('sr-urgent').checked=false;
1569     }
1570
1571     /* =====
1572     LOGOUT + ALERT
1573     ===== */
1574     function logout(){
1575         localStorage.removeItem('jwt_token');
1576         localStorage.removeItem('refresh_token');
1577         localStorage.removeItem('auth_type');
1578         localStorage.removeItem('last_role');
1579         fetch(`${API}/session/logout`,{method:'POST',credentials:'include'}).finally(()=>{
1580             document.cookie='JSESSIONID=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/;';
1581             window.location.href='/google.html';
1582         });
1583     }
1584     async function loadNotifCount() {
1585         try {
1586             const data = await apiFetch('/notifications/unread-count');
1587             const badge = document.getElementById('notif-badge');
1588             if (data && data.count > 0) {
1589                 badge.textContent = data.count;
1590                 badge.style.display = 'flex';
1591             } else {
1592                 badge.style.display = 'none';
1593             }
1594         } catch(e) {
1595             console.warn('Could not load notification count:', e);
1596         }
1597     }
1598
1599     async function toggleNotifPanel() {
1600         const panel = document.getElementById('notif-panel');
1601         if (panel.style.display === 'none') {
1602             panel.style.display = 'block';
1603             try {
1604                 await loadNotifications();
1605                 await apiFetch('/notifications/mark-read', { method: 'POST' });
1606                 document.getElementById('notif-badge').style.display = 'none';
1607             } catch(e) {
1608                 console.warn('Notification panel error:', e);
1609             }
1610         } else {
1611             panel.style.display = 'none';
1612         }
1613     }
1614
1615     async function loadNotifications() {
1616         try {
1617             const data = await apiFetch('/notifications');
1618             const list = document.getElementById('notif-list');
1619             if (!data || data.length === 0) {
1620                 list.innerHTML = '<div class="notif-item">No notifications yet.</div>';
1621                 return;
1622             }

```

```

1623         list.innerHTML = data.map(n => `
1624             <div class="notif-item ${n.read ? '' : 'unread'}">
1625                 <div>${n.message}</div>
1626                 <div class="notif-time">${new Date(n.createdAt).toLocaleString()}</div>
1627             </div>
1628         `).join('');
1629     } catch(e) {
1630         document.getElementById('notif-list').innerHTML =
1631             '<div class="notif-item">Could not load notifications.</div>';
1632         console.warn('loadNotifications failed:', e);
1633     }
1634 }
1635
1636 // Load badge count on page load
1637 loadNotifCount();
1638 // Refresh count every 30 seconds
1639 setInterval(loadNotifCount, 30000);
1640
1641 document.getElementById('mat-start-date').addEventListener('change', function(){
1642     if(this.value) {
1643         const start = new Date(this.value);
1644         const end = new Date(start);
1645         end.setDate(end.getDate() + 179); // 180 days inclusive
1646         document.getElementById('mat-end-date').value =
1647             end.toISOString().split('T')[0];
1648     } else {
1649         document.getElementById('mat-end-date').value = '';
1650     }
1651 });
1652
1653 async function applyMaternityLeave() {
1654     const startDate = document.getElementById('mat-start-date').value;
1655     const reason = document.getElementById('mat-reason').value.trim();
1656
1657     if(!startDate) {
1658         showAlert('Please select a start date', 'error');
1659         return;
1660     }
1661
1662     try {
1663         await apiFetch('/leave_requests/maternity/apply', {
1664             method: 'POST',
1665             body: JSON.stringify({ startDate, reason })
1666         });
1667         showAlert('Maternity leave request submitted successfully!', 'success');
1668         // Clear form
1669         document.getElementById('mat-start-date').value = '';
1670         document.getElementById('mat-end-date').value = '';
1671         document.getElementById('mat-reason').value = '';
1672         // Refresh dashboard
1673         loadDashboard();
1674     } catch(e) {
1675         showAlert('Failed to submit: ' + e.message, 'error');
1676     }
1677 }
1678
1679 // Close panel when clicking outside
1680 document.addEventListener('click', e => {
1681     const panel = document.getElementById('notif-panel');
1682     const wrapper = document.querySelector('.notif-wrapper');
1683     if (!panel.contains(e.target) && !wrapper.contains(e.target)) {
1684         panel.style.display = 'none';
1685     }
1686 });
1687
1688 function showAlert(msg,type){
1689     const el=document.createElement('div');
1690     el.className=`alert alert-${type}`;
1691     el.textContent=msg;
1692     document.body.appendChild(el);
1693     setTimeout(()=>el.remove(),3000);
1694 }
1695 // ===== TWO-FACTOR AUTH =====
1696 async function loadTotpStatus() {
1697     const chip = document.getElementById('totp-status-chip');
1698     chip.textContent = 'Checking...';

```

```

1699     chip.className = 'chip';
1700     document.getElementById('qr-area').style.display = 'none';
1701     document.getElementById('totp-verify-input').value = '';
1702     document.getElementById('totp-verify-error').style.display = 'none';
1703     try {
1704         const data = await apiFetch('/totp/status');
1705         if (data.totpEnabled) {
1706             chip.textContent = 'ENABLED';
1707             chip.className = 'chip approved';
1708             document.getElementById('totp-setup-section').style.display = 'none';
1709             document.getElementById('totp-disable-section').style.display = 'block';
1710         } else {
1711             chip.textContent = 'DISABLED';
1712             chip.className = 'chip rejected';
1713             document.getElementById('totp-setup-section').style.display = 'block';
1714             document.getElementById('totp-disable-section').style.display = 'none';
1715         }
1716     } catch(e) {
1717         chip.textContent = 'Error';
1718         chip.className = 'chip pending';
1719     }
1720 }
1721
1722 async function generateTotpQr() {
1723     const btn = document.getElementById('generate-qr-btn');
1724     btn.textContent = 'Generating...';
1725     btn.disabled = true;
1726     try {
1727         const data = await apiFetch('/totp/setup', { method: 'POST' });
1728         document.getElementById('totp-secret-display').textContent = data.secret || '';
1729         const qrUri = encodeURIComponent(data.qrCodeUri || '');
1730         document.getElementById('qr-img').src =
1731             `https://api.qrserver.com/v1/create-qr-code/?size=180x180&data=${qrUri}`;
1732         document.getElementById('qr-area').style.display = 'block';
1733     } catch(e) {
1734         showAlert('Failed to generate QR: ' + e.message, 'error');
1735     } finally {
1736         btn.textContent = 'Regenerate QR Code';
1737         btn.disabled = false;
1738     }
1739 }
1740
1741 async function verifyAndActivateTotp() {
1742     const code = document.getElementById('totp-verify-input').value.trim();
1743     const errEl = document.getElementById('totp-verify-error');
1744     errEl.style.display = 'none';
1745     if (code.length !== 6) {
1746         errEl.textContent = 'Please enter a 6-digit code.';
1747         errEl.style.display = 'block';
1748         return;
1749     }
1750     try {
1751         await apiFetch('/totp/verify-setup', {
1752             method: 'POST',
1753             body: JSON.stringify({ code })
1754         });
1755         showAlert('2FA activated! You will need your authenticator on next login.', 'success');
1756         loadTotpStatus();
1757     } catch(e) {
1758         errEl.textContent = 'Invalid code. Try again.';
1759         errEl.style.display = 'block';
1760     }
1761 }
1762
1763 async function disableTotp() {
1764     if (!confirm('Are you sure you want to disable two-factor authentication?')) return;
1765     try {
1766         await apiFetch('/totp/disable', { method: 'DELETE' });
1767         showAlert('2FA has been disabled.', 'success');
1768         loadTotpStatus();
1769     } catch(e) {
1770         showAlert('Failed to disable 2FA: ' + e.message, 'error');
1771     }
1772 }
1773 </script>
1774 </body>

```



```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>EMS – Sign In</title>
7      <link
href="https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;500;600;700&family=JetBrains+Mono:wght@400;500&display=swap" rel="stylesheet">
8      <style>
9          *{margin:0;padding:0;box-sizing:border-box}
10         :root{
11             --bg:#070910;
12             --panel:#0e1118;
13             --panel2:#131821;
14             --border:rgba(255,255,255,0.07);
15             --border-lit:rgba(255,255,255,0.14);
16             --text:#dde4f0;--text2:#6b7591;--text3:#3d4560;
17             --blue:#4f8cff;--blue-dim:rgba(79,140,255,0.12);
18             --violet:#a78bfa;--violet-dim:rgba(167,139,250,0.12);
19             --red:#f87171;--red-dim:rgba(248,113,113,0.12);
20             --orange:#f59e0b;--orange-dim:rgba(245,158,11,0.12);
21             --green:#34d399;--green-dim:rgba(52,211,153,0.12);
22             --font:'Outfit', sans-serif;--mono:'JetBrains Mono', monospace;
23         }
24
25         body{
26             font-family:var(--font);
27             background:var(--bg);
28             color:var(--text);
29             min-height:100vh;
30             display:flex;
31             overflow-y:auto;
32         }
33
34         /* ---- BACKGROUND ---- */
35         .bg-layer{
36             position:fixed;inset:0;z-index:0;
37             background:
38                 radial-gradient(ellipse 60% 50% at 20% 50%, rgba(79,140,255,0.06) 0%, transparent 70%),
39                 radial-gradient(ellipse 40% 60% at 80% 20%, rgba(167,139,250,0.05) 0%, transparent 60%),
40                 radial-gradient(ellipse 50% 40% at 70% 80%, rgba(52,211,153,0.04) 0%, transparent 60%);
41         }
42         .grid-overlay{
43             position:fixed;inset:0;z-index:0;
44             background-image:
45                 linear-gradient(rgba(255,255,255,0.025) 1px, transparent 1px),
46                 linear-gradient(90deg, rgba(255,255,255,0.025) 1px, transparent 1px);
47             background-size:40px 40px;
48             mask-image:radial-gradient(ellipse 80% 80% at center, black 20%, transparent 80%);
49         }
50
51         #oauth-loading {
52             display: none;
53             position: fixed; inset: 0;
54             background: linear-gradient(135deg, #0d0f14 0%, #131720 100%);
55             z-index: 9999;
56             justify-content: center;
57             align-items: center;
58             flex-direction: column;
59             gap: 20px;
60         }
61         #oauth-loading.visible { display: flex; }
62         .spinner {
63             width: 48px; height: 48px;
64             border: 3px solid #2a2f3f;
65             border-top-color: #4f8cff;
66             border-radius: 50%;
67             animation: spin 0.9s linear infinite;
68         }
69         @keyframes spin { to { transform: rotate(360deg); } }
70

```

```

71      /* ---- SPLIT LAYOUT ---- */
72      .left-panel{
73          flex:1;display:flex;flex-direction:column;justify-content:center;
74          padding:60px;position:relative;z-index:1;
75          animation:fadeLeft 0.8s ease both;
76      }
77      @keyframes fadeLeft{from{opacity:0;transform:translateX(24px)}to{opacity:1;transform:translateX(0)}}
78      .brand{margin-bottom:48px}
79      .brand-logo{display:flex;align-items:center;gap:12px;margin-bottom:20px}
80      .brand-icon{
81          width:44px;height:44px;background:linear-gradient(135deg,#4f8cff,#6366f1);
82          border-radius:12px;display:flex;align-items:center;justify-content:center;
83          font-weight:700;font-size:18px;color:#fff;
84          box-shadow:0 0 30px rgba(79,140,255,0.35);
85          flex-shrink:0;
86      }
87      .brand-name{font-size:22px;font-weight:700;color:var(--text);letter-spacing:-0.3px}
88      .brand-tagline{font-size:13px;color:var(--text2)}
89      .hero-text h1{
90          font-size:clamp(36px,4vw,52px);font-weight:700;
91          line-height:1.1;letter-spacing:-1px;color:var(--text);margin-bottom:16px;
92      }
93      .hero-text h1 .hl-blue{color:var(--blue)}
94      .hero-text h1 .hl-violet{color:var(--violet)}
95      .hero-text h1 .hl-orange{color:var(--orange)}
96      .hero-text p{font-size:15px;color:var(--text2);line-height:1.6;max-width:380px}
97      .feature-pills{display:flex;flex-wrap:wrap;gap:8px;margin-top:32px}
98      .pill{
99          display:inline-flex;align-items:center;gap:6px;
100         padding:6px 14px;border-radius:20px;font-size:12px;font-weight:500;
101         border:1px solid var(--border);background:var(--panel);color:var(--text2);
102     }
103     .pill .dot{width:6px;height:6px;border-radius:50%;flex-shrink:0}
104     .pill.blue .dot{background:var(--blue)} .pill.blue{border-
color:rgba(79,140,255,0.2);color:rgba(79,140,255,0.8)}
105     .pill.violet .dot{background:var(--violet)} .pill.violet{border-
color:rgba(167,139,250,0.2);color:rgba(167,139,250,0.8)}
106     .pill.orange .dot{background:var(--orange)} .pill.orange{border-
color:rgba(245,158,11,0.2);color:rgba(245,158,11,0.8)}
107
108     /* ---- RIGHT PANEL (FORM) ---- */
109     .right-panel{
110         width:460px;flex-shrink:0;
111         display:flex;align-items:flex-start;justify-content:center;
112         overflow-y:auto;
113         padding:32px 40px;position:relative;z-index:1;
114         border-left:1px solid var(--border);
115         background:rgba(14,17,24,0.8);
116         backdrop-filter:blur(20px);
117         animation:fadeRight 0.8s ease both;
118     }
119     @keyframes
fadeRight{from{opacity:0;transform:translateX(24px)}to{opacity:1;transform:translateX(0)}}
120     .form-box{width:100%}
121
122     /* ---- ROLE SELECTOR — 3 roles only: Employee | Manager | Admin ---- */
123     .role-label{font-size:10px;font-weight:600;color:var(--text3);letter-spacing:1.5px;text-
transform:uppercase;font-family:var(--mono);margin-bottom:10px}
124     .role-tabs{display:grid;grid-template-columns:repeat(3,1fr);gap:6px;margin-bottom:28px}
125     .role-tab{
126         padding:10px 8px;border-radius:10px;border:1.5px solid var(--border);
127         background:var(--bg);color:var(--text2);font-family:var(--font);
128         font-size:12px;font-weight:600;cursor:pointer;transition:all 0.2s;
129         display:flex;flex-direction:column;align-items:center;gap:4px;
130     }
131     .role-tab .role-icon{font-size:16px}
132     .role-tab:hover{border-color:var(--border-lit);color:var(--text)}
133     .role-tab.active-emp{border-color:var(--blue);background:var(--blue-dim);color:var(--blue)}
134     .role-tab.active-mgr{border-color:var(--violet);background:var(--violet-dim);color:var(--violet)}
135     .role-tab.active-adm{border-color:var(--orange);background:var(--orange-dim);color:var(--orange)}
136
137     /* ---- AUTH TABS ---- */
138     .auth-tabs{display:flex;background:var(--bg);border-radius:10px;padding:4px;gap:4px;margin-
bottom:24px;border:1px solid var(--border)}
139     .auth-tab{
140         flex:1;padding:9px;border:none;background:transparent;

```

```

141         color:var(--text2);font-family:var(--font);font-weight:600;
142         font-size:13px;cursor:pointer;border-radius:7px;transition:all 0.2s;
143     }
144     .auth-tab.active{background:var(--panel2);color:var(--text);box-shadow:0 1px 4px rgba(0,0,0,0.3)}
145
146     /* ---- FORM ELEMENTS ---- */
147     .form-group{margin-bottom:16px}
148     .form-label{display:block;font-size:10px;font-weight:600;color:var(--text2);text-
transform:uppercase;letter-spacing:0.8px;font-family:var(--mono);margin-bottom:7px}
149     .form-input{
150         width:100%;padding:11px 14px;
151         background:var(--bg);border:1px solid var(--border);
152         border-radius:9px;color:var(--text);
153         font-family:var(--font);font-size:14px;
154         transition:all 0.2s;
155     }
156     .form-input:focus{outline:none;border-color:var(--blue);box-shadow:0 0 3px rgba(79,140,255,0.08)}
157     .form-input::placeholder{color:var(--text3)}
158     .form-input.focus-violet:focus{border-color:var(--violet);box-shadow:0 0 3px
159     rgba(167,139,250,0.08)}
160     .form-input.focus-orange:focus{border-color:var(--orange);box-shadow:0 0 3px
161     rgba(245,158,11,0.08)}
162
163     /* ---- BUTTONS ---- */
164     .btn-login{
165         width:100%;padding:12px;border:none;border-radius:9px;
166         font-family:var(--font);font-size:14px;font-weight:600;
167         cursor:pointer;transition:all 0.2s;margin-top:4px;
168         position:relative;overflow:hidden;
169     }
170     .btn-login.blue{background:var(--blue);color:#fff}
171     .btn-login.blue:hover{background:#6ba0ff;box-shadow:0 6px 20px
172     rgba(79,140,255,0.35);transform:translateY(-1px)}
173     .btn-login.violet{background:var(--violet);color:#fff}
174     .btn-login.violet:hover{background:#b8a0fc;box-shadow:0 6px 20px
175     rgba(167,139,250,0.35);transform:translateY(-1px)}
176     .btn-login.orange{background:var(--orange);color:#fff}
177     .btn-login.orange:hover{background:#f7b23b;box-shadow:0 6px 20px
178     rgba(245,158,11,0.35);transform:translateY(-1px)}
179     .btn-login.active{transform:translateY(0)}
180     .btn-login.loading{opacity:0.7;pointer-events:none}
181
182     .divider{display:flex;align-items:center;gap:12px;margin:20px 0;color:var(--text3);font-
183     size:11px;font-family:var(--mono)}
184     .divider::before,.divider::after{content:'';flex:1;height:1px;background:var(--border)}
185
186     .btn-google{
187         width:100%;padding:11px;background:transparent;
188         border:1px solid var(--border-lit);border-radius:9px;
189         color:var(--text2);font-family:var(--font);font-size:13px;font-weight:500;
190         cursor:pointer;transition:all 0.2s;
191         display:flex;align-items:center;justify-content:center;gap:10px;
192     }
193     .btn-google:hover{background:rgba(255,255,255,0.04);border-color:rgba(255,255,255,0.2);color:var(--
194     text)}
195
196     /* ---- GOOGLE ROLE SELECT ---- */
197     .google-role-row{display:flex;gap:8px;margin-bottom:10px;align-items:center}
198     .google-role-label{font-size:11px;color:var(--text3);font-family:var(--mono);white-space:nowrap}
199     .google-role-select{
200         flex:1;padding:8px 10px;background:var(--bg);
201         border:1px solid var(--border);border-radius:8px;
202         color:var(--text);font-family:var(--font);font-size:13px;cursor:pointer;
203     }
204     .google-role-select:focus{outline:none;border-color:var(--border-lit)}
205     .google-role-select option{background:var(--panel)}
206
207     /* ---- VENDOR LINK ---- */
208     .vendor-link-row{
209         margin-top:18px;padding-top:16px;
210         border-top:1px solid var(--border);
211         display:flex;align-items:center;justify-content:space-between;
212     }
213     .vendor-link-row p{font-size:12px;color:var(--text3);}
214     .btn-vendor-link{
215         padding:7px 16px;border-radius:8px;

```

```

209         border:1px solid rgba(248,113,113,0.3);
210         background:var(--red-dim);color:var(--red);
211         font-family:var(--font);font-size:12px;font-weight:600;
212         cursor:pointer;text-decoration:none;transition:all 0.2s;
213         white-space:nowrap;
214     }
215     .btn-vendor-link:hover{background:rgba(248,113,113,0.2);border-color:var(--red);}
216
217     /* ---- ERROR / STATUS ---- */
218     .status-box{
219         border-radius:9px;padding:10px 14px;margin-top:14px;
220         font-size:13px;font-weight:500;display:none;
221         animation:fadeIn 0.3s ease;
222     }
223     @keyframes fadeIn{from{opacity:0;transform:translateY(-4px)}to{opacity:1;transform:translateY(0)}}
224     .status-box.error{background:var(--red-dim);color:var(--red);border:1px solid
225     rgba(248,113,113,0.2)}
226     .status-box.success{background:var(--green-dim);color:var(--green);border:1px solid
227     rgba(52,211,153,0.2)}
228     .status-box.loading{background:var(--blue-dim);color:var(--blue);border:1px solid
229     rgba(79,140,255,0.2)}
230
231     /* ---- FOOTER ---- */
232     .form-footer{margin-top:20px;padding-top:18px;border-top:1px solid var(--border)}
233     .tz-row{display:flex;align-items:center;gap:8px;margin-bottom:8px}
234     .tz-label{font-size:10px;color:var(--text3);font-family:var(--mono);flex-shrink:0}
235     .tz-select{
236         flex:1;padding:6px 8px;background:var(--bg);border:1px solid var(--border);
237         border-radius:7px;color:var(--text2);font-family:var(--mono);font-size:11px;cursor:pointer;
238     }
239     .tz-select:focus{outline:none;border-color:var(--border-lit)}
240     .tz-select option{background:var(--panel)}
241     .tz-time{font-size:10px;color:var(--blue);font-family:var(--mono)}
242
243     /* ---- RESPONSIVE ---- */
244     @media(max-width:900px){
245         .left-panel{display:none}
246         .right-panel{width:100%;border-left:none;background:var(--bg)}
247     }
248 </style>
249 </head>
250 <body>
251 <div id="oauth-loading">
252 <div class="spinner"></div>
253 <div style="text-align: center;">
254 <p style="color: #e0e0e0; font-size: 16px; font-weight: 600; margin: 0 0 6px; font-family:
255 'Outfit', sans-serif;">Signing you in...</p>
256 <p style="color: #8b92a8; font-size: 13px; margin: 0; font-family: 'Outfit', sans-serif;">Verifying
257 your Google account</p>
258 </div>
259 </div>
260 <div class="bg-layer"></div>
261 <div class="grid-overlay"></div>
262
263 <!-- LEFT HERO -->
264 <div class="left-panel">
265 <div class="brand">
266 <div class="brand-logo">
267 <div class="brand-icon">E</div>
268 <div>
269 <div class="brand-name">EMS Portal</div>
270 <div class="brand-tagline">Employee Management System</div>
271 </div>
272 </div>
273 </div>
274 <div class="hero-text">
275 <h1>Your workspace,<br><span class="hl-blue">one login</span><br>away.</h1>
276 <p>Employees, Managers, and Admins each get their own tailored experience. Sign in to access your
277 personalized dashboard.</p>
278 </div>
279 <div class="feature-pills">
280 <div class="pill blue"><span class="dot"></span>Employee Portal</div>
281 <div class="pill violet"><span class="dot"></span>Manager Console</div>
282 <div class="pill orange"><span class="dot"></span>Admin Control Center</div>
283 <div class="pill blue"><span class="dot"></span>Leave Management</div>
284 <div class="pill violet"><span class="dot"></span>Meal Subscriptions</div>

```

```

279     <div class="pill orange"><span class="dot"></span>Delivery Tracking</div>
280   </div>
281 </div>
282
283 <!-- RIGHT FORM -->
284 <div class="right-panel">
285   <div class="form-box">
286
287     <!-- ROLE SELECTOR: 3 roles only – Vendor removed -->
288     <div class="role-label">Sign in as</div>
289     <div class="role-tabs">
290       <button class="role-tab active-emp" id="role-emp" onclick="selectRole('EMPLOYEE')">
291         <span class="role-icon"></span>Employee
292       </button>
293       <button class="role-tab" id="role-mgr" onclick="selectRole('MANAGER')">
294         <span class="role-icon"></span>Manager
295       </button>
296       <button class="role-tab" id="role-adm" onclick="selectRole('ADMIN')">
297         <span class="role-icon"></span>Admin
298       </button>
299     </div>
300
301     <!-- AUTH METHOD TABS -->
302     <div class="auth-tabs">
303       <button class="auth-tab active" id="tab-jwt" onclick="switchTab('jwt')">JWT Login</button>
304       <button class="auth-tab" id="tab-session" onclick="switchTab('session')">Session Login</button>
305     </div>
306
307     <!-- JWT FORM -->
308     <div id="jwt-form">
309       <div class="form-group">
310         <label class="form-label">Enter your Email</label>
311         <input class="form-input" id="jwt-username" type="text"
placeholder="your.email@company.com" autocomplete="off">
312       </div>
313       <div class="form-group">
314         <label class="form-label">Password</label>
315         <input class="form-input" id="jwt-password" type="password" placeholder="....."
autocomplete="current-password">
316       </div>
317       <button class="btn-login blue" id="jwt-btn" onclick="jwtLogin()">Sign In</button>
318     </div>
319
320     <!-- SESSION FORM -->
321     <div id="session-form" style="display:none">
322       <div class="form-group">
323         <label class="form-label">Enter your Email</label>
324         <input class="form-input" id="session-username" type="text"
placeholder="your.email@company.com">
325       </div>
326       <div class="form-group">
327         <label class="form-label">Password</label>
328         <input class="form-input" id="session-password" type="password" placeholder=".....">
329       </div>
330       <button class="btn-login blue" id="session-btn" onclick="sessionLogin()">Sign In with
Session</button>
331     </div>
332
333     <!-- OAUTH – Vendor removed from Google role select -->
334     <div class="divider">or continue with</div>
335     <div class="google-role-row">
336       <span class="google-role-label">Sign in as</span>
337       <select class="google-role-select" id="oauth-role">
338         <option value="EMPLOYEE">Employee</option>
339         <option value="MANAGER">Manager</option>
340         <option value="ADMIN">Admin</option>
341       </select>
342     </div>
343     <button class="btn-google" onclick="googleLogin()">
344       <svg width="18" height="18" viewBox="0 0 24 24">
345         <path fill="#4285F4" d="M22.56 12.25c0-.78-.07-1.53-.2-2.25H12v4.26h5.92c-.26 1.37-1.04
2.53-2.21 3.31v2.77h3.57c2.08-1.92 3.28-4.74 3.28-8.09z"/>
346         <path fill="#34A853" d="M12 23c2.97 0 5.46-.98 7.28-2.66l-3.57-2.77c-.98.66-2.23 1.06-3.71
1.06-2.86 0-5.29-1.93-6.16-4.53H2.18v2.84C3.99 20.53 7.7 23 12 23z"/>
347         <path fill="#FBBC05" d="M5.84 14.09c-.22-.66-.35-1.36-.35-2.09s.13-1.43.35-
2.09V7.07H2.18c1.43 8.55 1 10.22 1 12s.43 3.45 1.18 4.93l2.85-2.22.81-.62z"/>

```

```

348         <path fill="#EA4335" d="M12 5.38c1.62 0 3.06.56 4.21 1.64l3.15-3.15C17.45 2.09 14.97 1 12 1
7.7 1 3.99 3.47 2.18 7.07l3.66 2.84c.87-2.6 3.3-4.53 6.16-4.53z"/>
349     </svg>
350     Continue with Google
351 </button>
352
353 <div class="status-box" id="status-box"></div>
354
355 <!-- Vendor Login Link -->
356 <div class="vendor-link-row">
357     <p>Are you a vendor?</p>
358     <a class="btn-vendor-link" href="/vendor-login.html">Vendor Login </a>
359 </div>
360
361 <!-- FOOTER -->
362 <div class="form-footer">
363     <div class="tz-row">
364         <span class="tz-label">TZ</span>
365         <select class="tz-select" id="tz-select" onchange="updateTz()">
366             <option value="Asia/Kolkata">India – IST UTC+5:30</option>
367             <option value="UTC">UTC</option>
368             <option value="America/New_York">New York – EST UTC-5</option>
369             <option value="America/Los_Angeles">Los Angeles – PST UTC-8</option>
370             <option value="Europe/London">London – GMT UTC+0</option>
371             <option value="Europe/Paris">Paris – CET UTC+1</option>
372             <option value="Asia/Tokyo">Tokyo – JST UTC+9</option>
373             <option value="Asia/Dubai">Dubai – GST UTC+4</option>
374             <option value="Australia/Sydney">Sydney – AEDT UTC+11</option>
375             <option value="Asia/Singapore">Singapore – SGT UTC+8</option>
376         </select>
377         <span class="tz-time" id="tz-time">--:--:--</span>
378     </div>
379 </div>
380
381 </div>
382 <!-- TOTP MODAL OVERLAY -->
383 <div id="totp-overlay" style="display:none;position:fixed;inset:0;z-
index:999;background:rgba(7,9,16,0.85);backdrop-filter:blur(8px);display:none;align-items:center;justify-
content:center;">
384     <div style="background:var(--panel);border:1px solid var(--border-lit);border-
radius:16px;padding:36px;width:340px;text-align:center;animation:fadeIn 0.25s ease;">
385         <div style="font-size:28px;margin-bottom:8px;"></div>
386         <h3 style="font-size:17px;font-weight:700;color:var(--text);margin-bottom:6px;">Two-Factor
Verification</h3>
387         <p style="font-size:13px;color:var(--text2);margin-bottom:24px;">Enter the 6-digit code from
your authenticator app.</p>
388         <input id="totp-code" class="form-input" type="text" inputmode="numeric" maxlength="6"
placeholder="000000"
389             style="text-align:center;font-size:22px;font-family:var(--mono);letter-
spacing:8px;padding:14px;"
390             oninput="this.value=this.value.replace(/\D/g,'')">
391         <div id="totp-status" style="display:none;margin-top:10px;font-size:13px;color:var(--
red);"></div>
392         <button id="totp-verify-btn" class="btn-login blue" style="margin-top:16px;"
onclick="submitTotp()">Verify</button>
393         <button onclick="cancelTotp()" style="margin-
top:10px;width:100%;padding:10px;border:none;background:transparent;color:var(--text2);font-family:var(--
font);font-size:13px;cursor:pointer...
394     </div>
395 </div>
396
397 </div>
398
399 <script>
400     const API = 'http://localhost:8080';
401     const GOOGLE_CLIENT_ID = '525324511556-s9eugd1o761n6cjje2nr60cv8ho6joq2.apps.googleusercontent.com';
402     const REDIRECT_URI = 'http://localhost:8080/google.html';
403
404     // Role dashboard mapping – VENDOR removed from this page
405     const DASHBOARDS = {
406         EMPLOYEE: '/employee-dashboard.html',
407         MANAGER: '/manager-dashboard.html',
408         ADMIN: '/dashboard.html',
409         USER: '/pending.html'
410     };
411
412     let currentRole = 'EMPLOYEE';

```

```

413
414     function buildAuthOptions(token, authType) {
415         const headers = {};
416         if (token) headers.Authorization = `Bearer ${token}`;
417         return {
418             headers,
419             credentials: authType === 'session' ? 'include' : 'same-origin'
420         };
421     }
422
423     async function resolveRoleFromServer(token, authType) {
424         const authOptions = buildAuthOptions(token, authType);
425
426         // Check employee/manager/admin endpoint – vendors are not expected here
427         const employeeResponse = await fetch(`${API}/employee/me`, authOptions);
428         if (employeeResponse.ok) {
429             const employee = await employeeResponse.json();
430             return employee.role || 'EMPLOYEE';
431         }
432
433         throw new Error('Unable to determine authenticated role');
434     }
435
436     async function redirectByRole(token, authType = localStorage.getItem('auth_type')) {
437         let role;
438
439         if (token) {
440             role = getRoleFromJWT(token);
441         } else {
442             try {
443                 const res = await fetch(`${API}/employee/me`, { credentials: 'include' });
444                 if (res.ok) {
445                     const emp = await res.json();
446                     role = emp.role || 'USER';
447                 } else {
448                     role = 'USER';
449                 }
450             } catch(e) {
451                 role = localStorage.getItem('last_role') || 'USER';
452             }
453         }
454
455         console.log('Final role for redirect:', role);
456         localStorage.setItem('last_role', role);
457
458         // Block ALL vendor types from using the main login page
459         if (role === 'FOOD_VENDOR' || role === 'TECH_VENDOR') {
460             localStorage.removeItem('jwt_token');
461             localStorage.removeItem('auth_type');
462             localStorage.removeItem('last_role');
463             // Reset button state
464             const jwtBtn = document.getElementById('jwt-btn');
465             const sesBtn = document.getElementById('session-btn');
466             if (jwtBtn) { jwtBtn.classList.remove('loading'); jwtBtn.textContent = 'Sign In'; }
467             if (sesBtn) { sesBtn.classList.remove('loading'); sesBtn.textContent = 'Sign In with Session'; }
468             showStatus('error', 'Vendors must use the Vendor Login page. ');
469             return;
470         }
471
472         const destination = DASHBOARDS[role];
473         if (!destination) {
474             // Unknown role – don't redirect anywhere, just show error
475             localStorage.removeItem('jwt_token');
476             localStorage.removeItem('auth_type');
477             showStatus('error', 'Unknown role: ' + role + '. Please contact your administrator. ');
478             const jwtBtn = document.getElementById('jwt-btn');
479             if (jwtBtn) { jwtBtn.classList.remove('loading'); jwtBtn.textContent = 'Sign In'; }
480             return;
481         }
482
483         console.log('Redirecting to:', destination);
484         window.location.href = destination;
485     }
486
487     // ---- Role selector (3 tabs: Employee | Manager | Admin) ----
488     function selectRole(role) {

```

```

489     currentRole = role;
490     document.getElementById('role-emp').className = 'role-tab';
491     document.getElementById('role-mgr').className = 'role-tab';
492     document.getElementById('role-adm').className = 'role-tab';
493
494     const classMap = { EMPLOYEE: 'active-emp', MANAGER: 'active-mgr', ADMIN: 'active-adm' };
495     const idMap     = { EMPLOYEE: 'role-emp',   MANAGER: 'role-mgr',   ADMIN: 'role-adm'   };
496     document.getElementById(idMap[role]).classList.add(classMap[role]);
497
498     document.getElementById('oauth-role').value = role;
499
500     const colorMap = { EMPLOYEE: 'blue', MANAGER: 'violet', ADMIN: 'orange' };
501     const col = colorMap[role];
502     document.getElementById('jwt-btn').className = `btn-login ${col}`;
503     document.getElementById('session-btn').className = `btn-login ${col}`;
504
505     document.querySelectorAll('.form-input').forEach(inp => {
506         inp.className = (`form-input focus-${col === 'blue' ? '' : col}`).trim();
507     });
508 }
509
510 // ---- Auth tab switching ----
511 function switchTab(type) {
512     document.getElementById('jwt-form').style.display = type === 'jwt' ? 'block' : 'none';
513     document.getElementById('session-form').style.display = type === 'session' ? 'block' : 'none';
514     document.getElementById('tab-jwt').classList.toggle('active', type === 'jwt');
515     document.getElementById('tab-session').classList.toggle('active', type === 'session');
516     hideStatus();
517 }
518
519 // ---- Status display ----
520 function showStatus(type, msg) {
521     const box = document.getElementById('status-box');
522     box.className = `status-box ${type}`;
523     box.textContent = msg;
524     box.style.display = 'block';
525 }
526 function hideStatus() {
527     document.getElementById('status-box').style.display = 'none';
528 }
529
530 // ---- JWT Login ----
531 let _preAuthToken = null; // holds token between step 1 and step 2
532
533 async function jwtLogin() {
534     const username = document.getElementById('jwt-username').value.trim();
535     const password = document.getElementById('jwt-password').value;
536     if (!username || !password) {
537         showStatus('error', 'Please enter your username and password');
538         return;
539     }
540
541     const btn = document.getElementById('jwt-btn');
542     btn.classList.add('loading'); btn.textContent = 'Signing in...';
543     showStatus('loading', 'Authenticating...');
544
545     try {
546         const res = await fetch(`${API}/Authenticate`, {
547             method: 'POST',
548             headers: { 'Content-Type': 'application/json' },
549             body: JSON.stringify({ username, password })
550         });
551
552         if (res.ok) {
553             const data = await res.json();
554
555             // --- TOTP required: show step-2 modal ---
556             if (data.requiresTotp) {
557                 _preAuthToken = data.preAuthToken;
558                 btn.classList.remove('loading'); btn.textContent = 'Sign In';
559                 hideStatus();
560                 openTotpModal();
561                 return;
562             }
563
564             // --- No TOTP: issue JWT directly ---

```



```

565         localStorage.setItem('jwt_token', data.accessToken);
566         localStorage.setItem('refresh_token', data.refreshToken || '');
567         localStorage.setItem('auth_type', 'jwt');
568         localStorage.setItem('last_role', currentRole);
569         localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
570         showStatus('success', 'Login successful! Redirecting...');
571         setTimeout(() => redirectByRole(data.accessToken, 'jwt'), 600);
572
573     } else {
574         const err = await res.text();
575         showStatus('error', 'Login failed: ' + (err || 'Invalid credentials'));
576         btn.classList.remove('loading'); btn.textContent = 'Sign In';
577     }
578 } catch (e) {
579     showStatus('error', 'Connection error: ' + e.message);
580     btn.classList.remove('loading'); btn.textContent = 'Sign In';
581 }
582 }
583
584 function openTotpModal() {
585     const overlay = document.getElementById('totp-overlay');
586     overlay.style.display = 'flex';
587     document.getElementById('totp-code').value = '';
588     document.getElementById('totp-status').style.display = 'none';
589     setTimeout(() => document.getElementById('totp-code').focus(), 100);
590 }
591
592 function cancelTotp() {
593     _preAuthToken = null;
594     document.getElementById('totp-overlay').style.display = 'none';
595 }
596
597 async function submitTotp() {
598     const code = document.getElementById('totp-code').value.trim();
599     if (code.length !== 6) {
600         document.getElementById('totp-status').textContent = 'Please enter a 6-digit code.';
601         document.getElementById('totp-status').style.display = 'block';
602         return;
603     }
604
605     const verifyBtn = document.getElementById('totp-verify-btn');
606     verifyBtn.classList.add('loading'); verifyBtn.textContent = 'Verifying...';
607     document.getElementById('totp-status').style.display = 'none';
608
609     try {
610         const res = await fetch(`${API}/Authenticate/totp`, {
611             method: 'POST',
612             headers: { 'Content-Type': 'application/json' },
613             body: JSON.stringify({ preAuthToken: _preAuthToken, code })
614         });
615
616         if (res.ok) {
617             const data = await res.json();
618             _preAuthToken = null;
619             document.getElementById('totp-overlay').style.display = 'none';
620             localStorage.setItem('jwt_token', data.accessToken);
621             localStorage.setItem('refresh_token', data.refreshToken || '');
622             localStorage.setItem('auth_type', 'jwt');
623             localStorage.setItem('last_role', currentRole);
624             localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
625             showStatus('success', 'Verified! Redirecting...');
626             setTimeout(() => redirectByRole(data.accessToken, 'jwt'), 600);
627         } else {
628             const err = await res.json().catch(() => ({}));
629             document.getElementById('totp-status').textContent = err.error || 'Invalid code. Try
again.';
630             document.getElementById('totp-status').style.display = 'block';
631             document.getElementById('totp-code').value = '';
632             document.getElementById('totp-code').focus();
633         }
634     } catch (e) {
635         document.getElementById('totp-status').textContent = 'Connection error: ' + e.message;
636         document.getElementById('totp-status').style.display = 'block';
637     } finally {
638         verifyBtn.classList.remove('loading'); verifyBtn.textContent = 'Verify';
639     }

```

```

640     }
641
642     // ---- Session Login ----
643     async function sessionLogin() {
644         const username = document.getElementById('session-username').value.trim();
645         const password = document.getElementById('session-password').value;
646         if (!username || !password) { showStatus('error', 'Please enter your username and password');
return; }
647
648         const btn = document.getElementById('session-btn');
649         btn.classList.add('loading'); btn.textContent = 'Signing in...';
650         showStatus('loading', 'Authenticating...');
651
652         try {
653             const res = await fetch(`${API}/session/login`, {
654                 method: 'POST',
655                 headers: { 'Content-Type': 'application/json' },
656                 body: JSON.stringify({ username, password }),
657                 credentials: 'include'
658             });
659             if (res.ok) {
660                 localStorage.setItem('auth_type', 'session');
661                 localStorage.setItem('last_role', currentRole);
662                 localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
663                 showStatus('success', 'Login successful! Redirecting...');
664                 setTimeout(() => redirectByRole(null, 'session'), 600);
665             } else {
666                 const err = await res.json().catch(() => ({}));
667                 showStatus('error', 'Login failed: ' + (err.error || 'Invalid credentials'));
668                 btn.classList.remove('loading'); btn.textContent = 'Sign In with Session';
669             }
670         } catch(e) {
671             showStatus('error', 'Connection error: ' + e.message);
672             btn.classList.remove('loading'); btn.textContent = 'Sign In with Session';
673         }
674     }
675
676     // ---- Google OAuth – goes through backend /auth/google/init ----
677     function googleLogin() {
678         const role = document.getElementById('oauth-role').value;
679         const timezone = document.getElementById('tz-select').value;
680         window.location.href =
681             `/auth/google/init` +
682             `?role=${encodeURIComponent(role)}` +
683             `&timezone=${encodeURIComponent(timezone)}`;
684     }
685
686     function getRoleFromJWT(token) {
687         try {
688             const payload = JSON.parse(atob(token.split('.')[1]));
689             console.log('JWT payload:', payload);
690             const role = payload.role || 'USER';
691             // Strip ROLE_ prefix if accidentally present
692             return typeof role === 'string' ? role.replace(/^ROLE_/, '') : 'USER';
693         } catch (e) {
694             console.error('Failed to parse JWT:', e);
695             return 'USER';
696         }
697     }
698
699     // ---- Spring redirects back here with tokens as query params ----
700     async function handleOAuthCallback() {
701         const params = new URLSearchParams(window.location.search);
702         const accessToken = params.get('accessToken');
703         const refreshToken = params.get('refreshToken');
704
705         if (!accessToken) return;
706
707         window.history.replaceState({}, document.title, window.location.pathname);
708
709         localStorage.setItem('jwt_token', accessToken);
710         localStorage.setItem('refresh_token', refreshToken || '');
711         localStorage.setItem('auth_type', 'jwt');
712
713         const role = getRoleFromJWT(accessToken);
714         console.log('Authenticated role:', role);

```

```

715     localStorage.setItem('last_role', role);
716
717     showStatus('success', 'Signed in! Redirecting...');
718     setTimeout(() => {
719         window.location.href = DASHBOARDS[role] || '/google.html';
720     }, 800);
721 }
722
723 // ---- Timezone ----
724 function updateTz() {
725     localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
726     refreshTzClock();
727 }
728 function refreshTzClock() {
729     const tz = document.getElementById('tz-select').value;
730     document.getElementById('tz-time').textContent =
731         new Date().toLocaleTimeString('en-GB', { timeZone: tz, hour12: false });
732 }
733
734 // ---- Enter key support ----
735 document.addEventListener('keydown', e => {
736     if (e.key !== 'Enter') return;
737     // If TOTP modal is open, submit TOTP
738     if (document.getElementById('totp-overlay').style.display === 'flex') {
739         submitTotp();
740         return;
741     }
742     if (document.getElementById('jwt-form').style.display !== 'none') jwtLogin();
743     else sessionLogin();
744 });
745 // ---- Init ----
746 window.onload = () => {
747     const savedTz = localStorage.getItem('user_timezone') || 'Asia/Kolkata';
748     document.getElementById('tz-select').value = savedTz;
749     refreshTzClock();
750     setInterval(refreshTzClock, 1000);
751
752     const params = new URLSearchParams(window.location.search);
753     const accessToken = params.get('accessToken');
754     if (accessToken) {
755         document.getElementById('oauth-loading').classList.add('visible');
756         document.querySelector('.right-panel').style.display = 'none';
757         document.querySelector('.left-panel').style.display = 'none';
758         handleOAuthCallback();
759     }
760 };
761 </script>
762 </body>
763 </html>
764

```

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>EMS - Leave Management</title>
7      <link
href="https://fonts.googleapis.com/css2?family=DM+Sans:wght@400;500;600;700&family=DM+Mono:wght@400;500&display=
swap" rel="stylesheet">
8      <style>
9          * { margin: 0; padding: 0; box-sizing: border-box; }
10         :root {
11             --bg: #0d0f14; --bg2: #131720; --bg3: #1a2030;
12             --panel: #1e2436; --panel2: #252c40;
13             --border: rgba(255,255,255,0.07); --border2: rgba(255,255,255,0.12);
14             --text: #e0e0e0; --text2: #8b92a8; --text3: #5557fa;
15             --accent: #4f8cff; --accent-dim: rgba(79,140,255,0.12);
16             --green: #36d399; --green-dim: rgba(54,211,153,0.12);
17             --amber: #f9c74f; --amber-dim: rgba(249,199,79,0.12);
18             --red: #f87171; --red-dim: rgba(248,113,113,0.12);
19             --violet: #a78bfa; --violet-dim: rgba(167,139,250,0.12);
20             --font: 'DM Sans', sans-serif; --mono: 'DM Mono', monospace;
21         }
22         body { font-family: var(--font); background: var(--bg); color: var(--text); font-size: 14px; line-
height: 1.5; overflow: hidden; }
23         .app { display: flex; height: 100vh; }
24
25         /* Sidebar */
26         .sidebar { width: 260px; background: var(--bg2); border-right: 1px solid var(--border); display:
flex; flex-direction: column; overflow-y: auto; }
27         .logo { padding: 20px; border-bottom: 1px solid var(--border); }
28         .logo h2 { font-size: 20px; color: var(--text); }
29         .logo span { color: var(--accent); }
30         .logo p { font-size: 11px; color: var(--text3); margin-top: 4px; }
31         .nav-section { padding: 12px 20px 4px; font-size: 10px; font-weight: 600; color: var(--text2); text-
transform: uppercase; letter-spacing: 0.08em; }
32         .nav-item { padding: 10px 20px; margin: 2px 12px; border-radius: 8px; cursor: pointer; transition:
all 0.2s; color: var(--text2); display: flex; align-items: center; gap: 10px; font-size: 1...
33         .nav-item svg { opacity: 0.6; flex-shrink: 0; }
34         .nav-item:hover, .nav-item.active { background: var(--accent-dim); color: var(--accent); }
35         .nav-item:hover svg, .nav-item.active svg { opacity: 1; }
36         .nav-item .badge-count { margin-left: auto; background: var(--amber-dim); color: var(--amber); font-
size: 10px; padding: 2px 6px; border-radius: 10px; font-weight: 600; }
37
38         /* Main */
39         .main-content { flex: 1; display: flex; flex-direction: column; overflow: hidden; }
40         .topbar { background: var(--bg2); padding: 16px 24px; display: flex; justify-content: space-between;
align-items: center; border-bottom: 1px solid var(--border); }
41         .page-title { font-size: 20px; font-weight: 600; }
42         .topbar-right { display: flex; align-items: center; gap: 16px; }
43         .user-pill { display: flex; align-items: center; gap: 8px; background: var(--bg3); border: 1px solid
var(--border2); border-radius: 20px; padding: 6px 12px; }
44         .user-avatar { width: 24px; height: 24px; border-radius: 50%; background: var(--accent-dim);
display: flex; align-items: center; justify-content: center; font-size: 10px; font-weight: 600; ...
45         .user-name { font-size: 12px; color: var(--text2); }
46         .logout-btn { padding: 8px 16px; background: var(--red-dim); border: 1px solid
rgba(248,113,113,0.3); border-radius: 6px; color: var(--red); cursor: pointer; font-family: var(--font); font-
...
47         .logout-btn:hover { background: rgba(248,113,113,0.2); }
48         .content { flex: 1; overflow-y: auto; padding: 24px; }
49
50         /* Views */
51         .view { display: none; }
52         .view.active { display: block; }
53
54         /* Stats */
55         .stats-grid { display: grid; grid-template-columns: repeat(4, 1fr); gap: 16px; margin-bottom: 24px;
}
56         .stat-card { background: var(--panel); padding: 20px; border-radius: 12px; border: 1px solid var(--
border); position: relative; overflow: hidden; }
57         .stat-card::before { content: ''; position: absolute; top: 0; left: 0; right: 0; height: 2px; }
58         .stat-card.blue::before { background: var(--accent); }

```

```

59     .stat-card.green::before { background: var(--green); }
60     .stat-card.amber::before { background: var(--amber); }
61     .stat-card.red::before { background: var(--red); }
62     .stat-label { font-size: 11px; color: var(--text2); text-transform: uppercase; letter-spacing:
0.05em; margin-bottom: 8px; }
63     .stat-value { font-size: 28px; font-weight: 600; }
64     .stat-value.blue { color: var(--accent); }
65     .stat-value.green { color: var(--green); }
66     .stat-value.amber { color: var(--amber); }
67     .stat-value.red { color: var(--red); }
68
69     /* Section header */
70     .section-head { display: flex; justify-content: space-between; align-items: center; margin-bottom:
16px; flex-wrap: wrap; gap: 10px; }
71     .section-title { font-size: 16px; font-weight: 600; }
72     .section-actions { display: flex; gap: 8px; align-items: center; }
73
74     /* Table */
75     .table-wrap { background: var(--panel); border-radius: 12px; border: 1px solid var(--border);
overflow: hidden; }
76     table { width: 100%; border-collapse: collapse; }
77     th { text-align: left; padding: 12px 16px; background: var(--bg3); color: var(--text2); font-size:
11px; font-weight: 600; text-transform: uppercase; letter-spacing: 0.04em; border-bottom: ...
78     td { padding: 12px 16px; border-top: 1px solid var(--border); font-size: 13px; }
79     tr:hover td { background: rgba(255,255,255,0.02); }
80     .empty-state { text-align: center; padding: 48px; color: var(--text2); font-size: 13px; }
81
82     /* Chips */
83     .chip { padding: 4px 10px; border-radius: 20px; font-size: 11px; font-weight: 600; display: inline-
block; }
84     .chip.pending, .chip.PENDING { background: var(--amber-dim); color: var(--amber); }
85     .chip.approved, .chip.APPROVED { background: var(--green-dim); color: var(--green); }
86     .chip.rejected, .chip.REJECTED { background: var(--red-dim); color: var(--red); }
87     .chip.cancelled, .chip.CANCELLED { background: rgba(139,146,168,0.12); color: var(--text2); }
88     .chip.sick, .chip.SICK { background: var(--red-dim); color: var(--red); }
89     .chip.casual, .chip.CASUAL { background: var(--accent-dim); color: var(--accent); }
90     .chip.maternity, .chip.MATERNITY { background: var(--violet-dim); color: var(--violet); }
91
92     /* Buttons */
93     .btn { padding: 8px 16px; border-radius: 6px; border: none; font-family: var(--font); font-size:
13px; font-weight: 500; cursor: pointer; transition: all 0.2s; }
94     .btn-primary { background: var(--accent); color: white; }
95     .btn-primary:hover { background: #6ba0ff; }
96     .btn-secondary { background: var(--bg3); color: var(--text2); border: 1px solid var(--border2); }
97     .btn-secondary:hover { color: var(--text); }
98     .btn-danger { background: var(--red-dim); color: var(--red); border: 1px solid
rgba(248,113,113,0.3); }
99     .btn-danger:hover { background: rgba(248,113,113,0.2); }
100    .btn-success { background: var(--green-dim); color: var(--green); border: 1px solid
rgba(54,211,153,0.3); }
101    .btn-success:hover { background: rgba(54,211,153,0.2); }
102    .btn-sm { padding: 4px 10px; font-size: 11px; }
103    .action-row { display: flex; gap: 6px; flex-wrap: wrap; }
104
105    /* Form */
106    .form-card { background: var(--panel); border-radius: 12px; border: 1px solid var(--border);
padding: 24px; max-width: 640px; }
107    .form-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 16px; }
108    .form-group { display: flex; flex-direction: column; gap: 6px; }
109    .form-group.full { grid-column: 1 / -1; }
110    label { font-size: 11px; font-weight: 600; color: var(--text2); text-transform: uppercase; letter-
spacing: 0.04em; }
111    input, select, textarea { padding: 10px 12px; background: var(--bg3); border: 1px solid var(--
border2); border-radius: 8px; color: var(--text); font-family: var(--font); font-size: 13px; tr...
112    input:focus, select:focus, textarea:focus { outline: none; border-color: var(--accent); }
113    input[readonly] { opacity: 0.5; cursor: not-allowed; }
114    select option { background: var(--bg3); }
115
116    /* Balance cards */
117    .balance-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(200px, 1fr)); gap:
16px; margin-bottom: 24px; }
118    .balance-card { background: var(--panel); border-radius: 12px; border: 1px solid var(--border);
padding: 20px; }
119    .balance-name { font-size: 12px; color: var(--text2); text-transform: uppercase; letter-spacing:
0.05em; margin-bottom: 4px; }
120    .balance-row { display: flex; align-items: baseline; gap: 6px; margin-bottom: 10px; }

```

```

121     .balance-big { font-size: 28px; font-weight: 600; }
122     .balance-of { font-size: 13px; color: var(--text2); }
123     .balance-bar { height: 4px; background: var(--border2); border-radius: 2px; overflow: hidden; }
124     .balance-fill { height: 100%; border-radius: 2px; transition: width 0.4s ease; }
125
126     /* Tabs */
127     .tab-bar { display: flex; gap: 2px; background: var(--bg3); border-radius: 8px; padding: 4px;
margin-bottom: 20px; width: fit-content; }
128     .tab-btn { padding: 7px 16px; border-radius: 6px; font-size: 12px; font-weight: 500; cursor:
pointer; color: var(--text2); border: none; background: transparent; font-family: var(--font); t...
129     .tab-btn.active { background: var(--panel); color: var(--text); }
130     .tab-btn:hover:not(.active) { color: var(--text); }
131
132     /* Modal */
133     .modal-overlay { display: none; position: fixed; top: 0; left: 0; right: 0; bottom: 0; background:
rgba(0,0,0,0.7); z-index: 1000; align-items: center; justify-content: center; }
134     .modal-overlay.open { display: flex; }
135     .modal { background: var(--panel); border-radius: 12px; border: 1px solid var(--border2); padding:
24px; width: 460px; max-width: 90%; }
136     .modal-title { font-size: 18px; font-weight: 600; margin-bottom: 16px; }
137     .modal-actions { display: flex; gap: 12px; margin-top: 20px; justify-content: flex-end; }
138
139     /* Alert */
140     .alert { position: fixed; top: 20px; right: 20px; padding: 12px 20px; border-radius: 8px; z-index:
1100; animation: slideIn 0.3s ease; font-size: 13px; font-weight: 500; }
141     .alert-success { background: var(--green-dim); color: var(--green); border: 1px solid
rgba(54,211,153,0.3); }
142     .alert-error { background: var(--red-dim); color: var(--red); border: 1px solid
rgba(248,113,113,0.3); }
143     @keyframes slideIn { from { transform: translateX(100%); opacity: 0; } to { transform:
translateX(0); opacity: 1; } }
144
145     /* Pending count indicator */
146     .pending-dot { width: 7px; height: 7px; border-radius: 50%; background: var(--amber); display:
inline-block; margin-left: 4px; animation: pulse 2s infinite; }
147     @keyframes pulse { 0%,100% { opacity: 1; } 50% { opacity: 0.4; } }
148 </style>
149 </head>
150 <body>
151 <div class="app">
152
153     <!-- Sidebar -->
154     <div class="sidebar">
155         <div class="logo">
156             <h2>E<span>M</span>S</h2>
157             <p>Leave Management</p>
158         </div>
159         <div style="flex:1; padding: 8px 0;">
160             <div class="nav-section">Overview</div>
161             <div class="nav-item active" onclick="showView('dashboard',this)">
162                 <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2"><rect x="3" y="3" width="7" height="7"/><rect x="14" y="3" width="7" height="7"/><...
163                 Dashboard
164             </div>
165             <div class="nav-section">My Leave</div>
166             <div class="nav-item" onclick="showView('apply',this)">
167                 <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2"><circle cx="12" cy="12" r="10"/><line x1="12" y1="8" x2="12" y2="16"/><line x1="8"...
168                 Apply for Leave
169             </div>
170             <div class="nav-item" onclick="showView('my-leaves',this)">
171                 <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2"><path d="M14 2H6a2 2 0 0 0 2 2V16a2 2 0 0 0 2 2h12a2 2 0 0 0 2 2V8z"/><polyline po...
172                 My Requests
173             </div>
174             <div class="nav-item" onclick="showView('balance',this)">
175                 <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2"><line x1="18" y1="20" x2="18" y2="10"/><line x1="12" y1="20" x2="12" y2="4"/><line...
176                 Leave Balance
177             </div>
178             <div class="nav-section">Admin</div>
179             <div class="nav-item" id="nav-pending" onclick="showView('pending',this)">
180                 <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2"><polyline points="9 11 12 14 22 4"/><path d="M21 12v7a2 2 0 0 1 2 2H5a2 2 0 0 1 2-...
181                 Pending Approvals
182                 <span class="badge-count" id="pending-count" style="display:none">0</span>

```

```

183         </div>
184         <div class="nav-item" onclick="showView('all-leaves',this)">
185             <svg width="15" height="15" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2"><path d="M17 21v-2a4 4 0 0 0-4-4H5a4 4 0 0 0-4 4v2"/><circle cx="9" cy="7" r="4"/>...
186             All Leave Requests
187         </div>
188     </div>
189     <div style="padding: 16px; border-top: 1px solid var(--border);">
190         <a href="/dashboard.html" style="color:var(--text2);font-size:12px;text-
decoration:none;display:flex;align-items:center;gap:8px;">
191             <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-
width="2"><polyline points="15 18 9 12 15 6"/></svg>
192             Back to Admin Console
193         </a>
194     </div>
195 </div>
196
197 <!-- Main -->
198 <div class="main-content">
199     <div class="topbar">
200         <div class="page-title" id="pageTitle">Dashboard</div>
201         <div class="topbar-right">
202             <div class="user-pill">
203                 <div class="user-avatar" id="user-initials">--</div>
204                 <span class="user-name" id="username-display">Loading...</span>
205             </div>
206             <button class="logout-btn" onclick="logout()">Logout</button>
207         </div>
208     </div>
209
210     <div class="content">
211
212         <!-- DASHBOARD VIEW -->
213         <div id="dashboard-view" class="view active">
214             <div class="stats-grid">
215                 <div class="stat-card blue"><div class="stat-label">Total Requests</div><div
class="stat-value blue" id="s-total">--</div></div>
216                 <div class="stat-card amber"><div class="stat-label">Pending</div><div class="stat-
value amber" id="s-pending">--</div></div>
217                 <div class="stat-card green"><div class="stat-label">Approved</div><div class="stat-
value green" id="s-approved">--</div></div>
218                 <div class="stat-card red"><div class="stat-label">Rejected</div><div class="stat-value
red" id="s-rejected">--</div></div>
219             </div>
220             <div class="section-head">
221                 <div class="section-title">Recent Leave Requests</div>
222                 <button class="btn btn-secondary btn-sm" onclick="loadDashboard()"> Refresh</button>
223             </div>
224             <div class="table-wrap">
225                 <table>
226                     <thead><tr><th>Employee ID</th><th>Leave Type</th><th>Start Date</th><th>End
Date</th><th>Days</th><th>Status</th></tr></thead>
227                     <tbody id="recent-leaves-body"></tbody>
228                 </table>
229             </div>
230         </div>
231
232         <!-- APPLY LEAVE VIEW -->
233         <div id="apply-view" class="view">
234             <div class="section-title" style="margin-bottom:20px;">Apply for Leave</div>
235             <div class="form-card">
236                 <div class="form-grid">
237                     <div class="form-group full">
238                         <label>Your Email</label>
239                         <input type="email" id="leave-email" readonly
placeholder="your.email@company.com">
240                     </div>
241                     <div class="form-group">
242                         <label>Leave Type</label>
243                         <select id="leave-type">
244                             <option value="SICK">Sick Leave</option>
245                             <option value="CASUAL">Casual Leave</option>
246                         </select>
247                         <div id="sick-leave-hint" style="display:none;margin-top:6px;font-
size:11px;color:var(--amber);background:var(--amber-dim);border:1px solid rgba(249,199,79,0.2);border-r...
248                         Sick leave is limited to <strong>1 day per month</strong>. If already used

```

this month, please select Unpaid Leave.

```
249             </div>
250             <div id="casual-leave-hint" style="display:none;margin-top:6px;font-
size:11px;color:var(--accent);background:var(--accent-dim);border:1px solid rgba(79,140,255,0.2);bord...
251                 You have <strong id="casual-balance-text">...</strong> casual leave days
available.
252             </div>
253         </div>
254         <div class="form-group">
255             <label>Priority</label>
256             <select id="leave-priority" style="color:var(--text2);">
257                 <option value="">- Optional -</option>
258                 <option value="NORMAL">Normal</option>
259                 <option value="URGENT">Urgent</option>
260             </select>
261         </div>
262         <div class="form-group">
263             <label>Start Date</label>
264             <input type="date" id="leave-start">
265         </div>
266         <div class="form-group">
267             <label>End Date</label>
268             <input type="date" id="leave-end">
269         </div>
270         <div class="form-group full">
271             <label>Reason</label>
272             <textarea id="leave-reason" rows="3" placeholder="Please provide a brief reason
for your leave request..."></textarea>
273         </div>
274     </div>
275     <div style="display:flex;gap:12px;margin-top:20px;">
276         <button class="btn btn-primary" onclick="applyLeave()">Submit Leave
Request</button>
277         <button class="btn btn-secondary" onclick="clearLeaveForm()">Clear</button>
278     </div>
279 </div>
280 </div>
281
282 <!-- MY LEAVES VIEW -->
283 <div id="my-leaves-view" class="view">
284     <div class="section-head">
285         <div class="section-title">My Leave Requests</div>
286         <button class="btn btn-secondary btn-sm" onclick="loadMyLeaves()"> Refresh</button>
287     </div>
288     <div class="table-wrap">
289         <table>
290             <thead><tr><th>ID</th><th>Type</th><th>Start Date</th><th>End
Date</th><th>Days</th><th>Status</th><th>Reason</th><th>Action</th></tr></thead>
291             <tbody id="my-leaves-list"></tbody>
292         </table>
293     </div>
294 </div>
295
296 <!-- LEAVE BALANCE VIEW -->
297 <div id="balance-view" class="view">
298     <div class="section-head">
299         <div class="section-title">Leave Balance</div>
300         <button class="btn btn-secondary btn-sm" onclick="loadBalance()"> Refresh</button>
301     </div>
302     <div class="balance-grid" id="balance-grid">
303         <div class="balance-card" style="color:var(--text2);text-
align:center;padding:40px;">Loading balance...</div>
304     </div>
305     <div class="section-head" style="margin-top:8px;">
306         <div class="section-title">Leave History</div>
307     </div>
308     <div class="table-wrap">
309         <table>
310             <thead><tr><th>ID</th><th>Type</th><th>Start Date</th><th>End
Date</th><th>Days</th><th>Status</th><th>Remarks</th></tr></thead>
311             <tbody id="balance-history-list"></tbody>
312         </table>
313     </div>
314 </div>
315
316 <!-- PENDING APPROVALS VIEW -->
```



```

317         <div id="pending-view" class="view">
318             <div class="section-head">
319                 <div class="section-title">Pending Approvals</div>
320                 <button class="btn btn-secondary btn-sm" onclick="loadPendingLeaves()">
Refresh</button>
321             </div>
322             <div class="table-wrap">
323                 <table>
324                     <thead><tr><th>ID</th><th>Employee
ID</th><th>Type</th><th>Start</th><th>End</th><th>Days</th><th>Reason</th><th>Actions</th></tr></thead>
325                     <tbody id="pending-leaves-list"></tbody>
326                 </table>
327             </div>
328         </div>
329
330         <!-- ALL LEAVES VIEW -->
331         <div id="all-leaves-view" class="view">
332             <div class="section-head">
333                 <div class="section-title">All Leave Requests</div>
334                 <div class="section-actions">
335                     <select id="status-filter" onchange="filterAllLeaves()" style="padding:6px
10px;font-size:12px;border-radius:6px;">
336                         <option value="">All Statuses</option>
337                         <option value="PENDING">Pending</option>
338                         <option value="APPROVED">Approved</option>
339                         <option value="REJECTED">Rejected</option>
340                         <option value="CANCELLED">Cancelled</option>
341                     </select>
342                     <button class="btn btn-secondary btn-sm" onclick="loadAllLeaves()">
Refresh</button>
343                 </div>
344             </div>
345             <div class="table-wrap">
346                 <table>
347                     <thead><tr><th>ID</th><th>Employee
ID</th><th>Type</th><th>Start</th><th>End</th><th>Days</th><th>Status</th><th>Actions</th></tr></thead>
348                     <tbody id="all-leaves-list"></tbody>
349                 </table>
350             </div>
351         </div>
352
353     </div><!-- /content -->
354 </div><!-- /main-content -->
355 </div><!-- /app -->
356
357 <!-- Approval Modal -->
358 <div class="modal-overlay" id="action-modal">
359     <div class="modal">
360         <div class="modal-title" id="action-modal-title">Review Leave Request</div>
361         <input type="hidden" id="action-leave-id">
362         <input type="hidden" id="action-type">
363         <div class="form-group" style="margin-bottom:12px;">
364             <label>Remarks (optional)</label>
365             <textarea id="action-remarks" rows="3" placeholder="Add remarks for the
employee..."></textarea>
366         </div>
367         <div class="modal-actions">
368             <button class="btn btn-secondary" onclick="closeModal('action-modal')">Cancel</button>
369             <button class="btn btn-primary" id="action-submit-btn"
onclick="submitApproval()">Approve</button>
370         </div>
371     </div>
372 </div>
373
374 <!-- Cancel Modal -->
375 <div class="modal-overlay" id="cancel-modal">
376     <div class="modal">
377         <div class="modal-title">Cancel Leave Request</div>
378         <p style="color:var(--text2);font-size:13px;margin-bottom:16px;">Are you sure you want to cancel
this leave request? This action cannot be undone.</p>
379         <input type="hidden" id="cancel-leave-id">
380         <div class="form-group" style="margin-bottom:12px;">
381             <label>Reason for cancellation (optional)</label>
382             <textarea id="cancel-remarks" rows="2" placeholder="Reason..."></textarea>
383         </div>
384         <div class="modal-actions">

```

```

385         <button class="btn btn-secondary" onclick="closeModal('cancel-modal')">Keep Request</button>
386         <button class="btn btn-danger" onclick="confirmCancel()">Yes, Cancel</button>
387     </div>
388 </div>
389 </div>
390
391 <script>
392     const API = '';
393     let currentUser = null;
394     let allLeavesCache = [];
395
396     // Auth
397     function getAuthHeader() {
398         const token = localStorage.getItem('jwt_token');
399         return token ? { 'Authorization': `Bearer ${token}` } : {};
400     }
401
402     async function apiFetch(path, opts = {}) {
403         const response = await fetch(`${API}${path}`, {
404             ...opts,
405             headers: { 'Content-Type': 'application/json', ...getAuthHeader(), ...opts.headers },
406             credentials: 'include'
407         });
408         if (response.status === 401) {
409             localStorage.removeItem('jwt_token');
410             localStorage.removeItem('auth_type');
411             showAlert('Session expired. Please login again.', 'error');
412             setTimeout(() => window.location.href = '/google.html', 1500);
413             throw new Error('Unauthorized');
414         }
415         if (!response.ok) {
416             const text = await response.text();
417             let msg = `Error ${response.status}`;
418             try {
419                 const body = JSON.parse(text);
420                 // handles: {error:...}, {message:...}, Spring's default {error:..., message:...}
421                 msg = body.error || body.message || body.detail || text || msg;
422             } catch(_) {
423                 msg = text || msg;
424             }
425             throw new Error(msg);
426         }
427         const text = await response.text();
428         return text ? JSON.parse(text) : {};
429     }
430
431     function logout() {
432         localStorage.removeItem('jwt_token');
433         localStorage.removeItem('auth_type');
434         window.location.href = '/google.html';
435     }
436
437     (async function initAuth() {
438         const token = localStorage.getItem('jwt_token');
439         const authType = localStorage.getItem('auth_type');
440         if (!token && authType !== 'session') { window.location.href = '/google.html'; return; }
441         try {
442             const employee = await apiFetch('/employee/me');
443             currentUser = employee;
444             const initials = (employee.name || employee.email || 'U').split(' ').map(w =>
w[0]).join('').substring(0,2).toUpperCase();
445             document.getElementById('user-initials').textContent = initials;
446             document.getElementById('username-display').textContent = employee.name || employee.email ||
'User';
447             document.getElementById('leave-email').value = employee.email || '';
448             loadDashboard();
449             loadPendingCount();
450         } catch(e) {
451             if (e.message !== 'Unauthorized') window.location.href = '/google.html';
452         }
453     })();
454
455     // Navigation
456     const viewTitles = {
457         dashboard: 'Leave Dashboard',
458         apply: 'Apply for Leave',

```

```

459     'my-leaves': 'My Leave Requests',
460     balance: 'Leave Balance',
461     pending: 'Pending Approvals',
462     'all-leaves': 'All Leave Requests'
463   };
464   const viewLoaders = {
465     dashboard: loadDashboard,
466     apply: () => {},
467     'my-leaves': loadMyLeaves,
468     balance: loadBalance,
469     pending: loadPendingLeaves,
470     'all-leaves': loadAllLeaves
471   };
472
473   function showView(name, el) {
474     document.querySelectorAll('.view').forEach(v => v.classList.remove('active'));
475     document.querySelectorAll('.nav-item').forEach(n => n.classList.remove('active'));
476     document.getElementById(name + '-view').classList.add('active');
477     if (el) el.classList.add('active');
478     document.getElementById('pageTitle').textContent = viewTitles[name] || name;
479     if (viewLoaders[name]) viewLoaders[name]();
480   }
481
482   // Helpers
483   function daysBetween(start, end) {
484     if (!start || !end) return '-';
485     const d = (new Date(end) - new Date(start)) / 86400000 + 1;
486     return d > 0 ? d : '-';
487   }
488
489   function chip(val) {
490     if (!val) return '-';
491     const cls = val.toLowerCase();
492     return `${val}</span>`;
493   }
494
495   function showAlert(msg, type = 'success') {
496     const el = document.createElement('div');
497     el.className = `alert alert-${type}`;
498     el.textContent = msg;
499     document.body.appendChild(el);
500     setTimeout(() => el.remove(), 3000);
501   }
502
503   function closeModal(id) {
504     document.getElementById(id).classList.remove('open');
505   }
506
507   document.querySelectorAll('.modal-overlay').forEach(m => {
508     m.addEventListener('click', e => { if (e.target === m) m.classList.remove('open'); });
509   });
510
511   // Dashboard
512   async function loadDashboard() {
513     try {
514       const [all, mine] = await Promise.all([
515         apiFetch('/leave_requests').catch(() => []),
516         apiFetch('/leave_requests/employee').catch(() => ({ requests: [] }))
517       ]);
518       const leaves = Array.isArray(all) ? all : [];
519       document.getElementById('s-total').textContent = leaves.length;
520       document.getElementById('s-pending').textContent = leaves.filter(l => l.status ===
521 'PENDING').length;
521       document.getElementById('s-approved').textContent = leaves.filter(l => l.status ===
522 'APPROVED').length;
522       document.getElementById('s-rejected').textContent = leaves.filter(l => l.status ===
523 'REJECTED').length;
523       const recent = leaves.slice(-8).reverse();
524       const tbody = document.getElementById('recent-leaves-body');
525       tbody.innerHTML = recent.length
526         ? recent.map(l => `<tr>
527 <td>${l.employeeName || '-'}</td>
528 <td>${chip(l.leaveType)}</td>
529 <td>${l.startDate || '-'}</td>
530 <td>${l.endDate || '-'}</td>
531 <td>${daysBetween(l.startDate, l.endDate)}</td>

```

```

532         <td>${chip(l.status)}</td>
533     </tr>`).join('')
534     : `<tr><td colspan="6" class="empty-state">No leave requests yet</td></tr>`;
535 } catch(e) { showAlert('Failed to load dashboard: ' + e.message, 'error'); }
536 }
537
538 async function loadPendingCount() {
539     try {
540         const leaves = await apiFetch('/leave_requests/pending').catch(() => []);
541         const count = Array.isArray(leaves) ? leaves.filter(l => l.status === 'PENDING').length : 0;
542         const el = document.getElementById('pending-count');
543         if (count > 0) { el.textContent = count; el.style.display = 'inline'; }
544         else el.style.display = 'none';
545     } catch(e) {}
546 }
547
548 // Apply Leave
549 async function applyLeave() {
550     const leaveType = document.getElementById('leave-type').value;
551     const startDate = document.getElementById('leave-start').value;
552     const endDate = document.getElementById('leave-end').value;
553     const reason = document.getElementById('leave-reason').value.trim();
554     if (!startDate || !endDate) { showAlert('Please select both start and end dates.', 'error'); return; }
555     if (new Date(endDate) < new Date(startDate)) { showAlert('End date cannot be before start date.',
556 'error'); return; }
557     try {
558         const data = await apiFetch('/leave_requests', {
559             method: 'POST',
560             body: JSON.stringify({ leaveType, startDate, endDate, reason })
561         });
562         if(data && data.warning) {
563             showAlert(data.warning, 'warning');
564         } else {
565             showAlert('Leave request submitted successfully!', 'success');
566             clearLeaveForm();
567             loadPendingCount();
568         } catch(e) {
569             let msg = e.message;
570             try {
571                 const parsed = JSON.parse(msg);
572                 if(parsed.error) msg = parsed.error;
573             } catch(_) {}
574             showAlert(msg, 'error');
575         }
576     }
577
578     function clearLeaveForm() {
579         document.getElementById('leave-start').value = '';
580         document.getElementById('leave-end').value = '';
581         document.getElementById('leave-reason').value = '';
582     }
583
584 // My Leaves
585 async function loadMyLeaves() {
586     try {
587         const data = await apiFetch('/leave_requests/employee');
588         const leaves = data.requests || (Array.isArray(data) ? data : []);
589         const tbody = document.getElementById('my-leaves-list');
590         tbody.innerHTML = leaves.length
591             ? leaves.map(l => `<tr>
592 <td>${l.leaveRequestId}</td>
593 <td>${chip(l.leaveType)}</td>
594 <td>${l.startDate || '-'}</td>
595 <td>${l.endDate || '-'}</td>
596 <td>${daysBetween(l.startDate, l.endDate)}</td>
597 <td>${chip(l.status)}</td>
598 <td style="max-width:180px;overflow:hidden;text-overflow:ellipsis;white-space:nowrap;"
599 title="${l.reason|| ''}">${l.reason || '-'}</td>
600 <td>
601     ${l.status === 'PENDING'
602         ? `<button class="btn btn-danger btn-sm"
603 onclick="openCancelModal(${l.leaveRequestId})">Cancel</button>`
604         : '-'}
605     </td>
606 </tr>`).join('')

```

```

605         : `<tr><td colspan="8" class="empty-state">No leave requests found</td></tr>`;
606     } catch(e) { showAlert('Failed to load your leaves: ' + e.message, 'error'); }
607 }
608
609 function openCancelModal(id) {
610     document.getElementById('cancel-leave-id').value = id;
611     document.getElementById('cancel-remarks').value = '';
612     document.getElementById('cancel-modal').classList.add('open');
613 }
614
615 async function confirmCancel() {
616     const id = document.getElementById('cancel-leave-id').value;
617     try {
618         await apiFetch(`/leave_requests/cancel/${id}`, { method: 'PUT' });
619         showAlert('Leave request cancelled.', 'success');
620         closeModal('cancel-modal');
621         loadMyLeaves();
622         loadPendingCount();
623     } catch(e) { showAlert('Failed to cancel: ' + e.message, 'error'); }
624 }
625
626 // Leave Balance
627 const balanceColors = {
628     SICK: '#f87171', CASUAL: '#4f8cff', MATERNITY: '#a78bfa', UNPAID: '#8b92a8'
629 };
630
631 function normalizeDashboardBalances(data) {
632     if (!data) return [];
633     if (data.leaveBalances) {
634         return Object.values(data.leaveBalances).map(balance => ({
635             type: balance.leaveType,
636             opening: Number(balance.opening ?? 0),
637             accrued: Number(balance.accrued ?? 0),
638             used: Number(balance.used ?? 0),
639             available: Number(balance.available ?? 0)
640         }));
641     }
642     return ['sick', 'casual', 'maternity']
643         .map(key => data[key])
644         .filter(Boolean)
645         .map(balance => ({
646             type: balance.leaveType,
647             opening: Number(balance.opening ?? 0),
648             accrued: Number(balance.accrued ?? 0),
649             used: Number(balance.used ?? 0),
650             available: Number(balance.available ?? 0)
651         }));
652 }
653
654 async function loadBalance() {
655     try {
656         const data = await apiFetch('/api/employees/me/leave-dashboard');
657         const grid = document.getElementById('balance-grid');
658         const balances = normalizeDashboardBalances(data);
659
660         if (balances.length) {
661             grid.innerHTML = balances.map(info => {
662                 const type = info.type;
663                 const used = info.used;
664                 const total = info.opening + info.accrued;
665                 const left = info.available;
666                 const pct = total ? Math.min(100, Math.round((used / total) * 100)) : 0;
667                 const color = balanceColors[type] || '#4f8cff';
668                 return `<div class="balance-card">
669                     <div class="balance-name">${type.replace(/_/g, ' ')}</div>
670                     <div class="balance-row">
671                         <span class="balance-big" style="color:${color}">${left}</span>
672                         <span class="balance-of"> / ${total} days left</span>
673                     </div>
674                     <div class="balance-bar"><div class="balance-fill"
675 style="width:${pct}%;background:${color}"></div></div>
676                 </div>`;
677             }).join('');
678         } else {
679             grid.innerHTML = `<div class="balance-card" style="color:var(--text2);grid-column:1/-
1;text-align:center;padding:32px;">

```

```

679         Leave balance data not available from the API. Check <code>/api/employees/me/leave-dashboard</code>
endpoint.
680     </div>`;
681     }
682
683     const myData = await apiFetch('/leave_requests/employee');
684     const myLeaves = myData.requests || (Array.isArray(myData) ? myData : []);
685     const histTbody = document.getElementById('balance-history-list');
686     histTbody.innerHTML = myLeaves.length
687         ? myLeaves.map(l => `<tr>
688             <td>${l.leaveRequestId}</td>
689             <td>${chip(l.leaveType)}</td>
690             <td>${l.startDate || '-'}</td>
691             <td>${l.endDate || '-'}</td>
692             <td>${daysBetween(l.startDate, l.endDate)}</td>
693             <td>${chip(l.status)}</td>
694             <td style="font-size:12px;color:var(--text2)">${l.managerRemarks || l.remarks || '-'}</td>
695         </tr>`).join('')
696         : `<tr><td colspan="7" class="empty-state">No leave history</td></tr>`;
697     } catch(e) { showAlert('Failed to load balance: ' + e.message, 'error'); }
698 }
699
700 // Pending Approvals
701 async function loadPendingLeaves() {
702     try {
703         const leaves = await apiFetch('/leave_requests/pending');
704         const arr = Array.isArray(leaves) ? leaves : [];
705         const count = arr.filter(l => l.status === 'PENDING').length;
706         const el = document.getElementById('pending-count');
707         if (count > 0) { el.textContent = count; el.style.display = 'inline'; }
708         else el.style.display = 'none';
709         const tbody = document.getElementById('pending-leaves-list');
710         tbody.innerHTML = arr.length
711             ? arr.map(l => `<tr>
712                 <td>${l.leaveRequestId}</td>
713                 <td>${l.employeeName || '-'}</td>
714                 <td>${chip(l.leaveType)}</td>
715                 <td>${l.startDate || '-'}</td>
716                 <td>${l.endDate || '-'}</td>
717                 <td>${daysBetween(l.startDate, l.endDate)}</td>
718                 <td style="max-width:140px;overflow:hidden;text-overflow:ellipsis;white-space:nowrap;"
title="${l.reason||''}">${l.reason || '-'}</td>
719                 <td class="action-row">
720                     <button class="btn btn-success btn-sm"
onclick="openActionModal(${l.leaveRequestId},'APPROVED')>Approve</button>
721                     <button class="btn btn-danger btn-sm"
onclick="openActionModal(${l.leaveRequestId},'REJECTED')>Reject</button>
722                 </td>
723             </tr>`).join('')
724             : `<tr><td colspan="8" class="empty-state">No pending leave requests</td></tr>`;
725     } catch(e) { showAlert('Failed to load pending: ' + e.message, 'error'); }
726 }
727
728 function openActionModal(id, action) {
729     document.getElementById('action-leave-id').value = id;
730     document.getElementById('action-type').value = action;
731     document.getElementById('action-remarks').value = '';
732     document.getElementById('action-modal-title').textContent = `${action === 'APPROVED' ? 'Approve' :
'Reject'} Leave Request #${id}`;
733     const btn = document.getElementById('action-submit-btn');
734     btn.textContent = action === 'APPROVED' ?
735
736     'Approve' : 'Reject';
737     btn.className = action === 'APPROVED' ? 'btn btn-success' : 'btn btn-danger';
738     document.getElementById('action-modal').classList.add('open');
739 }
740
741 async function submitApproval() {
742     const leaveRequestId = parseInt(document.getElementById('action-leave-id').value);
743     const action = document.getElementById('action-type').value;
744     const remarks = document.getElementById('action-remarks').value;
745     try {
746         await apiFetch('/leave_requests/approval', {
747             method: 'PUT',
748             body: JSON.stringify({ leaveRequestId, action, remarks })
749         });

```

```

750         showAlert(`Leave request ${action.toLowerCase()} successfully!`, 'success');
751         closeModal('action-modal');
752         loadPendingLeaves();
753         loadDashboard();
754     } catch(e) { showAlert('Failed: ' + e.message, 'error'); }
755 }
756
757 // All Leaves
758 async function loadAllLeaves() {
759     try {
760         const leaves = await apiFetch('/leave_requests');
761         allLeavesCache = Array.isArray(leaves) ? leaves : [];
762         renderAllLeaves(allLeavesCache);
763     } catch(e) { showAlert('Failed to load all leaves: ' + e.message, 'error'); }
764 }
765
766 function filterAllLeaves() {
767     const status = document.getElementById('status-filter').value;
768     const filtered = status ? allLeavesCache.filter(l => l.status === status) : allLeavesCache;
769     renderAllLeaves(filtered);
770 }
771
772 function renderAllLeaves(leaves) {
773     const tbody = document.getElementById('all-leaves-list');
774     tbody.innerHTML = leaves.length
775         ? leaves.map(l => `<tr>
776             <td>${l.leaveRequestId}</td>
777             <td>${l.employeeName || '-'}</td>
778             <td>${chip(l.leaveType)}</td>
779             <td>${l.startDate || '-'}</td>
780             <td>${l.endDate || '-'}</td>
781             <td>${daysBetween(l.startDate, l.endDate)}</td>
782             <td>${chip(l.status)}</td>
783             <td class="action-row">
784                 ${l.status === 'PENDING'
785                     ? `<button class="btn btn-success btn-sm"
786 onlick="openActionModal(${l.leaveRequestId}, 'APPROVED')">Approve</button>
787 <button class="btn btn-danger btn-sm"
788 onlick="openActionModal(${l.leaveRequestId}, 'REJECTED')">Reject</button>`
789                     : '-'}
790             </td>
791         `).join('')
792         : `<tr><td colspan="8" class="empty-state">No leave requests found</td></tr>`;
793 }
794
795 document.getElementById('leave-type').addEventListener('change', async function() {
796     document.getElementById('sick-leave-hint').style.display =
797         this.value === 'SICK' ? 'block' : 'none';
798
799     const casualHint = document.getElementById('casual-leave-hint');
800     if(this.value === 'CASUAL') {
801         casualHint.style.display = 'block';
802         document.getElementById('casual-balance-text').textContent = 'loading...';
803         try {
804             const dashboard = await apiFetch('/api/employees/me/leave-dashboard');
805             const balances = dashboard?.leaveBalances || {};
806             const cas = balances['CASUAL'] || balances['casual'] || {};
807             const available = Number(cas.available ?? cas.availableBalance ?? 0);
808             document.getElementById('casual-balance-text').textContent =
809                 available > 0 ? `${available}` : '0 (no balance remaining)';
810         } catch(e) {
811             document.getElementById('casual-balance-text').textContent = 'unavailable';
812         }
813     } else {
814         casualHint.style.display = 'none';
815     }
816 });
817 </script>
818 </body>
819 </html>

```

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4     <meta charset="UTF-8">
5     <title>Title</title>
6 </head>
7 <body>
8 <h1>hi</h1>
9 </body>
10 </html>
```



```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>EMS – Manager Console</title>
7      <link
href="https://fonts.googleapis.com/css2?family=Plus+Jakarta+Sans:wght@300;400;500;600;700&family=IBM+Plex+Mono:w
ght@400;500&display=swap" rel="stylesheet">
8      <link href="https://cdn.jsdelivr.net/npm/fullcalendar@6.1.8/index.global.min.css" rel="stylesheet">
9      <script src="https://cdn.jsdelivr.net/npm/fullcalendar@6.1.8/index.global.min.js"></script>
10     <style>
11         *{margin:0;padding:0;box-sizing:border-box}
12         :root{
13             --bg:#0a0b0f;--bg2:#0f1117;--bg3:#14171f;
14             --panel:#171c28;--panel2:#1d2336;
15             --border:rgba(167,139,250,0.08);--border2:rgba(167,139,250,0.18);
16             --text:#e8e0f5;--text2:#8b82a8;--text3:#504d68;
17             --accent:#a78bfa;--accent-dim:rgba(167,139,250,0.1);
18             --green:#34d399;--green-dim:rgba(52,211,153,0.1);
19             --amber:#fbbf24;--amber-dim:rgba(251,191,36,0.1);
20             --red:#f87171;--red-dim:rgba(248,113,113,0.1);
21             --blue:#60a5fa;--blue-dim:rgba(96,165,250,0.1);
22             --font:'Plus Jakarta Sans',sans-serif;--mono:'IBM Plex Mono',monospace;
23             --sidebar:250px;
24         }
25         body{font-family:var(--font);background:var(--bg);color:var(--text);font-
size:14px;overflow:hidden;height:100vh}
26         .app{display:flex;height:100vh}
27
28         /* SIDEBAR */
29         .sidebar{width:var(--sidebar);background:var(--bg2);border-right:1px solid var(--
border);display:flex;flex-direction:column;flex-shrink:0;position:relative;overflow:hidden}
30         .sidebar::after{content:'';position:absolute;bottom:0;left:0;right:0;height:200px;background:linear-
gradient(to top,rgba(167,139,250,0.04),transparent);pointer-events:none}
31         .logo{padding:22px 20px;border-bottom:1px solid var(--border);position:relative}
32         .logo-row{display:flex;align-items:center;gap:10px;margin-bottom:4px}
33         .logo-icon{width:34px;height:34px;background:linear-gradient(135deg,var(--accent),#7c3aed);border-
radius:10px;display:flex;align-items:center;justify-content:center;font-weight:700;font-siz...
34         .logo h2{font-size:16px;font-weight:700;color:var(--text)}
35         .logo p{font-size:11px;color:var(--text3);font-family:var(--mono)}
36         .role-badge{display:inline-flex;align-items:center;gap:6px;padding:4px 12px;background:linear-
gradient(90deg,rgba(167,139,250,0.15),rgba(124,58,237,0.08));border:1px solid rgba(167,139,250,...
37         .role-dot{width:6px;height:6px;background:var(--accent);border-radius:50%;animation:glow 2s
infinite}
38         @keyframes glow{0%,100%{box-shadow:0 0 4px var(--accent)}50%{box-shadow:0 0 10px var(--accent)},0 0
20px rgba(167,139,250,0.4)}}
39         .nav{flex:1;padding:10px 0;overflow-y:auto}
40         .nav-section{padding:12px 20px 6px;font-size:9px;font-weight:700;color:var(--text3);letter-
spacing:2px;text-transform:uppercase}
41         .nav-item{display:flex;align-items:center;gap:10px;padding:10px 20px;margin:1px 8px;border-
radius:8px;cursor:pointer;transition:all 0.2s;color:var(--text2);font-size:13px;font-weight:500}
42         .nav-item:hover{background:var(--accent-dim);color:var(--accent)}
43         .nav-item.active{background:linear-gradient(90deg,var(--accent-
dim),rgba(124,58,237,0.06));color:var(--accent);border:1px solid rgba(167,139,250,0.2)}
44         .nav-badge{margin-left:auto;background:var(--red);color:#fff;font-size:10px;font-
weight:700;padding:2px 7px;border-radius:10px;min-width:18px;text-align:center}
45         .nav-badge.hidden{display:none}
46         .sidebar-footer{padding:16px 20px;border-top:1px solid var(--border)}
47         .user-card{display:flex;align-items:center;gap:10px}
48         .avatar{width:34px;height:34px;background:linear-gradient(135deg,#7c3aed,var(--accent));border-
radius:50%;display:flex;align-items:center;justify-content:center;font-size:14px;font-weight:7...
49         .user-info-side .name{font-size:12px;font-weight:600;color:var(--text)}
50         .user-info-side .role-text{font-size:10px;color:var(--accent);font-family:var(--mono);text-
transform:uppercase;letter-spacing:0.8px}
51
52         /* MAIN */
53         .main{flex:1;display:flex;flex-direction:column;overflow:hidden}
54         .topbar{background:var(--bg2);padding:14px 24px;display:flex;justify-content:space-between;align-
items:center;border-bottom:1px solid var(--border);flex-shrink:0}
55         .page-title{font-size:16px;font-weight:700;color:var(--text)}
56         .topbar-right{display:flex;align-items:center;gap:12px}

```

```

57     .logout-btn{padding:7px 16px;background:transparent;border:1px solid var(--border2);border-
radius:6px;color:var(--text2);font-family:var(--font);font-size:12px;cursor:pointer;transition:all...
58     .logout-btn:hover{background:var(--red-dim);border-color:var(--red);color:var(--red)}
59     .content{flex:1;overflow-y:auto;padding:24px}
60     .view{display:none}
61     .view.active{display:block;animation:fadeIn 0.3s ease}
62     @keyframes fadeIn{from{opacity:0;transform:translateY(8px)}to{opacity:1;transform:translateY(0)}}
63     .notif-wrapper { position: relative; cursor: pointer; }
64
65     .notif-badge {
66         position: absolute; top: -5px; right: -6px;
67         background: #e53e3e; color: white;
68         font-size: 10px; font-weight: 600;
69         min-width: 16px; height: 16px;
70         border-radius: 50%; display: flex;
71         align-items: center; justify-content: center;
72         padding: 0 3px;
73     }
74
75     .notif-panel {
76         position: absolute; top: 48px; right: 16px;
77         width: 320px; max-height: 400px; overflow-y: auto;
78         background: var(--panel); border: 1px solid #e2e8f0;
79         border-radius: 10px; box-shadow: 0 4px 16px rgba(0,0,0,0.1);
80         z-index: 999;
81     }
82
83     .notif-header {
84         padding: 12px 16px; font-weight: 600;
85         border-bottom: 1px solid #e2e8f0;
86     }
87
88     .notif-item {
89         padding: 12px 16px; font-size: 14px;
90         border-bottom: 1px solid #f1f5f9;
91     }
92
93     .notif-item.unread { background: #eff6ff; }
94     .notif-time { font-size: 11px; color: #94a3b8; margin-top: 4px; }
95
96     /* STATS */
97     .stats-grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(160px,1fr));gap:16px;margin-
bottom:24px}
98     .stat-card{background:var(--panel);border:1px solid var(--border);border-
radius:14px;padding:20px;position:relative;overflow:hidden;transition:all 0.2s;cursor:default}
99     .stat-card:hover{border-color:var(--border2);transform:translateY(-1px)}
100     .stat-card::before{content:'';position:absolute;top:0;left:0;right:0;height:3px}
101     .stat-card.purple::before{background:linear-gradient(90deg,#7c3aed,var(--accent))}
102     .stat-card.green::before{background:var(--green)}
103     .stat-card.amber::before{background:var(--amber)}
104     .stat-card.blue::before{background:var(--blue)}
105     .stat-label{font-size:11px;color:var(--text3);text-transform:uppercase;letter-spacing:0.8px;margin-
bottom:10px;font-weight:600}
106     .stat-value{font-size:30px;font-weight:700;color:var(--text);font-family:var(--mono);line-height:1}
107     .stat-sub{font-size:11px;color:var(--text3);margin-top:6px}
108
109     /* SECTION */
110     .section{background:var(--panel);border:1px solid var(--border);border-radius:14px;margin-
bottom:20px;overflow:hidden}
111     .section-head{padding:16px 20px;border-bottom:1px solid var(--border);display:flex;justify-
content:space-between;align-items:center}
112     .section-title{font-size:14px;font-weight:700;color:var(--text)}
113     .section-sub{font-size:12px;color:var(--text3);margin-top:2px}
114
115     /* TABLE */
116     .tablewrap{overflow-x:auto}
117     table{width:100%;border-collapse:collapse}
118     th{padding:10px 16px;text-align:left;font-size:10px;color:var(--text3);font-weight:700;letter-
spacing:1px;text-transform:uppercase;background:var(--bg3);border-bottom:1px solid var(--border)}
119     td{padding:12px 16px;border-bottom:1px solid var(--border);font-size:13px;color:var(--text2)}
120     tr:last-child td{border-bottom:none}
121     tr:hover td{background:rgba(167,139,250,0.03);color:var(--text)}
122
123     /* CHIPS */
124     .chip{display:inline-flex;align-items:center;padding:3px 10px;border-radius:20px;font-
size:11px;font-weight:600}

```

```

125     .chip.pending{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(251,191,36,0.2)}
126     .chip.approved{background:var(--green-dim);color:var(--green);border:1px solid
rgba(52,211,153,0.2)}
127     .chip.rejected{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
128     .chip.active{background:var(--green-dim);color:var(--green);border:1px solid rgba(52,211,153,0.2)}
129     .chip.open { background: var(--accent-dim); color: var(--accent); }
130     .chip.sick{background:var(--red-dim);color:var(--red)}
131     .chip.casual{background:var(--amber-dim);color:var(--amber)}
132     .chip.maternity{background:var(--accent-dim);color:var(--accent)}
133     .chip.employee{background:var(--blue-dim);color:var(--blue)}
134     .chip.manager{background:var(--accent-dim);color:var(--accent)}
135     .chip.under_repair { background: var(--amber-dim); color: var(--amber); }
136     .chip.unpaid { background: rgba(139,92,246,0.15); color: #a78bfa; border: 1px solid
rgba(139,92,246,0.2); }
137     .chip.assigned { background: var(--green-dim); color: var(--green);}
138     .chip.condemned { background: var(--red-dim); color: var(--red);}
139
140     /* BUTTONS */
141     .btn{padding:8px 16px;border-radius:8px;border:none;font-family:var(--font);font-size:12px;font-
weight:600;cursor:pointer;transition:all 0.2s}
142     .btn-primary{background:var(--accent);color:#fff}
143     .btn-primary:hover{background:#7c3aed;box-shadow:0 4px 12px rgba(167,139,250,0.3)}
144     .btn-approve{background:var(--green-dim);color:var(--green);border:1px solid rgba(52,211,153,0.2)}
145     .btn-approve:hover{background:var(--green);color:#fff}
146     .btn-reject{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
147     .btn-reject:hover{background:var(--red);color:#fff}
148     .btn-secondary{background:var(--panel2);color:var(--text2);border:1px solid var(--border)}
149     .btn-secondary:hover{border-color:var(--border2);color:var(--text)}
150     .btn-sm{padding:5px 12px;font-size:11px}
151     .action-row{display:flex;gap:6px}
152
153     /* FORM */
154     .form-card{padding:24px}
155     .form-grid{display:grid;grid-template-columns:1fr 1fr;gap:16px}
156     .form-group{display:flex;flex-direction:column;gap:6px}
157     .form-group.full{grid-column:1/-1}
158     label{font-size:10px;font-weight:700;color:var(--text3);text-transform:uppercase;letter-
spacing:0.8px}
159     input,select,textarea{background:var(--bg3);border:1px solid var(--border);border-
radius:8px;padding:10px 14px;color:var(--text);font-family:var(--font);font-size:13px;transition:border-col...
160     input:focus,select:focus,textarea:focus{outline:none;border-color:var(--accent)}
161     input::placeholder{color:var(--text3)}
162     select option{background:var(--bg2)}
163     .form-actions{display:flex;gap:10px;margin-top:20px}
164
165     /* ALERT */
166     .alert{position:fixed;top:20px;right:20px;padding:12px 20px;border-radius:10px;z-
index:1100;animation:slideIn 0.3s ease;font-size:13px;font-weight:500}
167     .alert-success{background:var(--green-dim);color:var(--green);border:1px solid
rgba(52,211,153,0.3)}
168     .alert-error{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.3)}
169     .alert-warning{background:var(--amber-dim);color:var(--amber);border:1px solid
rgba(251,191,36,0.3)}
170     @keyframes slideIn{from{transform:translateX(120%);opacity:0}to{transform:translateX(0);opacity:1}}
171     .empty-state{text-align:center;color:var(--text3);font-style:italic;padding:30px}
172
173     /* GRID DASHBOARD */
174     .dash-grid{display:grid;grid-template-columns:1fr 1fr;gap:20px;margin-bottom:0}
175
176     /* SEARCH */
177     .search-bar{display:flex;align-items:center;gap:10px;margin-bottom:16px}
178     .search-input{flex:1;background:var(--bg3);border:1px solid var(--border);border-
radius:8px;padding:9px 14px;color:var(--text);font-family:var(--font);font-size:13px}
179     .search-input:focus{outline:none;border-color:var(--accent)}
180     .search-input::placeholder{color:var(--text3)}
181
182     /* ===== */
183     /* CALENDAR VIEW STYLES */
184     /* ===== */
185
186     /* Leave balance summary cards above the calendar */
187     .leave-balance-grid {
188         display: grid;
189         grid-template-columns: repeat(auto-fit, minmax(160px, 1fr));
190         gap: 14px;
191         padding: 20px;

```

```

192         border-bottom: 1px solid var(--border);
193     }
194     .leave-bal-card {
195         background: var(--bg3);
196         border: 1px solid var(--border);
197         border-radius: 10px;
198         padding: 16px 18px;
199         display: flex;
200         flex-direction: column;
201         gap: 6px;
202         position: relative;
203         overflow: hidden;
204         transition: border-color 0.2s;
205     }
206     .leave-bal-card:hover { border-color: var(--border2); }
207     .leave-bal-card::before {
208         content: '';
209         position: absolute;
210         left: 0; top: 0; bottom: 0;
211         width: 3px;
212         border-radius: 3px 0 0 3px;
213     }
214     .leave-bal-card.sick::before { background: var(--red); }
215     .leave-bal-card.casual::before { background: var(--amber); }
216     .leave-bal-card.mat::before { background: var(--accent); }
217     .leave-bal-card.total::before { background: var(--green); }
218     .lbc-icon { font-size: 18px; }
219     .lbc-label { font-size: 10px; color: var(--text3); text-transform: uppercase; letter-spacing: 1px;
font-weight: 600; }
220     .lbc-count { font-size: 24px; font-weight: 700; color: var(--text); font-family: var(--mono); line-
height: 1; }
221     .lbc-sub { font-size: 11px; color: var(--text3); }
222
223     /* Legend row */
224     .cal-legend {
225         display: flex;
226         align-items: center;
227         gap: 20px;
228         padding: 12px 20px;
229         border-bottom: 1px solid var(--border);
230         flex-wrap: wrap;
231     }
232     .legend-label { font-size: 11px; color: var(--text3); font-weight: 600; text-transform: uppercase;
letter-spacing: 0.8px; margin-right: 4px; }
233     .legend-item { display: flex; align-items: center; gap: 6px; font-size: 12px; color: var(--text2);
}
234     .legend-dot { width: 10px; height: 10px; border-radius: 50%; flex-shrink: 0; }
235
236     /* FullCalendar dark-theme overrides */
237     #employee-calendar {
238         padding: 16px 20px 20px;
239     }
240     /* Toolbar */
241     #employee-calendar .fc .fc-toolbar-title {
242         font-family: var(--font);
243         font-size: 15px;
244         font-weight: 600;
245         color: var(--text);
246     }
247     #employee-calendar .fc .fc-button {
248         background: var(--panel2);
249         border: 1px solid var(--border2);
250         color: var(--text2);
251         font-family: var(--font);
252         font-size: 12px;
253         font-weight: 600;
254         border-radius: 6px;
255         padding: 5px 12px;
256         transition: all 0.2s;
257         box-shadow: none;
258         text-transform: capitalize;
259     }
260     #employee-calendar .fc .fc-button:hover,
261     #employee-calendar .fc .fc-button-active,
262     #employee-calendar .fc .fc-button:not(:disabled):active {
263         background: var(--accent-dim);

```

```

264         border-color: var(--accent);
265         color: var(--accent);
266         box-shadow: none;
267     }
268     #employee-calendar .fc .fc-button:focus { box-shadow: none; outline: none; }
269
270     /* Grid */
271     #employee-calendar .fc-theme-standard td,
272     #employee-calendar .fc-theme-standard th,
273     #employee-calendar .fc-theme-standard .fc-scrollgrid {
274         border-color: var(--border);
275     }
276     #employee-calendar .fc .fc-col-header-cell-cushion {
277         font-family: var(--font);
278         font-size: 11px;
279         font-weight: 600;
280         color: var(--text3);
281         text-transform: uppercase;
282         letter-spacing: 0.8px;
283         padding: 8px 4px;
284     }
285     #employee-calendar .fc .fc-daygrid-day-number {
286         font-family: var(--mono);
287         font-size: 12px;
288         color: var(--text3);
289         padding: 6px 8px;
290     }
291     #employee-calendar .fc .fc-daygrid-day.fc-day-today {
292         background: var(--accent-dim) !important;
293     }
294     #employee-calendar .fc .fc-daygrid-day.fc-day-today .fc-daygrid-day-number {
295         color: var(--accent);
296         font-weight: 700;
297     }
298     #employee-calendar .fc .fc-daygrid-day:hover {
299         background: rgba(56,189,248,0.03);
300     }
301     #employee-calendar .fc-daygrid-day-frame {
302         background: var(--bg3);
303     }
304     #employee-calendar .fc .fc-day-other .fc-daygrid-day-frame {
305         background: var(--bg2);
306     }
307
308     /* Events */
309     #employee-calendar .fc-event {
310         border: none !important;
311         border-radius: 4px;
312         font-family: var(--font);
313         font-size: 11px;
314         font-weight: 600;
315         padding: 2px 6px;
316         cursor: pointer;
317     }
318     #employee-calendar .fc-event-title { font-weight: 600; }
319
320     /* Event tooltip */
321     .cal-tooltip {
322         position: fixed;
323         background: var(--panel2);
324         border: 1px solid var(--border2);
325         border-radius: 8px;
326         padding: 10px 14px;
327         font-size: 12px;
328         color: var(--text);
329         z-index: 9999;
330         pointer-events: none;
331         max-width: 200px;
332         box-shadow: 0 8px 24px rgba(0,0,0,0.4);
333     }
334     .cal-tooltip .tt-title { font-weight: 600; margin-bottom: 4px; font-size: 13px; }
335     .cal-tooltip .tt-row { color: var(--text2); margin-top: 2px; }
336
337     /* balance loading state */
338     .lbc-count.loading { color: var(--text3); font-size: 16px; }
339 </style>

```

```
340 </head>
341 <body>
342 <div class="app">
343   <!-- SIDEBAR -->
344   <div class="sidebar">
345     <div class="logo">
346       <div class="logo-row">
347         <div class="logo-icon">M</div>
348         <h2>EMS Console</h2>
349       </div>
350       <p>Management Portal</p>
351       <div class="role-badge"><span class="role-dot"></span>Manager</div>
352     </div>
353     <nav class="nav">
354       <div class="nav-section">Overview</div>
355       <div class="nav-item active" onclick="showView('dashboard',this)">
356         <span></span> Dashboard
357       </div>
358       <div class="nav-section">Team</div>
359       <div class="nav-item" onclick="showView('employees',this)"><span></span> All Employees</div>
360       <div class="nav-item" onclick="showView('add-employee',this)"><span></span> Add Employee</div>
361       <div class="nav-item" onclick="showView('promote-users',this)"><span></span> Promote Users
362       <span class="nav-badge" id="users-badge">0</span></div>
363       <div class="nav-section">Leave Management</div>
364       <div class="nav-item" onclick="showView('pending-leaves',this)">
365         <span></span> Pending Approvals
366         <span class="nav-badge" id="pending-badge">0</span>
367       </div>
368       <div class="nav-item" onclick="showView('all-leaves',this)"><span></span> All Leave
369       Requests</div>
370       <div class="nav-section">Personal</div>
371       <div class="nav-item" onclick="showView('apply-leave',this)"><span></span> Apply Leave</div>
372       <div class="nav-item" onclick="showView('my-leaves',this)"><span></span> My Leaves</div>
373       <div class="nav-item" onclick="showView('my-calendar',this)"><span class="icon"></span> My
374       Calendar</div>
375       <div class="nav-item" onclick="showView('profile',this)"><span></span> My Profile</div>
376       <div class="nav-item" onclick="showView('my-subscriptions',this)"><span></span> My
377       Subscriptions</div>
378       <div class="nav-item" onclick="showView('add-subscription',this)"><span></span> Subscribe to
379       Meal</div>
380       <div class="nav-item" onclick="showView('restaurants',this)"><span></span> Restaurants</div>
381       <div class="nav-item" onclick="showView('my-devices',this)"><span></span> My Devices</div>
382       <div class="nav-item" onclick="showView('service-requests', this)"><span class="icon"></span>
383       Service Requests</div>
384       <div class="nav-item" onclick="showView('two-factor',this)"><span class="icon"></span> Two-
385       Factor Auth</div>
386       <div class="nav-item" onclick="showView('search-employee',this)"><span></span> Search
387       Employee</div>
388     </nav>
389     <div class="sidebar-footer">
390       <div class="user-card">
391         <div class="avatar" id="avatar-initials">M</div>
392         <div class="user-info-side">
393           <div class="name" id="sidebar-name">Manager</div>
394           <div class="role-text">Manager</div>
395         </div>
396       </div>
397     </div>
398   </div>
399   <!-- MAIN -->
400   <div class="main">
401     <div class="topbar">
402       <div class="page-title" id="pageTitle">Dashboard</div>
403       <!-- Bell icon with badge -->
404       <div class="notif-wrapper" onclick="toggleNotifPanel()">
405         <svg width="22" height="22" viewBox="0 0 24 24" fill="none"
406           stroke="currentColor" stroke-width="2" stroke-linecap="round">
407           <path d="M18 8A6 6 0 0 0 6 8c0 7-3 9-3 9h18s-3-2-3-9"/>
408           <path d="M13.73 21a2 2 0 0 1-3.46 0"/>
409         </svg>
410         <span id="notif-badge" class="notif-badge" style="display:none"></span>
411       </div>
412       <!-- Dropdown panel -->
413       <div id="notif-panel" class="notif-panel" style="display:none">
```

```

408         <div class="notif-header">Notifications</div>
409         <div id="notif-list"></div>
410     </div>
411     <div class="topbar-right">
412         <span style="font-size:12px;color:var(--text3);font-family:var(--mono)" id="topbar-
email"></span>
413         <button class="logout-btn" onclick="logout()">Sign Out</button>
414     </div>
415 </div>
416 <div class="content">
417
418     <!-- DASHBOARD -->
419     <div id="dashboard-view" class="view active">
420         <div class="stats-grid">
421             <div class="stat-card purple">
422                 <div class="stat-label">Total Employees</div>
423                 <div class="stat-value" id="stat-total">--</div>
424                 <div class="stat-sub">in your organization</div>
425             </div>
426             <div class="stat-card green">
427                 <div class="stat-label">Active Now</div>
428                 <div class="stat-value" id="stat-active">--</div>
429                 <div class="stat-sub">currently working</div>
430             </div>
431             <div class="stat-card amber">
432                 <div class="stat-label">Pending Approvals</div>
433                 <div class="stat-value" id="stat-pending">--</div>
434                 <div class="stat-sub">need your action</div>
435             </div>
436             <div class="stat-card blue">
437                 <div class="stat-label">On Leave Today</div>
438                 <div class="stat-value" id="stat-on-leave">--</div>
439                 <div class="stat-sub">employees absent</div>
440             </div>
441         </div>
442         <div class="dash-grid">
443             <div class="section">
444                 <div class="section-head">
445                     <div><div class="section-title"> Needs Attention</div><div class="section-
sub">Pending leave requests</div></div>
446                     <button class="btn btn-secondary btn-sm" onclick="showView('pending-
leaves',null)">View All </button>
447                 </div>
448                 <div class="tablewrap">
449                     <table>
450
451                         <thead><tr><th>Employee</th><th>Type</th><th>Dates</th><th>Action</th></tr></thead>
452                         <tbody id="dash-pending"><tr><td colspan="4" class="empty-
state">Loading...</td></tr></tbody>
453                     </table>
454                 </div>
455             </div>
456             <div class="section">
457                 <div class="section-head">
458                     <div><div class="section-title"> Team Overview</div><div class="section-
sub">Recent employees</div></div>
459                     <button class="btn btn-secondary btn-sm"
onclick="showView('employees',null)">View All </button>
460                 </div>
461                 <div class="tablewrap">
462                     <table>
463                         <thead><tr><th>Name</th><th>Dept</th><th>Status</th></tr></thead>
464                         <tbody id="dash-employees"><tr><td colspan="3" class="empty-
state">Loading...</td></tr></tbody>
465                     </table>
466                 </div>
467             </div>
468         </div>
469
470     <!-- ALL EMPLOYEES -->
471     <div id="employees-view" class="view">
472         <div class="section">
473             <div class="section-head">
474                 <div><div class="section-title">All Employees</div><div class="section-sub">Manage
your team</div></div>

```

```

475         <button class="btn btn-secondary btn-sm" onclick="loadEmployees()">
Refresh</button>
476     </div>
477     <div style="padding:16px 20px;border-bottom:1px solid var(--border)">
478         <div class="search-bar">
479             <input class="search-input" id="emp-search" placeholder="Search by name, email
or department..." oninput="filterEmployees()">
480         </div>
481     </div>
482     <div class="tablewrap">
483         <table>
484
<thead><tr><th>ID</th><th>Name</th><th>Email</th><th>Department</th><th>Role</th><th>Status</th><th>Actions</th>
</tr></thead>
485         <tbody id="employees-list"><tr><td colspan="7" class="empty-
state">Loading...</td></tr></tbody>
486     </table>
487 </div>
488 </div>
489 </div>
490
491 <!-- ADD EMPLOYEE -->
492 <div id="add-employee-view" class="view">
493     <div class="section">
494         <div class="section-head"><div class="section-title">Add New Employee</div></div>
495         <div class="form-card">
496             <div class="form-grid">
497                 <div class="form-group"><label>Full Name</label><input id="emp-name"
placeholder="John Doe"></div>
498                 <div class="form-group"><label>Email</label><input type="email" id="emp-email"
placeholder="john@company.com"></div>
499                 <div class="form-group"><label>Password</label><input type="password" id="emp-
pass" placeholder="Temporary password"></div>
500                 <div class="form-group"><label>Gender</label>
501                 <select id="emp-gender">
502                     <option value="M">Male</option>
503                     <option value="F">Female</option>
504                     <option value="O">Other</option>
505                 </select>
506             </div>
507             <div class="form-group"><label>Timezone</label>
508             <select id="emp-timezone">
509                 <option value="UTC">UTC</option>
510                 <option value="Asia/Kolkata">Asia/Kolkata (IST)</option>
511                 <option value="America/New_York">America/New_York (EST)</option>
512                 <option value="America/Los_Angeles">America/Los_Angeles (PST)</option>
513                 <option value="Europe/London">Europe/London (GMT)</option>
514                 <option value="Asia/Singapore">Asia/Singapore (SGT)</option>
515             </select>
516             </div>
517         </div>
518         <div class="form-actions">
519             <button class="btn btn-primary" onclick="addEmployee()">Add Employee</button>
520             <button class="btn btn-secondary" onclick="clearEmpForm()">Clear</button>
521         </div>
522     </div>
523 </div>
524 </div>
525
526 <!-- SEARCH EMPLOYEE -->
527 <div id="search-employee-view" class="view">
528     <div class="section">
529         <div class="section-head">
530             <div><div class="section-title"> Search Employee</div><div class="section-sub">Look
up any employee across the organisation</div></div>
531         </div>
532         <div style="padding:20px;border-bottom:1px solid var(--border)">
533             <div style="display:flex;gap:10px">
534                 <input class="search-input" id="org-search-input" placeholder="Enter exact
employee name..."
535                     style="flex:1" onkeydown="if(event.key==='Enter')searchEmployee()">
536                 <button class="btn btn-primary" onclick="searchEmployee()">Search</button>
537             </div>
538             <div id="org-search-error" style="font-size:12px;color:var(--red);margin-
top:8px;display:none"></div>
539         </div>

```



```

540         <div id="org-search-results" style="padding:20px;display:none">
541
542             <!-- Employee Info Card -->
543             <div style="display:grid;grid-template-columns:1fr 1fr;gap:16px;margin-
bottom:20px">
544                 <div style="background:var(--bg3);border:1px solid var(--border);border-
radius:10px;padding:16px">
545                     <div style="font-size:10px;font-weight:700;color:var(--text3);text-
transform:uppercase;letter-spacing:0.8px;margin-bottom:12px">Employee Info</div>
546                     <div style="display:flex;flex-direction:column;gap:8px">
547                         <div style="display:flex;justify-content:space-between">
548                             <span style="font-size:12px;color:var(--text3)">Name</span>
549                             <span style="font-size:13px;font-weight:600;color:var(--text)"
id="sr-name"></span>
550                         </div>
551                         <div style="display:flex;justify-content:space-between">
552                             <span style="font-size:12px;color:var(--text3)">ID</span>
553                             <span style="font-size:12px;font-family:var(--mono);color:var(--
accent)" id="sr-id"></span>
554                         </div>
555                         <div style="display:flex;justify-content:space-between">
556                             <span style="font-size:12px;color:var(--text3)">Email</span>
557                             <span style="font-size:12px;font-family:var(--mono);color:var(--
text2)" id="sr-email"></span>
558                         </div>
559                         <div style="display:flex;justify-content:space-between">
560                             <span style="font-size:12px;color:var(--text3)">Department</span>
561                             <span style="font-size:13px;color:var(--text)" id="sr-
dept"></span>
562                         </div>
563                     </div>
564                 </div>
565                 <div style="background:var(--bg3);border:1px solid var(--border);border-
radius:10px;padding:16px">
566                     <div style="font-size:10px;font-weight:700;color:var(--text3);text-
transform:uppercase;letter-spacing:0.8px;margin-bottom:12px">Status</div>
567                     <div style="display:flex;flex-direction:column;gap:8px">
568                         <div style="display:flex;justify-content:space-between;align-
items:center">
569                             <span style="font-size:12px;color:var(--text3)">Work Status</span>
570                             <span id="sr-status"></span>
571                         </div>
572                         <div style="display:flex;justify-content:space-between;align-
items:center">
573                             <span style="font-size:12px;color:var(--text3)">Attendance</span>
574                             <span style="font-size:13px;font-weight:600;color:var(--text)"
id="sr-attendance"></span>
575                         </div>
576                     </div>
577                 </div>
578             </div>
579
580             <!-- Devices -->
581             <div style="background:var(--bg3);border:1px solid var(--border);border-
radius:10px;overflow:hidden">
582                 <div style="padding:12px 16px;border-bottom:1px solid var(--border)">
583                     <span style="font-size:12px;font-weight:700;color:var(--text3);text-
transform:uppercase;letter-spacing:0.8px">Assigned Devices</span>
584                 </div>
585                 <table style="width:100%;border-collapse:collapse">
586                     <thead>
587                         <tr>
588                             <th style="padding:10px 16px;text-align:left;font-size:10px;color:var(-
-text3);font-weight:700;letter-spacing:1px;text-transform:uppercase;background:var(--bg3)"...
589                             <th style="padding:10px 16px;text-align:left;font-size:10px;color:var(-
-text3);font-weight:700;letter-spacing:1px;text-transform:uppercase;background:var(--bg3)"...
590                             <th style="padding:10px 16px;text-align:left;font-size:10px;color:var(-
-text3);font-weight:700;letter-spacing:1px;text-transform:uppercase;background:var(--bg3)"...
591                             <th style="padding:10px 16px;text-align:left;font-size:10px;color:var(-
-text3);font-weight:700;letter-spacing:1px;text-transform:uppercase;background:var(--bg3)"...
592                             <th style="padding:10px 16px;text-align:left;font-size:10px;color:var(-
-text3);font-weight:700;letter-spacing:1px;text-transform:uppercase;background:var(--bg3)"...
593                             <th style="padding:10px 16px;text-align:left;font-size:10px;color:var(-
-text3);font-weight:700;letter-spacing:1px;text-transform:uppercase;background:var(--bg3)"...
594                         </tr>
595                     </thead>

```

```

596                 <tbody id="sr-devices"></tbody>
597             </table>
598         </div>
599     </div>
600
601     <!-- Empty / initial state -->
602     <div id="org-search-empty" style="padding:40px;text-align:center;color:var(--
text3);font-style:italic">
603         Enter a name above to look up an employee
604     </div>
605 </div>
606 </div>
607
608 <!-- PROMOTE USERS -->
609 <div id="promote-users-view" class="view">
610     <div class="section">
611         <div class="section-head">
612             <div><div class="section-title"> Promote Users</div><div class="section-
sub">Onboard pending user registrations to your department</div></div>
613             <button class="btn btn-secondary btn-sm" onclick="loadUsers()"> Refresh</button>
614         </div>
615         <div class="tablewrap">
616             <table>
617                 <thead><tr><th>Name</th><th>Email</th><th>Timezone</th><th>Temp
Password</th><th>Action</th></tr></thead>
618                 <tbody id="users-list"><tr><td colspan="5" class="empty-
state">Loading...</td></tr></tbody>
619             </table>
620         </div>
621     </div>
622 </div>
623
624 <!-- PENDING LEAVE APPROVALS -->
625 <div id="pending-leaves-view" class="view">
626     <div class="section">
627         <div class="section-head">
628             <div><div class="section-title"> Pending Leave Approvals</div><div class="section-
sub">Review and take action</div></div>
629             <button class="btn btn-secondary btn-sm" onclick="loadPendingLeaves()">
Refresh</button>
630         </div>
631         <div class="tablewrap">
632             <table>
633 <thead><tr><th>Employee</th><th>Type</th><th>From</th><th>To</th><th>Days</th><th>Reason</th><th>Actions</th></tr></thead>
634             <tbody id="pending-leaves-list"><tr><td colspan="7" class="empty-
state">Loading...</td></tr></tbody>
635         </table>
636     </div>
637 </div>
638 </div>
639
640
641
642 <!-- ALL LEAVES -->
643 <div id="all-leaves-view" class="view">
644     <div class="section">
645         <div class="section-head">
646             <div><div class="section-title">All Leave Requests</div></div>
647             <button class="btn btn-secondary btn-sm" onclick="loadAllLeaves()">
Refresh</button>
648         </div>
649         <div class="tablewrap">
650             <table>
651 <thead><tr><th>Employee</th><th>Type</th><th>From</th><th>To</th><th>Reason</th><th>Status</th></tr></thead>
652             <tbody id="all-leaves-list"><tr><td colspan="7" class="empty-
state">Loading...</td></tr></tbody>
653         </table>
654     </div>
655 </div>
656 </div>
657
658 <!-- APPLY LEAVE -->
659 <div id="apply-leave-view" class="view">

```

```

660         <div class="section">
661             <div class="section-head">
662                 <div>
663                     <div class="section-title"> Apply for Leave</div>
664                     <div class="section-sub">Submit a new leave request</div>
665                 </div>
666             </div>
667             <div class="form-card">
668                 <div class="form-grid">
669                     <div class="form-group">
670                         <label>Leave Type</label>
671                         <select id="leave-type">
672                             <option value="SICK">Sick Leave</option>
673                             <option value="CASUAL">Casual Leave</option>
674                             <option value="UNPAID">Unpaid Leave</option>
675                         </select>
676                         <div id="sick-leave-hint" style="display:none;margin-top:6px;font-
size:11px;color:var(--amber);background:var(--amber-dim);border:1px solid rgba(251,191,36,0.2);bord...
677                             Sick leave is limited to <strong>1 day per month</strong>. If already
used this month, please select Unpaid Leave.
678                         </div>
679                         <div id="casual-leave-hint" style="display:none; margin-top:6px; font-
size:11px; color:var(--accent); background:var(--accent-dim); border:1px solid rgba(167,139,250...
680                             You have <strong id="casual-balance-text">...</strong> casual leave
days available.
681                         </div>
682                     </div>
683                     <div class="form-group">
684                         <label>Reason</label>
685                         <input id="leave-reason" placeholder="Brief reason for leave">
686                     </div>
687                     <div class="form-group">
688                         <label>Start Date</label>
689                         <input type="date" id="leave-start">
690                     </div>
691                     <div class="form-group">
692                         <label>End Date</label>
693                         <input type="date" id="leave-end">
694                     </div>
695                 </div>
696                 <div class="form-actions">
697                     <button class="btn btn-primary" onclick="applyLeave()">Submit Request</button>
698                     <button class="btn btn-secondary" onclick="clearLeaveForm()">Clear</button>
699                 </div>
700             </div>
701         </div>
702     </div>
703
704     <!-- MY LEAVES -->
705     <div id="my-leaves-view" class="view">
706         <div class="section">
707             <div class="section-head">
708                 <div class="section-title">My Leave Requests</div>
709                 <button class="btn btn-secondary btn-sm" onclick="loadMyLeaves()"> Refresh</button>
710             </div>
711             <div class="tablewrap">
712                 <table>
713
714 <thead><tr><th>Type</th><th>From</th><th>To</th><th>Days</th><th>Status</th></tr></thead>
715 <tbody id="my-leaves-list"><tr><td colspan="5" class="empty-
state">Loading...</td></tr></tbody>
716                 </table>
717             </div>
718         </div>
719
720     <!-- ===== -->
721     <!-- MY CALENDAR VIEW (NEW) -->
722     <!-- ===== -->
723     <div id="my-calendar-view" class="view">
724         <div class="section">
725             <div class="section-head">
726                 <div>
727                     <div class="section-title">My Leave Calendar</div>
728                     <div class="section-sub">All approved leaves and public holidays</div>
729                 </div>

```

```

730             <button class="btn btn-secondary btn-sm" onclick="refreshCalendar()">
Refresh</button>
731         </div>
732
733         <!-- Leave Balance Summary Cards -->
734         <div class="leave-balance-grid">
735             <!-- Sick Leave Card -->
736             <div class="leave-bal-card sick">
737                 <span class="lbc-icon"></span>
738                 <span class="lbc-label">Sick Leave</span>
739                 <span class="lbc-count" id="cal-sick-remaining">--</span>
740                 <span class="lbc-sub" id="cal-sick-sub">loading...</span>
741             </div>
742             <!-- Casual Leave Card -->
743             <div class="leave-bal-card casual">
744                 <span class="lbc-icon"></span>
745                 <span class="lbc-label">Casual Leave</span>
746                 <span class="lbc-count" id="cal-casual-remaining">--</span>
747                 <span class="lbc-sub" id="cal-casual-sub">loading...</span>
748             </div>
749             <!-- Maternity Leave Card -->
750             <div class="leave-bal-card mat">
751                 <span class="lbc-icon"></span>
752                 <span class="lbc-label">Maternity Leave</span>
753                 <span class="lbc-count" id="cal-mat-remaining">--</span>
754                 <span class="lbc-sub" id="cal-mat-sub">loading...</span>
755             </div>
756             <!-- Total Used Card -->
757             <div class="leave-bal-card total">
758                 <span class="lbc-icon"></span>
759                 <span class="lbc-label">Total Used</span>
760                 <span class="lbc-count" id="cal-total-used">--</span>
761                 <span class="lbc-sub" id="cal-total-sub">loading...</span>
762             </div>
763         </div>
764
765         <!-- Legend -->
766         <div class="cal-legend">
767             <span class="legend-label">Legend:</span>
768             <div class="legend-item">
769                 <div class="legend-dot" style="background:#34d399"></div>
770                 Approved Leave
771             </div>
772             <div class="legend-item">
773                 <div class="legend-dot" style="background:#fbbf24"></div>
774                 Pending Leave
775             </div>
776             <div class="legend-item">
777                 <div class="legend-dot" style="background:#f87171"></div>
778                 Rejected Leave
779             </div>
780             <div class="legend-item">
781                 <div class="legend-dot" style="background:#38bdf8"></div>
782                 Public Holiday
783             </div>
784         </div>
785
786         <!-- FullCalendar Mount Point -->
787         <div id="employee-calendar"></div>
788     </div>
789 </div>
790 <!-- END CALENDAR VIEW -->
791
792 <!-- MY SUBSCRIPTIONS -->
793 <div id="my-subscriptions-view" class="view">
794     <div class="section">
795         <div class="section-head">
796             <div><div class="section-title"> My Subscriptions</div><div class="section-
sub">Your active meal subscriptions</div></div>
797             <button class="btn btn-secondary btn-sm" onclick="loadMySubscriptions()">
Refresh</button>
798         </div>
799         <div class="tablewrap">
800             <table>
801 <thead><tr><th>Slot</th><th>Restaurant</th><th>Address</th><th>Schedule</th><th>Day</th><th>Status</th><th>Next

```

```

Delivery</th></tr></thead>
802                <tbody id="my-subscriptions-list"><tr><td colspan="7" class="empty-
state">Loading...</td></tr></tbody>
803                </table>
804            </div>
805        </div>
806    </div>
807
808    <!-- ADD SUBSCRIPTION -->
809    <div id="add-subscription-view" class="view">
810        <div class="section">
811            <div class="section-head">
812                <div><div class="section-title"> Subscribe to Meal</div><div class="section-
sub">Add a new meal slot subscription</div></div>
813            </div>
814            <div class="form-card">
815                <div class="form-grid">
816                    <div class="form-group">
817                        <label>Schedule Type</label>
818                        <select id="sub-schedule" onchange="toggleDayOfWeek()">
819                            <option value="DAILY">Daily</option>
820                            <option value="WEEKLY">Weekly</option>
821                        </select>
822                    </div>
823                    <div class="form-group" id="day-of-week-group" style="display:none">
824                        <label>Day of Week</label>
825                        <select id="sub-day">
826                            <option value="MONDAY">Monday</option>
827                            <option value="TUESDAY">Tuesday</option>
828                            <option value="WEDNESDAY">Wednesday</option>
829                            <option value="THURSDAY">Thursday</option>
830                            <option value="FRIDAY">Friday</option>
831                            <option value="SATURDAY">Saturday</option>
832                            <option value="SUNDAY">Sunday</option>
833                        </select>
834                    </div>
835                </div>
836
837
838                <!-- Slot Orders -->
839                <div style="margin-top:20px">
840                    <div style="display:flex;justify-content:space-between;align-
items:center;margin-bottom:12px">
841                        <label style="font-size:11px;font-weight:700;color:var(--text3);letter-
spacing:0.8px;text-transform:uppercase">Meal Slots & Restaurants</label>
842                        <button class="btn btn-secondary btn-sm" onclick="addSlotRow()"> Add
Slot</button>
843                    </div>
844                    <div id="slot-rows"></div>
845                </div>
846
847                <div class="form-actions">
848                    <button class="btn btn-primary"
onclick="submitSubscription()">Subscribe</button>
849                    <button class="btn btn-secondary"
onclick="clearSubscriptionForm()">Clear</button>
850                </div>
851            </div>
852        </div>
853    </div>
854    <!-- RESTAURANTS -->
855    <div id="restaurants-view" class="view">
856        <div class="section">
857            <div class="section-head">
858                <div><div class="section-title"> Restaurants</div><div class="section-sub">All
available restaurants</div></div>
859                <button class="btn btn-secondary btn-sm" onclick="loadRestaurants()">
Refresh</button>
860            </div>
861            <div class="tablewrap">
862                <table>
863                    <thead><tr><th>ID</th><th>Name</th><th>Address</th><th>Status</th></tr></thead>
864                    <tbody id="restaurants-list"><tr><td colspan="4" class="empty-
state">Loading...</td></tr></tbody>
865                </table>
866            </div>

```

[illegible]

```

941             <option value="">-- Select your device --</option>
942         </select>
943     </div>
944     <div class="form-group">
945         <label>Request Type</label>
946         <select id="sr-type">
947             <option value="REPAIR">Repair</option>
948             <option value="REPLACEMENT">Replacement</option>
949             <option value="UPGRADE">Upgrade</option>
950             <option value="MAINTENANCE">Maintenance</option>
951             <option value="RETURN">Return</option>
952         </select>
953     </div>
954     <div class="form-group" style="display:flex;align-
items:center;gap:10px;padding-top:22px">
955         <input type="checkbox" id="sr-urgent"
style="width:16px;height:16px;padding:0;flex-shrink:0">
956         <label for="sr-urgent" style="text-transform:none;font-size:13px;font-
weight:500;color:var(--text);cursor:pointer;letter-spacing:0">
957             Mark as Urgent
958         </label>
959     </div>
960     <div class="form-group full">
961         <label>Issue Description</label>
962         <textarea id="sr-description" rows="3" placeholder="Describe the issue in
detail..."></textarea>
963     </div>
964 </div>
965     <div class="form-actions">
966         <button class="btn btn-primary" onclick="submitServiceRequest()">Submit
Request</button>
967         <button class="btn btn-secondary"
onclick="clearServiceRequestForm()">Clear</button>
968     </div>
969 </div>
970 </div>
971 </div>
972
973 <!-- TWO-FACTOR AUTH -->
974 <div id="two-factor-view" class="view">
975     <div class="section">
976         <div class="section-head">
977             <div>
978                 <div class="section-title"> Two-Factor Authentication</div>
979                 <div class="section-sub">Protect your account with an authenticator app</div>
980             </div>
981             <span id="totp-status-chip" class="chip">Checking...</span>
982         </div>
983
984         <div style="padding:24px;max-width:480px">
985
986             <!-- DISABLED state – setup flow -->
987             <div id="totp-setup-section">
988                 <p style="font-size:13px;color:var(--text2);margin-bottom:16px;line-
height:1.6">
989                     Use Google Authenticator, Authy, or any TOTP app.
990                     Scan the QR code below, then enter the 6-digit code to activate.
991                 </p>
992                 <button class="btn btn-primary" onclick="generateTotpQr()" id="generate-qr-
btn">
993                     Generate QR Code
994                 </button>
995
996                 <div id="qr-area" style="display:none;margin-top:24px;text-align:center">
997                     <img id="qr-img" src="" alt="QR Code"
998                         style="width:180px;height:180px;border-radius:10px;
999                         border:1px solid var(--border2);display:block;margin:0 auto">
1000                     <div style="margin-top:12px;font-size:11px;color:var(--text3)">
1001                         Can't scan? Enter this key manually:
1002                     </div>
1003                     <div id="totp-secret-display"
1004                         style="font-family:var(--mono);font-size:13px;color:var(--accent);
1005                         background:var(--bg3);border:1px solid var(--border2);
1006                         border-radius:6px;padding:8px 12px;margin-top:6px;
1007                         word-break:break-all"></div>
1008

```

```

1009         <div style="margin-top:20px;text-align:left">
1010             <label style="font-size:11px;color:var(--text3);font-weight:600;
1011                 text-transform:uppercase;letter-spacing:0.8px;
1012                 display:block;margin-bottom:8px">
1013                 Enter 6-digit code to activate
1014             </label>
1015             <input id="totp-verify-input" type="text" inputmode="numeric"
1016                 maxlength="6" placeholder="000000"
1017                 style="text-align:center;font-size:20px;
1018                 font-family:var(--mono);letter-spacing:8px;padding:12px"
1019                 oninput="this.value=this.value.replace(/\D/g, '')">
1020             <button class="btn btn-primary"
1021                 onclick="verifyAndActivateTotp()"
1022                 style="width:100%;margin-top:10px">
1023                 Activate 2FA
1024             </button>
1025             <div id="totp-verify-error"
1026                 style="display:none;margin-top:8px;font-size:12px;
1027                 color:var(--red)"></div>
1028         </div>
1029     </div>
1030 </div>
1031
1032     <!-- ENABLED state -->
1033     <div id="totp-disable-section" style="display:none">
1034         <div style="background:var(--green-dim);border:1px solid rgba(52,211,153,0.2);
1035             border-radius:8px;padding:12px 16px;font-size:13px;
1036             color:var(--green);margin-bottom:16px">
1037             Two-factor authentication is active on your account.
1038         </div>
1039         <button class="btn btn-danger" onclick="disableTotp()">
1040             Disable 2FA
1041         </button>
1042     </div>
1043 </div>
1044 </div>
1045 </div>
1046
1047 <!-- PROFILE -->
1048 <div id="profile-view" class="view">
1049     <div class="section">
1050         <div class="section-head">
1051             <div class="section-title">My Profile</div>
1052         </div>
1053         <div style="display:grid;grid-template-columns:1fr 1fr;gap:16px;padding:24px">
1054             <div style="display:flex;flex-direction:column;gap:4px">
1055                 <div style="font-size:10px;color:var(--text3);text-transform:uppercase;
1056                     letter-spacing:0.8px;font-weight:700">Name</div>
1057                 <div style="font-size:14px;color:var(--text);padding:10px 14px;
1058                     background:var(--bg3);border-radius:8px;border:1px solid var(--border);
1059                     font-family:var(--mono)" id="p-name"></div>
1060             </div>
1061             <div style="display:flex;flex-direction:column;gap:4px">
1062                 <div style="font-size:10px;color:var(--text3);text-transform:uppercase;
1063                     letter-spacing:0.8px;font-weight:700">Email</div>
1064                 <div style="font-size:14px;color:var(--text);padding:10px 14px;
1065                     background:var(--bg3);border-radius:8px;border:1px solid var(--border);
1066                     font-family:var(--mono)" id="p-email"></div>
1067             </div>
1068             <div style="display:flex;flex-direction:column;gap:4px">
1069                 <div style="font-size:10px;color:var(--text3);text-transform:uppercase;
1070                     letter-spacing:0.8px;font-weight:700">Department</div>
1071                 <div style="font-size:14px;color:var(--text);padding:10px 14px;
1072                     background:var(--bg3);border-radius:8px;border:1px solid var(--border);
1073                     font-family:var(--mono)" id="p-dept"></div>
1074             </div>
1075             <div style="display:flex;flex-direction:column;gap:4px">
1076                 <div style="font-size:10px;color:var(--text3);text-transform:uppercase;
1077                     letter-spacing:0.8px;font-weight:700">Role</div>
1078                 <div style="font-size:14px;color:var(--text);padding:10px 14px;
1079                     background:var(--bg3);border-radius:8px;border:1px solid var(--border);
1080                     font-family:var(--mono)" id="p-role"></div>
1081             </div>
1082             <div style="display:flex;flex-direction:column;gap:4px">
1083                 <div style="font-size:10px;color:var(--text3);text-transform:uppercase;
1084                     letter-spacing:0.8px;font-weight:700">Status</div>

```



```

1085         <div style="font-size:14px;color:var(--text);padding:10px 14px;
1086         background:var(--bg3);border-radius:8px;border:1px solid var(--border);
1087         font-family:var(--mono)" id="p-status"></div>
1088     </div>
1089     <div style="display:flex;flex-direction:column;gap:4px">
1090         <div style="font-size:10px;color:var(--text3);text-transform:uppercase;
1091         letter-spacing:0.8px;font-weight:700">Timezone</div>
1092         <div style="font-size:14px;color:var(--text);padding:10px 14px;
1093         background:var(--bg3);border-radius:8px;border:1px solid var(--border);
1094         font-family:var(--mono)" id="p-tz"></div>
1095     </div>
1096 </div>
1097 </div>
1098
1099 <!-- MATERNITY SECTION – only visible for female managers -->
1100 <div class="section" id="maternity-apply-btn" style="display:none;margin-top:0">
1101     <div class="section-head">
1102         <div>
1103             <div class="section-title"> Maternity Leave</div>
1104             <div class="section-sub">Apply for 180 days maternity leave</div>
1105         </div>
1106     </div>
1107     <div class="form-card">
1108         <div class="form-grid">
1109             <div class="form-group">
1110                 <label>Start Date</label>
1111                 <input type="date" id="mat-start-date">
1112             </div>
1113             <div class="form-group">
1114                 <label>End Date (Auto)</label>
1115                 <input type="text" id="mat-end-date"
1116                     placeholder="Auto: start + 180 days"
1117                     disabled
1118                     style="color:var(--text3);cursor:not-allowed">
1119             </div>
1120             <div class="form-group full">
1121                 <label>Reason / Notes</label>
1122                 <textarea id="mat-reason" rows="3"
1123                     placeholder="Optional notes..."></textarea>
1124             </div>
1125             <div class="form-group full">
1126                 <div style="background:var(--accent-dim);
1127                 border:1px solid rgba(167,139,250,0.2);
1128                 border-radius:8px;padding:12px 16px;
1129                 font-size:12px;color:var(--text2)">
1130                     Maternity leave is fixed at
1131                     <strong style="color:var(--accent)">180 days</strong>.
1132                     End date is calculated automatically.
1133                     You can apply <strong style="color:var(--accent)">once per
1134 year</strong>.
1135                 </div>
1136             </div>
1137             <div class="form-actions">
1138                 <button class="btn btn-primary" onclick="applyMaternityLeave()">
1139                     Submit Maternity Leave
1140                 </button>
1141             </div>
1142         </div>
1143     </div>
1144 </div>
1145
1146 </div>
1147 </div>
1148 </div>
1149 <!-- Tooltip for calendar events -->
1150 <div class="cal-tooltip" id="cal-tooltip" style="display:none"></div>
1151 <script>
1152     const API='http://localhost:8080';
1153     let allEmployeesCache=[];
1154
1155     // Add these near the top of your <script>, right after: let allEmployeesCache=[];
1156
1157     let employeeCalendar = null;
1158     let calendarInitialized = false;
1159

```

```

1160 const STATUS_COLORS = {
1161     APPROVED : { bg:'rgba(52,211,153,0.25)', border:'#34d399', text:'#34d399' },
1162     PENDING  : { bg:'rgba(251,191,36,0.25)',  border:'#fbbf24', text:'#fbbf24' },
1163     REJECTED  : { bg:'rgba(248,113,113,0.25)', border:'#f87171', text:'#f87171' },
1164     HOLIDAY   : { bg:'rgba(167,139,250,0.25)', border:'#a78bfa', text:'#a78bfa' },
1165 };
1166
1167 const TYPE_LABEL = { SICK:' Sick', CASUAL:' Casual', MATERNITY:' Maternity', UNPAID:' Unpaid'
1168 };
1169
1170 (function(){
1171     const token=localStorage.getItem('jwt_token');
1172     const authType=localStorage.getItem('auth_type');
1173     if(!token&&authType!=='session'){window.location.href='/login.html';return;}
1174     if(token){
1175         try{
1176             const p=JSON.parse(atob(token.split('.')[1]));
1177             const rawRole = p.role || p.roles || p.authorities || '';
1178             const normalizedRoles = Array.isArray(rawRole)
1179                 ? rawRole.map(v => String(v).replace(/^ROLE_/, ''))
1180                 : [String(rawRole).replace(/^ROLE_/, '')];
1181             if(normalizedRoles.includes('MANAGER')) { initUser(); return; }
1182             if(normalizedRoles.includes('ADMIN')){window.location.href='/dashboard.html';return;}
1183             if(normalizedRoles.includes('EMPLOYEE')){window.location.href='/employee-
dashboard.html';return;}
1184             if(normalizedRoles.includes('VENDOR')){window.location.href='/vendor-
dashboard.html';return;}
1185         }catch(e){}
1186     }
1187     initUser();
1188 })();
1189
1190 function getAuthHeader(){
1191     const t=localStorage.getItem('jwt_token');
1192     return t?{'Authorization':`Bearer ${t}`}:{};
1193 }
1194 async function apiFetch(path, opts={}){
1195     const res = await fetch(`${API}${path}`, {
1196         ...opts,
1197         headers: {'Content-Type':'application/json',...getAuthHeader(),...(opts.headers||{})},
1198         credentials: 'include'
1199     });
1200     if(res.status===401){
1201         localStorage.removeItem('jwt_token');
1202         localStorage.removeItem('auth_type');
1203         showAlert('Session expired','error');
1204         setTimeout(()=>window.location.href='/google.html',1500);
1205         throw new Error('Unauthorized');
1206     }
1207     const text = await res.text();
1208     if(!res.ok){
1209         let message = res.statusText || `Error ${res.status}`;
1210         try {
1211             const body = text ? JSON.parse(text) : null;
1212             message = body?.message || body?.error || message;
1213         } catch {
1214             message = text || message;
1215         }
1216         throw new Error(message);
1217     }
1218     if(!text) return null;
1219     try { return JSON.parse(text); } // try JSON first
1220     catch { return text; } // fall back to plain string
1221 }
1222
1223 function asArray(value){
1224     return Array.isArray(value) ? value : [];
1225 }
1226
1227 async function initUser(){
1228     const token=localStorage.getItem('jwt_token');
1229     let email='';
1230     if(token){try{const p=JSON.parse(atob(token.split('.')[1]));email=p.sub||p.email||'';}catch(e){}}
1231     document.getElementById('avatar-initials').textContent=(email||'M').charAt(0).toUpperCase();
1232     document.getElementById('sidebar-name').textContent=email||'Manager';
1233     document.getElementById('topbar-email').textContent=email;

```

```

1234         loadDashboard();
1235         loadUsers();
1236     }
1237
1238     function showView(name,el){
1239         document.querySelectorAll('.view').forEach(v=>v.classList.remove('active'));
1240         document.getElementById(`${name}-view`).classList.add('active');
1241         document.querySelectorAll('.nav-item').forEach(n=>n.classList.remove('active'));
1242         if(el)el.classList.add('active');
1243         document.getElementById('pageTitle').textContent=name.split('-
1244         ').map(w=>w.charAt(0).toUpperCase()+w.slice(1)).join(' ');
1245         const loaders = {
1246             dashboard: loadDashboard,
1247             employees: loadEmployees,
1248             'pending-leaves': loadPendingLeaves,
1249             'all-leaves': loadAllLeaves,
1250             'my-leaves': loadMyLeaves,
1251             'my-calendar': loadMyCalendar,
1252             'add-employee': () => {},
1253             'promote-users': loadUsers,
1254             'my-subscriptions': loadMySubscriptions,
1255             'add-subscription': () => {},
1256             restaurants: loadRestaurants,
1257             'my-devices': loadMyAssignedDevices,
1258             'service-requests': loadServiceRequests,
1259             'two-factor':loadTotpStatus,
1260             'search-employee': () => {},
1261             'profile': loadProfile,      // ADD THIS
1262         };
1263         if(loaders[name])loaders[name]();
1264     }
1265
1266     async function loadDashboard(){
1267         const [empsResult, leavesResult] = await Promise.allSettled([
1268             apiFetch('/manager/employees'),
1269             apiFetch('/manager/pending')
1270         ]);
1271
1272         const emps = asArray(empsResult.status === 'fulfilled' ? empsResult.value : []);
1273         const leaves = asArray(leavesResult.status === 'fulfilled' ? leavesResult.value : []);
1274
1275         if (empsResult.status === 'rejected') {
1276             console.error('Failed to load manager employees:', empsResult.reason);
1277             document.getElementById('dash-employees').innerHTML =
1278                 '<tr><td colspan="3" class="empty-state">Could not load employees</td></tr>';
1279         }
1280         if (leavesResult.status === 'rejected') {
1281             console.error('Failed to load pending leaves:', leavesResult.reason);
1282             document.getElementById('dash-pending').innerHTML =
1283                 '<tr><td colspan="4" class="empty-state">Could not load pending requests</td></tr>';
1284         }
1285
1286         // Stats
1287         document.getElementById('stat-total').textContent = emps.length;
1288         document.getElementById('stat-active').textContent = emps.filter(e => e.status ===
1289         'ACTIVE').length;
1290         document.getElementById('stat-on-leave').textContent = emps.filter(e => e.status ===
1291         'ON_LEAVE').length;
1292         const pending = leaves.filter(l => l.status === 'PENDING');
1293         document.getElementById('stat-pending').textContent = pending.length;
1294         document.getElementById('pending-badge').textContent = pending.length;
1295         document.getElementById('pending-badge').classList.toggle('hidden', pending.length === 0);
1296
1297         // Build employeeId name lookup from employees list
1298         const empMap = {};
1299         emps.forEach(e => { empMap[e.employeeId] = e.name || '-' });
1300
1301         // Dash pending
1302         if (leavesResult.status === 'fulfilled') {
1303             document.getElementById('dash-pending').innerHTML = pending.length
1304             ? pending.slice(0, 5).map(l => {
1305                 const leaveId = l.leaveRequestId || 0;
1306                 const empName = empMap[l.employeeId] || `#${l.employeeId}`;
1307                 return `<tr>
1308                     <td>${empName}</td>
1309                     <td><span class="chip ${l.leaveType||''}.toLowerCase()>${l.leaveType}</span></td>

```

```

1307         <td>${l.startDate} ${l.endDate}</td>
1308         <td><div class="action-row">
1309             <button class="btn btn-approve btn-sm"
onclick="approveLeave(${leaveId})"></button>
1310             <button class="btn btn-reject btn-sm" onclick="rejectLeave(${leaveId})"></button>
1311         </div></td>
1312     </tr>`;
1313     }).join('')
1314     : '<tr><td colspan="4" class="empty-state">No pending requests </td></tr>';
1315 }
1316
1317 // Dash employees
1318 if (empsResult.status === 'fulfilled') {
1319     document.getElementById('dash-employees').innerHTML = emps.length
1320     ? emps.slice(0, 6).map(e => `<tr>
1321         <td>${e.name} || '-`</td>
1322         <td>${e.dept} || '-`</td>
1323         <td><span class="chip ${e.status} || ''>${e.status} || '-`</span></td>
1324     </tr>`).join('')
1325     : '<tr><td colspan="3" class="empty-state">No employees</td></tr>';
1326 }
1327 }
1328
1329 async function loadEmployees(){
1330     try{
1331         const emps=await apiFetch('/manager/employees');
1332         allEmployeesCache=emps||[];
1333         renderEmployees(allEmployeesCache);
1334     }catch(e){document.getElementById('employees-list').innerHTML='<tr><td colspan="7" class="empty-
state">Could not load employees</td></tr>';}
1335 }
1336
1337 function renderEmployees(emps){
1338     document.getElementById('employees-list').innerHTML=emps.length
1339     ? emps.map(e=>`<tr><td style="font-family:var(--mono);font-
size:12px">#${e.employeeId}</td><td>${e.name}||'-`</td><td style="font-family:var(--mono);font-
size:12px">${e.email}||'-`</td>...
1340     : '<tr><td colspan="7" class="empty-state">No employees found</td></tr>';
1341 }
1342
1343 function filterEmployees(){
1344     const q=document.getElementById('emp-search').value.toLowerCase();
1345     renderEmployees(allEmployeesCache.filter(e=>(e.name||'').toLowerCase().includes(q)||e.email||'').toLowerCase().
includes(q)||e.dept||'').toLowerCase().includes(q)));
1346 }
1347
1348 async function addEmployee(){
1349     const name = document.getElementById('emp-name').value.trim();
1350     const email = document.getElementById('emp-email').value.trim();
1351     const password = document.getElementById('emp-pass').value;
1352     const gender = document.getElementById('emp-gender').value;
1353     const timezone = document.getElementById('emp-timezone').value;
1354     if(!name || !email || !password){
1355         showAlert('Please fill all required fields','error');
1356         return;
1357     }
1358     try{
1359         await apiFetch('/manager/employees', {
1360             method: 'POST',
1361             body: JSON.stringify({ name, email, password, timezone, gender })
1362         });
1363         showAlert('Employee added successfully!','success');
1364         clearEmpForm();
1365         loadEmployees();
1366     }catch(e){ showAlert('Failed: '+e.message,'error'); }
1367 }
1368
1369 function clearEmpForm(){
1370     ['emp-name', 'emp-email', 'emp-pass'].forEach(id=>document.getElementById(id).value='');
1371     document.getElementById('emp-timezone').value = 'UTC';
1372     document.getElementById('emp-gender').selectedIndex = 0;
1373 }
1374
1375 async function deleteEmployee(id){
1376     if(!confirm('Remove this employee?'))return;
1377     try{await apiFetch(`/employee/${id}`, {method: 'DELETE'});showAlert('Employee

```

```

removed', 'success');loadEmployees();}
1377     catch(e){
1378     console.log(e);
1379     showAlert('Failed to remove employee','error');
1380     }
1381 }
1382
1383 async function loadPendingLeaves(){
1384 try{
1385     const leaves = await apiFetch('/manager/pending');
1386     console.log('Leave sample:', leaves?.[0]); // remove after confirming
1387     const pending=(leaves||[]).filter(l=>l.status==='PENDING');
1388     document.getElementById('pending-leaves-list').innerHTML=pending.length
1389     ? pending.map(l=>{
1390         // Try all possible ID field names your DTO might use
1391         const leaveId = l.leaveRequestId || l.leaveId || l.id || 0;
1392         return `<tr>
1393             <td>${l.employeeName||l.employeeId||'-'}</td>
1394             <td><span class="chip ${l.leaveType||''}.toLowerCase()>${l.leaveType}</span></td>
1395             <td>${l.startDate}</td>
1396             <td>${l.endDate}</td>
1397             <td>${l.numberOfDays||'-'}</td>
1398             <td>${(l.reason||'-').substring(0,40)}</td>
1399             <td><div class="action-row">
1400                 <button class="btn btn-approve btn-sm" onclick="approveLeave(${leaveId})">
Approve</button>
1401                 <button class="btn btn-reject btn-sm" onclick="rejectLeave(${leaveId})">
Reject</button>
1402             </div></td>
1403         </tr>`;
1404     }).join('')
1405     : `<tr><td colspan="7" class="empty-state">No pending approvals </td></tr>`;
1406 }catch(e){
1407     document.getElementById('pending-leaves-list').innerHTML=`<tr><td colspan="7" class="empty-
state">Could not load</td></tr>`;
1408 }
1409 }
1410
1411 async function approveLeave(id) {
1412 console.log('Approving leaveRequestId:', id); // verify this is not 0
1413 try {
1414     const res = await fetch(`${API}/leave_requests/approval`, {
1415         method: 'PUT',
1416         headers: { 'Content-Type': 'application/json', ...getAuthHeader() },
1417         credentials: 'include',
1418         body: JSON.stringify({
1419             leaveRequestId: id, // matches ActionDTO exactly
1420             action: 'APPROVED', // check if your service expects uppercase or lowercase
1421             remarks: ''
1422         })
1423     });
1424     const text = await res.text();
1425     if (res.ok) {
1426         showAlert('Leave approved ', 'success');
1427         loadPendingLeaves(); loadDashboard(); loadAllLeaves();
1428     } else {
1429         showAlert('Error: ' + text, 'error');
1430     }
1431 } catch(e) { showAlert(e.message, 'error'); }
1432 }
1433
1434 async function rejectLeave(id) {
1435 console.log('Rejecting leaveRequestId:', id); // verify this is not 0
1436 try {
1437     const res = await fetch(`${API}/leave_requests/approval`, {
1438         method: 'PUT',
1439         headers: { 'Content-Type': 'application/json', ...getAuthHeader() },
1440         credentials: 'include',
1441         body: JSON.stringify({
1442             leaveRequestId: id, // matches ActionDTO exactly
1443             action: 'REJECTED', // check if your service expects uppercase or lowercase
1444             remarks: ''
1445         })
1446     });
1447     const text = await res.text();
1448     if (res.ok) {
1449         showAlert('Leave rejected', 'success');

```

```

1449         loadPendingLeaves(); loadDashboard(); loadAllLeaves();
1450     } else {
1451         showAlert('Error: ' + text, 'error');
1452     }
1453 } catch(e) { showAlert('Failed: ' + e.message, 'error'); }
1454 }
1455
1456 async function loadAllLeaves(){
1457     try{
1458         const leaves=await apiFetch('/manager/requests');
1459         document.getElementById('all-leaves-list').innerHTML=(leaves||[]).length
1460             ? (leaves).map(l=><tr>
1461 <td>${l.employeeName||'-'}</td>
1462 <td><span class="chip ${l.leaveType||''}.toLowerCase()">${l.leaveType||'-'}</span></td>
1463 <td>${l.startDate||'-'}</td>
1464 <td>${l.endDate||'-'}</td>
1465 <td>${(l.reason||'-').substring(0,50)}</td>
1466 <td><span class="chip ${l.status||''}.toLowerCase()">${l.status||'-'}</span></td>
1467 </tr>` )
1468 : '<tr><td colspan="7" class="empty-state">No leave requests</td></tr>';
1469     }catch(e){document.getElementById('all-leaves-list').innerHTML='<tr><td colspan="7" class="empty-
state">Could not load</td></tr>';}
1470 }
1471
1472 async function applyLeave() {
1473     const leaveType = document.getElementById('leave-type').value;
1474     const reason = document.getElementById('leave-reason').value.trim();
1475     const startDate = document.getElementById('leave-start').value;
1476     const endDate = document.getElementById('leave-end').value;
1477
1478     if (!reason || !startDate || !endDate) {
1479         showAlert('Please fill all fields', 'error');
1480         return;
1481     }
1482     if (endDate < startDate) {
1483         showAlert('End date cannot be before start date', 'error');
1484         return;
1485     }
1486
1487     try {
1488         const data = await apiFetch('/leave_requests', {
1489             method: 'POST',
1490             body: JSON.stringify({ leaveType, reason, startDate, endDate })
1491         });
1492         if (data && data.warning) {
1493             showAlert(data.warning, 'warning');
1494         } else {
1495             showAlert('Leave request submitted successfully!', 'success');
1496         }
1497         clearLeaveForm();
1498         loadMyLeaves();
1499     } catch(e) {
1500         let msg = e.message;
1501         try {
1502             const parsed = JSON.parse(msg);
1503             if(parsed.error) msg = parsed.error;
1504         } catch(_) {}
1505         showAlert(msg, 'error');
1506     }
1507 }
1508
1509 document.getElementById('leave-type').addEventListener('change', async function() {
1510     document.getElementById('sick-leave-hint').style.display =
1511         this.value === 'SICK' ? 'block' : 'none';
1512
1513     const casualHint = document.getElementById('casual-leave-hint');
1514     if(this.value === 'CASUAL') {
1515         casualHint.style.display = 'block';
1516         document.getElementById('casual-balance-text').textContent = 'loading...';
1517         try {
1518             const dashboard = await apiFetch('/api/employees/me/leave-dashboard');
1519             const balances = dashboard?.leaveBalances || {};
1520             const cas = balances['CASUAL'] || balances['casual'] || {};
1521             const available = Number(cas.available ?? cas.availableBalance ?? 0);
1522             document.getElementById('casual-balance-text').textContent =
1523                 available > 0 ? `${available}` : '0 (no balance remaining)';
1524         } catch(e) {

```

```

1524         document.getElementById('casual-balance-text').textContent = 'unavailable';
1525     }
1526 } else {
1527     casualHint.style.display = 'none';
1528 }
1529 });
1530
1531 function clearLeaveForm() {
1532     document.getElementById('leave-type').value = 'SICK';
1533     document.getElementById('leave-reason').value = '';
1534     document.getElementById('leave-start').value = '';
1535     document.getElementById('leave-end').value = '';
1536 }
1537
1538 async function loadMyLeaves(){
1539     try{
1540         const data=await apiFetch('/leave_requests/employee');
1541         const leaves=data?.requests||data||[];
1542         document.getElementById('my-leaves-list').innerHTML=leaves.length
1543             ? leaves.map(l=><tr><td><span class="chip
1544             ${l.leaveType||''}.toLowerCase()}>${l.leaveType}</span></td><td>${l.startDate}</td><td>${l.endDate}</td><td>${
1545             l.numberOfDays||'-'}</td><...
1546             : <tr><td colspan="5" class="empty-state">No leave requests</td></tr>';
1547     }catch(e){document.getElementById('my-leaves-list').innerHTML=<tr><td colspan="5" class="empty-
1548     state">Could not load</td></tr>';}
1549 }
1550
1551 async function loadUsers(){
1552     try{
1553         const users = await apiFetch('/manager/Users');
1554         const badge = document.getElementById('users-badge');
1555         if(badge){ badge.textContent = (users||[]).length;
1556         badge.classList.toggle('hidden',(users||[]).length===0); }
1557         document.getElementById('users-list').innerHTML = (users||[]).length
1558             ? users.map(u=><tr>
1559             <td>${u.name||'-'}</td>
1560             <td style="font-family:var(--mono);font-size:12px">${u.email||'-'}</td>
1561             <td style="font-family:var(--mono);font-size:12px">${u.timezone||'-'}</td>
1562             <td><input type="password" placeholder="Set password" id="pwd-${CSS.escape(u.email)}"
1563             style="padding:6px 10px;font-size:12px;width:160px"></td>
1564             <td><button class="btn btn-approve btn-sm" onclick="promoteUser('${u.email}')">
1565             Promote</button></td>
1566             </tr>`).join('')
1567             : <tr><td colspan="5" class="empty-state">No pending user registrations</td></tr>';
1568     }catch(e){
1569         document.getElementById('users-list').innerHTML=<tr><td colspan="5" class="empty-state">Could not
1570         load</td></tr>';
1571     }
1572 }
1573
1574 async function promoteUser(email){
1575     const pwdInput = document.getElementById('pwd-'+CSS.escape(email));
1576     const password = pwdInput?.value?.trim();
1577     if(!password){ showAlert('Please set a password before promoting','error'); return; }
1578     try{
1579         const res = await fetch(`${API}/manager/promote`,{
1580             method:'PUT',
1581             headers:{'Content-Type':'application/json',...getAuthHeader()},
1582             credentials:'include',
1583             body: JSON.stringify({ email, password })
1584         });
1585         const text = await res.text();
1586         if(res.ok){
1587             showAlert(`${email} promoted to Employee `, 'success');
1588             loadUsers();
1589         } else {
1590             showAlert('Error: '+text, 'error');
1591         }
1592     }catch(e){ showAlert('Failed: '+e.message, 'error'); }
1593 }
1594
1595 async function loadRestaurants(){
1596     try{
1597         const data = await apiFetch('/restaurants');
1598         document.getElementById('restaurants-list').innerHTML = (data||[]).length
1599             ? data.map(r=><tr>
1600             <td style="font-family:var(--mono);font-size:12px">#${r.restaurantId||r.id}</td>

```

```

1594         <td>${r.name||'-'}</td>
1595         <td style="font-size:12px;color:var(--text3)">${r.address||'-'}</td>
1596         <td><span class="chip active">${r.status||'ACTIVE'}</span></td>
1597     </tr>`).join('')
1598     : '<tr><td colspan="4" class="empty-state">No restaurants found</td></tr>';
1599 }catch(e){
1600     document.getElementById('restaurants-list').innerHTML='<tr><td colspan="4" class="empty-
state">Could not load</td></tr>';
1601 }
1602 }
1603
1604 // SUBSCRIPTIONS
1605
1606 async function loadMySubscriptions(){
1607     try{
1608         const subs = await apiFetch('/subscription/user');
1609         document.getElementById('my-subscriptions-list').innerHTML = (subs||[]).length
1610             ? subs.map(s=>`<tr>
1611                 <td><span class="chip pending">${s.mealSlot||'-'}</span></td>
1612                 <td>${s.restaurantName||'-'}</td>
1613                 <td style="font-size:12px;color:var(--text3)">${s.restaurantAddress||'-'}</td>
1614                 <td>${s.scheduleType||'-'}</td>
1615                 <td>${s.dayOfWeek||'-'}</td>
1616                 <td><span class="chip ${s.status||''}.toLowerCase()">${s.status||'-'}</span></td>
1617                 <td style="font-family:var(--mono);font-size:11px">${s.nextDeliveryTime ? new
Date(s.nextDeliveryTime).toLocaleString() : '-'}</td>
1618             </tr>`).join('')
1619             : '<tr><td colspan="7" class="empty-state">No subscriptions yet</td></tr>';
1620     }catch(e){
1621         document.getElementById('my-subscriptions-list').innerHTML='<tr><td colspan="7" class="empty-
state">Could not load</td></tr>';
1622     }
1623 }
1624
1625 let slotRowCount = 0;
1626 async function addSlotRow(){
1627     slotRowCount++;
1628     const id = slotRowCount;
1629     const div = document.createElement('div');
1630     div.id = `slot-row-${id}`;
1631     div.style.cssText = 'display:grid;grid-template-columns:1fr 1fr auto;gap:12px;margin-
bottom:10px;align-items:end';
1632
1633     // Fetch restaurants for the dropdown
1634     let options = '<option value="">Loading...</option>';
1635     try{
1636         const data = await apiFetch('/restaurants');
1637         options = data.map(r => `<option
value="${r.restaurantId||r.id}">${r.name}</option>`).join('');
1638     }catch(e){ options = '<option value="">Could not load</option>'; }
1639
1640     div.innerHTML = `
1641     <div class="form-group" style="margin:0">
1642         <label>Meal Slot</label>
1643         <select id="slot-meal-${id}">
1644             <option value="BREAKFAST">Breakfast</option>
1645             <option value="LUNCH">Lunch</option>
1646             <option value="DINNER">Dinner</option>
1647         </select>
1648     </div>
1649     <div class="form-group" style="margin:0">
1650         <label>Restaurant</label>
1651         <select id="slot-restaurant-${id}">${options}</select>
1652     </div>
1653     <button class="btn btn-reject btn-sm" onclick="removeSlotRow(${id})" style="margin-
bottom:1px"></button>
1654 `;
1655     document.getElementById('slot-rows').appendChild(div);
1656 }
1657
1658 function removeSlotRow(id){
1659     document.getElementById(`slot-row-${id}`)?.remove();
1660 }
1661
1662 function toggleDayOfWeek(){
1663     const isWeekly = document.getElementById('sub-schedule').value === 'WEEKLY';

```



```

1664     document.getElementById('day-of-week-group').style.display = isWeekly ? 'flex' : 'none';
1665 }
1666
1667     async function submitSubscription(){
1668         const scheduleType = document.getElementById('sub-schedule').value;
1669         const dayOfWeek    = scheduleType === 'WEEKLY' ? document.getElementById('sub-day').value : null;
1670
1671         const slotOrders = [];
1672         let valid = true;
1673
1674         document.querySelectorAll('[id^="slot-row-"]').forEach(row => {
1675             const idNum      = row.id.split('-')[2];
1676             const mealSlot    = document.getElementById(`slot-meal-${idNum}`)?.value;
1677             const restaurantVal = document.getElementById(`slot-restaurant-${idNum}`)?.value;
1678
1679             if(!mealSlot || !restaurantVal){ valid = false; return; }
1680
1681             const restaurantId = parseInt(restaurantVal);
1682             if(isNaN(restaurantId)){ valid = false; return; }
1683
1684             slotOrders.push({ mealSlot, restaurantId });
1685         });
1686
1687         if(!valid || slotOrders.length === 0){
1688             showAlert('Please add at least one valid slot','error');
1689             return;
1690         }
1691
1692         // Derive mealSlots from slotOrders to satisfy @NotEmpty
1693         const mealSlots = slotOrders.map(s => s.mealSlot);
1694
1695         try{
1696             const res = await fetch(`${API}/subscription`, {
1697                 method: 'POST',
1698                 headers: {'Content-Type':'application/json', ...getAuthHeader()},
1699                 credentials: 'include',
1700                 body: JSON.stringify({ scheduleType, dayOfWeek, mealSlots, slotOrders })
1701             });
1702             const text = await res.text();
1703             if(res.ok){
1704                 showAlert('Subscription added successfully! ','success');
1705                 clearSubscriptionForm();
1706                 loadMySubscriptions();
1707             } else {
1708                 showAlert('Error: '+text,'error');
1709             }
1710         }catch(e){ showAlert('Failed: '+e.message,'error'); }
1711     }
1712
1713     function clearSubscriptionForm(){
1714         document.getElementById('sub-schedule').value = 'DAILY';
1715         document.getElementById('day-of-week-group').style.display = 'none';
1716         document.getElementById('slot-rows').innerHTML = '';
1717         slotRowCount = 0;
1718     }
1719
1720     /* =====
1721     MY CALENDAR (NEW)
1722     ===== */
1723     async function loadMyCalendar(){
1724         // 1. Load balance cards
1725         await loadCalendarBalanceCards();
1726         // 2. Build or refresh FullCalendar
1727         if(!calendarInitialized){
1728             initFullCalendar();
1729         } else {
1730             await refreshCalendarEvents();
1731         }
1732     }
1733
1734     async function loadCalendarBalanceCards(){
1735         // Default state
1736         ['cal-sick-remaining','cal-casual-remaining','cal-mat-remaining','cal-total-used']
1737             .forEach(id=>{ document.getElementById(id).textContent='--'; });
1738
1739         try{

```

```

1740     const dashboard = await apiFetch('/api/employees/me/leave-dashboard');
1741     const balances = dashboard?.leaveBalances || {};
1742
1743     let totalUsed=0, totalAllowed=0;
1744
1745     // SICK
1746     const sick = balances['SICK'] || balances['sick'] || {};
1747     const sickRem = Number(sick.available || 0);
1748     const sickTotal = Number(sick.total || 0);
1749     const sickUsed = sickTotal - sickRem;
1750     document.getElementById('cal-sick-remaining').textContent = sickRem;
1751     document.getElementById('cal-sick-sub').textContent = `${sickUsed} used of ${sickTotal}
days`;
1752
1753     // CASUAL
1754     const cas = balances['CASUAL'] || balances['casual'] || {};
1755     const casRem = Number(cas.available || 0);
1756     const casTotal = Number(cas.total || 0);
1757     const casUsed = casTotal - casRem;
1758     document.getElementById('cal-casual-remaining').textContent = casRem;
1759     document.getElementById('cal-casual-sub').textContent = `${casUsed} used of ${casTotal}
days`;
1760
1761     // MATERNITY
1762     const mat = balances['MATERNITY'] || balances['maternity'] || {};
1763     const matRem = Number(mat.available || 0);
1764     const matTotal = Number(mat.total || 0);
1765     const matUsed = matTotal - matRem;
1766     document.getElementById('cal-mat-remaining').textContent = matRem;
1767     document.getElementById('cal-mat-sub').textContent = `${matUsed} used of ${matTotal}
days`;
1768
1769     // TOTAL
1770     Object.values(balances).forEach(b=>{
1771         totalUsed += (Number(b.total||0) - Number(b.available||0));
1772         totalAllowed += Number(b.total||0);
1773     });
1774     document.getElementById('cal-total-used').textContent = totalUsed;
1775     document.getElementById('cal-total-sub').textContent = `out of ${totalAllowed} days total`;
1776
1777     }catch(e){
1778         console.error('Could not load leave balance for calendar:', e);
1779         ['cal-sick-sub', 'cal-casual-sub', 'cal-mat-sub', 'cal-total-sub']
1780             .forEach(id=>{ document.getElementById(id).textContent='unavailable'; });
1781     }
1782 }
1783
1784 async function buildCalendarEvents(){
1785     const events = [];
1786     try{
1787         // - Employee leaves -
1788         const data = await apiFetch('/leave_requests/employee');
1789         const leaves = data?.requests || data || [];
1790         leaves.forEach(l=>{
1791             const s = (l.status||'PENDING').toUpperCase();
1792             const c = STATUS_COLORS[s] || STATUS_COLORS.PENDING;
1793             const typeLabel = TYPE_LABEL[l.leaveType] || l.leaveType || 'Leave';
1794
1795             // FullCalendar end date is exclusive, so add 1 day for multi-day events
1796             let endDate = l.endDate;
1797             if(endDate){
1798                 const d = new Date(endDate);
1799                 d.setDate(d.getDate()+1);
1800                 endDate = d.toISOString().split('T')[0];
1801             }
1802
1803             events.push({
1804                 id : `leave-${l.leaveRequestId}`,
1805                 title : `${typeLabel} (${s.charAt(0)+s.slice(1).toLowerCase()}`,
1806                 start : l.startDate,
1807                 end : endDate,
1808                 allDay : true,
1809                 backgroundColor : c.bg,
1810                 borderColor : c.border,
1811                 textColor : c.text,
1812                 extendedProps : {

```

```

1813         type    : 'leave',
1814         status   : s,
1815         reason   : l.reason || '-',
1816         leaveType: l.leaveType
1817     }
1818 });
1819 });
1820 }catch(e){ console.warn('Could not load leaves for calendar:', e); }
1821
1822 try{
1823     // - Public holidays (admin-added) -
1824     const holidays = await apiFetch('/holidays');
1825     (holidays||[]).forEach(h=>{
1826         const c = STATUS_COLORS.HOLIDAY;
1827         events.push({
1828             id      : `holiday-${h.id||h.holidayId||Math.random()}`,
1829             title   : `${h.name||h.holidayName||'Holiday'}`,
1830             start   : h.date||h.holidayDate,
1831             allDay  : true,
1832             backgroundColor : c.bg,
1833             borderColor  : c.border,
1834             textColor    : c.text,
1835             extendedProps : { type:'holiday', description: h.description||'' }
1836         });
1837     });
1838 }catch(e){ console.warn('Could not load holidays for calendar:', e); }
1839
1840 return events;
1841 }
1842
1843 function initFullCalendar(){
1844     const calendarEl = document.getElementById('employee-calendar');
1845
1846     employeeCalendar = new FullCalendar.Calendar(calendarEl, {
1847         initialView: 'dayGridMonth',
1848         height     : 'auto',
1849         headerToolbar:{
1850             left : 'prev,next today',
1851             center: 'title',
1852             right : 'dayGridMonth,dayGridWeek,listMonth'
1853         },
1854         buttonText:{
1855             today : 'Today',
1856             month : 'Month',
1857             week  : 'Week',
1858             listMonth: 'List'
1859         },
1860         dayMaxEvents: 3, // "+N more" link if overflowing
1861         navLinks     : true, // click day number day view
1862         events       : [], // start empty, populate below
1863
1864         // Event hover tooltip
1865         eventMouseEnter: function(info){
1866             const ep = info.event.extendedProps;
1867             const tooltip = document.getElementById('cal-tooltip');
1868             let html = `<div class="tt-title">${info.event.title}</div>`;
1869             if(ep.type==='leave'){
1870                 html += `<div class="tt-row"> ${info.event.startStr}
1871                 ${info.event.endStr||info.event.startStr}</div>`;
1872                 if(ep.reason && ep.reason!=='-')
1873                     html += `<div class="tt-row"> ${ep.reason.substring(0,60)}</div>`;
1874                 html += `<div class="tt-row">Status: <strong
1875                 style="color:${STATUS_COLORS[ep.status]?.text||'#fff'}">${ep.status}</strong></div>`;
1876             } else {
1877                 if(ep.description) html += `<div class="tt-row">${ep.description}</div>`;
1878             }
1879             tooltip.innerHTML = html;
1880             tooltip.style.display = 'block';
1881         },
1882         eventMouseLeave: function(){
1883             document.getElementById('cal-tooltip').style.display='none';
1884         },
1885         eventClick: function(info){
1886             // Click navigates to leave history
1887             const ep = info.event.extendedProps;
1888             if(ep.type==='leave'){

```

```

1887         showView('my-leaves', document.querySelector('.nav-item:nth-child(4)'));
1888     }
1889 },
1890 });
1891
1892     employeeCalendar.render();
1893     calendarInitialized = true;
1894
1895     // Load events asynchronously after render
1896     refreshCalendarEvents();
1897 }
1898
1899     async function refreshCalendarEvents(){
1900         if(!employeeCalendar) return;
1901         const events = await buildCalendarEvents();
1902         // Remove all existing events and re-add
1903         employeeCalendar.removeAllEvents();
1904         events.forEach(e=>employeeCalendar.addEvent(e));
1905     }
1906
1907     async function refreshCalendar(){
1908         await loadCalendarBalanceCards();
1909         await refreshCalendarEvents();
1910         showAlert('Calendar refreshed!', 'success');
1911     }
1912
1913     // Move tooltip with mouse
1914     document.addEventListener('mousemove', function(e){
1915         const tt = document.getElementById('cal-tooltip');
1916         if(tt.style.display==='block'){
1917             tt.style.left = (e.clientX+14)+'px';
1918             tt.style.top = (e.clientY+14)+'px';
1919             // Keep tooltip on screen
1920             const rect = tt.getBoundingClientRect();
1921             if(rect.right > window.innerWidth)
1922                 tt.style.left = (e.clientX - rect.width - 10)+'px';
1923             if(rect.bottom > window.innerHeight)
1924                 tt.style.top = (e.clientY - rect.height - 10)+'px';
1925         }
1926     });
1927
1928     // ===== MY DEVICES =====
1929     async function loadMyAssignedDevices() {
1930         try {
1931             const devices = await apiFetch('/devices/my');
1932             const tbody = document.getElementById('my-devices-list');
1933             if (!devices || !devices.length) {
1934                 tbody.innerHTML = '<tr><td colspan="7" class="empty-state">No devices assigned to
you</td></tr>';
1935                 return;
1936             }
1937             const today = new Date();
1938             tbody.innerHTML = devices.map(d => {
1939                 const warrantyExpiry = d.warrantyExpiryDate ? new Date(d.warrantyExpiryDate) : null;
1940                 const warrantyDisplay = warrantyExpiry
1941                     ? (warrantyExpiry > today
1942                       ? `<span style="color:var(--green);font-size:11px"> ${d.warrantyExpiryDate}</span>`
1943                       : `<span style="color:var(--red);font-size:11px"> Expired
${d.warrantyExpiryDate}</span>`)
1944                   : `<span style="color:var(--text3);font-size:11px"></span>`;
1945                 return `<tr>
1946 <td>#${d.id}</td>
1947 <td><strong>${d.deviceName || '-'}</strong></td>
1948 <td>${d.brand || '-'}</td>
1949 <td><span class="chip">${(d.deviceType || '').toLowerCase()}</span></td>
1950 <td>${warrantyDisplay}</td>
1951 <td style="font-size:12px;color:var(--text2)">${d.assignedDate || '-'}</td>
1952 <td><span class="chip">${(d.deviceStatus || '').toLowerCase()}>${d.deviceStatus || '-'}</span></td>
1953 </tr>`;
1954             }).join('');
1955         } catch (e) {
1956             document.getElementById('my-devices-list').innerHTML =
1957                 '<tr><td colspan="7" class="empty-state">Could not load devices</td></tr>';
1958         }
1959     }
1960

```

```

1961 // ===== SERVICE REQUESTS =====
1962 async function loadServiceRequests() {
1963     await Promise.all([loadServiceRequestsList(), loadMyDevicesForSR()]);
1964 }
1965
1966 async function loadServiceRequestsList() {
1967     try {
1968         const requests = await apiFetch('/service-requests/my');
1969         const tbody = document.getElementById('service-requests-list');
1970         if (!requests || !requests.length) {
1971             tbody.innerHTML = '<tr><td colspan="7" class="empty-state">No service requests yet</td></tr>';
1972             return;
1973         }
1974         tbody.innerHTML = requests.map(r => `<tr>
1975             <td style="color:var(--text3);font-family:var(--mono)">#${r.id}</td>
1976             <td>${r.deviceName || '-'}</td>
1977             <td><span class="chip pending">${r.requestType}</span></td>
1978             <td style="max-width:200px;overflow:hidden;text-overflow:ellipsis;white-space:nowrap">
1979                 ${r.issueDescription || '-'}
1980             </td>
1981             <td>${r.urgent
1982                 ? '<span class="chip rejected">YES</span>'
1983                 : '<span style="color:var(--text3)">No</span>'}
1984             </td>
1985             <td><span class="chip ${r.status || ''}.toLowerCase()">${r.status || '-'}</span></td>
1986             <td style="font-size:12px;color:var(--text3)">
1987                 ${r.raisedAt ? new Date(r.raisedAt).toLocaleDateString() : '-'}
1988             </td>
1989         </tr>`).join('');
1990     } catch (e) { console.error(e); }
1991 }
1992
1993 async function loadMyDevicesForSR() {
1994     try {
1995         const devices = await apiFetch('/devices/my');
1996         const select = document.getElementById('sr-device-select');
1997         select.innerHTML = '<option value="">-- Select your device --</option>';
1998         (devices || []).forEach(d => {
1999             const opt = document.createElement('option');
2000             opt.value = d.id;
2001             opt.textContent = `${d.deviceName} (${d.brand || d.deviceType || 'Device'})`;
2002             select.appendChild(opt);
2003         });
2004     } catch (e) { console.error('Failed to load devices for SR', e); }
2005 }
2006
2007 async function submitServiceRequest() {
2008     const deviceId = parseInt(document.getElementById('sr-device-select').value);
2009     const requestType = document.getElementById('sr-type').value;
2010     const issueDescription = document.getElementById('sr-description').value.trim();
2011     const urgent = document.getElementById('sr-urgent').checked;
2012
2013     if (!deviceId) { showAlert('Please select a device', 'error'); return; }
2014     if (!issueDescription) { showAlert('Please describe the issue', 'error'); return; }
2015
2016     try {
2017         await apiFetch('/service-requests', {
2018             method: 'POST',
2019             body: JSON.stringify({ deviceId, requestType, issueDescription, urgent })
2020         });
2021         showAlert('Service request submitted!', 'success');
2022         clearServiceRequestForm();
2023         loadServiceRequestsList();
2024     } catch (e) { showAlert('Failed: ' + e.message, 'error'); }
2025 }
2026
2027 function clearServiceRequestForm() {
2028     document.getElementById('sr-device-select').value = '';
2029     document.getElementById('sr-type').value = 'REPAIR';
2030     document.getElementById('sr-description').value = '';
2031     document.getElementById('sr-urgent').checked = false;
2032 }
2033
2034 async function searchEmployee() {
2035     const name = document.getElementById('org-search-input').value.trim();
2036     const errorEl = document.getElementById('org-search-error');

```

```

2037     const resultsEl = document.getElementById('org-search-results');
2038     const emptyEl = document.getElementById('org-search-empty');
2039
2040     errorEl.style.display = 'none';
2041     resultsEl.style.display = 'none';
2042     emptyEl.style.display = 'block';
2043
2044     if (!name) {
2045         errorEl.textContent = 'Please enter a name to search.';
2046         errorEl.style.display = 'block';
2047         return;
2048     }
2049
2050     try {
2051         const emp = await apiFetch(`/manager/employees/search?name=${encodeURIComponent(name)}`);
2052
2053         // Populate info card
2054         document.getElementById('sr-name').textContent = emp.name || '-';
2055         document.getElementById('sr-id').textContent = `#${emp.employeeId}`;
2056         document.getElementById('sr-email').textContent = emp.email || '-';
2057         document.getElementById('sr-dept').textContent = emp.dept || '-';
2058         document.getElementById('sr-attendance').textContent = emp.attendance || '-';
2059
2060         // Status chip
2061         const statusMap = { ACTIVE: 'approved', ON_LEAVE: 'pending', INACTIVE: 'rejected' };
2062         const statusKey = (emp.status || '').toString();
2063         document.getElementById('sr-status').innerHTML =
2064             `${statusKey || '-'}`;
2065
2066         // Devices table
2067         const today = new Date();
2068         const devices = emp.devices || [];
2069         document.getElementById('sr-devices').innerHTML = devices.length
2070             ? devices.map(d => {
2071                 const exp = d.warrantyExpiryDate ? new Date(d.warrantyExpiryDate) : null;
2072                 const warranty = exp
2073                     ? (exp > today
2074                         ? `

```

```

2105     localStorage.removeItem('refresh_token');
2106     localStorage.removeItem('auth_type');
2107     localStorage.removeItem('last_role');
2108
2109     // Call server logout to invalidate session
2110     fetch(`${API}/session/logout`, {
2111         method: 'POST',
2112         credentials: 'include'
2113     }).finally(() => {
2114         // Force expire the session cookie on client side
2115         document.cookie = 'JSESSIONID=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/;';
2116         window.location.href = '/google.html';
2117     });
2118 }
2119 async function loadNotifCount() {
2120     try {
2121         const data = await apiFetch('/notifications/unread-count');
2122         const badge = document.getElementById('notif-badge');
2123         if (data.count > 0) {
2124             badge.textContent = data.count;
2125             badge.style.display = 'flex';
2126         } else {
2127             badge.style.display = 'none';
2128         }
2129     } catch(e) { console.warn('Could not load notif count', e); }
2130 }
2131
2132 async function toggleNotifPanel() {
2133     const panel = document.getElementById('notif-panel');
2134     if (panel.style.display === 'none') {
2135         panel.style.display = 'block';
2136         await loadNotifications();
2137         try { await apiFetch('/notifications/mark-read', { method: 'POST' }); } catch(e) {}
2138         document.getElementById('notif-badge').style.display = 'none';
2139     } else {
2140         panel.style.display = 'none';
2141     }
2142 }
2143
2144 async function loadNotifications() {
2145     try {
2146         const data = await apiFetch('/notifications');
2147         const list = document.getElementById('notif-list');
2148         list.innerHTML = data.length
2149             ? data.map(n => `
2150                 <div class="notif-item ${n.read ? '' : 'unread'}">
2151                     <div>${n.message}</div>
2152                     <div class="notif-time">${new Date(n.createdAt).toLocaleString()}</div>
2153                 </div>`).join('')
2154             : '<div class="notif-item">No notifications yet.</div>';
2155     } catch(e) { console.warn('Could not load notifications', e); }
2156 }
2157
2158 loadNotifCount();
2159 setInterval(loadNotifCount, 30000);
2160
2161 document.addEventListener('click', e => {
2162     const panel = document.getElementById('notif-panel');
2163     const wrapper = document.querySelector('.notif-wrapper');
2164     if (panel && wrapper && !panel.contains(e.target) && !wrapper.contains(e.target)) {
2165         panel.style.display = 'none';
2166     }
2167 });
2168 function showAlert(msg,type){
2169     const el=document.createElement('div');
2170     el.className=`alert alert-${type}`;el.textContent=msg;
2171     document.body.appendChild(el);setTimeout(()=>el.remove(),3000);
2172 }
2173
2174 // Gender store
2175 let currentManagerGender = null;
2176
2177 // Load profile data
2178 async function loadProfile() {
2179     try {
2180         const me = await apiFetch('/employee/me');

```

```

2181         if(me) {
2182             currentManagerGender = me.gender;
2183
2184             document.getElementById('p-name').textContent = me.name || '-';
2185             document.getElementById('p-email').textContent = me.email || '-';
2186             document.getElementById('p-dept').textContent = me.dept || '-';
2187             document.getElementById('p-role').textContent = me.role || 'MANAGER';
2188             document.getElementById('p-status').textContent = me.status || '-';
2189             document.getElementById('p-tz').textContent = me.timezone || '-';
2190
2191             // Show maternity section only for female managers
2192             const maternityBtn = document.getElementById('maternity-apply-btn');
2193             if(maternityBtn) {
2194                 maternityBtn.style.display = me.gender === 'F' ? 'block' : 'none';
2195             }
2196
2197             // Also update sidebar name with real name
2198             document.getElementById('sidebar-name').textContent = me.name || '-';
2199             document.getElementById('avatar-initials').textContent =
2200                 (me.name || me.email || 'M').charAt(0).toUpperCase();
2201         }
2202     } catch(e) {
2203         console.error('Could not load profile:', e);
2204     }
2205 }
2206
2207 // Auto calculate maternity end date
2208 document.getElementById('mat-start-date').addEventListener('change', function(){
2209     if(this.value) {
2210         const start = new Date(this.value);
2211         const end = new Date(start);
2212         end.setDate(end.getDate() + 179);
2213         document.getElementById('mat-end-date').value =
2214             end.toISOString().split('T')[0];
2215     } else {
2216         document.getElementById('mat-end-date').value = '';
2217     }
2218 });
2219
2220 // Submit maternity leave
2221 async function applyMaternityLeave() {
2222     const startDate = document.getElementById('mat-start-date').value;
2223     const reason = document.getElementById('mat-reason').value.trim();
2224
2225     if(!startDate) {
2226         showAlert('Please select a start date', 'error');
2227         return;
2228     }
2229
2230     try {
2231         await apiFetch('/leave_requests/maternity/apply', {
2232             method: 'POST',
2233             body: JSON.stringify({ startDate, reason })
2234         });
2235         showAlert('Maternity leave submitted successfully!', 'success');
2236         document.getElementById('mat-start-date').value = '';
2237         document.getElementById('mat-end-date').value = '';
2238         document.getElementById('mat-reason').value = '';
2239         loadMyLeaves();
2240     } catch(e) {
2241         showAlert('Failed: ' + e.message, 'error');
2242     }
2243 }
2244
2245 // ===== TWO-FACTOR AUTH =====
2246 async function loadTotpStatus() {
2247     const chip = document.getElementById('totp-status-chip');
2248     chip.textContent = 'Checking...';
2249     chip.className = 'chip';
2250     document.getElementById('qr-area').style.display = 'none';
2251     document.getElementById('totp-verify-input').value = '';
2252     document.getElementById('totp-verify-error').style.display = 'none';
2253     try {
2254         const data = await apiFetch('/totp/status');
2255         if (data.totpEnabled) {
2256             chip.textContent = 'ENABLED';
2257             chip.className = 'chip approved';

```



```

2257         document.getElementById('totp-setup-section').style.display = 'none';
2258         document.getElementById('totp-disable-section').style.display = 'block';
2259     } else {
2260         chip.textContent = 'DISABLED';
2261         chip.className = 'chip rejected';
2262         document.getElementById('totp-setup-section').style.display = 'block';
2263         document.getElementById('totp-disable-section').style.display = 'none';
2264     }
2265 } catch(e) {
2266     chip.textContent = 'Error';
2267     chip.className = 'chip pending';
2268 }
2269 }
2270
2271 async function generateTotpQr() {
2272     const btn = document.getElementById('generate-qr-btn');
2273     btn.textContent = 'Generating...';
2274     btn.disabled = true;
2275     try {
2276         const data = await apiFetch('/totp/setup', { method: 'POST' });
2277         document.getElementById('totp-secret-display').textContent = data.secret || '';
2278         const qrUri = encodeURIComponent(data.qrCodeUri || '');
2279         document.getElementById('qr-img').src =
2280             `https://api.qrserver.com/v1/create-qr-code/?size=180x180&data=${qrUri}`;
2281         document.getElementById('qr-area').style.display = 'block';
2282     } catch(e) {
2283         showAlert('Failed to generate QR: ' + e.message, 'error');
2284     } finally {
2285         btn.textContent = 'Regenerate QR Code';
2286         btn.disabled = false;
2287     }
2288 }
2289
2290 async function verifyAndActivateTotp() {
2291     const code = document.getElementById('totp-verify-input').value.trim();
2292     const errEl = document.getElementById('totp-verify-error');
2293     errEl.style.display = 'none';
2294     if (code.length !== 6) {
2295         errEl.textContent = 'Please enter a 6-digit code.';
2296         errEl.style.display = 'block';
2297         return;
2298     }
2299     try {
2300         await apiFetch('/totp/verify-setup', {
2301             method: 'POST',
2302             body: JSON.stringify({ code })
2303         });
2304         showAlert('2FA activated! You will need your authenticator on next login.', 'success');
2305         loadTotpStatus();
2306     } catch(e) {
2307         errEl.textContent = 'Invalid code. Try again.';
2308         errEl.style.display = 'block';
2309     }
2310 }
2311
2312 async function disableTotp() {
2313     if (!confirm('Are you sure you want to disable two-factor authentication?')) return;
2314     try {
2315         await apiFetch('/totp/disable', { method: 'DELETE' });
2316         showAlert('2FA has been disabled.', 'success');
2317         loadTotpStatus();
2318     } catch(e) {
2319         showAlert('Failed to disable 2FA: ' + e.message, 'error');
2320     }
2321 }
2322 </script>
2323 </body>
2324 </html>
2325

```

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <title>Account Pending</title>
6      <style>
7          * { margin: 0; padding: 0; box-sizing: border-box; }
8          body {
9              font-family: 'Outfit', sans-serif;
10             background: #070910;
11             color: #dde4f0;
12             min-height: 100vh;
13             display: flex;
14             align-items: center;
15             justify-content: center;
16         }
17         .card {
18             background: #0e1118;
19             border: 1px solid rgba(255,255,255,0.07);
20             border-radius: 16px;
21             padding: 48px;
22             max-width: 420px;
23             text-align: center;
24         }
25         .icon { font-size: 48px; margin-bottom: 20px; }
26         h1 { font-size: 22px; font-weight: 700; margin-bottom: 12px; }
27         p { font-size: 14px; color: #6b7591; line-height: 1.6; margin-bottom: 24px; }
28         .email {
29             background: #131821;
30             border: 1px solid rgba(79,140,255,0.2);
31             border-radius: 8px;
32             padding: 10px 16px;
33             font-size: 13px;
34             color: #4f8cff;
35             margin-bottom: 28px;
36             font-family: monospace;
37         }
38         .btn {
39             padding: 10px 24px;
40             background: transparent;
41             border: 1px solid rgba(255,255,255,0.1);
42             border-radius: 8px;
43             color: #6b7591;
44             font-size: 13px;
45             cursor: pointer;
46         }
47         .btn:hover { border-color: rgba(255,255,255,0.2); color: #dde4f0; }
48     </style>
49 </head>
50 <body>
51 <div class="card">
52     <div class="icon"></div>
53     <h1>Account Pending Approval</h1>
54     <p>You've successfully signed in with Google. Your account is currently awaiting approval from an admin
or manager.</p>
55     <div class="email" id="user-email">Loading...</div>
56     <p>Once approved, you'll have full access to the portal. Please contact your manager or admin.</p>
57     <button class="btn" onclick="logout()">Sign out</button>
58 </div>
59
60 <script>
61     // Show the user their email from JWT
62     const token = localStorage.getItem('jwt_token');
63     if (token) {
64         try {
65             const payload = JSON.parse(atob(token.split('.')[1]));
66             document.getElementById('user-email').textContent = payload.sub || 'Unknown';
67         } catch(e) {}
68     }
69
70     function logout() {
71         localStorage.clear();

```

```
72         window.location.href = '/google.html';
73     }
74 </script>
75 </body>
76 </html>
```

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>TechVendor Hub</title>
7      <link
href="https://fonts.googleapis.com/css2?family=Syne:wght@400;500;600;700;800&family=DM+Mono:wght@400;500&display
=swap" rel="stylesheet">
8      <style>
9          *{margin:0;padding:0;box-sizing:border-box}
10         :root{
11             --bg:#080b10;--bg2:#0c1018;--bg3:#111620;--bg4:#161d2a;
12             --panel:#131a26;--panel2:#1a2235;
13             --border:rgba(0,210,255,0.08);--border2:rgba(0,210,255,0.18);--border3:rgba(0,210,255,0.35);
14             --text:#d4e8f5;--text2:#6b8aaa;--text3:#3a5270;
15             --cyan:#00d2ff;--cyan-dim:rgba(0,210,255,0.08);--cyan-glow:rgba(0,210,255,0.15);
16             --amber:#f59e0b;--amber-dim:rgba(245,158,11,0.1);
17             --green:#10d48e;--green-dim:rgba(16,212,142,0.1);
18             --red:#f05454;--red-dim:rgba(240,84,84,0.1);
19             --violet:#a78bfa;--violet-dim:rgba(167,139,250,0.1);
20             --orange:#fb923c;--orange-dim:rgba(251,146,60,0.1);
21             --font:'Syne',sans-serif;--mono:'DM Mono',monospace;
22             --sidebar:260px;
23         }
24         body{font-family:var(--font);background:var(--bg);color:var(--text);font-
size:14px;overflow:hidden;height:100vh}
25         .app{display:flex;height:100vh}
26
27         body::before{content:'';position:fixed;inset:0;background:repeating-linear-
gradient(0deg,transparent,transparent 2px,rgba(0,0,0,0.03) 2px,rgba(0,0,0,0.03) 4px);pointer-events:none;z-
index:9...
28
29         .sidebar{width:var(--sidebar);background:var(--bg2);border-right:1px solid var(--
border);display:flex;flex-direction:column;flex-shrink:0;overflow:hidden;position:relative}
30         .sidebar::after{content:'';position:absolute;bottom:0;left:0;right:0;height:300px;background:linear-
gradient(to top,rgba(0,210,255,0.03),transparent);pointer-events:none}
31
32         .logo{padding:24px 20px 20px;border-bottom:1px solid var(--border)}
33         .logo-mark{display:flex;align-items:center;gap:10px;margin-bottom:6px}
34         .logo-hex{width:36px;height:36px;background:linear-gradient(135deg,#00d2ff,#0066ff);clip-
path:polygon(50% 0%,100% 25%,100% 75%,50% 100%,0% 75%,0% 25%);display:flex;align-items:center;justif...
35         .logo-name{font-size:17px;font-weight:800;color:var(--text);letter-spacing:-0.5px}
36         .logo-sub{font-size:10px;color:var(--text3);font-family:var(--mono);letter-spacing:1px;text-
transform:uppercase}
37         .vendor-badge{display:inline-flex;align-items:center;gap:6px;margin-top:10px;padding:5px
12px;background:var(--cyan-dim);border:1px solid var(--border2);border-radius:4px;font-size:10px;font...
38         .badge-dot{width:5px;height:5px;background:var(--cyan);border-radius:50%;animation:pulse 2s
infinite}
39         @keyframes pulse{0%,100%{opacity:1;box-shadow:0 0 0 0 var(--cyan-glow)}50%{opacity:0.7;box-shadow:0
0 0 4px transparent}}
40
41         .nav{flex:1;padding:12px 0;overflow-y:auto}
42         .nav::-webkit-scrollbar{width:3px}
43         .nav::-webkit-scrollbar-thumb{background:var(--border2);border-radius:2px}
44         .nav-section{padding:14px 20px 5px;font-size:9px;font-weight:700;color:var(--text3);letter-
spacing:2.5px;text-transform:uppercase;font-family:var(--mono)}
45         .nav-item{display:flex;align-items:center;gap:10px;padding:10px 20px;margin:1px 8px;border-
radius:6px;cursor:pointer;transition:all 0.15s;color:var(--text2);font-size:13px;font-weight:500;p...
46         .nav-item:hover{background:var(--cyan-dim);color:var(--cyan)}
47         .nav-item.active{background:var(--cyan-dim);color:var(--cyan);border:1px solid var(--border2)}
48         .nav-item.active::before{content:'';position:absolute;left:-8px;top:50%;transform:translateY(-
50%);width:3px;height:60%;background:var(--cyan);border-radius:0 2px 2px 0}
49         .nav-icon{width:16px;height:16px;opacity:0.7;flex-shrink:0}
50         .nav-item.active .nav-icon,.nav-item:hover .nav-icon{opacity:1}
51         .nav-badge{margin-left:auto;background:var(--red);color:#fff;font-size:9px;font-
weight:700;padding:2px 6px;border-radius:3px;font-family:var(--mono)}
52         .nav-badge.amber{background:var(--amber)}
53
54         .sidebar-footer{padding:16px 20px;border-top:1px solid var(--border)}
55         .user-row{display:flex;align-items:center;gap:10px}
56         .avatar{width:34px;height:34px;background:linear-gradient(135deg,#00d2ff22,#0066ff22);border:1px

```

```

solid var(--border2);border-radius:6px;display:flex;align-items:center;justify-content:cente...
57     .user-name{font-size:12px;font-weight:600;color:var(--text)}
58     .user-role{font-size:10px;color:var(--cyan);font-family:var(--mono);text-transform:uppercase;letter-
spacing:0.8px}
59
60     .main{flex:1;display:flex;flex-direction:column;overflow:hidden}
61     .topbar{background:var(--bg2);padding:0 24px;height:56px;display:flex;justify-content:space-
between;align-items:center;border-bottom:1px solid var(--border);flex-shrink:0}
62     .page-title{font-size:15px;font-weight:700;color:var(--text);display:flex;align-
items:center;gap:8px}
63     .page-title-prefix{color:var(--cyan);font-family:var(--mono);font-size:12px;font-weight:400}
64     .topbar-right{display:flex;align-items:center;gap:10px}
65     .topbar-email{font-size:11px;color:var(--text3);font-family:var(--mono)}
66     .signout-btn{padding:6px 14px;background:transparent;border:1px solid var(--border2);border-
radius:5px;color:var(--text2);font-family:var(--font);font-size:11px;font-weight:600;cursor:point...
67     .signout-btn:hover{background:var(--red-dim);border-color:var(--red);color:var(--red)}
68
69     .content{flex:1;overflow-y:auto;padding:24px}
70     .content::-webkit-scrollbar{width:4px}
71     .content::-webkit-scrollbar-thumb{background:var(--border2)}
72     .view{display:none}
73     .view.active{display:block;animation:viewIn 0.25s ease}
74     @keyframes viewIn{from{opacity:0;transform:translateY(6px)}to{opacity:1;transform:translateY(0)}}
75
76     .stats-row{display:grid;grid-template-columns:repeat(4,1fr);gap:14px;margin-bottom:22px}
77     .stat{background:var(--panel);border:1px solid var(--border);border-radius:8px;padding:18px
20px;position:relative;overflow:hidden;cursor:default;transition:border-color 0.2s}
78     .stat:hover{border-color:var(--border2)}
79     .stat::after{content:'';position:absolute;top:0;left:0;right:0;height:2px}
80     .stat.c{--c:var(--cyan)} .stat.g{--c:var(--green)} .stat.a{--c:var(--amber)} .stat.r{--c:var(--red)}
.stat.v{--c:var(--violet)} .stat.o{--c:var(--orange)}
81     .stat::after{background:var(--c)}
82     .stat-label{font-size:10px;color:var(--text3);text-transform:uppercase;letter-spacing:1px;font-
weight:600;margin-bottom:8px;font-family:var(--mono)}
83     .stat-val{font-size:28px;font-weight:800;color:var(--text);line-height:1;font-family:var(--mono)}
84     .stat-sub{font-size:11px;color:var(--text3);margin-top:5px}
85     .stat-icon{position:absolute;right:16px;top:50%;transform:translateY(-50%);font-
size:24px;opacity:0.12}
86
87     .card{background:var(--panel);border:1px solid var(--border);border-radius:10px;margin-
bottom:18px;overflow:hidden}
88     .card-head{padding:16px 20px;border-bottom:1px solid var(--border);display:flex;justify-
content:space-between;align-items:center}
89     .card-title{font-size:13px;font-weight:700;color:var(--text);letter-spacing:0.2px}
90     .card-sub{font-size:11px;color:var(--text3);margin-top:2px;font-family:var(--mono)}
91     .card-body{padding:20px}
92     .dash-grid{display:grid;grid-template-columns:1fr 1fr;gap:18px}
93
94     .tablewrap{overflow-x:auto}
95     table{width:100%;border-collapse:collapse}
96     th{padding:9px 16px;text-align:left;font-size:9px;color:var(--text3);font-weight:700;letter-
spacing:1.5px;text-transform:uppercase;background:var(--bg3);border-bottom:1px solid var(--border...
97     td{padding:12px 16px;border-bottom:1px solid var(--border);font-size:12px;color:var(--text2);font-
family:var(--mono)}
98     tr:last-child td{border-bottom:none}
99     tr:hover td{background:rgba(0,210,255,0.02);color:var(--text)}
100     td.main-cell{font-family:var(--font);font-size:13px;color:var(--text);font-weight:500}
101
102     .chip{display:inline-flex;align-items:center;gap:5px;padding:3px 9px;border-radius:3px;font-
size:10px;font-weight:700;letter-spacing:0.8px;text-transform:uppercase;font-family:var(--mono)}
103     .chip::before{content:'';width:4px;height:4px;border-radius:50%;flex-shrink:0}
104     .chip.available{background:var(--green-dim);color:var(--green);border:1px solid
rgba(16,212,142,0.2)} .chip.available::before{background:var(--green)}
105     .chip.assigned{background:var(--cyan-dim);color:var(--cyan);border:1px solid var(--border2)}
.chip.assigned::before{background:var(--cyan)}
106     .chip.under_repair{background:var(--amber-dim);color:var(--amber);border:1px solid
rgba(245,158,11,0.2)} .chip.under_repair::before{background:var(--amber)}
107     .chip.condemned{background:var(--red-dim);color:var(--red);border:1px solid rgba(240,84,84,0.2)}
.chip.condemned::before{background:var(--red)}
108     .chip.returned_to_vendor{background:var(--violet-dim);color:var(--violet);border:1px solid
rgba(167,139,250,0.2)} .chip.returned_to_vendor::before{background:var(--violet)}
109     .chip.open{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(245,158,11,0.2)}
.chip.open::before{background:var(--amber)}
110     .chip.acknowledged{background:var(--cyan-dim);color:var(--cyan);border:1px solid var(--border2)}
.chip.acknowledged::before{background:var(--cyan)}
111     .chip.sent_for_repair{background:var(--orange-dim);color:var(--orange);border:1px solid

```

```

111 rgba(251,146,60,0.2)} .chip.sent_for_repair::before{background:var(--orange)}
112 .chip.repair_done{background:var(--green-dim);color:var(--green);border:1px solid
113 rgba(16,212,142,0.2)} .chip.repair_done::before{background:var(--green)}
114 .chip.resolved{background:var(--green-dim);color:var(--green);border:1px solid
115 rgba(16,212,142,0.2)} .chip.resolved::before{background:var(--green)}
116 .chip.rejected{background:var(--red-dim);color:var(--red);border:1px solid rgba(240,84,84,0.2)}
117 .chip.rejected::before{background:var(--red)}
118 .chip.sent{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(245,158,11,0.2)}
119 .chip.sent::before{background:var(--amber)}
120 .chip.in_progress{background:var(--orange-dim);color:var(--orange);border:1px solid
121 rgba(251,146,60,0.2)} .chip.in_progress::before{background:var(--orange)}
122 .chip.fixed{background:var(--green-dim);color:var(--green);border:1px solid rgba(16,212,142,0.2)}
123 .chip.fixed::before{background:var(--green)}
124 .chip.irreparable{background:var(--red-dim);color:var(--red);border:1px solid rgba(240,84,84,0.2)}
125 .chip.irreparable::before{background:var(--red)}
126 .chip.laptop{background:var(--cyan-dim);color:var(--cyan);border:1px solid var(--border2)}
127 .chip.laptop::before{background:var(--cyan)}
128 .chip.phone{background:var(--violet-dim);color:var(--violet);border:1px solid
129 rgba(167,139,250,0.2)} .chip.phone::before{background:var(--violet)}
130 .chip.tablet{background:var(--green-dim);color:var(--green);border:1px solid rgba(16,212,142,0.2)}
131 .chip.tablet::before{background:var(--green)}
132 .chip.monitor,.chip.other,.chip.desktop,.chip.keyboard,.chip.mouse,.chip.headset{background:var(--
133 amber-dim);color:var(--amber);border:1px solid rgba(245,158,11,0.2)} .chip.monitor::before,...
134 .chip.active_assign{background:var(--green-dim);color:var(--green);border:1px solid
135 rgba(16,212,142,0.2)} .chip.active_assign::before{background:var(--green)}
136 .chip.returned{background:var(--text3);color:var(--text2);border:1px solid var(--border)}
137 .chip.returned::before{background:var(--text2)}
138 .chip.not_fixable{background:var(--red-dim);color:var(--red);border:1px solid rgba(240,84,84,0.2)}
139 .chip.not_fixable::before{background:var(--red)}
140
141
142 .btn{padding:8px 16px;border-radius:6px;border:none;font-family:var(--font);font-size:12px;font-
143 weight:700;cursor:pointer;transition:all 0.2s;letter-spacing:0.3px}
144 .btn-primary{background:var(--cyan);color:#080b10}
145 .btn-primary:hover{background:#33dbff;box-shadow:0 0 20px var(--cyan-glow)}
146 .btn-ghost{background:transparent;color:var(--text2);border:1px solid var(--border2)}
147 .btn-ghost:hover{border-color:var(--border3);color:var(--text)}
148 .btn-danger{background:var(--red-dim);color:var(--red);border:1px solid rgba(240,84,84,0.2)}
149 .btn-danger:hover{background:var(--red);color:#fff}
150 .btn-warn{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(245,158,11,0.2)}
151 .btn-warn:hover{background:var(--amber);color:#080b10}
152 .btn-success{background:var(--green-dim);color:var(--green);border:1px solid rgba(16,212,142,0.2)}
153 .btn-success:hover{background:var(--green);color:#080b10}
154 .btn-sm{padding:5px 10px;font-size:11px}
155 .btn-xs{padding:3px 8px;font-size:10px}
156 .action-row{display:flex;gap:6px;flex-wrap:wrap}
157
158
159 .form-grid{display:grid;grid-template-columns:1fr 1fr;gap:14px}
160 .form-group{display:flex;flex-direction:column;gap:6px}
161 .form-group.full{grid-column:1/-1}
162 .form-label{font-size:9px;font-weight:700;color:var(--text3);text-transform:uppercase;letter-
163 spacing:1.5px;font-family:var(--mono)}
164 .form-input{background:var(--bg3);border:1px solid var(--border);border-radius:6px;padding:10px
165 13px;color:var(--text);font-family:var(--mono);font-size:12px;transition:border-color 0.2s;wi...
166 .form-input:focus{outline:none;border-color:var(--cyan);box-shadow:0 0 3px var(--cyan-dim)}
167 .form-input:placeholder{color:var(--text3)}
168 select.form-input option{background:var(--bg2)}
169 .form-actions{display:flex;gap:10px;margin-top:18px}
170
171
172 .empty{text-align:center;padding:40px 20px;color:var(--text3);font-family:var(--mono);font-
173 size:12px}
174
175 .empty-icon{font-size:32px;margin-bottom:10px;opacity:0.4}
176
177 .alert{position:fixed;top:20px;right:20px;padding:12px 20px;border-radius:6px;z-
178 index:2000;animation:alertIn 0.25s ease;font-size:12px;font-weight:600;font-family:var(--mono);display:flex;a...
179 .alert-success{background:var(--green-dim);color:var(--green);border:1px solid
180 rgba(16,212,142,0.3)}
181 .alert-error{background:var(--red-dim);color:var(--red);border:1px solid rgba(240,84,84,0.3)}
182 @keyframes alertIn{from{transform:translateX(120%);opacity:0}to{transform:translateX(0);opacity:1}}
183
184
185 .search-row{display:flex;gap:10px;padding:14px 16px;border-bottom:1px solid var(--border)}
186 .search-input{flex:1;background:var(--bg3);border:1px solid var(--border);border-
187 radius:6px;padding:8px 13px;color:var(--text);font-family:var(--mono);font-size:12px}
188 .search-input:focus{outline:none;border-color:var(--cyan)}
189 .search-input:placeholder{color:var(--text3)}
190
191
192 .timeline{padding:20px;display:flex;flex-direction:column;gap:0}

```

```

166     .tl-item{display:flex;gap:14px;padding-bottom:20px;position:relative}
167     .tl-item:last-child{padding-bottom:0}
168     .tl-item:not(:last-
child)::after{content:'';position:absolute;left:14px;top:28px;bottom:0;width:1px;background:var(--border)}
169     .tl-dot{width:28px;height:28px;border-radius:50%;border:2px solid var(--border2);background:var(--
bg3);display:flex;align-items:center;justify-content:center;flex-shrink:0;font-size:11px;z-...
170     .tl-dot.cyan{border-color:var(--cyan);background:var(--cyan-dim)}
171     .tl-dot.amber{border-color:var(--amber);background:var(--amber-dim)}
172     .tl-dot.green{border-color:var(--green);background:var(--green-dim)}
173     .tl-dot.red{border-color:var(--red);background:var(--red-dim)}
174     .tl-content{flex:1;padding-top:4px}
175     .tl-title{font-size:12px;font-weight:600;color:var(--text);margin-bottom:2px}
176     .tl-meta{font-size:10px;color:var(--text3);font-family:var(--mono)}
177
178     /* Outcome hint box */
179     .outcome-hint{margin:12px 0 0;padding:10px 14px;border-radius:6px;font-size:11px;font-family:var(--
mono);transition:all 0.2s}
180     .outcome-hint.success{background:var(--green-dim);color:var(--green);border:1px solid
rgba(16,212,142,0.2)}
181     .outcome-hint.danger{background:var(--red-dim);color:var(--red);border:1px solid
rgba(240,84,84,0.2)}
182
183     /* Inline repair panel */
184     .repair-inline{background:var(--bg3);border:1px solid var(--border2);border-radius:8px;padding:18px
20px;margin:0 16px 16px;animation:viewIn 0.2s ease}
185
186     .modal-overlay{display:none;position:fixed;inset:0;background:rgba(8,11,16,0.85);z-
index:1500;align-items:center;justify-content:center;backdrop-filter:blur(4px)}
187     .modal-overlay.open{display:flex}
188     .modal{background:var(--panel2);border:1px solid var(--border2);border-radius:12px;width:560px;max-
width:95%;max-height:85vh;overflow-y:auto;animation:modalIn 0.2s ease}
189     @keyframes modalIn{from{opacity:0;transform:scale(0.96)}to{opacity:1;transform:scale(1)}}
190     .modal-head{padding:20px 24px;border-bottom:1px solid var(--border);display:flex;justify-
content:space-between;align-items:center}
191     .modal-title{font-size:16px;font-weight:700;color:var(--text)}
192     .modal-close{background:none;border:none;color:var(--text3);font-size:20px;cursor:pointer;padding:0
4px;transition:color 0.2s}
193     .modal-close:hover{color:var(--text)}
194     .modal-body{padding:24px}
195     .detail-grid{display:grid;grid-template-columns:1fr 1fr;gap:16px;margin-bottom:20px}
196     .detail-item{display:flex;flex-direction:column;gap:4px}
197     .detail-label{font-size:9px;font-weight:700;color:var(--text3);text-transform:uppercase;letter-
spacing:1.5px;font-family:var(--mono)}
198     .detail-value{font-size:13px;color:var(--text);font-family:var(--mono)}
199 </style>
200 </head>
201 <body>
202 <div class="app">
203
204     <!-- SIDEBAR -->
205     <div class="sidebar">
206         <div class="logo">
207             <div class="logo-mark">
208                 <div class="logo-hex">TV</div>
209                 <div>
210                     <div class="logo-name">TechVendor</div>
211                     <div class="logo-sub">Asset Hub</div>
212                 </div>
213             </div>
214             <div class="vendor-badge"><span class="badge-dot"></span>Tech Vendor</div>
215         </div>
216
217         <nav class="nav">
218             <div class="nav-section">Overview</div>
219             <div class="nav-item active" onclick="showView('dashboard',this)">
220                 <svg class="nav-icon" fill="none" viewBox="0 0 16 16"><rect x="1" y="1" width="6"
height="6" rx="1" stroke="currentColor" stroke-width="1.5"/><rect x="9" y="1" width="6" height="6" ...
221                 Dashboard
222             </div>
223
224             <div class="nav-section">Devices</div>
225             <div class="nav-item" onclick="showView('my-devices',this)">
226                 <svg class="nav-icon" fill="none" viewBox="0 0 16 16"><rect x="1" y="3" width="14"
height="10" rx="2" stroke="currentColor" stroke-width="1.5"/><path d="M5 13v2M11 13v2M3 15h10" str...
227                 My Devices
228             </div>

```

```

229         <div class="nav-item" onclick="showView('add-device',this)">
230             <svg class="nav-icon" fill="none" viewBox="0 0 16 16"><circle cx="8" cy="8" r="7"
stroke="currentColor" stroke-width="1.5"/><path d="M8 5v6M5 8h6" stroke="currentColor" stroke-width...
231             Add Device
232         </div>
233
234         <div class="nav-section">Service</div>
235         <div class="nav-item" onclick="showView('service-requests',this)">
236             <svg class="nav-icon" fill="none" viewBox="0 0 16 16"><path d="M8 11l1.5 4.5H14l-3.5 2.5 1.5 1.5
4.5L8 10l-4 2.5 1.5-4.5L2 5.5h4.5L8 1z" stroke="currentColor" stroke-width="1.5" stroke-l...
237             Service Requests
238             <span class="nav-badge amber" id="sr-badge" style="display:none">0</span>
239         </div>
240
241         <div class="nav-section">Assignments</div>
242         <div class="nav-item" onclick="showView('assignments',this)">
243             <svg class="nav-icon" fill="none" viewBox="0 0 16 16"><circle cx="6" cy="5" r="3"
stroke="currentColor" stroke-width="1.5"/><path d="M1 14c0-3 2-5 5-5" stroke="currentColor" stroke-...
244             Assignments
245         </div>
246
247         <div class="nav-section">Account</div>
248         <div class="nav-item" onclick="showView('two-factor',this)">
249             <svg class="nav-icon" fill="none" viewBox="0 0 16 16"><rect x="1" y="7" width="14"
height="8" rx="2" stroke="currentColor" stroke-width="1.5"/><path d="M4 7V5a4 4 0 0 1 8 0v2" strok...
250             Two-Factor Auth
251         </div>
252         <div class="nav-item" onclick="showView('profile',this)">
253             <svg class="nav-icon" fill="none" viewBox="0 0 16 16"><circle cx="8" cy="5" r="3"
stroke="currentColor" stroke-width="1.5"/><path d="M2 14c0-3.3 2.7-6 6-6s6 2.7 6 6" stroke="current...
254             My Profile
255         </div>
256     </nav>
257
258     <div class="sidebar-footer">
259         <div class="user-row">
260             <div class="avatar" id="sidebar-avatar">TV</div>
261             <div>
262                 <div class="user-name" id="sidebar-name">TechVendor</div>
263                 <div class="user-role">Tech Vendor</div>
264             </div>
265         </div>
266     </div>
267 </div>
268
269 <!-- MAIN -->
270 <div class="main">
271     <div class="topbar">
272         <div class="page-title">
273             <span class="page-title-prefix">// </span>
274             <span id="pageTitle">dashboard</span>
275         </div>
276         <div class="topbar-right">
277             <span class="topbar-email" id="topbar-email"></span>
278             <button class="signout-btn" onclick="logout()">Sign Out</button>
279         </div>
280     </div>
281
282     <div class="content">
283
284         <!-- DASHBOARD -->
285         <div id="dashboard-view" class="view active">
286             <div class="stats-row">
287                 <div class="stat c">
288                     <div class="stat-label">Total Devices</div>
289                     <div class="stat-val" id="stat-total">--</div>
290                     <div class="stat-sub">in catalog</div>
291                     <div class="stat-icon"></div>
292                 </div>
293                 <div class="stat g">
294                     <div class="stat-label">Available</div>
295                     <div class="stat-val" id="stat-avail">--</div>
296                     <div class="stat-sub">ready to assign</div>
297                     <div class="stat-icon"></div>
298                 </div>
299                 <div class="stat a">

```



```

300         <div class="stat-label">Pending Repairs</div>
301         <div class="stat-val" id="stat-repair">--</div>
302         <div class="stat-sub">awaiting your update</div>
303         <div class="stat-icon"></div>
304     </div>
305     <div class="stat r">
306         <div class="stat-label">Open Requests</div>
307         <div class="stat-val" id="stat-open">--</div>
308         <div class="stat-sub">sent for repair</div>
309         <div class="stat-icon">!</div>
310     </div>
311 </div>
312
313     <div class="dash-grid">
314         <div class="card">
315             <div class="card-head">
316                 <div>
317                     <div class="card-title">Repair Queue</div>
318                     <div class="card-sub">devices sent to you for repair</div>
319                 </div>
320                 <button class="btn btn-ghost btn-sm" onclick="showView('service-
requests',null)">View all </button>
321             </div>
322             <div class="tablewrap">
323                 <table>
324                     <thead><tr><th>Device</th><th>Issue</th><th>Status</th></tr></thead>
325                     <tbody id="dash-sr"></tbody>
326                 </table>
327             </div>
328         </div>
329
330         <div class="card">
331             <div class="card-head">
332                 <div>
333                     <div class="card-title">Active Repairs</div>
334                     <div class="card-sub">currently being serviced</div>
335                 </div>
336                 <button class="btn btn-ghost btn-sm" onclick="showView('repair-
logs',null)">View all </button>
337             </div>
338             <div class="tablewrap">
339                 <table>
340                     <thead><tr><th>Device</th><th>Sent</th><th>Status</th></tr></thead>
341                     <tbody id="dash-repairs"></tbody>
342                 </table>
343             </div>
344         </div>
345     </div>
346
347     <div class="card">
348         <div class="card-head">
349             <div class="card-title">Device Status Overview</div>
350         </div>
351         <div class="tablewrap">
352             <table>
353                 <thead><tr><th>Device</th><th>Type</th><th>Status</th><th>Assigned
To</th></tr></thead>
354                 <tbody id="dash-devices"></tbody>
355             </table>
356         </div>
357     </div>
358 </div>
359
360 <!-- MY DEVICES -->
361 <div id="my-devices-view" class="view">
362     <div class="card">
363         <div class="card-head">
364             <div>
365                 <div class="card-title">My Device Catalog</div>
366                 <div class="card-sub">all devices supplied by you</div>
367             </div>
368             <div class="action-row">
369                 <select class="form-input btn-sm" id="device-filter" onchange="filterDevices()"
style="width:140px;padding:5px 10px;font-size:11px">
370                     <option value="">All Status</option>
371                     <option value="AVAILABLE">Available</option>

```

```

372             <option value="ASSIGNED">Assigned</option>
373             <option value="UNDER_REPAIR">Under Repair</option>
374             <option value="CONDEMNED">Condemned</option>
375             <option value="RETURNED_TO_VENDOR">Returned</option>
376         </select>
377         <button class="btn btn-ghost btn-sm" onclick="loadMyDevices()">
Refresh</button>
378     </div>
379 </div>
380 <div class="search-row">
381     <input class="search-input" id="device-search" placeholder="Search by name, serial,
brand..." oninput="filterDevices()">
382 </div>
383 <div class="tablewrap">
384     <table>
385         <thead><tr><th>ID</th><th>Device
Name</th><th>Brand</th><th>Type</th><th>Warranty</th><th>Status</th><th>Actions</th></tr></thead>
386         <tbody id="devices-list"></tbody>
387     </table>
388 </div>
389 </div>
390 </div>
391
392 <!-- ADD DEVICE -->
393 <div id="add-device-view" class="view">
394     <div class="card" style="max-width:700px">
395         <div class="card-head">
396             <div>
397                 <div class="card-title">Add New Device</div>
398                 <div class="card-sub">register a device to your catalog</div>
399             </div>
400         </div>
401         <div class="card-body">
402             <div class="form-grid">
403                 <div class="form-group">
404                     <label class="form-label">Device Name</label>
405                     <input class="form-input" id="d-name" placeholder="Dell XPS 15 9520">
406                 </div>
407                 <div class="form-group">
408                     <label class="form-label">Brand</label>
409                     <input class="form-input" id="d-brand" placeholder="Dell">
410                 </div>
411                 <div class="form-group">
412                     <label class="form-label">Device Type</label>
413                     <select class="form-input" id="d-type">
414                         <option value="LAPTOP">Laptop</option>
415                         <option value="DESKTOP">Desktop</option>
416                         <option value="PHONE">Phone</option>
417                         <option value="TABLET">Tablet</option>
418                         <option value="MONITOR">Monitor</option>
419                         <option value="KEYBOARD">Keyboard</option>
420                         <option value="MOUSE">Mouse</option>
421                         <option value="HEADSET">Headset</option>
422                         <option value="OTHER">Other</option>
423                     </select>
424                 </div>
425                 <div class="form-group">
426                     <label class="form-label">Warranty Expiry</label>
427                     <input class="form-input" type="date" id="d-warranty">
428                 </div>
429             </div>
430             <div class="form-actions">
431                 <button class="btn btn-primary" onclick="addDevice()">Register Device</button>
432                 <button class="btn btn-ghost" onclick="clearDeviceForm()">Clear</button>
433             </div>
434         </div>
435     </div>
436 </div>
437
438 <!--
439     SERVICE REQUESTS - vendor repair queue
440     GET /service-requests/vendor/me SENT_FOR_REPAIR list
441     PUT /service-requests/vendor RepairDTO { requestId, resolution, status }
442     status=REPAIR_DONE device becomes ASSIGNED
443     status=anything else device becomes CONDEMNED
444 -->

```

```

445         <div id="service-requests-view" class="view">
446             <div class="card">
447                 <div class="card-head">
448                     <div>
449                         <div class="card-title">Repair Queue</div>
450                         <div class="card-sub">devices sent to you – mark as repaired or condemned</div>
451                     </div>
452                     <button class="btn btn-ghost btn-sm" onclick="loadServiceRequests()">
Refresh</button>
453                 </div>
454                 <div class="tablewrap">
455                     <table>
456                         <thead>
457                             <tr>
458                                 <th>ID</th>
459                                 <th>Device</th>
460                                 <th>Issue Type</th>
461                                 <th>Description</th>
462                                 <th>Urgent</th>
463                                 <th>Raised</th>
464                                 <th>Actions</th>
465                             </tr>
466                         </thead>
467                         <tbody id="sr-list"></tbody>
468                     </table>
469                 </div>
470             </div>
471
472             <!-- INLINE REPAIR UPDATE PANEL -->
473             <div class="card" id="vendor-repair-panel" style="max-width:640px;display:none">
474                 <div class="card-head">
475                     <div>
476                         <div class="card-title">Update Repair Result</div>
477                         <div class="card-sub" id="vr-panel-subtitle">Request #–</div>
478                     </div>
479                     <button class="btn btn-ghost btn-sm" onclick="closeRepairPanel()"> Cancel</button>
480                 </div>
481                 <div class="card-body">
482                     <input type="hidden" id="vr-request-id">
483
484                     <div class="form-grid">
485                         <!-- Outcome selector -->
486                         <div class="form-group full">
487                             <label class="form-label">Repair Outcome</label>
488                             <select class="form-input" id="vr-status" onchange="onOutcomeChange()">
489                                 <option value="REPAIR_DONE"> Repair Done – device returns to
employee</option>
490                                 <option value="NOT_FIXABLE"> Could Not Repair – device will be
condemned</option>
491                             </select>
492                         </div>
493
494                         <!-- Resolution notes -->
495                         <div class="form-group full">
496                             <label class="form-label">Resolution Notes</label>
497                             <input class="form-input" id="vr-resolution"
placeholder="Describe what was repaired, parts replaced, or why it
could not be fixed...">
498                         </div>
499                     </div>
500
501                     <!-- Dynamic outcome hint -->
502                     <div id="vr-outcome-hint" class="outcome-hint success">
503                         Device status <strong>ASSIGNED</strong> – returned to the employee
504                     </div>
505
506                     <div class="form-actions">
507                         <button class="btn btn-primary" onclick="submitRepairUpdate()">Submit
Update</button>
508                         <button class="btn btn-ghost" onclick="closeRepairPanel()">Cancel</button>
509                     </div>
510                 </div>
511             </div>
512         </div>
513     </div>
514
515     <!-- ASSIGNMENTS -->

```

```

516         <div id="assignments-view" class="view">
517             <div class="card">
518                 <div class="card-head">
519                     <div>
520                         <div class="card-title">Device Assignments</div>
521                         <div class="card-sub">who has your devices</div>
522                     </div>
523                     <button class="btn btn-ghost btn-sm" onclick="loadAssignments()"> Refresh</button>
524                 </div>
525                 <div class="tablewrap">
526                     <table>
527                         <thead>
528                             <tr>
529                                 <th>Device</th>
530                                 <th>Type</th>
531                                 <th>Employee</th>
532                                 <th>Assigned Date</th>
533                                 <th>Status</th>
534                             </tr>
535                         </thead>
536                         <tbody id="assignments-list"></tbody>
537                     </table>
538                 </div>
539             </div>
540         </div>
541         <!-- TWO-FACTOR AUTH -->
542         <div id="two-factor-view" class="view">
543             <div class="card" style="max-width:560px">
544                 <div class="card-head">
545                     <div>
546                         <div class="card-title">Two-Factor Authentication</div>
547                         <div class="card-sub">secure your account with an authenticator app</div>
548                     </div>
549                     <span id="totp-status-chip" class="chip">checking...</span>
550                 </div>
551                 <div class="card-body">
552                     <!-- DISABLED – setup flow -->
553                     <div id="totp-setup-section">
554                         <p style="font-size:12px;color:var(--text2);margin-bottom:16px;
555                             line-height:1.7;font-family:var(--mono)">
556                             Use Google Authenticator, Authy, or any TOTP app.<br>
557                             Scan the QR code, then enter the 6-digit code to activate.
558                         </p>
559                         <button class="btn btn-primary" onclick="generateTotpQr()"
560                             id="generate-qr-btn">
561                             Generate QR Code
562                         </button>
563                     </div>
564                     <div id="qr-area" style="display:none;margin-top:24px;text-align:center">
565                         <img id="qr-img" src="" alt="QR Code"
566                             style="width:180px;height:180px;border-radius:8px;
567                             border:1px solid var(--border2);display:block;margin:0 auto">
568                         <div style="margin-top:12px;font-size:10px;color:var(--text3);
569                             font-family:var(--mono);letter-spacing:0.5px">
570                             CAN'T SCAN? ENTER KEY MANUALLY:
571                         </div>
572                         <div id="totp-secret-display"
573                             style="font-family:var(--mono);font-size:12px;color:var(--cyan);
574                             background:var(--bg3);border:1px solid var(--border2);
575                             border-radius:6px;padding:8px 12px;margin-top:6px;
576                             word-break:break-all;text-align:center"></div>
577                     </div>
578                     <div style="margin-top:20px;text-align:left">
579                         <label class="form-label" style="display:block;margin-bottom:8px">
580                             Enter 6-digit code to activate
581                         </label>
582                         <input id="totp-verify-input" type="text"
583                             inputmode="numeric" maxlength="6"
584                             placeholder="000000"
585                             class="form-input"
586                             style="text-align:center;font-size:20px;
587                             letter-spacing:10px;padding:12px"
588                             oninput="this.value=this.value.replace(/\D/g,'')">
589                         <button class="btn btn-primary"
590                             onclick="verifyAndActivateTotp()"
591

```

```

592             style="width:100%;margin-top:10px">
593             Activate 2FA
594         </button>
595         <div id="totp-verify-error"
596             style="display:none;margin-top:8px;font-size:11px;
597             color:var(--red);font-family:var(--mono)"></div>
598         </div>
599     </div>
600 </div>
601
602     <!-- ENABLED – disable option -->
603     <div id="totp-disable-section" style="display:none">
604         <div style="background:var(--green-dim);border:1px solid rgba(16,212,142,0.2);
605         border-radius:6px;padding:12px 16px;font-size:12px;
606         color:var(--green);font-family:var(--mono);margin-bottom:16px">
607             TWO-FACTOR AUTHENTICATION IS ACTIVE
608         </div>
609         <button class="btn btn-danger" onclick="disableTotp()">
610             Disable 2FA
611         </button>
612     </div>
613
614 </div>
615 </div>
616 </div>
617 <!-- PROFILE -->
618 <div id="profile-view" class="view">
619     <div class="card" style="max-width:600px">
620         <div class="card-head">
621             <div class="card-title">My Profile</div>
622         </div>
623         <div class="card-body">
624             <div style="display:flex;align-items:center;gap:16px;margin-bottom:24px;padding-
625             bottom:20px;border-bottom:1px solid var(--border)">
626                 <div style="width:56px;height:56px;background:var(--cyan-dim);border:2px solid
627                 var(--border2);border-radius:10px;display:flex;align-items:center;justify-content:center;f...
628                 <div style="font-size:18px;font-weight:700;color:var(--text)" id="profile-
629                 name">--</div>
630                 <div style="font-size:11px;color:var(--cyan);font-family:var(--
631                 mono);margin-top:3px">TECH_VENDOR</div>
632             </div>
633             <div class="detail-grid">
634                 <div class="detail-item"><div class="detail-label">Email</div><div
635                 class="detail-value" id="profile-email">--</div></div>
636                 <div class="detail-item"><div class="detail-label">Phone</div><div
637                 class="detail-value" id="profile-phone">--</div></div>
638                 <div class="detail-item"><div class="detail-label">Registered</div><div
639                 class="detail-value" id="profile-reg">--</div></div>
640                 <div class="detail-item"><div class="detail-label">Status</div><div
641                 class="detail-value"><span class="chip active_assign">ACTIVE</span></div></div>
642             </div>
643         </div>
644     </div>
645 </div>
646 <!-- DEVICE DETAIL MODAL -->
647 <div class="modal-overlay" id="device-modal">
648     <div class="modal">
649         <div class="modal-head">
650             <div class="modal-title" id="modal-device-name">Device Details</div>
651             <button class="modal-close" onclick="closeModal()"></button>
652         </div>
653         <div class="modal-body">
654             <div class="detail-grid" id="modal-detail-grid"></div>
655             <div style="margin-top:16px">
656                 <div class="card-title" style="margin-bottom:12px;font-size:12px;color:var(--text3);font-
657                 family:var(--mono);text-transform:uppercase;letter-spacing:1px">Repair History</div>
658                 <div id="modal-repair-history"></div>
659             </div>
660         </div>
661     </div>
662 </div>

```

```

659     </div>
660 </div>
661
662 <script>
663     const API = 'http://localhost:8080';
664     let devicesCache = [], srCache = [], repairsCache = [], assignmentsCache = [];
665
666     // AUTH
667     (function(){
668         const token = localStorage.getItem('jwt_token');
669         if(!token){ window.location.href='/login.html'; return; }
670         try{
671             const p = JSON.parse(atob(token.split('.')[1]));
672             const role = (p.role||'').replace(/^ROLE_/, '');
673             if(role !== 'TECH_VENDOR'){ window.location.href='/google.html'; return; }
674             const email = p.sub||p.email||'TechVendor';
675             document.getElementById('topbar-email').textContent = email;
676             document.getElementById('sidebar-name').textContent = email.split('@')[0];
677             document.getElementById('sidebar-avatar').textContent = email.charAt(0).toUpperCase();
678             document.getElementById('profile-avatar').textContent = email.charAt(0).toUpperCase();
679             document.getElementById('profile-email').textContent = email;
680         }catch(e){ window.location.href='/login.html'; }
681         initDashboard();
682     })();
683
684     function getAuthHeader(){
685         const t = localStorage.getItem('jwt_token');
686         return t ? {'Authorization': `Bearer ${t}` } : {};
687     }
688
689     async function apiFetch(path, opts={}){
690         const res = await fetch(`${API}${path}`, {
691             ...opts,
692             headers: {'Content-Type': 'application/json', ...getAuthHeader(), ...(opts.headers||{})},
693             credentials: 'include'
694         });
695         if(res.status===401){
696             localStorage.removeItem('jwt_token');
697             window.location.href='/login.html';
698             throw new Error('Unauthorized');
699         }
700         const text = await res.text();
701         if(!res.ok) throw new Error(text || 'Request failed');
702         return text ? JSON.parse(text) : null;
703     }
704
705     // NAV
706     function showView(name, el){
707         document.querySelectorAll('.view').forEach(v=>v.classList.remove('active'));
708         document.getElementById(`${name}-view`).classList.add('active');
709         document.querySelectorAll('.nav-item').forEach(n=>n.classList.remove('active'));
710         if(el) el.classList.add('active');
711         document.getElementById('pageTitle').textContent = name.replace(/-/g, ' ');
712         const loaders = {
713             'dashboard': loadDashboard,
714             'my-devices': loadMyDevices,
715             'service-requests': loadServiceRequests,
716             'repair-logs': loadRepairLogs,
717             'assignments': loadAssignments,
718             'profile': loadProfile,
719             'two-factor': loadTotpStatus,
720             'add-device': ()=>{}
721         };
722         if(loaders[name]) loaders[name]();
723     }
724
725     // INIT
726     async function initDashboard(){
727         await Promise.all([loadMyDevices(), loadServiceRequests(), loadRepairLogs(), loadAssignments()]);
728         loadDashboard();
729     }
730
731     // DASHBOARD
732     function loadDashboard(){
733         document.getElementById('stat-total').textContent = devicesCache.length || 0;
734         document.getElementById('stat-avail').textContent =

```

```

devicesCache.filter(d=>d.deviceStatus==='AVAILABLE').length || 0;
735     document.getElementById('stat-repair').textContent = srCache.length || 0;
736     document.getElementById('stat-open').textContent = srCache.length || 0;
737
738     const badge = document.getElementById('sr-badge');
739     if(srCache.length){ badge.textContent=srCache.length; badge.style.display=''; }
740     else badge.style.display='none';
741
742     document.getElementById('dash-sr').innerHTML = srCache.slice(0,5).map(s=>`
743         <tr>
744             <td class="main-cell">${s.deviceName||'Device #' +s.deviceId}</td>
745             <td>${(s.requestType||'').toLowerCase().replace('_',' ')}</td>
746             <td>${chip('SENT_FOR_REPAIR')}</td>
747         </tr>`).join('') ||
748         `<tr><td colspan="3" class="empty">No pending repairs</td></tr>`;
749
750     document.getElementById('dash-repairs').innerHTML =
repairsCache.filter(r=>r.repairStatus!=='FIXED'&&r.repairStatus!=='IRREPARABLE').slice(0,5).map(r=>`
751         <tr>
752             <td class="main-cell">${r.deviceName||'Device #' +r.deviceId}</td>
753             <td>${r.sentDate||'-'}</td>
754             <td>${chip(r.repairStatus)}</td>
755         </tr>`).join('') ||
756         `<tr><td colspan="3" class="empty">No active repairs</td></tr>`;
757
758     document.getElementById('dash-devices').innerHTML = devicesCache.slice(0,8).map(d=>`
759         <tr>
760             <td class="main-cell">${d.deviceName||'-'}</td>
761             <td>${chip(d.deviceType)}</td>
762             <td>${chip(d.deviceStatus)}</td>
763             <td>${d.assignedEmployeeName||'-'}</td>
764         </tr>`).join('') ||
765         `<tr><td colspan="4" class="empty">No devices</td></tr>`;
766     }
767
768     // DEVICES
769     async function loadMyDevices(){
770         try{
771             const data = await apiFetch('/devices/vendor/me');
772             devicesCache = data || [];
773             renderDevices(devicesCache);
774         }catch(e){ renderDevices([]); }
775     }
776
777     function renderDevices(list){
778         document.getElementById('devices-list').innerHTML = list.length
779         ? list.map(d=>`<tr>
780             <td style="font-family:var(--mono);font-size:11px;color:var(--text3)">#${d.id}</td>
781             <td class="main-cell">${d.deviceName||'-'}</td>
782             <td>${d.brand||'-'}</td>
783             <td>${chip(d.deviceType)}</td>
784             <td style="font-size:11px;font-family:var(--mono)">
785                 ${d.warrantyExpiryDate
786                 ? (new Date(d.warrantyExpiryDate) > new Date()
787                     ? `<span style="color:var(--green)"> ${d.warrantyExpiryDate}</span>`
788                     : `<span style="color:var(--red)"> ${d.warrantyExpiryDate}</span>`)
789                 : `<span style="color:var(--text3)"></span>`}
790             </td>
791             <td>${chip(d.deviceStatus)}</td>
792             <td><div class="action-row">
793                 <button class="btn btn-ghost btn-xs" onclick="viewDevice(${d.id})">Details</button>
794             </div></td>
795         </tr>`).join('')
796         : `<tr><td colspan="7"><div class="empty"><div class="empty-icon"></div>No devices registered
yet</div></td></tr>`;
797     }
798
799     function filterDevices(){
800         const q = document.getElementById('device-search').value.toLowerCase();
801         const f = document.getElementById('device-filter').value;
802         renderDevices(devicesCache.filter(d=>{
803             const matchQ = !q ||
(d.deviceName||'').toLowerCase().includes(q)|| (d.serialNumber||'').toLowerCase().includes(q)|| (d.brand||'').toLo
werCase().includes(q);
804             const matchF = !f || d.deviceStatus === f;
805             return matchQ && matchF;

```

```

806     }));
807 }
808
809 async function addDevice(){
810     const body = {
811         deviceName: document.getElementById('d-name').value.trim(),
812         brand: document.getElementById('d-brand').value.trim(),
813         deviceType: document.getElementById('d-type').value,
814         warrantyExpiryDate: document.getElementById('d-warranty').value || null,
815     };
816     if(!body.deviceName || !body.brand || !body.deviceType){
817         showAlert('Device name, brand and type are required','error'); return;
818     }
819     try{
820         await apiFetch('/devices', {method:'POST', body:JSON.stringify(body)});
821         showAlert('Device registered successfully!','success');
822         clearDeviceForm();
823         loadMyDevices();
824     }catch(e){ showAlert('Failed: '+e.message,'error'); }
825 }
826
827 function clearDeviceForm(){
828     ['d-name','d-brand','d-warranty'].forEach(id=>document.getElementById(id).value='');
829     document.getElementById('d-type').value='LAPTOP';
830 }
831
832 // SERVICE REQUESTS (vendor repair queue)
833 // GET /service-requests/vendor/me list of SENT_FOR_REPAIR requests
834 // PUT /service-requests/vendor { requestId, resolution, status }
835 // REPAIR_DONE device becomes ASSIGNED (back to employee)
836 // anything else device becomes CONDEMNED
837 //
838 async function loadServiceRequests(){
839     try{
840         const data = await apiFetch('/service-requests/vendor/me');
841         srCache = data || [];
842         renderServiceRequests(srCache);
843         // refresh badge
844         const badge = document.getElementById('sr-badge');
845         if(srCache.length){ badge.textContent=srCache.length; badge.style.display=''; }
846         else badge.style.display='none';
847     }catch(e){ renderServiceRequests([]); }
848 }
849
850 function renderServiceRequests(list){
851     document.getElementById('sr-list').innerHTML = list.length
852     ? list.map(s=><tr>
853         <td style="font-family:var(--mono);font-size:11px;color:var(--text3)">#${s.id}</td>
854         <td class="main-cell">${s.deviceName}||'Device #'+s.deviceId}</td>
855         <td>${(s.requestType||'').toLowerCase().replace(/_/g,' ')}</td>
856         <td style="max-width:200px;overflow:hidden;text-overflow:ellipsis;white-
857 space:nowrap;color:var(--text2)">${s.issueDescription}||'-'</td>
858         <td>${s.urgent
859         ? '<span style="color:var(--red);font-family:var(--mono);font-size:10px;font-weight:700">
URGENT</span>'
860         : '<span style="color:var(--text3);font-family:var(--mono);font-size:10px">—</span>'}</td>
861         <td style="font-size:10px;font-family:var(--mono);color:var(--text3)">${s.raisedAt ?
s.raisedAt.split('T')[0] : '-'}</td>
862         <td>
863             <button class="btn btn-success btn-xs" onclick="openRepairPanel(${s.id},
'${(s.deviceName}||'Device #'+s.deviceId).replace(/'/g,'\\')}')">
864                 Update Repair
865             </button>
866         </td>
867     </tr>`).join('')
868     : `<tr><td colspan="7"><div class="empty"><div class="empty-icon"></div>No devices in repair
queue</div></td></tr>`;
869 }
870
871 // REPAIR PANEL
872 function openRepairPanel(requestId, deviceName){
873     document.getElementById('vr-request-id').value = requestId;
874     document.getElementById('vr-panel-subtitle').textContent = `Request #${requestId} – ${deviceName}`;
875     document.getElementById('vr-status').value = 'REPAIR_DONE';
876     document.getElementById('vr-resolution').value = '';
877     onOutcomeChange(); // set correct hint

```



```

877     const panel = document.getElementById('vendor-repair-panel');
878     panel.style.display = 'block';
879     panel.scrollIntoView({behavior:'smooth', block:'start'});
880 }
881
882 function closeRepairPanel(){
883     document.getElementById('vendor-repair-panel').style.display = 'none';
884 }
885
886 function onOutcomeChange(){
887     const status = document.getElementById('vr-status').value;
888     const hint = document.getElementById('vr-outcome-hint');
889     if(status === 'REPAIR_DONE'){
890         hint.className = 'outcome-hint success';
891         hint.innerHTML = ' Device status <strong>ASSIGNED</strong> – returned to the employee';
892     } else {
893         hint.className = 'outcome-hint danger';
894         hint.innerHTML = ' Device status <strong>CONDEMNED</strong> – device will be marked unusable';
895     }
896 }
897
898 async function submitRepairUpdate(){
899     const requestId = document.getElementById('vr-request-id').value;
900     const status = document.getElementById('vr-status').value;
901     const resolution = document.getElementById('vr-resolution').value.trim();
902
903     if(!resolution){
904         showAlert('Please enter resolution notes','error'); return;
905     }
906
907     // RepairDTO: { requestId, resolution, status }
908     const body = {
909         requestId: parseInt(requestId),
910         resolution,
911         status // "REPAIR_DONE" ASSIGNED | anything else CONDEMNED
912     };
913
914     try{
915         await apiFetch('/service-requests/vendor', {method:'PUT', body:JSON.stringify(body)});
916         const label = status === 'REPAIR_DONE' ? 'Repair marked done – device returned to employee' :
917         'Device condemned';
918         showAlert(label, 'success');
919         closeRepairPanel();
920         // Refresh everything that could have changed
921         await Promise.all([loadServiceRequests(), loadMyDevices(), loadRepairLogs()]);
922         loadDashboard();
923     }catch(e){ showAlert('Failed: '+e.message,'error'); }
924
925     // REPAIR LOGS
926     async function loadRepairLogs(){
927         try{
928             const data = await apiFetch('/tech-vendor/repair-logs');
929             repairsCache = data || [];
930             renderRepairLogs(repairsCache);
931         }catch(e){ renderRepairLogs([]); }
932     }
933
934     function renderRepairLogs(list){
935         document.getElementById('repair-list').innerHTML = list.length
936         ? list.map(r=><tr>
937             <td style="font-family:var(--mono);font-size:11px;color:var(--text3)">#${r.id}</td>
938             <td class="main-cell">${r.deviceName} | Device #'+r.deviceId}</td>
939             <td style="max-width:140px;overflow:hidden;text-overflow:ellipsis;white-
940 space:nowrap">${r.issueReported} | '-'|</td>
941             <td>${r.technicianName} | '-'|</td>
942             <td style="font-size:11px;font-family:var(--mono)">${r.sentDate} | '-'|</td>
943             <td style="font-size:11px;font-family:var(--mono)">${r.expectedReturnDate} | '-'|</td>
944             <td style="font-family:var(--mono)">${r.repairCost ? ''+r.repairCost : '-'}</td>
945             <td>${chip(r.repairStatus)}</td>
946             <td><button class="btn btn-ghost btn-xs"
947 onclick="openRepairUpdate(${r.id})">Update</button></td>
948             </tr>` ).join('')
949         : `<tr><td colspan="9"><div class="empty"><div class="empty-icon"></div>No repair
950 logs</div></td></tr>`;
951     }

```

```

949
950     function openRepairUpdate(id){
951         document.getElementById('repair-id').value = id;
952         const form = document.getElementById('repair-update-form');
953         form.style.display = 'block';
954         form.scrollIntoView({behavior:'smooth'});
955     }
956
957     async function updateRepair(){
958         const id = document.getElementById('repair-id').value;
959         const body = {
960             repairStatus: document.getElementById('repair-status-input').value,
961             technicianName: document.getElementById('repair-tech').value.trim(),
962             repairCost: parseFloat(document.getElementById('repair-cost').value)||null,
963             invoiceNumber: document.getElementById('repair-invoice').value.trim(),
964             actualReturnDate: document.getElementById('repair-return-date').value||null,
965             fixedSuccessfully: document.getElementById('repair-fixed').value==='true',
966             repairNotes: document.getElementById('repair-notes').value.trim(),
967             postRepairAction: document.getElementById('repair-action').value
968         };
969         try{
970             await apiFetch(`/tech-vendor/repair-logs/${id}`, {method:'PUT', body:JSON.stringify(body)});
971             showAlert('Repair log updated!', 'success');
972             document.getElementById('repair-update-form').style.display='none';
973             loadRepairLogs(); loadMyDevices(); loadDashboard();
974         }catch(e){ showAlert('Failed: '+e.message, 'error'); }
975     }
976
977     // ASSIGNMENTS
978     async function loadAssignments(){
979         try{
980             const data = await apiFetch('/devices/assignments');
981             assignmentsCache = data || [];
982             document.getElementById('assignments-list').innerHTML = assignmentsCache.length
983                 ? assignmentsCache.map(a=>`<tr>
984                     <td class="main-cell">${a.deviceName||'-'}</td>
985                     <td>${chip(a.deviceType)}</td>
986                     <td>${a.employeeName||'-'}</td>
987                     <td style="font-size:11px;font-family:var(--mono)">${a.assignedDate||'-'}</td>
988                     <td>${chip(a.status==='ACTIVE'? 'active_assign':a.status?.toLowerCase())}</td>
989                 </tr>`).join('')
990             : `<tr><td colspan="5"><div class="empty"><div class="empty-icon"></div>No assignments
found</div></td></tr>`;
991         }catch(e){}
992     }
993
994     // PROFILE
995     async function loadProfile(){
996         try{
997             const data = await apiFetch('/vendors/me');
998             document.getElementById('profile-name').textContent = data.name || '--';
999             document.getElementById('profile-email').textContent = data.email || '--';
1000             document.getElementById('profile-phone').textContent = data.phone || '--';
1001             document.getElementById('profile-reg').textContent = data.registeredAt || '--';
1002             document.getElementById('profile-avatar').textContent = (data.name||'T').charAt(0).toUpperCase();
1003             document.getElementById('sidebar-name').textContent = data.name || 'TechVendor';
1004         }catch(e){}
1005     }
1006
1007     // DEVICE DETAIL MODAL
1008     async function viewDevice(id){
1009         const d = devicesCache.find(x=>x.id===id);
1010         if(!d) return;
1011         document.getElementById('modal-device-name').textContent = d.deviceName;
1012
1013         const today = new Date();
1014         const warrantyExpiry = d.warrantyExpiryDate ? new Date(d.warrantyExpiryDate) : null;
1015         const warrantyDisplay = warrantyExpiry
1016             ? (warrantyExpiry > today
1017                 ? `<span style="color:var(--green);font-family:var(--mono)"> Valid until
${d.warrantyExpiryDate}</span>`
1018                 : `<span style="color:var(--red);font-family:var(--mono)"> Expired
${d.warrantyExpiryDate}</span>`)
1019             : `<span style="color:var(--text3);font-family:var(--mono)">—</span>`;
1020
1021         document.getElementById('modal-detail-grid').innerHTML = `

```

```

1022         <div class="detail-item"><div class="detail-label">Brand</div><div class="detail-
value">${d.brand||'-'}</div></div>
1023         <div class="detail-item"><div class="detail-label">Type</div><div class="detail-
value">${chip(d.deviceType)}</div></div>
1024         <div class="detail-item"><div class="detail-label">Status</div><div class="detail-
value">${chip(d.deviceStatus)}</div></div>
1025         <div class="detail-item"><div class="detail-label">Warranty</div><div class="detail-
value">${warrantyDisplay}</div></div>
1026         <div class="detail-item"><div class="detail-label">Assigned To</div><div class="detail-
value">${d.assignedEmployeeName||'<span style="color:var(--text3)">Unassigned</span>'}</div></div>
1027         <div class="detail-item"><div class="detail-label">Added On</div><div class="detail-
value">${d.createdAt ? d.createdAt.split('T')[0] : '-'}</div></div>
1028     `;
1029
1030     const repairs = repairsCache.filter(r=>r.deviceId===id);
1031     document.getElementById('modal-repair-history').innerHTML = repairs.length
1032     ? `<div class="timeline">${repairs.map(r=>`
1033         <div class="tl-item">
1034             <div class="tl-dot
${r.repairStatus=== 'FIXED'? 'green':r.repairStatus=== 'IRREPARABLE'? 'red':r.repairStatus=== 'IN_PROGRESS'? 'amber': '
cyan'}"></div>
1035             <div class="tl-content">
1036                 <div class="tl-title">${r.issueReported||'Repair'}</div>
1037                 <div class="tl-meta">${chip(r.repairStatus)} · ${r.sentDate||'-'}
${r.actualReturnDate||'pending'} · ${r.repairCost ? ''+r.repairCost : 'cost pending'}</div>
1038             </div>
1039         </div>`}).join('')}</div>`
1040     : `<div class="empty" style="padding:16px">No repair history</div>`;
1041
1042     document.getElementById('device-modal').classList.add('open');
1043 }
1044
1045 function closeModal(){
1046     document.getElementById('device-modal').classList.remove('open');
1047 }
1048
1049 // HELPERS
1050 function chip(val){
1051     if(!val) return '-';
1052     const cls = val.toLowerCase().replace(/ /g, '_');
1053     return `<span class="chip ${cls}">${val.replace(/_/g, ' ')}</span>`;
1054 }
1055
1056 function showAlert(msg, type){
1057     const el = document.createElement('div');
1058     el.className = `alert alert-${type}`;
1059     el.innerHTML = `<span>${type==='success'? '' : ''}</span>${msg}`;
1060     document.body.appendChild(el);
1061     setTimeout(()=>el.remove(), 3500);
1062 }
1063
1064 function logout(){
1065     localStorage.removeItem('jwt_token');
1066     localStorage.removeItem('auth_type');
1067     fetch(`${API}/session/logout`, {method: 'POST'}).finally(()=>window.location.href='/vendor-
login.html');
1068 }
1069
1070 // TWO-FACTOR AUTH
1071 async function loadTotpStatus() {
1072     const chip = document.getElementById('totp-status-chip');
1073     chip.textContent = 'checking...';
1074     chip.className = 'chip';
1075     document.getElementById('qr-area').style.display = 'none';
1076     document.getElementById('totp-verify-input').value = '';
1077     document.getElementById('totp-verify-error').style.display = 'none';
1078     try {
1079         const data = await apiFetch('/totp/status');
1080         if (data.totpEnabled) {
1081             chip.textContent = 'enabled';
1082             chip.className = 'chip repair_done';
1083             document.getElementById('totp-setup-section').style.display = 'none';
1084             document.getElementById('totp-disable-section').style.display = 'block';
1085         } else {
1086             chip.textContent = 'disabled';
1087             chip.className = 'chip rejected';
1088             document.getElementById('totp-setup-section').style.display = 'block';

```

```

1088         document.getElementById('totp-disable-section').style.display = 'none';
1089     }
1090 } catch(e) {
1091     chip.textContent = 'error';
1092     chip.className = 'chip open';
1093 }
1094 }
1095
1096 async function generateTotpQr() {
1097     const btn = document.getElementById('generate-qr-btn');
1098     btn.textContent = 'Generating...';
1099     btn.disabled = true;
1100     try {
1101         const data = await apiFetch('/totp/setup', { method: 'POST' });
1102         document.getElementById('totp-secret-display').textContent = data.secret || '';
1103         const qrUri = encodeURIComponent(data.qrCodeUri || '');
1104         document.getElementById('qr-img').src =
1105             `https://api.qrserver.com/v1/create-qr-code/?size=180x180&data=${qrUri}`;
1106         document.getElementById('qr-area').style.display = 'block';
1107     } catch(e) {
1108         showAlert('Failed to generate QR: ' + e.message, 'error');
1109     } finally {
1110         btn.textContent = 'Regenerate QR Code';
1111         btn.disabled = false;
1112     }
1113 }
1114
1115 async function verifyAndActivateTotp() {
1116     const code = document.getElementById('totp-verify-input').value.trim();
1117     const errEl = document.getElementById('totp-verify-error');
1118     errEl.style.display = 'none';
1119     if (code.length !== 6) {
1120         errEl.textContent = 'Please enter a 6-digit code.';
1121         errEl.style.display = 'block';
1122         return;
1123     }
1124     try {
1125         await apiFetch('/totp/verify-setup', {
1126             method: 'POST',
1127             body: JSON.stringify({ code })
1128         });
1129         showAlert('2FA activated – required on next login', 'success');
1130         loadTotpStatus();
1131     } catch(e) {
1132         errEl.textContent = 'Invalid code. Try again.';
1133         errEl.style.display = 'block';
1134     }
1135 }
1136
1137 async function disableTotp() {
1138     if (!confirm('Disable two-factor authentication?')) return;
1139     try {
1140         await apiFetch('/totp/disable', { method: 'DELETE' });
1141         showAlert('2FA has been disabled', 'success');
1142         loadTotpStatus();
1143     } catch(e) {
1144         showAlert('Failed: ' + e.message, 'error');
1145     }
1146 }
1147 </script>
1148 </body>
1149 </html>

```

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>EMS – Two-Factor Authentication Setup</title>
7      <link
href="https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;500;600;700&family=JetBrains+Mono:wght@400;50
0&display=swap" rel="stylesheet">
8      <!-- qrcode.js for QR rendering -->
9      <script src="https://cdnjs.cloudflare.com/ajax/libs/qrcodejs/1.0.0/qrcode.min.js"></script>
10     <style>
11         *{margin:0;padding:0;box-sizing:border-box}
12         :root{
13             --bg:#070910;--panel:#0e1118;--panel2:#131821;
14             --border:rgba(255,255,255,0.07);--border-lit:rgba(255,255,255,0.16);
15             --text:#dde4f0;--text2:#6b7591;--text3:#3d4560;
16             --blue:#4f8cff;--blue-dim:rgba(79,140,255,0.13);
17             --green:#34d399;--green-dim:rgba(52,211,153,0.12);
18             --red:#f87171;--red-dim:rgba(248,113,113,0.12);
19             --amber:#f9c74f;
20             --font:'Outfit',sans-serif;--mono:'JetBrains Mono',monospace;
21         }
22         body{font-family:var(--font);background:var(--bg);color:var(--text);min-height:100vh;}
23
24         .bg-layer{position:fixed;inset:0;z-index:0;
25             background:
26                 radial-gradient(ellipse 50% 50% at 20% 30%, rgba(79,140,255,0.06) 0%,transparent 70%),
27                 radial-gradient(ellipse 40% 40% at 80% 70%, rgba(52,211,153,0.04) 0%,transparent 60%);
28         }
29         .grid-overlay{position:fixed;inset:0;z-index:0;
30             background-image:linear-gradient(rgba(255,255,255,0.015) 1px,transparent 1px),linear-
gradient(90deg,rgba(255,255,255,0.015) 1px,transparent 1px);
31             background-size:40px 40px;
32         }
33
34         /* Nav */
35         .nav{position:relative;z-index:10;display:flex;align-items:center;justify-content:space-
between;padding:18px 40px;border-bottom:1px solid var(--border);}
36         .nav-brand{display:flex;align-items:center;gap:10px;}
37         .nav-icon{width:36px;height:36px;background:linear-gradient(135deg,#4f8cff,#818cf8);border-
radius:9px;display:flex;align-items:center;justify-content:center;font-weight:700;font-size:14px;c...
38         .nav-name{font-size:16px;font-weight:700;}
39         .nav-back{font-size:13px;color:var(--text2);text-decoration:none;display:flex;align-
items:center;gap:6px;transition:color .2s;}
40         .nav-back:hover{color:var(--blue);}
41
42         /* Page */
43         .page{position:relative;z-index:1;max-width:680px;margin:0 auto;padding:48px 24px 80px;}
44
45         .page-header{margin-bottom:36px;}
46         .page-header h1{font-size:28px;font-weight:700;letter-spacing:-.4px;margin-bottom:6px;}
47         .page-header p{font-size:14px;color:var(--text2);line-height:1.6;}
48
49         /* Status badge */
50         .status-bar{display:flex;align-items:center;justify-content:space-between;background:var(--
panel);border:1px solid var(--border);border-radius:14px;padding:18px 22px;margin-bottom:28px;}
51         .status-left{display:flex;align-items:center;gap:14px;}
52         .status-icon{width:44px;height:44px;border-radius:12px;display:flex;align-items:center;justify-
content:center;font-size:22px;flex-shrink:0;}
53         .status-icon.on{background:var(--green-dim);}
54         .status-icon.off{background:var(--border);}
55         .status-title{font-size:15px;font-weight:600;margin-bottom:2px;}
56         .status-desc{font-size:12px;color:var(--text2);}
57         .badge{font-size:11px;font-weight:700;padding:4px 10px;border-radius:20px;letter-spacing:.04em;text-
transform:uppercase;}
58         .badge-on{background:var(--green-dim);color:var(--green);border:1px solid rgba(52,211,153,0.25);}
59         .badge-off{background:var(--border);color:var(--text2);border:1px solid var(--border-lit);}
60
61         /* Cards */
62         .card{background:var(--panel);border:1px solid var(--border);border-radius:14px;padding:24px;margin-
bottom:20px;}

```

```

63     .card-title{font-size:15px;font-weight:600;margin-bottom:4px;display:flex;align-
items:center;gap:8px;}
64     .card-subtitle{font-size:13px;color:var(--text2);margin-bottom:20px;}
65
66     /* Steps list */
67     .steps-list{display:flex;flex-direction:column;gap:16px;margin-bottom:24px;}
68     .step-row{display:flex;gap:14px;align-items:flex-start;}
69     .step-num{width:28px;height:28px;border-radius:50%;background:var(--blue-dim);border:1px solid
rgba(79,140,255,0.3);color:var(--blue);font-size:13px;font-weight:700;display:flex;align-items...
70     .step-text strong{font-size:13px;color:var(--text);display:block;margin-bottom:2px;}
71     .step-text span{font-size:12px;color:var(--text2);line-height:1.5;}
72
73     /* QR area */
74     .qr-area{display:none;flex-direction:column;align-
items:center;gap:20px;padding:24px;background:var(--panel2);border:1px solid var(--border);border-
radius:12px;margin-bottom:20px;}
75     .qr-area.show{display:flex;}
76     #qr-container{background:#fff;padding:14px;border-radius:10px;display:inline-block;}
77     .qr-container canvas, .qr-container img{display:block;}
78     .secret-box{width:100%;background:var(--bg);border:1px solid var(--border);border-
radius:9px;padding:12px 16px;text-align:center;}
79     .secret-label{font-size:11px;color:var(--text3);text-transform:uppercase;letter-
spacing:.06em;margin-bottom:6px;}
80     .secret-val{font-family:var(--mono);font-size:14px;color:var(--amber);letter-spacing:.12em;word-
break:break-all;}
81     .copy-btn{font-size:11px;color:var(--blue);background:none;border:none;cursor:pointer;font-
family:var(--font);padding:0;margin-top:4px;}
82     .copy-btn:hover{text-decoration:underline;}
83
84     /* OTP row */
85     .otp-row{display:flex;gap:10px;justify-content:center;margin:0 0 8px;}
86     .otp-digit{width:48px;height:58px;text-align:center;font-size:24px;font-family:var(--mono);font-
weight:500;background:var(--panel2);border:1px solid var(--border);border-radius:10px;color:v...
87     .otp-digit:focus{border-color:var(--blue);box-shadow:0 0 3px var(--blue-dim);}
88     .otp-digit.filled{border-color:rgba(79,140,255,0.4);}
89
90     /* Alerts */
91     .alert{border-radius:9px;padding:11px 14px;font-size:13px;margin-bottom:14px;display:none;align-
items:flex-start;gap:9px;}
92     .alert.show{display:flex;}
93     .alert-err{background:var(--red-dim);border:1px solid rgba(248,113,113,0.25);color:var(--red);}
94     .alert-ok{background:var(--green-dim);border:1px solid rgba(52,211,153,0.25);color:var(--green);}
95     .alert-icon{font-size:15px;flex-shrink:0;}
96
97     /* Buttons */
98     .btn{padding:12px 22px;border-radius:10px;border:none;font-family:var(--font);font-size:14px;font-
weight:600;cursor:pointer;transition:all .2s;}
99     .btn-primary{background:var(--blue);color:#fff;}
100    .btn-primary:hover{background:#6ba1ff;box-shadow:0 4px 16px rgba(79,140,255,0.3);}
101    .btn-primary.disabled{opacity:.5;cursor:not-allowed;box-shadow:none;}
102    .btn-danger{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.25);}
103    .btn-danger:hover{background:rgba(248,113,113,0.2);}
104    .btn-ghost{background:transparent;color:var(--text2);border:1px solid var(--border);}
105    .btn-ghost:hover{border-color:var(--border-lit);color:var(--text);}
106
107    .btn-row{display:flex;gap:10px;flex-wrap:wrap;}
108
109    .spinner{display:inline-block;width:14px;height:14px;border:2px solid rgba(255,255,255,0.2);border-
top-color:#fff;border-radius:50%;animation:spin .6s linear infinite;vertical-align:middle;...
110    @keyframes spin{to{transform:rotate(360deg)}}
111
112    /* Danger zone */
113    .danger-card{background:var(--red-dim);border:1px solid rgba(248,113,113,0.2);border-
radius:14px;padding:20px 24px;display:flex;align-items:center;justify-content:space-between;gap:16px;}
114    .danger-text strong{font-size:14px;color:var(--red);display:block;margin-bottom:3px;}
115    .danger-text span{font-size:12px;color:rgba(248,113,113,0.7);}
116
117    .loading{text-align:center;padding:60px 0;color:var(--text2);}
118    .loading .spinner{border-top-color:var(--blue);border-color:var(--border-
lit);width:24px;height:24px;}
119    </style>
120 </head>
121 <body>
122 <div class="bg-layer"></div>
123 <div class="grid-overlay"></div>
124

```

```

125 <nav class="nav">
126     <div class="nav-brand">
127         <div class="nav-icon">EMS</div>
128         <div class="nav-name">EmployeeMS</div>
129     </div>
130     <a class="nav-back" href="javascript:history.back()"> Back to dashboard</a>
131 </nav>
132
133 <div class="page">
134     <div class="page-header">
135         <h1>Two-Factor Authentication</h1>
136         <p>Add an extra layer of security to your account. Once enabled, you'll need to provide a 6-digit
code from your authenticator app every time you sign in.</p>
137     </div>
138
139     <!-- Loading state -->
140     <div class="loading" id="loading">
141         <span class="spinner"></span>
142     </div>
143
144     <!-- Main content (shown after status loads) -->
145     <div id="main-content" style="display:none;">
146
147         <!-- Status bar -->
148         <div class="status-bar">
149             <div class="status-left">
150                 <div class="status-icon" id="status-icon"></div>
151                 <div>
152                     <div class="status-title">Two-Factor Authentication</div>
153                     <div class="status-desc" id="status-desc">Loading...</div>
154                 </div>
155             </div>
156             <span class="badge" id="status-badge"></span>
157         </div>
158
159         <!-- ===== SETUP SECTION (shown when disabled) ===== -->
160         <div id="setup-section">
161             <div class="card">
162                 <div class="card-title"> Enable 2FA</div>
163                 <div class="card-subtitle">Follow these steps to add two-factor authentication to your
account.</div>
164
165                 <div class="steps-list">
166                     <div class="step-row">
167                         <div class="step-num">1</div>
168                         <div class="step-text">
169                             <strong>Install an authenticator app</strong>
170                             <span>Download <a href="https://support.google.com/accounts/answer/1066447"
target="_blank" style="color:var(--blue)">Google Authenticator</a> or <a href="https://authy...
171                         </div>
172                     </div>
173                     <div class="step-row">
174                         <div class="step-num">2</div>
175                         <div class="step-text">
176                             <strong>Scan the QR code</strong>
177                             <span>Click "Generate QR Code" below, then scan it with your authenticator app.
You can also enter the secret key manually.</span>
178                         </div>
179                     </div>
180                     <div class="step-row">
181                         <div class="step-num">3</div>
182                         <div class="step-text">
183                             <strong>Verify the setup</strong>
184                             <span>Enter the 6-digit code shown in your app to confirm everything is working
correctly.</span>
185                         </div>
186                     </div>
187                 </div>
188
189                 <!-- QR area -->
190                 <div class="qr-area" id="qr-area">
191                     <div id="qr-container"></div>
192                     <div class="secret-box">
193                         <div class="secret-label">Manual entry key (backup)</div>
194                         <div class="secret-val" id="secret-display"></div>
195                         <button class="copy-btn" onclick="copySecret()">Copy key</button>

```

```

196         </div>
197     </div>
198
199     <!-- Alerts -->
200     <div class="alert alert-err" id="setup-err"><span class="alert-icon"></span><span
id="setup-err-msg"></span></div>
201     <div class="alert alert-ok" id="setup-ok"><span class="alert-icon"></span><span id="setup-
ok-msg"></span></div>
202
203     <!-- Verify code input (shown after QR generated) -->
204     <div id="verify-area" style="display:none;">
205         <p style="font-size:13px;color:var(--text2);margin-bottom:16px;">Enter the code shown
in your authenticator app to activate 2FA:</p>
206         <div class="otp-row" id="setup-otp-row">
207             <input class="otp-digit" type="text" inputmode="numeric" maxlength="1" id="s0">
208             <input class="otp-digit" type="text" inputmode="numeric" maxlength="1" id="s1">
209             <input class="otp-digit" type="text" inputmode="numeric" maxlength="1" id="s2">
210             <input class="otp-digit" type="text" inputmode="numeric" maxlength="1" id="s3">
211             <input class="otp-digit" type="text" inputmode="numeric" maxlength="1" id="s4">
212             <input class="otp-digit" type="text" inputmode="numeric" maxlength="1" id="s5">
213         </div>
214     </div>
215
216     <div class="btn-row" style="margin-top:8px;">
217         <button class="btn btn-ghost" id="btn-generate" onclick="generateQr()"> Generate QR
Code</button>
218         <button class="btn btn-primary" id="btn-verify" style="display:none;" disabled
onclick="verifySetup()">Activate 2FA</button>
219     </div>
220 </div>
221 </div>
222
223     <!-- ===== ACTIVE SECTION (shown when enabled) ===== -->
224     <div id="active-section" style="display:none;">
225         <div class="card">
226             <div class="card-title"> 2FA is active</div>
227             <div class="card-subtitle">Your account is protected. Every sign-in requires your
authenticator app code.</div>
228             <p style="font-size:13px;color:var(--text2);line-height:1.6;">
229                 If you lose access to your authenticator app, contact your admin to reset 2FA. We
recommend also saving backup codes.
230             </p>
231         </div>
232
233         <div class="danger-card">
234             <div class="danger-text">
235                 <strong>Disable Two-Factor Authentication</strong>
236                 <span>This will remove the extra protection from your account.</span>
237             </div>
238             <button class="btn btn-danger" onclick="disableTotp()">Disable 2FA</button>
239         </div>
240     </div>
241
242 </div>
243 </div>
244
245 <script>
246     const token = () => localStorage.getItem('jwt_token');
247     const authHeader = () => ({ 'Authorization': 'Bearer ' + token(), 'Content-Type': 'application/json'
});
248
249     // ---- Load status on page open ----
250     async function loadStatus() {
251         try {
252             const res = await fetch('/totp/status', { headers: authHeader() });
253             if (res.status === 401) { window.location.href = '/google.html'; return; }
254             const data = await res.json();
255             renderStatus(data.totpEnabled);
256         } catch (e) {
257             console.error(e);
258         } finally {
259             document.getElementById('loading').style.display = 'none';
260             document.getElementById('main-content').style.display = 'block';
261         }
262     }
263

```



```

264     function renderStatus(enabled) {
265         const icon = document.getElementById('status-icon');
266         const desc = document.getElementById('status-desc');
267         const badge = document.getElementById('status-badge');
268         const setup = document.getElementById('setup-section');
269         const active = document.getElementById('active-section');
270
271         if (enabled) {
272             icon.textContent = '';
273             icon.className = 'status-icon on';
274             desc.textContent = 'Your account requires a 6-digit code on every sign-in.';
275             badge.textContent = 'Enabled';
276             badge.className = 'badge badge-on';
277             setup.style.display = 'none';
278             active.style.display = 'block';
279         } else {
280             icon.textContent = '';
281             icon.className = 'status-icon off';
282             desc.textContent = 'Not enabled – only your password protects your account.';
283             badge.textContent = 'Disabled';
284             badge.className = 'badge badge-off';
285             setup.style.display = 'block';
286             active.style.display = 'none';
287         }
288     }
289
290     // ---- Generate QR ----
291     let currentSecret = null;
292
293     async function generateQr() {
294         const btn = document.getElementById('btn-generate');
295         btn.disabled = true;
296         btn.innerHTML = '<span class="spinner"></span>Generating...';
297         clearAlerts();
298
299         try {
300             const res = await fetch('/totp/setup', { method: 'POST', headers: authHeader() });
301             const data = await res.json();
302             if (!res.ok) { showAlert('setup-err', data.error || 'Failed to generate QR code'); return; }
303
304             currentSecret = data.secret;
305
306             // Render QR code
307             const qrContainer = document.getElementById('qr-container');
308             qrContainer.innerHTML = '';
309             new QRCode(qrContainer, {
310                 text: data.qrCodeUri,
311                 width: 180,
312                 height: 180,
313                 colorDark: '#000000',
314                 colorLight: '#ffffff',
315                 correctLevel: QRCode.CorrectLevel.M
316             });
317
318             document.getElementById('secret-display').textContent = formatSecret(data.secret);
319             document.getElementById('qr-area').classList.add('show');
320             document.getElementById('verify-area').style.display = 'block';
321             document.getElementById('btn-verify').style.display = 'inline-block';
322             btn.textContent = 'Regenerate';
323
324             // Focus first digit
325             setTimeout(() => document.getElementById('s0').focus(), 200);
326         } catch (e) {
327             showAlert('setup-err', 'Network error. Please try again.');
```

```

340     navigator.clipboard.writeText(currentSecret || '');
341     const btn = document.querySelector('.copy-btn');
342     btn.textContent = 'Copied!';
343     setTimeout(() => btn.textContent = 'Copy key', 1500);
344 }
345
346 // ---- OTP inputs for setup ----
347 const setupDigits = Array.from({ length: 6 }, (_, i) => document.getElementById('s' + i));
348 const btnVerify = document.getElementById('btn-verify');
349
350 setupDigits.forEach((d, i) => {
351     d.addEventListener('input', () => {
352         const val = d.value.replace(/\D/g, '');
353         d.value = val;
354         d.classList.toggle('filled', val.length > 0);
355         if (val && i < 5) setupDigits[i + 1].focus();
356         btnVerify.disabled = setupDigits.map(x => x.value).join('').length < 6;
357     });
358     d.addEventListener('keydown', e => {
359         if (e.key === 'Backspace' && !d.value && i > 0) {
360             setupDigits[i - 1].focus();
361             setupDigits[i - 1].value = '';
362             setupDigits[i - 1].classList.remove('filled');
363             btnVerify.disabled = true;
364         }
365         if (e.key === 'Enter' && !btnVerify.disabled) verifySetup();
366     });
367     d.addEventListener('paste', e => {
368         const pasted = (e.clipboardData || window.clipboardData).getData('text').replace(/\D/g,
369             '').slice(0, 6);
370         if (pasted.length === 6) {
371             pasted.split('').forEach((ch, idx) => { setupDigits[idx].value = ch;
372                 setupDigits[idx].classList.add('filled'); });
373             setupDigits[5].focus();
374             btnVerify.disabled = false;
375             e.preventDefault();
376         }
377     });
378 });
379
380 // ---- Verify setup ----
381 async function verifySetup() {
382     clearAlerts();
383     const code = setupDigits.map(d => d.value).join('');
384     if (code.length < 6) return;
385
386     btnVerify.disabled = true;
387     btnVerify.innerHTML = '<span class="spinner"></span>Activating...';
388
389     try {
390         const res = await fetch('/totp/verify-setup', {
391             method: 'POST',
392             headers: authHeader(),
393             body: JSON.stringify({ code })
394         });
395         const data = await res.json();
396         if (!res.ok) {
397             showAlert('setup-err', data.error || 'Invalid code. Try again.');
```

398 setupDigits.forEach(d => { d.value = ''; d.classList.remove('filled'); });

399 setupDigits[0].focus();

400 btnVerify.disabled = true;

401 return;

402 }

403 showAlert('setup-ok', '2FA activated! Your account is now protected.');

404 setTimeout(() => renderStatus(true), 1200);

405 } catch (e) {

406 showAlert('setup-err', 'Network error. Please try again.');

407 } finally {

408 btnVerify.disabled = false;

409 btnVerify.textContent = 'Activate 2FA';

410 }

411 }

412 // ---- Disable TOTP ----

413 async function disableTotp() {

414 if (!confirm('Are you sure you want to disable two-factor authentication? This will make your

```
account less secure.')) return;
414
415     try {
416         const res = await fetch('/totp/disable', { method: 'DELETE', headers: authHeader() });
417         if (!res.ok) { alert('Failed to disable 2FA.');
```

return; }

```
418         renderStatus(false);
419     } catch (e) {
420         alert('Network error.');
```

421 }

```
422 }
423
424 // ---- Alerts ----
425 function showAlert(id, msg) {
426     const el = document.getElementById(id);
427     el.classList.add('show');
```

428 document.getElementById(id + '-msg').textContent = msg;

```
429 }
430 function clearAlerts() {
431     ['setup-err', 'setup-ok'].forEach(id => document.getElementById(id).classList.remove('show'));
```

432 }

```
433
434 // ---- Boot ----
435 if (!localStorage.getItem('jwt_token')) {
436     window.location.href = '/google.html';
437 } else {
438     loadStatus();
439 }
440 </script>
441 </body>
442 </html>
```

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>EMS – Vendor Hub</title>
7      <link
href="https://fonts.googleapis.com/css2?family=DM+Sans:wght@300;400;500;600;700&family=Space+Mono:wght@400;700&
isplay=swap" rel="stylesheet">
8      <style>
9          *{margin:0;padding:0;box-sizing:border-box}
10         :root{
11             --bg:#08100a;--bg2:#0c160e;--bg3:#101c12;
12             --panel:#131f15;--panel2:#182219;
13             --border:rgba(52,211,153,0.08);--border2:rgba(52,211,153,0.18);
14             --text: #0a41ce;--text2:#6b9474;--text3:#3d5e44;
15             --accent:#34d399;--accent-dim:rgba(52,211,153,0.1);
16             --amber:#fbbf24;--amber-dim:rgba(251,191,36,0.1);
17             --red:#f87171;--red-dim:rgba(248,113,113,0.1);
18             --blue:#60a5fa;--blue-dim:rgba(96,165,250,0.1);
19             --font:'DM Sans',sans-serif;--mono:'Space Mono',monospace;
20             --sidebar:245px;
21         }
22         body{font-family:var(--font);background:var(--bg);color:var(--text);font-
size:14px;overflow:hidden;height:100vh}
23         .app{display:flex;height:100vh}
24
25         /* SIDEBAR */
26         .sidebar{width:var(--sidebar);background:var(--bg2);border-right:1px solid var(--
border);display:flex;flex-direction:column;flex-shrink:0;position:relative}
27         .sidebar::before{content:'';position:absolute;left:0;top:0;bottom:0;width:2px;background:linear-
gradient(to bottom,transparent,var(--accent),transparent)}
28         .logo{padding:22px 20px;border-bottom:1px solid var(--border)}
29         .logo-row{display:flex;align-items:center;gap:10px;margin-bottom:4px}
30         .logo-icon{width:34px;height:34px;background:linear-gradient(135deg,#16a34a,var(--accent));border-
radius:10px;display:flex;align-items:center;justify-content:center;font-weight:700;font-siz...
31         .logo h2{font-size:16px;font-weight:700;color:var(--text)}
32         .logo p{font-size:10px;color:var(--text3);font-family:var(--mono)}
33         .role-badge{display:inline-flex;align-items:center;gap:6px;padding:4px 12px;background:var(--accent-
dim);border:1px solid rgba(52,211,153,0.2);border-radius:4px;font-size:10px;font-weight:7...
34         .nav{flex:1;padding:10px 0;overflow-y:auto}
35         .nav-section{padding:12px 20px 6px 6px;font-size:9px;font-weight:700;color:var(--text3);letter-
spacing:2px;text-transform:uppercase;font-family:var(--mono)}
36         .nav-item{display:flex;align-items:center;gap:10px;padding:10px 20px;margin:2px 8px;border-
radius:6px;cursor:pointer;transition:all 0.2s;color:var(--text2);font-size:13px;font-weight:500}
37         .nav-item:hover{background:var(--accent-dim);color:var(--accent)}
38         .nav-item.active{background:var(--accent-dim);color:var(--accent);border-left:2px solid var(--
accent);margin-left:6px}
39         .sidebar-footer{padding:16px 20px;border-top:1px solid var(--border)}
40         .user-card{display:flex;align-items:center;gap:10px}
41         .avatar{width:34px;height:34px;background:linear-gradient(135deg,#16a34a,var(--accent));border-
radius:6px;display:flex;align-items:center;justify-content:center;font-size:14px;font-weight:7...
42         .user-info-side .name{font-size:12px;font-weight:600;color:var(--text)}
43         .user-info-side .role-text{font-size:9px;color:var(--accent);font-family:var(--mono);text-
transform:uppercase;letter-spacing:1px}
44
45         /* MAIN */
46         .main{flex:1;display:flex;flex-direction:column;overflow:hidden}
47         .topbar{background:var(--bg2);padding:14px 24px;display:flex;justify-content:space-between;align-
items:center;border-bottom:1px solid var(--border);flex-shrink:0}
48         .page-title{font-size:16px;font-weight:700;color:var(--text);font-family:var(--mono)}
49         .topbar-right{display:flex;align-items:center;gap:12px}
50         .logout-btn{padding:7px 16px;background:transparent;border:1px solid var(--border2);border-
radius:4px;color:var(--text2);font-family:var(--mono);font-size:11px;cursor:pointer;transition:all...
51         .logout-btn:hover{background:var(--red-dim);border-color:var(--red);color:var(--red)}
52         .content{flex:1;overflow-y:auto;padding:24px}
53         .view{display:none}
54         .view.active{display:block;animation:fadeIn 0.25s ease}
55         @keyframes fadeIn{from{opacity:0;transform:translateY(6px)}to{opacity:1;transform:translateY(0)}}
56
57         /* STATS */
58         .stats-grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(150px,1fr));gap:14px;margin-

```

```

bottom:24px}
59     .stat-card{background:var(--panel);border:1px solid var(--border);border-
radius:10px;padding:18px;position:relative;transition:border-color 0.2s}
60     .stat-card:hover{border-color:var(--border2)}
61     .stat-card::before{content:'';position:absolute;top:0;left:0;right:0;height:2px;border-radius:10px
10px 0 0}
62     .stat-card.green::before{background:var(--accent)}
63     .stat-card.amber::before{background:var(--amber)}
64     .stat-card.blue::before{background:var(--blue)}
65     .stat-card.red::before{background:var(--red)}
66     .stat-label{font-size:10px;color:var(--text3);text-transform:uppercase;letter-spacing:1.2px;margin-
bottom:10px;font-weight:600;font-family:var(--mono)}
67     .stat-value{font-size:28px;font-weight:700;color:var(--text);font-family:var(--mono)}
68     .stat-sub{font-size:11px;color:var(--text3);margin-top:4px}
69
70     /* SECTION */
71     .section{background:var(--panel);border:1px solid var(--border);border-radius:10px;margin-
bottom:18px;overflow:hidden}
72     .section-head{padding:14px 20px;border-bottom:1px solid var(--border);display:flex;justify-
content:space-between;align-items:center}
73     .section-title{font-size:13px;font-weight:700;color:var(--text);font-family:var(--mono)}
74     .section-sub{font-size:11px;color:var(--text3);margin-top:2px}
75
76     /* TABLE */
77     .tablewrap{overflow-x:auto}
78     table{width:100%;border-collapse:collapse}
79     th{padding:10px 16px;text-align:left;font-size:10px;color:var(--text3);font-weight:700;letter-
spacing:1px;text-transform:uppercase;background:var(--bg3);border-bottom:1px solid var(--border...
80     td{padding:11px 16px;border-bottom:1px solid var(--border);font-size:13px;color:var(--text2)}
81     tr:last-child td{border-bottom:none}
82     tr:hover td{background:rgba(52,211,153,0.03);color:var(--text)}
83
84     /* CHIPS */
85     .chip{display:inline-flex;align-items:center;padding:3px 8px;border-radius:4px;font-size:10px;font-
weight:700;font-family:var(--mono);letter-spacing:0.5px}
86     .chip.active{background:var(--accent-dim);color:var(--accent);border:1px solid rgba(52,211,153,0.2)}
87     .chip.inactive{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
88     .chip.pending{background:var(--amber-dim);color:var(--amber)}
89     .chip.delivered{background:var(--accent-dim);color:var(--accent)}
90     .chip.processing{background:var(--blue-dim);color:var(--blue)}
91     .chip.breakfast{background:var(--amber-dim);color:var(--amber)}
92     .chip.lunch{background:var(--accent-dim);color:var(--accent)}
93     .chip.dinner{background:var(--blue-dim);color:var(--blue)}
94
95     /* BUTTONS */
96     .btn{padding:8px 16px;border-radius:6px;border:none;font-family:var(--mono);font-size:11px;font-
weight:700;cursor:pointer;transition:all 0.2s;letter-spacing:0.5px;text-transform:uppercase}
97     .btn-primary{background:var(--accent);color:#08100a}
98     .btn-primary:hover{background:#22c77a;box-shadow:0 4px 12px rgba(52,211,153,0.3)}
99     .btn-secondary{background:var(--panel2);color:var(--text2);border:1px solid var(--border)}
100    .btn-secondary:hover{border-color:var(--border2);color:var(--text)}
101    .btn-danger{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
102    .btn-danger:hover{background:var(--red);color:#fff}
103    .btn-warning{background:var(--amber-dim);color:var(--amber);border:1px solid rgba(251,191,36,0.2)}
104    .btn-warning:hover{background:var(--amber);color:#000}
105    .btn-sm{padding:5px 10px;font-size:10px}
106    .action-row{display:flex;gap:6px}
107
108    /* FORM */
109    .form-card{padding:22px}
110    .form-grid{display:grid;grid-template-columns:1fr 1fr;gap:14px}
111    .form-group{display:flex;flex-direction:column;gap:5px}
112    .form-group.full{grid-column:1/-1}
113    label{font-size:10px;font-weight:700;color:var(--text3);text-transform:uppercase;letter-
spacing:0.8px;font-family:var(--mono)}
114    input,select,textarea{background:var(--bg3);border:1px solid var(--border);border-
radius:6px;padding:9px 12px;color:var(--text);font-family:var(--font);font-size:13px;transition:border-colo...
115    input:focus,select:focus,textarea:focus{outline:none;border-color:var(--accent)}
116    input::placeholder,textarea::placeholder{color:var(--text3)}
117    select option{background:var(--bg2)}
118    .form-actions{display:flex;gap:10px;margin-top:18px}
119
120    /* ALERT */
121    .alert{position:fixed;top:20px;right:20px;padding:12px 20px;border-radius:8px;z-
index:1100;animation:slideIn 0.3s ease;font-size:12px;font-weight:700;font-family:var(--mono)}
122    .alert-success{background:var(--accent-dim);color:var(--accent);border:1px solid

```

```

rgba(52,211,153,0.3)}
123     .alert-error{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.3)}
124     @keyframes slideIn{from{transform:translateX(120%);opacity:0}to{transform:translateX(0);opacity:1}}
125     .empty-state{text-align:center;color:var(--text3);font-style:italic;padding:28px;font-family:var(--
mono);font-size:12px}
126
127     /* RESTAURANT CARDS */
128     .restaurant-grid{display:grid;grid-template-columns:repeat(auto-
fill,minmax(240px,1fr));gap:14px;padding:20px}
129     .restaurant-card{background:var(--bg3);border:1px solid var(--border);border-
radius:10px;padding:18px;transition:all 0.2s;position:relative;overflow:hidden}
130     .restaurant-
card::before{content:'';position:absolute;top:0;left:0;bottom:0;width:3px;background:var(--accent)}
131     .restaurant-card:hover{border-color:var(--border2);transform:translateX(2px)}
132     .restaurant-card.inactive::before{background:var(--red)}
133     .rest-name{font-weight:700;color:var(--text);margin-bottom:6px;font-size:14px}
134     .rest-meta{font-size:11px;color:var(--text3);font-family:var(--mono);margin-bottom:12px}
135     .rest-actions{display:flex;gap:8px}
136 </style>
137 </head>
138 <body>
139 <div class="app">
140     <!-- SIDEBAR -->
141     <div class="sidebar">
142         <div class="logo">
143             <div class="logo-row">
144                 <div class="logo-icon">V</div>
145                 <h2>Vendor Hub</h2>
146             </div>
147             <p>EMS Vendor Portal</p>
148             <div class="role-badge">// FOOD VENDOR</div>
149         </div>
150         <nav class="nav">
151             <div class="nav-section">// overview</div>
152             <div class="nav-item active" onclick="showView('dashboard',this)"><span></span> Dashboard</div>
153             <div class="nav-section">// restaurants</div>
154             <div class="nav-item" onclick="showView('my-restaurants',this)"><span></span> My
Restaurants</div>
155             <div class="nav-item" onclick="showView('add-restaurant',this)"><span></span> Add
Restaurant</div>
156             <div class="nav-section">// operations</div>
157             <div class="nav-item" onclick="showView('subscriptions',this)"><span></span>
Subscriptions</div>
158             <div class="nav-item" onclick="showView('deliveries',this)"><span></span> Deliveries</div>
159             <div class="nav-section">// account</div>
160             <div class="nav-item" onclick="showView('profile',this)"><span></span> My Profile</div>
161         </nav>
162         <div class="sidebar-footer">
163             <div class="user-card">
164                 <div class="avatar" id="avatar-initials">V</div>
165                 <div class="user-info-side">
166                     <div class="name" id="sidebar-name">Vendor</div>
167                     <div class="role-text">// vendor</div>
168                 </div>
169             </div>
170         </div>
171     </div>
172
173     <!-- MAIN -->
174     <div class="main">
175         <div class="topbar">
176             <div class="page-title" id="pageTitle">// dashboard</div>
177             <div class="topbar-right">
178                 <span style="font-size:11px;color:var(--text3);font-family:var(--mono)" id="topbar-
email"></span>
179                 <button class="logout-btn" onclick="logout()">SIGN OUT</button>
180             </div>
181         </div>
182         <div class="content">
183
184             <!-- DASHBOARD -->
185             <div id="dashboard-view" class="view active">
186                 <div class="stats-grid">
187                     <div class="stat-card green">
188                         <div class="stat-label">Restaurants</div>
189                         <div class="stat-value" id="stat-restaurants">--</div>

```

```

190         <div class="stat-sub">active listings</div>
191     </div>
192     <div class="stat-card amber">
193         <div class="stat-label">Subscriptions</div>
194         <div class="stat-value" id="stat-subs">--</div>
195         <div class="stat-sub">active customers</div>
196     </div>
197     <div class="stat-card blue">
198         <div class="stat-label">Deliveries</div>
199         <div class="stat-value" id="stat-deliveries">--</div>
200         <div class="stat-sub">total today</div>
201     </div>
202     <div class="stat-card red">
203         <div class="stat-label">Pending</div>
204         <div class="stat-value" id="stat-pending">--</div>
205         <div class="stat-sub">deliveries pending</div>
206     </div>
207 </div>
208 <div style="display:grid;grid-template-columns:1fr 1fr;gap:18px">
209     <div class="section">
210         <div class="section-head">
211             <div><div class="section-title">// my_restaurants</div></div>
212             <button class="btn btn-secondary btn-sm" onclick="showView('my-
restaurants',null)">view </button>
213         </div>
214         <div id="dash-restaurants" style="padding:4px 0"></div>
215     </div>
216     <div class="section">
217         <div class="section-head">
218             <div><div class="section-title">// recent_deliveries</div></div>
219             <button class="btn btn-secondary btn-sm"
onclick="showView('deliveries',null)">view </button>
220         </div>
221         <div class="tablewrap">
222             <table>
223                 <thead><tr><th>Meal</th><th>Slot</th><th>Status</th></tr></thead>
224                 <tbody id="dash-deliveries"><tr><td colspan="3" class="empty-
state">loading...</td></tr></tbody>
225             </table>
226         </div>
227     </div>
228 </div>
229 </div>
230
231 <!-- MY RESTAURANTS -->
232 <div id="my-restaurants-view" class="view">
233     <div class="section">
234         <div class="section-head">
235             <div><div class="section-title">// my_restaurants</div><div class="section-
sub">Manage your listings</div></div>
236             <button class="btn btn-secondary btn-sm" onclick="loadMyRestaurants()">
refresh</button>
237         </div>
238         <div class="restaurant-grid" id="restaurants-grid">
239             <div class="empty-state">loading...</div>
240         </div>
241     </div>
242 </div>
243
244 <!-- ADD RESTAURANT -->
245 <div id="add-restaurant-view" class="view">
246     <div class="section">
247         <div class="section-head"><div class="section-title">// add_restaurant</div></div>
248         <div class="form-card">
249             <div class="form-grid">
250                 <div class="form-group full">
251                     <label>Restaurant Name</label>
252                     <input id="rest-name" placeholder="My Restaurant Name">
253                 </div>
254
255                 <div class="form-group full">
256                     <label>Address</label>
257                     <input id="rest-address" placeholder="Street, area, city">
258                 </div>
259
260                 <div class="form-group full">

```

```

261             <label>Meal Slots</label>
262             <div style="display:flex;gap:14px;flex-wrap:wrap;padding:10px
12px;background:var(--bg3);border:1px solid var(--border);border-radius:6px">
263                 <label style="display:flex;align-items:center;gap:8px;color:var(--
text);font-family:var(--font);text-transform:none;letter-spacing:0;font-size:13px">
264                     <input type="checkbox" class="rest-slot-check" value="BREAKFAST"
style="width:auto"> Breakfast
265                 </label>
266                 <label style="display:flex;align-items:center;gap:8px;color:var(--
text);font-family:var(--font);text-transform:none;letter-spacing:0;font-size:13px">
267                     <input type="checkbox" class="rest-slot-check" value="LUNCH"
style="width:auto"> Lunch
268                 </label>
269                 <label style="display:flex;align-items:center;gap:8px;color:var(--
text);font-family:var(--font);text-transform:none;letter-spacing:0;font-size:13px">
270                     <input type="checkbox" class="rest-slot-check" value="DINNER"
style="width:auto"> Dinner
271                 </label>
272             </div>
273         </div>
274
275         <div class="form-group full">
276             <label>Description (optional)</label>
277             <textarea id="rest-desc" rows="2" placeholder="Brief description of your
restaurant..."></textarea>
278         </div>
279     </div>
280     <div class="form-actions">
281         <button class="btn btn-primary" onclick="addRestaurant()">CREATE
RESTAURANT</button>
282         <button class="btn btn-secondary" onclick="clearRestForm()">CLEAR</button>
283     </div>
284 </div>
285 </div>
286 </div>
287
288 <!-- SUBSCRIPTIONS -->
289 <div id="subscriptions-view" class="view">
290     <div class="section">
291         <div class="section-head">
292             <div><div class="section-title">// subscriptions</div><div class="section-
sub">Customers subscribed to your restaurants</div></div>
293             <button class="btn btn-secondary btn-sm" onclick="loadSubscriptions()">
refresh</button>
294         </div>
295         <div class="tablewrap">
296             <table>
297                 <thead><tr><th>Employee</th><th>Restaurant</th><th>Meal
Slot</th><th>From</th><th>To</th><th>Status</th></tr></thead>
298                 <tbody id="subscriptions-list"><tr><td colspan="6" class="empty-
state">loading...</td></tr></tbody>
299             </table>
300         </div>
301     </div>
302 </div>
303
304 <!-- DELIVERIES -->
305 <div id="deliveries-view" class="view">
306     <div class="section">
307         <div class="section-head">
308             <div><div class="section-title">// deliveries</div></div>
309             <button class="btn btn-secondary btn-sm" onclick="loadDeliveries()">
refresh</button>
310         </div>
311         <div class="tablewrap">
312             <table>
313                 <thead><tr><th>ID</th><th>Restaurant</th><th>Meal
Slot</th><th>Date</th><th>Status</th><th>Action</th></tr></thead>
314                 <tbody id="deliveries-list"><tr><td colspan="6" class="empty-
state">loading...</td></tr></tbody>
315             </table>
316         </div>
317     </div>
318 </div>
319
320 <!-- PROFILE -->

```



```

321         <div id="profile-view" class="view">
322             <div class="section">
323                 <div class="section-head"><div class="section-title">// my_profile</div></div>
324                 <div style="padding:24px;display:grid;grid-template-columns:1fr 1fr;gap:16px"
id="vendor-profile">
325                     <div><div style="font-size:10px;color:var(--text3);font-family:var(--mono);text-
transform:uppercase;letter-spacing:0.8px;margin-bottom:6px">NAME</div><div style="font-family...
326                     <div><div style="font-size:10px;color:var(--text3);font-family:var(--mono);text-
transform:uppercase;letter-spacing:0.8px;margin-bottom:6px">EMAIL</div><div style="font-famil...
327                     <div><div style="font-size:10px;color:var(--text3);font-family:var(--mono);text-
transform:uppercase;letter-spacing:0.8px;margin-bottom:6px">VENDOR ID</div><div style="font-f...
328                     <div><div style="font-size:10px;color:var(--text3);font-family:var(--mono);text-
transform:uppercase;letter-spacing:0.8px;margin-bottom:6px">ROLE</div><div style="font-family...
329                     </div>
330                 </div>
331             </div>
332
333         </div>
334     </div>
335 </div>
336
337 <script>
338     const API='http://localhost:8080';
339     let vendorId=null;
340
341     (function(){
342         const token=localStorage.getItem('jwt_token');
343         const authType=localStorage.getItem('auth_type');
344         if(!token&&authType!=='session'){window.location.href='/login.html';return;}
345         if(token){
346             try{
347                 const p=JSON.parse(atob(token.split('.')[1]));
348                 const roles=p.roles||p.authorities||'';
349                 if(roles.includes('ROLE_VENDOR')) { initUser(); return; }
350                 if(roles.includes('ROLE_EMPLOYEE')){window.location.href='/employee-
dashboard.html';return;}
351                 if(roles.includes('ROLE_MANAGER')){window.location.href='/manager-dashboard.html';return;}
352                 if(roles.includes('ROLE_ADMIN')){window.location.href='/dashboard.html';return;}
353             }catch(e){}
354         }
355         initUser();
356     })();
357
358     function getAuthHeader(){const t=localStorage.getItem('jwt_token');return t?{'Authorization':`Bearer
${t}`}:{};}
359     async function apiFetch(path,opts={}){
360         const res=await fetch(`${API}${path}`,{...opts,headers:{'Content-
Type':'application/json',...getAuthHeader(),...(opts.headers||{})},credentials:'include'});
361         if(res.status===401){localStorage.removeItem('jwt_token');localStorage.removeItem('auth_type');showAlert('SESSIO
N EXPIRED','error');setTimeout(()=>window.location.href='/login.html',1500);t...
362         const text=await res.text();return text?JSON.parse(text):null;
363     }
364
365     async function initUser(){
366         const token=localStorage.getItem('jwt_token');
367         let email='';
368         if(token){try{const p=JSON.parse(atob(token.split('.')[1]));email=p.sub||p.email||'';}catch(e){}}
369         document.getElementById('avatar-initials').textContent=(email||'V').charAt(0).toUpperCase();
370         document.getElementById('sidebar-name').textContent=(email.split('@')[0]||'Vendor');
371         document.getElementById('topbar-email').textContent=email;
372         document.getElementById('p-email').textContent=email;
373
374         try{
375             const me = await apiFetch('/vendors/me');
376             if(me){
377                 vendorId = me.id || me.vendorId || null;
378                 document.getElementById('p-name').textContent = me.name || '-';
379                 document.getElementById('p-id').textContent = vendorId || '-';
380             }
381         }catch(e){
382             console.error('Could not load vendor profile', e);
383         }
384
385         loadDashboard();
386     }

```

```

387
388     function showView(name,el){
389         document.querySelectorAll('.view').forEach(v=>v.classList.remove('active'));
390         document.getElementById(`${name}-view`).classList.add('active');
391         document.querySelectorAll('.nav-item').forEach(n=>n.classList.remove('active'));
392         if(el)el.classList.add('active');
393         document.getElementById('pageTitle').textContent='// '+name.replace(/-/g, '_');
394         const loaders={dashboard:loadDashboard,'my-restaurants':loadMyRestaurants,'add-
restaurant':()=>{},'subscriptions':loadSubscriptions,'deliveries':loadDeliveries,'profile':()=>{}};
395         if(loaders[name])loaders[name]();
396     }
397
398     async function loadDashboard(){
399         try{
400             const [rests,subs,deliveries]=await Promise.all([
401                 apiFetch('/restaurants/vendor').catch(()=>[]),
402                 apiFetch('/subscriptions').catch(()=>[]),
403                 apiFetch('/deliveries').catch(()=>[])
404             ]);
405
406             const restArr=rests||[];
407             const subArr=subs||[];
408             const delArr=deliveries||[];
409
410             document.getElementById('stat-
restaurants').textContent=restArr.filter(r=>r.active!==false).length;
411             document.getElementById('stat-subs').textContent=subArr.length;
412             document.getElementById('stat-deliveries').textContent=delArr.length;
413             document.getElementById('stat-
pending').textContent=delArr.filter(d=>d.status==='PENDING').length;
414
415             document.getElementById('dash-restaurants').innerHTML=restArr.length
416             ? restArr.slice(0,4).map(r=><div style="padding:12px 20px;border-bottom:1px solid var(--
border);display:flex;justify-content:space-between;align-items:center"><div><div style="font...
417                 : '<div class="empty-state">no restaurants yet</div>';
418
419             document.getElementById('dash-deliveries').innerHTML=delArr.length
420             ? delArr.slice(0,5).map(d=><tr><td>${d.restaurantName||d.restaurantId||'-'}</td><td><span
class="chip ${((d.mealSlot||'')).toLowerCase()}>${d.mealSlot||'-'}</span></td><td><span cla...
421                 : '<tr><td colspan="3" class="empty-state">no deliveries</td></tr>';
422             }catch(e){console.error(e);}
423         }
424
425
426     async function loadMyRestaurants(){
427         try{
428             const rests=await apiFetch('/restaurants/vendor');
429             const arr=rests||[];
430             document.getElementById('restaurants-grid').innerHTML=arr.length
431             ? arr.map(r=><div class="restaurant-card ${r.active===false?'inactive':''}>
432                 <div class="rest-name">${r.name||'-'}</div>
433                 <div class="rest-meta">${r.supportedMealSlots?.join(', ')}||'-'} · ID:
${r.id||r.restaurantId||'-'}</div>
434                 <div style="font-size:12px;color:var(--text2);margin-bottom:8px">${r.address||'-'}</div>
435                 <div style="font-size:11px;color:var(--text3);margin-bottom:12px">${r.description||''}</div>
436                 <div class="rest-actions">
437                     ${r.active!==false
438                     ? <button class="btn btn-warning btn-sm"
onclick="deactivateRestaurant(${r.id}|r.restaurantId)">DEACTIVATE</button>`
439                     : `<span style="font-size:10px;color:var(--red);font-family:var(--
mono)">INACTIVE</span>`}
440                 </div>
441             </div>`).join('')
442             : '<div class="empty-state">no restaurants found. create one!</div>';
443         }catch(e){
444             document.getElementById('restaurants-grid').innerHTML='<div class="empty-state">could not load
restaurants</div>';
445         }
446     }
447
448
449     async function addRestaurant(){
450         const name=document.getElementById('rest-name').value.trim();
451         const address=document.getElementById('rest-address').value.trim();
452         const description=document.getElementById('rest-desc').value.trim();
453         const supportedMealSlots=Array.from(document.querySelectorAll('.rest-slot-

```

```

check:checked')).map(el=>el.value);
454
455     if(!name || !address || supportedMealSlots.length===0){
456         showAlert('NAME, ADDRESS AND AT LEAST ONE MEAL SLOT REQUIRED','error');
457         return;
458     }
459
460     try{
461         await apiFetch('/restaurants',{
462             method:'POST',
463             body:JSON.stringify({name,address,description,supportedMealSlots})
464         });
465         showAlert('RESTAURANT CREATED!','success');
466         clearRestForm();
467         loadMyRestaurants();
468         loadDashboard();
469     }catch(e){
470         showAlert('FAILED: '+e.message,'error');
471     }
472 }
473
474
475 function clearRestForm(){
476     ['rest-name','rest-address','rest-desc'].forEach(id=>document.getElementById(id).value='');
477     document.querySelectorAll('.rest-slot-check').forEach(el=>el.checked=false);
478 }
479
480 async function loadSubscriptions(){
481     try{
482         const data=await apiFetch('/subscriptions');
483         document.getElementById('subscriptions-list').innerHTML=(data||[]).length
484             ? data.map(s=><tr><td style="font-family:var(--mono);font-size:12px">${s.employeeEmail}|${s.employeeId}|-</td><td>${s.restaurantName}|${s.restaurantId}|-</td><td><span class="chip...
485             : '<tr><td colspan="6" class="empty-state">no subscriptions found</td></tr>';
486     }catch(e){document.getElementById('subscriptions-list').innerHTML='<tr><td colspan="6"
class="empty-state">could not load subscriptions</td></tr>';}
487 }
488
489 async function loadDeliveries(){
490     try{
491         const data=await apiFetch('/deliveries');
492         document.getElementById('deliveries-list').innerHTML=(data||[]).length
493             ? data.map(d=><tr><td style="font-family:var(--mono);font-size:11px">#${d.id}|-</td><td>${d.restaurantName}|${d.restaurantId}|-</td><td><span class="chip
${(d.mealSlot||'').toL...
494             : '<tr><td colspan="6" class="empty-state">no deliveries found</td></tr>';
495     }catch(e){document.getElementById('deliveries-list').innerHTML='<tr><td colspan="6" class="empty-
state">could not load deliveries</td></tr>';}
496 }
497
498 async function markDelivered(id){
499     try{await apiFetch(`/deliveries/${id}/delivered`,{method:'PUT'});showAlert('MARKED AS
DELIVERED','success');loadDeliveries();}
500     catch(e){showAlert('FAILED','error');}
501 }
502
503 function logout(){
504     // Clear all localStorage
505     localStorage.removeItem('jwt_token');
506     localStorage.removeItem('refresh_token');
507     localStorage.removeItem('auth_type');
508     localStorage.removeItem('last_role');
509
510     // Call server logout to invalidate session
511     fetch(`${API}/session/logout`, {
512         method: 'POST',
513         credentials: 'include'
514     }).finally(() => {
515         // Force expire the session cookie on client side
516         document.cookie = 'JSESSIONID=; expires=Thu, 01 Jan 1970 00:00:00 UTC; path=/;';
517         window.location.href = '/google.html';
518     });
519 }
520
521 function showAlert(msg,type){

```

```
522     const el=document.createElement('div');
523     el.className=`alert alert-${type}`;el.textContent=msg;
524     document.body.appendChild(el);setTimeout(()=>el.remove(),3000);
525 }
526 </script>
527 </body>
528 </html>
529
```

```

1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>EMS – Vendor Login</title>
7      <link
href="https://fonts.googleapis.com/css2?family=Outfit:wght@300;400;500;600;700&family=JetBrains+Mono:wght@400;50
0&display=swap" rel="stylesheet">
8      <style>
9          *{margin:0;padding:0;box-sizing:border-box}
10         :root{
11             --bg:#070910;
12             --panel:#0e1118;
13             --panel2:#131821;
14             --border:rgba(255,255,255,0.07);
15             --border-lit:rgba(255,255,255,0.14);
16             --text:#dde4f0;--text2:#6b7591;--text3:#3d4560;
17             --red:#f87171;--red-dim:rgba(248,113,113,0.12);
18             --blue:#4f8cff;--blue-dim:rgba(79,140,255,0.12);
19             --green:#34d399;--green-dim:rgba(52,211,153,0.12);
20             --amber:#f9c74f;--amber-dim:rgba(249,199,79,0.12);
21             --font:'Outfit',sans-serif;--mono:'JetBrains Mono',monospace;
22         }
23         body{font-family:var(--font);background:var(--bg);color:var(--text);min-
height:100vh;display:flex;overflow:auto;}
24
25         .bg-layer{position:fixed;inset:0;z-index:0;
26             background:
27                 radial-gradient(ellipse 60% 50% at 20% 50%, rgba(248,113,113,0.06) 0%,transparent 70%),
28                 radial-gradient(ellipse 40% 60% at 80% 20%, rgba(249,199,79,0.05) 0%,transparent 60%),
29                 radial-gradient(ellipse 50% 40% at 70% 80%, rgba(248,113,113,0.04) 0%,transparent 60%);
30         }
31         .grid-overlay{position:fixed;inset:0;z-index:0;background-image:linear-
gradient(rgba(255,255,255,0.02) 1px,transparent 1px),linear-gradient(90deg,rgba(255,255,255,0.02)
1px,transparent 1px)...
32
33         /* LEFT */
34         .left-panel{flex:1;display:flex;flex-direction:column;justify-
content:center;padding:60px;position:relative;z-index:1;animation:fadeLeft 0.8s ease both;}
35         @keyframes fadeLeft{from{opacity:0;transform:translateX(-
24px)}to{opacity:1;transform:translateX(0)}}
36         .brand-logo{display:flex;align-items:center;gap:12px;margin-bottom:32px}
37         .brand-icon{width:44px;height:44px;background:linear-gradient(135deg,#f87171,#fb923c);border-
radius:12px;display:flex;align-items:center;justify-content:center;font-weight:700;font-size:18p...
38         .brand-name{font-size:22px;font-weight:700;color:var(--text);letter-spacing:-0.3px}
39         .brand-tagline{font-size:13px;color:var(--text2)}
40         .hero-text h1{font-size:clamp(36px,4vw,50px);font-weight:700;line-height:1.1;letter-spacing:-
1px;color:var(--text);margin-bottom:16px;}
41         .hl-red{color:var(--red)}
42         .hero-text p{font-size:15px;color:var(--text2);line-height:1.6;max-width:360px}
43
44         .info-cards{display:flex;flex-direction:column;gap:12px;margin-top:40px;max-width:360px;}
45         .info-card{background:var(--panel);border:1px solid var(--border);border-radius:10px;padding:14px
18px;display:flex;align-items:flex-start;gap:12px;}
46         .info-card-icon{font-size:18px;flex-shrink:0;margin-top:1px}
47         .info-card-text strong{font-size:13px;color:var(--text);display:block;margin-bottom:2px}
48         .info-card-text span{font-size:12px;color:var(--text2)}
49
50         .back-link{display:inline-flex;align-items:center;gap:6px;font-size:12px;color:var(--text3);text-
decoration:none;margin-top:36px;transition:color 0.2s;}
51         .back-link:hover{color:var(--text2)}
52
53         /* RIGHT */
54         .right-panel{width:460px;flex-shrink:0;display:flex;align-items:flex-start;overflow-y:auto;justify-
content:center;padding:32px 40px;position:relative;z-index:1;border-left:1px solid var(--b...
55         @keyframes
fadeRight{from{opacity:0;transform:translateX(24px)}to{opacity:1;transform:translateX(0)}}
56         .form-box{width:100%}
57
58         .vendor-badge{display:inline-flex;align-items:center;gap:8px;background:var(--red-dim);border:1px
solid rgba(248,113,113,0.25);border-radius:20px;padding:6px 14px;font-size:12px;font-weight...

```

```

59     .vendor-badge .dot{width:7px;height:7px;border-radius:50%;background:var(--red);animation:pulse 1.4s
infinite;}}
60     @keyframes pulse{0%,100%{opacity:1}50%{opacity:0.4}}
61
62     .form-title{font-size:22px;font-weight:700;color:var(--text);margin-bottom:6px;}
63     .form-subtitle{font-size:13px;color:var(--text2);margin-bottom:28px;}
64
65     /* AUTH TABS */
66     .auth-tabs{display:flex;background:var(--bg);border-radius:10px;padding:4px;gap:4px;margin-
bottom:24px;border:1px solid var(--border)}
67     .auth-tab{flex:1;padding:9px;border:none;background:transparent;color:var(--text2);font-family:var(-
font);font-weight:600;font-size:13px;cursor:pointer;border-radius:7px;transition:all 0.2s;}
68     .auth-tab.active{background:var(--panel2);color:var(--text);box-shadow:0 1px 4px rgba(0,0,0,0.3)}
69
70     /* FORM ELEMENTS */
71     .form-group{margin-bottom:16px}
72     .form-label{display:block;font-size:10px;font-weight:600;color:var(--text2);text-
transform:uppercase;letter-spacing:0.8px;font-family:var(--mono);margin-bottom:7px}
73     .form-input{width:100%;padding:11px 14px;background:var(--bg);border:1px solid var(--border);border-
radius:9px;color:var(--text);font-family:var(--font);font-size:14px;transition:all 0.2s;}
74     .form-input:focus{outline:none;border-color:var(--red);box-shadow:0 0 3px rgba(248,113,113,0.08)}
75     .form-input::placeholder{color:var(--text3)}
76
77     /* BUTTONS */
78     .btn-login{width:100%;padding:12px;border:none;border-radius:9px;font-family:var(--font);font-
size:14px;font-weight:600;cursor:pointer;transition:all 0.2s;margin-top:4px;background:var(--re...
79     .btn-login:hover{background:#f98a8a;box-shadow:0 6px 20px
rgba(248,113,113,0.35);transform:translateY(-1px)}
80     .btn-login.active{transform:translateY(0)}
81     .btn-login.loading{opacity:0.7;pointer-events:none}
82
83     /* STATUS */
84     .status-box{border-radius:9px;padding:10px 14px;margin-top:14px;font-size:13px;font-
weight:500;display:none;animation:fadeIn 0.3s ease;}
85     @keyframes fadeIn{from{opacity:0;transform:translateY(4px)}to{opacity:1;transform:translateY(0)}}
86     .status-box.error{background:var(--red-dim);color:var(--red);border:1px solid rgba(248,113,113,0.2)}
87     .status-box.success{background:var(--green-dim);color:var(--green);border:1px solid
rgba(52,211,153,0.2)}
88     .status-box.loading{background:var(--blue-dim);color:var(--blue);border:1px solid
rgba(79,140,255,0.2)}
89
90     .notice{background:rgba(249,199,79,0.06);border:1px solid rgba(249,199,79,0.15);border-
radius:9px;padding:12px 14px;margin-top:20px;font-size:12px;color:var(--text2);line-height:1.5;}
91     .notice strong{color:var(--amber)}
92
93     /* FOOTER */
94     .form-footer{margin-top:20px;padding-top:18px;border-top:1px solid var(--border)}
95     .tz-row{display:flex;align-items:center;gap:8px;margin-bottom:8px}
96     .tz-label{font-size:10px;color:var(--text3);font-family:var(--mono);flex-shrink:0}
97     .tz-select{flex:1;padding:6px 8px;background:var(--bg);border:1px solid var(--border);border-
radius:7px;color:var(--text2);font-family:var(--mono);font-size:11px;cursor:pointer;}
98     .tz-select:focus{outline:none;border-color:var(--border-lit)}
99     .tz-time{font-size:10px;color:var(--red);font-family:var(--mono)}
100
101     @media(max-width:900px){
102         .left-panel{display:none}
103         .right-panel{width:100%;border-left:none;background:var(--bg)}
104     }
105 </style>
106 </head>
107 <body>
108 <div class="bg-layer"></div>
109 <div class="grid-overly"></div>
110
111 <!-- LEFT -->
112 <div class="left-panel">
113     <div class="brand-logo">
114         <div class="brand-icon">V</div>
115         <div>
116             <div class="brand-name">Vendor Hub</div>
117             <div class="brand-tagline">Employee Management System</div>
118         </div>
119     </div>
120     <div class="hero-text">
121         <h1>Manage your<br><span class="hl-red">vendor account</span><br>securely.</h1>
122         <p>Access your deliveries, restaurants, and subscriptions. Your account is created and managed by

```

```

the system admin.</p>
123     </div>
124     <div class="info-cards">
125         <div class="info-card">
126             <span class="info-card-icon"></span>
127             <div class="info-card-text">
128                 <strong>Delivery Tracking</strong>
129                 <span>Monitor your active and scheduled meal deliveries in real-time.</span>
130             </div>
131         </div>
132         <div class="info-card">
133             <span class="info-card-icon"></span>
134             <div class="info-card-text">
135                 <strong>Restaurant Management</strong>
136                 <span>View and update restaurant details and meal offerings.</span>
137             </div>
138         </div>
139         <div class="info-card">
140             <span class="info-card-icon"></span>
141             <div class="info-card-text">
142                 <strong>Account by Admin Only</strong>
143                 <span>Vendor accounts are created by the system admin. Contact your admin if you don't have
credentials.</span>
144             </div>
145         </div>
146     </div>
147     <a class="back-link" href="/google.html"> Back to main login</a>
148 </div>
149
150 <!-- RIGHT FORM -->
151 <div class="right-panel">
152     <div class="form-box">
153
154         <div class="vendor-badge"><span class="dot"></span>Vendor Portal</div>
155         <div class="form-title">Vendor Sign In</div>
156         <div class="form-subtitle">Enter the credentials provided by your system admin.</div>
157
158         <!-- AUTH TABS -->
159         <div class="auth-tabs">
160             <button class="auth-tab active" id="tab-jwt" onclick="switchTab('jwt')">JWT Login</button>
161             <button class="auth-tab" id="tab-session" onclick="switchTab('session')">Session Login</button>
162         </div>
163
164         <!-- JWT FORM -->
165         <div id="jwt-form">
166             <div class="form-group">
167                 <label class="form-label">Email / Username</label>
168                 <input class="form-input" id="jwt-username" type="text" placeholder="vendor@company.com"
autocomplete="off">
169             </div>
170             <div class="form-group">
171                 <label class="form-label">Password</label>
172                 <input class="form-input" id="jwt-password" type="password" placeholder="....."
autocomplete="current-password">
173             </div>
174             <button class="btn-login" id="jwt-btn" onclick="jwtLogin()">Sign In as Vendor</button>
175         </div>
176
177         <!-- SESSION FORM -->
178         <div id="session-form" style="display:none">
179             <div class="form-group">
180                 <label class="form-label">Email / Username</label>
181                 <input class="form-input" id="session-username" type="text"
placeholder="vendor@company.com">
182             </div>
183             <div class="form-group">
184                 <label class="form-label">Password</label>
185                 <input class="form-input" id="session-password" type="password" placeholder=".....">
186             </div>
187             <button class="btn-login" id="session-btn" onclick="sessionLogin()">Sign In with
Session</button>
188         </div>
189
190     <div class="status-box" id="status-box"></div>
191     <!-- TOTP MODAL -->
192     <div id="totp-overlay" style="display:none;position:fixed;inset:0;z-index:999;

```

```

193     background:rgba(7,9,16,0.88);backdrop-filter:blur(8px);
194     align-items:center;justify-content:center;">
195         <div style="background:var(--panel);border:1px solid var(--border-lit);
196             border-radius:16px;padding:36px;width:340px;text-align:center;
197             animation:fadeIn 0.25s ease;">
198             <div style="font-size:28px;margin-bottom:8px;"></div>
199             <h3 style="font-size:17px;font-weight:700;color:var(--text);margin-bottom:6px;">
200                 Two-Factor Verification
201             </h3>
202             <p style="font-size:13px;color:var(--text2);margin-bottom:24px;">
203                 Enter the 6-digit code from your authenticator app.
204             </p>
205             <input id="totp-code" class="form-input" type="text"
206                 inputmode="numeric" maxlength="6" placeholder="000000"
207                 style="text-align:center;font-size:22px;font-family:var(--mono);
208                 letter-spacing:8px;padding:14px;"
209                 oninput="this.value=this.value.replace(/\D/g, '')">
210             <div id="totp-error" style="display:none;margin-top:10px;
211                 font-size:12px;color:var(--red);font-family:var(--mono)"></div>
212             <button id="totp-verify-btn" class="btn-login"
213                 onclick="submitTotp()"
214                 style="margin-top:16px;">
215                 Verify
216             </button>
217             <button onclick="cancelTotp()"
218                 style="margin-top:10px;width:100%;padding:10px;border:none;
219                 background:transparent;color:var(--text2);
220                 font-family:var(--font);font-size:13px;cursor:pointer;">
221                 Back to login
222             </button>
223         </div>
224     </div>
225     <div class="notice">
226         <strong>Don't have an account?</strong> Vendor accounts are created exclusively by the system
227         admin. Please contact your administrator to set up access.
228     </div>
229     <!-- FOOTER -->
230     <div class="form-footer">
231         <div class="tz-row">
232             <span class="tz-label">TZ</span>
233             <select class="tz-select" id="tz-select" onchange="updateTz()">
234                 <option value="Asia/Kolkata">India – IST UTC+5:30</option>
235                 <option value="UTC">UTC</option>
236                 <option value="America/New_York">New York – EST UTC-5</option>
237                 <option value="America/Los_Angeles">Los Angeles – PST UTC-8</option>
238                 <option value="Europe/London">London – GMT UTC+0</option>
239                 <option value="Europe/Paris">Paris – CET UTC+1</option>
240                 <option value="Asia/Tokyo">Tokyo – JST UTC+9</option>
241                 <option value="Asia/Dubai">Dubai – GST UTC+4</option>
242                 <option value="Australia/Sydney">Sydney – AEDT UTC+11</option>
243                 <option value="Asia/Singapore">Singapore – SGT UTC+8</option>
244             </select>
245             <span class="tz-time" id="tz-time">--:--:--</span>
246         </div>
247     </div>
248
249 </div>
250 </div>
251
252 <script>
253     const API = 'http://localhost:8080';
254
255     const DASHBOARDS = {
256         FOOD_VENDOR: '/vendor-dashboard.html',
257         TECH_VENDOR: '/tech-vendor-dashboard.html'
258     };
259
260     // ALREADY LOGGED IN?
261     (function(){
262         const token = localStorage.getItem('jwt_token');
263         if(!token) return;
264         try{
265             const role = getRoleFromJWT(token);
266             if(DASHBOARDS[role]) window.location.href = DASHBOARDS[role];
267         }catch(e){}

```



```

268     });
269
270     // HELPERS
271     function getRoleFromJWT(token){
272         const payload = JSON.parse(atob(token.split('.')[1]));
273         const raw = payload.role || payload.roles || payload.authorities || payload.scope || '';
274         const role = Array.isArray(raw) ? raw[0] : raw;
275         return String(role).replace(/^ROLE_/, '').toUpperCase().trim();
276     }
277
278     function showStatus(type, msg){
279         const box = document.getElementById('status-box');
280         box.className = `status-box ${type}`;
281         box.textContent = msg;
282         box.style.display = 'block';
283     }
284     function hideStatus(){
285         document.getElementById('status-box').style.display = 'none';
286     }
287
288     function switchTab(type){
289         document.getElementById('jwt-form').style.display = type === 'jwt' ? 'block' : 'none';
290         document.getElementById('session-form').style.display = type === 'session' ? 'block' : 'none';
291         document.getElementById('tab-jwt').classList.toggle('active', type === 'jwt');
292         document.getElementById('tab-session').classList.toggle('active', type === 'session');
293         hideStatus();
294     }
295
296     function redirectByRole(role){
297         const dest = DASHBOARDS[role];
298         if(!dest){
299             showStatus('error', 'This portal is for vendors only. Please use the main login page.');
```

```

343     }
344
345     localStorage.setItem('jwt_token', token);
346     localStorage.setItem('refresh_token', data.refreshToken || '');
347     localStorage.setItem('auth_type', 'jwt');
348     localStorage.setItem('last_role', role);
349     localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
350
351     btn.textContent = 'Signed In ';
352     redirectByRole(role);
353
354     }else{
355         const err = await res.text();
356         showStatus('error', 'Login failed: ' + (err || 'Invalid credentials'));
357         btn.classList.remove('loading'); btn.textContent = 'Sign In as Vendor';
358     }
359 }catch(e){
360     showStatus('error', 'Connection error: ' + e.message);
361     btn.classList.remove('loading'); btn.textContent = 'Sign In as Vendor';
362 }
363 }
364
365 // SESSION LOGIN
366 async function sessionLogin(){
367     const username = document.getElementById('session-username').value.trim();
368     const password = document.getElementById('session-password').value;
369     if(!username || !password){ showStatus('error', 'Please enter your email and password'); return; }
370
371     const btn = document.getElementById('session-btn');
372     btn.classList.add('loading'); btn.textContent = 'Signing in...';
373     showStatus('loading', 'Authenticating...');
374
375     try{
376         const res = await fetch(`${API}/session/login`, {
377             method: 'POST',
378             headers: {'Content-Type': 'application/json'},
379             body: JSON.stringify({username, password}),
380             credentials: 'include'
381         });
382
383         if(res.ok){
384             // Session login – fetch /employee/me or /vendor/me to get role
385             try{
386                 const meRes = await fetch(`${API}/vendor/me`, {credentials: 'include'});
387                 if(meRes.ok){
388                     const me = await meRes.json();
389                     const role = String(me.role || '').replace(/^ROLE_/, '').toUpperCase();
390                     localStorage.setItem('auth_type', 'session');
391                     localStorage.setItem('last_role', role);
392                     localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
393                     btn.textContent = 'Signed In ';
394                     redirectByRole(role);
395                     return;
396                 }
397             }catch(e){}
398
399             // Fallback if /vendor/me not available
400             showStatus('error', 'Could not determine vendor type. Please use JWT login.');
```

```

419 function refreshTzClock(){
420     const tz = document.getElementById('tz-select').value;
421     document.getElementById('tz-time').textContent =
422         new Date().toLocaleTimeString('en-GB', {timeZone: tz, hour12: false});
423 }
424
425 document.addEventListener('keydown', e => {
426     if(e.key !== 'Enter') return;
427     if(document.getElementById('jwt-form').style.display !== 'none') jwtLogin();
428     else sessionLogin();
429 });
430
431 window.onload = () => {
432     const savedTz = localStorage.getItem('user_timezone') || 'Asia/Kolkata';
433     document.getElementById('tz-select').value = savedTz;
434     refreshTzClock();
435     setInterval(refreshTzClock, 1000);
436 };
437 // TOTP
438 let _preAuthToken = null;
439
440 function openTotpModal() {
441     const overlay = document.getElementById('totp-overlay');
442     overlay.style.display = 'flex';
443     document.getElementById('totp-code').value = '';
444     document.getElementById('totp-error').style.display = 'none';
445     setTimeout(() => document.getElementById('totp-code').focus(), 100);
446 }
447
448 function cancelTotp() {
449     _preAuthToken = null;
450     document.getElementById('totp-overlay').style.display = 'none';
451 }
452
453 async function submitTotp() {
454     const code = document.getElementById('totp-code').value.trim();
455     const errEl = document.getElementById('totp-error');
456     errEl.style.display = 'none';
457
458     if(code.length !== 6) {
459         errEl.textContent = 'Please enter a 6-digit code.';
460         errEl.style.display = 'block';
461         return;
462     }
463
464     const btn = document.getElementById('totp-verify-btn');
465     btn.classList.add('loading'); btn.textContent = 'Verifying...';
466
467     try {
468         const res = await fetch(`${API}/Authenticate/totp`, {
469             method: 'POST',
470             headers: {'Content-Type': 'application/json'},
471             body: JSON.stringify({ preAuthToken: _preAuthToken, code })
472         });
473
474         if(res.ok) {
475             const data = await res.json();
476             const token = data.accessToken || data.token;
477             const role = getRoleFromJWT(token);
478
479             _preAuthToken = null;
480             document.getElementById('totp-overlay').style.display = 'none';
481
482             localStorage.setItem('jwt_token', token);
483             localStorage.setItem('refresh_token', data.refreshToken || '');
484             localStorage.setItem('auth_type', 'jwt');
485             localStorage.setItem('last_role', role);
486             localStorage.setItem('user_timezone', document.getElementById('tz-select').value);
487
488             showStatus('success', 'Verified! Redirecting...');
489             setTimeout(() => redirectByRole(role), 600);
490         } else {
491             const err = await res.json().catch(() => ({}));
492             errEl.textContent = err.error || 'Invalid code. Try again.';
493             errEl.style.display = 'block';
494             btn.classList.remove('loading'); btn.textContent = 'Verify';

```

```
495         }
496     } catch(e) {
497         errEl.textContent = 'Connection error. Try again.';
498         errEl.style.display = 'block';
499         btn.classList.remove('loading'); btn.textContent = 'Verify';
500     }
501 }
502
503 // Allow Enter key inside TOTP input
504 document.addEventListener('keydown', e => {
505     if(e.key === 'Enter' &&
506         document.getElementById('totp-overlay').style.display === 'flex') {
507         submitTotp();
508     }
509 });
510 </script>
511 </body>
512 </html>
```

170 Current:

src\test\java\com\example\EmployeeManagementSystem\EmployeeManagementSystemApplication

```
1 package com.example.EmployeeManagementSystem;
2
3 import org.junit.jupiter.api.Test;
4 import org.springframework.boot.test.context.SpringBootTest;
5
6 @SpringBootTest
7 class EmployeeManagementSystemApplicationTests {
8
9     @Test
10     void contextLoads() {
11     }
12
13 }
14
```

```

1  package com.example.EmployeeManagementSystem.service;
2
3
4  import com.example.EmployeeManagementSystem.Entity.Employee;
5  import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
6  import com.example.EmployeeManagementSystem.Entity.LeaveType;
7  import com.example.EmployeeManagementSystem.Enum.Gender;
8  import com.example.EmployeeManagementSystem.Enum.Status;
9  import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
10 import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
11 import com.example.EmployeeManagementSystem.Repository.LeaveTypeRepository;
12 import com.example.EmployeeManagementSystem.Service.LeaveAccrualService;
13 import org.junit.jupiter.api.BeforeEach;
14 import org.junit.jupiter.api.Test;
15 import org.mockito.*;
16
17 import java.math.BigDecimal;
18 import java.time.LocalDate;
19 import java.util.List;
20 import java.util.Optional;
21
22 import static org.assertj.core.api.Assertions.assertThat;
23 import static org.mockito.ArgumentMatchers.any;
24 import static org.mockito.Mockito.*;
25
26 class LeaveAccrualServiceTest {
27
28     @Mock
29     EmployeeRepo employeeRepo;
30     @Mock
31     LeaveTypeRepository leaveTypeRepo;
32     @Mock
33     LeaveEntitlementRepository entitlementRepo;
34
35     @InjectMocks
36     LeaveAccrualService service;
37
38     @BeforeEach void setup() { MockitoAnnotations.openMocks(this); }
39
40     @Test
41     void maleEmployee_doesNotAccrueMaternity() {
42         Employee emp = employee(Gender.M);
43         LeaveType maternity = leaveType("MATERNITY", true, Gender.F);
44
45         when(employeeRepo.findByStatus(Status.ACTIVE)).thenReturn(List.of(emp));
46         when(leaveTypeRepo.findAll()).thenReturn(List.of(maternity));
47
48         service.runMonthlyAccrual(LocalDate.of(2025, 6, 1));
49
50         verify(entitlementRepo, never()).save(any());
51     }
52
53     @Test
54     void femaleEmployee_accruesMaternity() {
55         Employee emp = employee(Gender.F);
56         LeaveType maternity = leaveType("MATERNITY", true, Gender.F);
57
58         when(employeeRepo.findByStatus(Status.ACTIVE)).thenReturn(List.of(emp));
59         when(leaveTypeRepo.findAll()).thenReturn(List.of(maternity));
60         when(entitlementRepo.findByEmployeeIdAndLeaveTypeIdAndYear(any(), any(), any()))
61             .thenReturn(Optional.empty());
62
63         // Simulate new entitlement creation
64         when(entitlementRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));
65
66         service.runMonthlyAccrual(LocalDate.of(2025, 6, 1));
67
68         ArgumentCaptor<LeaveEntitlement> captor = ArgumentCaptor.forClass(LeaveEntitlement.class);
69         verify(entitlementRepo, atLeastOnce()).save(captor.capture());
70
71         LeaveEntitlement saved = captor.getAllValues().stream()

```

```

72         .filter(e -> e.getAccruedThisYear().compareTo(BigDecimal.ONE) == 0)
73         .findFirst().orElseThrow();
74
75         assertThat(saved.getAccruedThisYear()).isEqualToComparingTo("1");
76     }
77
78     @Test
79     void employeeJoiningAfterAccrualDate_isSkipped() {
80         Employee emp = employee(Gender.M);
81         emp.setJoined_at(LocalDate.of(2025, 6, 15)); // joins mid-month
82         LeaveType casual = leaveType("CASUAL", false, null);
83
84         when(employeeRepo.findByStatus(Status.ACTIVE)).thenReturn(List.of(emp));
85         when(leaveTypeRepo.findAll()).thenReturn(List.of(casual));
86
87         service.runMonthlyAccrual(LocalDate.of(2025, 6, 1));
88
89         verify(entitlementRepo, never()).save(any());
90     }
91
92     // --- helpers ---
93     private Employee employee(Gender gender) {
94         Employee e = new Employee();
95         e.setEmployeeId(1L);
96         e.setGender(gender);
97         e.setStatus(Status.ACTIVE);
98         e.setJoined_at(LocalDate.of(2023, 1, 1));
99         return e;
100     }
101
102     private LeaveType leaveType(String name, boolean genderRestricted, Gender restriction) {
103         LeaveType lt = new LeaveType();
104         lt.setId(1);
105         lt.setName(name);
106         lt.setGenderRestricted(genderRestricted);
107         lt.setGenderRestriction(restriction);
108         lt.setCarriesForward(!name.equals("SICK"));
109         lt.setMonthlyReset(name.equals("SICK"));
110         return lt;
111     }
112 }

```

```

1  package com.example.EmployeeManagementSystem.service;
2
3  import com.example.EmployeeManagementSystem.DTO.LeaveRequestDTO;
4  import com.example.EmployeeManagementSystem.Entity.Employee;
5  import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
6  import com.example.EmployeeManagementSystem.Entity.LeaveRequest;
7  import com.example.EmployeeManagementSystem.Enum.LeaveStatus;
8  import com.example.EmployeeManagementSystem.Enum.Role;
9  import com.example.EmployeeManagementSystem.Enum.Status;
10 import com.example.EmployeeManagementSystem.Enum.LeaveType;
11 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
12 import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
13 import com.example.EmployeeManagementSystem.Repository.LeaveRequestRepo;
14 import com.example.EmployeeManagementSystem.Repository.LeaveTypeRepository;
15 import com.example.EmployeeManagementSystem.Service.LeaveRequestService;
16 import com.example.EmployeeManagementSystem.Service.NotificationService;
17 import org.junit.jupiter.api.AfterEach;
18 import org.junit.jupiter.api.BeforeEach;
19 import org.junit.jupiter.api.Test;
20 import org.mockito.MockedStatic;
21 import org.mockito.InjectMocks;
22 import org.mockito.Mock;
23 import org.mockito.MockitoAnnotations;
24 import org.springframework.http.HttpStatus;
25 import org.springframework.http.ResponseEntity;
26 import org.springframework.security.core.Authentication;
27
28 import com.example.EmployeeManagementSystem.Util.AuthUtil;
29
30 import java.math.BigDecimal;
31 import java.time.LocalDate;
32 import java.util.List;
33 import java.util.Map;
34 import java.util.Optional;
35
36 import static org.assertj.core.api.Assertions.assertThat;
37 import static org.assertj.core.api.Assertions.assertThatThrownBy;
38 import static org.mockito.ArgumentMatchers.any;
39 import static org.mockito.Mockito.doNothing;
40 import static org.mockito.Mockito.mock;
41 import static org.mockito.Mockito.when;
42
43 class LeaveRequestServiceTest {
44
45     @Mock
46     LeaveRequestRepo leaveRequestRepo;
47     @Mock
48     EmployeeRepo employeeRepo;
49     @Mock
50     LeaveTypeRepository leaveTypeRepository;
51     @Mock
52     LeaveEntitlementRepository leaveEntitlementRepository;
53     @Mock
54     NotificationService notificationService;
55
56     @InjectMocks
57     LeaveRequestService service;
58
59     private MockedStatic<AuthUtil> authUtilMock;
60
61     @BeforeEach
62     void setup() {
63         MockitoAnnotations.openMocks(this);
64         authUtilMock = org.mockito.Mockito.mockStatic(AuthUtil.class);
65     }
66
67     @AfterEach
68     void tearDown() {
69         authUtilMock.close();
70     }
71

```



```

72     @Test
73     void approveLeave_updatesUsedEntitlementDays() {
74         Employee employee = new Employee();
75         employee.setEmployeeId(1L);
76
77         LeaveRequest request = new LeaveRequest();
78         request.setId(10L);
79         request.setEmployee(employee);
80         request.setLeaveType(LeaveType.CASUAL);
81         request.setStartDate(LocalDate.of(2026, 5, 4));
82         request.setEndDate(LocalDate.of(2026, 5, 6));
83         request.setStatus(LeaveStatus.PENDING);
84
85         com.example.EmployeeManagementSystem.Entity.LeaveType casual =
86             new com.example.EmployeeManagementSystem.Entity.LeaveType();
87         casual.setId(2);
88         casual.setName("CASUAL");
89         casual.setCarriesForward(true);
90
91         LeaveEntitlement entitlement = new LeaveEntitlement();
92         entitlement.setEmployee(employee);
93         entitlement.setLeaveType(casual);
94         entitlement.setYear(2026);
95         entitlement.setOpeningBalance(BigDecimal.ZERO);
96         entitlement.setAccruedThisYear(BigDecimal.TEN);
97         entitlement.setUsedThisYear(BigDecimal.ONE);
98         entitlement.setClosingBalance(BigDecimal.ZERO);
99
100        when(leaveRequestRepo.findById(10L)).thenReturn(Optional.of(request));
101        when(leaveRequestRepo.save(any())).thenReturn(inv -> inv.getArgument(0));
102        when(leaveTypeRepository.findByName("CASUAL")).thenReturn(Optional.of(casual));
103        when(leaveEntitlementRepository.findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(1L, 2, 2026))
104            .thenReturn(Optional.of(entitlement));
105        when(leaveEntitlementRepository.save(any())).thenReturn(inv -> inv.getArgument(0));
106        doNothing().when(notificationService).notify(any(), any(), any());
107
108        service.approveLeave(10L);
109
110        assertThat(entitlement.getUsedThisYear()).isEqualToByComparingTo("4");
111    }
112
113     @Test
114     void createRequest_allowsCasualLeaveEqualToAvailableBalance() {
115         Authentication authentication = mock(Authentication.class);
116         authUtilMock.when(() -> AuthUtil.extractEmail(authentication)).thenReturn("emp@test.com");
117
118         Employee employee = new Employee();
119         employee.setEmployeeId(1L);
120         employee.setEmail("emp@test.com");
121         employee.setStatus(Status.ACTIVE);
122         employee.setRole(Role.EMPLOYEE);
123         employee.setTimezone("UTC");
124         employee.setName("Emp");
125
126         LeaveRequestDTO dto = new LeaveRequestDTO();
127         dto.setLeaveType(LeaveType.CASUAL);
128         dto.setStartDate(LocalDate.now().plusDays(1));
129         dto.setEndDate(LocalDate.now().plusDays(2));
130         dto.setReason("Personal work");
131
132         com.example.EmployeeManagementSystem.Entity.LeaveType casual =
133             new com.example.EmployeeManagementSystem.Entity.LeaveType();
134         casual.setId(2);
135         casual.setName("CASUAL");
136         casual.setCarriesForward(true);
137
138         LeaveEntitlement entitlement = new LeaveEntitlement();
139         entitlement.setEmployee(employee);
140         entitlement.setLeaveType(casual);
141         entitlement.setYear(dto.getStartDate().getYear());
142         entitlement.setOpeningBalance(BigDecimal.ONE);
143         entitlement.setAccruedThisYear(BigDecimal.valueOf(3));
144         entitlement.setUsedThisYear(BigDecimal.valueOf(2));
145
146         when(employeeRepo.findByEmail("emp@test.com")).thenReturn(Optional.of(employee));
147         when(leaveTypeRepository.findByName("CASUAL")).thenReturn(Optional.of(casual));

```

```

148         when(leaveEntitlementRepository.findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
149             1L, 2, dto.getStartDate().getYear()).thenReturn(Optional.of(entitlement)));
150         when(leaveRequestRepo.checkDuplicate(1L, dto.getStartDate(), dto.getEndDate(),
LeaveStatus.PENDING.name()))
151             .thenReturn(0L);
152         when(leaveRequestRepo.countOverlappingLeave(1L, dto.getStartDate(), dto.getEndDate()))
153             .thenReturn(0L);
154         when(leaveRequestRepo.save(any())).thenReturn(inv -> inv.getArgument(0));
155         when(employeeRepo.findByRole(Role.ADMIN)).thenReturn(List.of());
156
157         ResponseEntity<?> response = service.createRequest(authentication, dto);
158
159         assertThat(response.getStatusCode()).isEqualTo(HttpStatus.CREATED);
160     }
161
162     @Test
163     void createRequest_rejectsCasualLeaveAboveAvailableBalance() {
164         Authentication authentication = mock(Authentication.class);
165         authUtilMock.when(() -> AuthUtil.extractEmail(authentication)).thenReturn("emp@test.com");
166
167         Employee employee = new Employee();
168         employee.setEmployeeId(1L);
169         employee.setEmail("emp@test.com");
170         employee.setStatus(Status.ACTIVE);
171         employee.setRole(Role.EMPLOYEE);
172         employee.setTimezone("UTC");
173
174         LeaveRequestDTO dto = new LeaveRequestDTO();
175         dto.setLeaveType(LeaveType.CASUAL);
176         dto.setStartDate(LocalDate.now().plusDays(1));
177         dto.setEndDate(LocalDate.now().plusDays(3));
178
179         com.example.EmployeeManagementSystem.Entity.LeaveType casual =
180             new com.example.EmployeeManagementSystem.Entity.LeaveType();
181         casual.setId(2);
182         casual.setName("CASUAL");
183         casual.setCarriesForward(true);
184
185         LeaveEntitlement entitlement = new LeaveEntitlement();
186         entitlement.setEmployee(employee);
187         entitlement.setLeaveType(casual);
188         entitlement.setYear(dto.getStartDate().getYear());
189         entitlement.setOpeningBalance(BigDecimal.ONE);
190         entitlement.setAccruedThisYear(BigDecimal.ONE);
191         entitlement.setUsedThisYear(BigDecimal.ONE);
192
193         when(employeeRepo.findByEmail("emp@test.com")).thenReturn(Optional.of(employee));
194         when(leaveTypeRepository.findByName("CASUAL")).thenReturn(Optional.of(casual));
195         when(leaveEntitlementRepository.findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(
196             1L, 2, dto.getStartDate().getYear()).thenReturn(Optional.of(entitlement)));
197
198         ResponseEntity<?> response = service.createRequest(authentication, dto);
199
200         assertThat(response.getStatusCode()).isEqualTo(HttpStatus.BAD_REQUEST);
201         assertThat(response.getBody()).isInstanceOf(Map.class);
202         assertThat(((Map<?, ?>) response.getBody()).get("error").toString())
203             .contains("exceeds available balance");
204     }
205
206     @Test
207     void approveLeave_rejectsWhenPendingRequestWouldOverdrawBalance() {
208         Employee employee = new Employee();
209         employee.setEmployeeId(1L);
210
211         LeaveRequest request = new LeaveRequest();
212         request.setId(10L);
213         request.setEmployee(employee);
214         request.setLeaveType(LeaveType.CASUAL);
215         request.setStartDate(LocalDate.of(2026, 5, 4));
216         request.setEndDate(LocalDate.of(2026, 5, 6));
217         request.setStatus(LeaveStatus.PENDING);
218
219         com.example.EmployeeManagementSystem.Entity.LeaveType casual =
220             new com.example.EmployeeManagementSystem.Entity.LeaveType();
221         casual.setId(2);
222         casual.setName("CASUAL");

```

```
223     casual.setCarriesForward(true);
224
225     LeaveEntitlement entitlement = new LeaveEntitlement();
226     entitlement.setEmployee(employee);
227     entitlement.setLeaveType(casual);
228     entitlement.setYear(2026);
229     entitlement.setOpeningBalance(BigDecimal.ONE);
230     entitlement.setAccruedThisYear(BigDecimal.ONE);
231     entitlement.setUsedThisYear(BigDecimal.ONE); // only 1 day left
232
233     when(leaveRequestRepo.findById(10L)).thenReturn(Optional.of(request));
234     when(leaveTypeRepository.findByName("CASUAL")).thenReturn(Optional.of(casual));
235     when(leaveEntitlementRepository.findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(1L, 2, 2026))
236         .thenReturn(Optional.of(entitlement));
237
238     assertThatThrownBy(() -> service.approveLeave(10L))
239         .assertInstanceOf(IllegalStateException.class)
240         .hasMessageContaining("Insufficient casual leave balance");
241 }
242 }
243
```

```

1  package com.example.EmployeeManagementSystem.service;
2
3  import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
4  import com.example.EmployeeManagementSystem.Repository.LeaveTypeRepository;
5  import org.junit.jupiter.api.BeforeEach;
6  import org.junit.jupiter.api.Test;
7  import org.mockito.*;
8  import com.example.EmployeeManagementSystem.Service.SickLeaveResetService;
9  import com.example.EmployeeManagementSystem.Entity.LeaveType;
10 import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
11 import com.example.EmployeeManagementSystem.Entity.Employee;
12
13 import java.math.BigDecimal;
14 import java.time.LocalDate;
15 import java.time.YearMonth;
16 import java.util.List;
17
18 import static org.assertj.core.api.Assertions.assertThat;
19 import static org.mockito.Mockito.*;
20
21 class SickLeaveResetServiceTest {
22
23     @Mock
24     LeaveTypeRepository leaveTypeRepo;
25     @Mock
26     LeaveEntitlementRepository entitlementRepo;
27     @InjectMocks
28     SickLeaveResetService service;
29
30     @BeforeEach
31     void setup() {
32         MockitoAnnotations.openMocks(this);
33     }
34
35     @Test
36     void unusedSickDays_areZeroedAtMonthEnd() {
37         LeaveType sick = sickLeaveType();
38         LeaveEntitlement entitlement = entitlement(sick, BigDecimal.valueOf(3), BigDecimal.ONE);
39         // accrued=3, used=1, available=2 should be wiped
40
41         when(leaveTypeRepo.findByMonthlyResetTrue()).thenReturn(List.of(sick));
42         when(entitlementRepo.findByLeaveTypeIdAndYear(sick.getId(), 2025))
43             .thenReturn(List.of(entitlement));
44         when(entitlementRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));
45
46         service.runMonthlyReset(LocalDate.of(2025, 6, 30));
47
48         // After reset: accrued == used, available == 0
49         assertThat(entitlement.getAccruedThisYear())
50             .isEqualToComparingTo(entitlement.getUsedThisYear());
51         assertThat(entitlement.getAvailableBalance())
52             .isEqualToComparingTo(BigDecimal.ZERO);
53     }
54
55     @Test
56     void fullyUsedSickLeave_noChangeNeeded() {
57         LeaveType sick = sickLeaveType();
58         LeaveEntitlement entitlement = entitlement(sick, BigDecimal.ONE, BigDecimal.ONE);
59         // accrued=1, used=1, available=0 nothing to wipe
60
61         when(leaveTypeRepo.findByMonthlyResetTrue()).thenReturn(List.of(sick));
62         when(entitlementRepo.findByLeaveTypeIdAndYear(sick.getId(), 2025))
63             .thenReturn(List.of(entitlement));
64         when(entitlementRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));
65
66         service.runMonthlyReset(LocalDate.of(2025, 6, 30));
67
68         // accrued unchanged – still equals used
69         assertThat(entitlement.getAccruedThisYear()).isEqualToComparingTo("1");
70     }
71 }

```

```

72 // FIX: New test – ensures reset does NOT alter a negative balance (over-used edge case)
73 @Test
74 void negativeBalance_isNotTouched() {
75     LeaveType sick = sickLeaveType();
76     // accrued=0, used=1, available=-1 (negative – already over-consumed)
77     LeaveEntitlement entitlement = entitlement(sick, BigDecimal.ZERO, BigDecimal.ONE);
78
79     when(leaveTypeRepo.findByMonthlyResetTrue()).thenReturn(List.of(sick));
80     when(entitlementRepo.findByLeaveTypeIdAndYear(sick.getId(), 2025))
81         .thenReturn(List.of(entitlement));
82     when(entitlementRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));
83
84     service.runMonthlyReset(LocalDate.of(2025, 6, 30));
85
86     // available is <= 0 so no reset should happen; accrued stays 0
87     assertThat(entitlement.getAccruedThisYear()).isEqualByComparingTo(BigDecimal.ZERO);
88 }
89
90 // FIX: New test – verifies correct year/month is used when resetDate is end-of-month
91 // (guards against the midnight-fire fragility in the scheduler)
92 @Test
93 void resetDate_usesCorrectYearFromPassedDate_notFromSystemClock() {
94     LeaveType sick = sickLeaveType();
95     LeaveEntitlement entitlement = entitlement(sick, BigDecimal.valueOf(2), BigDecimal.ZERO);
96
97     LocalDate endOfJune2025 = YearMonth.of(2025, 6).atEndOfMonth(); // 2025-06-30
98
99     when(leaveTypeRepo.findByMonthlyResetTrue()).thenReturn(List.of(sick));
100    when(entitlementRepo.findByLeaveTypeIdAndYear(sick.getId(), 2025))
101        .thenReturn(List.of(entitlement));
102    when(entitlementRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));
103
104    service.runMonthlyReset(endOfJune2025);
105
106    // Should query year=2025, not system year – verified by mock setup using 2025 only
107    verify(entitlementRepo).findByLeaveTypeIdAndYear(sick.getId(), 2025);
108    assertThat(entitlement.getAvailableBalance()).isEqualByComparingTo(BigDecimal.ZERO);
109 }
110
111 // --- helpers ---
112
113 private LeaveType sickLeaveType() {
114     LeaveType lt = new LeaveType();
115     lt.setId(1);
116     lt.setName("SICK");
117     lt.setMonthlyReset(true);
118     lt.setCarriesForward(false);
119     return lt;
120 }
121
122 private LeaveEntitlement entitlement(LeaveType type, BigDecimal accrued, BigDecimal used) {
123     Employee emp = new Employee();
124     emp.setEmployeeId(1L);
125     LeaveEntitlement e = new LeaveEntitlement();
126     e.setEmployee(emp);
127     e.setLeaveType(type);
128     e.setYear(2025);
129     e.setOpeningBalance(BigDecimal.ZERO);
130     e.setAccruedThisYear(accrued);
131     e.setUsedThisYear(used);
132     e.setClosingBalance(BigDecimal.ZERO);
133     return e;
134 }
135 }

```

```

1 package com.example.EmployeeManagementSystem.service;
2
3 import com.example.EmployeeManagementSystem.Entity.Employee;
4 import com.example.EmployeeManagementSystem.Entity.LeaveEntitlement;
5 import com.example.EmployeeManagementSystem.Entity.LeaveType;
6 import com.example.EmployeeManagementSystem.Enum.Status;
7 import com.example.EmployeeManagementSystem.Repository.EmployeeRepo;
8 import com.example.EmployeeManagementSystem.Repository.LeaveEntitlementRepository;
9 import com.example.EmployeeManagementSystem.Repository.LeaveWarningRepository;
10 import com.example.EmployeeManagementSystem.Repository.YearEndCarryForwardLogRepository;
11 import com.example.EmployeeManagementSystem.Service.YearEndCarryForwardService;
12
13 import org.junit.jupiter.api.BeforeEach;
14 import org.junit.jupiter.api.Test;
15
16 import org.mockito.InjectMocks;
17 import org.mockito.Mock;
18 import org.mockito.MockitoAnnotations;
19
20 import java.math.BigDecimal;
21 import java.util.List;
22 import java.util.Optional;
23
24 import static org.mockito.ArgumentMatchers.any;
25 import static org.mockito.ArgumentMatchers.eq;
26
27 import static org.mockito.Mockito.when;
28 import static org.mockito.Mockito.verify;
29 import static org.mockito.Mockito.never;
30 import static org.mockito.Mockito.atLeastOnce;
31
32 import static org.assertj.core.api.Assertions.assertThat;
33 class YearEndCarryForwardServiceTest {
34
35     @Mock
36     EmployeeRepo employeeRepo;
37     @Mock
38     LeaveEntitlementRepository entitlementRepo;
39     @Mock
40     YearEndCarryForwardLogRepository logRepo;
41     @Mock
42     LeaveWarningRepository warningRepo;
43     @InjectMocks
44     YearEndCarryForwardService service;
45
46     @BeforeEach
47     void setup() { MockitoAnnotations.openMocks(this); }
48
49     @Test
50     void whenTotalUnderCap_noCapApplied() {
51         Employee emp = employee();
52         LeaveType casual = carryForwardType("CASUAL");
53
54         // casual: opening=0, accrued=8, used=2 closing=6
55         LeaveEntitlement ent = entitlement(emp, casual, 0, 8, 2);
56
57         when(logRepo.existsByProcessedYear(2025)).thenReturn(false);
58         when(employeeRepo.findById(Status.ACTIVE)).thenReturn(List.of(emp));
59         when(entitlementRepo.findByIdByEmployeeIdAndYear(emp.getId(), 2025))
60             .thenReturn(List.of(ent));
61         when(entitlementRepo.findByIdByEmployeeIdAndLeaveTypeIdAndYear(any(), any(), eq(2026)))
62             .thenReturn(Optional.empty());
63         when(entitlementRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));
64         when(logRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));
65
66         service.runYearEnd(2025);
67
68         // closing balance = 6, no cap needed, nothing wasted
69         assertThat(ent.getClosingBalance()).isEqualToByComparingTo("6");
70         verify(warningRepo, never()).save(any());
71     }

```

72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137

```
@Test
void whenTotalExceedsCap_capAppliedProportionally() {
    Employee emp = employee();
    LeaveType casual = carryForwardType("CASUAL");
    LeaveType maternity = carryForwardType("MATERNITY");

    // casual: closing=20, maternity: closing=16 total=36, over cap by 6
    LeaveEntitlement casualEnt = entitlement(emp, casual, 0, 20, 0);
    LeaveEntitlement maternityEnt = entitlement(emp, maternity, 0, 16, 0);

    when(logRepo.existsByProcessedYear(2025)).thenReturn(false);
    when(employeeRepo.findByStatus(Status.ACTIVE)).thenReturn(List.of(emp));
    when(entitlementRepo.findByEmployeeEmployeeIdAndYear(emp.getEmployeeId(), 2025))
        .thenReturn(List.of(casualEnt, maternityEnt));
    when(entitlementRepo.findByEmployeeEmployeeIdAndLeaveTypeIdAndYear(any(), any(), eq(2026)))
        .thenReturn(Optional.empty());
    when(entitlementRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));
    when(logRepo.save(any())).thenAnswer(inv -> inv.getArgument(0));

    service.runYearEnd(2025);

    // casual: 20/36 * 30 = 16.67, maternity: 16/36 * 30 = 13.33, total = 30
    BigDecimal casualCapped = casualEnt.getClosingBalance();
    BigDecimal maternityCapped = maternityEnt.getClosingBalance();
    BigDecimal total = casualCapped.add(maternityCapped);

    assertThat(total).isEqualByComparingTo("30.00");
    verify(warningRepo, atLeastOnce()).save(any()); // wasted warning issued
}

@Test
void idempotencyGuard_preventsDoubleRun() {
    when(logRepo.existsByProcessedYear(2025)).thenReturn(true);

    service.runYearEnd(2025);

    verify(employeeRepo, never()).findByStatus(any());
}

// --- helpers ---
private Employee employee() {
    Employee e = new Employee(); e.setEmployeeId(1L); return e;
}

private LeaveType carryForwardType(String name) {
    LeaveType lt = new LeaveType();
    lt.setId(name.equals("CASUAL") ? 2 : 3);
    lt.setName(name);
    lt.setCarriesForward(true);
    lt.setMonthlyReset(false);
    return lt;
}

private LeaveEntitlement entitlement(Employee emp, LeaveType type,
    int opening, int accrued, int used) {
    LeaveEntitlement e = new LeaveEntitlement();
    e.setEmployee(emp); e.setLeaveType(type); e.setYear(2025);
    e.setOpeningBalance(BigDecimal.valueOf(opening));
    e.setAccruedThisYear(BigDecimal.valueOf(accrued));
    e.setUsedThisYear(BigDecimal.valueOf(used));
    e.setClosingBalance(BigDecimal.ZERO);
    return e;
}
}
```